

プログラムの設計方法 第二版（日本語訳）

Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, Shriram Krishnamurthi

目次

プログラムの設計方法 第二版	8
序文	9
体系的なプログラム設計	11
DrRacket とティーチング言語	13
移行可能なスキル	14
この本とその各部	15
違い点	16
プロローグ: プログラムの仕方	17
算術と算術	19
多くの方法で計算する	19
1 つのプログラム、多くの定義	19
あなたはもうプログラマだ	19
I 固定サイズのデータ (Fixed-Size Data)	20
1 算術 (Arithmetic)	21
1.1 数の算術 (The Arithmetic of Numbers)	21
1.2 文字列の算術 (The Arithmetic of Strings)	21
1.3 混ぜ合わせ (Mixing It Up)	22
1.4 画像の算術 (The Arithmetic of Images)	22
1.5 ブール値の算術 (The Arithmetic of Booleans)	22
1.6 ブール値と混ぜ合わせ (Mixing It Up with Booleans)	22
1.7 述語: 自分のデータを知れ (Predicates: Know Thy Data)	23
2 関数とプログラム (Functions and Programs)	24
2.1 関数 (Functions)	24
2.2 計算 (Computing)	24
2.3 関数の合成 (Composing Functions)	25
2.4 グローバル定数 (Global Constants)	25
2.5 プログラム (Programs)	25
3 プログラムの設計方法	26
3.1 関数の設計	28
情報とデータ	28
設計プロセス	30
関数の例から設計へ	31

3.2 指の運動: 関数	32
3.3 ドメイン知識	33
3.4 関数からプログラムへ	34
3.5 テストについて	35
3.6 World プログラムの設計	36
3.7 バーチャルペットの世界	37
4.1 条件式を使ったプログラミング (Programming with Conditionals)	38
4.2 条件付きの計算 (Computing Conditionally)	40
4.3 列挙 (Enumerations)	42
4.4 区間 (Intervals)	44
4.5 項目化 (Itemizations)	46
4.6 項目化を用いた設計 (Designing with Itemizations)	47
4.7 有限状態世界 (Finite State Worlds)	48
5 構造の追加 (Adding Structure)	51
5.1 位置から posn 構造体へ (From Positions to posn Structures)	51
5.2 posn を用いた計算 (Computing with posns)	52
5.3 posn を用いたプログラミング (Programming with posn)	53
5.4 構造体型の定義 (Defining Structure Types)	55
5.5 構造体を使った計算 (Computing with Structures)	59
5.6 構造体を用いたプログラミング (Programming with Structures)	63
5.7 データの宇宙 (The Universe of Data)	71
5.8 構造体を用いた設計 (Designing with Structures)	74
5.9 ワールド内の構造 (Structure in the World)	76
5.10 グラフィカルエディタ (A Graphical Editor)	77
5.11 さらにバーチャルペット (More Virtual Pets)	79
6 項目化と構造 (Itemizations and Structures)	82
6.1 項目化を用いた設計、再び (Designing with Itemizations, Again)	82
6.2 世界を混ぜ合わせる (Mixing Up Worlds)	94
6.3 入力エラー (Input Errors)	97
6.4 ワールドのチェック (Checking the World)	102
6.5 等価述語 (Equality Predicates)	104
7 まとめ (Summary)	106
Intermezzo 1: Beginning Student Language (初級学生言語)	107
目次 (Contents)	107
BSL の語彙 (BSL Vocabulary)	108
BSL の文法 (BSL Grammar)	109
BSL の意味 (BSL Meaning)	113
意味とコンピューティング (Meaning and Computing)	116
BSL のエラー (BSL Errors)	117
ブール式 (Boolean Expressions)	120

定数定義 (Constant Definitions)	122
構造型定義 (Structure Type Definitions)	124
BSL のテスト (BSL Tests)	127
BSL のエラーメッセージ (BSL Error Messages)	129
II 任意サイズのデータ (Arbitrarily Large Data)	136
8 リスト (Lists)	137
8.1 リストの作成 (Creating Lists)	137
8.2 '()' とは何か、cons とは何か (What Is '()', What Is cons)	141
8.3 リストを用いたプログラミング (Programming with Lists)	144
8.4 リストによる計算 (Computing with Lists)	148
9 自己参照データ定義を用いた設計 (Designing with Self-Referential Data Definitions)	151
9.1 指の運動：リスト (Finger Exercises: Lists)	158
9.3 自然数 (Natural Numbers)	166
9.4 マトリョーシカ人形 (Russian Dolls)	170
9.5 リストとワールド (Lists and World)	173
9.6 リストと集合についての注意 (A Note on Lists and Sets)	178
10 リストのさらなる活用 (More on Lists)	184
10.1 リストを生成する関数 (Functions that Produce Lists)	184
10.2 リスト内の構造体 (Structures in Lists)	186
10.3 リストの中のリスト、ファイル (Lists in Lists, Files)	191
10.4 グラフィカルエディタ再訪 (A Graphical Editor, Revisited)	202
11 構成による設計 (Design by Composition)	215
11.1 list 関数 (The list Function)	215
11.2 関数の合成 (Composing Functions)	217
11.3 再帰する補助関数 (Auxiliary Functions that Recur)	219
11.4 一般化する補助関数 (Auxiliary Functions that Generalize)	225
12 プロジェクト：リスト (Projects: Lists)	236
12.1 現実世界のデータ：辞書 (Real-World Data: Dictionaries)	236
12.2 実世界データ：iTunes (Real-World Data: iTunes)	238
12.3 単語ゲーム、合成の例証 (Word Games, Composition Illustrated)	244
12.4 単語ゲーム、問題の中核 (Word Games, the Heart of the Problem)	248
12.5 ワームに餌を与える (Feeding Worms)	250
12.6 簡易テトリス (Simple Tetris)	253
12.7 本格的なスペースウォーズ (Full Space War)	256
12.8 有限状態機械 (Finite State Machines)	257
13 まとめ (Summary)	266
Intermezzo 2: Quote, Unquote	267
III 抽象化 (Abstraction)	268
導入：抽象化の必要性	269

14 類似性はどこにでもある (Similarities Everywhere)	270
14.1 関数における類似性 (Similarities in Functions)	270
14.2 異なる類似性 (Different Similarities)	272
14.3 データ定義における類似性 (Similarities in Data Definitions)	275
14.4 関数は値である (Functions Are Values)	277
14.5 関数を使った計算 (Computing with Functions)	278
15 抽象化の設計 (Designing Abstractions)	282
15.1 例からの抽象化 (Abstractions from Examples)	282
15.2 署名における類似性 (Similarities in Signatures)	285
15.3 単一の制御点 (Single Point of Control)	287
15.4 テンプレートからの抽象化 (Abstractions from Templates)	287
17 無名関数 (Nameless Functions)	289
17.1 lambda から生まれる関数 (Functions from lambda)	289
17.2 lambda を使った計算 (Computing with lambda)	289
17.3 lambda を使った抽象化 (Abstracting with lambda)	289
17.4 lambda を使った仕様 (Specifying with lambda)	289
17.5 lambda を使った表現 (Representing with lambda)	289
Intermezzo 3: Scope and Abstraction	290
IV 絡み合ったデータ (Intertwined Data)	291
Intermezzo 4: The Nature of Numbers	292
V 生成的再帰 (Generative Recursion)	293
Intermezzo 5: The Cost of Computation	294
VI 蓄積子 (Accumulators)	295
エピローグ: 先へ進む (Epilogue: Moving On)	296
付録 A: Quick — 絵で学ぶ Racket 入門	297
1 用意.....	297
2 セット.....	297
3 スタート！	297
4 定義	298
5 局所束縛	299
6 関数は値である	300
7 レキシカルスコープ	301
8 リスト	301
9 モジュール	302
10 マクロ	303
11 オブジェクト	304
12 ここからどこへ行くか	305
付録 B: How to Design Programs 言語リファレンス	306
How to Design Programs 言語	306
目次	306

付録 B-1: Beginning Student Language (#lang htdp/bsl)	310
1 Beginning Student (初級学生言語)	310
1.1 あらかじめ定義された変数	311
1.2 テンプレート変数	312
1.3 構文	312
1.4 シグネチャ	322
1.5 あらかじめ定義された関数	324
1.6 数: 整数・有理数・実数・複素数・正確数・非正確数	325
1.7 真偽値	341
1.8 シンボル	342
1.9 リスト	343
1.10 Posn	355
1.11 文字	356
1.12 文字列	361
1.13 画像	371
1.14 その他	371
1.15 シグネチャ	374
付録 B-2: Beginning Student with List Abbreviations (#lang htdp/bsl+)	377
2.1 あらかじめ定義された変数	379
2.2 テンプレート変数	379
2.3 リスト略記つき初級の構文	380
2.4 共通の構文	381
2.5 シグネチャ	390
2.6 あらかじめ定義された関数	393
2.7 数: 整数・有理数・実数・複素数・正確数・非正確数	393
2.8 真偽値	409
2.9 シンボル	410
2.10 リスト	411
2.11 Posn	424
2.12 文字	425
2.13 文字列	430
2.14 画像	439
2.15 その他	440
2.16 シグネチャ	443
付録 B-3: Intermediate Student (#lang htdp/isl)	446
3 Intermediate Student (中級学生言語)	446
3.1 あらかじめ定義された変数	448
3.2 テンプレート変数	448
3.3 中級の構文	449
3.4 共通の構文	450

3.5 シグネチャ	460
3.6 あらかじめ定義された関数	463
3.7 数: 整数・有理数・実数・複素数・正確数・非正確数	463
3.8 真偽値	478
3.9 シンボル	480
3.10 リスト	480
3.11 Posn	493
3.12 文字	494
3.13 文字列	499
3.14 画像	506
3.15 その他	506
3.16 シグネチャ	509
3.17 数 (緩和条件)	511
3.18 文字列 (緩和条件)	513
3.19 Posn	516
3.20 高階関数	516
付録 B-4: Intermediate Student with lambda (#lang htdp/isl+)	523
4 Intermediate Student with Lambda (lambda つき中級)	523
4.1 あらかじめ定義された変数	525
4.2 テンプレート変数	525
4.3 lambda つき中級の構文	526
4.4 共通の構文 (Common Syntaxes)	528
4.5 あらかじめ定義された関数	537
4.6 シグネチャ	537
4.7 数: 整数・有理数・実数・複素数・正確数・非正確数	540
4.8 真偽値	555
4.9 シンボル	557
4.10 リスト	557
4.11 Posn	570
4.12 文字	571
4.13 文字列	576
4.14 画像	583
4.15 その他	583
4.16 シグネチャ	586
4.17 数 (緩和条件)	588
4.18 文字列 (緩和条件)	588
4.19 Posn	592
4.20 高階関数	592
4.21 数 (緩和条件・拡張)	592
4.22 高階関数 (lambda つき)	593

付録 B-5: Advanced Student (#lang htdp/asl)	600
5 Advanced Student (上級学生言語)	600
5.1 あらかじめ定義された変数	603
5.2 テンプレート変数	603
5.3 上級の構文	604
5.4 共通の構文	608
5.5 あらかじめ定義された関数	618
5.6 シグネチャ	618
5.7 数: 整数・有理数・実数・複素数・正確数・非正確数	620
5.8 真偽値	636
5.9 シンボル	637
5.10 リスト	638
5.11 Posn	650
5.12 文字	652
5.13 文字列	657
5.14 画像	664
5.15 その他	664
5.16 シグネチャ	669
5.17 数 (緩和条件)	671
5.18 文字列 (緩和条件)	671
5.19 Posn	674
5.20 高階関数	674
5.21 数 (緩和条件・拡張)	674
5.22 高階関数 (lambda つき)	676
5.23 読み取りと表示	681
5.24 ベクタ	684
5.25 ボックス	686
5.26 ハッシュ表	687

プログラムの設計方法 第二版

How to Design Programs, Second Edition

著者: Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, Shriram Krishnamurthi

© 2014 (更新版 2026) MIT Press

この資料は著作権保護されており、Creative Commons CC BY-NC-ND ライセンスの下で提供されています。

序文

Preface

序文	
	プロローグ: プログラムの仕方
I	固定サイズのデータ
	Intermezzo 1: Beginning Student Language
II	任意サイズのデータ
	Intermezzo 2: Quote, Unquote
III	抽象化
	Intermezzo 3: Scope and Abstraction
IV	絡み合ったデータ
	Intermezzo 4: The Nature of Numbers
V	生成的再帰
	Intermezzo 5: The Cost of Computation
VI	蓄積子
	エピローグ: 先へ進む

多くの職業で何らかのプログラミングが必要です。会計士はスプレッドシートをプログラムし、ミュージシャンはシンセサイザーをプログラムし、著者はワードプロセッサをプログラムし、Web デザイナーはスタイルシートをプログラムします。私たちが初版のこれらの言葉を書いたとき (1995–2000)、読者はそれらを未来的だと考えたかもしれません。今や、プログラミングは必須のスキルとなり、雇用見通しを高めるという目的で、数多くの書籍、オンラインコース、K-12 カリキュラムがこのニーズに応えています。

典型的なプログラミングのコースでは、「動くまでいじくり回す」アプローチを教えます。それが動くと、学生は「動いた!」と叫んで次に進みます。悲しいことに、このフレーズはコンピューティングにおける最短の嘘でもあり、多くの人々の人生から何時間も奪ってきました。対照的に、本書は良いプログラミングの習慣に焦点を当て、プロフェッショナルおよび職業プログラマの両方を対象としています。

「良いプログラミング」とは、ソフトウェア作成へのアプローチで、最初から、すべての段階で、すべてのステップで体系的な思考、計画、理解に依拠するものを意味します。この点を強調するために、私たちは「体系的なプログラム設計」と「体系的に設計されたプログラム」について語ります。後者は、望まれる機能の根拠を明確に述べます。良いプログラミングは達成感の美的感覚も満たします。良いプログラムのエレガンスは、時の試練

に耐えた詩や過去の時代の白黒写真に匹敵します。要するに、プログラミングと良いプログラミングの違いは、ダイナーのクレヨン画と美術館の油絵のようなものです。

いいえ、この本は誰をもマスターペインターには変えません。しかし、私たちがこの版を書くのに 15 年を費やしたのは、以下を信じているからに他なりません。

誰もがプログラムを設計できる

そして

誰もが創造的設計から得られる満足感を経験できる

実際、私たちはさらに進んで主張します。

プログラム設計—— しかしプログラミングではない—— は、リベラルアーツ教育において数学や言語スキルと同じ役割を果たすべきである。

設計を学ぶ学生が二度とプログラムに触れなくても、普遍的に有用な問題解決スキルを身につけ、深く創造的な活動を経験し、新しい形式の美を理解することを学ぶでしょう。この序文の残りでは、「体系的設計」とは何を意味するのか、誰がどのように恩恵を受けるのか、そしてそれをどのように教えるのかを詳しく説明します。

体系的なプログラム設計

プログラムは人々（ユーザーと呼ばれる）と他のプログラム（サーバーとクライアントコンポーネントの場合）と相互作用します。したがって、合理的に完全なプログラムは多くの構成要素から成ります。一部は入力を扱い、一部は出力を作成し、一部はそれらの間のギャップを橋渡しします。私たちは関数を基本的な構成要素として選ぶ理由は、誰もが前代数で関数に遭遇するからであり、最も単純なプログラムがまさにそのような関数だからです。鍵は、どの関数が必要か、それらをどのように接続するか、そして基本的な材料からそれらをどのように構築するかを発見することです。

この文脈で、「体系的なプログラム設計」とは、2つの概念の混合を指します。**設計レシピ**と**反復的洗練**です。私たちはMichael JacksonのCOBOLプログラム作成法、Daniel Friedmanとの再帰に関する会話、Robert Harperとの型理論、Daniel Jacksonとのソフトウェア設計から着想を得ました。設計レシピは著者らの創作であり、ここでは後者の使用を可能にします。

問題分析からデータ定義へ 表現しなければならない情報と、それが選択したプログラミング言語でどのように表現されるかを特定せよ。データ定義を定式化し、例で説明せよ。署名、目的文、ヘッダ 望まれる関数がどのような種類のデータを消費し、生成するかを述べよ。その関数が何を計算するのかという問いに対する簡潔な答えを定式化せよ。署名に沿ったスタブを定義せよ。関数の例 関数の目的を説明する例に取り組み。関数のテンプレート データ定義を関数のアウトラインに翻訳せよ。関数定義 関数のテンプレートの隙間を埋めよ。目的文と例を活用せよ。テスト 例をテストとして明確に述べ、関数がすべてに合格することを確認せよ。これにより誤りが発見される。テストはまた、必要が生じたときに他の人が定義を読み理解するのを助ける——そしてそれは深刻なプログラムでは必ず生じる。

図 1: 関数設計レシピの基本ステップ

設計レシピは完全なプログラムと個々の関数の両方に適用されます。本書は完全なプログラムのための2つのレシピだけを扱います。GUIを持つプログラム用とバッチプログラム用です。一方、関数用の設計レシピにはさまざまなバリエーションがあります。数値のような原子的なデータ形式用、さまざまな種類のデータの列挙用、固定された方法で他のデータを複合するデータ用、有限だが任意に大きいデータ用、など。

関数レベルの設計レシピは共通の設計プロセスを共有します。図 1 はその 6 つの必須ステップを示しています。各ステップのタイトルは期待される成果を指定し、「コマンド」は主要な活動を示唆します。例はほぼすべての段階で中心的な役割を果たします。

各ステップには鋭い質問が付随しており、本書の 6 つのパートを通じて導入されます。特定のステップ——例えば関数の例の作成やテンプレート——では、質問はデータ定義に訴えかけるかもしれません。答えはほぼ自動的に中間成果物を作成します。

このアプローチの新しさは、初心者レベルのプログラムのための中間成果物の作成にあります。初心者が詰まったとき、専門家や講師は既存の中間成果物を検査できます。検査は設計プロセスの一般的な質問を使う可能性が高く、したがって初心者に自分で修正させることになります。そしてこの自己強化プロセスこそが、プログラミングとプログラム設計の重要な違いです。

反復的洗練は、問題が複雑で多面的であるという問題に対処します。一度にすべてを正しくすることはほぼ不可能です。代わりに、コンピュータ科学者はこの設計問題に取り組むために物理科学から反復的洗練を借用します。本質的に、反復的洗練は、最初にすべての非本質的な詳細を取り除き、残された中核問題の解決策を見つけることを推奨します。洗練ステップは、これらの省略された詳細の1つを追加し、既存の解決策を可能な限り使用して拡張された問題を再解決します。これらの洗練ステップの繰り返し（反復とも呼ばれる）は、最終的に完全な解決策につながります。

この意味で、プログラマはミニ科学者です。科学者は世界の理想化されたバージョンのための近似モデルを作成し、それについての予測を行います。モデルの予測が当たっている限り、すべてはうまくいきます。予測された出来事が実際のもものと異なる場合、科学者はその不一致を減らすためにモデルを改訂します。同様に、プログラマにタスクが与えられたとき、彼らは最初の設計を作成し、それをコードにし、実際のユーザーで評価し、プログラムの振る舞いが望まれる製品に密接に一致するまで設計を反復的に洗練します。

本書は2つの異なる方法で反復的洗練を導入します。洗練による設計がプログラムの設計が複雑になると有用になるため、本書は問題がある程度の難易度を獲得した第4部でこの技法を明示的に導入します。さらに、私たちは第1部から第3部を通じて同じ問題のますます複雑なバリエーションを述べるために反復的洗練を使用します。つまり、中核問題を選び、1つの章でそれを扱い、その後、後続の章で新たに導入された概念に合致する詳細で類似の問題を提示します。

DrRacket とティーチング言語

プログラムを設計することを学ぶには、繰り返しのハンズオン練習が必要です。ピアノを弾かずにピアノ奏者になれないのと同じように、実際のプログラムを作成して正しく動作させることなしにプログラム設計者にはなれません。したがって、本書には最小限のソフトウェアサポートが付属しています。プログラムを書き留めるための言語と、プログラムをワード文書のように編集し、プログラムを実行できるプログラム開発環境です。

多くの人が「コードの書き方を学びたい」と言い、次にどのプログラミング言語を学ぶべきかを尋ねます。いくつかのプログラミング言語が得ている注目を考えると、この質問は驚くべきことではありません。しかし、それは全く不適切でもあります。初心者を対象としないコースでは、既製の言語を設計レシピとともに使用できるかもしれません。現在流行のプログラミング言語でプログラミングを学ぶことは、学生を最終的な失敗に導くことがよくあります。この世界での流行は非常に短命です。「Xで素早くプログラミング」本やコースは、次の流行の言語に移行できる原則を教えることに失敗することが多いです。さらに悪いことに、言語自体が、解決策を表現するレベルでもプログラミングミスに対処するレベルでも、移行可能なスキルの習得から気を逸らします。

対照的に、プログラムを設計することを学ぶことは、主に原則の研究と移行可能なスキルの習得についてです。理想的なプログラミング言語はこれら 2 つの目標をサポートしなければなりませんが、既製の産業用言語はそうしません。重要な問題は、初心者が言語の多くを知る前に間違いを犯すことです。しかしプログラミング言語は、プログラマがすでに言語全体を知っているかのように常にこれらのエラーを診断します。その結果、診断レポートは初心者をしばしば困惑させます。

私たちの解決策は、私たち自身の仕立てられたティーチング言語から始めることです。「Beginning Student Language」または BSL と呼ばれます。この言語は本質的に、学生が前代数のコースで習得する「外国語」です。関数定義、関数適用、条件式のための記法を含みます。また、式を入れ子にすることもできます。この言語は非常に小さいので、言語全体の観点からのエラー診断でも、前代数しか持たない読者にとってまだアクセス可能です。

構造的設計原則を習得した学生は、次に「Intermediate Student Language」と他の高度な方言（総称して*SL）に進むことができます。本書はこれらの方言を使用して抽象化と一般再帰の設計原則を教えます。私たちは、このような一連のティーチング言語を使用することが、幅広い専門プログラミング言語（JavaScript、Python、Ruby、Java など）でプログラムを作成するための優れた準備を読者に提供すると固く信じています。

注: ティーチング言語は Racket で実装されています。Racket はプログラミング言語を構築するためのプログラミング言語として私たちが構築したものです。Racket はラボから現実世界に逃れ出ました。それはプログラミング言語です。

(続き: DrRacket の使用、ティーチング言語の選択など詳細)

移行可能なスキル

(中略：設計レシピから得られる普遍的な問題解決スキル、創造的活動としてのプログラミングの価値について)

この本とその各部

本書は6つの主要なパートと5つの Intermezzo に分かれています。各パートは設計レシピの特定の側面に焦点を当て、徐々に複雑さを増していきます。

- Part I: 固定サイズのデータ — 基本的な算術から設計レシピの導入まで。
- Part II: 任意サイズのデータ — リストと自己参照データ定義。
- Part III: 抽象化 — 類似性の発見と高階関数。
- Part IV: 絡み合ったデータ — 複雑なデータ定義の相互作用。
- Part V: 生成的再帰 — 構造的再帰を超えたアルゴリズム。
- Part VI: 蓄積子 — 効率的な再帰のための追加の知識の保持。

各 Intermezzo では、言語の形式的な側面（文法、意味、計算モデル）を扱います。

違い点

(初版との違いの説明：より明確な設計レシピ、プロジェクトの強化など)

謝辞 (Acknowledgments)

… (詳細は原典を参照。多くの貢献者への感謝)

本翻訳は原典の内容を忠実に日本語化したものです。サンプルコードは原文を維持しています。

原典 URL: https://htdp.org/2026-5-28/Book/part_preface.html

プロローグ: プログラムの仕方

Prologue: How to Program

(目次リンクは省略)

あなたが小さな子供だったとき、親はあなたに数を数え、指で簡単な計算をすることを教えました。「 $1+1$ は 2 」;「 $1+2$ は 3 」; など。そして「 $3+2$ は？」と尋ね、あなたは片手の指を数えました。彼らはプログラムし、あなたは計算しました。そしてある意味で、それが本当にプログラミングとコンピューティングのすべてです。

今度は役割を交代する時です。DrRacket を起動しましょう。そうすると DrRacket のウィンドウが表示されます。「Language」メニューから「Choose language」を選び、「How to Design Programs」の「Teaching Languages」から「Beginning Student」(BSL) を選択し、OK をクリックして DrRacket を設定します。

これで準備完了です。あなたがプログラムし、DrRacket ソフトウェアが子供の役割を果たします。

最も単純な計算から始めましょう。DrRacket の上部に次のように入力します：

(+ 1 1)

RUN をクリックすると、下部に 2 が表示されます。

(図: DrRacket の画面)

それほど単純なのがプログラミングです。DrRacket を子供のように質問し、DrRacket があなたのために計算します。一度に複数のリクエストを処理させることもできます：

(+ 2 2)

(* 3 3)

(- 4 2)

(/ 6 2)

RUN をクリックすると、期待される結果 4923 が表示されます。

少しペースを落として用語を導入しましょう。

DrRacket の上半分は「定義領域 (definitions area)」と呼ばれます。この領域でプログラムを作成します。これを編集と呼びます。定義領域に単語を追加したり変更したりすると、左上に SAVE ボタンが表示されます。初めて SAVE をクリックすると、DrRacket はファイルを保存するための名前を尋ねます。一度ファイルに関連付けられると、SAVE をクリックすると定義領域の内容が安全にファイルに保存されます。

プログラムは式 (expressions) で構成されます。数学で式に出会ったことがあるでしょう。今のところ、式は単なる数値か、左括弧「(」で始まり対応する右括弧「)」で終わるものです——DrRacket はその括弧の間の領域を強調表示します。

RUN をクリックすると、DrRacket は定義領域の式を評価し、その結果を対話領域 (interactions area) に表示します。その後、DrRacket はプロンプト ($>$) であなたのコマン

ドを待ちます。プロンプトが表示されたら、追加の式を入力でき、DrRacket は定義領域と同じようにそれを評価します。

このようなプログラミングとコンピューティングを DrRacket で簡単に探求できます。

算術と算術

プログラミングが数と算術だけだったら、数学と同じくらい退屈でしょう。幸い、プログラミングには数値以外にも多くのものがあります。テキスト、真偽値、画像などです。

BSL で知っておくべき最初のことは、テキストは二重引用符 (") で囲まれた任意のキーボード文字のシーケンスであるということです。これを文字列 (string) と呼びます。したがって、

```
"hello world"
```

は完全に正しい文字列であり、DrRacket がこの文字列を評価すると、対話領域で数値と同じようにそのままエコーバックします。

BSL には数の算術に加えて、文字列の算術もあります。以下に 2 つの例を示します：

```
(string-append "hello" "world")  
; "helloworld"
```

```
(string-append "hello " "world")  
; "hello world"
```

+ と同様に、string-append は演算子です。2 番目の文字列を最初の文字列の末尾に追加して新しい文字列を作ります。最初の例が示すように、2 つの文字列の間に何も挿入せずに文字通り結合します。空白、カンマ、何もありません。

(以下、画像の算術、ブール値、条件式、関数の定義などプロローグの続きが展開されます。)

多くの方法で計算する

(条件式の導入例)

1 つのプログラム、多くの定義

(define を使った定数と関数の導入)

あなたはもうプログラマだ

ここまで来たら、あなたはすでに小さなプログラムを書けるプログラマです。以降の章で設計レシピを学び、体系的にプログラムを構築していきます。

注意: サンプルコードは原文のままです。

原典: https://htdp.org/2026-5-28/Book/part_prologue.html

I 固定サイズのデータ (Fixed-Size Data)

本書の第I部では、基本的な原子データ（数値、文字列、画像、真偽値）を操作する「算術」と、それらを組み合わせた最も単純なプログラムである「関数」について学び、体系的なプログラム設計の最初の一步を踏み出します。

1 算術 (Arithmetic)

BSL (Beginning Student Language) における算術は、前置表記 (Prefix Notation) で記述されます。基本規則：1. 開き括弧 (を書く 2. 基本演算の名前 (+ など) を書く 3. 引数をスペースで区切って書く 4. 閉じ括弧) を書く

例：

```
(+ 1 2)
```

ネスト (入れ子) された例：

```
(+ 1 (+ 1 (+ 1 1) 2) 3 4 5)
```

評価は、まず引数となる式をすべて評価し、その結果のデータ片を演算に与えて計算します。

```
(+ 1 2)
```

```
==
```

```
3
```

1.1 数の算術 (The Arithmetic of Numbers)

BSL は自然数、整数、有理数、実数、複素数をサポートし、正確数 (Exact) と不正確数 (Inexact) を区別します。

演算例：+, -, *, /, abs, add1, ceiling, denominator, exact->inexact, expt, floor, gcd, log, max, numerator, quotient, random, remainder, sqr, tan

```
> (sin 0)
```

```
0
```

また、円周率 pi やオイラーの定数 e も事前定義されています。不正確数 (近似値) には #i プレフィックスが付きます (例: #i1.4142135623730951)。

Exercise 1. 定義領域に以下を追加し、デカルト平面上の点 (x, y) から原点 (0, 0) までの距離を計算する式を記述せよ。

```
(define x 3)
```

```
(define y 4)
```

1.2 文字列の算術 (The Arithmetic of Strings)

文字列はダブルクォーテーション " で囲まれた文字列です。

演算例：string-append, string-length, substring, string-ith

```
(string-append "hello" " " "world")
```

```
; "hello world"
```

Exercise 2. prefix と suffix という名前の文字列定数を定義し、それらを結合する式を記述せよ。

1.3 混ぜ合わせ (Mixing It Up)

文字列と数値を相互に変換したり、型を操作する演算です。

演算例： number->string, string->number, string-length

```
(string-length "42")  
==  
2
```

Exercise 3. 文字列 `str` と数値 `i` を定義し、`str` の `i` 番目の位置に `_` を挿入する式を記述せよ。

```
(define str "helloworld")  
(define i 5)
```

Exercise 4. 文字列 `str` と数値 `i` が与えられたとき、`i` 番目の文字を削除する式を記述せよ。

1.4 画像の算術 (The Arithmetic of Images)

2http/image ティーチパックを使用すると、画像をデータとして操作できます。

演算例： circle, rectangle, empty-scene, place-image, overlay, image-width, image-height

Exercise 5. 1 枚の画像のピクセル数を計算する式を作成せよ。

```
(define cat <画像>)
```

Exercise 6. 任意の画像 `cat` が与えられたとき、それが縦長か、横長か、正方形かを判定する式を作成せよ。

Exercise 7. 以下の画像作成を試せ。

```
(circle 10 "solid" "red")  
(rectangle 10 20 "outline" "blue")
```

Exercise 8. 青いバナーの上に赤い文字でテキストを表示する画像を生成する式を記述せよ。

1.5 ブール値の算術 (The Arithmetic of Booleans)

真偽値 (`#true`, `#false`) と、論理演算 (`and`, `or`, `not`)、および比較述語 (`<`, `>`, `=`, `string=?`) です。

```
(or #false #true)  
==  
#true
```

Exercise 9. 画像 `in` に対して、それが縦長であるか、または横長であるかを判定するブール値の式を作成せよ。

1.6 ブール値と混ぜ合わせ (Mixing It Up with Booleans)

条件に応じて処理を分岐させるための `if` 式です。

```
(if (string? "red" "green") "stop" "go")  
==  
"go"
```

Exercise 10. 与えられた画像が縦長なら "tall"、横長なら "wide"、正方形なら "square" を返す if 式を記述せよ。

1.7 述語: 自分のデータを知れ (Predicates: Know Thy Data)

データ型を検査するための述語です。number?, string?, image?, boolean?, any?

Exercise 11. number? や string? の動作を確認する式を記述せよ。

Exercise 12. 値 v が与えられたとき、それが画像ならその面積、文字列ならその長さ、数値ならその値自体を返す条件分岐式を記述せよ。

2 関数とプログラム (Functions and Programs)

プログラムの半分は算術であり、もう半分はそれらを再利用可能にする「関数」です。

2.1 関数 (Functions)

define を使って新しい関数を定義します。

```
(define (f x) (+ x 10))
```

Exercise 13. 与えられた文字列の最初の文字を返す関数 `string-first` を定義せよ。

Exercise 14. 与えられた文字列の最後の文字を返す関数 `string-last` を定義せよ。

Exercise 15. 2つのブール値 `sunny` と `friday` を消費し、金曜日で晴れの日であるかを判定する関数 `play` を定義せよ。

Exercise 16. 画像を消費し、そのピクセル面積を計算する関数 `image-area` を定義せよ。

Exercise 17. 画像を消費し、それが縦長であるかを判定する関数 `image-classify` を定義せよ。

Exercise 18. 2つの文字列を消費し、それらを "_" で結合する関数 `string-join` を定義せよ。

Exercise 19. 文字列 `str` と位置 `i` を消費し、`i` 番目に "_" を挿入する関数 `string-insert` を定義せよ。

Exercise 20. 文字列 `str` と位置 `i` を消費し、`i` 番目の文字を削除する関数 `string-delete` を定義せよ。

2.2 計算 (Computing)

関数の評価は、実引数 (Argument) を仮引数 (Parameter) に置き換える「代入モデル」に従って行われます。

```
(f 2)
==
(+ 2 10)
==
12
```

Exercise 21. `(ff 2)` がどのように評価されるか代入ステップを記述せよ。

```
(define (ff x) (* 10 x))
```

Exercise 22. `(distance 3 4)` の評価ステップを記述せよ。

Exercise 23. `(string-first "hello")` の評価ステップを記述せよ。

Exercise 24. `(implies #true #false)` の評価ステップを記述せよ。

Exercise 25. 画像分類関数の評価ステップを記述せよ。

Exercise 26. `(string-insert "hello" 2)` の評価ステップを記述せよ。

2.3 関数の合成 (Composing Functions)

複雑なタスクは、単純な関数を組み合わせることで解決します。

```
(define (profit price)
  (- (revenue price) (cost price)))
```

Exercise 27. 映画館のチケット価格と利益の関係をモデル化するプログラムを構築せよ。

Exercise 28. チケット価格ごとの利益をシミュレーションし、最適なチケット価格を見つけよ。

Exercise 29. コスト構造が変化した場合の最適なチケット価格を再計算せよ。

2.4 グローバル定数 (Global Constants)

変更に強いプログラムを作るために、マジックナンバーをグローバル定数として定義します。

```
(define BASE-PRICE 5)
```

Exercise 30. 映画館の利益計算プログラムにおける定数（基本価格、固定費、変動費など）を定数定義として分離せよ。

2.5 プログラム (Programs)

BSL では 2htdp/batch-io を使ったバッチ処理プログラムや、2htdp/universe を使った対話型（イベント駆動）プログラム（big-bang）を設計できます。

```
(big-bang 0
  [on-tick sub1]
  [to-draw render]
  [on-key handle-key])
```

Exercise 31. ファイルからテキストを読み込んで処理するバッチプログラムを作成せよ。

Exercise 32. big-bang を使って、画面を動く UFO やロケットのアニメーションプログラムを作成せよ。

3 プログラムの設計方法

本書の最初の数章では、プログラミングを学ぶには多くの概念をある程度習得する必要があることを示してきました。一方では、プログラミングには言語が必要です。つまり、計算したいことを伝えるための表記法です。プログラムを定式化するための言語は人工的な構成物ですが、プログラミング言語の習得は自然言語の習得といくつかの要素を共有しています。どちらも語彙、文法、そして「句」が何を意味するかの理解を伴います。

他方では、問題文からプログラムに至る方法を学ぶことが重要です。問題文の中で何が関連していて、何を無視できるかを判断する必要があります。プログラムが何を消費し、何を生成し、入力と出力をどのように関連付けるかを明らかにする必要があります。選択した言語とそのティーチパックが、プログラムが処理するデータに対する特定の基本演算を提供しているかどうかを知る（または調べる）必要があります。提供していない場合は、それらの演算を実装する補助関数を開発しなければならないかもしれません。最後に、プログラムができたなら、それが実際に意図した計算を行っているかどうかを確認しなければなりません。そして、これはあらゆる種類のエラーを明らかにする可能性があり、それらを理解して修正できなければなりません。

これらすべてはかなり複雑に聞こえ、なぜただ実験しながら適当にやって、結果がまあまあならそのままにしておけばいいのかと思うかもしれません。このプログラミングへのアプローチはしばしば「ガレージプログラミング」と呼ばれ、多くの場合に成功します。時にはスタートアップ企業の立ち上げの足がかりになることもあります。しかし、スタートアップはその「ガレージ努力」の結果を販売することはできません。なぜなら、元のプログラムとその友人しか使えないからです。

良いプログラムには、それが何をするか、どのような入力を期待するか、何を生成するかを説明する短い書き込みが付属しています。理想的には、それが実際に動作することをある程度保証するものも付きます。最良の場合、プログラムの問題文へのつながりが明白なので、問題文への小さな変更をプログラムへの小さな変更簡単に翻訳できます。ソフトウェアエンジニアはこれを「プログラミング製品」と呼びます。

このすべての追加作業が必要なのは、プログラマが自分自身のためにプログラムを作成するわけではないからです。プログラマは、他のプログラマが読むためにプログラムを書きます。そして時折、人々は仕事をするためにこれらのプログラムを実行します。

「他の」という言葉には、通常、若い頃の自分がプログラムの制作に込めた思考のすべてを思い出すことができない年上のバージョンのプログラマも含まれます。

ほとんどのプログラムは、協力し合う関数の大規模で複雑な集合であり、誰もが1日でこれらすべての関数を書くことはできません。プログラマはプロジェクトに参加し、コードを書き、プロジェクトを去ります。他の人がそのプログラムを引き継いで作業します。もう一つの困難は、プログラマのクライアントが、本当に解決してほしい問題について考えを変えがちだということです。彼らは通常、ほぼ正しく理解していますが、多くの場合、いくつかの詳細を間違えます。さらに悪いことに、プログラムのような複雑な論理的構成物は、ほとんど常に人間のエラーに悩まされます。要するに、プログラマは間違いを犯し

ます。最終的に誰かがこれらのエラーを発見し、プログラマはそれらを修正しなければなりません。彼らは1ヶ月前、1年前、または20年前のプログラムを読み直して変更する必要があります。

Exercise 33. 「year 2000」問題を調べよ。

ここでは、ステップバイステップのプロセスと、問題データを中心にプログラムを組織する方法を統合した**設計レシピ**を提示します。長い間空白の画面を見つめ続けるのが嫌いな読者にとって、この設計レシピは体系的な方法で進捗を達成する方法を提供します。他の人をプログラム設計に教える人にとって、レシピは初心者の困難を診断するための道具です。他の人々にとっては、このレシピは医学、ジャーナリズム、工学など他の分野にも適用できるものかもしれません。本物のプログラマになりたい人にとって、設計レシピは既存のプログラムを理解し作業するための方法も提供します——ただし、すべてのプログラマがこのような設計レシピのような方法を使ってプログラムを考え出すわけではありません。この章の残りは、設計レシピの世界への最初の一步を捧げます。以降の章とパートは、レシピを何らかの形で洗練・拡張していきます。

3.1 関数の設計

情報とデータ

プログラムの目的は、ある情報を消費し、新しい情報を生成する計算プロセスを記述することです。この意味で、プログラムは数学の教師が生徒に与える指示のようなものです。しかし生徒とは異なり、プログラムは数値以上のものを扱います。ナビゲーション情報を計算し、人の住所を調べ、スイッチを入れ、ビデオゲームの状態を検査します。これらすべての情報は現実世界の一部——しばしばプログラムの**ドメイン**と呼ばれる——から来ており、プログラムの計算結果はこのドメイン内のさらなる情報を表します。

情報は私たちの説明で中心的な役割を果たします。情報をプログラムのドメインに関する事実として考えてください。家具カタログを扱うプログラムにとって、「5本脚のテーブル」や「2メートル四方の正方形テーブル」は情報の断片です。ゲームプログラムは異なる種類のドメインを扱い、「5」はあるオブジェクトがキャンバスのある部分から別の部分へ移動する際の、クロックティックあたりのピクセル数を指すかもしれません。また、給与計算プログラムは「5つの控除」を扱う可能性が高いでしょう。

プログラムが情報を処理するためには、それをプログラミング言語の何らかの形の**データ**に変換しなければなりません。そしてデータを処理し、完了したら結果のデータを再び情報に変換します。対話型プログラムはこれらのステップをさらに混ぜ合わせ、必要に応じて世界からより多くの情報を取得し、その間に情報を配信するかもしれません。

私たちはBSLとDrRacketを使用するので、情報からデータへの変換について心配する必要はありません。DrRacketのBSLでは、関数をデータに直接適用して何を生成するかを観察できます。その結果、情報をデータに変換したりその逆を行う関数を書くという深刻な二つとりと卵の問題を避けられます。単純な種類の情報の場合、そのようなプログラム部品の設計は些細です。単純な情報以外のものについては、例えばパーズングを知る必要があり、それは直ちにプログラム設計における多くの専門知識を必要とします。

ソフトウェアエンジニアは、BSLとDrRacketがデータ処理を情報→データのパーズングやデータ→情報への変換から分離する方法を **model-view-controller (MVC)** というスローガンで表現します。実際、十分に設計されたソフトウェアシステムはこの分離を強制することが今や受け入れられた知恵となっています。入門書の多くは依然としてそれらを混同していますが、したがって、BSLとDrRacketで作業することで、プログラムの核心の設計に集中でき、そこに十分な経験を積んだ後、情報-データ変換部分の設計を学ぶことができます。

ここでは、データと情報の分離を示すために、2つのプリインストール済みティーチパックを使用します：2htdp/batch-ioと2htdp/universeです。この章から、バッチプログラムと対話型プログラムのための設計レシピを開発し、完全なプログラムがどのように設計されるかのアイデアを提供します。完全なプログラミング言語のティーチパックは完全なプログラムのためのより多くの文脈を提供しており、設計レシピを適切に適応させる必要があることを心に留めておいてください。

図 15: 情報からデータへ、そして戻る

情報とデータの中心的な役割を考えると、プログラム設計はそれらの間のつながりから始めなければなりません。具体的には、私たちプログラマは、選択したプログラミング言語を使って関連する情報の断片をデータとしてどのように**表現**し、データをどのように**解釈**して情報とするかを決定しなければなりません。図 15 はこの考えを抽象的な図で説明しています。

この考えを具体的にするために、いくつかの例を見てみましょう。数値の形で情報を消費・生成するプログラムを設計しているとします。表現の選択は簡単ですが、解釈には 42 のような数値がドメインで何を表すかを説明する必要があります：

- 42 は画像のドメインでは上端からのピクセル数を表すかもしれない
- 42 はシミュレーションやゲームオブジェクトが移動するクロックティックあたりのピクセル数を表すかもしれない
- 42 は物理学のドメインでは華氏、摂氏、または絶対温度目盛りでの温度を意味するかもしれない
- 42 はプログラムのドメインが家具カタログの場合、あるテーブルのサイズを指定するかもしれない
- 42 は単に文字列内の文字数を数えるかもしれない

鍵は、数値を情報として数値をデータとして（そしてその逆へ）どのように移行するかを知ることです。

この知識はプログラムを読むすべての者にとって非常に重要なので、私たちはしばしばそれをコメントの形で書き留め、**データ定義**と呼びます。データ定義には 2 つの目的があります。第一に、意味のある単語を使ってデータのあるコレクション（クラス）に名前を付けます。第二に、そのクラスの要素をどのように作成し、任意のデータ片がそのクラスに属するかどうかをどのように判断するかを読者に伝えます。

例：

```
; Temperature は Number  
; 摂氏での温度を表す
```

このデータ定義は、任意の数値を温度として解釈できることを述べています。-400 も Temperature です。読者はデータ定義から、-400 が有効な Temperature であることを理解します。

あるデータ定義が与えられたとき、読者は「このクラスに属するデータ片の例を挙げよ」と尋ねられるべきです。Temperature の場合、-273、0、100 は良い例です。読者はまた「この特定のデータはどのクラスに属するか？」と尋ねられるべきです。42 は Number、String、Image、Boolean のいずれでもありません。なぜなら文字列は Temperature に属さないからです。そして、Temperature のサンプルを作れと言われたら、-400 のようなものを思いつくかもしれません。

可能な最低温度が約-273°Cであることを知っている場合、この知識をデータ定義で表現できるかどうかを疑問に思うかもしれません。データ定義は実際にはクラスの英語による記述に過ぎないので、確かにここに示したよりもはるかに正確な方法で温度のクラスを定義できます。本書では、このようなデータ定義に様式化された英語を使用し、次の章で「-274 より大きい」などの制約を課すためのスタイルを導入します。

これまで、4つのデータクラスの名前に出会いました：Number、String、Image、Booleanです。この知識があれば、新しいデータ定義を定式化することは、既存のデータ形式に新しい名前を導入することに他なりません。例えば「temperature」を数値に与えることです。この限られた知識でも、私たちの設計プロセスの概要を説明するのに十分です。

設計プロセス

入力情報をデータとして表現する方法と、出力データを情報として解釈する方法を理解したら、個々の関数の設計は簡単なプロセスに従って進められます。

1. 情報をデータとしてどのように表現したいかを表現せよ。1行のコメントで十分です：
；センチメートルを表すために数値を使います。
2. プログラムの成功する設計に関連すると考えるデータクラスに対して、Temperatureのようなデータ定義を定式化せよ。
3. 署名、目的文、関数ヘッダを書け。

関数署名 は、設計の読者にあなたの関数がいくつの入力消費し、それらがどのクラスから来ているか、何種類のデータを生成するかを伝えるコメントです。以下は3つの例です：

```
; String -> String  
; String -> Number  
; Number String Image -> Image
```

目的文 は、関数が何をするかを1文で述べるコメントです。良い目的文は、関数を読まずに何をするかを誰でも理解できるようにします。

ヘッダ（スタブとも呼ばれる）は単純化された関数定義です。署名内の各入力クラスに対して1つの変数名を選び、関数の本体は出力クラスからの任意のデータ片にできます。

```
(define (f a-string) 0)  
(define (g n) "a")  
(define (h num str img) (empty-scene 100 100))
```

パラメータ名は、パラメータが表すデータの種類を反映すべきです。時には、パラメータの目的を示唆する名前を使いたいと思うかもしれません。

目的文を定式化するとき、パラメータ名を使って何が計算されるかを明確にすると役立つことがよくあります。

4. 関数の例を作成せよ。

各関数について、少なくとも 2~3 個の具体例を挙げ、入力と期待される出力を書きましょう。これを「指の運動」と呼びます。

```
; given: "hello", expect: "h"  
; given: "world", expect: "w"
```

5. テンプレートを書け。

データ定義から関数のアウトラインを導き出します。条件分岐が必要な場合は `cond` の骨組みを、構造体が必要ななら選択子を使った骨組みを書きます。

6. 関数定義を完成させよ。

テンプレートを埋め、目的文と例を使って実際の計算ロジックを記述します。

7. テストを行え。

例を `check-expect` に変換し、すべてが通ることを確認します。

図 1: 関数設計レシピの基本ステップ（前述の 6 ステップの要約図）

このプロセスは初心者が進捗を確実にし、エラーを早期に発見するのを助けます。

関数の例から設計へ

ここでは、文字列の最初の文字を抽出する簡単な関数 `string-first` を設計する例を示します。

（コード例は原文のまま）

```
; String -> String  
; 文字列 s の最初の文字を抽出する  
(define (string-first s) "a")  
  
; 例:  
; given: "hello", expect: "h"  
; given: "world", expect: "w"  
; given: " ", expect: " "  
  
(check-expect (string-first "hello") "h")  
(check-expect (string-first "world") "w")
```

（以降の章の例も同様に設計レシピを適用して進めます。）

3.2 指の運動: 関数

この節の最初のいくつかの練習問題は、前の「関数」節のものとほぼコピーです。ただし、後方で「define」という単語を使っているところを、ここでは「design」という単語を使っています。この違いが意味するのは、設計レシピに従ってこれらの関数を作成し、解答には関連するすべての部品を含めるべきだということです。

節のタイトルが示唆するように、これらはプロセスを身体化するための練習問題です。ステップが第二の天性になるまで、1つも飛ばさないでください。飛ばすと簡単に避けられるエラーにつながります。プログラミングには複雑なエラーの余地がたくさん残されています。ばかばかしいエラーに時間を無駄にする必要はありません。

Exercise 34. 関数 `string-first` を設計せよ。文字列 `s` から最初の文字を抽出する。

Exercise 35. 関数 `string-last` を設計せよ。文字列 `s` から最後の文字を抽出する。

Exercise 36. 関数 `image-area` を設計せよ。画像 `img` の面積を計算する。

Exercise 37. 関数 `string-rest` を設計せよ。文字列 `s` から最初の文字を除いた残りを返す。

Exercise 38. 関数 `string-remove-last` を設計せよ。文字列 `s` から最後の文字を除いた残りを返す。

(本書ではさらに多くの練習問題が続き、すべて設計レシピの適用を練習するものです。)

3.3 ドメイン知識

関数の本体をコード化するのにどのような知識が必要かを不思議に思うのは自然です。少し反省すると、このステップにはプログラムのドメインに対する適切な把握が必要であることがわかります。実際、そのような**ドメイン知識**には2つの形態があります：

1. 外部ドメインからの知識——数学、音楽、生物学、土木工学、芸術など。プログラマはコンピューティングのすべての応用ドメインを知ることはできないため、ドメイン専門家と問題を議論できるように、さまざまな応用分野の言語を理解する準備ができていなければなりません。数学は多くの（すべてではない）ドメインの交差点にあります。したがって、プログラマはドメイン専門家と問題を処理する際に新しい言語を習得しなければならないことがよくあります。
2. ティーチング言語とそのライブラリに関する知識。プログラマは、与えられた問題に対して言語が何を提供しているかを知る必要があります。

設計プロセスの中で、ドメイン知識は主にステップ5（関数定義を完成させる）で必要になりますが、良い例を作成するためにも役立ちます。

3.4 関数からプログラムへ

すべてのプログラムが単一の関数定義から成るわけではありません。いくつかの関数が必要とするものもあります。多くのものは定数定義も使用します。いずれにせよ、すべての関数を体系的に設計することは常に重要ですが、グローバル定数や補助関数は設計プロセスを少し変えます。

グローバル定数を定義した場合、関数は結果を計算するためにそれらを使用するかもしれませんが。それらの存在を自分に思い出させるために、テンプレートにこれらの定数を追加するとよいでしょう。結局のところ、それらは関数定義に寄与し得るものの目録に属します。

複数関数のプログラムは、対話型プログラムが自動的にキーイベントやマウスイベントを処理する関数、状態を画像として描画する関数などを必要とするために生じます。各補助関数も設計レシピに従って設計すべきです。

プログラム全体にはトップレベルの目的文も書くべきです。

3.5 テストについて

テストはすぐに労働集約的な作業になります。小さなプログラムを対話領域で実行するのは簡単ですが、多くの機械的な作業と複雑な検査が必要です。プログラマがシステムを成長させるにつれ、多くのテストを実施したいと思うようになります。やがてこの作業は圧倒的になり、プログラマはそれを怠るようになります。同時に、テストは基本的な欠陥を発見・防止するための最初の道具です。ずさんなテストはすぐにバグのある関数——隠れた問題を抱えた関数——につながり、バグのある関数はプロジェクトをしばしば複数の方法で遅らせます。

したがって、手動でテストを行うのではなく、テストを機械化することが重要です。多くのプログラミング言語と同様に、BSL にはテスト機能が含まれており、DrRacket はこの機能を認識しています。

check-expect を使って例を自動テストに変換します：

```
(check-expect (string-first "hello") "h")
```

DrRacket は RUN 時にすべてのテストを実行し、結果を表示します。失敗したテストは赤で表示され、デバッグに役立ちます。

良いテストには以下が含まれます： - 典型的なケース - 境界値（空文字列、長さ 1 の文字列など） - 特殊ケース

テストはまた、プログラムのドキュメントとしても機能します。

3.6 World プログラムの設計

前の章では 2htdp/universe ティーチパックをアドホックな方法で紹介しましたが、この節では**設計レシピ**がどのようにして world プログラムを体系的に作成するのを助けるかを示します。まず、データ定義と関数署名に基づく 2htdp/universe ティーチパックの簡単な要約から始めます。次に、world プログラムのための設計レシピを詳述します。

ティーチパックは、プログラマが世界の状態を表すデータ定義と、すべての可能な世界状態に対して画像を作成する方法を知っている関数 `render` を開発することを期待します。プログラムの必要性に応じて、プログラマは次にクロックティック、キー入力、マウスイベントに応答する関数を設計しなければなりません。最後に、対話型プログラムは `big-bang` を使って状態、描画関数、イベントハンドラを結び付ける必要があります。

world プログラム設計レシピの概要：

1. 世界の状態を表すデータ定義を書く。
2. `render` 関数の署名と目的文を書く：`WorldState -> Image`
3. 必要に応じて `on-tick`、`on-key`、`on-mouse` などのハンドラの署名を書く。
4. 各関数について設計レシピを適用する。
5. `big-bang` でプログラムを組み立てる。

例として、シンプルなカウンターやアニメーションを設計できます。

3.7 バーチャルペットの世界

この練習問題の節は、バーチャルペットゲームの最初の2つの要素を紹介します。キャンバスを横切って歩き続ける猫の表示から始まります。もちろん、歩き続けると猫は不幸になり、その不幸が現れます。すべてのペットと同様に、撫でて少し助けるか、餌を与えてたくさん助けることができます。

まず、私たちのお気に入りの猫の画像から始めましょう：

```
(define cat1 <画像>)
```

猫の画像をコピーして DrRacket に貼り付け、define で画像に名前を付けましょう。

Exercise 45. 「バーチャルキャット」の world プログラムを設計せよ。猫を左から右に継続的に動かす。cat-prog と呼び、猫の開始位置を消費すると仮定せよ。さらに、猫をクロックティックあたり 3 ピクセル動かせ。猫が右端で消えたら左端に再出現させる。modulo 関数について読むとよい。

Exercise 46. 少し異なる画像で猫のアニメーションを改善せよ：

```
(define cat2 <画像>)
```

Exercise 45 の描画関数を調整し、x 座標が奇数かどうかで 2 つの猫画像のどちらかを使うようにせよ。HelpDesk で odd? を調べ、cond 式を使って猫画像を選択せよ。

Exercise 47. 「幸福ゲージ」を維持・表示する world プログラムを設計せよ。gauge-prog と呼び、幸福度は 0 から 100（両端を含む）の数値であると合意せよ。

各クロックティックで幸福レベルは 0.1 減少する。ただし 0 を下回らない。下矢印キーが押されるたびに幸福度は 1/5 減少、上矢印が押されると 1/3 ジャンプする。

幸福レベルを表示するために、黒い枠のある塗りつぶしの赤い長方形を持つシーンを使い。幸福レベル 0 では赤いバーは消え、最大幸福レベル H ではバーがシーン全体にわたるようにせよ。

Note. 十分な知識を得たら、gauge-prog を Exercise 45 の解と組み合わせる方法を説明する。その後、猫を助けることができるようになる。無視し続けると猫はますます不幸になる。撫でるとより幸せに、餌をやるとずっとずっと幸せになる。だからこそ、これらの最初の 3 章が教えてくれるよりもずっと多くの world プログラムの設計について知りたいと思う理由がわかるだろう。

注意: この章のすべての Racket コード、関数定義、check-expect、画像リテラル、big-bang 式などは**原文のまま**保持しています。翻訳は説明文、演習の記述、注記、設計プロセスの解説のみです。

原典: How to Design Programs, Second Edition — Chapter 3 “How to Design Programs”
URL: https://htdp.org/2026-5-28/Book/part_one.html ## 4 区間、列挙、項目化 (Intervals, Enumerations, and Itemizations)

現時点では、情報をデータとして表現するための選択肢は、数値、文字列、画像、真偽値の4つしかありません。多くの問題に対してはこれで十分ですが、これら4つの基本データの集合だけでは対応しきれない複雑な問題も多く存在します。優れたソフトウェア設計者には、情報を表現するためのさらなる仕組みが必要です。

少なくとも、プログラムを設計する際には、これらの組み込みデータの集合に対して「制約」を課す方法を学ばねばなりません。制約を課す一つの方法は、データ集合からいくつかの具体的なデータだけをリストアップし、この問題においてはそれらしか使用しないと合意することです。このように要素をリストアップする方法を**列挙 (Enumeration)**と呼びますが、これは要素数が有限の場合にのみ有効です。一方で、「無限に」多くの要素を含む集合を扱うために、特定の条件を満たす要素の範囲を指定する**区間 (Interval)**を導入します。

[!NOTE] ここでいう「無限」とは、「要素数が多すぎて、一つひとつリストアップするのが完全に不可能である」という現実的な意味も含まれます。

列挙や区間を定義することは、異なる種類の要素を区別することを意味します。プログラムコード内でこれらを区別するには、**条件分岐関数 (Conditional Functions)**、すなわち引数の値に応じて異なる計算方法を選択する関数が必要です。「プロローグ：プログラムの仕方」や「ブール値と混ぜ合わせ」では、こうした条件分岐関数の書き方をいくつかの例で示しましたが、いずれも体系的な設計手法（設計レシピ）は用いていませんでした。本章では、列挙、区間、そしてこれらを組み合わせた新しいデータ表現方法である**項目化 (Itemization)**の一般的な設計手法を解説します。

まず、cond 式の使い方を詳細に見直し、その後に列挙、区間、項目化の3つのデータ定義方法について説明します。列挙はそのクラスに属するすべてのデータを一つずつリストアップするものであり、区間はデータの範囲を指定するものです。最後の項目化は、定義の一部で範囲を指定し、別の部分で特定のデータを指定するという、両者を混ぜ合わせたものです。最後に、これらのデータ型に対する一般的な設計レシピ（設計戦略）を整理します。

4.1 条件式を使ったプログラミング (Programming with Conditionals)

「プロローグ：プログラムの仕方」で学んだ条件式を思い出してください。cond 式は本書で最も複雑な式の形であるため、その一般的な形式を詳しく見てみましょう。

```
(cond
  [ConditionExpression1 ResultExpression1]
  [ConditionExpression2 ResultExpression2]
  ...
  [ConditionExpressionN ResultExpressionN])
```

cond 式はキーワードである (cond で始まり、対応する) で終わります。キーワードに続いて、必要なだけの cond 行（これを **cond 節** と呼びます）を記述します。各 cond 節は、角括弧 [と] で囲まれた2つの式で構成されます（※丸括弧 () を使っても文法上問題ありませんが、視認性を高めるために角括弧を使うのが一般的です）。

条件式を使用した関数定義の例を見てみましょう：

```
(define (next traffic-light-state)
  (cond
    [(string=? "red" traffic-light-state) "green"]
    [(string=? "green" traffic-light-state) "yellow"]
    [(string=? "yellow" traffic-light-state) "red"])))
```

この例は、cond 式を使用することの便利さを示しています。多くの問題において、関数は複数の異なる状況を区別しなければなりません。cond 式を使用すれば、発生しうる可能性ごとに1つの cond 節を割り当てることができ、問題文で示された異なる状況とプログラムコードとの対応関係を読者に対して明確に示すことができます。

if 式（「ブール値と混ぜ合わせ」を参照）との使い分けについての注意：if 式は「ある1つの状況と、それ以外のすべての状況」を区別するために使われます。そのため、3つ以上の状況を区別する必要がある文脈には適していません。if 式は「A か B か」の二者択一を簡潔に表現する場合にのみ使用すべきです。したがって、データ定義から導き出される複数の明確に区別された状況をコードで表現する場合には、常に cond 式を使用します。

cond 式の条件が複雑になり、「その他のすべての場合」と表現したい場合があります。このような場合、cond 式の最後の節に限って else キーワードを使用することが許可されています。

```
(cond
  [ConditionExpression1 ResultExpression1]
  [ConditionExpression2 ResultExpression2]
  ...
  [else DefaultResultExpression])
```

もし最後以外の cond 節で else を使用すると、DrRacket の BSL はエラーを返します：

```
> (cond
  [(> x 0) 10]
  [else 20]
  [< x 10) 30])
cond:found an else clause that isn't the last clause in its cond expression
```

文法的に正しくないプログラムコードは意味を持たないため、BSL は実行前にこれを拒否します。

ゲームプログラムの一部として、ゲーム終了時のスコアに応じて賞（アワード）を計算する関数 reward を設計するとします。関数のヘッダとデータ定義は以下の通りです：

```
; A PositiveNumber is a Number greater than/equal to 0.
; PositiveNumber -> String
; computes the reward level from the given score s
```

この関数を実装する2つのバリエーションを比較してみましょう：

左側の実装（3つの明確な条件を使用）：

```
(define (reward s)
  (cond
    [(<= 0 s 10) "bronze"]
    [(and (< 10 s) (<= s 20)) "silver"]
    [(< 20 s) "gold"])))
```

右側の実装 (else 節を使用) :

```
(define (reward s)
  (cond
    [(<= 0 s 10) "bronze"]
    [(and (< 10 s) (<= s 20)) "silver"]
    [else "gold"])))
```

左側の実装では、3 番目の条件として ($< 20\ s$) を正しく記述するために、以下の計算が成り立つことをプログラマが頭の中で保証する必要があります： s は非負の数値であること。 $(\leq 0\ s\ 10)$ が `#false` であること。 $(\text{and } (< 10\ s) (\leq s\ 20))$ もまた `#false` であること。

この例では計算は簡単に見えますが、複雑な条件分岐では境界値の計算ミスによってプログラムにバグ (不具合) が混入しやすくなります。したがって、最後のケースが他の条件の完全な「補集合 (それ以外)」であることが明らかな場合は、右側のように `else` 節を使用する方が安全で優れています。

4.2 条件付きの計算 (Computing Conditionally)

DrRacket が `cond` 式をどのように評価するかをより正確に見ていきましょう。先ほどの `reward` 関数の定義を考えます：

```
(define (reward s)
  (cond
    [(<= 0 s 10) "bronze"]
    [(and (< 10 s) (<= s 20)) "silver"]
    [else "gold"])))
```

関数を呼び出す際、`cond` 式単体を見ただけではどの節が実行されるかを予測することはできません。これが関数の目的そのものです。関数は多様な入力 (例えば 3, 12, 21 など) を受け取り、それぞれの入力に対して異なる振る舞いをします。入力データのバリエーションを区別して処理することが `cond` 式の役割です。

具体的な評価ステップを見てみましょう。まず、`(reward 3)` を呼び出した場合です：

```
(reward 3)
==
(cond
  [(<= 0 3 10) "bronze"]
  [(and (< 10 3) (<= 3 20)) "silver"]
  [else "gold"])
==
(cond
  [#true "bronze"]
  [(and (< 10 3) (<= 3 20)) "silver"]
  [else "gold"])))
```



```

    [else "gold"])
==
"bronze"

```

ここでは、最初の条件 ($\leq 0 \ 3 \ 10$) が `#true` に評価されたため、対応する結果式である `"bronze"` が `cond` 式全体の値となります。

次に、`(reward 21)` を呼び出した場合を見てみます：

```

(reward 21)
==
(cond
  [( $\leq 0 \ 21 \ 10$ ) "bronze"]
  [(and ( $< 10 \ 21$ ) ( $\leq 21 \ 20$ )) "silver"]
  [else "gold"])
==
(cond
  [#false "bronze"]
  [(and ( $< 10 \ 21$ ) ( $\leq 21 \ 20$ )) "silver"]
  [else "gold"])
==
(cond
  [(and ( $< 10 \ 21$ ) ( $\leq 21 \ 20$ )) "silver"]
  [else "gold"])

```

最初の条件が `#false` と評価されたため、その `cond` 節全体が評価対象から取り除かれ、次の節へと計算が進みます：

```

(cond
  [(and ( $< 10 \ 21$ ) ( $\leq 21 \ 20$ )) "silver"]
  [else "gold"])
==
(cond
  [(and #true ( $\leq 21 \ 20$ )) "silver"]
  [else "gold"])
==
(cond
  [(and #true #false) "silver"]
  [else "gold"])
==
(cond
  [#false "silver"]
  [else "gold"])
==
(cond
  [else "gold"])
==
"gold"

```

このように、1 番目と 2 番目の条件が共に #false であったため、最後の else 節へと処理が移行し、結果として "gold" が得られます。

Exercise 48. DrRacket の定義領域に reward 関数の定義と (reward 18) を入力し、ステップ実行 (Stepper) を使用して評価のプロセスを確認せよ。

Exercise 49. cond 式は式的一种であるため、他の式の中にネスト (入れ子に) して記述することができます。

```
(- 200 (cond [(> y 200) 0] [else y]))
```

y の値が 100 の場合と 210 の場合について、Stepper を使ってこの式がどのように評価されるかを確認せよ。

また、図 20 に示す UFO/ロケットのアニメーションシーン描画関数 create-rocket-scene.v5 をリファクタリングし、cond 式をネストさせることで、place-image の記述が一度だけで済むように実装を書き換えよ。

```
(define WIDTH 100)
(define HEIGHT 60)
(define MTSCN (empty-scene WIDTH HEIGHT))
(define ROCKET <画像>)
(define ROCKET-CENTER-TO-TOP (- HEIGHT (/ (image-height ROCKET) 2)))

; 元の実装
(define (create-rocket-scene.v5 h)
  (cond
    [(<= h ROCKET-CENTER-TO-TOP) (place-image ROCKET 50 h MTSCN)]
    [(> h ROCKET-CENTER-TO-TOP) (place-image ROCKET 50 ROCKET-CENTER-TO-TOP MTSCN)]))
```

4.3 列挙 (Enumerations)

すべての文字列が任意に使えるわけではありません。例えば、マウスイベントをプログラムに通知する文字列は以下の 6 つに制限されています：

```
; A MouseEvent is one of these Strings:
; - "button-down"
; - "button-up"
; - "drag"
; - "move"
; - "enter"
; - "leave"
```

このように、取りうるすべての具体的なデータを網羅して定義するデータ型を**列挙 (Enumeration)**と呼びます。日常のプログラム設計において、列挙型は非常によく現れます。例えば、信号機の状態を以下のように定義できます：

```
; A TrafficLight is one of the following Strings:
; - "red"
; - "green"
```

```
; - "yellow"
; interpretation: the three strings represent the three possible states of a traffic light
```

列挙データ型を消費する関数の設計は非常にシンプルです。入力データがケースバイケースで列挙されている場合、関数側でもそれぞれのケースを `cond` 節で区別して処理します。信号機の状態を受け取って、次の信号機の状態を返す関数 `traffic-light-next` は以下のように設計できます：

```
; TrafficLight -> TrafficLight
; yields the next state given current state s
(check-expect (traffic-light-next "red") "green")
```

```
(define (traffic-light-next s)
  (cond
    [(string=? "red" s) "green"]
    [(string=? "green" s) "yellow"]
    [(string=? "yellow" s) "red"]))
```

`TrafficLight` の定義が3つのデータ要素で構成されているため、`traffic-light-next` 関数もまた自然に3つの `cond` 節で分岐を行います。

Exercise 50. 上記の `traffic-light-next` 関数の定義を `DrRacket` にコピー＆ペーストして `RUN` すると、テストが不十分なため、いくつかの `cond` 節がハイライト表示（テスト未通過）されます。すべての節がカバーされるようにテストケースを追加せよ。

Exercise 51. 信号機のアニメーションをシミュレートする `big-bang` プログラムを設計せよ。信号機の状態を適切な色の円で描画し、クロックティックごとに状態（色）が遷移するようにせよ。最も適切な初期状態について検討せよ。

列挙型の本質は、データ集合を「有限個の具体的なデータ片」として定義することです。もう一つの重要な例を紹介します：

```
; A 1String is a String of length 1, including:
; - "\" (the backslash),
; - " " (the space bar),
; - "  " (tab),
; - "
" (return), and
; - "" (backspace).
; interpretation: represents keys on the keyboard
```

BSL では、キーボードの入力イベント (`KeyEvent`) を以下のように表現します：

```
; A KeyEvent is one of:
; - 1String
; - "left"
; - "right"
; - "up"
; - "down"
; - ...
```

この定義に従うと、キーイベントを処理する関数（キーイベントハンドラ）の設計テンプレートは自然と以下ようになります：

```
; WorldState KeyEvent -> WorldState
(define (handle-key-events w ke)
  (cond
    [(= (string-length ke) 1) ...]
    [(string=? "left" ke) ...]
    [(string=? "right" ke) ...]
    [(string=? "up" ke) ...]
    [(string=? "down" ke) ...]
    [else ...]))
```

Exercise 52. キーボードの左右の矢印キーが押されたときに、赤いドットを左右に移動させるキーイベントハンドラ `keh` を設計せよ。不要なキーイベントは `else` 節で無視すること。図 21 に示す以下のコードを参考にせよ。

```
; A Position is a Number.
; interpretation: distance between the left margin and the ball
; Position KeyEvent -> Position
; computes the next location of the ball
```

```
(check-expect (keh 13 "left") 8)
(check-expect (keh 13 "right") 18)
(check-expect (keh 13 "a") 13)
```

```
(define (keh p k)
  (cond
    [(string=? "left" k) (- p 5)]
    [(string=? "right" k) (+ p 5)]
    [else p]))
```

4.4 区間 (Intervals)

UFO が降下する様子を描画するプログラムを考えます（図 22 を参照）。

```
; A WorldState is a Number.
; interpretation: number of pixels between the top and the UFO
(define WIDTH 300)
(define HEIGHT 100)
(define CLOSE (/ HEIGHT 3))
(define MTSCN (empty-scene WIDTH HEIGHT))
(define UFO (overlay (circle 10 "solid" "green") (rectangle 30 4 "solid" "green")))
```

```
; WorldState -> WorldState
(define (main y0)
  (big-bang y0
    [on-tick nxt]
    [to-draw render]))
```

```
; WorldState -> WorldState
```

```
(define (nxt y) (+ y 3))
```

```
; WorldState -> Image
(define (render y)
  (place-image UFO (/ WIDTH 2) y MTSCN))
```

この UFO シミュレーションにおいて、UFO の高さ（位置）に応じて以下のようなステータス画面を表示させたいとします： * UFO の高さが画面上部の 3 分の 1（CLOSE）より上にあるときは "descending"（降下中）。 * それを超えて下部（HEIGHT）までの間にあるときは "closing in"（接近中）。 * 画面の底（HEIGHT）に達したときは "landed"（着陸）。

この場合、状態を表す数値は無限に存在するため、単純に個々の値を列挙することは不可能です。代わりに、境界値を用いて数値の範囲を切り分ける **区間 (Intervals)** を使用します。

区間は、境界値（左端と右端）によって定義されます。境界値自体が範囲に含まれる場合を「閉じた (closed)」、含まれない場合を「開いた (open)」と呼びます。数学の表記に倣い、閉じた境界にはブラケット [や]、開いた境界にはペアレントシス (や) を用いて表現します： * $[3, 5]$: 閉区間 (3 以上 5 以下) * $(3, 5]$: 左開・右閉区間 (3 より大きく 5 以下) * $[3, 5)$: 左閉・右开区間 (3 以上 5 未満) * $(3, 5)$: 开区間 (3 より大きく 5 未満)

これを用いると、世界の状態 WorldState は以下の 3 つの区間に分類できます：1. 0 から CLOSE の間（降下中） 2. CLOSE から HEIGHT の間（接近中） 3. HEIGHT 以上（着陸）

これをプログラムで表現する場合、境界値がどの区間に属するかという「重複」を避ける必要があります。BSL の cond 条件式は順番に評価されるため、この性質を利用して以下のように記述できます：

```
; WorldState -> Image
(define (render/status y)
  (cond
    [(<= 0 y CLOSE)
     (place-image (text "descending" 11 "green") 10 10 (render y))]
    [(and (< CLOSE y) (<= y HEIGHT))
     (place-image (text "closing in" 11 "orange") 10 10 (render y))]
    [(> y HEIGHT)
     (place-image (text "landed" 11 "red") 10 10 (render y))]))
```

UFO のアニメーションプログラムをこのステータス表示付きのバージョンにするには、main 関数で [to-draw render/status] を呼び出すように変更します。

UFO の位置情報や表示位置を一箇所で管理するために、UFO の座標やテキスト表示位置の「コピー」を避ける実装を設計するのが賢明です。図 24 のように、cond 式自体を place-image の引数として入れ子にすることができます：

```
(define (render/status y)
  (place-image
    (cond
      [(<= 0 y CLOSE) (text "descending" 11 "green")]
```

```

    [(and (< CLOSE y) (<= y HEIGHT)) (text "closing in" 11 "orange")]
    [else (text "landed" 11 "red")])
  20 20
  (render y)))

```

4.5 項目化 (Itemizations)

列挙型は具体的な有限のデータ片をリストアップし、区間は無限に存在する数値の範囲を指定します。これら両方を混在させて定義するデータ型を**項目化 (Itemizations)**と呼びます。

例えば、string->number 関数の出力結果を考えてみましょう。この関数は、文字列が数値に変換可能な場合は数値を返し、不可能な場合は #false を返します。この結果を表すデータ型 NorF は以下のように定義できます：

```

; An NorF is one of:
; - #false
; - a Number

```

これは、具体的なデータ片である #false と、無限のデータクラスである Number を組み合わせた項目化の典型例です。この NorF を消費する関数の設計は、やはりそれぞれのケース（項目）に対応する cond 節を持つ形になります：

```

; NorF -> Number
; adds 3 to the given number; 3 otherwise
(check-expect (add3 #false) 3)
(check-expect (add3 0.12) 3.12)

```

```

(define (add3 x)
  (cond
    [(false? x) 3]
    [else (+ x 3)]))

```

もう少し実用的な例として、ロケットの打ち上げカウントダウンをシミュレートするプログラムを設計します。

```

; An LRCD (for launching rocket countdown) is one of:
; - "resting"
; - a Number between -3 and -1
; - a NonnegativeNumber
; interpretation:
;   "resting" represents a grounded rocket.
;   a number in [-3, -1] denotes the countdown phase.
;   a nonnegative number denotes the height of the rocket in flight.

```

このデータ型に対するシミュレーションプログラムを以下のように構成します： * 定数定義:

```

(define HEIGHT 300)
(define WIDTH 100)
(define YDELTA 3)

```

```
(define BACKG (empty-scene WIDTH HEIGHT))
(define ROCKET (rectangle 5 30 "solid" "red"))
(define CENTER (/ (image-height ROCKET) 2))
```

• 描画関数 show:

```
; LRCD -> Image
(define (show x)
  (cond
    [(string? x)
     (place-image ROCKET 10 (- HEIGHT CENTER) BACKG)]
    [(<= -3 x -1)
     (place-image (text (number->string x) 20 "red")
                   10 (* 3/4 WIDTH)
                   (place-image ROCKET 10 (- HEIGHT CENTER) BACKG))]
    [(>= x 0)
     (place-image ROCKET 10 (- x CENTER) BACKG))]))
```

• キーイベントハンドラ launch:

```
; LRCD KeyEvent -> LRCD
(define (launch x ke)
  (cond
    [(string? x) (if (string=? " " ke) -3 x)]
    [(<= -3 x -1) x]
    [(>= x 0) x]))
```

• 時間経過ハンドラ fly (カウントダウンおよび飛行のシミュレーション) :

```
; LRCD -> LRCD
(define (fly x)
  (cond
    [(string? x) x]
    [(<= -3 x -1) (if (= x -1) HEIGHT (+ x 1))]
    [(>= x 0) (- x YDELTA))]))
```

• メインプログラム実行:

```
(define (main2 s)
  (big-bang s
    [to-draw show]
    [on-key launch]
    [on-tick fly 1])) ; クロックティックは1秒ごと
```

Exercise 53~57. 以上のカウントダウン付きロケットプログラムを実際に DrRacket 上で構築・実行し、終了処理 (stop-when) の追加などの拡張を試みよ。

4.6 項目化を用いた設計 (Designing with Itemizations)

これまでに学んだ内容を基に、「From Functions to Programs」で紹介した設計レシピを拡張し、列挙、区間、項目化を扱う関数を体系的に設計するためのステップを定義します。

以下の消費税計算を例に、ステップを確認します：> 安価な商品（1,000 ドル未満）には税金がかかりません。> 高級品（10,000 ドル超）には 8% の税率が適用されます。> それらの中間の商品には 5% の税金がかかります。

1. **データ定義 (Data Definition)** 問題文で区別されている状況をカバーする、重複のない明確なデータ定義を記述します。

```
; A Price falls into one of three intervals:  
; — 0 through 1000 (exclusive)  
; — 1000 through 10000 (exclusive)  
; — 10000 and above.
```

2. **署名、目的文、ヘッダ (Signature, Purpose, Header)**

```
; Price -> Number  
; computes the amount of tax charged for p  
(define (sales-tax p) 0)
```

3. **具体的な例 (Examples)** それぞれの区間、および境界値からテストデータを抽出します（境界値の扱いについて専門家や仕様書に確認することが重要です）。

```
(check-expect (sales-tax 537) 0)  
(check-expect (sales-tax 1000) (* 0.05 1000))  
(check-expect (sales-tax 12017) (* 0.08 12017))
```

4. **テンプレート (Template)** データ定義の項目（サブクラス）の数と同じ数の `cond` 節を持つ条件分岐のアウトラインを作成します。

```
(define (sales-tax p)  
  (cond  
    [(and (<= 0 p) (< p 1000)) ...]  
    [(and (<= 1000 p) (< p 10000)) ...]  
    [(>= p 10000) ...]))
```

5. **定義の完成 (Definition)** 各節の空欄 ... を、パラメータを用いた具体的な計算式で埋めます。

```
(define (sales-tax p)  
  (cond  
    [(and (<= 0 p) (< p 1000)) 0]  
    [(and (<= 1000 p) (< p 10000)) (* 0.05 p)]  
    [(>= p 10000) (* 0.08 p)]))
```

6. **テスト (Testing)** テストを実行し、すべての `cond` 節がカバーされ、期待通りの値が返ることを確認します。

4.7 有限状態世界 (Finite State Worlds)

信号機のように、「有限個の状態」と「状態間の遷移ルール」によって定義されるシステムを**有限状態機械 (Finite State Machine: FSM)** または**有限状態オートマトン (Finite State Automaton: FSA)** と呼びます。

信号機のシミュレーションプログラムは、以下の要素で構成できます： * **状態の定義**: "red", "green", "yellow" * **時間経過による遷移関数** `tl-next`: racket ; TrafficLight -> TrafficLight (define (tl-next cs) (cond [(string=? "red" cs) "green"] [(string=? "green" cs) "yellow"] [(string=? "yellow" cs) "red"]])) * **描画関数** `tl-render`: 状態に応じて適切な色のライトを表示する画像を生成する。

Exercise 59~61. 信号機の有限状態シミュレーションプログラムを完成させよ。

自動ドアクローザーのシミュレーション

自動ドアは以下の状態と遷移ルールを持ちます： * **状態 (DoorState)**: * LOCKED (施錠中) * CLOSED (閉扉中・未施錠) * OPEN (開扉中) * **イベントルール**: * LOCKED 状態で "u" キーを押すと CLOSED になる (解錠)。 * CLOSED 状態で "l" キーを押すと LOCKED になる (施錠)。 * CLOSED 状態でスペースキー " " を押すと OPEN になる (開扉)。 * OPEN 状態からは、クロックティック (時間経過) により自動的に CLOSED に戻る。

このシミュレーションプログラムは以下のように設計できます：

```
(define LOCKED "locked")
(define CLOSED "closed")
(define OPEN "open")

; DoorState -> DoorState
; closes an open door over the period of one tick
(define (door-closer state-of-door)
  (cond
    [(string=? LOCKED state-of-door) LOCKED]
    [(string=? CLOSED state-of-door) CLOSED]
    [(string=? OPEN state-of-door) CLOSED]))

; DoorState KeyEvent -> DoorState
; turns key event k into an action on state s
(define (door-action s k)
  (cond
    [(and (string=? LOCKED s) (string=? "u" k)) CLOSED]
    [(and (string=? CLOSED s) (string=? "l" k)) LOCKED]
    [(and (string=? CLOSED s) (string=? " " k)) OPEN]
    [else s]))

; DoorState -> Image
; translates the state s into a large text image
(define (door-render s)
  (text s 40 "red"))

; DoorState -> DoorState
(define (door-simulation initial-state)
  (big-bang initial-state
    [on-tick door-closer 3] ; 3秒ごとにドアが閉まる
```

```
[on-key door-action]  
[to-draw door-render]))
```

Exercise 62. ドアシミュレータを DrRacket で実行し、動作を確認せよ。

5 構造の追加 (Adding Structure)

ある架空の完璧な部屋の床と天井の間で、直線の垂直線上を往復して跳ね返るボールをシミュレートするワールドプログラムを設計したいとしましょう。ボールは 1 クロックティックごとに常に 2 ピクセル移動すると仮定します。デザインレシピに従うなら、最初の目標は時間の経過に伴って変化するもののデータ表現を開発することです。ここで、ボールの位置と方向は時間の経過とともに変化しますが、これは 2 つの値であるのに対し、big-bang が追跡するのは 1 つの値だけです。したがって、1 つのデータがどのようにして 2 つの変化する情報量を表現できるのかという疑問が生じます。

Note: 数学者は、元の数値を復元できるように 2 つの数値を 1 つの数値に「マージ (統合)」するトリックを知っています。プログラマーは、このようなトリックがプログラムの真の意図を曖昧にするため、悪質であると考えています。

同じ疑問を提起するもう 1 つのシナリオを紹介します。あなたの携帯電話は、要するにプラスチックで包まれた数百万行のコードです。とりわけ、携帯電話は連絡先を管理しています。各連絡先には、名前、電話番号、メールアドレス、そしておそらくその他のいくつかの情報が含まれています。多数の連絡先がある場合、単一の連絡先は 1 つのデータとして表現するのが最適です。そうでなければ、さまざまな情報が誤って混ざってしまう可能性があるからです。

このようなプログラミング上の問題があるため、すべてのプログラミング言語は、複数のデータを 1 つの複合データ (compound data) に組み合わせるメカニズムと、必要に応じて構成値を抽出する方法を提供しています。本章では、BSL のメカニズムであるいわゆる構造体型定義 (structure type definitions) と、複合データを扱うプログラムの設計方法について紹介します。

5.1 位置から posn 構造体へ (From Positions to posn Structures)

ワールドキャンバス上の位置は、左端からの距離と上端からの距離という 2 つのデータによって一義的に特定されます。前者は x 座標 (x-coordinate) と呼ばれ、後者は y 座標 (y-coordinate) と呼ばれます。

基本的に BSL プログラムである DrRacket は、そのような位置を posn 構造体 (posn structures) で表現します。posn 構造体は 2 つの数を 1 つの値に組み合わせます。2 つの数を消費して posn を作成する操作 make-posn を使用して、posn 構造体を作成できます。例えば、

```
(make-posn 3 4)
```

は、x 座標が 3 で y 座標が 4 である posn 構造体を作成する式です。

posn 構造体は、数やブーリアン、文字列と同じステータスを持ちます。特に、基本操作と関数の両方が構造体を消費し、生成することができます。また、プログラムは posn 構造体に名前を付けることができます。

```
(define one-posn (make-posn 8 6))
```

Stop! 座標の観点から one-posn を説明してください。

他のことを行う前に、posn 構造体の計算規則 (laws of computation) を見てみましょう。そうすることで、posn 構造体进行处理する関数を作成し、それが何を計算するかを予測できるようになります。

5.2 posn を用いた計算 (Computing with posns)

関数や関数の規則は初等代数から完全に馴染み深いものですが、posn 構造体は新しい考え方のように見えます。とはいえ、posn の概念は、以前に出会ったことがあるかもしれない直交座標系の点や平面上の位置に似ているはずです。

```
+-----+
| Figure 29: A Cartesian point |
|                               |
| [image: pict_65.png]        |
| [image: pict_65.png]        |
+-----+
```

直交座標の要素を選択することも馴染みのあるプロセスです。例えば、先生が「figure 29 のグラフを見て、px と py が何であるか教えてください」と言った場合、あなたはそれぞれ 31 と 26 と答えるでしょう。なぜなら、p から放射状に伸びる垂直線と水平線が軸と交わる値を読み取る必要があることを知っているからです。

Note: plot ライブラリを提供してくれた Neil Toronto に感謝します。

この考え方は BSL で表現できます。次を定義領域 (definitions area) に追加すると仮定します。

```
(define p (make-posn 31 26))
```

定義領域に追加し、RUN をクリックして、次の操作 (interactions) を実行します。

```
> (posn-x p)
31
> (posn-y p)
26
```

p を定義することは、直交座標平面上に点を打つようなものです。posn-x と posn-y を使用することは、p にインデックスで下付き文字を付けるようなものです (px と py)。

計算の観点から言えば、posn 構造体には 2 つの方程式が付属しています。

$$(\text{posn-x}(\text{make-posn } x_0 y_0)) = x_0 (\text{posn-y}(\text{make-posn } x_0 y_0)) = y_0$$

DrRacket は計算の際、これらの方程式を使用します。posn 構造体を含む計算の例を次に示します。

```
(posn-x p)
==; DrRacket replaces p with (make-posn 31 26)
(posn-x (make-posn 31 26))
==; DrRacket uses the law for posn-x
31
```

Stop! 上記の 2 番目の相互作用をご自身の計算で確認してください。また、DrRacket のステッパー (stepper) を使用してダブルチェックしてください。

5.3 posn を用いたプログラミング (Programming with posn)

次に、キャンバスの原点からある位置までの距離を計算する関数の設計について考えてみましょう。

[image: pict_66.png]

画像は、「距離」が指定された点からキャンバスの左上隅までの最も直接的な経路（「直線距離」）の長さであることを明確にしています。

目的記述 (purpose statement) とヘッダーを以下に示します。

```
; computes the distance of ap to the origin
(define (distance-to-0 ap)
  0)
```

鍵となるのは、distance-to-0 が単一の値、すなわち何らかの posn を消費することです。これは単一の値、すなわちその位置から原点までの距離を生成します。

例を作成するには、この距離の計算方法を知る必要があります。座標のいずれか一方が 0 である点の場合、結果はもう一方の座標になります。

```
(check-expect (distance-to-0 (make-posn 0 5)) 5)
(check-expect (distance-to-0 (make-posn 7 0)) 7)
```

一般の場合については、自力で公式を考え出すこともできますし、幾何学の授業で習った公式を思い出すこともできます。ご存知のように、これはあなたが持っているかもしれないドメイン知識ですが、持っていない場合に備えて提供します。結局のところ、このドメイン知識はコンピューターサイエンスではありません。それでは、(x,y) に対する距離の公式を改めて示します。

[image: pict_67.png]

この公式を踏まえると、さらにいくつかの機能的な例を簡単に作成できます。

```
(check-expect (distance-to-0 (make-posn 3 4)) 5)
(check-expect (distance-to-0 (make-posn 8 6)) 10)
(check-expect (distance-to-0 (make-posn 5 12)) 13)
```

念のためにお伝えすると、結果が簡単に計算できるように例を調整しています。すべての posn 構造体でこうなるわけではありません。

Stop! 例の x 座標と y 座標を公式に代入してください。5 つの例すべてについて期待される結果を確認してください。

次に、関数の定義に目を向けることができます。これらの例は、distance-to-0 の設計が異なる状況を区別する必要がないことを示唆しています。常に与えられた posn 構造体の内部にある x 座標と y 座標から距離を計算するだけです。しかし、関数は与えられた posn 構造体からこれらの座標を選択しなければなりません。そして、そのために posn-x と posn-y という基本操作を使用します。具体的には、ap は与えられた未知の posn 構造体の名前であるため、関数は (posn-xap) と (posn-yap) を計算する必要があります。

```
(define (distance-to-0 ap)
  (... (posn-x ap)...
  ... (posn-y ap)...))
```

このテンプレートと例を使用すれば、残りは簡単です。

```
(define (distance-to-0 ap)
  (sqrt
    (+ (sqr (posn-x ap))
       (sqr (posn-y ap))))))
```

この関数は、x 座標と y 座標を表す (posn-xap) と (posn-yap) を 2 乗し、その結果を足し合わせ、平方根をとります。DrRacket を使用すると、新しい関数が例に対して適切な結果を生成することを迅速に確認できます。

Exercise 63. 以下の式を:

- (distance-to-0(make-posn34))
- (distance-to-0(make-posn6(*24)))
- (+ (distance-to-0(make-posn125))10)

手計算で評価してください。すべてのステップを示してください。sqr はその計算を単一のステップで実行すると仮定します。DrRacket のステッパ (stepper) で結果を確認してください。

Exercise 64. ある点から原点へのマンハッタン距離 (Manhattan distance) は、マンハッタンに見られる格子状の道路網に沿った経路を考慮します。以下に 2 つの例を示します。

[image: pict_68.png] [image: pict_69.png]

左側の例は「直接 (direct)」戦略を示しており、必要な分だけ左に進み、続いて必要な分だけ上に進みます。これに対し、右側の例は「ランダムウォーク (random walk)」戦略を示しており、目的地（ここでは原点）に到達するまで、数ブロック左に進み、数ブロック上に進む、といった動作を繰り返します。

Stop! どちらの戦略に従うかは重要でしょうか？

与えられた posn から原点までのマンハッタン距離を計測する関数 manhattan-distance を設計してください。

5.4 構造体型の定義 (Defining Structure Types)

数値やブール値とは異なり、posn のような構造体は通常、プログラミング言語に最初から備わっているわけではありません。構造型を定義するためのメカニズムのみが提供されており、残りはプログラマーに委ねられています。これは BSL においても同様です。

構造型定義は、定数定義や関数定義とは異なる、定義のもう 1 つの形式です。DrRacket の制作者が BSL で posn 構造型をどのように定義したかを以下に示します：

```
(define-struct posn [x y])
```

一般に、構造型定義は次のような形式を持ちます：

Note: 構造型定義で角括弧（ブラケット）を使用するのは慣習であり、必須ではありません。角括弧を使うことでフィールド名が目立つようになります。角括弧を丸括弧（ペアレンセシス）に置き換えてもまったく問題ありません。

```
(define-struct StructureName [FieldName...])
```

キーワード define-struct は、新しい構造型の導入を示します。その後に構造体の名前が続きます。構造型定義の 3 番目の部分は、角括弧で囲まれた一連の名前であり、これらがフィールドです。

構造型定義は、実際には関数を定義します。しかし、通常関数定義とは異なり、構造型定義は同時に複数の関数を定義します。具体的には、以下の 3 種類の関数を定義します：

- 1 つのコンストラクター：構造体インスタンスを作成する関数です。フィールドの数と同じだけの値を受け取ります。前述したように、構造体は構造体インスタンスの略称です。「構造型」というフレーズは、考えられるすべてのインスタンスの集まりを指す総称です。
- フィールドごとに 1 つのセクター：構造体インスタンスからそのフィールドの値を抽出します。
- 1 つの構造体述語：通常の述語と同様に、そのインスタンスを他のすべての種類の値から区別します。

プログラムは、これらがあたかも通常関数や組み込みプリミティブであるかのように使用できます。

面白いことに、構造型定義は、新しく作成する多様な操作のために名前を自動的に生成します。コンストラクターの名前として、構造体名の前に “make-” という接頭辞を付けます。セクターの名前としては、構造体名の後に各フィールド名を接尾辞として付けます。

最後に、述語は構造体名に“?”を付けただけであり、音読するときは「ハ (huh)」と発音されます。

この命名規約は複雑で、混乱を招くように思えるかもしれません。しかし、少し練習すればコツを掴むことができます。これは、posn 構造体に付属している関数についても説明しています。make-posn はコンストラクターであり、posn-x と posn-y はセクターです。まだ posn? には出くわしていませんが、それが存在することが分かりました。次の章では、これらの述語の役割について詳しく説明します。

Exercise 65. 次の構造型定義を見てみましょう：

- (define-structmovie[titleproduceryear])
- (define-structperson[namehaireyesphone])
- (define-structpet[namenumber])
- (define-structCD[artisttitleprice])
- (define-structsweater[materialsizeproducer])

それぞれが導入する関数（コンストラクター、セクター、述語）の名前を書き出してください。

posn 構造体の話はひとまずこれくらいにしましょう。携帯電話の連絡先などを管理するために使用できるような構造型定義を見てみましょう：

```
(define-struct entry [name phone email])
```

この定義が導入する関数の名前は以下のとおりです：

- make-entry：3つの値を受け取り、entry のインスタンスを構築します。
- entry-name、entry-phone、entry-email：entry の1つのインスタンスを受け取り、3つのフィールド値のいずれか1つを抽出します。
- entry?：述語です。

それぞれの entry は3つの値を組み合わせるため、次の式：

```
(make-entry "Al Abe" "666-7771" "Lee@x.me")
```

は、name フィールドに“Al Abe”、phone フィールドに“666-7771”、email フィールドに“lee@x.me”を持つ entry 構造体を作成します。

Exercise 66. 練習問題 65 の構造型定義を再確認してください。どのフィールドにどんな種類の値が入るか妥当な推測を行ってください。その上で、構造型定義ごとに少なくとも1つのインスタンスを作成してください。

あらゆる構造型定義は、他のすべての構造体とは区別される、新しい種類の構造体を導入します。プログラマーがこのような表現力を求めるのは、構造体名によって意図を伝えたいからです。構造体が作成、選択、またはテストされるすべての場所で、プログラムのコードは読み手にその意図を明示的に思い出させます。もしコードの将来の読み手が存在しないのであれば、プログラマーは1つのフィールドを持つ構造体用に1つの構造体定義

を、2つのフィールドを持つ構造体用に別の定義を、3つのフィールドを持つ構造体用に3つ目の定義を…といった使い分けで済ませることもできるでしょう。

この文脈において、別のプログラミング問題を検討してみましょう：

サンプル問題 本章の冒頭で簡単に触れた「跳ねるボール」を扱うプログラムのための構造型定義を開発しなさい。ボールの位置は単一の数値、すなわち上端からのピクセル距離です。ボールの一定の速さは、クロックの1目盛りごとに移動するピクセル数です。ボールの速度は、速さに移動方向を加えたものです。

ボールは直線的な垂直線に沿って移動するため、その速度をデータ表現するには数値が完全に適しています：

- 正の数値はボールが下に移動することを意味します。
- 負の数値はボールが上に移動することを意味します。

この領域知識を利用して、構造型定義を定式化することができます：

```
(define-struct ball [location velocity])
```

両方のフィールドに数値が入るため、(make-ball 10 -3) は良いデータ例となります。これは、上端から 10 ピクセルの位置にあり、クロックの 1 目盛りあたり 3 ピクセルの速さで上に移動するボールを表しています。

原理的には、ball 構造体が posn 構造体と同じように、単に 2 つの数値を組み合わせているだけである点に注目してください。プログラムに (ball-velocity a-ball) という式が含まれている場合、そのプログラムがボールとその速度の表現を扱っていることが即座に伝わります。対照的に、もしプログラムが代わりに posn 構造体を使用していた場合、(posn-y a-ball) はコードの読み手に、その式が y 座標に関するものであるという誤解を与える可能性があります。

Exercise 67. 跳ねるボールを表現する別の方法を以下に示します：

```
(define SPEED 3)
(define-struct balld [location direction])
(make-balld 10 "up")
```

このコード断片を解釈し、balld の他のインスタンスを作成してください。

構造体は数値やブール値や文字列と同様に値であるため、ある構造体のインスタンスが別のインスタンスの内部に現れるのは理にかなっています。ゲームオブジェクトを考えてみましょう。跳ねるボールとは異なり、そのようなオブジェクトは常に垂直線に沿って移動するとは限りません。代わりに、キャンバス上を「斜め」に移動します。2次元のワールドキャンバス上を移動するボールの位置と速度の両方を記述するには、各方向に 1 つずつ、計 2 つの数値が必要になります。位置のパートについては、2 つの数値が x 座標と y 座標を表します。速度は水平方向と垂直方向の変化を記述します。言い換えれば、これらの「変

化の数値」をそれぞれの座標に加算することで、オブジェクトが次にどこに位置するのかを突き止めることができます。

Note: オブジェクトの次の位置を得るために位置に速度を加算するというのは、物理学の教えるところです。開発者は、どの領域について誰に尋ねるべきかを学ぶ必要があります。

明らかに、`posn` 構造体は位置を表すことができます。速度については、`vel` 構造型を定義します：

```
(define-struct vel [deltax deltay])
```

これには `deltax` と `deltay` という 2 つのフィールドがあります。「デルタ」という単語は、物理的活動のシミュレーションにおいて変化について語る際に一般的に使用され、`x` と `y` の部分はどの軸が関係しているかを示します。

これで、`ball` のインスタンスを使用して `posn` 構造体と `vel` 構造体を組み合わせることで、直線的に移動するものの、必ずしも垂直（または水平）線に沿ってのみ移動するとは限らないボールを表現できます：

```
(define ball1  
  (make-ball (make-posn 30 40) (make-vel -10 5)))
```

このインスタンスを解釈する 1 つの方法は、左端から 30 ピクセル、上端から 40 ピクセルの位置にあるボールを考えることです。このボールは、クロックの 1 目盛りごとに左に向かって 10 ピクセル移動します。なぜなら、`x` 座標から 10 ピクセルを引くと左端に近づくからです。垂直方向に関しては、ボールはクロックの 1 目盛りあたり 5 ピクセルの速さで落下します。なぜなら、`y` 座標に正の数を加えると上端からの距離が増加するからです。

Exercise 68. ボールのネストされたデータ表現に代わる方法として、4 つのプロパティを追跡するために 4 つのフィールドを使用するものがあります：

Note: さらに代替案として、複素数を使用する方法もあります。複素数について知っている方は、位置と速度の両方に複素数を使用するデータ表現について考えてみてください。例えば BSL では、 $4-3i$ は複素数であり、位置や速度 $(4, -3)$ を表現するために使用できます。

```
(define-struct ballf [x y deltax deltay])
```

プログラマーはこれをフラット表現と呼びます。`ball1` と同じ解釈を持つ `ballf` のインスタンスを作成してください。

ネストされた構造体の 2 つ目の例として、連絡先リストの例を簡単に見てみましょう。多くの携帯電話の連絡先リストは、1 つの名前に対して複数の電話番号（自宅用、職場用、携帯電話用など）を保持できるようになっています。電話番号については、市外局番と加

入者番号の両方を含めたいと考えています。これは情報をネストするため、ネストされたデータ表現を作成するのが最善です：

```
(define-struct centry [name home office cell])
```

```
(define-struct phone [area number])
```

```
(make-centry "Shriram Fisler"  
  (make-phone 207 "363-2421")  
  (make-phone 101 "776-1099")  
  (make-phone 208 "112-9981"))
```

ここでの意図は、連絡先リストの1つのエントリが4つのフィールド（1つの名前と3つの電話レコード）を持つことです。後者は市外局番と加入者番号を分離する phone のインスタンスで表現されます。

要約すると、情報をネストさせるのは自然なことです。このような情報をデータで表現する最善の方法は、ネストされた構造体インスタンスを用いてそのネスト構造を反映させることです。そうすることで、プログラムのアプリケーション領域においてデータを解釈することが容易になり、情報の例からデータへの移行も明快になります。もちろん、情報とデータの間をどのように行き来するかを規定するのは、本来データ定義の役割です。しかし、構造型定義に対するデータ定義を学ぶ前に、まず構造体を用いた計算や思考について体系的に見ていきましょう。どのように指定するかは、本来データ定義の役割です。しかし、構造型定義のためのデータ定義を学ぶ前に、まず構造体を用いた計算とそれに対する考え方について体系的に見ていきましょう。

5.5 構造体を使った計算 (Computing with Structures)

構造体型はデカルト座標 (Cartesian points) を2つの方法で一般化します。第一に、構造体型は任意の数のフィールドを指定できます。ゼロ、1つ、2つ、3つ、その他いくらかでも可能です。第二に、構造体型はフィールドに名前を付けるのであり、番号を振るわけではありません。これはプログラマーがコードを読むのに役立ちます。なぜなら、姓 (family name) が7番目のフィールドにあると思いつくよりも、last-name というフィールドにあると思いつく方がはるかに簡単だからです。

注: ほとんどのプログラミング言語は、数値のフィールド名を使用する構造体風のデータもサポートしています。

同じ趣旨で、構造体インスタンスを使った計算は、デカルト座標の操作を一般化したものです。この考え方をよく理解するために、まず構造体インスタンスをフィールドの数だけ区画 (compartment) を持つ鍵付きの箱 (lockbox) として考える図的な捉え方を見てみましょう。以下は、

```
(define pl (make-entry "Al Abe" "666-7771" "lee@x.me"))
```

のような図による表現です。

[image: pict_70.png]

箱の斜体ラベルは、それが特定の構造体型のインスタンスであることを識別します。各区画にもラベルが付けられています。以下は別のインスタンスです。

```
(make-entry "Tara Harp" "666-7770" "th@smlu.edu")
```

これは内容が異なりますが、同様の箱の図に対応します。

[image: pict_71.png]

驚くことではありませんが、入れ子になった構造体インスタンスは、箱の中に箱が入れ子になった図になります。したがって、前述の `ball1` は次の図と等価です。

[image: pict_72.png]

この場合、外側の箱には2つの箱が含まれており、各フィールドに1つずつ割り当てられています。

Exercise 69. 演習 65 の解答に対する箱表現を描きなさい。

このイメージの文脈では、セクタ (selector) は鍵のようなものです。特定の種類の箱の特定の区画を開け、それにより保持者はその中身を取り出すことができます。したがって、前述の `pl` に `entry-name` を適用すると文字列が得られます。

```
> (entry-name pl)
"Al Abe"
```

しかし、`posn` 構造体に `entry-name` を適用するとエラーが送出されます。

```
> (entry-name (make-posn 42 5))
entry-name:expects an entry, given (make-posn 42 5)
```

区画に箱が含まれている場合、目的の数値を得るために2つのセクタを続けて使用する必要があるかもしれません。

```
> (ball-velocity ball1)
(make-vel -10 5)
```

`ball1` に `ball-velocity` を適用すると、`vel` のインスタンスである `velocity` フィールドの値が取り出されます。x 軸に沿った速度を得るには、最初の選択結果に対してセクタを適用します。

```
> (vel-deltax (ball-velocity ball1))
-10
```

内側の式が `ball1` から速度を取り出すので、外側の式は `deltax` フィールドの値を取り出します。この場合は `-10` です。

この対話 (interaction) 結果は、構造体インスタンスが値であることを示しています。DrRacket は数値のような単純な値と同様に、入力されたとおりにそのまま印字します。

```
> (make-vel -10 5)
(make-vel -10 5)
> (make-entry "Tara Harp" "666-7770" "th@smlu.edu")
(make-entry "Tara Harp" "666-7770" "th@smlu.edu")
> (make-centry "Shriram Fisler" (make-phone 207 "363-2421") (make-phone 101 "776-1099"))
(make-phone 208 "112-9981"))
(make-centry...)
```

Stop! 自分でこの最後の対話を試してみて、正しい結果を確認してください。

一般的に言えば、構造体型の定義は新しい関数や値を生成する新しい方法を作成するだけでなく、DrRacket の知識に新しい計算規則 (laws of computation) を追加します。これらの規則は「posn による計算 (Computing with posns)」における posn 構造体の規則を一般化したものであり、例を通して理解するのが最も適切です。

DrRacket が 2 つのフィールドを持つ構造体型定義に遭遇すると、

```
(define-struct ball [location velocity])
```

セクタごとに 1 つずつ、2 つの規則を導入します。

$$(\text{ball-location}(\text{make-ball } l_0 v_0)) == l_0 (\text{ball-velocity}(\text{make-ball } l_0 v_0)) == v_0$$

異なる構造体型定義に対しては、同様の規則が導入されます。したがって、

```
(define-struct vel [deltax deltay])
```

DrRacket はこれら 2 つの規則を知識に追加します。

$$(\text{vel-deltax}(\text{make-vel } dx_0 dy_0)) == dx_0 (\text{vel-deltay}(\text{make-vel } dx_0 dy_0)) == dy_0$$

これらの規則を使用すると、前述の対話を説明できるようになります。

```
(vel-deltax (ball-velocity ball1))
==; DrRacket replaces ball1 with its value
(vel-deltax
 (ball-velocity
  (make-ball (make-posn 30 40) (make-vel -10 5))))
==; DrRacket uses the law for ball-velocity
(vel-deltax (make-vel -10 5))
==; DrRacket uses the law for vel-deltax
-10
```

Exercise 70. 以下の構造体型定義に対する計算規則を詳細に書き下しなさい：

```
(define-struct centry [name home office cell])
(define-struct phone [area number])
```

DrRacket のステッパーを使用して、次の式の値が 101 であることを確認しなさい：

```
(phone-area
  (centry-office
    (make-centry "Shriram Fisler"
      (make-phone 207 "363-2421")
      (make-phone 101 "776-1099")
      (make-phone 208 "112-9981")))))
```

述語 (predicate) は、構造体型定義を理解するために説明しなければならない最後の概念です。前述のように、すべての構造体型定義は 1 つの新しい述語を導入します。DrRacket はこれらの述語を使用して、セレクトが適切な種類の値に適用されているかどうかを検出します。次の章では、この考え方を詳しく説明します。ここでは、これらの述語が「算術 (arithmetic)」の述語とまったく同じであることをお伝えしたいだけです。number? が数値かを識別し、string? が文字列かを識別するように、posn? や entry? といった述語は posn 構造体や entry 構造体を識別します。これらがどのように機能するかについての考えは、対話領域での実験で確認できます。定義領域に以下の定義が含まれていると仮定します。

```
(define ap (make-posn 7 0))
```

```
(define pl (make-entry "Al Abe" "666-7771" "lee@x.me"))
```

もし posn? が posn を他のすべての値から区別する述語であるなら、数値に対しては #false を返し、ap に対しては #true を返すことが期待されます。

```
> (posn? ap)
#true
> (posn? 42)
#false
> (posn? #true)
#false
> (posn? (make-posn 3 4))
#true
```

同様に、entry? は entry 構造体を他のすべての値から区別します。

```
> (entry? pl)
#true
> (entry? 42)
#false
> (entry? #true)
#false
```

一般に、述語は同名のコンストラクタで構築された値だけを正確に識別します。「合間 1: 初級学生言語 (Intermezzo 1: Beginning Student Language)」ではこの規則を詳しく説明しており、BSL の計算規則も 1 箇所にまとめています。

Exercise 71. DrRacket の定義領域に以下を配置しなさい：


```

; distances in terms of pixels:
(define HEIGHT 200)
(define MIDDLE (quotient HEIGHT 2))
(define WIDTH 400)
(define CENTER (quotient WIDTH 2))

(define-struct game [left-player right-player ball])

(define game0
  (make-game MIDDLE MIDDLE (make-posn CENTER CENTER)))

```

RUN（実行）をクリックし、以下の式を評価しなさい：

```

(game-ball game0)
(posn? (game-ball game0))
(game-left-player game0)

```

ステップごとの計算で結果を説明しなさい。DrRacket のステッパーを使用して、自分の計算を再確認しなさい。

5.6 構造体を用いたプログラミング (Programming with Structures)

適切なプログラミングにはデータ定義が必要です。構造体型定義の導入によって、データ定義は興味深いものになります。データ定義とは、情報をデータへと表現する方法と、そのデータを情報として解釈する方法を提供するものであることを思い出してください。構造体型の場合、これはどのフィールドにどのような種類のデータが入るのかを記述することを意味します。いくつかの構造体型定義においては、そうした記述を策定することは簡単で明白です。

```

(define-struct posn [x y])
; A Posn is a structure:
;   (make-posn Number Number)
; interpretation a point x pixels from left, y from top

```

posn を作成するために他の種類のデータを使用することは何の意味もありません。同様に、連絡先リストのエントリのための構造体型定義である entry のすべてのフィールドも、前の節での使い方に従えば、明らかに文字列であることが想定されています。

```

(define-struct entry [name phone email])
; An Entry is a structure:
;   (make-entry String String String)
; interpretation a contact's name, phone#, and email

```

posn と entry のどちらについても、読者はアプリケーションドメインにおけるこれらの構造体のインスタンスを簡単に解釈できます。

この単純さを、明らかに少なくとも 2 通りの異なる解釈が可能な ball の構造体型定義と対比してみましょう。

```

(define-struct ball [location velocity])
; A Ball-1d is a structure:

```

```
; (make-ball Number Number)
; interpretation 1 distance to top and velocity
; interpretation 2 distance to left and velocity
```

プログラムでそのうちのどちらを使用するにしても、一貫してそれを使い続けなければなりません。しかし、「構造体型の定義 (Defining Structure Types)」で示されているように、ball 構造体をまったく異なる方法で使用することも可能です。

```
; A Ball-2d is a structure:
; (make-ball Posn Vel)
; interpretation a 2-dimensional position and velocity
```

```
(define-struct vel [deltax deltay])
; A Vel is a structure:
; (make-vel Number Number)
; interpretation (make-vel dx dy) means a velocity of
; dx pixels [per tick] along the horizontal and
; dy pixels [per tick] along the vertical direction
```

ここでは、ワールドキャンバス上を直線状に移動するボールのデータ表現を記述するために、Ball-1d とは区別される 2 つ目のデータコレクションを Ball-2d と命名しています。要するに、まったく同じ構造体型を 2 つの異なる方法で使うことが可能です。もちろん、1 つのプログラム内では、唯一の用途だけに絞るのが最善です。さもないと、自ら問題を招くことになります。

また、Ball-2d は私たちが定義した別のデータ定義、すなわち Vel の定義を参照しています。これまで他のすべてのデータ定義は組み込みのデータコレクション (Number、Boolean、String) を参照していましたが、自分のデータ定義の 1 つが別のデータ定義を参照することは完全に許容されますし、実際によくあることです。とです。

Exercise 72. 上記の phone 構造型定義に対し、与えられた例に適合するデータ定義を作成しなさい。

次に、以下の構造型定義を用いて電話番号のデータ定義を作成しなさい：

```
(define-struct phone# [area switch num])
```

歴史的には、最初の 3 桁が市外局番 (area code)、次の 3 桁があなたの地域の電話交換局 (phone switch (exchange)) のコード、最後の 4 桁はその地域内での個人の電話番号を表します。これら 3 つのフィールドの内容を、区間 (interval) を用いて可能な限り正確に記述しなさい。

この時点で、データ定義が実際に何を意味するのか疑問に思っているかもしれません。この疑問とその答えは次節のトピックです。今のところは、プログラム設計においてデータ定義をどのように使うかを示します。

文脈を設定するための問題文を以下に示します：

Sample Problem あなたのチームは、赤い点を [image: pict_73.png] キャンバス上で移動させ、プレイヤーがマウスを使ってその点をリセットできるようにするインタラクティブなゲームプログラムを設計しています。チームでここまでに進めた内容は以下の通りです：(define MTS (empty-scene 100 100))(define DOT (circle 3 "solid" "red"));APosn represents the state of the world.;Posn -> Posn(define (main p0) (big-bang p0 [on-tick x+] [on-mouse reset-dot] [to-draw scene+dot]))あなたのタスクは、指定された位置にある空のキャンバスに赤い点を追加する関数 scene+dot を設計することです。

問題の文脈によって、関数のシグネチャが決定されます：

```
; Posn -> Image
; adds a red spot to MTS at p
(define (scene+dot p) MTS)
```

目的記述 (purpose statement) の追加は簡単です。「関数の設計 (Designing Functions)」で述べるように、目的記述は関数のパラメータを使用して関数が何を計算するかを表現します。

ここで、いくつかの例を作成し、それらをテストとして定式化します：

```
(check-expect (scene+dot (make-posn 10 20))
               (place-image DOT 10 20 MTS))
(check-expect (scene+dot (make-posn 88 73))
               (place-image DOT 88 73 MTS))
```

この関数が Posn を受け取る (consume) ことから、関数は x フィールドと y フィールドの値を取り出すことができますとわかります：

```
(define (scene+dot p)
  (... (posn-x p) ... (posn-y p) ...))
```

関数の本体の中にこれらの追加の構成要素が揃えば、残りの定義は簡単です。place-image を使用して、この関数は p に含まれる座標の位置で MTS に DOT を配置します：

```
(define (scene+dot p)
  (place-image DOT (posn-x p) (posn-y p) MTS))
```

関数は構造体を生成する (produce) こともあります。まさにそのようなタスクが含まれているため、上記の例題を再開しましょう：

Sample Problem 同僚が Posn を受け取り、x 座標を 3 増加させる関数 x+ を定義するよう頼まれました。

x+ 関数がクロックティック (clock tick) を処理することを思い出してください。

scene+dot の設計の最初のいくつかのステップを適応させることができます：

```

; Posn -> Posn
; increases the x-coordinate of p by 3
(check-expect (x+ (make-posn 10 0)) (make-posn 13 0))
(define (x+ p)
  (... (posn-x p)... (posn-y p)...))

```

シグネチャ、目的、および例はすべて問題文から導き出されます。ヘッダー（デフォルトの結果を返す関数）の代わりに、私たちのスケッチには Posn に対する 2 つのセレクトタ式が含まれています。結局のところ、結果のための情報は入力から来なければならない、入力は 2 つの値を含む構造体だからです。

これで定義を完成させるのは簡単です。望まれる結果は Posn であるため、関数は make-posn を使用して各要素を組み合わせます：

```

(define (x+ p)
  (make-posn (+ (posn-x p) 3) (posn-y p)))

```

Exercise 73. Posn である p と Number である n を受け取る関数 posn-up-x を設計 کنید。この関数は、x フィールドが n に置き換わった p のような Posn を生成します。

興味深い観察として、posn-up-x を使って x+ を定義できることが挙げられます：

```

(define (x+ p)
  (posn-up-x p (+ (posn-x p) 3)))

```

Note: posn-up-x のような関数は、しばしばアップデータ (updater) や関数型セッター (functional setter) と呼ばれます。これらは大きなプログラムを書く際に非常に便利です。

関数は、アトミックなデータ (atomic data) からインスタンスを生成することもあります。make-posn はそれを行う組み込みのプリミティブですが、進行中の例題がもう一つの好例を提供しています：

Sample Problem 別の同僚が、マウスがクリックされたときに点をリセットする関数 reset-dot を設計するタスクを担当しています。

この問題に取り組むには、「ワールドプログラムの設計 (Designing World Programs)」から、マウスイベントハンドラが 4 つの値 (ワールドの現在の状態、マウスクリックの x 座標と y 座標、および MouseEvent) を受け取ることを思い出す必要があります。

例題からの知識をプログラム設計レシピに追加することで、シグネチャ、目的記述、およびヘッダーが得られます：

```

; Posn Number Number MouseEvent -> Posn
; for mouse clicks, (make-posn x y); otherwise p
(define (reset-dot p x y me) p)

```

マウスイベントハンドラの例には、Posn、2つの数値、および String の特別な種類に過ぎない MouseEvent が必要です。例えばマウスクリックは、“button-down”と“button-up”という2つの文字列のいずれかで表されます。前者はユーザーがマウスボタンをクリックしたことを示し、後者はボタンが離されたことを示します。これを念頭に置いて、以下に示す2つの例を熟考し解釈してみてください：

```
(check-expect
  (reset-dot (make-posn 10 20) 29 31 "button-down")
  (make-posn 29 31))
(check-expect
  (reset-dot (make-posn 10 20) 29 31 "button-up")
  (make-posn 10 20))
```

この関数はアトミックな形式のデータのみを受け取りますが、その目的記述と例は、2種類の MouseEvent (“button-down”とそれ以外のすべて) を区別することを示唆しています。このような場合の分岐は cond 式を示唆します：

```
(define (reset-dot p x y me)
  (cond
    [(mouse=? "button-down" me) (... p... x y...)]
    [else (... p... x y...)]))
```

設計レシピに従い、このスケルトンは利用可能なデータを思い出させるためにパラメータを記述しています。

目的記述自体が2つのケースのそれぞれで関数が何を計算するかを指示しているため、残りの部分は再び簡単です：

```
(define (reset-dot p x y me)
  (cond
    [(mouse=? me "button-down") (make-posn x y)]
    [else p]))
```

上記と同様に、make-posn が Posn のインスタンスを作成することに言及することもできましたが、皆さんはすでにこのことを知っており、常に思い起こさせる必要はないでしょう。

Exercise 74. 関連するすべての定数定義および関数定義を DrRacket の定義領域 (definitions area) にコピーしなさい。テストを追加して、それらがパスすることを確認しなさい。その後、プログラムを実行し、マウスを使って赤い点を配置しなさい。

多くのプログラムは入れ子になった構造体 (nested structures) を扱います。ワールドプログラムからのもう一つの短い抜粋を使ってこの点を示します：

Sample Problem あなたのチームは、変化する速度でキャンバス上を移動するオブジェクトを追跡するゲームプログラムを設計しています。選択されたデータ表現には2つのデータ定義が必要です： (define-struct ufo [loc vel]);AUFO is a structure;; (make-ufoPosnVel);interpretation(make-ufo pv) is at location;p moving

at velocity v 与えられた UFO が 1 クロックティック経過した後の位置を計算する関数 `ufo-move-1` を開発することがあなたのタスクです。

Note: 物理学に関することであることを思い出してください。

データ定義を少し探求するいくつかの例から始めましょう：

```
(define v1 (make-vel 8 -3))
(define v2 (make-vel -5 -3))
```

The order of these definitions matters. See Intermezzo 1: Beginning Student Language.

```
(define p1 (make-posn 22 80))
(define p2 (make-posn 30 77))
```

```
(define u1 (make-ufo p1 v1))
(define u2 (make-ufo p1 v2))
(define u3 (make-ufo p2 v1))
(define u4 (make-ufo p2 v2))
```

Note: これらの定義の順序は重要です。「合間 1: 初等学生言語 (Intermezzo 1: Beginning Student Language)」を参照してください。

最初の 4 つは Vel と Posn の要素です。最後の 4 つは、最初の 4 つを考えられるあらゆる組み合わせで組み合わせたものです。

次に、シグネチャ、目的、いくつかの例、および関数ヘッダーを書き下します：

```
; UFO -> UFO
; determines where u moves in one clock tick;
; leaves the velocity as is
```

```
(check-expect (ufo-move-1 u1) u3)
(check-expect (ufo-move-1 u2)
              (make-ufo (make-posn 17 77) v2))
```

```
(define (ufo-move-1 u) u)
```

関数の例については、データの例と、位置と速度に関するドメイン知識を使用します。具体的には、時速 60 マイルで北へ、時速 10 マイルで西へ移動している乗り物は、1 時間の走行後にスタート地点から北へ 60 マイル、西へ 10 マイル離れた場所に到達することがわかっています。2 時間後には、スタート地点から北へ 120 マイル、西へ 20 マイルの場所になります。

いつものように、構造体のインスタンスを受け取る関数は、その結果を計算するために構造体から情報を抽出することができます（そしておそらくそうしなければなりません）。したがって、再び関数の定義にセレクト式を追加します：

```
(define (ufo-move-1 u)
  (... (ufo-loc u)... (ufo-vel u)...))
```

Note: セレクタ式は、このスケッチをさらに洗練させる必要があるかという疑問を生じさせます。結局のところ、2つの式はそれぞれ Posn と Vel のインスタンスを抽出します。これら2つも構造体インスタンスであり、今度はそれらから順番に値を抽出することもできます。その結果得られるスケルトンは以下のようになります：

```
; UFO -> UFO
(define (ufo-move-1 u)
  (... (posn-x (ufo-loc u))...
    ... (posn-y (ufo-loc u))...
    ... (vel-deltax (ufo-vel u))...
    ... (vel-deltay (ufo-vel u))...))
```

しかし、そうすることでスケッチは明らかにかなり複雑に見えるようになります。真に現実的なプログラムにおいて、この考え方を論理的な極限まで進めると、信じられないほど複雑なプログラムのアウトラインが作成されてしまいます。より一般的には、

関数が入れ子になった構造体を扱う場合は、入れ子のレベルごとに1つの関数を開発しなさい。

本書の第2部では、このガイドラインはさらに重要になり、少し洗練させていきます。(終わり)

ここでは、与えられた Posn と与えられた Vel を組み合わせて UFO の次の位置を取得する方法に焦点を当てます。なぜなら、それが私たちの物理学の知識が教えてくれることだからです。具体的には、この2つを「加算する (add)」ことを意味しますが、ここでの「加算」は私たちが通常数値に適用する操作を意味することはできません。そこで、Vel を Posn に加算するための関数が存在すると仮定してみましょう：

```
; Posn Vel -> Posn
; adds v to p
(define (posn+ p v) p)
```

このようにシグネチャ、目的、およびヘッダーを書き下すことは、プログラミングの正当な方法です。これは「願い事をする (making a wish)」と呼ばれ、「関数からプログラムへ (From Functions to Programs)」で説明されている「ウィッシュリスト (願い事リスト) を作る (making a wish list)」ことの一部です。

重要なのは、現在取り組んでいる関数を完成させることができるように願い事をすることです。このようにして、難解なプログラミングタスクを異なるタスクに分割することができ、これは合理的に小さなステップで問題を解決するのに役立つテクニックです。この例題では、ufo-move-1 の完全な定義が得られます：

```
(define (ufo-move-1 u)
  (make-ufo (posn+ (ufo-loc u) (ufo-vel u))
            (ufo-vel u)))
```

ufo-move-1 と posn+ は完全な定義であるため、RUN をクリックすることさえでき、これによって DrRacket がこれまでの作業における文法上の問題について文句を言わないことを確認できます。当然ながら、posn+ は単なる願い事であり、必要な関数そのものではないため、テストは失敗します。

今こそ posn+ に集中するときです。設計の最初の 2 つのステップ（データ定義、シグネチャ/目的/ヘッダー）を完了したため、例を作成しなければなりません。「願い事」に対する機能的な例を作成する簡単な方法の 1 つは、元の関数の例を使用して、それらを新しい関数の例に変換することです：

Note: 幾何学では、posn+ に対応する操作は平行移動 (translation) と呼ばれます。

```
(check-expect (posn+ p1 v1) p2)
(check-expect (posn+ p1 v2) (make-posn 17 77))
```

この問題において、(ufo-move-1(make-ufo p1 v1)) が p2 を生成することを知っています。同時に、ufo-move-1 が posn+ を p1 と v1 に適用することを知っており、これは posn+ がこれらの入力に対して p2 を生成しなければならないことを意味します。手を止めてください！(Stop!) 私たちが何をしているかを理解できているか、手計算を確認してください。

これで、設計スケッチにセレクト式を追加できるようになりました：

```
(define (posn+ p v)
  (... (posn-x p)... (posn-y p)...
  ... (vel-deltax v)... (vel-deltay v)...))
```

posn+ は Posn と Vel のインスタンスを受け取り、各データは 2 つのフィールドを持つ構造体のインスタンスであるため、4 つの式が得られます。上記の入れ子になったセレクト式とは対照的に、これらはパラメータに対するセレクトの単純な適用です。

これら 4 つの式が何を表しているのかを思い出すか、または 2 つの構造体から望ましい結果をどのように計算したかを思い出せば、posn+ の定義の完成は簡単です：

```
(define (posn+ p v)
  (make-posn (+ (posn-x p) (vel-deltax v))
             (+ (posn-y p) (vel-deltay v))))
```

最初のステップは、水平方向の速度を x 座標に加算し、垂直方向の速度を y 座標に加算することです。これにより、新しい座標ごとに 1 つずつ、計 2 つの式が得られます。make-posn を使用することで、これらを再び単一の Posn に結合できます。

Exercise 75. これらの定義とそのテストケースを DrRacket の定義領域に入力し、正常に動作することを確認しなさい。これは皆さんが「願い事 (wish)」を扱った初めての機会であり、2 つの関数がどのように連携して動作するかを確実に理解する必要があります。

5.7 データの宇宙 (The Universe of Data)

すべての言語にはデータの宇宙が存在します。このデータは外部世界からの、そして外部世界についての情報を表現しており、プログラムが操作する対象です。このデータの宇宙とは、組み込みのデータをすべて含むだけでなく、あらゆるプログラムが作成し得るあらゆるデータをも含むコレクション（集まり）です。

注: 数学者はデータのコレクションやデータクラスのことを集合 (sets) と呼ぶことを思い出してください。

figure 30 の左側は、BSL の宇宙を想像するための一つの方法を示しています。数値や文字列は無限に存在するため、すべてのデータのコレクションは無限です。この図では「無限」を「…」で表していますが、実際の定義ではこのような曖昧さを回避しなければなりません。

プログラムもプログラム内の個々の関数も、データの宇宙全体を扱うわけではありません。データの定義の目的は、この宇宙の一部を説明し、それらに名前を付けることで、簡潔に参照できるようにすることです。言い換えれば、名前付きのデータ定義とはデータのコレクションの記述であり、その名前は他のデータ定義や関数のシグネチャで使用できます。関数のシグネチャにおいて、その名前は関数がどのデータを扱うかを規定し、暗黙のうちにデータの宇宙のどの部分を扱わないかも規定します。

```
+-----+
| Figure 30: The universe of data |
|                                 |
| [image: pict_74.png]           |
| [image: pict_75.png]           |
| [image: pict_74.png] [image: pict_75.png] |
+-----+
```

実際問題として、最初の 4 章におけるデータ定義は、組み込みのデータのコレクションを制限するものでした。それらは、含まれるすべての値を明示的または暗黙的に箇条書き (itemization) にすることによって制限を行います。例えば、figure 30 の右側に灰色で陰影が付けられた領域は、以下のデータ定義を描写しています。

```
; A BS is one of:
; — "hello",
; — "world", or
; — pi.
```


この特定のデータ定義は一見おかしなものに見えるかもしれませんが、英語と BSL が様式化されて混ざり合っていることに注目してください。その意味は正確かつ一義的であり、どの要素が BS に属し、どの要素が属さないかを正確に明確化しています。

構造体型 (structure type) の定義は、この構図を完全に一新しました。プログラマーが構造体型を定義すると、宇宙は可能なすべての構造体インスタンスとともに拡張されます。例えば、posn の追加は、2 つのフィールドに可能なあらゆる値を持つ posn のインスタンスが登場することを意味します。figure 31 の中央のバブルは、(make-posn "hello" 0) や (make-posn (make-posn 0 1) 2) のような一見無意味に見えるものを含め、これらの値が追加された様子を描写しています。そして確かに、これらの posn のインスタンスのいくつかは私たちが理解できる意味を持ちません。しかし、BSL プログラムはそれらのどれでも構築することができるのです。

```
+-----+
| Figure 31: Adding structure to a universe |
|                                           |
| [image: pict_76.png]                   |
| [image: pict_76.png]                   |
+-----+
```

さらにもう一つの構造体型定義を追加すると、すべてが再び組み合わせられます。例えば、同じく 2 つのフィールドを持つ ball の定義を追加したとしましょう。figure 31 の 3 つ目のバブルが示すように、この追加により、数値や posn 構造体などを含む ball のインスタンスが生み出されるだけでなく、ball のインスタンスを含む posn のインスタンスも生み出されます。DrRacket で試してみてください！

```
(define-struct ball [location velocity])
```

を定義エリア (definitions area) に追加し、RUN を押して、いくつかの構造体インスタンスを作成してみましょう。

データ定義の運用面に関して言えば、構造体型のデータ定義は、既存のデータ定義とインスタンスの組み合わせを通じて、大きなデータのコレクションを記述します。私たちが

```
; Posn is (make-posn Number Number)
```

と書くとき、posn の可能なインスタンスの無限の数を記述していることになります。上記と同様に、データ定義は自然言語、他で定義されたデータのコレクション、およびデータコンストラクタの組み合わせを使用します。現時点では、データ定義にそれ以外のものが現れるべきではありません。

構造体のデータ定義は、私たちの関数によって使用されるインスタンスで構成された、新しいデータのコレクションを指定します。例えば、Posns のデータ定義は、figure 31 の宇宙の中央のバブルにおいて灰色で陰影が付けられた領域を特定します。ここには、2 つのフィールドに数値を含むすべての posn 構造体が含まれます。同時に、両方のフィールドに数値が含まれるという要件を満たさない posn のインスタンスを構築することも完全に可能です。


```
(make-posn (make-posn 1 1) "hello")
```

この構造体は、x フィールドに posn を、y フィールドに文字列を含んでいます。

Exercise 76. 以下の構造体型定義に対するデータ定義を作成しなさい。

- (define-struct movie [title producer year])
- (define-struct person [name hair eyes phone])
- (define-struct pet [name number])
- (define-struct CD [artist title price])
- (define-struct sweater [material size producer])

各フィールドにどのような種類の値が入るかについて、妥当な仮定を立てなさい。

Exercise 77. 深夜 (午前 0 時) からの経過時刻を表す構造体型定義およびデータ定義を提供しなさい。時刻は、時間 (hours)、分 (minutes)、秒 (seconds) の 3 つの数値で構成されます。

Exercise 78. 3 文字の単語を表す構造体型およびデータ定義を提供しなさい。単語は、“a” から “z” までの 1Strings および #false で表される小文字で構成されます。注: この問題はハングマン (hangman) ゲームの設計の一部です。exercise 396 を参照してください。

プログラマーはデータ定義を書くだけでなく、プログラムを理解するため、扱うことのできるデータの種類の拡張するため、エラーを排除するためなどにデータ定義を読みます。私たちは、指定されたコレクションに属するデータをどのように作成するかを理解し、あるデータが指定されたクラスに属するかどうかを判断するためにデータ定義を読みます。

データ定義は設計プロセスにおいて非常に中心かつ重要な役割を果たすため、関数の挙動を例で説明するのと同様に、データ定義も例を用いて説明するのが最善であることがよくあります。そして実際、データ定義からデータの例を作成することは簡単です。

- 組み込みのデータコレクション (数値、文字列、ブール値、画像) については、お気に入りの例を選びます。注: 時として、NegativeNumber や OneLetterString のように、組み込みデータコレクションを修飾するために説明的な名前が使われることがあります。これらは適切に書かれたデータ定義の代わりにはなりません。注終わり
- 列挙型 (enumeration) については、列挙型の項目をいくつか使用します。
- 区間 (interval) については、端点 (含まれている場合) および少なくとも一つの内部の点を使用します。
- 箇条書き (itemization) については、各部分を個別に扱います。そして
- 構造体のデータ定義については、自然言語による記述に従います。つまり、コンストラクタを使用し、各フィールドに指定されたデータコレクションから例を選びます。

本書の大部分においてデータ定義から例を構築するために必要なことはこれですべてですが、データ定義はこれまで見てきたものよりもずっと複雑になっていきます。

Exercise 79. 以下のデータ定義の例を作成しなさい。

- ;AColor is one of;;—"white";—"yellow";—"orange";—"green";—"red";—"blue";—"black" 注: DrRacket はさらに多くの文字列を色として認識します。注終わり
- ;H is aNumber between0 and100.;interpretation represents a happiness value
- (define-struct person [fstname lstname male?]);APerson is a structure;; (make-personStringStringBoolean) 述語 (predicate) の名前のように見えるフィールド名を使用するのは良い考えでしょうか？
- (define-struct dog [owner name age happiness]);ADog is a structure;; (make-dogPersonStringPositiveIntegerH) このデータ定義にも解釈 (interpretation) を追加しなさい。
- ;AWeapon is one of;;—#false;—Posn;interpretation#false means the missile hasn't;been fired yet; aPosn means it is in flight

最後の定義は変則的な箇条書き (itemization) であり、組み込みのデータと構造体型を組み合わせています。次の章では、このような定義について詳しく扱います。

5.8 構造体を用いた設計 (Designing with Structures)

構造体型の導入は、設計レシピ (design recipe) の6つのステップすべてを行う必要性を補強します。情報を表現するのに組み込みのデータコレクションに頼るだけではもはや不十分であり、プログラマーは最も単純な問題を除くすべての問題についてデータ定義を作成しなければならないことが明確になります。

本節では、設計レシピを追加し、それを以下の問題を用いて説明します。

問題例 3 次元空間におけるオブジェクトの原点からの距離を計算する関数を設計しなさい。

それでは始めましょう。

1. 問題が、互いに関連し合う情報や自然なまとまり (全体) を表現することを求めている場合、構造体型の定義が必要です。それには、関連する属性の数だけのフィールドが必要となります。この構造体型のインスタンスは全体に対応し、フィールド内の値はその属性に対応します。構造体型のデータ定義は、正当なインスタンスのコレクションに対する名前を導入します。さらに、どの種類のデータがどのフィールドに対応するかを記述しなければなりません。組み込みデータコレクションの名前、または以前に定義されたデータ定義の名前のみを使用してください。最終的に、私たちが (そして他の人も) そのデータ定義を使用してサンプルとなる構造体インスタンスを作成できなければなりません。そうでなければ、私たちのデータ定義のどこかが間違っています。インスタンスを作成できることを確実にするため、データ定義にはデータの例を添付する必要があります。この考え方を問題例に適用する方法は以下のとおりです。 (define-struct r3 [x y z]);AnR3 is a structure;; (make-r3NumberNumberNumber) (define ex1 (make-r3 1 2 13))(define ex2 (make-r3 -1 0 3))構造体型定義は新しい種類の構造体 r3

を導入し、データ定義は数値のみを含む r3 のすべてのインスタンスに対する名前として R3 を導入します。

2. シグネチャ、目的文、関数のヘッダーが引き続き必要ですが、これらは従来と同じです。手を止めて、問題例に対してこれを行ってください。
3. 最初のステップの例を使用して、関数の例を作成します。区間や列挙型に関連付けられた各フィールドについて、端点や中間点を選んで関数の例を作成するようにしてください。あなたが問題例への取り組みを続けることを期待します。
4. 構造体を消費する関数は、通常（常にではありませんが）、構造体内の様々なフィールドから値を抽出します。この可能性を自分自身に思い起こさせるために、そのような関数のテンプレートに各フィールドのセクタを追加します。問題例に対して作成したテンプレートは以下のとおりです。
;R3->Number;determines the distance of p to the origin
(define (r3-distance-to-0 p) (... (r3-x p) ... (r3-y p) ... (r3-z p) ...))
各セクタ式の隣に、それが与えられた構造体からどのような種類のデータを抽出するかを書き留めておくといいでしょう。この情報はデータ定義の中で見つけることができます。手を止めて、今すぐやってみてください！
5. 関数を定義する際には、テンプレートのセクタ式を使用します。それらのいくつかは必要ないかもしれないことに注意してください。
6. テスト。関数のヘッダーが書かれたらすぐにテストします。すべての式がカバーされるまでテストします。変更を加えたときには再びテストします。

問題例を完成させなさい。3次元の点の原点からの距離が思い出せない場合は、幾何学の本で調べてみてください。

注: そこには のような公式が見つかるでしょう。

Exercise 80. 以下の構造体型のインスタンスを消費する関数用のテンプレートを作成しなさい。

- (define-struct movie [title directory year])
- (define-struct pet [name number])
- (define-struct CD [artist title price])
- (define-struct sweater [material size color])

いいえ、このタスクにデータ定義は必要ありません。

Exercise 81. time 構造体のインスタンス（exercise 77 を参照）を消費し、深夜からの経過秒数を生成する関数 time->seconds を設計しなさい。例えば、12 時間 30 分 2 秒をこれらの構造体の一つで表現し、そのインスタンスに time->seconds を適用した場合、正しい結果は 45002 となります。

Exercise 82. 関数 compare-word を設計しなさい。この関数は 2 つの 3 文字の単語（exercise 78 を参照）を消費します。そして、与えられた単語がどこで一致し、どこで一致しないかを示す単語を生成します。2 つの単語が一致する場合、関数は構造体フィール

ドの内容を維持し、そうでない場合は結果の単語のフィールドに #false を配置します。ヒント: この問題では「単語の比較」と「文字の比較」という 2 つのタスクに言及しています。容を保持します。そうでない場合、結果となる単語のフィールドに #false を配置します。ヒント: この演習問題では、単語の比較と「文字」の比較という 2 つのタスクについて言及されています。

5.9 ワールド内の構造 (Structure in the World)

ワールドプログラムが独立した 2 つの情報を追跡しなければならない場合、ワールドの状態データを表現するために構造体のコレクションを使用する必要があります。一方のフィールドが 1 つの情報を追跡し、もう一方のフィールドが 2 つ目の情報を追跡します。当然のことながら、対象領域 (ドメイン) のワールドに 2 つより多くの独立した情報が含まれている場合、構造体型の定義には異なる情報の数と同じだけのフィールドを指定しなければなりません。

UFO と戦車 (tank) で構成されるインベーダーゲームを考えてみましょう。UFO は真っ直ぐな垂直線に沿って降下し、戦車はシーンの底部で水平に移動します。両方のオブジェクトが既知の一定速度で移動する場合、これら 2 つのオブジェクトを記述するために必要なのは、オブジェクトごとに 1 つの情報だけです。すなわち、UFO の y 座標と戦車の x 座標です。これらをまとめるには、2 つのフィールドを持つ構造体が必要です。

```
(define-struct space-game [ufo tank])
```

解釈を含め、この構造体型定義に対する適切なデータ定義を定式化することは皆さんにお任せします。構造体の名前にあるハイフンについて考えてみてください。BSL では、変数、関数、構造体、およびフィールドの名前にあらゆる種類の文字を使用することが実際に許可されています。この構造体のセクタ名は何でしょうか？ 述語の名前は何でしょうか？

「1 つの情報」と言うとき、それが必ずしも単一の数値や単一の単語を意味するとは限りません。1 つの情報自体が、複数の情報を組み合わせたものである場合もあります。そのような種類の情報のデータ表現を作成すると、自然と入れ子構造 (ネストされた構造体) に至ります。

架空のインベーダーゲームに少しのスパイスを加えてみましょう。垂直線に沿って降下するだけの UFO は退屈です。このアイデアを、戦車が何らかの武器で UFO を攻撃する面白いゲームに変えるには、UFO が単純ではない線に沿って降下できるようにする必要があります。おそらくランダムにジャンプするかもしれません。このアイデアの実装は、UFO の位置を記述するために 2 つの座標が必要であることを意味し、その結果、スペースゲームのための改訂されたデータ定義は以下のようになります。

```
; A SpaceGame is a structure:  
;   (make-space-game Posn Number).  
; interpretation (make-space-game (make-posn ux uy) tx)  
; describes a configuration where the UFO is  
; at (ux,uy) and the tank's x-coordinate is tx
```

ワールドプログラムにどのようなデータ表現が必要であるかを理解するには練習が必要です。続く2つの節では、ある程度複雑な問題文をいくつか紹介します。ご自身で設計したくなるような種類のゲームに進む前に、これらを解決してください。

5.10 グラフィカルエディタ (A Graphical Editor)

BSL でプログラミングをするには、DrRacket を開き、キーボードを打ち、テキストが表示されるのを見ます。キーボードの左矢印キーを押すとカーソルが左に移動し、バックスペース（または delete）キーを押すと、カーソルの左側にある1文字が消去されます（そもそも文字が存在する場合）。

このプロセスは「編集 (editing)」と呼ばれますが、キーボードで入力するよりも要求の厳しいタスクのために「編集」という言葉を使用するため、その正確な名称は「プログラムのテキスト編集」とするべきでしょう。英語の課題など、他の種類のドキュメントを執筆・改訂するときは、ワードプロセッサと呼ばれる他のソフトウェアアプリケーションを使用することが多いでしょうが、コンピュータ科学者はそれらすべてをエディタ、あるいはグラフィカルエディタと呼びます。

皆さんは今や、プレーンテキスト用の1行エディタとして機能するワールドプログラムを設計できる立場にあります。ここでの編集には、文字の入力や、文字の削除・挿入を含む既存テキストの何らかの変更が含まれます。これは、テキスト内における位置という概念を意味します。人々はこの位置をカーソル (cursor) と呼び、ほとんどのグラフィカルエディタは簡単に目につくようにそれを表示します。

以下のエディタ構成を見てみてください。

[image: pict_78.png]

誰かが「helloworld」というテキストを入力し、左矢印キーを5回押したとします。これにより、カーソルはテキストの末尾から「o」と「w」の間の位置に移動します。ここでスペースキーを押すと、エディタは表示を次のように変更します。

[image: pict_79.png]

要約すると、この操作は「」を挿入し、カーソルをそれと「w」の間に配置します。

以上のことから、エディタは2つの情報を追跡しなければなりません。

1. これまでに入力されたテキスト
2. カーソルの現在の位置

そして、これは2つのフィールドを持つ構造体型を示唆しています。

情報からデータへ、そしてその逆へ変換するさまざまな方法を想像することができます。たとえば、構造体の一方のフィールドに入力されたテキスト全体が含まれ、他方のフィールドに最初の文字（左から数えて）とカーソルの間の文字数が含まれるかもしれません。

もう1つのデータ表現は、2つのフィールドで2つの文字列を使用することです。すなわち、カーソルの左側のテキスト部分と、その右側のテキスト部分です。以下は、エディタの状態を表現するための私たちの推奨するアプローチです。

```
(define-struct editor [pre post])  
; An Editor is a structure:  
; (make-editor String String)  
; interpretation (make-editor s t) describes an editor  
; whose visible text is (string-append s t) with  
; the cursor displayed between s and t
```

このデータ表現に基づいて、続くいくつかの練習問題を解いてください。

Exercise 83. 関数 `render` を設計してください。この関数は `Editor` を受け取り、イメージを生成します。

この関数の目的は、`[image: pict_80.png]` ピクセルの空のシーン (empty scene) 内にテキストを描画することです。カーソルには `[image: pict_81.png]` の赤い長方形を使用し、文字列にはサイズ 16 の黒いテキストを使用してください。

DrRacket の交互作用エリア (interactions area) でサンプル文字列のイメージを作成してください。私たちは次の式から始めました。

```
(overlay/align "left" "center"  
  (text "hello world" 11 "black")  
  (empty-scene 200 20))
```

`beside` や `above` などの関数について調べてみるとよいでしょう。イメージの見た目に満足したら、その式をテストとして、また `render` の設計のガイドとして使用してください。

Exercise 84. `edit` を設計してください。この関数は2つの入力、エディタ `ed` と `KeyEvent` `ke` を受け取り、別のエディタを生成します。そのタスクは、`ke` がバックスペース ("`\b`") キーを表していない限り、1文字の `KeyEvent` `ke` を `ed` の `pre` フィールドの末尾に追加することです。バックスペースキーを表している場合、カーソルの直前 (左側) にある文字を消去します (文字が存在する場合)。この関数はタブキー ("`\t`") とリターンキー ("`\r`") を無視します。

この関数は、1文字より長い `KeyEvents` のうち、"`left`" と "`right`" の2つだけに注意を払います。左矢印はカーソルを左に1文字移動させ (文字が存在する場合)、右矢印は右に1文字移動させます (文字が存在する場合)。これら以外のそのような `KeyEvents` はすべて無視されます。

特殊なケースに注意を払いながら、`edit` の例をかなりの数作成してください。私たちがこの練習問題を解いたときは、20個の例を作成し、それらすべてをテストに変換しました。

ヒント: この関数を、列挙型 (enumeration) として指定されたコレクションである `KeyEvents` を受け取るものとして考えてください。Editor 構造体を扱うために補助関数を使用します。ウィッシュリストを手元に用意しておきましょう。string-first、string-

rest、string-last、string-remove-last など、これらの補助関数の大部分に対して追加の関数を設計する必要があります。まだ行っていない場合は、『Functions (関数)』の練習問題を解いてください。

Exercise 85. 関数 run を定義してください。エディタの pre フィールドが与えられると、前述の 2 つの練習問題の render と edit をそれぞれ to-draw 句と on-key 句に使用して、対話型エディタを起動します。

Exercise 86. たくさん入力すると、エディタプログラムがテキストのすべてを表示しないことに注意してください。代わりに、テキストは右マージンで切り取られます。Exercise 84 の関数 edit を修正し、pre フィールドの末尾にキー入力を追加すると描画されたテキストがキャンバスに対して幅広になりすぎる場合に、そのキー入力を無視するようにしてください。

Exercise 87. 文字列とインデックスを使用する私たちの最初のアイデアに基づいて、エディタのデータ表現を開発してください。その後、前の練習問題をもう一度解いてください。設計レシピ (design recipe) をたどり直してください。ヒント: まだ行っていない場合は、『Functions (関数)』の練習問題を解いてください。

設計上の選択に関する注記 (Note on Design Choices) この練習問題は、設計上の選択を行うための最初の調査です。ごく最初の設計上の選択がデータ表現に関するものであることを示しています。正しい選択をするには、事前計画を立て、それぞれの複雑さを秤にかけする必要があります。もちろん、これを得意になることは経験を積むことの問題です。

5.11 さらにバーチャルペット (More Virtual Pets)

この節では、『Virtual Pet Worlds (バーチャルペットワールド)』からのバーチャル動物園プロジェクトを継続します。具体的には、この練習問題の目標は、猫のワールドプログラムと、その幸福度ゲージ (happiness gauge) を管理するプログラムを組み合わせることです。組み合わせられたプログラムが実行されると、猫がキャンバスを横切って歩くのが見られ、一歩進むごとにその幸福度が下がります。猫を幸せにする唯一の方法は、餌を与えるか、撫でる (上矢印キー) ことです。最後に、この節の最後の練習問題の目標は、別の幸せなバーチャルペットを作成することです。

Exercise 88. 猫の x 座標とその幸福度を追跡する構造体型を定義してください。次に、解釈を含めて vcat と呼ばれる猫のデータ定義を定式化してください。

Exercise 89. 歩く猫とその幸福度レベルを管理する happy-cat ワールドプログラムを設計してください。猫は完璧な幸福度から始めると仮定しましょう。

ヒント: (1) 『Virtual Pet Worlds (バーチャルペットワールド)』のワールドプログラムの関数を再利用してください。(2) ワールドの状態を表現するために前の練習問題の構造体型を使用してください。

Exercise 90. 猫の幸福度が 0 に落ちたときはいつでも停止するように、前の練習問題の happy-cat プログラムを修正してください。

Exercise 91. Exercise 88 の構造体型定義とデータ定義を拡張して、`direction` フィールドを含めてください。指定された方向に猫が移動するように `happy-cat` プログラムを調整してください。プログラムは猫を現在の方向に移動させ、シーンのどちらかの端に達したときに猫を反転させる必要があります。

```
(define cham [image:chameleon.png])
```

上記のカメレオンの図は透明なイメージです。これを DrRacket に挿入するには、「Insert Image (イメージの挿入)」メニュー項目を使用して挿入します。この手順を使用すると、図のピクセルの透明度が保持されます。

部分的に透明なイメージが、たとえば長方形のような色のついた図形と組み合わせられると、イメージは下にある色を帯びます。カメレオンの図では、実際に透明なのは動物の内側であり、外側の領域は純白です。お使いの DrRacket で次の式を試してみてください。

```
(define background
  (rectangle (image-width cham)
             (image-height cham)
             "solid"
             "red"))
```

```
(overlay cham background)
```

Exercise 92. カメレオンがキャンバスを左から右へと連続的に歩く `cham` プログラムを設計してください。キャンバスの右端に達すると、消えてすぐに左側に再出現します。猫と同様に、カメレオンは歩き回ることによって腹が減り、時間が経過するにつれて、この空腹は不幸（不機嫌）として現れます。

カメレオンの幸福度ゲージを管理するには、バーチャル猫の幸福度ゲージを再利用できます。カメレオンを幸せにするには、餌を与えます（下矢印キー、2 ポイントのみ）。撫でることは許可されていません。もちろん、すべてのカメレオンと同様に、私たちのカメレオンも色を変えることができます。`"r"` は赤色に、`"b"` は青色に、`"g"` は緑色に変えます。カメレオンのワールドプログラムをバーチャル猫ゲームに追加し、可能であれば後者（バーチャル猫ゲーム）の関数を再利用してください。

カメレオンを表現するためのデータ定義 `VCham` から始めてください。

Exercise 93. Exercise 92 の解法をコピーし、カメレオンが3色の背景を横切って歩くようにそのコピーを修正してください。私たちの解法では以下の色を使用しています。

```
(define BACKGROUND
  (beside (empty-scene WIDTH HEIGHT "green") Have some Italian pizza when you're done.
          (empty-scene WIDTH HEIGHT "white")
          (empty-scene WIDTH HEIGHT "red")))
```

Note: 終わったらイタリアンピザでも食べてみてください。

ただし、どんな色を使用してもかまいません。カメレオンが2つの色の境界を越えるときに、どのように色を変えて背景に溶け込むかを観察してください。

注記 (Note) アニメーションを注意深く観察すると、カメレオンが白い長方形の上に乗っているのが見えます。画像編集ソフトウェアの使用方法を知っている場合は、白い長方形が見えなくなるように画像を修正してください。そうすれば、カメレオンは本当に溶け込むようになります。

6 項目化と構造 (Itemizations and Structures)

前章までの2つの章では、データ定義を定式化するための2つの方法を紹介しました。項目化（列挙と区間）を用いるものは、大きなコレクションから小さなコレクションを作成するために使用されます。構造体を使用するものは、複数のコレクションを結合します。データ表現の開発が適切なプログラム設計の出発点であるため、プログラマが構造体を含むデータ定義を項目化したいと考えたり、項目化されたデータを結合するために構造体を使いたいと頻繁に考えるのは驚くことではありません。

前章の『Structure in the World』で扱った架空のインベーダーゲームを思い出してください。これまでのところ、ゲームには宇宙から下降してくる1つのUFOと、地上で水平移動する1つの戦車（タンク）が登場します。私たちのデータ表現では、2つのフィールドを持つ構造体を使用しています。1つはUFOのデータ表現のため、もう1つは戦車のデータ表現のためです。当然ながら、プレイヤーはミサイルを発射できる戦車を望むでしょう。突然ですが、私たちは3つの独立して移動するオブジェクト（UFO、戦車、ミサイル）を含む2つ目の種類の状態を考えることができます。したがって、2つの独立して移動するオブジェクトを表現するためのものと、3つ目のオブジェクトを含むものを表現するための、2つの異なる構造体を持つことになります。ワールド状態（world state）はこれら2つの構造体のいずれかになり得るため、起こり得るすべての状態を記述するために項目化を使用するのが自然です：

1. ワールドの状態は2つのフィールドを持つ構造体である、または
2. ワールドの状態は3つのフィールドを持つ構造体である。

私たちのドメイン（実際のゲーム）に関する限り、1番目の種類の状態は戦車が唯一のミサイルを発射する前の時間を表し、2番目の状態はミサイルが発射された後の時間を表します。

Note: ご安心ください。本書の次のパートでは、リロードなしで好きなだけミサイルを発射する方法について扱います。

本章では、構造体を含むデータ定義を項目化するという基本的な考え方を紹介します。必要な他の要素はすべて揃っているため、構造体の項目化からすぐに始めます。その後、この新しい能力を活用するワールドプログラムを含め、いくつかの例について議論します。最後のセクションはプログラミングにおけるエラーについて扱います。

6.1 項目化を用いた設計、再び (Designing with Itemizations, Again)

『Programming with Structures』で扱ったインベーダーゲームの洗練された問題記述から始めましょう。

例題 (Sample Problem) ²<http://universe> 教材パック（teachpack）を使用して、簡単なインベーダーゲームをプレイするためのゲームプログラムを設計しなさい。プレイヤーは戦車（小さな長方形）を操作し、キャンバスの上部から下部へ下降してくる

UFO (ひとつの可能性については『Intervals』を参照) から、私たちの惑星 (キャンパスの最下部) を守らなければなりません。UFO の着陸を阻止するために、プレイヤーはスペースキーを押すことで単一のミサイル (戦車より小さな三角形) を発射することができます。キー操作に応答して、ミサイルが戦車から現れます。UFO がミサイルと衝突した場合、プレイヤーの勝利となります。そうでない場合、UFO が着陸してプレイヤーの敗北となります。ここでは、3つのゲームオブジェクトとその移動に関するいくつかの詳細を述べます。第一に、戦車はキャンパスの底面に沿って一定の速度で移動しますが、プレイヤーは左矢印キーと右矢印キーを使って方向を変更できます。第二に、UFO は一定の速度で下降しますが、左右に小さなランダムなジャンプを行います。第三に、一度発射されると、ミサイルはUFO が下降する速度の少なくとも2倍以上の一定速度で、垂直な直線に沿って上昇します。最後に、UFO とミサイルの参照点が十分に接近している場合、それらは衝突したとみなされます (「十分に接近している」の定義は自由に決定してかまいません)。

続く2つのサブセクションでは、この例題を継続的な例として使用します。そのため、先へ進む前にこれをよく学び、次の演習問題を解いてください。そうすることで、問題を十分な深さで理解するのに役立ちます。

Exercise 94. 様々な段階におけるゲーム風景がどのように見えるか、いくつかのスケッチを描きなさい。そのスケッチを使って、ゲームの定数部分と変数部分を特定しなさい。前者については、ワールド (キャンパス) とそのオブジェクトの寸法を表す物理定数および画像定数 (graphical constants) を作成しなさい。また、いくつかの背景風景も作成しなさい。最後に、戦車、UFO、背景の定数から初期シーン (initial scene) を作成しなさい。

項目化の定義 (Defining Itemizations) 設計レシピの最初のステップでは、データ定義の定式化が求められます。データ定義の1つの目的は、ワールドの状態を表すデータの構築方法を記述することであり、もう1つの目的は、ワールドプログラムのイベントハンドリング関数が消費する可能性のあるすべてのデータを記述することです。構造体を含む項目化はまだ見ていないため、この最初のサブセクションではこの考え方を紹介します。これにはおそらく驚かないでしょうが、細心の注意を払ってください。

Note: このインベーダーゲームでは、3つのフィールドを持つ1つの構造体型定義だけで済ませることもできます。その場合、3番目のフィールドにはミサイルが発射されるまで `#false` が格納され、発射後はミサイルの座標を表す `Posn` が格納されます。下記を参照してください。

本章の導入部で論じたように、ミサイルを発射する戦車を伴うインベーダーゲームには、2種類の異なるゲーム状態を表すデータ表現が必要です。私たちは2つの構造体型定義を選択します：

```
(define-struct aim [ufo tank])  
(define-struct fired [ufo tank missile])
```

```

; A UFO is a Posn.
; interpretation (make-posn x y) is the UFO's location
; (using the top-down, left-to-right convention)

(define-struct tank [loc vel])
; A Tank is a structure:
;   (make-tank Number Number).
; interpretation (make-tank x dx) specifies the position:
; (x, HEIGHT) and the tank's speed: dx pixels/tick

; A Missile is a Posn.
; interpretation (make-posn x y) is the missile's place

```

最初の構造体は、プレイヤーが射撃のために戦車の位置を合わせようとしている期間のためのものであり、2番目の構造体はミサイルが発射された後の状態を表現するためのものです。しかし、ゲーム全体の状態に対するデータ定義を立案する前に、戦車、UFO、ミサイルのためのデータ表現が必要です。

演習問題 94 の対象である WIDTH や HEIGHT といった物理定数の定数定義が存在すると仮定すると、データ定義を次のように立案できます：

```

; A UFO is a Posn.
; interpretation (make-posn x y) is the UFO's location
; (using the top-down, left-to-right convention)

(define-struct tank [loc vel])
; A Tank is a structure:
;   (make-tank Number Number).
; interpretation (make-tank x dx) specifies the position:
; (x, HEIGHT) and the tank's speed: dx pixels/tick

```

； A Missile is a Pに関して、`Posn` が世界の2つの異なる側面を表すために使用されていることに少し驚くかもしれません。しかし、私たちは実世界の多くの異なる種類の情報を表すために数値（および文字列やブール値）を使用してきたので、`Posn` のような構造体のコレクションを再利用することは大きな問題ではありません。

これで、インベーダーゲームの状態に関するデータ定義を公式化する準備が整いました。

```

````racket
; A SIGS is one of:
; - (make-aim UFO Tank)
; - (make-fired UFO Tank Missile)
; interpretation represents the complete state of a
; space invader game

```

このデータ定義の形は項目化の形をしています。しかし、各条項（clause）はこれまで見てきた構造体型のデータ定義と同じように、構造体型の内容を記述しています。それでも、このデータ定義は、すべてのデータ定義がちょうど1つの構造体型定義を伴うわけで

はないことを示しています。ここでは、1つのデータ定義に2つの異なる構造体型定義が関わっています。

そのようなデータ定義の意味も明快です。これは、定義に従って作成できるすべての構造体インスタンスのコレクションに対して SIGS という名前を導入します。それでは、いくつか作成してみましょう。

- ミサイルを発射するために戦車が位置につこうと操縦している様子を記述するインスタンスです。(`make-aim (make-posn 20 10) (make-tank 28 -3)`)
- これは前述のものとはほぼ同じですが、ミサイルが発射された後の状態です。(`make-fired (make-posn 20 10) (make-tank 28 -3) (make-posn 28 (- HEIGHT TANK-HEIGHT))`) もちろん、大文字の名前はあなたが定義した物理定数を参照しています。
- 最後に、ミサイルが UFO に今にも衝突しそうな状態です。(`make-fired (make-posn 20 100) (make-tank 100 3) (make-posn 22 103)`) この例は、キャンバスの高さが 100 ピクセル以上であることを前提としています。

SIGS の最初のインスタンスはデータ定義の第 1 条項に従って生成され、2 番目と 3 番目は第 2 条項に従っていることに注目してください。当然ながら、各フィールドの数値はグローバルなゲーム定数の選択に依存します。

**Exercise 95.** なぜ 3 つのインスタンスがデータ定義の第 1 条項または第 2 条項に従って生成されるのかを説明しなさい。

**Exercise 96.** [`image: pict_82.png`] のキャンバスを仮定して、3 つのゲーム状態のそれぞれがどのように描写されるかをスケッチしなさい。

**設計レシピ** データ定義を公式化する新しい方法に伴い、設計レシピの再点検が行われます。本章ではデータを記述する 2 つ以上の手段を組み合わせる方法を紹介し、改訂された設計レシピ、特に最初のステップがこれを反映しています。

1. **いつこの新しいデータ定義方法が必要になるか？** 項目化の必要性が問題文における異なるクラスの情報区別に由来することは既に知っています。同様に、構造体ベースのデータ定義の必要性は、いくつかの異なる情報をグループ化するという要求に由来します。異なる形式のデータ（構造体のコレクションを含む）の項目化が必要となるのは、問題文が異なる種類の情報を区別しており、かつ、これらの情報のうち少なくともいくつかは複数の異なる要素で構成されている場合です。1 つ心に留めておくべきことは、データ定義が他のデータ定義を参照できるということです。したがって、データ定義内の特定の条項が複雑すぎるように思われる場合は、その条項のために別のデータ定義を記述し、その補助的な定義を参照することが認められます。そして、いつものように、データ定義を使用してデータの例を公式化します。
2. **2 番目のステップは同じままです。** 定義済みまたは組み込みのデータコレクションの名前のみに言及する関数のシグネチャを公式化し、目的のステートメントを追加し、関数ヘッダーを作成します。

3. **3 番目のステップでも何も変わりません。** 2 番目のステップからの目的のステートメントを例示する機能的な例を公式化する必要があり、項目化の項目ごとに 1 つの例が依然として必要です。
4. **テンプレートの開発は、項目化自体と、その条項における構造体の使用という 2 つの異なる次元を活用するようになります。** 第一に、テンプレートの本体は、項目化が持つ項目の数と同じ数の cond 条項を持つ cond 式で構成されます。さらに、対応する項目のデータのサブクラスを特定する条件を各 cond 条項に追加しなければなりません。第二に、ある項目が構造体を扱う場合、テンプレートには、その項目で記述されたデータのサブクラスを扱う cond 条項内にセレクト式が含まれます。しかし、別個のデータ定義でデータを記述することを選択した場合は、セレクト式を追加しません。代わりに、手元のタスクに合わせて別個のデータ定義用のテンプレートを作成し、関数呼び出しによってそのテンプレートを参照します。後者は、このデータのサブクラスが別個に処理されていることを示します。テンプレートの開発作業に入る前に、関数の性質について簡単に熟考してください。問題文が実行すべきいくつかのタスクがあることを示唆している場合、テンプレートの代わりに、個別設計された複数の関数の合成が必要である可能性が高いです。その場合、テンプレートのステップをスキップしてください。
5. **テンプレートの隙間を埋めます。** データ定義を複雑にすればするほど、このステップは複雑になります。良い知らせは、この設計レシピが多くの状況で役立つということです。行き詰まった場合は、まず簡単なケースを埋め、他のケースにはデフォルト値を使用します。これにより一部のテストケースは失敗しますが、前進しており、その進捗を視覚化できます。項目化のいくつかのケースで行き詰まった場合は、それらのケースに対応する例を分析します。テンプレートの各パーツが与えられた入力から何を取得（計算）するかを決定します。次に、これらのパーツ（およびいくつかの定数）をどのように組み合わせて目的の出力を計算するかを考えます。補助関数が必要になる可能性があることを心に留めておいてください。また、データ定義が互いに参照し合っているためにテンプレートが別のテンプレートを「呼び出す」場合は、その他方の関数の定義がまだ完了していなくても、その他方の関数が目的のステートメントと例で約束されたものを提供すると仮定します。
6. **テスト。** テストが失敗した場合は、何が間違っているか（関数、テスト、またはその両方）を特定します。適切なステップに戻ります。

「関数の設計 (Designing Functions)」に戻り、シンプルな設計レシピの説明を読み返して、この改訂版と比較してみてください。

本節の冒頭にある例題に対する描写（レンダリング）関数の設計で、この設計レシピを解説しましょう。big-bang 式は、時計の各刻み（クロックティック）、マウスクリック、またはキー入力後に、世界の状態を画像に変換するために、このような描写関数を必要とすることを思い出してください。

この描写関数のシグネチャは、世界の状態クラスのエレメントを Image のクラスへとマッピングすることを示しています。

```

; SIGS -> Image
; adds TANK, UFO, and possibly MISSILE to
; the BACKGROUND scene
(define (si-render s) BACKGROUND)

```

ここで TANK、UFO、MISSILE、および BACKGROUND は、演習問題 94 で求められた画像定数です。このシグネチャは、常に世界の状態のコレクションを消費して何らかの画像を生成する、描写関数の一般的なシグネチャの 1 つのインスタンスにすぎないことを思い出してください。

Figure 32: Rendering space invader game states, by example	
[image: pict_83.png]	
[image: pict_84.png]	
[image: pict_85.png]	
s	(si-render s)
(make-aim (make-posn 10 20) (make-posn 10 20))	[image: pict_83.png]
(make-aim (make-posn 10 20) (make-tank 28 -3))	
(make-fired (make-posn 20 100) (make-posn 20 100) ...)	[image: pict_84.png]
(make-fired (make-posn 20 100) (make-tank 100 3) (make-posn 22 103))	
(make-fired (make-posn 10 20) (make-posn 10 20) (make-tank 28 -3))	[image: pict_85.png]
(make-fired (make-posn 10 20) (make-tank 28 -3) (make-posn 32 (- HEIGHT TANK-HEIGHT)))	

データ定義の項目化は 2 つの項目で構成されているため、上記のデータ例を使用して 3 つの例を作成しましょう。figure 32 を参照してください。数学の本に見られる関数表とは異なり、この表は垂直にレンダリングされます。左の列には目的の関数のサンプル入力が含まれ、右の列には対応する期待される結果が一覧表示されます。おわかりのように、設計レシピの最初のステップからのデータ例を使用しており、これらは項目化の双方の項目を網羅しています。

次に、設計プロセスの最も重要なステップであるテンプレートの開発に移ります。まず、si-render の本体は 2 つの cond 条項を持つ cond 式でなければならないことがわかります。設計レシピに従うと、2 つの条件は (aim? s) と (fired? s) であり、これらは si-render が消費する可能性のある 2 種類のデータを区別します。

```
(define (si-render s)
 (cond
 [(aim? s)...]
 [(fired? s)...]))
```

第二に、構造体を扱うすべての cond 条項にセレクト式を追加します。この場合、両方の条項が構造体の処理に関わっています (aim と fired)。前者は 2 つのフィールドを持つため、最初の cond 条項に対して 2 つのセレクト式を必要とし、後者の種類の構造体は 3 つの値で構成されるため、3 つのセレクト式を要求します。

```
(define (si-render s)
 (cond
 [(aim? s) (... (aim-tank s)... (aim-ufo s)...)]
 [(fired? s) (... (fired-tank s)... (fired-ufo s)
... (fired-missile s)...)]))
```

テンプレートには、タスクを完了するために必要なほぼすべてのものが含まれています。定義を完成させるために、各 cond 行について、期待される結果を計算するために手中にある値をどのように組み合わせるかを考えます。入力の要素以外にも、グローバルに定義された定数（例えば、ここでは明らかに役立つ BACKGROUND）、プリミティブまたは組み込みの操作、そして万策尽きた場合には「ウィッシュリスト（望むもの一覧）関数」（つまり、あればいいのにと望む関数を記述すること）を使用できます。

戦車のデータ表現（すなわち (aim-tank s)）と UFO のデータ表現（すなわち (aim-ufo s)）を持っている最初の cond 条項について考えます。figure 32 の最初の例から、背景風景にこれら 2 つのオブジェクトを追加する必要があることがわかります。さらに、設計レシピは、これらのデータが独自のデータ定義を伴う場合、ヘルパー（補助）関数を定義することを検討し、それらを使用して結果を計算することを提案しています。

```
... (tank-render (aim-tank s)
 (ufo-render (aim-ufo s) BACKGROUND))
```

ここで tank-render と ufo-render はウィッシュリスト関数です。

```
; Tank Image -> Image
; adds t to the given image im
(define (tank-render t im) im)
```

```
; UFO Image -> Image
; adds u to the given image im
(define (ufo-render u im) im)
```

```
+-----+
| Figure 33: The complete rendering function |
| | | | |
| ; SIGS -> Image |
| ; renders the given game state on top of BACKGROUND |
| ; for examples see figure 32 |
| (define (si-render s) |
```



```

| (cond
| [(aim? s)
| (tank-render (aim-tank s)
| (ufo-render (aim-ufo s) BACKGROUND))]
| [(fired? s)
| (tank-render
| (fired-tank s)
| (ufo-render (fired-ufo s)
| (missile-render (fired-missile s)
| BACKGROUND))))])
+-----+

```

少々類推を用いれば、2 番目の cond 条項も同様に扱うことができます。figure 33 に完全な定義を示します。何よりも素晴らしいことに、ウィッシュリスト関数である tank-render と ufo-render をすぐに再利用でき、追加する必要があるのはシーンにミサイルを含めるための関数だけです。適切なウィッシュリストの項目は以下の通りです。

```

; Missile Image -> Image
; adds m to the given image im
(define (missile-render m im) im)

```

上記と同様に、コメントは私たちが望むものを十分な詳細さで記述しています。定義を示しています。何よりも素晴らしいことに、ウィッシュリスト関数である tank-render と ufo-render をすぐに再利用でき、追加する必要があるのはシーンにミサイルを含めるための関数だけです。適切なウィッシュリストの項目は以下の通りです：

上記と同様に、コメントは私たちが望むものを十分な詳細さで説明しています。

**Exercise 97.** 関数 tank-render、ufo-render、missile-render を設計せよ。以下の式：

```

(tank-render
 (fired-tank s)
 (ufo-render (fired-ufo s)
 (missile-render (fired-missile s)
 BACKGROUND)))

```

と、こちらの式を比較せよ：

```

(ufo-render
 (fired-ufo s)
 (tank-render (fired-tank s)
 (missile-render (fired-missile s)
 BACKGROUND)))

```

これら 2 つの式は、どのような場合に同じ結果を生成するか？

**Exercise 98.** stop-when ハンドラとして使用する関数 si-game-over? を設計せよ。UFO が着陸した場合、またはミサイルが UFO に命中した場合にゲームは停止する。どちらの条件についても、あるオブジェクトと別のオブジェクトとの接近度を判定することを推奨する。

stop-when 句には、オプションで 2 番目の部分式、すなわちゲームの最終状態を描画する関数を指定できる。si-render-final を設計し、演習問題 100 のメイン関数における stop-when 句の 2 番目のパーツとして使用せよ。

**Exercise 99.** si-move を設計せよ。この関数は、オブジェクトが現在どの位置に移動するかを決定するために、クロックティックごとに呼び出される。したがって、この関数は SIGS の要素を 1 つ受け取り、別の SIGS 要素を生成する。

戦車とミサイル（存在する場合）の移動は比較的分かりやすい。これらは一定の速度で直線上に移動する。一方、UFO の移動には左右への小さなランダムなジャンプが求められる。あなたはこれまで乱数を生成する関数を扱ったことがないため、この演習問題の残りの部分は、この問題に対処する方法についての少し長めのヒントとなっている。

BSL には乱数を生成する関数が付属している。この関数を導入することは、なぜシングネチャと目的の記述が設計においてこれほど重要な役割を果たすのかを示している。必要な関数の関連資料は以下のとおりである：

```
; Number -> Number
; produces a number in the interval [0,n),
; possibly a different one each time it is called
(define (random n)...)
```

シングネチャと目的の記述は関数が何を計算するかを正確に説明しているため、DrRacket の対話領域で random を試してみることができる。手を止めて、実際に試してみよう！

random が呼び出されるたびに（ほぼ）異なる数値を生成する場合、random を使用する関数のテストは困難になる。まずは、si-move とその本来の機能を 2 つの部分に分割する：

**注:** random を使用しなければならないという考えは BSL の知識であり、習得すべき設計スキルのパートではないため、このヒントを提供している。また、random は数学的関数ではない BSL の最初にして唯一のプリミティブである。プログラミングにおける関数は数学的関数からインスピレーションを得ているが、同一の概念ではない。

```
(define (si-move w)
 (si-move-proper w (random...)))

; SIGS Number -> SIGS
; moves the space-invader objects predictably by delta
(define (si-move-proper w delta)
 w)
```

この定義により、乱数の生成とゲームオブジェクトの移動という行為を分離する。random は呼び出されるたびに異なる結果を生成するかもしれないが、si-move-proper は特定の数値入力でテストできるため、同じ入力を与えられた場合には確実に同じ結果を返すことが保証される。要するに、コードの大部分がテスト可能なまま残るのである。

random を直接呼び出す代わりに、UFO のランダムな x 座標を生成する関数を設計したいと思うかもしれない。そのような関数をテストするために、BSL のテストフレームワークに含まれる check-random の使用を検討せよ。

**Exercise 100.** キーイベント (KeyEvent) ハンドラの役割を果たす関数 si-control を設計せよ。この関数はゲーム状態と KeyEvent を受け取り、新しいゲーム状態を生成する。この関数は 3 つの異なるキーに反応する：

- 左矢印キーを押すと、戦車が左に移動する。
- 右矢印キーを押すと、戦車が右に移動する。
- スペースキーを押すと、ミサイルがまだ発射されていない場合にミサイルを発射する。

この関数ができあがったら、big-bang を使用してゲームプレイウィンドウを起動する si-main 関数を定義できる。楽しんでほしい！

```
+-----+
| Figure 34: Rendering game states again |
| |
| ; SIGS.v2 -> Image |
| ; renders the given game state on top of BACKGROUND |
| (define (si-render.v2 s) |
| (tank-render |
| (sigs-tank s) |
| (ufo-render (sigs-ufo s) |
| (missile-render.v2 (sigs-missile s) |
| BACKGROUND)))) |
+-----+
```

データ表現が唯一であることはめったにない。例えば、インベーダーゲームの状態を表すために単一の構造体型を使用することもできる：

```
(define-struct sigs [ufo tank missile])
; A SIGS.v2 (short for SIGS version 2) is a structure:
; (make-sigs UFO Tank MissileOrNot)
; interpretation represents the complete state of a
; space invader game

; A MissileOrNot is one of:
; - #false
; - Posn
; interpretation #false means the missile is in the tank;
; Posn says the missile is at that location
```

ゲーム状態の最初のデータ表現とは異なり、この 2 番目のバージョンではミサイルの発射前と発射後を区別しない。代わりに、各状態にはミサイルに関するデータが含まれる。ただし、このデータはミサイルがまだ発射されていないことを示す #false である場合もある。

その結果、この 2 番目の状態のデータ表現に対する関数は、最初のための関数とは異なる。特に、SIGS.v2 の要素を受け取る関数は cond 式を使用しない。なぜなら、そのコレクションには 1 種類の要素しか存在しないからである。設計アプローチの観点からは、「Designing with Structures」の構造体のための設計レシピで十分である。Figure 34 は、このデータ表現に対する描画関数を設計した結果を示している。

```
; MissileOrNot Image -> Image
; adds an image of missile m to scene s
(define (missile-render.v2 m s)
 s)
```

```

+-----+
| Figure 35: Rendering the space invader games, with tanks |
+-----+
| [image: pict_86.png] |
| [image: pict_87.png] |
+-----+
| m | | (missile-render.v2 m s) |
+-----+
| #false | | [image: pict_86.png] |
| (make-posn ↻ 32 ↻ (- HEIGHT ↻ TANK-... | | [image: pict_87.png] |
| (make-posn | |
| 32 | |
| (- HEIGHT | |
| TANK-HEIGHT | |
| 10)) | |
+-----+

```

これでテンプレートを作成する準備が整った。主要な引数 (m) のデータ定義は 2 つの要素を持つ箇条書きであるため、関数本体は 2 つの句を持つ cond 式で構成される可能性が高い：

```
[(boolean? m)...]
[(posn? m)...]))
```

再びデータ定義に従うと、最初の cond 句は m が Boolean 値であるかどうかをチェックし、2 番目の句はそれが Posn の要素であるかどうかをチェックする。そして、仮に誰かが誤って missile-render.v2 を #true と何らかの画像に適用した場合、関数は最初の cond 句を使用して結果を計算することになる。このようなエラーについては、以下でさらに詳しく述べる。

テンプレートの第2ステップでは、構造体を扱うすべての cond 句にセレクト式を含めることを要求している。今回の例では、これは2番目の句に当てはまり、セレクト式は与えられた Posn から x 座標と y 座標を抽出する：

```
(define (missile-render.v2 m s)
 (cond
 [(boolean? m)...]
 [(posn? m) (... (posn-x m)... (posn-y m)...)]))
```

このテンプレートを上記の si-render のテンプレートと比較してみよう。後者のデータ定義は2つの異なる構造体型を扱っているため、si-render の関数テンプレートには両方の cond 句にセレクト式が含まれている。しかし、MissileOrNot のデータ定義は、単なる値である要素と構造体を記述する要素を混在させている。どちらの種類の定義もまったく問題ない。重要なのは、レシピに従い、データ定義に一致するコード構造を見つけることである。

以下が完全な関数定義である：

```
(define (missile-render.v2 m s)
 (cond
 [(boolean? m) s]
 [(posn? m)
 (place-image MISSILE (posn-x m) (posn-y m) s)]))
```

これをステップバイステップで行うには、まず簡単な句から取り組む。この関数では、それが最初の句である。ミサイルが発射されていないことを示しているため、関数は与えられた s をそのまま返す。2 番目の句については、(posn-x m) と (posn-y m) がミサイル画像の座標を選択することを思い出す必要がある。この関数は s に MISSILE を追加しなければならないため、4つの値を組み合わせるのに最適なプリミティブ操作と独自の関数の組み合わせを見つける必要がある。この結合操作の選択こそが、プログラマーとしてのあなたの創造的洞察力が活かされる場所である。

**Exercise 102.** この2番目のデータ定義に対してゲームを完成させるために必要な、他のすべての関数を設計せよ。 と(posn-y m)がミサイルの画像の座標を選択することを思い出す必要があります。この関数は s に MISSILE` を追加しなければならないため、4つの値を組み合わせるためにプリミティブ操作と独自の関数の最適な組み合わせを見つける必要があります。この結合操作の選択こそが、プログラマーとしてのあなたの創造的な洞察力が発揮される場所です。

**Exercise 102.** この 2 番目のデータ定義において、ゲームを完成させるために必要な他のすべての関数を設計しなさい。

**Exercise 103.** 次の 4 種類の動物園の動物に関するデータ表現を作成しなさい：

- クモ (spiders)：関連する属性は残りの足の数（クモは事故で足を失う可能性があるとは仮定します）と、輸送時に必要なスペースです。
- ゾウ (elephants)：唯一の属性は輸送時に必要なスペースです。
- ボアコンストリクター (boa constrictors)：属性には長さとお回りが含まれます。
- アルマジロ (armadillos)：輸送に必要なスペースを決定する属性を含め、適切な属性をあなたが決定しなければなりません。

動物園の動物を受け取る関数のためのテンプレートを作成しなさい。

動物園の動物とケージの説明を受け取る `fits?` 関数をデザインしなさい。この関数は、ケージの容積がその動物にとって十分な大きさであるかどうかを判定します。

**Exercise 104.** あなたの地元の町では、自動車 (automobiles)、バン (vans)、バス (buses)、SUV という車両群 (fleet of vehicles) を管理しています。乗り物のためのデータ表現を作成しなさい。各乗り物の表現には、乗車可能な乗客数、ナン、および燃料消費量 (1 ガロンあたりのマイル数、miles per gallon) を記述しなければなりません。乗り物を受け取る関数のためのテンプレートを作成しなさい。

**Exercise 105.** あるプログラムに次のデータ定義が含まれています。

```
; A Coordinate is one of:
; - a NegativeNumber
; interpretation on the y axis, distance from top
; - a PositiveNumber
; interpretation on the x axis, distance from left
; - a Posn
; interpretation an ordinary Cartesian point
```

データ定義の各節 (clause) について、少なくとも 2 つのデータ例を作成しなさい。それぞれの例について、キャンバスのスケッチを用いてその意味を説明しなさい。

## 6.2 世界を混ぜ合わせる (Mixing Up Worlds)

このセクションでは、バーチャルペットに関する簡単な拡張問題をはじめとして、ワールドプログラムについてのいくつかの設計問題を提案します。

**Exercise 106.** 「さらにいくつかのバーチャルペット (More Virtual Pets)」では、幸福度ゲージが付いたバーチャルペットの作成について議論しました。バーチャルペットの一方はネコで、もう一方はカメレオンです。しかし、それぞれのプログラムは単一のペット専用になっていました。

位置と動物の両方を受け取り、与えられた位置から出発して後者の動物をキャンバスを横切って歩かせる `cat-cham` ワールドプログラムを設計しなさい。動物として選択されたデータ表現は以下の通りです。



```
; A VAnimal is either
; - a VCat
; - a VCham
```

ここで VCat と VCham は、演習問題 88 と 92 で作成したデータ定義です。

VAnimal がワールド状態の集合であるとして、以下を設計する必要があります。

- VAnimal から Image への描画関数
- VAnimal から VAnimal へのクロックティックを処理する関数
- 該当する場合に応じて、動物に餌を与えたり、撫でたり、色を変えたりできるように、キーイベントを扱う関数

ネコの色を変えたり、カメレオンを撫でたりすることは依然として不可能です。

**Exercise 107.** バーチャルネコとバーチャルカメレオンの両方を扱う `cham-and-cat` プログラムを設計しなさい。両方の動物を含む「zoo (動物園)」のデータ定義と、それを扱う関数が必要となります。

問題文では、キーが2匹の動物をどのように操作するかについては開かれたままになっています。以下に2つの可能な解釈を示します。

1. 各キーイベントが両方の動物に届く。
2. 各キーイベントが2匹の動物のうち一方のみに適用される。この選択肢では、フォーカスされている動物、つまり現在操作可能な動物を指定するデータ表現が必要です。フォーカスを切り替えるには、キー処理関数に「kitty (子猫)」を表す "k" と「lizard (トカゲ)」を表す "l" を解釈イヤてくださいますとイレイ、"k" が “押す押す以降、それ以降のキーストロークは猫にのみ適用されます。

選択肢のいずれか1つを選び、適切なプログラムを設計しなさい。

**Exercise 108.** デフォルト状態では、歩行者用信号機は黒の背景の上にオレンジ色の立っている人の姿を表示します。歩行者が道路を渡る時間になると、信号機はシグナルを受信し、緑色の歩いている人の姿に切り替わります。このフェーズは10秒間続きます。その後、信号機は数字 9, 8, ..., 0 を表示し、奇数はオレンジ色、偶数は緑色で着色されます。カウントダウンが0に達すると、信号機はデフォルト状態に戻ります。

そのような歩行者用信号機を実装する世界プログラムを設計しなさい。キーボードのスペースキーを押すと、信号機はデフォルト状態から切り替わります。それ以外のすべての遷移はクロックティックに対する反応でなければなりません。以下の画像を使用しても構いませんし、

[image: pedestrian\_traffic\_light\_red.png]

[image: pedestrian\_traffic\_light\_green.png]

あるいは `2http/image` ティーチパックを使って独自の棒人間を作成しても構いません。

**Exercise 109.** 一することもできます。

**Exercise 109.** 一連の KeyEvent におけるパターンを認識するワールドプログラムを設計しなさい。初期状態では、プログラムは 100 × 100 の白い長方形を表示します。プログラムが最初の意図された文字を検出すると、同じサイズの黄色の長方形を表示します。最後の文字を検出した後、長方形の色は緑色に変わります。何か「不正な (bad)」キーイベントが発生した場合、プログラムは赤色の長方形を表示します。

```
+-----+
| Figure 36: Two ways of writing a data definition for FSMs |
|-----+-----+-----+
| conventional | | defined abbreviations |
|-----+-----+-----+
| ; ExpectsToSee.v1 is one of: ⌞; - ... | ; ExpectsToSee.v2 is one of: ⌞; - |
| ... |
| ; ExpectsToSee.v1 is one of: | | |
| ; -"start, expect an 'a'" | | |
| ; -"expect 'b', 'c', or 'd'" | | |
| ; -"finished" | | |
| ; -"error, illegal key" | | |
| ; ExpectsToSee.v2 is one of: | | |
| ; -AA | | |
| ; -BB | | |
| ; -DD | | |
| ; -ER | | |
| (define AA "start,...") | | |
| (define BB "expect...") | | |
| (define DD "finished") | | |
| (define ER "error,...") | | |
|-----+-----+-----+
+-----+
```

プログラムが探索する特定のシーケンスは "a" で始まり、その後に "b" と "c" の任意の長さの混合が続き、"d" で終わります。明らかに、"acbd" は受け入れ可能な文字列のひとつの例です。その他の 2 つの例は "ad" と "abcbbbcd" です。もちろん、"da"、"aa"、"d" などはマッチしません。

```
+-----+
| Figure 37: A finite state machine as a diagram |
|-----+-----+-----+
| [image: pict_88.png] |
| [image: pict_88.png] |
|-----+-----+-----+
+-----+
```

ヒント あなたの解法は有限状態機械 (FSM: finite state machine) を実装します。これは、ワールドプログラムの背後にあるひとつの設計原則として「有限状態の世界 (Finite State Worlds)」で導入された概念です。その名前が示すように、FSM プログラムは有限の個数



の状態のうちの1つをとります。最初の状態は初期状態 (initial state) と呼ばれます。それぞれのキーイベントにより、マシンは現在の状態を再検討します。同じ状態に遷移することあれば、別の状態に遷移することもあります。プログラムが適切なキーイベントのシーケンスを認識すると、最終状態 (final state) へと遷移します。

シーケンス認識問題では、状態は通常、マシンが次に現れることを期待している文字を表します。データ定義については figure 36 を参照してください。不正な入力が発出されたことを示す最後の状態を確認してください。Figure 37 は、これらの状態とその関係を図形式で捉える方法を示しています。各ノードは4つの有限状態のうちの1つに対応し、各矢印はどの KeyEvent によってプログラムがある状態から別の状態へ遷移するかを指定しています。

**Note:** 右側のデータ定義は、演習問題 61 で導入された命名手法を使用しています。

歴史 1950 年代、私たちがコンピュータ科学者と呼ぶであろうスティーヴン・C・クリーネ (Stephen C. Kleene) は、テキストパターンを認識する問題のための記法として正規表現 (regular expressions) を発明しました。上記の問題に対して、クリーネなら次のように記述するでしょう。

$a(b|c)^*d$

これは、 $d$  に遭遇するまで  $a$  の後に任意回数の  $b$  または  $c$  が続くことを意味します。

### 6.3 入力エラー (Input Errors)

本章の中心的なポイントの1つは、述語 (predicates) の役割に関するものです。述語は、データの混合 (mixes of data) を処理する関数を設計しなければならない場合に不可欠となります。このような混合は、問題文で多様な種類の情報が言及されている場合に自然と現れますが、自分が作成した関数やプログラムを他人に渡す際にも現れます。結局のところ、あなた自身は自分のデータ定義や関数のシグネチャを理解し、尊重しています。しかし、友人や同僚がどのように振る舞うかは分かりませんし、特に BSL やプログラミングの知識がない人があなたのプログラムをどのように使うかは全く予想できません。したがって、本節では不適切な入力からプログラムを保護するための一つの方法を紹介します。

**Note:** 私たちが常に自分自身の関数のシグネチャを尊重していると期待するのは、一種の自己欺瞞です。誤った種類のデータに対して関数を呼び出してしまうことは、誰にでも起こり得ます。多くの言語は BSL のようにプログラマ自身がシグネチャをチェックすることを期待していますが、他の言語では多少の複雑さと引き換えにそれを自動的に行います。

この点を単純なプログラム、すなわち円盤の面積を計算する関数を使って実証してみましょう。

```
; Number -> Number
; computes the area of a disk with radius r
(define (area-of-disk r)
 (* 3.14 (* r r)))
```

私たちの友人は、幾何学の宿題のためにこの関数を使いたいと思うかもしれません。あいにく、友人がこの関数を使う際、誤って数値ではなく文字列に適用してしまう可能性があります。それが起こると、関数は不可解なエラーメッセージとともにプログラムの実行を停止します。

```
> (area-of-disk "my-disk")
*:expects a number, given "my-disk"
```

述語を使用すると、このような分かりにくいエラーメッセージを防ぎ、自分で選択した分かりやすいエラーを発生させることができます。

```
+-----+
| Figure 38: The universe of BSL data |
| |
| ; Any BSL value is one of: |
| ; - Number |
| ; - Boolean |
| ; - String |
| ; - Image |
| ; - (make-posn Any Any) |
| ; ... |
| ; - (make-tank Any Any) |
| ; ... |
+-----+
```

具体的には、関数を友人に渡したい場合、関数のチェック機能付きバージョン (checked versions) を定義できます。友人は BSL をあまり知らないかもしれないため、彼らがこのチェック機能付き関数を任意の BSL 値 (数値、文字列、画像、Posn など) に適用することを想定しなければなりません。BSL でどの構造体型が定義されるかをあらかじめ予測することはできませんが、すべての BSL 値のコレクションに対するデータ定義の大まかな形状は分かっています。Figure 38 は、このデータ定義の形状を示しています。「データの宇宙 (The Universe of Data)」で議論したように、あらゆる構造体型の定義が新しいインスタンスを追加するため、Any に対するデータ定義はオープンエンド (拡張可能) です。これらのインスタンスには再び Any の値を保持する可能性があるため、これは Any のデータ定義が自分自身を参照しなければならないことを意味します。これは最初は恐ろしく思える考え方です。

この箇条書き (itemization) に基づくと、チェック付き関数のテンプレートはおおむね次のような形状になります。

```

; Any ->???
(define (checked-f v)
 (cond
 [(number? v)...]
 [(boolean? v)...]
 [(string? v)...]
 [(image? v)...]
 [(posn? v) (... (posn-x v)... (posn-y v)...)]
 ...
; which selectors are needed in the next clause?
 [(tank? v)...]
 ...))

```

もちろん、誰もこの定義のすべての節 (clause) を列挙することはできません。幸いなことに、その必要はありません。私たちが知っているのは、元の関数が定義されている値のクラスに含まれるすべての値に対して、チェック付きバージョンが同じ結果を生成しなければならないということ、そしてそれ以外のすべての値に対してはエラーを発行しなければならないということです。

具体的には、サンプル関数 `checked-area-of-disk` は任意の BSL 値を受け取り、入力が数値である場合は `area-of-disk` を使って円盤の面積を計算します。そうでない場合はエラーメッセージとともに停止しなければなりません。BSL では、これを実現するために関数 `error` を使用します。error 関数は文字列を受け取り、プログラムを停止させます。

```
(error "area-of-disk: number expected")
```

したがって、`checked-area-of-disk` のおおまかな定義は次のようになります。

```

(define MESSAGE "area-of-disk: number expected")

(define (checked-area-of-disk v)
 (cond
 [(number? v) (area-of-disk v)]
 [(boolean? v) (error MESSAGE)]
 [(string? v) (error MESSAGE)]
 [(image? v) (error MESSAGE)]
 [(posn? v) (error MESSAGE)]
 ...
 [(tank? v) (error MESSAGE)]
 ...))

```

`else` を使用することで、この定義を自然な形で完成させることができます。

```

; Any -> Number
; computes the area of a disk with radius v,
; if v is a number
(define (checked-area-of-disk v)
 (cond
 [(number? v) (area-of-disk v)]
 [else (error "area-of-disk: number expected")]))

```

意図通りの結果が得られることを確認するために、実験してみましょう。

```
> (checked-area-of-disk "my-disk")
area-of-disk:number expected
```

自作のプログラムを他人に使ってもらうために配布する場合、チェック付き関数を書くことは重要です。しかし、正しく動作するプログラムを設計することの方がはるかに重要です。本書は適切なプログラム設計のための設計プロセスに焦点を当てており、注意を散漫にさせることなくこれを行うため、私たちは常にデータ定義とシグネチャを厳守することに同意します。少なくとも、ほぼ常にそうすることとし、ごくまれに関数やプログラムのチェック付きバージョンを設計するようお願いする場合があります。

**Exercise 110.** `area-of-disk` のチェック付きバージョンは、関数の引数が単なる任意の数値ではなく正の数であることを強制することもできます。`checked-area-of-disk` をこのように変更してください。

**Exercise 111.** 以下の定義を見てください。

```
(define-struct vec [x y])
; A vec is
; (make-vec PositiveNumber PositiveNumber)
; interpretation represents a velocity vector
```

基本操作 `make-vec` のチェック付きバージョンとして理解される関数 `checked-make-vec` を作成してください。この関数は、`make-vec` への引数が正の数であることを保証します。言い換えれば、`checked-make-vec` は非形式的なデータ定義を強制します。

## 述語 (Predicates)

どのようにして独自の述語を設計できるのか疑問に思うかもしれません。結局のところ、チェック付き関数は実際には次のような一般的な形状をしているように見えます。

```
; Any ->...
; checks that a is a proper input for function g
(define (checked-g a)
 (cond
 [(XYZ? a) (g a)]
 [else (error "g: bad input")]))
```

ここで、`g` 自体は次のように定義されています。

```
; XYZ ->...
(define (g some-x)...)
```

`XYZ` というラベルのついたデータ定義が存在し、`a` が `XYZ` の要素である場合には `(XYZ? a)` が `#true` を生成し、そうでない場合には `#false` を生成すると仮定します。

`Number` を受け取る `area-of-disk` については、`number?` が明らかに適切な述語です。対照的に、前出の `missile-render` のようないくつかの関数については、`MissileOrNot` が組み込

みのデータコレクションではなく作り出されたものであるため、明らかに独自の述語を定義する必要があります。それでは、MissileOrNot のための述語を設計してみましょう。

述語のシグネチャを思い出してみましょう。

```
; Any -> Boolean
; is a an element of the MissileOrNot collection
(define (missile-or-not? a) #false)
```

述語の目的文 (purpose statements) として疑問文を使用するのは良い習慣です。なぜなら、述語を適用することは値について質問することに似ているからです。名前の末尾にある疑問符「?」もこの考え方を強固にします。そのような関数の名前を発音する際に、末尾に「huh」を付け足す人もいます。

例を作成するのも簡単です。

```
(check-expect (missile-or-not? #false) #true)
(check-expect (missile-or-not? (make-posn 9 2)) #true)
(check-expect (missile-or-not? "yellow") #false)
```

最初の 2 つの例は、MissileOrNot のすべての要素が #false または何らかの Posn のいずれかであることを思い出させます。3 番目のテストは、文字列がこのコレクションの要素ではないことを示しています。さらに 3 つのテストを示します。

```
(check-expect (missile-or-not? #true) #false)
(check-expect (missile-or-not? 10) #false)
(check-expect (missile-or-not? empty-image) #false)
```

期待される答えの理由を説明してください！

述語は可能なすべての BSL 値を受け取るため、そのテンプレートは checked-f のテンプレートとまったく同じです。手を止めてください！読み進める前に、テンプレートを見つけてもう一度確認してください。

チェック付き関数と同様に、述語はすべての可能な cond 行を必要とするわけではありません。#true を生成する可能性がある行だけが必要です。

```
(define (missile-or-not? v)
 (cond
 [(boolean? v)...]
 [(posn? v) (... (posn-x v)... (posn-y v)...)]
 [else #false]))
```

他のすべてのケースは、#false を生成する else 行を介してまとめられます。

テンプレートが与えられれば、missile-or-not? の定義は、各ケースについて熟考するだけの簡単な問題です。

```
(define (missile-or-not? v)
 (cond
 [(boolean? v) (boolean=? #false v)]
```

```
[(posn? v) #true]
[else #false]))
```

#false のみが正当な MissileOrNot であり、#true は違います。私たちはこのアイデアを (boolean=? #false v) で表現しますが、(false? v) でも構いません。

```
(define (missile-or-not? v)
 (cond
 [(false? v) #true]
 [(posn? v) #true]
 [else #false]))
```

当然ながら、Posn のすべての要素は MissileOrNot のメンバーでもあり、これが 2 行目の #true の理由を説明しています。

**Exercise 112.** or 式を使用して、この述語を再定式化してください。

**Exercise 113.** 前節のデータ定義である SIGS、Coordinate (exercise 105)、および VAnimal に対する述語を設計してください。

締めくくりとして、ワールドプログラムで使用したくなるかもしれない 2 つの重要な述語として key-event? と mouse-event? を挙げておきます。これらは期待される性質を検証しますが、何が計算されるのかを確実に理解するために、それらのドキュメントを確認しておく必要があります。

## 6.4 ワールドのチェック (Checking the World)

ワールドプログラムでは、多くのことがうまく行かなくなる可能性があります。関数は常に適切な種類のデータに適用されると信頼することに同意したばかりですが、ワールドプログラムでは一度にあまりにも多くのことを同時にこなすため、自分自身にそこまでの信頼を寄せることはできないかもしれません。クロックティック、マウスクリック、キー入力、そして描画を処理するワールドプログラムを設計するとき、それらの相互作用のどれかを誤ってしまうのはあまりにも簡単なことです。もちろん、誤りが生じたからといって、BSL がその間違いを即座に認識するわけではありません。例えば、私たちの関数の 1 つが、ワールド状態のためのデータ表現の要素とは言えない結果を生成してしまうかもしれません。それと同時に、big-bang はこのデータを受け取り、次のイベントが発生するまでそれを保持し続けます。プログラムが失敗するのは、後続のイベントハンドラがこの不適切なデータを受け取った時だけかもしれません。しかし、事態はさらに悪化することもあります。というのも、2 番目、3 番目、あるいは 4 番目のイベント処理ステップでさえ、実際には不適切な状態値をなんとか処理できてしまい、処理のずっと後になってからすべてが破綻する可能性があるからです。

この種の問題に対処するため、big-bang には、ワールド状態に対する述語を受け取るオプションの check-with 節が用意されています。例えば、すべてのワールド状態を Number で表現することを選択した場合、この事実を次のように簡単に表現できます：

```
(define (main s0)
 (big-bang s0... [check-with number?]....))
```

いずれかのイベント処理関数が数値以外のものを生成した瞬間、ワールドは適切なエラーメッセージを出して停止します。

check-with 節は、データ定義が number? のような組み込み述語を持つ単なるデータのクラスではなく、次のような区間定義のように繊細なものである場合、さらに役立ちます：

```
; A UnitWorld is a number
; between 0 (inclusive) and 1 (exclusive).

; Any -> Boolean
; is x between 0 (inclusive) and 1 (exclusive)
```

```
(check-expect (between-0-and-1? "a") #false)
(check-expect (between-0-and-1? 1.2) #false)
(check-expect (between-0-and-1? 0.2) #true)
(check-expect (between-0-and-1? 0.0) #true)
(check-expect (between-0-and-1? 1.0) #false)
```

```
(define (between-0-and-1? x)
 (and (number? x) (<= 0 x) (< x 1)))
```

その場合、この区間のための述語を定式化したくなります：

```
; Any -> Boolean
; is x between 0 (inclusive) and 1 (exclusive)

(check-expect (between-0-and-1? 0.0) #true)
(check-expect (between-0-and-1? 1.0) #false)

(define (between-0-and-1? x)
 (and (number? x) (<= 0 x) (< x 1)))
```

この述語を使えば、ワールドプログラムにおけるあらゆる単一の遷移を監視できるようになります：

```
(define (main s0)
 (big-bang s0
 ...
 [check-with between-0-and-1?]
 ...))
```

ワールドを生成するハンドラのいずれかが区間外の数値を生成したり、さらに悪いことに非数値の値を生成したりした場合、プログラムはその間違いを即座に発見し、修正する機会を与えてくれます。

Exercise 114. 演習 113 の述語を使用して、インベーダーゲームのワールドプログラム、バーチャルペットのプログラム（演習 106）、およびエディタプログラム（グラフィカルエディタ）をチェックしなさい。



## 6.5 等価述語 (Equality Predicates)

等価述語とは、同じデータコレクションの2つの要素を比較する関数です。3つの文字列“red”、“green”、“yellow”のコレクションであるTrafficLightの定義を思い出してください。light=?関数を定義する1つの方法を以下に示します：

```
; TrafficLight TrafficLight -> Boolean
; are the two (states of) traffic lights equal

(check-expect (light=? "red" "red") #true)
(check-expect (light=? "red" "green") #false)
(check-expect (light=? "green" "green") #true)
(check-expect (light=? "yellow" "yellow") #true)

(define (light=? a-value another-value)
 (string=? a-value another-value))
```

RUNをクリックすると、すべてのテストが成功しますが、残念ながら他の対話によって意図との不一致が明らかになります：

```
> (light=? "salad" "greens")
#false
> (light=? "beans" 10)
string=?:expects a string, given 10
```

これらの対話を、他の組み込み等価述語と比較してみてください：

```
> (boolean=? "#true" 10)
boolean=?:expects a boolean as 1st argument, given "#true"
```

ご自身で (string=? 10 #true) や (= 20 “help”) を試してみてください。それらはすべて、誤った種類の引数に適用されたというエラーを発生させます。

チェック機能を備えた改訂版の light=? は、両方の引数が TrafficLight に属していることを強制します。属していない場合、組み込みの等価述語が発生させるようなエラーを送出します。簡潔にするため、TrafficLight に対する述語を light? と呼ぶことにします：

**注:** 文字の大文字と小文字は区別されます。“red”は“Red”や“RED”とは異なります。

```
; Any -> Boolean
; is the given value an element of TrafficLight
(define (light? x)
 (cond
 [(string? x) (or (string=? "red" x)
 (string=? "green" x)
 (string=? "yellow" x))]
 [else #false]))
```



これで、最初の分析に従って light=? の改訂を締めくくることができます。まず、関数は 2 つの入力が TrafficLight の要素であるかどうかを判定し、そうでない場合は error を使用して間違いを通知します：

```
(define MESSAGE
 "traffic light expected, given some other value")

; Any Any -> Boolean
; are the two values elements of TrafficLight and,
; if so, are they equal

(check-expect (light=? "red" "red") #true)
(check-expect (light=? "red" "green") #false)
(check-expect (light=? "green" "green") #true)
(check-expect (light=? "yellow" "yellow") #true)

(define (light=? a-value another-value)
 (if (and (light? a-value) (light? another-value))
 (string=? a-value another-value)
 (error MESSAGE)))
```

Exercise 115. 2つの引数のうちどちらが TrafficLight の要素でないかをエラーメッセージで明示するように、light=? を改訂しなさい。

作成するプログラムで light=? を使う可能性は低いかもしれませんが、前サブセクションの最後で簡単に触れた 2 つの等価述語である key=? と mouse=? は使用すべきです。当然ながら、key=? は 2 つの KeyEvent を比較するための操作であり、同様に mouse=? は 2 つの MouseEvent を比較します。どちらのイベントの種類も文字列として表現されますが、すべての文字列がキーイベントやマウスイベントを表すわけではないことを理解しておくことが重要です。

今後は、キーイベントハンドラでは key=? を、マウスイベントハンドラでは mouse=? を使用することを推奨します。キーイベントハンドラで key=? を使用することで、その関数が任意の文字列ではなく、確実にキーイベントを表す文字列同士を比較するようになります。例えば、誤って関数が “helloworld” に適用されると即座に、key=? はエラーを発生させ、何か異常があることを知らせてくれます。

## 7 まとめ (Summary)

本書のこの第1部では、シンプルでありながら重要な教訓をいくつも学んだ。以下にまとめる：

1. 優れたプログラマーはプログラムを**設計する**。下手なプログラマーは、プログラムが動いているように見えるまでいじくり回す。
2. 設計レシピには2つの側面がある。1つは設計の**プロセス**、すなわち実行すべき一連の手順に関わる。もう1つは、選んだデータの**表現**が設計プロセスにどのように影響するかを説明する。
3. 適切に設計されたプログラムはすべて、多くの定数定義、構造体型定義、データ定義、および関数定義で構成される。バッチプログラムでは、1つの関数が「main」関数となり、通常その計算を行うために他のいくつかの関数を組み合わせる。対話型プログラムでは、big-bang 関数が main 関数の役割を果たす。これはプログラムの初期状態◆◆、画像を生成する出力関数、および高◆◆3つのイベントハンドラ（クロックティック用、マウスクリック用、キーイベント用にそれぞれ1つ）を指定する。どちらの種類プログラムでも、関数定義は「トップダウン」で提示される。つまり main 関数から始まり、main 関数◆◆で言及されている関数が続き、以下同様に続く。
4. すべてのプログラミング言語と同様に、Beginning Student Language には語彙と文法がある。プログラマーは、◆◆力が与えられたときにプログラムがどのように計算を実行するかを予測できるよう、言語◆◆の各文の意味を判断できなければならない。続く Intermezzo では、この考え方を詳細に説明する。
5. BSL を含むプログラミング言語には、プログラマーが毎回車輪の再発明をしなくて済むよう、豊富なティーチパックが付属している。プログラマーは、ティーチパックが提供する関数、特にそのシグネチャと目的記述に◆◆れ親しむべきである。そうすることで作業がシンプルになる。
6. プログラマーは、選んだプログラミング言語が提供する「ツール」を熟知しなければならない。これらのツールは、cond や max のように言語自体の一部であるか、ティーチパックから「インポート」されたもののいずれかである。この精神に則り、以下の用語を確実に理解しているか確認せよ：構造体型定義（structure type definition）、関数定義（function definition）、定数定義（constant definition）、構造体インスタンス（structure instance）、データ定義（data definition）、big-bang、およびイベントハンドラ関数（event-handling function）。

## Intermezzo 1: Beginning Student Language (初級学生言語)

### 目次 (Contents)

- BSL の語彙 (BSL Vocabulary)
- BSL の文法 (BSL Grammar)
- BSL の意味 (BSL Meaning)
- 意味とコンピューティング (Meaning and Computing)
- BSL のエラー (BSL Errors)
- ブール式 (Boolean Expressions)
- 定数定義 (Constant Definitions)
- 構造型定義 (Structure Type Definitions)
- BSL のテスト (BSL Tests)
- BSL のエラーメッセージ (BSL Error Messages)

第 I 部「固定サイズのデータ」は、BSL を自然言語であるかのように扱いました。言語の「基本的な単語」を紹介し、「単語」を「文」に組み立てる方法を示唆し、これらの「文」を直感的に理解するために代数学の知識に訴えかけました。この種の導入はある程度は機能しますが、真に効果的なコミュニケーションには、ある程度の形式的な学習が必要です。

多くの点で、第 I 部のアナロジーは正しいものです。プログラミング言語には語彙と文法があり、プログラマはこれらの要素を **構文 (syntax)** と呼びます。BSL における「文」は、**式 (expression)** または **定義 (definition)** です。BSL の文法は、これらの句をどのように形成するかを規定します。しかし、文法的に正しい文であっても、すべてが意味を持つわけではありません——英語でもプログラミング言語でも同じです。例えば、英語の文「the cat is round」は意味を持ちますが、「the brick is a car」は文法的には完全に正しくても意味がありません。文が意味を持つかどうかを判断するには、言語の **意味 (meaning)** を知る必要があります。プログラマはこれを **意味論 (semantics)** と呼びます。

この Intermezzo では、BSL を中学校で馴染みのある算数・代数の言語の拡張であるかのように提示します。結局のところ、計算はこの単純な数学の形式から始まるのですから、この数学とコンピューティングのつながりを理解すべきです。最初の 3 つの節では、BSL のかなりの部分の構文と意味論を提示します。この新しい BSL の理解に基づき、4 つ目の節ではエラーの議論を再開します。残りの節はこの理解を完全な言語に拡張し、最後の節ではテストを表現するためのツールを拡張します。

**注 (Note):** プログラマは最終的にこれらの計算の原理を理解しなければなりません  
が、それらは設計の原理を補完するものです。

## BSL の語彙 (BSL Vocabulary)

図 39 は、BSL の基本的な語彙を紹介し定義しています。語彙は、数値や真偽値などのリテラル定数、BSL によって意味が与えられている名前（例: `cond` や `+`）、そしてプログラムが `define` や関数のパラメータを通じて意味を与えることができる名前から構成されます。

```
+-----+
| Figure 39: BSL core vocabulary |
| |
| * A primitive is a name to which BSL assigns meaning, for |
| example, + or sqrt. |
| * A variable is a name without preassigned meaning. |
| |
| * A number is one of: 1, -1, 3/5, 1.22, #i1.22, 0+1i, and so on. The syntax |
| for BSL numbers is complicated because it accommodates a range of formats: |
| positive and negative numbers, fractions and decimal numbers, exact and |
| inexact numbers, real and complex numbers, numbers in bases other than 10, and |
| more. Understanding the precise notation for numbers requires a thorough |
| understanding of grammars and parsing, which is out of scope for this |
| intermezzo. |
| * A Boolean is one of: #true or #false. |
| * A string is one of: "", "he says \"hello world\" to you", "doll", and so on. |
| In general, it is a sequence of characters enclosed by a pair of ". |
| * An image is a png, jpg, tiff, and various other formats. We intentionally |
| omit a precise definition. |
+-----+
```

それぞれの説明は、要素の示唆的な列挙によって集合を定義しています。これらの集まりを完全に指定することも可能ですが、ここではそれは余分と考え、直感を信頼します。ただし、これらの集合のそれぞれに追加の要素が含まれる場合があることだけは念頭に置いてください。

## BSL の文法 (BSL Grammar)

図 40 は、BSL 文法の大部分を示しています。他の言語と比べて極めて単純です。BSL の表現力については見た目に惑わされないでください。ただし最初の課題は、このような文法の読み方を議論することです。= のある各行は統語範疇を導入します。= は「～のいずれかである」、| は「または」と読むのがよいです。三点リーダ ... があるところでは、その直前のものを好きな回数だけ繰り返すと想像してください。これは例えば、program が何もないか、def-expr が 1 回、または 2 回・3 回・4 回・5 回……といくらでも続く列であることを意味します。この例は特に明快ではないので、2 つ目の統語範疇を見てみましょう。それは def が次のいずれかであることを述べています。

**注 (Note):** 文法を声に出して読むと、データ定義のように聞こえます。実際、多くのデータ定義を文法で書き下ろすこともできます。

(define (variable variable) expr)

なぜなら「好きな回数」にはゼロも含まれるからです。あるいは

(define (variable variable variable) expr)

これは 1 回の繰り返しです。あるいは

(define (variable variable variable variable) expr)

これは 2 回使っています。

+-----+   Figure 40: BSL core grammar 				
+-----+-----+-----+-----+   program   =   def-expr...				
+-----+-----+-----+-----+   def-expr   =   def         expr   def   =   (define (variable variable variable...     expr   =   variable         value         (primitive expr expr...)         (variable expr expr...)         (cond [expr expr]... [expr expr])         (cond [expr expr]... [else expr])   +-----+-----+-----+-----+ +-----+				

文法についての最後のポイントは、特別なフォントで書かれた 3 つの「語」——define、cond、else——に関するものです。BSL 語彙の定義によれば、これらの 3 語は名前です。語彙定義が教えてくれないのは、これらの名前があらかじめ定められた意味を持つことです。

BSL では、これらの語はいくつかの複合文を他と区別するマーカーとして働き、その役割を認めて **キーワード (keywords)** と呼ばれます。

これで文法の目的を述べる準備ができました。プログラミング言語の文法は、その語彙から文をどのように形成するかを規定します。ある文は単に語彙の要素です。例えば、図 40 によれば 42 は BSL の文です：

- 最初の統語範疇は、プログラムが `def-expr` であることを言います。式は定義を参照できます。
- 2 つ目は、`def-expr` が `def` または `expr` であることを教えます。> **注 (Note):** DrRacket では、プログラムは実際には 2 つの異なる部分——定義領域と、対話領域の式——から成ります。
- 最後の定義は `expr` を形成するすべての方法を列挙し、その 2 つ目が `value` です。

**注 (Note):** DrRacket では、プログラムは実際には 2 つの異なる部分——定義領域と、対話領域の式——から成ります。

図 39 から 42 が値であることがわかるので、確認が取れます。

文法の興味深い部分は、他の文から組み立てられる複合文の作り方を示します。例えば、`def` の部分は、関数定義が「(」に続いてキーワード `define`、さらに「(」、少なくとも 2 つの変数の列、「)」」、`expr`、そして最初のものに対応する閉じの「)」で形成されることを教えています。先頭のキーワード `define` が定義を式から区別することに注意してください。

式 (`expr`) には 6 つの種類があります：変数、定数、プリミティブ適用、(関数)適用、および 2 種類の条件式です。最初の 2 つは原子的な文で、残りの 4 つは複合文です。`define` と同様に、キーワード `cond` が条件式を適用から区別します。

式の例を 3 つ挙げます：`"all"`、`x`、そして `(f x)`。最初のは文字列のクラスに属し、したがって式です。2 つ目は変数であり、すべての変数は式です。3 つ目は関数適用です。なぜなら `f` と `x` が変数だからです。

対照的に、次の括弧付きの文は合法的式ではありません：`(f define)`、`(cond x)`、`((f 2) 10)`。最初のは関数適用の形に部分的に一致しますが、`define` を変数であるかのように使っています。2 つ目は正しい `cond` 式になることに失敗します。なぜなら 2 番目の要素が変数であり、括弧で囲まれた式の対ではないからです。最後のものは条件式でも適用でもありません。最初の部分が式だからです。

最後に、文法が空白（スペース、タブ、改行）に言及していないことに気づくかもしれません。BSL は寛容な言語です。プログラム内の任意の列の要素の間に何らかの空白があれば、DrRacket はあなたの BSL プログラムを理解できます。しかし、優れたプログラムはあなたが書くものを好まないかもしれません。これらのプログラムは空白を使ってプログラムを読みやすくします。最も重要なのは、プログラムを処理するソフトウェア（DrRacket など）より人間の読者を優先するスタイルを採用することです。彼らは本

のコード例を注意深く読み、どのように整形されているかに注意を払うことでこのスタイルを身につけます。

**注 (Note):** あなたの BSL プログラムを調べる読者は 2 種類いることを念頭に置いてください：人と DrRacket です。

**練習問題 116 (Exercise 116).** 次の文を見てください：

1. `x`
2. `(= y z)`
3. `(= (= y z) 0)`

これらが統語的に合法的な式である理由を説明せよ。

**練習問題 117 (Exercise 117).** 次の文を考えよ：

1. `(3 + 4)`
2. `number?`
3. `(x)`

これらが統語的に非合法である理由を説明せよ。

**練習問題 118 (Exercise 118).** 次の文を見てください：

1. `(define (f x) x)`
2. `(define (f x) y)`
3. `(define (f x y) 3)`

これらが統語的に合法的な定義である理由を説明せよ。

**練習問題 119 (Exercise 119).** 次の文を考えよ：

1. `(define (f "x") x)`
2. `(define (f x y z) (x))`

これらが統語的に非合法である理由を説明せよ。

**練習問題 120 (Exercise 120).** 合法的な文と非合法的な文を区別せよ：

1. `(x)`
2. `(+ 1 (not x))`
3. `(+ 1 2 3)`

なぜ合法または非合法かを説明せよ。合法的なものが範疇 `expr` と `def` のどちらに属するかを判定せよ。

**文法用語についての注 (Note on Grammatical Terminology)** 複合文の構成要素には名前があります。これらの名前のいくつかは非形式的に導入してきました。図 41 は慣習の要約を示します。

+-----+	
	Figure 41: Syntactic naming conventions
	; function application:
	(function argument... argument)
	; function definition:
	(define (function-name parameter... parameter)
	function-body)
	; conditional expression:
	(cond
	cond-clause
	...
	cond-clause)
	; cond clause
	[condition answer]
+-----+	

図 41 の用語に加えて、定義の第 2 成分を **関数ヘッダ (function header)** と呼びます。それに応じて、式の成分は **関数本体 (function body)** と呼ばれます。プログラミング言語を数学の一形態と考える人々は、ヘッダを **左辺 (left-hand side)**、本体を **右辺 (right-hand side)** と呼びます。時折、関数適用における引数を **実引数 (actual arguments)** と呼ぶこともあります。以上。



## BSL の意味 (BSL Meaning)

キーボードのリターンキーを押して DrRacket に式の評価を依頼すると、DrRacket は算数と代数の法則を使って値を求めます。これまでに扱った BSL の変種については、図 39 が文法的に値とは何かを定義しています——値の集合は、すべての式の部分集合にすぎません。この集合には真偽値 (Boolean)、文字列 (String)、画像 (Image) が含まれます。

評価の規則は 2 つのカテゴリに分かれます。無限個の規則——算術の規則のようなもの——が、プリミティブ操作を値に適用した結果の値の求め方を説明します：

```
(+ 1 1) = 2
(- 2 1) = 1
...
```

`=` は、BSL の計算法則に従って 2 つの式が等しいことを表すことを思い出してください。しかし BSL の算術は単なる数値の処理より一般的です。真偽値や文字列などを扱う規則も含まれます：

```
(not #true) == #false
(string=? "a" "a") == #true
...
```

そして代数と同様に、等しいものは常に等しいもので置き換えられます。図 42 に計算の例があります。

```
+-----+
| Figure 42: Replacing equals by equals |
| (boolean? (= (string-length (string-append "h" "w")) |
| (+ 1 3))) |
| == |
| (boolean? (= (string-length (string-append "h" "w")) 4)) |
| == |
| (boolean? (= (string-length "hw") 4)) |
| == |
| (boolean? (= 2 4)) |
| == |
| (boolean? #false) |
| == #true |
+-----+
```

第二に、関数を引数に適用することを理解するために、代数からの規則が必要です。プログラムに次の定義が含まれているとします：

```
(define (f x-1... x-n)
 f-body)
```

すると、関数適用は次の法則に支配されます：

```
(f v-1... v-n) == f-body
; with all occurrences of x-1... x-n
; replaced with v-1... v-n, respectively
```

BSL のような言語の歴史のため、この規則を **ベータ (beta)** または **ベータ値 (beta-value)** 規則と呼びます。

**注 (Note):** この規則の詳細は `lambda` を使った計算 (Computing with lambda) を参照してください。

この規則はできるだけ一般的に定式化されているので、具体例を見るのが最善です。定義が

```
(define (poly x y)
 (+ (expt 2 x) y))
```

であり、DrRacket に式 `(poly 3 5)` が与えられたとします。すると式の評価の最初のステップはベータ規則を使います：

```
(poly 3 5) == (+ (expt 2 3) 5)... == (+ 8 5) == 13
```

ベータに加えて、`cond` 式の値を決める規則も必要です。これらの規則は代数的ですが、標準カリキュラムの一部として明示的に教えられるとは限りません。最初の条件が `#false` のとき、最初の `cond` 行は消え、残りの行はそのまま残ります：

```
(cond
 [#false...] ; first line removed
 [condition2 answer2] [condition2 answer2]
 ...)...
```

この規則の名前は **condfalse** です。こちらが **condtrue** です：

```
(cond
 [#true answer-1]
 [condition2 answer2]
 ...)
```

最初の条件が `else` のときにもこの規則は適用されます。

次の評価を考えてください：

```
(cond
 [(zero? 3) 1]
 [(= 3 3) (+ 1 1)]
 [else 3])
==; by plain arithmetic and equals-for-equals
(cond
 [#false 1]
 [(= 3 3) (+ 1 1)]
 [else 3])
==; by rule condfalse
```

```
(cond
 [(= 3 3) (+ 1 1)]
 [else 3])
```

==; by plain arithmetic and equals-for-equals

```
(cond
 [#true (+ 1 1)]
 [else 3])
```

==; by rule condtrue

```
(+ 1 1)
```

この計算は、通常の算術の規則、等しいものの置き換え、および両方の cond 規則を示しています。

**練習問題 121 (Exercise 121).** 次の式をステップごとに評価せよ：

1.  $(+ (* (/ 12 8) 2/3) (- 20 (\text{sqrt } 4)))$
2.  $(\text{cond } [(= 0 0) \text{\#false}] [(> 0 1) (\text{string=? } "a" "a")] [\text{else } (= (/ 1 0) 9)])$
3.  $(\text{cond } [(= 2 0) \text{\#false}] [(> 2 1) (\text{string=? } "a" "a")] [\text{else } (= (/ 1 2) 9)])$

DrRacket のステッパで計算を確認せよ。

**練習問題 122 (Exercise 122).** プログラムに次の定義が含まれているとする：

```
(define (f x y)
 (+ (* 3 x) (* y y)))
```

次の式を DrRacket がどのようにステップごとに評価するかを示せ：

1.  $(+ (f 1 2) (f 2 1))$
2.  $(f 1 (* 2 3))$
3.  $(f (f 1 (* 2 3)) 19)$

DrRacket のステッパで計算を確認せよ。



## BSL のエラー (BSL Errors)

DrRacket が、ある括弧で囲まれた句が BSL に属しないと発見すると、**構文エラー (syntax error)** を通知します。完全に括弧付けされたプログラムが統語的に合法かどうかを判定するために、DrRacket は図 40 の文法を使い、上で説明した方針に沿って推論します。しかし、統語的に合法的なプログラムのすべてが意味を持つわけではありません。

**注 (Note):** エラーメッセージのほぼ完全な一覧は、この Intermezzo の最後の節を参照してください。

DrRacket が統語的に合法的なプログラムを評価し、ある操作が誤った種類の値に使われていることを発見すると、**実行時エラー (run-time error)** を上げます。統語的に合法的な式  $(/ \ 1 \ 0)$  を考えてください。これは数学からわかるように値を持ちません。BSL の計算は数学と一貫していなければならないので、DrRacket はエラーを通知します：

```
> (/ 1 0)
/:division by zero
```

もちろん、 $(/ \ 1 \ 0)$  のような式が別の式の奥深くに入れ子になっているときにもエラーを通知します：

```
> (+ (* 20 2) (/ 1 (- 10 10)))
/:division by zero
```

DrRacket の振る舞いは、私たちの計算では次のように翻訳されます。値ではない式を見つけ、評価規則がこれ以上の簡約を許さないとき、計算は **行き詰まった (stuck)** と言います。この行き詰まりの概念が実行時エラーに対応します。例えば、上の式の値を計算すると行き詰まり状態に至ります：

```
(+ (* 20 2) (/ 1 (- 10 10)))
==
(+ (* 20 2) (/ 1 0))
==
(+ 40 (/ 1 0))
```

この計算が示すのは、DrRacket がエラーを通知するときに行き詰まった式の文脈を取り除くということです。この具体例では、行き詰まった式  $(/ \ 1 \ 0)$  への 40 の加算を取り除きます。

入れ子になった行き詰まり式のすべてがエラー通知に終わるわけではありません。プログラムに次の定義が含まれているとします：

```
(define (my-divide n)
 (cond
 [(= n 0) "inf"]
 [else (/ 1 n)]))
```

ここで my-divide を 0 に適用すると、DrRacket は次のように計算します：

```
(my-divide 0)
==
(cond
 [(= 0 0) "inf"]
 [else (/ 1 0)])
```

網掛けの部分式の評価がそれを示唆するとしても、関数がいまゼロ除算エラーを通知するというのは明らかに誤りです。理由は `(= 0 0)` が `#true` に評価され、したがって 2 番目の `cond` 節は何の役割も果たさないからです：

```
(my-divide 0)
==
(cond
 [(= 0 0) "inf"]
 [else (/ 1 0)])
==
(cond
 [#true "inf"]
 [else (/ 1 0)])
== "inf"
```

幸い、私たちの評価法則はこれらの状況を自動的に処理します。法則がいつ適用されるかを覚えておくだけでよいのです。例えば、

```
(+ (* 20 2) (/ 20 2))
```

では、乗算や除算の前に加算は行えません。同様に、

```
(cond
 [(= 0 0) "inf"]
 [else (/ 1 0)])
```

の網掛けの除算は、対応する行が `cond` の最初の条件になるまで、完全な `cond` 式の代わりにはなりません。

経験則として、次を心に留めておくのが最善です：

評価の準備ができた、最も外側で最も左の入れ子式を常に選べ。

この指針は単純に見えるかもしれませんが、BSL の結果を常に説明します。

場合によっては、プログラマもエラーを上げる関数を定義したいことがあります。入力エラー (Input Errors) からのチェック付きバージョン `area-of-disk` を思い出してください：

```
(define (checked-area-of-disk v)
 (cond
 [(number? v) (area-of-disk v)]
 [else (error "number expected")]))
```

ここで `checked-area-of-disk` を文字列に適用することを想像してください：

```

(- (checked-area-of-disk "a")
 (checked-area-of-disk 10))
==
(- (cond
 [(number? "a") (area-of-disk "a")]
 [else (error "number expected")])
 (checked-area-of-disk 10))
==
(- (cond
 [#false (area-of-disk "a")]
 [else (error "number expected")])
 (checked-area-of-disk 10))
==
(- (error "number expected")
 (checked-area-of-disk 10))

```

この時点で2番目の式を評価しようと試みるかもしれませんが、その結果がおよそ314だとわかったとしても、計算は最終的に error 式を扱わねばならず、それは行き詰まり式と同じです。要するに、計算は次で終わります：

```

(error "number expected")

```

## ブール式 (Boolean Expressions)

現在の BSL の定義は or と and 式を省略しています。これらを追加することは、新しい言語構成子をどう学ぶかのケーススタディになります。まず構文を理解し、次に意味論を理解しなければなりません。

こちらが改訂された式の文法です：

```
+-----+---+---+---+-----+
| expr | | = | | ... |
+-----+---+---+---+-----+
| | | | | | (and expr expr) |
| | | | | | (or expr expr) |
+-----+---+---+---+-----+
```

この文法は、and と or がキーワードであり、それぞれに 2 つの式が続くことを述べています。それらは関数適用ではありません。

なぜ and と or が BSL で定義された関数ではないのかを理解するには、まずその語用論 (pragmatics) を見なければなりません。(/ 1 n) が r であるかどうかを判定する条件を定式化する必要があるとします：

```
(define (check n r)
 (and (not (= n 0)) (= (/ 1 n) r)))
```

条件を and の組み合わせとして定式化します。誤って 0 で割らないためです。ここで check を 0 と 1/5 に適用しましょう：

```
(check 0 1/5)
=> (and (not (= 0 0)) (= (/ 1 0) 1/5))
```

もし and が普通の操作であれば、両方の部分式を評価しなければならず、そうするとエラーが引き起こされます。代わりに、and は最初の式が #false のとき単に 2 番目の式を評価しません。つまり、and は評価を **短絡 (short-circuits)** します。

and と or の評価規則を定式化するのは簡単でしょう。意味を説明する別の方法は、他の式へ翻訳することです：

**注 (Note):** expr-2 が真偽値に評価されることを確実にするため、これらの省略形は単に expr-2 の代わりに (if expr-2 #true #false) を使うべきです。ここではこの詳細を省略します。

(and expr-1 expr-2) は (cond [expr-1 expr-2] [else #false]) の省略です

そして

(or expr-1 expr-2) は (cond [expr-1 #true] [else expr-2]) の省略です



したがって、and や or 式の評価の仕方に疑いがあるときは、上の同値を使って計算してください。しかし、これらの操作を直感的に理解していると信じており、それでほぼ常に十分です。

**練習問題 123 (Exercise 123).** if の使用は別の意味で驚かせたかもしれません。なぜならこの Intermezzo は他の場所でこの形に言及していないからです。要するに、Intermezzo は説明のない形で and を説明しているように見えます。この時点では、if を cond の省略形として直感的に理解していることに頼っています。次を cond 式に書き換えられることを示す規則を書きなさい。

(if expr-test expr-then expr-else)

## 定数定義 (Constant Definitions)

プログラムは関数定義だけでなく定数定義からも成りますが、これらは最初の文法には含まれていませんでした。そこで定数定義を含む拡張文法を示します：

```
definition = ... | (define name expr)
```

定数定義の形は関数定義の形に似ています。キーワード `define` は定数定義を式から区別しますが、関数定義との区別はしません。そのためには、人間の読者が定義の第2成分を見なければなりません。

**注 (Note):** 実のところ、DrRacket には関数定義を扱う別の方法もあります。無名関数 (Nameless Functions) を参照してください。

次に、定数定義が何を意味するかを理解しなければなりません。右辺がリテラル定数である定数定義、例えば

```
(define RADIUS 5)
```

では、変数はその値の単なる省略形です。評価中に DrRacket が `RADIUS` に会うところではどこでも、それを 5 で置き換えます。

右辺がきちんとした式である定義、例えば

```
(define DIAMETER (* 2 RADIUS))
```

では、直ちにその式の値を求めなければなりません。この過程は、この定数定義より前にある定義をすべて使います。したがって、

```
(define RADIUS 5)
(define DIAMETER (* 2 RADIUS))
```

は次と等価です：

```
(define RADIUS 5)
(define DIAMETER 10)
```

この過程は関数定義が関与するときにも働きます：

```
(define RADIUS 10)
(define DIAMETER (* 2 RADIUS))
(define (area r) (* 3.14 (* r r)))
(define AREA-OF-RADIUS (area RADIUS))
```

DrRacket がこの定義の列をステップ実行するとき、まず `RADIUS` が 10 を、`DIAMETER` が 20 を表し、`area` が関数の名前であることを判定します。最後に `(area RADIUS)` を 314 に評価し、`AREA-OF-RADIUS` をその値と関連付けます。

定数定義と関数定義を混ぜると、新しい種類の実行時エラーも生じます。次のプログラムを見てください：

```
(define RADIUS 10)
(define DIAMETER (* 2 RADIUS))
(define AREA-OF-RADIUS (area RADIUS))
(define (area r) (* 3.14 (* r r)))
```

これは上のものと最後の2つの定義が入れ替わっている点を除いて同じです。最初の2つの定義については、評価は前と同様に進みます。しかし3つ目については評価がうまくいきません。過程は (area RADIUS) の評価を要求します。RADIUS の定義はこの式より前にありますが、area の定義にはまだ出会っていません。このプログラムを DrRacket で評価すると、「this function is not defined」（この関数は定義されていない）と説明するエラーが出ます。したがって、定数定義の中で関数を使うのは、それらが定義されているとわかっているときに限るよう注意してください。

**練習問題 124 (Exercise 124).** 次のプログラムをステップごとに評価せよ：

```
(define PRICE 5)
(define SALES-TAX (* 0.08 PRICE))
(define TOTAL (+ PRICE SALES-TAX))
```

次のプログラムの評価はエラーを通知するか？

```
(define COLD-F 32)
(define COLD-C (fahrenheit->celsius COLD-F))
(define (fahrenheit->celsius f)
 (* 5/9 (- f 32)))
```

次のはどうか？

```
(define LEFT -100)
(define RIGHT 100)
(define (f x) (+ (* 5 (expt x 2)) 10))
(define f@LEFT (f LEFT))
(define f@RIGHT (f RIGHT))
```

DrRacket のステッパーで計算を確認せよ。

## 構造型定義 (Structure Type Definitions)

想像できるように、define-struct は最も複雑な BSL 構成子です。そのため説明を最後に残しました。文法は次のとおりです：

```
definition = ... | (define-struct name [name ...])
```

構造型定義は定義の第 3 の形です。キーワードが関数定義とも定数定義とも区別します。簡単な例です：

```
(define-struct point [x y z])
```

point、x、y、z は変数であり、括弧は文法パターンに従って置かれているので、これは構造型の適切な定義です。対照的に、次の 2 つの括弧付き文

```
(define-struct [point x y z])
(define-struct point x y z)
```

は非合法的な定義です。なぜなら define-struct の後に単一の変数名と、括弧内の変数の列が続いていないからです。

define-struct の構文は素直ですが、その意味を評価規則で書き下ろすのは難しいです。何度か述べたように、define-struct 定義は一度にいくつかの関数を定義します：コンストラクタ、いくつかのセレクタ、および述語です。したがって

```
(define-struct c [s-1... s-n])
```

の評価は、プログラムに次の関数を導入します：

1. make-c：コンストラクタ；
2. c-s-1 ... c-s-n：一連のセレクタ；および
3. c?：述語。

これらの関数は +、-、\* と同じ地位を持ちます。しかしこれらの新しい関数を支配する規則を理解する前に、値の定義に戻らなければなりません。結局のところ、define-struct の目的の 1 つは、既存のすべての値と区別される値のクラスを導入することだからです。

端的に言えば、define-struct の使用は値の宇宙を拡張します。まず、それは複数の値を 1 つに複合する構造体も含むようになります。プログラムに define-struct 定義が含まれると、その評価は値の定義を次のように変更します：

値とは次のいずれかである：数、真偽値、文字列、画像、または構造体値：(make-c \_value-1 ... \_value-n) (構造型 c が定義されていると仮定)

例えば、point の定義は次の形の値を追加します：

```
(make-point 1 2 -1)
(make-point "one" "hello" "world")
```

```
(make-point 1 "one" (make-point 1 2 -1))
...
```

これで新しい関数の評価規則を理解する立場になりました。c-s-1 が c 構造体に適用されると、値の第 1 成分を返します。同様に、第 2 セレクタは第 2 成分を取り出し、第 3 セレクタは第 3 成分を、といった具合です。新しいデータコンストラクタとセレクタの関係は、BSL の規則に追加される n 個の等式で最もよく特徴づけられます：

```
(c-s-1 (make-c V-1... V-n)) == V-1
(c-s-n (make-c V-1... V-n)) == V-n
```

実行中の例では、次の具体的な等式が得られます：

```
(point-x (make-point V U W)) == V
(point-y (make-point V U W)) == U
(point-z (make-point V U W)) == W
```

DrRacket が (point-y (make-point 3 4 5)) を見ると、式を 4 で置き換えます。一方 (point-x (make-point (make-point 1 2 3) 4 5)) は (make-point 1 2 3) に評価されます。

述語 c? は任意の値に適用できます。値が種類 c なら #true を、そうでなければ #false を返します。両方の部分を 2 つの等式に翻訳できます：

```
(c? (make-c V-1... V-n)) == #true
(c? V) == #false
```

ただし V は make-c で構築されていない値です。再び、等式は例で理解するのが最善です：

```
(point? (make-point U V W)) == #true
(point? X) == #false
```

ただし X は値だが point 構造体ではない場合です。

**練習問題 125 (Exercise 125).** 合法的な文と非合法的な文を区別せよ：

1. (define-struct oops [])
2. (define-struct child [parents dob date])
3. (define-struct (child person) [dob date])

なぜ合法または非合法かを説明せよ。

**練習問題 126 (Exercise 126).** 定義領域に次の構造型定義があるとして、次の式のうち値であるものを特定せよ：

```
(define-struct point [x y z])
(define-struct none [])
```

1. (make-point 1 2 3)
2. (make-point (make-point 1 2 3) 4 5)
3. (make-point (+ 1 2) 3 4)
4. (make-none)
5. (make-point (point-x (make-point 1 2 3)) 4 5)

なぜ値である／でないかを説明せよ。

**練習問題 127 (Exercise 127).** プログラムに次が含まれるとする：

```
(define-struct ball [x y speed-x speed-y])
```

次の式を評価した結果を予測せよ：

1. (number? (make-ball 1 2 3 4))
2. (ball-speed-y (make-ball (+ 1 2) (+ 3 3) 2 3))
3. (ball-y (make-ball (+ 1 2) (+ 3 3) 2 3))
4. (ball-x (make-posn 1 2))
5. (ball-speed-y 5)

対話領域とステッパーで予測を確認せよ。

Figure 43: BSL, full grammar			
def-expr	=	definition	
definition	=	expr	
		test-case	
		(define (name variable variable..... (define name expr) (define-struct name [name...])	
expr	=	(name expr expr...)	
		(cond [expr expr]... [expr expr])	
		(cond [expr expr]... [else expr])	
		(and expr expr expr...)	
		(or expr expr expr...)	
		name	
test-case	=	number	
		string	
		image	
		(check-expect expr expr)	
		(check-within expr expr expr)	
		(check-member-of expr expr...)	
		(check-range expr expr expr)	
		(check-error expr)	
		(check-random expr expr)	
		(check-satisfied expr name)	

## BSL のテスト (BSL Tests)

図 43 は BSL 全体に加え、いくつかのテスト形式を示しています。

テスト式の一般的な意味は説明しやすいです。RUN ボタンをクリックすると、DrRacket はすべてのテスト式を集め、現れた順序を保ったままプログラムの末尾へ移します。その後、定義領域の内容を評価します。各テストはその各片を評価し、何らかの述語を介して期待される結果と比較します。それに加えて、テストは統計とテスト失敗の表示方法に関する情報を集めるために DrRacket と通信します。

詳細はこれらのテスト形式のドキュメントを読んでください。以下は説明的な例です：

```
; check-expect compares the outcome and the expected value with equal?
(check-expect 3 3)

; check-member-of compares the outcome and the expected values with equal?
; if one of them yields #true, the test succeeds
(check-member-of "green" "red" "yellow" "green")

; check-within compares the outcome and the expected value with a predicate
; like equal? but allows for a tolerance of epsilon for each inexact number
(check-within (make-posn #i1.0 #i1.1) (make-posn #i0.9 #i1.2) 0.2)

; check-range is like check-within
; but allows for a specification of an interval
(check-range 0.9 #i0.6 #i1.0)

; check-error checks whether an expression signals (any) error
(check-error (/ 1 0))

; check-random evaluates the sequences of calls to random in the
; two expressions such that they yield the same number
(check-random (make-posn (random 3) (random 9))
 (make-posn (random 3) (random 9)))

; check-satisfied determines whether a predicate produces #true
; when applied to the outcome, that is, whether outcome has a certain property
(check-satisfied 4 even?)
```

上のテストはすべて成功します。本書の残りの部分は、必要に応じてこれらのテスト形式を再導入します。

**練習問題 128 (Exercise 128).** 次のテストを DrRacket の定義領域にコピーせよ：

```
(check-member-of "green" "red" "yellow" "grey")
(check-within (make-posn #i1.0 #i1.1)
 (make-posn #i0.9 #i1.2) 0.01)
(check-range #i0.9 #i0.6 #i0.8)
(check-random (make-posn (random 3) (random 9))
```

```
(make-posn (random 9) (random 3)))
(check-satisfied 4 odd?)
```

それらがすべて失敗することを検証し、なぜかを説明せよ。



## BSL のエラーメッセージ (BSL Error Messages)

BSL プログラムは多くの種類の構文エラーを通知することがあります。BSL とそのエラー報告は、誤りを犯すのが当たり前の初心者向けに作られています。メッセージ自体には慣れが必要です。

以下では出会うかもしれないエラーメッセージの種類をいくつか示します。各一覧の各項目は 3 つの部分から成ります：

- エラーメッセージを通知するコード片；
- エラーメッセージ；および
- 誤りを直す提案を伴う説明。

次の例を考えてください。これはおそらくこれまでに見る最悪のエラーメッセージです：

```
+-----+
+-----+
| (define (absolute n) ↵ (cond ↵ [< 0 (- n)] ↵ [e... | A cond expression consists of the
keyword follo... |
+-----+
+-----+
| (define (absolute n) ↵ (cond ↵ [< 0 (- n)] ↵ [e... |
|
| (define (absolute n) |
| (cond |
| [< 0 (- n)] |
| [else n])) |
|
| <: expected a function call, but there is no op... |
|
+-----+
+-----+
```

関数定義内の < の強調表示がエラーを指し示しています。定義の下に、RUN をクリックしたときに DrRacket が対話ウィンドウに提示するエラーメッセージが見えます。右のエラー説明を調べて、このやや自己矛盾したメッセージへの対処を理解してください。そして、他のどのエラーメッセージもこれほど不透明でないことを安心してよいです。

したがって、エラーが出て助けが必要なときは、適切な図を見つけ、一致する項目を探し、その完全な項目を研究してください。

BSL における関数適用に関するエラーメッセージ (Error Messages about Function Applications in BSL)

定義領域に次だけが含まれていると仮定します：

```

; Number Number -> Number
; finds the average of x and y
(define (average x y)
 (/ (+ x y)
 2))

```

RUN ボタンを押します。すると以下のエラーメッセージに出会うかもしれません。

```

+-----+
| (f 1)f: this function is not defined | The application names f as the function, ↵ and ... |
+-----+
| (f 1) |
| `f: this function is not defined` |
+-----+

+-----+
+-----+
| (1 3 "three" #true)function call: expected a fu... | An open parenthesis must always be
followed by ... |
+-----+
+-----+
| (1 3 "three" #true) |
| |
| function call: expected a function after the ↵ ... |
| |
+-----+
+-----+

+-----+
+-----+
| (average 7)average: expects 2 arguments, but fo... | This function call applies average to ↵
one arg... |
+-----+
+-----+
| (average 7) |
| |
| `average: expects 2 arguments, but found only 1` |
| |
+-----+
+-----+

+-----+
+-----+
| (average 1 2 3)average: expects 2 arguments, bu... | Here average is applied to three numbers
instea... |
+-----+
+-----+
| (average 1 2 3) |
| |
| `average: expects 2 arguments, but found 3` |
| |
+-----+

```

```

+-----+
+-----+
+-----+
+-----+
| (make-posn 1)make-posn: expects 2 arguments, bu... | Functions defined by BSL must also be
applied t... |
+-----+
+-----+
| (make-posn 1) |
| |
| `make-posn: expects 2 arguments, but found only... |
| |
+-----+
+-----+

```

## BSL における誤ったデータに関するエラーメッセージ (Error Messages about Wrong Data in BSL)

以下のエラーシナリオでも、定義領域に次が含まれていると仮定します：

```

; Number Number -> Number
; find the average of x and y
(define (average x y)...)

```

posn はあらかじめ定義された構造型であることを思い出してください。

```

+-----+
+-----+
| (posn-x #true)posn-x: expects a posn, given #tr... | A function must be applied to the
arguments it ... |
+-----+
+-----+
| (posn-x #true) |
| |
| `posn-x: expects a posn, given #true` |
| |
+-----+
+-----+
+-----+
+-----+
| (average "one" "two")+: expects a number as 1st... | A function defined to consume two ↩
Numbers mus... |
+-----+
+-----+
| (average "one" "two") |
| |
| `+: expects a number as 1st argument, given "on... |
| |
+-----+
+-----+

```

## BSL における条件式に関するエラーメッセージ (Error Messages about Conditionals in BSL)

今度は定義領域に定数定義があることを想定します：

```
+-----+
| ; N in [0,1,...10) ↵ (define 0-to-9 (random 10)) |
+-----+
| ;N in [0,1,...10) |
| (define 0-to-9 (random 10)) |
+-----+

+-----+
+-----+
| (cond ↵ [(>= 0-to-9 5)]) | Every cond clause must consist of exactly
two p... |
+-----+

+-----+
| (cond ↵ [(>= 0-to-9 5)]) |
| (cond |
| [(>= 0-to-9 5)]) |
| cond: expected a clause with a question and an ... |
|
+-----+
+-----+

+-----+
+-----+
| (cond ↵ [(>= 0-to-9 5) ↵ "head" ↵ "tail"]) | In this case, the cond clause consists
of three... |
+-----+

+-----+
| (cond ↵ [(>= 0-to-9 5) ↵ "head" ↵ "tail"]) |
| (cond |
| [(>= 0-to-9 5) |
| "head" |
| "tail"]) |
| cond: expected a clause with a question and an ... |
|
+-----+
+-----+
```

```

+-----+
+-----+
| (cond)cond: expected a clause after cond, but n... | A conditional must come with at least one
cond ... |
+-----+
+-----+
| (cond) |
| |
| `cond: expected a clause after cond, but nothin... |
| |
+-----+
+-----+

```

## BSL における関数定義に関するエラーメッセージ (Error Messages about Function Definitions in BSL)

以下のエラーシナリオはすべて、コード片を定義領域に置き RUN を押したと仮定します。

```

+-----+
+-----+
| (define f(x) x)define: expected only one expres... | A definition consist of three parts: the
↵ defi... |
+-----+
+-----+
| (define f(x) x) |
| |
| define: expected only one expression after the ... |
| |
+-----+
+-----+
+-----+
+-----+
| (define (f x x) x)define: found a variable that... | The sequence of parameters in a function
↵ defi... |
+-----+
+-----+
| (define (f x x) x) |
| |
| `define: found a variable that is used more tha... |
| |
+-----+
+-----+
+-----+
+-----+
| (define (g) x)define: expected at least one var... | In BSL a function header must contain at
↵ leas... |
+-----+
+-----+
| (define (g) x) |
| |

```

```

|
| define: expected at least one variable after ↵ ... |
|
+-----+
+-----+
+-----+
+-----+
| (define (f (x)) x)define: expected a variable, ... | The function header contains ↵ (x),
which is no... |
+-----+
+-----+
| (define (f (x)) x) |
| `define: expected a variable, but found a part` |
|
+-----+
+-----+
+-----+
+-----+
| (define (h x y) x y)define: expected only one e... | This function definition comes with two
↵ expre... |
+-----+
+-----+
| (define (h x y) x y) |
|
| define: expected only one expression for the fu... |
|
+-----+
+-----+

```

## BSL における構造型定義に関するエラーメッセージ (Error Messages about Structure Type Definitions in BSL)

今度は構造型定義を定義領域に置き、RUN を押して次のエラーを試す必要があります。

```

+-----+
+-----+
| (define-struct [x]) ↵ (define-struct [x y]) | A structure type definition consists of
three ↵... |
+-----+
+-----+
| (define-struct [x]) ↵ (define-struct [x y]) |
|
| (define-struct [x]) |
|
| (define-struct [x y]) |
|
| define-struct: expected the structure name afte... |
|

```

```

+-----+
+-----+
+-----+
+-----+
| (define-struct x ↵ [y y]) | The sequence of field names in a structure
↵ ty... |
+-----+
+-----+
| (define-struct x ↵ [y y]) |
|
| (define-struct x |
| [y y]) |
|
| `define-struct: found a field name that is used... |
|
+-----+
+-----+
+-----+
+-----+
| (define-struct x y) ↵ (define-struct x y z) | These structure type definitions lack the
↵ seq... |
+-----+
+-----+
| (define-struct x y) ↵ (define-struct x y z) |
|
| (define-struct x y) |
|
| (define-struct x y z) |
|
| define-struct: expected at least one field name... |
|
+-----+
+-----+

```

---

**免責:** 本ファイルは HtDP/2e の個人学習用 AI 意識であり、公式翻訳ではありません。コードフェンス内は英語原本と一致させています。

## II 任意サイズのデータ (Arbitrarily Large Data)



## 8 リスト (Lists)

おそらくこれまでに、自己参照による定義 (self-referential definitions) に出会ったことはないでしょう。英語の先生は間違いなくこうした定義を避けますし、多くの数学の授業でも、このような定義に関しては曖昧な説明にとどまります。プログラマーが曖昧さを受け入れるわけにはいきません。プログラマーの仕事には精密さが求められるからです。一般的に、定義は自身への参照を複数含むこともあります。本章では、リストに関する定義を切り出しとして、参照が 1 つだけで足りる有用な例を紹介します。

リストの導入は、私たちが取り扱うことのできるアプリケーションの種類をいっそう刺激的なものにします。本章では、例を通じてあなたの直感を丁寧に養うとともに、自己参照型のデータ定義を扱う関数を体系的に作成する方法を説明する次章でのデザインレシピの改訂へと橋渡しを行います。

### 8.1 リストの作成 (Creating Lists)

私たちは誰しも、日常的にリストを作成しています。食料品の買い物に行く前には、購入したい品物のリストを書き出します。毎朝 ToDo リストを書く人もいます。12 月になると、多くの子供たちがクリスマスに欲しいもののリストを準備します。パーティーを計画する際には、招待客のリストを作成します。このように情報をリストの形で整理することは、私たちの生活において至る所で見られる日常的な行為です。

情報がリストの形態をとる以上、そうしたリストを BSL データとしてどのように表現するかを明確に学ぶ必要があります。実際、リストは非常に重要であるため、BSL には直交座標上の点 (posn) に対するサポートと同様に、リストの作成や操作のための組込みサポートが備わっています。点とは対照的に、リストのデータ定義を記述することは常にあなた自身に委ねられます。しかし、まずは第一歩から始めましょう。リストの作成からスタートします。

リストを生成するとき、私たちは常に空のリスト (empty list) から出発します。BSL では、空のリストを次のように表現します。

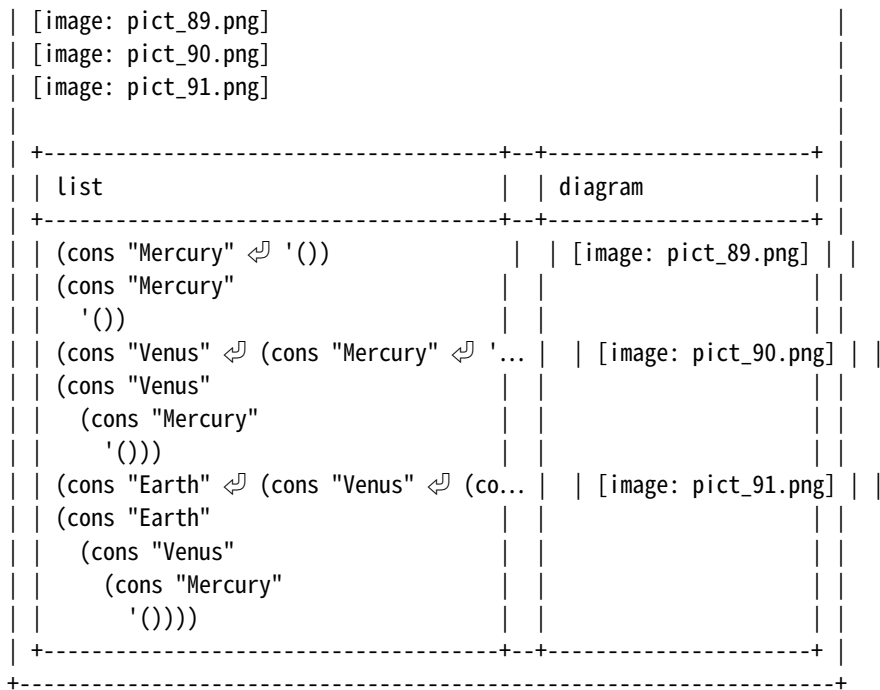
```
'()
```

これは「エンプティ (empty)」と発音され、「空のリスト (empty list)」の略です。#true や 5 と同様に、'() も単なる定数です。リストに何かを追加するとき、私たちは別のリストを構築します。BSL では、cons 操作がこの目的を果たします。例えば、

```
(cons "Mercury" '())
```

は、'() リストと文字列 “Mercury” からリストを構築します。Figure 44 は、構造体で用いたのと同じ図的表現でこのリストを示しています。cons のボックスには、first と rest という 2 つのフィールドがあります。この特定の例では、first フィールドに “Mercury” が含まれ、rest フィールドに '() が含まれています。

```
+-----+
| Figure 44: Building a list |
| |
```



1つの要素を含むリストが得られたら、再び cons を使用することで、2つの要素を含むリストを構築できます。その1例がこちらです：

```
(cons "Venus" (cons "Mercury" '()))
```

そして、もう1つの例がこちらです：

```
(cons "Earth" (cons "Mercury" '()))
```

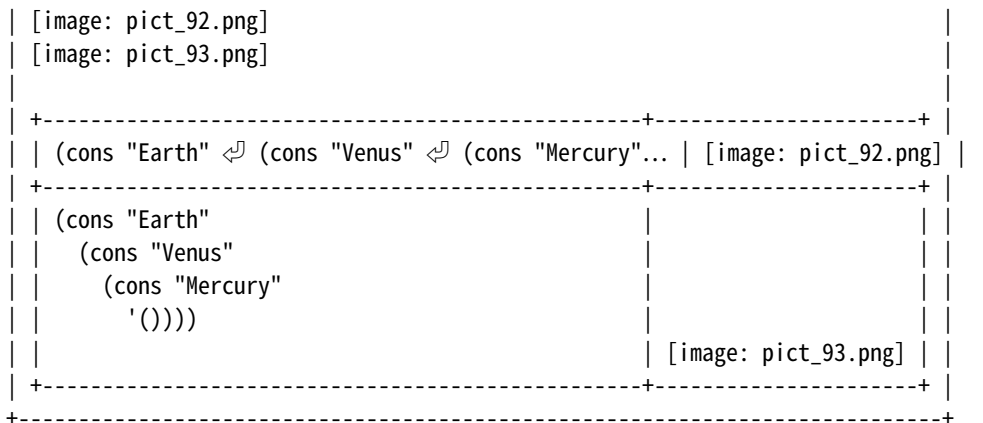
Figure 44 の中央の行は、2つの要素を含むリストのイメージを示しています。これも2つのフィールドを持つボックスですが、今回の rest フィールドにはボックスが含まれています。実際、そこには同じ図の上の行のボックスが含まれています。

最後に、3つの要素を持つリストを構築します：

```
(cons "Earth" (cons "Venus" (cons "Mercury" '())))
```

Figure 44 の最後の行は、3つの要素を持つリストを示しています。その rest フィールドには、さらにボックスを含むボックスが入っています。したがって、リストを作成するにつれて、私たちはボックスの中にボックス、さらにその中にボックスを入れていくことになります。一見すると奇妙に思えるかもしれませんが、これは誕生日に時々もらうような、中国の入れ子式の贈り物用ボックスや入れ子のコップのセットと全く同じです。唯一の違いは、BSL プログラムはどんな職人が物理的な箱を入れ子にするよりもはるかに深くリストを入れ子にできるという点です。

Figure 45: Drawing a List



腕の良い芸術家であっても、深く入れ子になった構造を描くのには苦勞するため、コンピュータ科学者は代わりに箱と矢印による図 (box-and-arrow diagrams) を用います。Figure 45 は、Figure 44 の最後の行をどのように再配置するかを示しています。各 cons 構造は別々のボックスになります。rest フィールドが複雑すぎてボックスの内部に描けない場合は、代わりに黒丸 (bullet) を描き、そこからそれが含んでいるボックスへと矢印付きの線を引きます。ボックスの配置方法によって、2 種類の図が得られます。Figure 45 の上の行に表示されている最初の図は、ボックスが作成された順序で並んでいます。下の行に表示されている 2 番目の図は、ボックスが cons によって結合された順序で並んでいます。したがって、2 番目の図を見れば、リストがどれほど長くても、そのリストに first を適用したときに何が生成されるかがすぐにわかります。そのため、プログラマーは 2 番目の配置を好みます。

**Exercise 129.** 以下を表現する BSL リストを作成しなさい：

1. 天体のリスト (例えば、少なくとも太陽系のすべての惑星)
2. 食事の品目のリスト (例えば、ステーキ、フライドポテト、豆、パン、水、ブリー・チーズ、アイスクリーム)
3. 色のリスト

figure 44 や figure 45 にあるような、これらのリストのボックス表現のスケッチを描きなさい。あなたはどちらのスケッチが好みですか。

数値のリストを作成することもできます。以下は 10 個の一桁の数字を含むリストです：

```
(cons 0
 (cons 1
 (cons 2
 (cons 3
 (cons 4
 (cons 5
 (cons 6
 (cons 7
 (cons 8
 (cons 9 '()))))))))))))
```

このリストを構築するには、10 回のリスト構築と 1 つの '()' が必要です。例えば、任意の 3 つの数値のリストの場合、

```
(cons pi
 (cons e
 (cons -22.3 '()))))
```

3 回の cons が必要となります。

一般に、リストは 1 種類の値だけを含む必要はなく、任意の値を含めることができます：

```
(cons "Robbie Round"
 (cons 3
 (cons #true
 '()))))
```

このリストの最初の要素は文字列、2 番目の要素は数値、最後の要素は Boolean です。このリストを、従業員の名前、勤続年数、会社経由の医療保険に加入しているかどうかという 3 つのデータからなる人事記録の表現とみなすこともできます。あるいは、ゲームのバーチャルプレイヤーを表しているとも考えることもできるでしょう。データ定義がなければ、そのデータが何を表しているのかを知ることはできません。

**Note:** とはいえ、このリストが固定された数のデータ項目からなる記録を表現するものである場合は、代わりに構造体型 (structure type) を使用してください。

以下は cons を含む最初のデータ定義です：

```
; A 3LON is a List of three numbers:
; (cons Number (cons Number (cons Number '())))
; interpretation a point in 3-dimensional space
```

もちろん、このデータ定義は、構造体のインスタンスに対するコンストラクタを他の定義で使うのと同様に cons を使用しており、ある意味で cons は特別なコンストラクタにすぎません。このデータ定義が示すことができていないのは、任意の長さのリスト、つまり要素が 0 個、1 個、2 個、10 個、あるいは 1,438,901 個含まれるリストをどのように形成するかということです。

そこでお試しに、もう一度試してみましょう：

```
; A List-of-names is one of:
; - '()
; - (cons String List-of-names)
; interpretation a list of invitees, by last name
```

深呼吸して、もう一度読んでみてください。このデータ定義は、あなたがこれまでに巡り合った定義の中で最も異質なものの 1 つです。自分自身を参照する定義など、以前に見たことがないでしょう。これが意味をなしているのかさえ即座にはわかりません。なにしろ、英語の先生に『テーブルとはテーブルである』というのが『テーブル』という単語

の定義です」と言ったとしたら、「ナンセンス！」と返ってくるのが関の山だからです。自己参照の定義は、単語の意味を説明してくれないからです。

しかし、コンピュータサイエンスやプログラミングにおいて、自己参照型の定義 (self-referential definitions) は中心的な役割を果たしており、注意深く扱えば、そのような定義は実際に意味をなします。ここで「意味をなす」とは、データ定義を本来の目的のために使用できること、すなわち、定義されているクラスに属するデータの例を生成したり、与えられたデータが定義されたデータクラスに属しているかどうかをチェックしたりできることを意味します。この観点から見ると、List-of-names の定義は完全に意味をなしています。最低限、箇条書きの最初の条項を使用して、1つの例として '()' を生成することができます。List-of-names の要素として '()' が与えられれば、2つ目の例を作るのは簡単です：

```
(cons "Findler" '())
```

ここでは、箇条書きの2番目の条項に従い、String と List-of-names から得られた唯一のリストを使用して、データを生成しています。同じルールを使って、この種のリストをさらに多く生成できます：

```
(cons "Flatt" '())
(cons "Felleisen" '())
(cons "Krishnamurthi" '())
```

そして、これらのリストはいずれも1つの名前 (String として表現) を含んでいますが、データ定義の2行目を使用して、より多くの名前を含むリストを作成することも実際に可能です：

```
(cons "Felleisen" (cons "Findler" '()))
```

このデータは、“Felleisen” が String であり、(cons "Findler" '()) が確認済みの List-of-names であるため、List-of-names に属します。

**Exercise 130.** 5つのStringを含むList-of-namesの要素を作成しなさい。figure 44にあるものと同様の、そのリストのボックス表現をスケッチしなさい。

なぜ

```
(cons "1" (cons "2" '()))
```

が List-of-names の要素であり、なぜ (cons2 '()) はそうでないのかを説明しなさい。

**Exercise 131.** Boolean 値のリストを表現するためのデータ定義を提供しなさい。そのクラスには、任意の長さのすべての Booleans のリストが含まれます。れます。

## 8.2 '()' とは何か、cons とは何か (What Is '()', What Is cons)

少し立ちし立ってまっとう c'() につに詳して詳てみ見てようしよ述の前述り、おは単は定数なる定数です。5 や “this is a string” のような定数と比較する数名関数名や変数のように見えます。しかし、#true や #false と比較してみれば、それが単に空リストを表す BSL の表現にすぎないことが容易に理解できるでしょう。

評価規則の観点から見ると、'()' は新しい種類の原子値 (atomic value など、他のあらゆる種類とは区別される新しい種類の原子値 (atomic value) です。また、Posn のような複合値 (compound value) でもありません。実に'()' は非常にユニークであり、それ自体だけで単独のクラスに属しています。そのため、この値のクラスには、'()' だけを認識し、それ以外は一切認識しない述語が用意されています。

```
; Any -> Boolean
; is the given value '()'
(define (empty? x)...)

```

すべての述語と同様に、empty? は BSL の値の全体集合からの任意の値に対して適用できます。これが #true を出力するのは、'()' に適用されたときに限られます。

```
> (empty? '())
#true
> (empty? 5)
#false
> (empty? "hello world")
#false
> (empty? (cons 1 '()))
#false
> (empty? (make-posn 0 0))
#false

```

次に cons について見ていきます。これまで見てきたすべてのことから、cons は構造体型の定義によって導入されるコンストラクタとまったく同様のコンストラクタであることが示唆されます。より正確には、cons は 2 つのフィールドを持つ構造体のコンストラクタであるように見えます。1 つ目のフィールドはあらゆる種類の値のためのものであり、2 つ目のフィールドはリスト状のあらゆる値のためのものです。以下の定義は、この考え方を BSL に変換したものです。

```
(define-struct pair [left right])
; A ConsPair is a structure:
; (make-pair Any Any).

; Any Any -> ConsPair
(define (our-cons a-value a-list)
 (make-pair a-value a-list))

```

唯一の問題は、our-cons が 2 番目の引数としてありとあらゆる BSL の値を受け入れられるのに対し、cons は受け入れないという点です。これは以下の実験によって確認できます。

```
> (cons 1 2)
cons:second argument must be a list, but received 1 and 2

```

言い方を変えれば、cons は実質的に「要素の列挙と構造体 (Itemizations and Structures)」で議論された種類のチェック付き関数 (checked function) であり、このことから次のような洗練形が示唆されます。

```
; A ConsOrEmpty is one of:
; - '()
; - (make-pair Any ConsOrEmpty)
; interpretation ConsOrEmpty is the class of all lists
```

```
; Any Any -> ConsOrEmpty
(define (our-cons a-value a-list)
 (cond
 [(empty? a-list) (make-pair a-value a-list)]
 [(pair? a-list) (make-pair a-value a-list)]
 [else (error "cons: second argument...")]))
```

もし cons がチェック付きコンストラクタ関数であるなら、結果として得られる構造体からどのようにして各要素を取り出せばよいのか疑問に思うかもしれません。結局のところ、「構造体の追加 (Adding Structure)」で述べられているように、構造体を用いたプログラミングにはセクタが必要です。cons 構造体には2つのフィールドがあるため、2つのセクタ first と rest が存在します。これらも pair 構造体を用いて簡単に定義できます。

```
; ConsOrEmpty -> Any
; extracts the left part of the given pair
(define (our-first a-list)
 (if (empty? a-list)
 (error 'our-first "...")
 (pair-left a-list)))
```

Stop! our-rest を定義しなさい。

プログラムが pair の構造体型定義にアクセスできる場合、右側のフィールドに '() や別の pair を含まない pair を作成することは容易です。そのような不良なインスタンスが意図的であれ偶発的であれ作成されると、それらは関数やプログラムを奇妙な方法で破壊しがちです。したがって BSL は、これらの問題を回避するために cons の実際の構造体型定義を隠蔽しています。「局所定義 (Local Definitions)」では、プログラムがそのような定義を隠す方法の1つを示していますが、現時点ではこの機能は必要ありません。

Figure 46: List primitives		
'()	a special value, mostly to represent the empty list	
empty?	a predicate to recognize '() and no other values	
cons	a checked constructor to create two-element lists	
first	the selector to extract the first element of a list	
rest	the selector to extract the extended part of a list	
cons?	a predicate to recognize instances of the cons constructor	



Figure 46: List primitives		
'()		a special value, mostly to represen...
empty?		a predicate to recognize '()' and no...
cons		a checked constructor to create two...
first		the selector to extract the last it...
rest		the selector to extract the extende...
cons?		a predicate to recognize instances ...

Figure 46 はこの節をまとめたものです。重要な洞察は、'()' がユニークな値であり、cons はリスト値を生成するチェック付きコンストラクタであるということです。さらに、first、rest、cons? は通常の述語やセレクタに対する単なる異なる名称にすぎません。したがって、この章が教えているのは、データの新しい作成方法ではなく、データ定義の新しい定式化方法なのです。

### 8.3 リストを用いたプログラミング (Programming with Lists)

あるリストを使って友人を管理しているとしましょう。そして、そのリストが非常に大きくなったため、ある名前がリストにあるかどうかを判定するプログラムが必要になったとします。このアイデアを具体的にするために、例題として定式化してみましょう：

**注記:** ここでは「友人 (friend)」という言葉を、現実世界ではなくソーシャルネットワークの意味で使っています。

**例題** あなたは新しい携帯電話の連絡先リストの作成に取り組んでいます。電話の持ち主は様々な機会にこのリストを更新し、参照します。今のとりとえ、あなたはこの連絡先リストを受け取り、そこに“Flatt”という名前が含まれているかどうかを判定する関数を設計するタスクを割り当てられました。

この例題に対する解法が得られたら、それを一般化して、リスト上の任意の名前を見つける関数にします。

前節の List-of-names のデータ定義は、関数が検索する名前のリストを表現するのに適しています。次にヘッダー部分に取り組みます。

```
; List-of-names -> Boolean
; determines whether "Flatt" is on a-list-of-names
(define (contains-flatt? a-list-of-names)
 #false)
```

a-list-of-names は関数が受け取る名前のリストの名前として優れていますが、長くて口にしがたいため、along に短縮します。



一般的な設計レシピに従い、次に関数の目的を説明する例をいくつか作成します。まず、最も単純な入力である '()' に対する出力を決定します。このリストには文字列が一切含まれていないため、“Flatt” が含まれていないことは確実です。

```
(check-expect (contains-flatt? '()) #false)
```

次に、要素が1つだけのリストを考えます。以下に2つの例を示します。

```
(check-expect (contains-flatt? (cons "Find" '()))
 #false)
(check-expect (contains-flatt? (cons "Flatt" '()))
 #true)
```

最初のケースでは、リスト上の唯一の要素が“Flatt”ではないため、答えは #false になります。2番目のケースでは、唯一の要素が“Flatt”であるため、答えは #true になります。最後に、より一般的な例を挙げます。

```
(check-expect
 (contains-flatt?
 (cons "A" (cons "Flatt" (cons "C" '()))))
 #true)
```

この場合も、リストに“Flatt”が含まれているため、答えは #true でなければなりません。手を止めて考えてみましょう！ 答えが #false にならない一般的な例を作成してください。

ひと息ついて、プログラムを実行してください。ヘッダーは関数の「ダミー」定義であり、いくつかの例を作成し、それらはテストに変換されました。そして素晴らしいことに、そのうちのいくつかは実際に成功します。間違った理由で成功しているのですが、成功することには変わりありません。ここまでの内容が理解できたら、読み進めてください。

第4のステップは、データ定義に一致する関数テンプレートを設計することです。文字列のリストに対するデータ定義には2つの節があるため、関数の本体は2つの節を持つ cond 式でなければなりません。2つの条件によって、関数が2種類のリストのどちらを受け取ったかが決定されます。

```
(define (contains-flatt? alon)
 (cond
 [(empty? alon)...]
 [(cons? alon)...]))
```

(cons? alon) の代わりに、2番目の節で else を使用することもできます。

cond 式の各節を順に調べることで、テンプレートにもう1つのヒントを追加できます。具体的には、対応する入力クラスが複合体で構成されている場合、各節にセレクト式をアノテーションとして注記することが設計レシピで提案されていたことを思い出してください。今回のケースでは、'()' には複合体がないため、構成要素はありません。そうでない場合、リストは文字列と別の文字列のリストから構築されるため、テンプレートに (first alon) と (rest alon) を追加することで、この事実を思い出せるようにします。

```
(define (contains-flatt? alon)
 (cond
 [(empty? alon)...]
 [(cons? alon)
 (... (first alon)... (rest alon)...)]))
```

いよいよ本題のプログラミング作業、つまり設計レシピの第5ステップに移行します。これはテンプレートから始まり、各 cond 節を個別に処理します。(empty? alon) が真の場合、入力空リストであり、この場合、関数は結果 #false を生成しなければなりません。2 番目のケースでは、(cons? alon) が真になります。テンプレート内の注記は、最初の文字列が存在し、その後に残りのリストが続くことを思い出させてくれます。そこで、このカテゴリに該当する例を考えてみましょう。

```
(cons "A"
 (cons...
 ... '()))
```

関数は人間と同じように、最初の要素を“Flatt”と比較しなければなりません。この例では、最初の要素は“A”であり“Flatt”ではないため、比較結果は #false になります。代わりに別の例、例えば以下のようなものを考えていたとしましょう。

```
(cons "Flatt"
 (cons...
 ... '()))
```

関数は入力の最初の要素が“Flatt”であると判定し、したがって #true を返します。このことは、cond 式の 2 行目にリスト上の最初の名前と“Flatt”を比較する式が含まれるべきであることを意味しています。

```
(define (contains-flatt? alon)
 (cond
 [(empty? alon) #false]
 [(cons? alon)
 (... (string=? (first alon) "Flatt")
 ... (rest alon)...)]))
```

さらに、比較結果が #true を生成した場合、関数は #true を出力しなければなりません。比較結果が #false の場合、残るのは別の文字列のリスト、すなわち (rest alon) です。明らかに、この場合関数は最終的な答えを知ることができません。なぜなら、答えは「...」が何を表しているかに依存するからです。言い換えれば、最初の要素が“Flatt”でない場合、リストの残りに“Flatt”が含まれているかどうかをチェックする何らかの方法が必要になります。

幸いなことに、私達には contains-flatt? があり、それが目的にぴったり合致しています。その目的記述によれば、この関数はリストに“Flatt”が含まれているかどうかを判定します。この記述は、(contains-flatt? l) が文字列のリスト l に“Flatt”が含まれているかどうかを教えてくれることを意味します。そして同様に、(contains-flatt? (rest alon)) は“Flatt”が

(rest alon) の要素であるかどうかを判定します。これこそが、まさに私たちが知る必要のあることです。

手短に言えば、最後の行は (contains-flatt? (rest alon)) となるべきです。

```
; List-of-names -> Boolean
(define (contains-flatt? alon)
 (cond
 [(empty? alon) #false]
 [(cons? alon)
 (... (string=? (first alon) "Flatt")...
 ... (contains-flatt? (rest alon))...)]))
```

コツは、2つの式の値を適切な方法で組み合わせることです。前述のように、最初の式が #true を生成する場合、リストの残りを検索する必要はありません。しかし、それが #false であっても、2番目の式が依然として #true を生成する可能性があり、それは名前 “Flatt” がリストの残りに存在することを意味します。これらすべては、最後の行の最初の式または2番目の式のいずれかが #true を生成する場合、(contains-flatt? alon) の結果が #true になることを示唆しています。

```
+-----+
| Figure 47: Searching a List |
| |
| ; List-of-names -> Boolean |
| ; determines whether "Flatt" occurs on alon |
| (check-expect |
| (contains-flatt? (cons "X" (cons "Y" (cons "Z" '())))) |
| #false) |
| (check-expect |
| (contains-flatt? (cons "A" (cons "Flatt" (cons "C" '())))) |
| #true) |
| (define (contains-flatt? alon) |
| (cond |
| [(empty? alon) #false] |
| [(cons? alon) |
| (or (string=? (first alon) "Flatt") |
| (contains-flatt? (rest alon)))])) |
+-----+
```

図 47 (Figure 47) は完全な定義を示しています。全体として、本書の第 1 章にある定義とそれほど違いはないように見えます。シグネチャ、目的記述、2つの例、そして定義で構成されています。この関数定義がこれまでに見たものと唯一異なる点は、自己参照 (self-reference)、つまり define の本体内で contains-flatt? を参照している点です。しかしよく考えると、データ定義も自己参照的であるため、ある意味では関数内の自己参照もそれほど驚くべきことではありません。

**Exercise 132.** この例で contains-flatt? を実行するために DrRacket を使用してください:

```
(cons "Fagan"
 (cons "Findler"
 (cons "Fisler"
 (cons "Flanagan"
 (cons "Flatt"
 (cons "Felleisen"
 (cons "Friedman" '()))))))))
```

どのような答えを予想しますか？

**Exercise 133.** 以下は `contains-flatt?` の 2 番目の `cond` 節を定式化するもう 1 つの方法です:

```
... (cond
 [(string=? (first alon) "Flatt") #true]
 [else (contains-flatt? (rest alon))])...
```

なぜこの式が図 47 (figure 47) のバージョンの `or` 式と同じ答えを生成するのか説明してください。あなたにとってどちらのバージョンがよりわかりやすいですか？理由を説明してください。

**Exercise 134.** 与えられた文字列が、与えられた文字列のリストに含まれているかどうかを判定する `contains?` 関数を開発してください。

注: BSL には実際には `member?` が付属しています。これは 2 つの値を受け取り、最初の値が 2 番目の値 (リスト) に含まれているかどうかをチェックする関数です:

```
> (member? "Flatt" (cons "b" (cons "Flatt" '())))
#true
```

`contains?` 関数を定義するために `member?` を使用しないでください。

```
+-----+
| Figure 48: Computing with lists, step 1 |
| |
| (contains-flatt? (cons "Flatt" (cons "C" '()))) |
| == |
| (cond |
| [(empty? (cons "Flatt" (cons "C" '()))) #false] |
| [(cons? (cons "Flatt" (cons "C" '()))) |
| (or |
| (string=? (first (cons "Flatt" (cons "C" '()))) "Flatt") |
| (contains-flatt? (rest (cons "Flatt" (cons "C" '()))))] |
|) |
|] |
+-----+
```

## 8.4 リストによる計算 (Computing with Lists)

私たちはまだ BSL を使用しているため、代数のルール (合間 1 (intermezzo 1) を参照) に従えば、DrRacket を使わなくても次のような式の値を求める方法が分かります。

```
(contains-flatt? (cons "Flatt" (cons "C" '())))
```

プログラマはこの種の計算がどのように動作するかを直感的に理解している必要があるため、この簡単な例についてステップごとに追っていきます。

図 48 は最初のステップを示しており、関数適用の値を決定するために通常の代入規則を使用しています。代数の先生が言うように関数が段階的に定義されているため、結果は条件式になります。

```
+-----+
| Figure 49: Computing with lists, step 2 |
| |
| ... |
| == |
| (cond |
| [#false #false] |
| [(cons? (cons "Flatt" (cons "C" '()))) |
| (or (string=? (first (cons "Flatt" (cons "C" '()))) "Flatt") |
| (contains-flatt? (rest (cons "Flatt" (cons "C" '())))))] |
| == |
| (cond |
| [(cons? (cons "Flatt" (cons "C" '()))) |
| (or (string=? (first (cons "Flatt" (cons "C" '()))) "Flatt") |
| (contains-flatt? (rest (cons "Flatt" (cons "C" '())))))] |
| == |
| (cond |
| [#true |
| (or (string=? (first (cons "Flatt" (cons "C" '()))) "Flatt") |
| (contains-flatt? (rest (cons "Flatt" (cons "C" '())))))] |
| == |
| (or (string=? (first (cons "Flatt" (cons "C" '()))) "Flatt") |
| (contains-flatt? (rest (cons "Flatt" (cons "C" '())))) |
+-----+
```

計算は図 49 で続きを行います。cond 式の正しい節を見つけるために、条件の値を 1 つずつ決定していく必要があります。cons で作られたリストは空ではないため、最初の条件の結果は #false となり、したがって最初の cond 節を取り除きます。最終的に、cons で作られたリストに対する cons? が成り立つため、2 番目の節の条件は #true に評価されます。

```
+-----+
| Figure 50: Computing with lists, step 3 |
| |
| ... |
| == |
| (or (string=? "Flatt" "Flatt") |
| (contains-flatt? (rest (cons "Flatt" (cons "C" '())))) |
| == (or #true (contains-flatt?...)) |
| == #true |
+-----+
```

ここからは、最終的な結果を得るまであとわずか3ステップの算術計算です。図 50 はその3つのステップを示しています。最初のステップは、first の法則に従って (first (cons "Flatt" ...)) を "Flatt" に評価します。2番目のステップでは、"Flatt" が文字列であり "Flatt" と等しいことを確認します。3番目のステップでは、x が何であれ (or #true x) は #true になるとしています。

**Exercise 135.** DrRacket のステッパーを使って、次に対する計算を確認しなさい。

```
(contains-flatt? (cons "Flatt" (cons "C" '())))
```

また、ステッパーを使って次の値を決定しなさい。

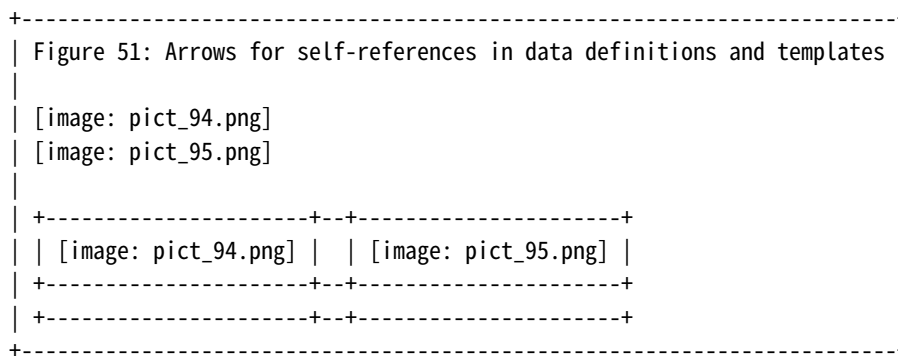
```
(contains-flatt?
 (cons "A" (cons "Flatt" (cons "C" '()))))
```

“Flatt” を “B” に置き換えると何が起きますか。

**Exercise 136.** DrRacket のステッパーで以下を検証しなさい。

```
(our-first (our-cons "a" '())) == "a"
(our-rest (our-cons "a" '())) == '()
```

これらの関数の定義については What Is ‘(), What Is cons を参照してください。



## 9 自己参照データ定義を用いた設計 (Designing with Self-Referential Data Definitions)

一見すると、自己参照データ定義 (self-referential data definitions) は混合データ (mixed data) の定義よりも遥かに複雑に思えるかもしれません。しかし、contains-flatt? の例が示すように、設計レシピの6つのステップは引き続き機能します。それにもかかわらず、本節では設計レシピを一般化し、自己参照データ定義に対してさらに効果的に機能するようにします。新しい部分は、自己参照データ定義がいつ必要であるかを発見するプロセス、テンプレートを導出するプロセス、そして関数本体を定義するプロセスに関わっています：

1. 問題文が任意サイズの情報 (information of arbitrary size) に関するものである場合、それを表現するために自己参照データ定義が必要となります。現時点では、そのようなクラスとして List-of-names の1つしか見ていません。figure 51 の左側は、同じ方法で List-of-strings を定義する方法を示しています。アトミックデータの他のリストも同様に機能します。自己参照データ定義が有効であるためには、2つの条件を満たす必要があります。第一に、少なくとも2つの節 (clauses) を含んでいなければなりません。第二に、節のうち少なくとも1つは、定義しようとしているデータのクラスを参照 (逆参照) してはなりません。データ定義内の参照から定義対象の用語へと戻る矢印を用いて、自己参照を明示的に特定することは優れた実践です。そのような注釈の例については figure 51 を参照してください。自己参照データ定義の妥当性は、データ例を作成することによって確認しなければなりません。データ定義を参照しない節から始め、次に節自体が定義を参照する箇所まで最初の例を使用しながら、もう一方の節へと進みます。figure 51 のデータ定義の場合、次のような3つのリストが得られます：

'() 第1節による (cons "a" '()) 第2節および直前の例による (cons "b" (cons "a" '())) 第2節および直前の例による

データ定義から例を生成することが不可能な場合、その定義は無効です。例を生成できても、段階的に大きな例を生成する方法が見当たらない場合、その定義は解釈 (interpretation) に合致していない可能性があります。> 注: 数値も任意に大きく思えるかもしれません。不正確な数値 (inexact numbers) においては、これは錯覚です。正確な整数 (precise integers) においては、実際にその通りです。したがって、整数を扱うことも本章の一部となります。

2. ヘッダー部分 (シグネチャ、目的の文、ダミー定義) に関しては何も変わりません。目的の文 (purpose statement) を作成するときは、関数がどのように処理を進めるか (特に与えられたデータのインスタンスをどのように巡回するか) ではなく、関数が何を計算するかに焦点を当てます。この設計レシピを具体化するための例を以下に示します：

```
;List-of-strings ->Number;counts how many stringsalos contains(define (how-many alos) 0)
```

目的の文は、関数が与えられた入力に含まれる文字列を単純に数えるだけであることを明確に述べています。この考えを BSL 関数としてどのように定式化できるかについて、事前に思い悩む必要はありません。

3. 機能的な例 (functional examples) に関しては、データ定義の自己参照節を複数回使用する入力を必ず試してください。これは、後で関数定義全体をカバーするテストを定式化するための最良の方法です。今回の進行中の例では、目的の文がデータ例からほぼ自然に機能的な例を生成します：

与えられた入力 (given)	期待される出力 (wanted)
'()	0
(cons "a" '())	1
(cons "b" (cons "a" '()))	2

1 行目は空のリスト (empty list) に関するものであり、空のリストには何も含まれていないことがわかっています。2 行目は 1 つの文字列からなるリストであるため、1 が期待される答えです。最後の行は 2 つの文字列からなるリストに関するものです。

4. 本質的に、自己参照データ定義は混合データのデータ定義と似ています。したがって、テンプレートの開発は「項挙と構造体 (Itemizations and Structures)」のレシピに従って進めることができます。具体的には、データ定義に含まれる節の数と同じ数の cond 節を持つ cond 式を定式化し、各識別条件 (recognizing condition) をデータ定義の対応する節に一致させ、複合値 (compound values) を処理するすべての cond 行に適切なセレクト式を書き出します。

質問 (Question)	回答 (Answer)
データ定義はデータの異なるサブクラスを区別していますか？	テンプレートには、データ定義が区別するサブクラスの数だけ cond 節が必要です。
サブクラスは互いにどのように異なりますか？	その違いを利用して、節ごとに条件を定式化します。
構造化された値を扱う節はありますか？	もしそうなら、適切なセレクト式をその節に追加します。
データ定義は自己参照を使用していますか？	データ定義の自己参照を表現するために、テンプレートに対して「自然再帰 (natural recursions)」を定式化します。
データ定義が他のデータ定義を参照している場合、その相互参照はどこにありますか？	その他のデータ定義に合わせてテンプレートを特化させます。このテンプレートを参照してください。「再・項挙と構造体を用いた設計 (Designing with Itemizations, Again)」の設計レシピのステップ 4 および 5 を参照してください。



figure 52: データ定義をテンプレートへと変換する方法 (How to translate a data definition into a template)

figure 52 はこの考え方を一問一答形式で表しています。左列には引数のデータ定義に関する質問が記載され、右列にはその回答がテンプレートの構築において何を意味するかが説明されています。

最後の行を無視して、List-of-strings を受け取る任意の関数に最初の 3 つの質問を適用すると、次のような形に至ります：

```
(define (fun-for-los alos) (cond [(empty? alos)...] [else (... (first alos)... (rest alos)...)]))
```

ただし、テンプレートの目的はデータ定義を関数のレイアウトとして表現することであることを思い出してください。つまり、テンプレートは入力されたデータ定義が英語と BSL の混在として表現している内容をコードとして表現します。したがって、データ定義のすべての重要な部分はテンプレート内に対応するものを持たなければならず、このガイドラインはデータ定義が自己参照的である場合（定義内部から定義対象の用語への矢印が含まれている場合）にも当てはまるべきです。特に、データ定義が  $i$  番目の節およびそこで言及されている構造体の  $k$  番目のフィールドで自己参照的である場合、テンプレートは  $i$  番目の `cond` 節および  $k$  番目のフィールドに対するセレクトタ式において自己参照的であるべきです。そのようなセレクトタ式ごとに、関数のパラメータへと戻る矢印を追加します。最終的に、テンプレートはデータ定義にあるのと同じ数の矢印を持たなければなりません。

figure 51 は、List-of-strings を受け取る関数のテンプレートをデータ定義と並べて表示することでこの考え方を図示しています。両者とも、2 番目の節（それぞれ `rest` フィールドおよびセレクトタ）から発生し、それぞれの定義の先頭を指し示す 1 本の矢印を含んでいます。

BSL およびほとんどのプログラミング言語はテキスト指向であるため、矢印の代替手段、すなわち適切なセレクトタ式に対する関数の自己適用 (self-application) を使用する必要があります：

```
(define (fun-for-los alos) (cond [(empty? alos)...] [else (... (first alos)..... (fun-for-los (rest alos))...)]))
```

関数自体の呼び出しを再帰 (recursion) と呼び、本書の最初の 4 つのパートでは自然再帰 (natural recursion) と呼びます。

5. 関数本体については、基底ケース (base cases) として知られる、再帰的な関数呼び出しを含まない `cond` 行から始めます。対応する回答は通常、定式化が容易であるか、すでに例として与えられています。

次に、自己参照ケースを扱います。まず、テンプレート行の各式が何を計算するかを自分自身に思い出させることから始めます。自然再帰については、関数が目的の文で指定された通りにすでに機能していると仮定します。この最後のステップは「信仰の跳躍 (leap of faith)」ですが、これから見るように、常に機能します。

残りは、さまざまな値を組み合わせる問題です。

質問 (Question)	回答 (Answer)
非再帰的な cond 節に対する回答は何ですか？	例を見れば、ここでどの値が必要かがわかるはずです。そうでない場合は、適切な例とテストを定式化してください。
再帰節のセレクト式は何を計算しますか？	データ定義はこれらの式がどの種類のデータを抽出するかを示し、データ定義の解釈はそのデータが何を表現しているかを示します。
自然再帰は何を計算しますか？	関数の目的の文を使用して、再帰の値が何を意味するかを判断します(その答えをどのように計算するかではありません)。目的の文から答えが得られない場合は、目的の文を改善してください。
関数は希望する答えを得るためにこれらの値をどのように組み合わせることができますか？	値を組み合わせる BSL の関数を見つけます。あるいは、それが機能しない場合は、補助関数を要請 (make a wish) します。多くの関数にとって、この最後のステップは簡単です。目的、例、およびテンプレートが組み合わせられて、利用可能な値を適切な結果へと組み合わせる関数または式を示してくれます。既存の用語を少し乱用して、この関数または式をコンビネータ (combinator) と呼びます。

figure 53: テンプレートを関数定義へと変換する方法 (How to turn a template into a function definition)

質問 (Question)	回答 (Answer)
したがって、ここで行き詰まった場合は…	…第 3 ステップの例を表に整理します。1 列目に与えられた入力を、最後の列に希望する出力を配置します。中間列にはセレクト式と自然再帰の値を入力します。コンビネータを示唆するパターンが現れるまで例を追加します。
テンプレートが他のテンプレートを参照している場合、補助関数は何を計算しますか？	もう一方の関数の目的の文と例を参照して何が計算されるかを判断し、その補助関数の設計が終わっていなくてもその結果を使用してよいと仮定します。

figure 54: テンプレートを関数へと変換するテーブル法 (Turning a template into a function, the table method)

figure 53 は、このステップに関する最初の 4 つの質問と回答を定式化しています。このゲームを使って `how-many` の定義を完成させましょう。fun-for-los テンプレートの名前を `how-many` に変更すると、次のようになります：

```
;List-of-strings ->Number;determines how many strings are onalos(define (how-many alos) (cond [(empty? alos)...] [else (... (first alos)..... (how-many (rest alos))...)]))
```

機能的な例がすでに示唆しているように、基底ケースに対する回答は 0 です。第 2 節の 2 つの式は、最初の項目と (rest alos) に含まれる文字列の数を計算します。alos 全体に何個の文字列があるかを計算するには、関数は後者の式の値に 1 を加えるだけで済みます：

```
(define (how-many alos) (cond [(empty? alos) 0] [else (+ (how-many (rest alos)) 1)]))
```

値を希望する答えに組み合わせる正しい方法を見つけることが、常にそれほど簡単であるとは限りません。初心者プログラマーはよくこのステップで行き詰まります。figure 54 が示唆するように、機能的な例を、テンプレート内の式の値も明記した表に整理することをお勧めします。figure 55 は、`how-many` の例におけるこの表がどのようなになるかを示しています。一番左の列にはサンプル入力が一覧表示され、一番右の列にはそれらの入力に対して希望する答えが含まれています。中間の 3 つの列は、テンプレート式の値を示しています：(first alos)、(rest alos)、および自然再帰である (how-many (rest alos)) です。この表をじっくり見つめると、結果の列が常に自然再帰の列の値よりも 1 大きいことに気づくでしょう。したがって、

```
(+ (how-many (rest alos)) 1)
```

が希望する結果を計算する式であると推測できるかもしれません。DrRacket はこの種の推測の検証が速いため、それを組み込んで RUN をクリックします。テストに変換された例が合格した場合は、式を熟考して、それがすべてのリストで正しく機能することを確認してください。合格しない場合は、別の考えが浮かぶまで表に例の行を追加してください。

表はまた、テンプレート内のいくつかのセクタ式が実際の定義においては無関係である可能性もあることを指摘しています。ここで (first alos) は最終的な答えを計算するのに必要ありません。これはテンプレートからの両方の式を使用する contains-flatt? とは極めて対照的です。

本書の残りを進めるにあたり、多くのケースにおいて組み合わせのステップは BSL の基本操作（例えば +、and、または cons）で表現できることを心に留めておいてください。ただし、場合によっては「願い事をする（make a wish）」、つまり補助関数を設計しなければならないこともあります。さらに別のケースでは、入れ子になった条件式が必要になる場合もあります。

**注:** 好奇心旺盛な読者のために付言すると、任意に大きなデータに対する設計レシピは、数学におけるいわゆる「帰納法による証明 (proofs by induction)」に対応し、「信仰の跳躍 (leap of faith)」はそのような証明の帰納ステップ (inductive step) における帰納法の仮定 (induction hypothesis) の使用を表しています。論理学は帰納法定理 (Induction Theorem) によってこの証明手法の妥当性を証明しています。 **注:** Felix Klock がコンビネータを推測するためのこのテーブルベースのアプローチを提案しました。

- 最後に、すべての例をテストに変換し、これらのテストが合格し、それらを実行することで関数のすべての部分がカバーされていることを確認してください。以下は、テストに変換された how-many の例です：

```
(check-expect (how-many '()) 0)(check-expect (how-many (cons "a" '())) 1)(check-expect (how-many (cons "b" (cons "a" '())) 2)
```

例をテストとして直接定式化するのが最善であり、BSL ではこれが可能であることを覚えておいてください。そうすることは、前のステップのテーブルベースの推測アプローチに頼る必要がある場合にも役立ちます。

Figure 55: Tabulating arguments, intermediate values, and results											
+-----+--+-----+--+-----											
alos	(first ↵ alos)	(rest ↵ alos)	(how-many ↵ (rest alos))								
(how-many ↵ alos)											
+-----+--+-----+--+-----											
(first											
alos)											
(rest											
alos)											
(how-many											
(rest alos))											
(how-many											
alos)											
(cons "a" ↵ '())		"a"		'()		0		1			



```
| +-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+ |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

## 9.1 指の運動：リスト (Finger Exercises: Lists)

**Exercise 137.** `contains-flatt?` のテンプレートと `how-many` のテンプレートを比較しなさい。関数名を除けば、これらは同じです。この類似性について説明しなさい。

**Exercise 138.** 以下は金額の系列を表現するためのデータ定義です：

```
; A List-of-amounts is one of:
; - '()
; - (cons PositiveNumber List-of-amounts)
```

データ定義を確実に理解するために、いくつかの例を作成しなさい。また、自己参照を表す矢印を追加しなさい。

`List-of-amounts` を受け取り、それらの金額の合計を計算する `sum` 関数を設計しなさい。DrRacket のステッパーを使用して、`List-of-amounts` の短いリスト `l` に対して `(suml)` がどのように動作するかを確認しなさい。

**Exercise 139.** 次に、このデータ定義を見てみましょう：

```
; A List-of-numbers is one of:
; - '()
; - (cons Number List-of-numbers)
```

このデータクラスの要素のいくつかは演習 138 の `sum` に対する適切な入力となりますが、そうでないものもあります。

`List-of-numbers` を受け取り、すべての数値が正の数であるかどうかを判定する関数 `pos?` を設計しなさい。言い換えれば、`(pos?l)` が `#true` を産出する場合、`l` は `List-of-amounts` の要素です。DrRacket のステッパーを使用して、`(cons5'())` や `(cons-1'())` に対して `pos?` がどのように動作するかを理解しなさい。

また、`checked-sum` も設計しなさい。この関数は `List-of-numbers` を受け取ります。入力が `List-of-amounts` にも属している場合はそれらの合計を産出し、そうでなければエラーをシグナルします。ヒント: `check-error` を使用したことを思い出してください。

`sum` は `List-of-numbers` の要素に対して何を計算しますか？

**Exercise 140.** ブーリアン値のリストを受け取り、それらすべてが `#true` であるかどうかを判定する関数 `all-true` を設計しなさい。言い換えれば、リスト上に 1 つでも `#false` があれば、この関数は `#false` を産出します。

次に、ブーリアン値のリストを受け取り、リスト上の少なくとも 1 つの項目が `#true` であるかどうかを判定する関数 `one-true` を設計しなさい。

コーディングにはテーブルに基づくアプローチを採用しなさい。これはベースケースの検討に役立つかもしれません。DrRacket のステッパーを使用して、これらの関数がリスト

(cons#true'()), (cons#false'()), および (cons#true(cons#false'())) をどのように処理するかを確認しなさい。

**Exercise 141.** 文字列のリストを受け取り、それらをすべて 1 つの長い文字列に連結する関数 `cat` の設計を求められた場合、最終的に次のような部分的な定義に至ることが保証されています：

```
; List-of-string -> String
; concatenates all strings in l into one long string
```

```
(check-expect (cat '()) "")
(check-expect (cat (cons "a" (cons "b" '())))) "ab")
(check-expect
 (cat (cons "ab" (cons "cd" (cons "ef" '()))))
 "abcdef")
```

```
(define (cat l)
 (cond
 [(empty? l) ""]
 [else (... (first l) ... (cat (rest l))...)]))
```

Figure 57: A table for cat									
l	(first l)	(rest l)	(cat (rest l))	(cat l)					
(cons "a" (cons "b" '()))	???	???	???	"ab"					
(cons "a"									
(cons "b"									
'()))									
(cons (cons "ab" (cons "cd" (cons "... (cat (rest l))..."))	???	???	???	"abcdef"					
(cons									
"ab"									
(cons "cd"									
(cons "ef"									
'()))									

figure 57 の表を埋めなさい。部分式によって計算された値から望ましい結果を作成できる関数を推測しなさい。

DrRacket のステッパーを使用して (cat(cons"a"('()))) を評価しなさい。

**Exercise 142.** 画像のリスト `loi` と正の数 `n` を受け取る `ill-sized?` 関数を設計しなさい。この関数は、`loi` の中で `(n n)` の正方形ではない最初の画像を産出します。そのような画像が見つからない場合は `#false` を産出します。

ヒント シグネチャの結果部分には次を使用しなさい

```
; ImageOrFalse is one of:
; - Image
; - #false
```

is one of: ; - Image ; - #false

### ### 9.2 空でないリスト (Non-empty Lists)

これで、`cons` を使用し、リストのデータ定義を作成するのに十分な知識が得られました。前節の終わりの演習問題を（いくつかでも）解いたのであれば、さまざまな種類の数値のリスト、ブール値のリスト、画像のリストなどを扱うことができるようになっています。本節では、リストとは何か、そしてそれをどのように処理するかについて探求を続けます。

まず、一見単純に思える「気温のリストの平均値を計算する」という問題から始めましょう。単純化するために、以下のデータ定義を提供します。

```
``racket
; A List-of-temperatures is one of:
; - '()
; - (cons CTemperature List-of-temperatures)

(define ABSOLUTE0 -272)
; A CTemperature is a Number greater than ABSOLUTE0.
; A List-of-temperatures is one of:
; - '()
; - (cons CTemperature List-of-temperatures)

(define ABSOLUTE0 -272)
; A CTemperature is a Number greater than ABSOLUTE0.
```

ここでの目的上、気温は普通の数値として考えてかまいませんが、2 番目のデータ定義は、現実にはすべての数値が気温というわけではないということを思い出させます。このことを念頭に置いておいてください。

ヘッダー部分は極めてシンプルです。

```
; List-of-temperatures -> Number
; computes the average temperature
(define (average alot) 0)
```

この問題の例を作成するのも簡単ですので、テストを 1 つだけ作成します。

```
(check-expect
 (average (cons 1 (cons 2 (cons 3 '())))) 2)
```

期待される結果は、当然、気温の合計を気温の個数で割ったものになります。

少し考えれば、average のテンプレートはこれまで見てきたものと類似しているはずだとわかるでしょう。



```
(define (average alot)
 (cond
 [(empty? alot)...]
 [(cons? alot)
 (... (first alot)...
 ... (average (rest alot))...)]))
```

2つの cond 節はデータ定義の2つの節を反映しています。質問部分は空リストと空でないリストを区別し、データ定義における自己参照のために自然な再帰が必要となります。

しかしながら、このテンプレートを関数定義へと完成させるのは非常に困難です。最初の cond 節には空の気温コレクションの平均を表す数値が必要ですが、そのような数値は存在しません。同様に、2番目の節は気温と残りの気温の平均を組み合わせる新しい平均を求める関数を要求します。可能ではありますが、このような方法で平均を計算するのは極めて不自然です。

気温の平均を計算するとき、私たちはそれらの合計を個数で割ります。単純な小さい例を定式化した際にもそう述べました。この文は、average が合計 (summing)、カウント (counting)、除算 (dividing) の3つのタスクからなる関数であることを示唆しています。「固定サイズのデータ (Fixed-Size Data)」のガイドラインでは、タスクごとに関数を1つ書くよう規定されています。それに従えば、average の設計は明快になります。

```
; List-of-temperatures -> Number
; computes the average temperature
(define (average alot)
 (/ (sum alot) (how-many alot)))

; List-of-temperatures -> Number
; adds up the temperatures on the given list
(define (sum alot) 0)

; List-of-temperatures -> Number
; counts the temperatures on the given list
(define (how-many alot) 0)
```

もちろん、最後の2つの関数定義は希望 (wish) であり、完全な定義を設計する必要があります。上記で出てきた how-many は List-of-strings にも List-of-temperatures にもそのまま機能するため (なぜでしょうか?)、そして sum の設計もおなじみの手順に従うだけなので、これを行うのは簡単です。

```
; List-of-temperatures -> Number
; adds up the temperatures on the given list
(define (sum alot)
 (cond
 [(empty? alot) 0]
 [else (+ (first alot) (sum (rest alot)))])))
```

手を止めてください！ average の例を使って sum の例を作成し、テストが正しく実行されることを確認してください。その後、average のテストを実行してください。

いまこの average の定義を読むと、学校で誰もが学ぶ平均の計算方法と直接対応しているため、明らかに正しいことがわかります。とはいえ、プログラムは自分たちのためだけでなく、他の人のためにも実行されます。特に、他の人がシグネチャを読み、その関数を使って、有益な回答を期待できるようにすべきです。しかし、私たちの average の定義は、空の気温リストに対しては機能しません。

**Exercise 143.** 空リストに適用されたとき、average が DrRacket でどのように動作するかを確認しなさい。次に、'() に適用されたときに役立つエラーメッセージを出力する関数 checked-average を設計しなさい。

**Note:** 数学の言葉を借りれば、演習問題 143 は average が '() に対してエラーを発生させるため、部分関数 (partial function) であることを示している、と言えます。

代替の解決策は、average が空リストに対して動作しないことをシグネチャを通じて将来の読者に伝えることです。そのためには、'() を除外するようなリストのデータ表現が必要となります。たとえば次のようなものです。

```
; An NEList-of-temperatures is one of:
; -???
; - (cons CTemperature NEList-of-temperatures)
```

問題は、'() リストが除外されつつも、他のすべての気温リストが構築可能なままになるように、「???」を何に置き換えるべきかということです。1つのヒントとして、空リストが最も短いリストである一方で、1つの気温からなるリストがその次に短いリストであるということが挙げられます。ここから、最初の節は1つの気温からなるすべての可能なリストを記述すべきだということが示唆されます。

```
; An NEList-of-temperatures is one of:
; - (cons CTemperature '())
; - (cons CTemperature NEList-of-temperatures)
; interpretation non-empty lists of Celsius temperatures
```

この定義はこれまでのリスト定義とは異なりますが、重要な要素を共有しています。すなわち、自己参照と、自己参照を使用しない節です。設計レシピを厳密に順守するには、NEList-of-temperatures の例をいくつか作成して、定義が理にかなっていることを確認する必要があります。いつものように、基本節から始める必要があります。つまり、例は次のようになるはずです。

```
(cons c '())
```

ここで c は CTemperature を表します (例: (cons ABSOLUTE0 '())). また、List-of-temperatures のすべての空でない要素が、この新しいデータクラスの要素でもあることは明らかです。(cons2(cons3'())) が適合するのであれば (cons1(cons2(cons3'()))) も適合しますし、先ほど確認したように (cons3'()) が NEList-of-temperatures の要素であるため、(cons2(cons3'())) は NEList-of-temperatures に属します。NEList-of-temperatures のサイズに制限がないことを自分で確認してください。

それでは、average が空でないリスト専用であることを誰もが知ることができるよう、average を設計し直す問題に戻りましょう。NEList-of-temperatures の定義により、シグネチャで意図した内容を表現する手段が得られました。

**Note:** この代替の展開は、この場合、average の定義域を狭めることで全域関数 (total function) を作成できることを説明しています。

```
; NEList-of-temperatures -> Number
; computes the average temperature

(check-expect (average (cons 1 (cons 2 (cons 3 '()))))
 2)

(define (average ne-l)
 (/ (sum ne-l)
 (how-many ne-l)))
```

当然ながら、残りの部分（目的文、例とテスト、関数定義）は同じままです。なにしろ、平均を計算するという考え方自体が空でない数値のコレクションを前提としており、それこそが私たちの議論のすべてのポイントだったからです。

**Exercise 144.** sum や how-many は、List-of-temperatures からの入力を想定して設計されているにもかかわらず、NEList-of-temperatures に対しても動作するでしょうか。動かないと思う場合は反例を示しなさい。動作すると思う場合はその理由を説明しなさい。

それにもかかわらず、この定義は sum や how-many が NEList-of-temperatures のインスタンスを受け取るようになったため、これらをどのように設計するかという新たな問いを提起します。以下は設計レシピの最初の 3 つのステップから導かれる明確な結果です。

```
; NEList-of-temperatures -> Number
; computes the sum of the given temperatures
(check-expect
 (sum (cons 1 (cons 2 (cons 3 '())))) 6)
(define (sum ne-l) 0)
```

この例は average の例を適応させたものです。ダミー定義は数値を生成しますが、与えられたテストに対しては誤った数値となります。

第 4 ステップは、NEList-of-temperatures に対する sum の設計において最も興味深い部分です。これまでのすべての設計例では、データ定義が適切な形状を持っていたため、空リストと空でない（つまり cons された）リストを区別するテンプレートが必要でした。しかし、NEList-of-temperatures ではそうではありません。ここでは両方の節が cons されたリストを追加します。ただし、2 つの節はそれらのリストの rest フィールドにおいて異なります。特に、最初の節は rest フィールドに常に '() を使用し、2 番目の節は代わりに cons を使用します。したがって、1 番目のデータと 2 番目のデータを区別するための適切な条件は、rest フィールドを抽出してから empty? を使用することになります。

```
; NEList-of-temperatures -> Number
(define (sum ne-l)
 (cond
 [(empty? (rest ne-l))...]
 [else...]))
```

ここで else は (cons?(restne-l)) を置き換えています。

次に、両方の節を点検し、一方または両方の節が ne-l を構造体であるかのように扱っているかどうかを判断する必要があります。ne-l に対して無条件に rest が使用されていることが示す通り、もちろんこれは当てはまります。言い換えれば、2つの節に適切なセクター式を追加します。

```
(define (sum ne-l)
 (cond
 [(empty? (rest ne-l)) (... (first ne-l)...)]
 [else (... (first ne-l)... (rest ne-l)...)]))
```

読み進める前に、最初の節にセクター式 (restne-l) が含まれていない理由を説明してください。

テンプレート設計の最後の問いは、データ定義における自己参照に関するものです。ご存知のように、NEList-of-temperatures には1つの自己参照が含まれているため、sum のテンプレートには1つの再帰的呼び出しが必要となります。

```
(define (sum ne-l)
 (cond
 [(empty? (rest ne-l)) (... (first ne-l)...)]
 [else
 (... (first ne-l)... (sum (rest ne-l))...)]))
```

具体的には、データ定義の類似するポイントで自己参照されているため、2番目の節で sum は (restne-l) に対して呼び出されます。

第5の設計ステップとして、私たちがすでにどれだけのものを手にしているかを理解しましょう。最初の cond 節は、再帰的関数呼び出しを伴う2番目の節よりも大幅に単純に見えるため、そちらから始めるべきです。この特定のケースでは、条件は sum がちょうど1つの気温 (firstne-l) を持つリストに適用されることを意味しています。明らかに、この1つの気温が、与えられたリストのすべての気温の合計となります。

```
(define (sum ne-l)
 (cond
 [(empty? (rest ne-l)) (first ne-l)]
 [else
 (... (first ne-l)... (sum (rest ne-l))...)]))
```

2番目の節は、リストが1つの気温と少なくとももう1つの要素から構成されていることを表しています。(firstne-l) が先頭の位置を抽出し、(restne-l) が残りの部分を抽出します。さらに、テンプレートは (sum(restne-l)) の結果を使用することを示唆しています。しかし、

sum はまさにあなたが定義している最中の関数であり、それが (restne-l) をどのように利用するかをあらかじめ知る由もありません。知っているのは、目的文が述べていること、すなわち sum が与えられたリスト（ここでは (restne-l)）のすべての気温を足し合わせるということだけです。この文が正しいとすれば、(sum(restne-l)) は ne-l の 1 つの数値を除くすべての数値を足し合わせることになります。合計を得るには、関数は最初の気温を加算するだけでよいのです。

```
(define (sum ne-l)
 (cond
 [(empty? (rest ne-l)) (first ne-l)]
 [else (+ (first ne-l) (sum (rest ne-l)))]))
```

ここでこの関数のテストを実行すると、「信頼の飛躍 (leap of faith)」が正当化されることがわかります。実際、本書の範囲を超える理由により、この飛躍は常に正当化されます。それこそが、それが設計レシピの固有の構成要素となっている理由です。

**Exercise 145.** NEList-of-temperatures を受け取り、気温が降順にソートされている（つまり、2 番目が 1 番目より小さく、3 番目が 2 番目より小さく、以下同様である）場合に #true を返し、それ以外の場合に #false を返す述語 sorted? を設計しなさい。

ヒント: この問題は、コンビネータを推測するための表に基づいた方法がうまく機能するもう 1 つの問題です。figure 58 にいくつかの例の表（一部）を示します。表の残りを埋めなさい。そして、その構成要素から結果を計算する式を作成しなさい。

+-----+														
Figure 58: A table for sorted?														
+-----+-----+-----+-----+														
l	(first l)	(rest l)	(sorted? ↵ (rest l))	(sorted? ↵ l)										
+-----+-----+-----+-----+														
(sorted?														
(rest l)														
(sorted?														
l)														
(cons 1 ↵ (cons 2 ↵ '()))	1	???		#true		#false								
(cons 1														
(cons 2														
'())														
(cons 3 ↵ (cons 2 ↵ '()))	3		(cons 2 '())	???		#true								
(cons 3														
(cons 2														
'())														
(cons 0 ↵ (cons 3 ↵ (cons 2 ↵ '())))	0		(cons 3 ↵ (cons 2 ↵ '()))											
???	???													
(cons 0														
(cons 3														
(cons 2														
'()))														

```

| | (cons 3 | | | | | | | | | |
| | (cons 2 | | | | | | | | | |
| | '())) | | | | | | | | | |
| +-----+-----+-----+-----+
+-----+

```

**Exercise 146.** NEList-of-temperatures に対する how-many を設計しなさい。これを行うことで average が完成するので、average がすべてのテストに合格することも確認しなさい。

**Exercise 147.** ブール値の空でないリストの表現である NEList-of-Booleans のデータ定義を作成しなさい。次に、演習問題 140 の関数 all-true および one-true を再設計しなさい。

**Exercise 148.** 本節の関数定義 (sum、how-many、all-true、one-true) を、前節までの対応する関数定義と比較しなさい。空でないリストの定義と比較して、空リストを受け入れるデータ定義を扱う方がよいでしょうか。なぜでしょうか、あるいはなぜそうではないのでしょうか。

### 9.3 自然数 (Natural Numbers)

BSL プログラミング言語は、リストを消費する多くの関数だけでなく、リストを生成する関数もいくつか提供しています。その中にあるのが make-list で、これは数値  $n$  と他の何らかの値  $v$  を受け取り、 $v$  を  $n$  個含むリストを生成します。以下にいくつか例を示します。

```

> (make-list 2 "hello")
(cons "hello" (cons "hello" '()))
> (make-list 3 #true)
(cons #true (cons #true (cons #true '())))
> (make-list 0 17)
'()

```

要するに、この関数はアトムックな (不可分の) データを受け取るにもかかわらず、任意に大きなデータを生成します。疑問に思ふべきは、これがなぜ可能なのかということです。

私たちの答えは、make-list の入力は一単なる数値ではなく、特別な種類の数値であるということです。幼稚園でこれらの数値は「物を数える数 (counting numbers)」と呼ばれていました。つまり、これらの数値はオブジェクトを数えるために使用されます。コンピュータサイエンスでは、これらの数値は自然数 (natural numbers) と呼ばれます。通常の数値とは異なり、自然数にはデータ定義が存在します。

```

; An N is one of:
; - 0
; - (add1 N)
; interpretation represents the counting numbers

```

最初の条項は 0 が自然数であることを示しています。これは数えるべきオブジェクトが存在しないことを表すために用いられます。2 番目の条項は、もし  $n$  が自然数であれば  $n+1$  も自然数であることを示しています。なぜなら add1 は与えられた任意の数値に 1 を加える関数だからです。この 2 番目の条項を  $(+ n 1)$  と書くこともできますが、add1 を使用することで、この加算が特別なものであると示すことを意図しています。



この `add1` の使用における特別な点は、通常の関数というよりも構造体型定義のコンストラクタ（構造子）のように振る舞うことです。その理由から、BSL には `add1` に対応する「セクタ（選択子）」である関数 `sub1` も用意されています。0 に等しくない任意の自然数  $m$  が与えられたとき、`sub1` を使用して  $m$  の構築に使用された数値を調べることができます。言い換えれば、`add1` は `cons` のようなものであり、`sub1` は `first` や `rest` のようなものです。

この時点で、0 を 0 でない自然数と区別する述語（predicate）が何であるか疑問に思うかもしれません。リストの場合と同様に 2 つあります。与えられた数値が 0 であるかを判定する `zero?` と、数値が 0 より大きいかを判定する `positive?` です。

これで、`make-list` のような自然数を扱う関数を自分自身で設計する準備が整いました。データ定義はすでに利用可能なので、ヘッダー情報を追加しましょう。

```
; N String -> List-of-strings
; creates a list of n copies of s

(check-expect (copier 0 "hello") '())
(check-expect (copier 2 "hello")
 (cons "hello" (cons "hello" '())))

(define (copier n s)
 '())
```

テンプレートを作成することが次のステップです。テンプレートに関する問いは、`copier` の本体が 2 つの条項を持つ `cond` 式であることを示唆しています。1 つは 0 のための条項、もう 1 つは正の数のための条項です。さらに、0 はアトミックとみなされ、正の数は構造化された値とみなされるため、テンプレートの 2 番目の条項にはセクタ式が必要です。最後に非常に重要なこととして、 $N$  のデータ定義は 2 番目の条項において自己参照的（再帰的）です。したがって、テンプレートは 2 番目の条項で対応するセクタ式に対する再帰呼び出しを必要とします。

```
(define (copier n s)
 (cond
 [(zero? n)...]
 [(positive? n) (... (copier (sub1 n) s)...)]))
```

```
+-----+
| Figure 59: Creating a list of copies |
| |
| ; N String -> List-of-strings |
| ; creates a list of n copies of s |
| |
| (check-expect (copier 0 "hello") '()) |
| (check-expect (copier 2 "hello") |
| (cons "hello" (cons "hello" '()))) |
| |
| (define (copier n s) |
| (cond |
| [(zero? n)...] |
| [(positive? n) (... (copier (sub1 n) s)...)])) |
| |
+-----+
```

```
| [(zero? n) '()] |
| [(positive? n) (cons s (copier (sub1 n) s)))] |
+-----+
```

Figure 59 には、テンプレートから得られた `copier` 関数の完全な定義が含まれています。このステップを慎重に再構築してみましょう。いつものように、再帰呼び出しを持たない `cond` 条項から始めます。ここでは条件が（重要な）入力が 0 であることを伝えており、それは関数が 0 個の要素、つまり要素を持たないリストを生成しなければならないことを意味します。もちろん、2 番目の例を検討したことで、このケースはすでに明確になっています。次に、もう一方の `cond` 条項に移り、その式が何を計算するかを思い出してみましょう。

1. `(sub1 n)` は  $n$  の構築に使用された自然数を取り出します。 $n$  は 0 より大きいことがわかっています。
2. `(copier (sub1 n) s)` は `copier` の目的記述に従って、`(sub1 n)` 個の文字列 `s` のリストを生成します。

しかし、関数には  $n$  が与えられているため、 $n$  個の文字列 `s` を持つリストを生成しなければなりません。文字列が 1 つだけ不足しているリストが与えられた場合、関数が単に `(copier (sub1 n) s)` の結果に 1 つの `s` を `cons` すればよいことは容易にわかります。そしてそれこそが、2 番目の条項が正確に指定していることです。

この時点で、テストを実行して、この関数が少なくとも作成した 2 つの例で正しく動作することを確認する必要があります。さらに、いくつかの追加の入力に対して関数を使用してみるのもよいでしょう。

**Exercise 149.** 自然数と Boolean や画像 (image) に対して `copier` を適用したとき、`copier` は適切に動作しますか？それとも別の関数を設計する必要がありますか？答えについては「抽象化 (Abstraction)」を参照してください。

`copier` の代替定義では `else` を使用するかもしれません。

```
(define (copier.v2 n s)
 (cond
 [(zero? n) '()]
 [else (cons s (copier.v2 (sub1 n) s))]))
```

`copier` と `copier.v2` を 0.1 と "x" に適用したとき、それらはどのように振る舞いますか？説明しなさい。説明を確認するために DrRacket のステッパー (stepper) を使用しなさい。

**Exercise 150.** 関数 `add-to-pi` を設計しなさい。この関数は自然数  $n$  を受け取り、基本操作の `+` を使用せずにそれを  $\pi$  に加算します。以下が出発点です。

```
; N -> Number
; computes (+ n pi) without using +

(check-within (add-to-pi 3) (+ 3 pi) 0.001)
```



```
(define (add-to-pi n)
 pi)
```

完全な定義ができたなら、関数を `add` に一般化しなさい。この関数は、`+` を使用せずに自然数 `n` を任意の実数 `x` に加算します。スケルトン（骨組み）が `check-within` を使用しているのはなぜですか？

**Exercise 151.** 関数 `multiply` を設計しなさい。この関数は自然数 `n` を受け取り、`*` を使用せずに数値 `x` と掛け合わせます。

DrRacket のステッパーを使用して、好きな `x` に対する `(multiply 3 x)` を評価しなさい。`multiply` は小学校で学んだこととどのように関連していますか？

**Exercise 152.** 2つの関数 `col` と `row` を設計しなさい。

関数 `col` は自然数 `n` と画像 `img` を受け取ります。`img` の `n` 個のコピーを縦方向（垂直）に並べた列（column）を生成します。

関数 `row` は自然数 `n` と画像 `img` を受け取ります。`img` の `n` 個のコピーを横方向（水平）に並べた行（row）を生成します。

```
+-----+
| Figure 60: Random attacks |
| |
| [image: pict_96.png] |
| [image: pict_97.png] |
| [image: pict_98.png] |
| [image: pict_96.png] [image: pict_97.png] [image: pict_98.png] |
+-----+
```

**Exercise 153.** この演習問題の目的は、1968年スタイルのヨーロッパの学生運動（暴動）の結果を視覚化することです。大まかなアイディアは次のとおりです。学生の小さなグループが集まって絵の具の入った風船を作り、講義室に入り、出席者に向かってランダムに風船を投げます。あなたのプログラムは、風船が講義室の座席をどのように着色するかを表示します。

演習問題 152 の2つの関数を使用して、それぞれサイズが  $10 \times 10$  の正方形で構成される `COLUMNS`  $\times$  `ROWS` の長方形を作成しなさい。それを同じサイズの `empty-scene` に配置します。この画像があなたの講義室となります。

`add-balloons` を設計しなさい。この関数は、座標が講義室の寸法内に収まる `Posn` のリストを受け取ります。`Posn` のリストで指定された位置に赤いドットが追加された講義室の画像を生成します。

Figure 60 は、10列の `COLUMNS` と20行の `ROWS` に対する、ある `Posn` のリストを与えたときの私たちの解の出力を示しています。一番左は何も汚れていない講義室、2番目は2つの風船が当たった後、最後は10回の命中による極めて起こりそうにない分布です。10番目の

風船はどこにありますか？ が当たった後、そして最後は10回の命中という非常にありそうにない分布です。10個目はどこにあるのでしょうか？

#### 9.4 マトリョーシカ人形 (Russian Dolls)

Wikipedia では、マトリョーシカ人形 (Russian doll) を「大きさが徐々に小さくなる人形のセットで、一つがもう一つの内側に配置されているもの」と定義し、次の画像で説明しています。

[image: russian-doll-from-wikipedia.png]

この画像では、見る人が人形をすべて確認できるように、人形が分解されています。

**Note:** この問題り、あるいはばかばかしくさえ思えるかもしれません。なぜマトリョーシカを表現したいのか、あるいはそのような表現で何をするのかが明確ではないからです。とりあえず今は、付き合ってみてください。

さて、このようなマトリョーシカを BSL のデータで表現する問題を考えてみましょう。少し想像力を働かせれば、職人が任意の数の人形からなるマトリョーシカの入れ子を作成できることは簡単に理解できます。結局のところ、与えられたマトリョーシカの周りに、いつでも別の層 (レイヤー) を巻きつけることができるからです。一方で、その一番奥深くには、中身が何もない固形の人形が存在することも分かっています。

マトリョーシカの各層について、私たちはさまざまなことに興味を持つことができます。入れ子の深さに関連してはいますがそのサイズ、色、表面に描かれている絵、などです。ここではそのうちの1つ、すなわち人形の色だけを選び、それを文字列 (string) で表現することにします。それをふまえると、マトリョーシカの各層は2つのプロパティ (属性) を持つことがわかります。その色と、内側にある人形です。2つのプロパティを持つ情報を表現するために、私たちは常に構造体型 (structure type) を定義します。

```
(define-struct layer [color doll])
```

そして、次のようにデータ定義を追加します。

```
; An RD (short for Russian doll) is one of:
; - String
; - (make-layer String RD)
```

当然のことながら、このデータ定義の第1条項 (clause) は一番内側の人形、正確に言えばその色を表しています。第2条項は、与えられたマトリョーシカの周りに層を追加するためのものです。これを layer のインスタンスで表現しますが、これには明らかに人形の色と、もう1つのフィールド、すなわちこの人形のすぐ内側に入れ子になっている人形が含まれています。

次の人形を見てみましょう。

[image: pict\_99.png]

これは3つの人形から構成されています。赤い人形が一番内側にあり、緑色の人形が中央に位置し、黄色い人形が現在の最も外側のラッパー（包み）です。この人形を RD の要素として表現するには、どちらの端から始めても構いません。ここでは内側から外側へと進めていきます。赤い人形は RD として簡単に表現できます。中には何も入っておらず、赤色であるため、文字列 "red" で十分です。2 番目の層には、次のように記述します。

```
(make-layer "green" "red")
```

これは、緑色の（空洞の）人形が赤い人形を含んでいることを示しています。最後に、一番外側を得るには、この最後の人形の周りにさらにもう 1 つの層を巻きつけるだけです。

```
(make-layer "yellow" (make-layer "green" "red"))
```

このプロセスによって、任意の色付きマトリョーシカのセットからデータ表現へと変換する方法がよく理解できるはずです。ただし、プログラマは逆の操作、つまりデータから具体的な情報へと遡ることもできなければならないことを心に留めておいてください。この精神に基づき、次の RD の要素に対応するマトリョーシカの概略図を描いてみましょう。

```
(make-layer "pink" (make-layer "black" "white"))
```

これを BSL で実際に試してみるのもよいでしょう。

これでデータ定義が手に入り、実際の人形をどのように表現するか、また RD の要素をどのように人形として解釈するかを理解できたので、RD を受け取る関数を設計する準備が整いました。具体的には、マトリョーシカのセットに何個の人形が含まれているかを数える関数を設計してみましょう。この文は優れた目的文（purpose statement）であり、シグネチャ（signature）も決定します。

```
; RD -> Number
; how many dolls are part of an-rd
```

データの例については、(make-layer "yellow" (make-layer "green" "red")) から始めましょう。上の画像は、赤い人形、緑色の人形、黄色い人形の3つの人形があるため、期待される答えが3であることを教えてくれます。この例を1つ作業するだけでも、入力が次の人形の表現である場合には

[image: pict\_100.png]

答えが1になることが分かります。

ステップ4ではテンプレートの開発が求められます。このステップの標準的な質問を使用すると、次のテンプレートが作成されます。

```
; RD -> Number
; how many dolls are a part of an-rd
```

```
(define (depth an-rd)
 (cond
 [(string? an-rd)...]
 [(layer? an-rd)
 (... (layer-color an-rd)...
 ... (depth (layer-doll an-rd))...)]))
```

cond 条項の数は、RD の定義における条項の数によって決まります。それぞれの条項は対象とするデータがどのような種類のものであるかを具体的に明記しており、これにより string? 述語と layer? 述語を使用することがわかります。文字列は複合データ (compound data) ではありませんが、layer のインスタンスは2つの値を含んでいます。関数がこれらの値を必要とする場合、セクタ式 (layer-color an-rd) および (layer-doll an-rd) を使用します。最後に、データ定義の第2条項には、layer 構造体の doll フィールドから定義自体への自己参照が含まれています。したがって、2番目のセクタ式には再帰的な関数呼び出しが必要です。

例とテンプレートを見れば、関数の定義はほぼ自ずと決まります。非再帰的な cond 条項の場合、答えが明らかになるのは1です。再帰的な条項の場合、テンプレートの式は次のような結果を計算します。

- (layer-color an-rd) は、現在の層の色を表す文字列を抽出します。
- (layer-doll an-rd) は、現在の層に含まれている人形を抽出します。
- (depth (layer-doll an-rd)) は、depth の目的文にしたがって、(layer-doll an-rd) に何個の人形が含まれているかを決定します。

この最後の数値は求めたい答えにほぼ近いですが、完全ではありません。なぜなら、an-rd と (layer-doll an-rd) の差は1つの層、つまり1個追加の人形を意味するからです。言い換えると、実際の答えを得るには、関数は再帰的な結果に1を加えなければなりません。

```
; RD -> Number
; how many dolls are a part of an-rd
(define (depth an-rd)
 (cond
 [(string? an-rd) 1]
 [else (+ (depth (layer-doll an-rd)) 1)]))
```

関数の定義において、第2条項で (layer-color an-rd) が使用されていないことに注意してください。ここでも改めて分かるように、テンプレートはデータ定義に関して私たちが知っているすべてのことの整理スキーム (構成スキーマ) ですが、実際の定義においてそれらの要素すべてが必要になるとは限りません。

最後に、例をテストへと変換しましょう。

```
(check-expect (depth "red") 1)
(check-expect
 (depth
 (make-layer "yellow" (make-layer "green" "red"))))
3)
```

これらを DrRacket で実行すると、それらの評価が `depth` の定義のすべての部分に触れていることが確認できます。

**Exercise 154.** 関数 `colors` を設計しなさい。この関数はマトリョーシカを受け取り、すべての色をカンマとスペースで区切った 1 つの文字列を生成します。したがって、私たちの例では次を出力する必要があります。

```
"yellow, green, red"
```

**Exercise 155.** 関数 `inner` を設計しなさい。この関数は `RD` を受け取り、一番内側の人形の(色)を生成します。お気に入りの `rd` に対して `(inner rd)` を評価するために、DrRacket のステッパーを使用しなさい。

## 9.5 リストとワールド (Lists and World)

一般に、リストや自己参照的なデータ定義を使用すると、有限のデータを使用する場合よりもはるかに興味深いワールドプログラムを設計して実行できるようになります。『要素列挙と構造体 (Itemizations and Structures)』で扱ったスペースインベーダープログラムを拡張し、プレイヤーが戦車から好きなだけ弾を発射できるようにしたバージョンを作成できる場面を想像してみてください。まずは、この問題の単純化されたバージョンから始めてみましょう。

**Note:** しばらくワールドプログラムを設計していない場合は、『ワールドプログラムの設計 (Designing World Programs)』を読み返してください。

例題: 弾の発射をシミュレートするワールドプログラムを設計しなさい。「プレイヤー」がスペースキーを押すたびに、プログラムはキャンバスの最下部に弾を 1 つ追加します。これらの弾は、1 クロックティックあたり 1 ピクセルの割合で垂直に上昇します。

ワールドプログラムの設計は、情報を「定数」と「絶えず変化するワールドの状態の要素」に分離することから始まります。前者については物理的定数および画像定数を導入し、後者についてはワールド状態のデータ表現を構築する必要があります。この例題は具体的な詳細について比較的曖昧ですが、縦線に沿って弾が描画される長方形の背景を前提としていることは明らかなです。明らかに弾の位置はクロックティックごとに変化しますが、背景のサイズと弾のラインの `x` 座標は一定のままです。

```
(define HEIGHT 80); distances in terms of pixels
(define WIDTH 100)
(define XSHOTS (/ WIDTH 2))

; graphical constants
(define BACKGROUND (empty-scene WIDTH HEIGHT))
(define SHOT (triangle 3 "solid" "red"))
```

問題文のどこにもこれらの特定の選択を要求するものではありませんが、それらを簡単に変更できる（単一の定義を編集するだけで変更できることを意味します）限り、目標は達成されたと言えます。

変化する「ワールド」の側面に関して、問題文は2つのことに言及しています。第1に、スペースキーを押すと弾が追加されること。第2に、すべての弾が1クロックティックあたり1ピクセルずつまっすぐ上に移動することです。プレイヤーが何発の弾を「発射」するか予測できないため、それらを表現するためにリストを使用します。

```
; A List-of-shots is one of:
; - '()
; - (cons Shot List-of-shots)
; interpretation the collection of shots fired
```

残る唯一の疑問は、個々の弾をどのように表現するかです。すべての弾が同じx座標を持ち、その座標は終始一定であることはすでにわかっています。さらに、すべての弾の見た目は同じです。したがって、それらが互いに異なる唯一の属性はy座標です。それゆえ、各弾を数値として表現すれば十分です。

```
; A Shot is a Number.
; interpretation represents the shot's y-coordinate
```

すべての弾がキャンバスの最下部から発射され、その後上に移動すること、つまりy座標が継続的に減少することを知っているため、弾の表現を HEIGHT 未満の数値の区間に制限することもできます。

このワールドを表現するために、次のようなデータ定義を使用することもできます。

```
; A ShotWorld is List-of-numbers.
; interpretation each number on such a list
; represents the y-coordinate of a shot
```

結局のところ、上記の2つの定義はどちらも数値のリストを表しています。私たちはすでに数値のリストの定義を持っており、ShotWorldという名前は、このデータのクラスが何に関するものであるかを皆に伝えています。

定数を定義し、ワールドの状態に関するデータ表現を構築したら、主要なタスクはどのイベントハンドラを採用するかを選択し、そのシグネチャを与えられた問題に適合させることです。実行中の例ではクロックティックとスペースキーに言及しており、これらはすべて3つの関数からなるウィッシュリストに変換されます。

- ワールド状態を画像に変換する関数： ;ShotWorld ->Image;adds the image of a shot for each y onw;at (MID,y) to the background image(define (to-image w) BACKGROUND)問題が視覚的な描画を要求しているため。
- ティックイベントを処理するための関数： ;ShotWorld ->ShotWorld;moves each shot onw up by one pixel(define (tock w) w)
- そして、キーイベントを処理するための関数：;ShotWorldKeyEvent ->ShotWorld;adds a shot to the world;if the player presses the space bar(define (keyh w ke) w)



最初のウィッシュリストに加えて、実際にワールドをセットアップしハンドラを組み込む main 関数も定義する必要があることを忘れないでください。figure 61 には、設計されたものではなく標準的なスキーマの修正として定義されたこの関数の 1 つが含まれています。

to-image の設計から始めましょう。シグネチャ、目的ステートメント、およびヘッダーが揃っているの、次は例が必要です。データ定義には 2 つの節があるため、少なくとも 2 つの例が存在するはずです。'()' と、例えば (cons 9'()) のような cons されたリストです。'()' に対する期待される結果は明らかに BACKGROUND です。しかし、y 座標が存在する場合、関数は弾の画像を MID と指定された座標に配置しなければなりません。

```
(check-expect (to-image (cons 9 '()))
 (place-image SHOT XSHOTS 9 BACKGROUND))
```

読み進める前に、2 つの Shot のリストに to-image を適用する例を手作業で解いてみてください。そうすることで、関数がどのように動作するかを理解するのに役立ちます。

第 4 ステップは、データ定義をテンプレートに変換することです。

```
; ShotWorld -> Image
(define (to-image w)
 (cond
 [(empty? w)...]
 [else
 (... (first w)... (to-image (rest w))...)]))
```

リストのデータ定義に対するテンプレートはすでにおなじみのものであるため、詳細な説明は不要でしょう。疑問がある場合は、figure 52 の質問を読み返し、自分自身でテンプレートを設計してみてください。

ここから関数を定義するのは簡単です。重要なのは、例とテンプレートを組み合わせ、figure 53 の質問に答えることです。それらに従い、まず空の弾リストというベースケースから始めると、例から期待される答えが BACKGROUND であることがわかります。次に、2 番目の cond のテンプレート式が何を計算するかを定式化します。

- (first w) はリストから最初の座標を取り出します。
- (rest w) は残りの座標です。
- (to-image (rest w)) は、to-image の目的ステートメントに従って、(rest w) にある各弾を背景画像に追加します。

言い換えると、(to-image (rest w)) はリストの残りの部分を画像として描画するため、作業のほとんどを実行します。不足しているのは最初の弾、(first w) です。ここで、これら 2 つの式に目的ステートメントを適用すると、2 番目の cond 節に必要な式が得られます。

```
(place-image SHOT XSHOTS (first w)
 (to-image (rest w)))
```

追加されるアイコンは弾の標準画像です。2つの座標は目的ステートメントで明確に記述されています。そして place-image への引数の最後は、リストの残りの部分から構築された画像です。

figure 61 には、to-image の完全な関数定義、およびプログラムの残りの部分が表示されています。tock の設計は to-image の設計とまったく同様であるため、ご自身で取り組んでみてください。ただし、keyh ハンドラのシグネチャは1つの興味深い疑問を提起します。このハンドラは、軽微ではないデータ定義を持つ2つの入力を消費すると規定されています。一方では ShotWorld が自己参照的なデータ定義であり、他方では KeyEvents の定義が大きな列挙型です。今のところは、2つの引数のうちどちらがテンプレートの開発を主導すべきかを「推測」してもらいます。後ほど、このようなケースについて深く学習します。

```
+-----+
| Figure 61: A list-based world program |
| |
| ; ShotWorld -> ShotWorld |
| (define (main w0) |
| (big-bang w0 |
| [on-tick tock] |
| [on-key keyh] |
| [to-draw to-image])) |
| |
| ; ShotWorld -> ShotWorld |
| ; moves each shot up by one pixel |
| (define (tock w) |
| (cond |
| [(empty? w) '()] |
| [else (cons (sub1 (first w)) (tock (rest w)))])) |
| |
| ; ShotWorld KeyEvent -> ShotWorld |
| ; adds a shot to the world if the space bar is hit |
| (define (keyh w ke) |
| (if (key=? ke " ") (cons HEIGHT w) w)) |
| |
| ; ShotWorld -> Image |
| ; adds each shot y on w at (XSHOTS,y) to BACKGROUND |
| (define (to-image w) |
| (cond |
| [(empty? w) BACKGROUND] |
| [else (place-image SHOT XSHOTS (first w) |
| (to-image (rest w)))])) |
| |
+-----+
```

ワールドプログラムに関して言えば、keyh のようなキーハンドラは、それが消費するキーイベントに関係しています。したがって、それを主引数とみなし、そのデータ定義を使用してテンプレートを導出します。具体的には、『列挙型 (Enumerations)』で示した



KeyEvent のデータ定義に従うと、この関数には次のような多数の節を持つ cond 式が必要であることが決定付けられます。

```
(define (keyh w ke)
 (cond
 [(key=? ke "left")...]
 [(key=? ke "right")...]
 ...
 [(key=? ke " ")...]
 ...
 [(key=? ke "a")...]
 ...
 [(key=? ke "z")...]))
```

もちろん、すべての可能な BSL 値を消費する関数と同様に、キーハンドラが通常すべての可能なケースを検査する必要はありません。現在の問題では、キーハンドラがスペースキーにのみ反応し、他のすべてのキーは無視されることが具体的にわかっています。したがって、” ” の節を除き、すべての cond 節を else 節に折りたたむのが自然です。

**Exercise 156.** figure 61 のプログラムにテストを追加し、それらのテストに合格することを確認しなさい。main が何を行うかを説明しなさい。その後、main を介してプログラムを実行しなさい。

**Exercise 157.** 定数に関する任意の決定が容易に変更可能かどうかを実験して判断しなさい。例えば、単一の定数定義を変更するだけで、意図した結果が得られるかどうかを判断しなさい。

- キャンバスの高さを 220 ピクセルに変更する。
- キャンバスの幅を 30 ピクセルに変更する。
- 弾のラインの x 位置を「中央よりやや左側のどこか」に変更する。
- 背景を緑色の長方形に変更する。
- 弾の描画を赤色の細長い長方形に変更する。

また、他のものを何も変更せずに弾のサイズを 2 倍にすることや、その色を黒に変更することが可能かどうかを確認しなさい。

**Exercise 158.** main を実行し、スペースキーを押して（弾を発射し）、十分な時間待つと、弾はキャンバスから消えます。しかし、ワールドのキャンバスを終了すると、その結果には依然としてこの見えない弾が含まれているワールドが得られます。

1 クロックティックあたり弾を 1 ピクセル移動させるだけでなく、座標がキャンバスの上外に出た弾を除外する代替の tock 関数を設計しなさい。ヒント: 再帰的な cond 節のための補助関数の設計を検討するとよいでしょう。

**Exercise 159.** exercise 153 の解法をワールドプログラムに変換しなさい。riot と名付けられたその main 関数は、学生が投げたい風船の数を消費します。その視覚化は、1 秒間

に 1 個の割合で風船が次々と落下する様子を示します。この関数は風船が当たった Posn のリストを生成します。

ヒント (1) 可能なデータ表現の 1 つを以下に示します：

```
(define-struct pair [balloon# lob])
; A Pair is a structure (make-pair N List-of-posns)
; A List-of-posns is one of:
; - '()
; - (cons Posn List-of-posns)
; interpretation (make-pair n lob) means n balloons
; must yet be thrown and added to lob
```

(2) big-bang 式は実際には単なる式です。それを別の式の中にネストすることは正当です。

(3) random が乱数を生成することを思い出さないでください。それを別の式の中にネスト（入れ子）にすることは正当です。

(4) random は乱数を生成することを思い出してください。

## 9.6 リストと集合についての注意 (A Note on Lists and Sets)

本書は、集合を BSL 値のコレクションとする直感的な理解に依存しています。「データの宇宙 (The Universe of Data)」では特に、データ定義は BSL 値の集合に名前を導入するものであると述べています。本書が集合に関して一貫して投げかける問いが 1 つあり、それは「ある要素が与えられたある集合に含まれるかどうか」です。たとえば、4 は Number に含まれますが、“four” は含まれません。また、本書では、ある値が特定の名前付き集合の要素 (member) であるかどうかをチェックするためにデータ定義を使用する方法や、データ定義の一部を使用して集合のサンプル要素を生成する方法も示していますが、これら 2 つの手続きはデータ定義に関するものであり、集合そのものに関するものではありません。

同時に、リストも値のコン。

それと同時に、リストもまた値の集まりを表現します。したがって、リストと集合の違いは何なのか、あるいはこれは不要な区別なのではないか疑問に思っているかもしれません。もしそうなら、この節はあなたのためのものです。

現時点での集合とリストの主な違いは、前者がコードの設計におけるステップを議論するために使用する概念であるのに対し、後者は私たちが選択したプログラミング言語である BSL における多数のデータ形式の 1 つである点です。これら 2 つのアイデアは、私たちの議論においてかなり異なるレベルに位置しています。しかし、データ定義が BSL 内部の実際の情報のデータ表現を導入すること、そして集合が情報の集まりであることを考えると、集合が BSL 内部でデータとしてどのように表現されるのかという疑問が湧くかもしれません。

**注:** 多くの本格的なプログラミング言語は、リストと集合の両方のデータ表現を直接サポートしています。

BSL においてリストは特別な地位を占めていますが、集合にはそれがありません。しかし同時に、集合はどこかリストに似ています。主な違いは、プログラムが通常どちらかの形式のデータに対して使用する関数の種類にあります。BSL はリストに対していくつかの基本的な定数や関数——たとえば `empty`、`empty?`、`cons`、`cons?`、`first`、`rest`——を提供しており、また自分で定義できる関数もいくつか存在します——たとえば `member?`、`length`、`remove`、`reverse` などです。以下は、自分で定義できるものの BSL には標準で付属していない関数の例です。

```
; List-of-string String -> N
; determines how often s occurs in los
(define (count los s)
 0)
```

手を止めましょう！(Stop!) この関数の設計を完成させてください。

素直で、もしかすると単純すぎる方法で進め、集合とは基本的にリストであると仮定してみましょう。そして、さらに単純化するために、この節では数値のリストに注目することにします。ここで、数値が集合の一部であるか否かだけが重要であると受け入れるならば、集合を表現するためにリストを 2 つの異なる方法で使用できることがほぼ直ちに明らかになります。

```
+-----+
| Figure 62: Two data representations for sets |
| |
| +-----+ +-----+ |
| |; A Son.L is one of: 0; - empty 0 ... | |; A Son.R is one of: 0; - empty 0 |
| | ... | |
| +-----+ +-----+ |
| |;ASon.L is one of: | | |
| |;-empty | | |
| |;- (cons Number Son.L) | | |
| |; | | |
| |;Son is used when it | | |
| |;applies toSon.L andSon.R | | |
| |;ASon.R is one of: | | |
| |;-empty | | |
| |;- (cons Number Son.R) | | |
| |; | | |
| |;Constraint Ifs is aSon.R, | | |
| |;no number occurs twice ins | | |
| +-----+ +-----+ |
+-----+
```

図 62 (Figure 62) は 2 つのデータ定義を示しています。どちらも基本的には、集合が数値のリストとして表現されることを述べています。違いは、右側の定義には、リスト上に

どちらの定義を選択したとしても、すでに2つの重要な概念を定義することができます。

最初のは空集合 (empty set) であり、どちらの場合も空リスト (empty list) によって表現されます。2 つ目は要素判定 (membership test) です。

```
(cons 1 (cons 2 (cons 3 '())))
```

```
(cons 2 (cons 1 (cons 3 '())))
```

```
(cons 1 (cons 2 (cons 1 (cons 3 '()))))
```

```
Figure 63: Functions for the two data representations of sets
```

Left Representation	Right Representation
<pre> ; Number Son.L -&gt; Son.L ↵; removes...  </pre>	<pre> ; Number Son.R -&gt; Son.R ↵;</pre>
<pre>removes...  </pre>	
<pre> ;NumberSon.L -&gt;Son.L      </pre>	
<pre> ;removesx froms      </pre>	
<pre>  (define s1.L      </pre>	
<pre>  (cons 1 (cons 1 '()))      </pre>	
<pre>  (check-expect      </pre>	
<pre>  (set-.L 1 s1.L) es)      </pre>	
<pre>  (define (set-.L x s)      </pre>	

```

| | (remove-all x s)) | | |
| | ;NumberSon.R ->Son.R | | |
| | ;removesx froms | | |
| | (define s1.R | | |
| | (cons 1 '())) | | |
| | (check-expect | | |
| | (set-.R 1 s1.R) es) | | |
| | (define (set-.R x s) | | |
| | (remove x s)) | | |
| +-----+-----+
+-----+-----+

```

2つのデータ定義の違いは、関数を設計する際に現れます。たとえば、集合から数値を削除する関数が必要だとしましょう。両方の表現に適用できるウィッシュリストのエントリは以下のとおりです。

```

; Number Son -> Son
; subtracts x from s
(define (set- x s)
 s)

```

目的のステートメント (purpose statement) で「引く (subtract)」という言葉が使われているのは、論理学者や数学者が集合を扱う際にこの言葉を使用するからです。

図 63 (Figure 63) はその結果を示しています。2つの列は次の2つの点で異なります。

1. 左側のテストは1を2回含むリストを使用していますが、右側のテストは同じ集合を単一の cons で表現しています。
2. これらの違いにより、左側の set- は remove-all を使用しなければなりませんが、右側の set- は remove だけで済みます。

手を止めましょう！ (Stop!) コードを DrRacket の定義領域にコピーし、テストが合格することを確認してください。その後、読み進めながらコードで実験してみてください。

図 63 (Figure 63) の好ましくない点は、テストで期待される結果としてプレーンなリストである es を使用していることです。この問題は一見すると些細なことに思えるかもしれませんが、次の例を考えてみてください。

```
(set- 1 set123)
```

ここで set123 は、1、2、および3を含む集合を以下の2つの方法のいずれかで表現しています。

```

(define set123-version1
 (cons 1 (cons 2 (cons 3 '()))))

(define set123-version2
 (cons 1 (cons 3 (cons 2 '()))))

```

どちらの表現を選択したとしても、(set- 1 set123) はこれら 2 つのリストのいずれかに評価されます。

```
(define set23-version1
 (cons 2 (cons 3 '())))
```

```
(define set23-version2
 (cons 3 (cons 2 '())))
```

しかし、set- がそれら 2 つのどちらを生成するかを予測することはできません。

2 つの選択肢というシンプルなケースでは、次のように check-member-of テスト機能を使用することが可能です。

```
(check-member-of (set- .v1 1 set123.v1)
 set23-version1
 set23-version2)
```

期待される集合が 3 つの要素を含む場合、左側のデータ定義が許可する重複を伴う表現を除いたとしても、6 通りのバリエーションが存在します。

この問題を解決するには、2 つのアイデアを組み合わせる必要があります。第 1 に、set- は本質的に、指定された要素が結果の中に現れないことを保証するためのものであるということを思い出してください。これは、例をテストへと変換する私たちのやり方では伝わらないアイデアです。第 2 に、BSL の check-satisfied テスト機能を使用すれば、まさにこのアイデアを正確に記述することができます。

「合間 1 初等学生言語 (Intermezzo 1: Beginning Student Language)」では check-satisfied について簡単に触れていますが、かいつまんで言えば、この機能はある式が特定の性質 (property) を満たしているかどうかを判定します。性質とは、値から Boolean への関数です。今回の具体的なケースでは、1 が特定の集合のメンバではないことを表明したいと思います。

```
; Son -> Boolean
; #true if 1 is not a member of s; #false otherwise
(define (not-member-1? s)
 (not (in? 1 s)))
```

not-member-1? を使用すると、テストケースを次のように定式化できます。

```
(check-satisfied (set- 1 set123) not-member-1?)
```

そして、このバリエーションは関数が何を達成することになっているのかを明確に示しています。さらに都合が良いことに、この定式化は入力または出力の集合がどのように表現されているかにまったく依存しません。

要約すると、リストと集合はどちらも値の集まりに関するものであるという点で関連していますが、大きく異なる点もあります。

性質 (property)	リスト (lists)	集合 (sets)
メンバ関係 (membership)	多数の中の 1 つ (one among many)	極めて重要 (critical)
順序 (ordering)	極めて重要 (critical)	無関係 (irrelevant)
出現回数 (# of occurrences)	意味を持つ (sensible)	無関係 (irrelevant)
サイズ (size)	有限だが任意 (finite but arbitrary)	有限または無限 (finite or infinite)

この表の最後の行は、新しいアイデアを提示していますが、同時に明白なものでもあります。本書で言及されている集合の多くは無限に大きく、たとえば `Number`、`String`、そして `List-of-strings` などがあります。対照的に、リストは任意に多数の項目を含むことができますが、常に有限です。

要約すると、この節では集合とリストの本質的な違いと、有限集合を有限リストで表現する 2 つの異なる方法について説明しました。BSL は無限集合を表現できるほど表現力が高くありません。演習問題 299 (exercise 299) では、無限集合にも対処できる、まったく異なる集合の表現方法を導入します。ただし、実際のプログラミング言語がどのように集合を表現するかという問題は、本書の範囲を超えています。

**Exercise 160.** 左側および右側のデータ定義に対して、それぞれ与えられた集合 `s` に数値 `x` を追加することで集合を作成する関数 `set+.L` および `set+.R` を設計しなさい。

## 10 リストのさらなる活用 (More on Lists)

リストは、現在のほぼすべてのプログラミング言語に備わっている多用途なデータ形式です。プログラマは、大規模なアプリケーション、人工知能、分散システムなどを構築するためにリストを使用してきました。本章では、リストを生成する関数、リストの内部に構造体を必要とするデータ表現、テキストファイルをリストとして表現する方法など、この世界からいくつかのアイデアを取り上げて説明します。

### 10.1 リストを生成する関数 (Functions that Produce Lists)

以下は、時給制従業員の賃金を決定するための関数です。

```
; Number -> Number
; computes the wage for h hours of work
(define (wage h)
 (* 12 h))
```

この関数は、働いた時間を入力として受け取り、賃金を算出します。しかし、給与計算ソフトウェアを使用したい企業は、この単一の関数には関心がありません。企業が求めているのは、すべての従業員の賃金を計算する関数です。

この新しい関数を `wage*` と呼びましょう。その役割は、すべての従業員の就労時間を処理し、それぞれに支払われるべき賃金を算出することです。簡単のため、入力はいくつかの従業員の就労時間を表す数値のリストであり、期待される結果も各数値が獲得した週給を表す数値のリストであると仮定しましょう。

入力と出力のデータ定義はすでに手元にあるため、直ちに設計レシピの第2段階へ進むことができます。

```
; List-of-numbers -> List-of-numbers
; computes the weekly wages for the weekly hours
(define (wage* whrs)
 '())
```

次に、いくつかの入力の例と、それに対応する出力が必要になります。そこで、週の就労時間を表すいくつかの短い数値のリストを作成します。

```
given expected '() '() (cons28'()) (cons336'()) (cons4(cons2'())) (cons48(cons24'()))
```

出力を計算するために、与えられた入力リストにある各数値に対して週給を求めます。最初の例では、入力リストに数値が存在しないため、出力は `'()` となります。2番目と3番目の期待される出力がなぜそのようなになるのか、理解できているか確認してください。

`wage*` は「リスト」の章の他のいくつかの関数と同じ種類のデータを消費すること、そしてテンプレートはデータ定義の形状のみに依存することから、このテンプレートを再利用できます。

```
(define (wage* whrs)
 (cond
```



```

 [(empty? whrs)...]
 [else (... (first whrs)...
... (wage* (rest whrs))...))]))

```

テンプレート開発の練習をしたい場合は、figure 52 の質問を活用してください。

それでは、最も創造的な設計段階に進みましょう。設計レシピに従い、テンプレートの各 cond 行を個別に検討します。非再帰的なケースでは、(empty? whrs) が真になります。これは入力が '()' であることを意味します。上記の例によって望ましい答えは '()' と定義されているため、これで完了です。

2 番目のケースでは、設計上の質問に従ってテンプレートの各式が何を計算するかを明らかにします。

- (first whrs) は whrs の最初の数値を返します。これは最初に従業員が働いた時間です。
- (rest whrs) は与えられたリストの残りの部分です。
- (wage\* (rest whrs)) は、残りのリストがまさに今定義している関数自身によって処理されることを示しています。いつものように、この式の導き出す結果を把握するために、関数のシグネチャと目的文を利用します。シグネチャからはそれが数値のリストであることがわかり、目的文からは、このリストが入力（時間のリストの残り）に対する賃金のリストを表していることが説明されます。

重要なのは、関数がまだ定義されていなくても、このケースにおける結果を計算する式を組み立てる際に、これらの事実を依拠することです。

whrs の最初の要素を除くすべての賃金のリストはすでに手元にあるため、関数は whrs 全体に対する期待される出力を生成するために 2 つの計算を実行する必要があります。(first whrs) に対する週給を計算することと、whrs のすべての週給を表すリストを構築することです。最初の部分には wage を再利用します。2 番目の部分には、これら 2 つの情報を cons によって 1 つのリストに結合します。

```

(cons (wage (first whrs)) (wage* (rest whrs)))

```

これで定義は完了です。figure 64 を参照してください。

```

+-----+
| Figure 64: Computing the wages of all employees |
| |
| ; List-of-numbers -> List-of-numbers |
| ; computes the weekly wages for all given weekly hours |
| (define (wage* whrs) |
| (cond |
| [(empty? whrs) '()] |
| [else (cons (wage (first whrs)) (wage* (rest whrs)))])) |
| |
| ; Number -> Number |
| ; computes the wage for h hours of work |
| (define (wage h) |

```

| (\* 12 h)) |  
+-----+

**Exercise 161.** 例をテストに変換し、それらがすべて成功することを確認しなさい。次に、figure 64 の関数を変更して、全員の時給が \$14 になるようにしなさい。ここで、全員の時給を変更する作業が、プログラム全体の複数箇所の変更ではなく単一の変更で済むように、プログラム全体を改訂しなさい。

**Exercise 162.** 週に 100 時間を超えて働く従業員はおそらく存在しません。不正から会社を守るため、関数は wage\* の入力リストのどの要素も 100 を超えていないかチェックする必要があります。もし超えているものがあれば、関数は直ちにエラーを発生させるべきです。この基本的な妥当性チェックを実行したい場合、figure 64 の関数をどのように変更する必要がありますか。

**Note:** 設計レシピのさまざまな段階の成果物を示しなさい。行き詰まった場合は、設計レシピに従ってどこまで進んだかを誰かに見せてください。レシピは単にあなたが使うための設計ツールであるだけでなく、他の人があなたの自助を支援できるようにするための診断システムでもあります。

**Exercise 163.** convertFC を設計しなさい。この関数は、華氏 (Fahrenheit) の測定値のリストを摂氏 (Celsius) の測定値のリストに変換します。

**Exercise 164.** 米ドル (US\$) の金額のリストをユーロ (€) の金額のリストに変換する関数 convert-euro を設計しなさい。現在の為替レートをウェブで調べなさい。

convert-euro を関数 convert-euro\* に一般化しなさい。この関数は為替レートと米ドルの金額のリストを受け取り、後者をユーロの金額のリストに変換します。

**Exercise 165.** おもちゃの説明 (単語 1 つの文字列) のリストを受け取り、すべての “robot” の出現を “r2d2” に置換し、他のすべての説明はそのまま残す関数 subst-robot を設計しなさい。

subst-robot を substitute に一般化しなさい。後者は new と old という 2 つの文字列と、文字列のリストを受け取ります。old のすべての出現を new に置換した新しい文字列のリストを生成します。ます。

## 10.2 リスト内の構造体 (Structures in Lists)

労働時間を単なる数値として表現するのは悪い選択です。なぜなら、給料小切手を印刷するには 1 週間あたりの労働時間以上の情報が必要だからです。また、すべての従業員が 1 時間あたり同じ金額を稼ぐわけではありません。幸いなことに、リストにはアトム値以外の項目を含めることができます。実際、リストには私たちが望むあらゆる値を含めることができ、特に構造体を含めることができます。

私たちの実行例では、まさにそのようなデータ表現が求められています。数値の代わりに、従業員と彼らの労働時間および時給を表す構造体を使用します：

```
(define-struct work [employee rate hours])
; A (piece of) Work is a structure:
; (make-work String Number Number)
; interpretation (make-work n r h) combines the name
; with the pay rate r and the number of hours h
```

この表現はまだ単純ですが、構造体を含むリストのデータ定義を公式化することを強いるため、追加の課題としてちょうど十分なものです：

```
; Low (short for list of works) is one of:
; - '()
; - (cons Work Low)
; interpretation an instance of Low represents the
; hours worked for a number of employees
```

以下は Low の 3 つの要素です：

```
'()
(cons (make-work "Robby" 11.95 39)
 '())
(cons (make-work "Matthew" 12.95 45)
 (cons (make-work "Robby" 11.95 39)
 '()))
```

データ定義を使用して、なぜこれらのデータが Low に属しているのかを説明しなさい。止まってください！ また、データ定義を使用して、さらに 2 つの例を生成しなさい。

**注:** 実際のプロジェクトで作業する場合、このような接尾辞は使用しません。代わりに、コードの異なるバージョンを管理するためのツールを使用します。

Low の定義が意味をなすことがわかったので、単なる数値のリストではなく Low の要素を受け取るように関数 wage\* を再設計する 때가 来ました：

```
; Low -> List-of-numbers
; computes the weekly wages for the given records
(define (wage*.v2 an-low)
 '())
```

関数名の末尾にある接尾辞「.v2」は、これが関数の改訂された第 2 版であることをコードを読むすべての人に知らせます。この場合、改訂は新しいシグネチャと適合された目的の文 (purpose statement) から始まります。ヘッダーは上記と同じです。

デザインレシピの第 3 ステップは、例を使って考えることです。上記の 2 番目のリストから始めましょう。それには 1 つの work レコード、すなわち (make-work “Robby” 11.95 39) が含まれています。その解釈は、“Robby” が 39 時間働き、時給 \$11.95 で支払われるということです。したがって、彼のその週の賃金は \$466.05、つまり (\* 11.95 39) です。したがって、wage\*.v2 の期待される結果は (cons 466.05 '()) です。当然、入力リストに 2 つの work レコードが含まれている場合、この種の計算を 2 回実行し、結果は 2 つの数値の

リストになります。止まってください！上記の3番目のデータ例に対する期待される結果を決定しなさい。

**数値に関する注意** BSL は——他のほとんどのプログラミング言語とは異なり——あなたと同じように、小数を正確な分数として理解することを覚えておいてください。たとえば Java のような言語は、最初の work レコードの期待される賃金に対して 466.04999999999995 を生成します。小数に対する演算がいつこのような奇妙な振る舞いをするかを予測することはできないため、他のプログラミング言語に備えるためにも、そのような例は次のように記述する方がよいでしょう：

```
(check-expect
 (wage*.v2
 (cons (make-work "Robby" 11.95 39) '())))
 (cons (* 11.95 39) '()))
```

また、このスタイルで例を記述することは、賃金の計算方法を本当に理解したことも意味します。**注記終わり**

ここからテンプレートの開発に移ります。テンプレートの質問を使用すると、すぐに次のようになります：

```
(define (wage*.v2 an-low)
 (cond
 [(empty? an-low)...]
 [(cons? an-low)
 (... (first an-low)...
 ... (wage*.v2 (rest an-low))...)]))
```

なぜなら、データ定義は2つの節から構成され、最初の節で '() を導入し、2番目の節で cons された構造体を導入する、といった具合だからです。しかし、このテンプレートが表現している以上に、入力について多くのことを知っていることにも気づくでしょう。たとえば、(first an-low) が与えられたリストから3つのフィールドを持つ構造体を抽出することを知っています。これは、テンプレートにさらに3つの式を追加することを示唆しているようです：

```
(define (wage*.v2 an-low)
 (cond
 [(empty? an-low)...]
 [(cons? an-low)
 (... (first an-low)...
 (work-employee (first an-low))...
 (work-rate (first an-low))...
 (work-hours (first an-low))...
 (wage*.v2 (rest an-low))...)]))
```

このテンプレートは、関心を持つ可能性のあるすべてのデータをリストアップしています。

ここでは異なる戦略を使用します。具体的には、他のデータ定義を参照するデータ定義のテンプレートを開発するときにはいつでも、個別の関数テンプレートを作成して参照することを提案します：

```
(define (wage*.v2 an-low)
 (cond
 [(empty? an-low)...]
 [(cons? an-low)
 (... (for-work (first an-low))
 ... (wage*.v2 (rest an-low))...)]))

; Work ->???
; a template for processing elements of Work
(define (for-work w)
 (... (work-employee w)...
 ... (work-rate w)...
 ... (work-hours w)...))
```

テンプレートを分割することで、作業が自然に関数内および関数間に分割されます。どの関数も大きくなりすぎず、すべてが特定のデータ定義に関連付けられます。

いよいよプログラミングの準備が整いました。いつものように、シンプルに見えるケース、つまりここでの最初の cond 行から始めます。wage\*.v2 が '()' に適用された場合、'()' が返されることを期待し、これで決まりです。次に 2 行目に進み、これらの式が何を計算するかを自分に思い出させます：

1. (first an-low) はリストから最初の work 構造体を抽出します。
2. (for-work...) は、work 構造体を処理する関数を設計したいことを示しています。
3. (rest an-low) は与えられたリストの残りを抽出します。そして
4. (wage\*.v2 (rest an-low)) は、関数の目的文に従って、最初のを除くすべての work レコードの賃金リストを決定します。

ここで行き詰まった場合は、figure 54 の表の方法を使用してください。

すべてを理解していれば、for-work が最初の work レコードの賃金を計算すると仮定して、2つの式を cons するだけで十分であることがわかります：

```
... (cons (for-work (first an-low))
 (wage*.v2 (rest an-low)))...
```

要するに、関数のウィッシュリストに別のエントリを追加することで、関数を完了したことになります。

for-work は単に代理として機能する名前であり、この関数としては悪い名前であるため、この関数を wage.v2 と呼び、その完全なウィッシュリストのエントリを書き留めましょう：

```
; Work -> Number
; computes the wage for the given work record w
(define (wage.v2 w)
 0)
```

この種の関数の設計は Fixed-Size Data で広範囲に扱われているため、ここで追加の説明は必要ありません。Figure 65 は wage.v2 と wage\*.v2 を開発した最終結果を示しています。

```
+-----+
| Figure 65: Computing the wages from work records |
| |
| ; Low -> List-of-numbers |
| ; computes the weekly wages for all weekly work records |
| |
| (check-expect |
| (wage*.v2 (cons (make-work "Robby" 11.95 39) '())) |
| (cons (* 11.95 39) '())) |
| |
| (define (wage*.v2 an-low) |
| (cond |
| [(empty? an-low) '()] |
| [(cons? an-low) (cons (wage.v2 (first an-low)) |
| (wage*.v2 (rest an-low)))])) |
| |
| ; Work -> Number |
| ; computes the wage for the given work record w |
| (define (wage.v2 w) |
| (* (work-rate w) (work-hours w))) |
+-----+
```

**Exercise 166.** 関数 wage\*.v2 は work レコードのリストを受け取り、数値のリストを生成します。もちろん、関数は構造体のリストを生成することもできます。

給料小切手 (paycheck) のデータ表現を開発しなさい。給料小切手には、従業員の名前と金額という2つの特徴的な情報が含まれていると仮定します。次に関数 wage\*.v3 を設計しなさい。この関数は work レコードのリストを受け取り、それからレコードごとに1つの給料小切手のリストを計算します。

現実には、給料小切手には従業員番号も含まれています。従業員情報のデータ表現を開発し、work レコードのデータ定義を変更して、従業員の名前のための単なる文字列ではなく従業員情報を使用するようにしなさい。また、給料小切手のデータ表現も変更して、従業員の名前と番号も含まれるようにしなさい。最後に、改訂された work レコードのリストを改訂された給料小切手のリストにマッピングする関数 wage\*.v4 を設計しなさい。

**反復的洗練に関する注意** この演習はタスクの反復的洗練 (iterative refinement) を示しています。関連するすべての情報を含むデータ表現を使用する代わりに、給料小切手の単純化された表現から始めて、徐々にその表現を現実的なものにしていきました。この単純

なプログラムの場合、洗練は過剰 (overkill) ですが、後で反復的洗練が単なる選択肢ではなく必要不可欠となる状況に遭遇することになります。

**Exercise 167.** 関数 `sum` を設計しなさい。この関数は `Posns` のリストを受け取り、そのすべての `x` 座標の合計を生成します。

**Exercise 168.** 関数 `translate` を設計しなさい。この関数は `Posns` のリストを受け取り、`Posns` のリストを生成します。前者の中の各 `(make-posn x y)` に対して、後者には `(make-posn x (+ y 1))` が含まれます。「`translate` (平行移動する)」という言葉は幾何学から借りたもので、直線に沿って一定の距離だけ点を移動させることを平行移動 (translation) と呼びます。

**Exercise 169.** 関数 `legal` を設計しなさい。演習 168 の `translate` と同様に、この関数は `Posns` のリストを受け取り、`Posns` のリストを生成します。結果には、`x` 座標が 0 から 100 の間にあり、`y` 座標が 0 から 200 の間にあるすべての `Posns` が含まれます。

**Exercise 170.** 以下は電話番号を表現する 1 つの方法です：

```
(define-struct phone [area switch four])
; A Phone is a structure:
; (make-phone Three Three Four)
; A Three is a Number between 100 and 999.
; A Four is a Number between 1000 and 9999.
```

関数 `replace` を設計しなさい。この関数は `Phones` のリストを受け取り、`Phones` のリストを生成します。市外局番 (area code) 713 のすべての出現を 281 に置き換えます。`y` 座標が 0 から 200 の間にあるすべての `Posn` が含まれます。

**Exercise 170.** 以下は電話番号を表現する一つの方法です。

```
(define-struct phone [area switch four])
; A Phone is a structure:
; (make-phone Three Three Four)
; A Three is a Number between 100 and 999.
; A Four is a Number between 1000 and 9999.
```

関数 `replace` を設計してください。この関数は `Phone` のリストを受け取り、`Phone` のリストを生成します。市外局番 (area code) 713 のすべての出現を 281 に置き換えます。

### 10.3 リストの中のリスト、ファイル (Lists in Lists, Files)

「関数とプログラム」(Functions and Programs) では、テキストファイル全体を 1 つの文字列として読み込む関数である `read-file` を紹介します。言い換えれば、`read-file` の作成者はテキストファイルを文字列として表現することを選択し、この関数は (名前で指定された) 特定のファイルに対するデータ表現を作成します。しかし、テキストファイルは単なる長いテキストや文字列ではありません。それらは行や単語、行 (ロー) やセル、その他多くの方法で構成されています。要するに、ファイルの内容を単なる文字列として表現することは、稀な機会には機能するかもしれませんが、通常は悪い選択です。



**注:** 定義領域に ( require 2htdp/batch-io ) を追加してください。

```
+-----e +
| Figure
| 6: Things ke time |
|
| +-----+
| | `TTT`
|
| +-----+
| | `Put up in a place`
| | `where it's easy to see`
| | `the cryptic admonishment`
| | `When you feel how depressingly`
| | `slowly you climb`,`
| | `it's well to remember that`
| | `Things Take Time.`
| | `Piet Hein`
|
| +-----+
+-----+

+-----+
| Figure 66: Things take time
|
| +-----+
| | `TTT`
|
| +-----+
| | `Put up in a place`
| | `where it's easy to see`
| | `the cryptic admonishment`
| | `T.T.T.`
| | `When you feel how depressingly`
| | `slowly you climb`,`
| | `it's well to remember that`
| | `Things Take Time.`
| | `Piet Hein`
|
| +-----+
+-----+
```

具体的に説明するために、figure 66 のサンプルファイルを見てみましょう。ここには Piet Hein による詩が含まれており、多くの行と単語で構成されています。プログラム

```
(read-file "ttt.txt")
```

を使ってこのファイルを BSL 文字列に変換すると、次のようになります：

**Note:** ご想像の通り、ドットは実際の結果の一部ではありません。

```
"TTT\n\nPut up in a place\nwhere...."
```

ここで文字列内の "\n" は改行を示しています。



文字列に対する基本操作 (例えば `explode`) でこの文字列を分割することは確かに可能です  
が、BSL を含むほとんどのプログラミング言語は、ファイルの多様な表現や、既存のファ  
イルからそうした表現を作成する関数をサポートしています：

- このファイルを表現する 1 つの方法は行のリストであり、各行は 1 つの文字列として  
表現されます： `(cons "TTT" (cons "" (cons "Put up in a place" (cons... '()))))` ここで  
は、ファイルに空行が含まれているため、リストの 2 番目の項目は空文字列になってい  
ます。
- 別の方法は単語のリストを使用することであり、ここでも各単語は文字列として表現さ  
れます： `(cons "TTT" (cons "Put" (cons "up" (cons "in" (cons... '()))))` この表現では  
空の 2 行目が消失していることに注目してください。結局のところ、空行には単語が存  
在しないからです。
- そして 3 番目の表現は、単語のリストのリストに基づいています： `(cons (cons "TTT"`  
`'()) (cons'() (cons (cons "Put" (cons "up" (cons... '())) (cons... '()))))` この表現は 2 番  
目の表現に対して、2 行目が空行であることも含めてファイルの構成を保存できるという  
利点があります。その代償として、リストが突然リストを含むことになります。

リストを含むリストというアイデアは最初は恐ろしく思えるかもしれませんが、心配す  
る必要はありません。デザインレシピはこのような複雑な場合でも役立ちます。

```
+-----+
| Figure 67: Reading files |
| | |
| ; String -> String |
| ; produces the content of file f as a string |
| (define (read-file f)...) |
| | |
| ; String -> List-of-string |
| ; produces the content of file f as a list of strings, |
| ; one per line |
| (define (read-lines f)...) |
| | |
| ; String -> List-of-string |
| ; produces the content of file f as a list of strings, |
| ; one per word |
| (define (read-words f)...) |
| | |
| ; String -> List-of-list-of-string |
| ; produces the content of file f as a list of list of |
| ; strings, one list per line and one string per word |
| (define (read-words/line f)...) |
| | |
| ; The above functions consume the name of a file as a String |
| ; argument. If the specified file does not exist in the |
| ; same folder as the program, they signal an error. |
+-----+
```

使い始める前に、figure 67 を見てみましょう。ここには、ファイルを読み込むための便利な関数がいくつか紹介されています。これらは網羅的なものではありません。ファイルからのテキストを扱う方法は他にもたくさんあり、あらゆるテキストファイルを扱うにははるかに多くのことを知る必要があります。ここでの私たちの目的（体系的なプログラムデザインの原則を教え、学ぶこと）においては、これらで十分であり、十分に興味深いプログラムをデザインできるようになります。

figure 67 では、リストを含むリストに関わるものも含め、まだ存在しない2つのデータ定義の名前が使用されています。いつものようにデータ定義から始めますが、今回はこの作業をあなたにお任せします。したがって、先を読み進める前に、以下の練習問題を解いてください。図の意味を完全に理解するには解答が必要であり、解答を自力で導き出さなければ、このセクションの残りの部分を本当に理解することはできません。

**Exercise 171.** List-of-strings のデータ定義がどのようなものか、あなたは知っています。それを明確に書き出してください。Piet Hein の詩を、各行が1つの文字列として表現される定義のインスタンスとして表現できること、および各単語が文字列となる別のインスタンスとして表現できることを確認してください。read-lines と read-words を使用して、表現の選択を確認してください。

次に、List-of-list-of-strings のデータ定義を開発してください。再び Piet Hein の詩を、各行が単語ごとに1つの文字列のリストとして表現され、詩全体がそのような行表現のリストとなる定義のインスタンスとして表現してください。選択を確認するために read-words/line を使用してもかまいません。

ご存知かもしれませんが、オペレーティングシステムにはファイルを測定するプログラムが付属しています。あるものは行数を数え、別のものは行ごとの単語数を特定します。複雑な関数のデザインにおいてデザインレシピがどのように役立つかを説明するために、後者から始めましょう。

最初のステップは、必要なすべてのデータ定義が揃っているか確認することです。上記の練習問題を解いたのであれば、目的の関数のすべての可能な入力に対するデータ定義があり、前節ではすべての可能な出力を記述する List-of-numbers を定義しました。簡潔にするため、文字列のリストのリストのクラスを指すのに LN を使用し、それを使って目的の関数のヘッダー素材を書き出します：

```
; LN -> List-of-numbers
; determines the number of words on each line
(define (words-on-line lls) '())
```

目的の文を1つのフレーズで捉えており適切であるため、この関数を words-on-line と命名します。

しかし、本当に必要なのはデータ例のセットです：

```
(define line0 (cons "hello" (cons "world" '())))
(define line1 '())
```

```
(define lls0 '())
(define lls1 (cons line0 (cons line1 '())))
```

最初の2つの定義は行の2つの例を導入しています。1つは2つの単語を含み、もう1つは何も含みません。最後の2つの定義は、これらの行の例から LN のインスタンスを構築する方法を示しています。関数にこれら2つの例が与えられたときに期待される結果を特定してください。

データ例を用意できれば、機能的な例を定式化するのは簡単です。データ例のそれぞれに関数を適用することを想像するだけです。words-on-line を lls0 に適用すると、行が存在しないため空のリストが返されるはずです。words-on-line を lls1 に適用すると、2つの行があるため、2つの数値のリストが返されるはずです。lls1 の2つの行にはそれぞれ2つと0の単語が含まれているため、2つの数値はそれぞれ2と0です。

これをすべてテストケースに変換する方法は次のとおりです：

```
(check-expect (words-on-line lls0) '())
(check-expect (words-on-line lls1)
 (cons 2 (cons 0 '())))
```

第2ステップの最後にこれを行うことで、実行するとテストケースの一部が失敗するものの、完全なプログラムが得られます。

このサンプル問題において、テンプレートの開発は興味深いステップです。figure 52 のテンプレートの質問に答えることで、通常のリスト処理テンプレートがすぐに得られます：

```
(define (words-on-line lls)
 (cond
 [(empty? lls)...]
 [else
 (... (first lls); a list of strings
 ... (words-on-line (rest lls))...)]))
```

前節と同様に、式 (first lls) が List-of-strings を抽出すること、そしてそれもまた複雑な構成を持っていることを私たちは知っています。この知識を表現するためにネストされたテンプレートを挿入したくなりますが、思い出してほしいように、より良いアイデアは2番目の補助テンプレートを開発し、2番目の条件の最初の行を変更してこの補助テンプレートを参照するようにすることです。

この補助テンプレートはリストを消費する関数のためのものなので、テンプレートは直前のものとほぼ同じに見えます：

```
(define (line-processor ln)
 (cond
 [(empty? ln)...]
 [else
 (... (first ln); a string
 ... (line-processor (rest ln))...)]))
```

重要な違いは、(first ln) がリストから文字列を抽出し、文字列をアトミックな値として扱うことです。このテンプレートを手に入れると、words-on-line の 2 番目のケースの最初の行を次のように変更できます：

```
... (line-processor (first lls))...
```

これは第 5 ステップにおいて、words-on-line の定義が補助関数のデザインを要求する可能性があることを思い出させてくれます。

さあ、プログラミングの時です。いつものように、このステップをガイドするために figure 53 の質問を使用します。行の空リストに関する最初のケースは簡単なケースです。私たちの例によると、この場合の答えは '()', つまり数値の空リストです。cons に関する 2 番目のケースにはいくつかの式が含まれており、それらが何を計算するかの確認から始めます：

- (first ln) は、(表現された) 行の空でないリストから最初の行を抽出します。
- (line-processor (first ln)) は、この行を処理するための補助関数をデザインしたいかもしれないことを示唆しています。
- (rest ln) は、行のリストの残りです。
- (words-on-line (rest ln)) は、リストの残りの行ごとの単語のリストを計算します。なぜこれがわかるのでしょうか？ words-on-line のシグネチャと目的の文でまさにそれを約束したからです。

1 行を消費してその 1 行の単語数をカウントする補助関数 (words# と呼びましょう) をデザインできると仮定すると、2 番目の条件を完了するのは簡単です：

```
(cons (words# (first ln))
 (words-on-line (rest ln)))
```

この式は、ln の最初の行の単語数を、ln の残りの行の単語数を表す数値のリストに cons します。

残るは words# 関数のデザインです。そのテンプレートは line-processor と呼ばれ、その目的は 1 行 (これは単なる文字列のリストです) の単語数をカウントすることです。したがって、ウィッシュリストのエントリは次のようになります：

```
; List-of-strings -> Number
; counts the number of words on los
(define (words# los) 0)
```

この時点で、Designing with Self-Referential Data Definitions で自己参照データのデザインレシピを説明するために使用された例を思い出すかもしれません。その関数は how-many と呼ばれ、これも文字列のリストにある文字列の数をカウントします。how-many の入力名前前のリストを表すことになっていたとしても、この違いはまったく問題になりません。文字列のリストにある文字列の数を正確にカウントする限り、how-many は私たちの問題を解決します。

既存の関数を再利用することは良いことなので、words# を次のように定義できます：

```
(define (words# los)
 (how-many los))
```

しかし実際には、プログラミング言語にはそのような問題をすでに解決する関数が付属しています。BSLはこの関数を `length` と呼び、値が何であるかに関わらず、あらゆる値のリストにある値の数をカウントします。

Figure 68: Counting the words on a line

```
; An LN is one of: | ; - '() | ; - (cons Los LN) | ; interpretation a list of lines,
 each is a list of Strings |
(define line0 (cons "hello" (cons "world" '()))) (define line1'())
(define ln0 '()) (define ln1 (cons line0 (cons line1'())))
; LN -> List-of-numbers | ; determines the number of words on each line |
(check-expect (words-on-line ln0) '()) (check-expect (words-on-line ln1)
 (cons 2 (cons 0'())))
(define (words-on-line ln) (cond [(empty? ln) '()] [else (cons (length (first
 ln)) (words-on-line (rest ln))))])
```

Figure 68: Counting the words on a line

```
; An LN is one of: | ; - '() | ; - (cons Los LN) | ; interpretation a list of lines,
 each is a list of Strings |
(define line0 (cons "hello" (cons "world" '()))) (define line1'())
(define ln0 '()) (define ln1 (cons line0 (cons line1'())))
; LN -> List-of-numbers | ; determines the number of words on each line |
(check-expect (words-on-line ln0) '()) (check-expect (words-on-line ln1)
 (cons 2 (cons 0'())))
(define (words-on-line ln) (cond [(empty? ln) '()] [else (cons (length (first
 ln)) (words-on-line (rest ln))))])
```

Figure 68: Counting the words on a line

```
; An LN is one of: | ; - '() | ; - (cons Los LN) | ; interpretation a list of lines,
each is a list of Strings |

(define line0 (cons "hello" (cons "world" '()))) (define line1 '())

(define ln0 '()) (define ln1 (cons line0 (cons line1 '())))

; LN -> List-of-numbers | ; determines the number of words on each line |
(check-expect (words-on-line ln0) '()) (check-expect (words-on-line ln1)
 (cons 2 (cons 0 '())))

(define (words-on-line ln) (cond [(empty? ln) '()] [else (cons (length (first
 ln)) (words-on-line (rest ln)))]))
```

Figure 68: Counting the words on a line

```
; An LN is one of: | ; - '() | ; - (cons Los LN) | ; interpretation a list of lines,
each is a list of Strings |

(define line0 (cons "hello" (cons "world" '()))) (define line1 '())

(define ln0 '()) (define ln1 (cons line0 (cons line1 '())))

; LN -> List-of-numbers | ; determines the number of words on each line |
(check-expect (words-on-line ln0) '()) (check-expect (words-on-line ln1)
 (cons 2 (cons 0 '())))

(define (words-on-line ln) (cond [(empty? ln) '()] [else (cons (length (first
 ln)) (words-on-line (rest ln)))]))
```

> **Note:** BSL に付属している関数の一覧を見直しておくといよいでしょう。中には馴染みの薄いものに見えるかもしれませんが、今後の問題のどれかで役立つ可能性があります。そうした関数を使用することは、私たちの時間ではなく、あなた自身の時間を節約することになります。

Figure 68 は、サンプル問題に対する完全な設計をまとめたものです。この図には 2 つのテストケースが含まれています。また、独立した関数 `words#` を使う代わりに、`words-on-line` の定義では BSL に付属している `length` 関数を単に呼び出しています。DrRacket でこの定義を実験し、これら 2 つのテストケースが関数定義全体をカバーしていることを確認してください。

小さな一歩を踏み出すことで、最初のファイルユーティリティを設計できるようになります：

```
``racket
; String -> List-of-numbers
; counts the words on each line in the given file
```

```

(define (file-statistic file-name)
 (words-on-line
 (read-words/line file-name)))

; String -> List-of-numbers
; counts the words on each line in the given file
(define (file-statistic file-name)
 (words-on-line
 (read-words/line file-name)))

```

これは単に、ティーチパックの関数と `words-on-line` を合成しているだけです。前者はファイルを `List-of-list-of-strings` として読み込み、その値を後者に渡します。

組み込み関数と新しく設計した関数を合成するというこの考え方は一般的なものです。当然ながら、無作為に関数を設計した上で、その設計を補完するものが選択したプログラミング言語の中に見つかることなど期待しません。代わりに、プログラムの設計者はあらかじめ計画を立て、利用可能な関数が提供する出力に合わせて関数を設計します。さらに一般的に言えば、前述のように、解決策を2つの計算の合成として捉え、一方の計算の結果をもう一方の計算に伝えるための適切なデータコレクションを開発することはよくあることです。ここで、それぞれの計算は関数として実装されます。

```

+-----+
| Figure 69: Encoding strings |
| |
| ; 1String -> String |
| ; converts the given 1String to a 3-letter numeric String |
| |
| (check-expect (encode-letter "z") (code1 "z")) |
| (check-expect (encode-letter "\t") |
| (string-append "00" (code1 "\t"))) |
| (check-expect (encode-letter "a") |
| (string-append "0" (code1 "a"))) |
| |
| (define (encode-letter s) |
| (cond |
| [(>= (string->int s) 100) (code1 s)] |
| [(< (string->int s) 10) |
| (string-append "00" (code1 s))] |
| [(< (string->int s) 100) |
| (string-append "0" (code1 s))])) |
| |
| ; 1String -> String |
| ; converts the given 1String into a String |
| |
| (check-expect (code1 "z") "122") |
| |
| (define (code1 c) |
| (number->string (string->int c))) |
+-----+

```

**Exercise 172.** 行のリスト (list of lines) を 1 つの文字列に変換する関数 `collapse` を設計しなさい。文字列どうしは半角スペース (" ") で区切られる必要があります。行どうしは改行 ("\n") で区切られる必要があります。

チャレンジ：完成したら、次のようにプログラムを使用してください：

```
(write-file "ttt.dat"
 (collapse (read-words/line "ttt.txt")))
```

2 つのファイル "ttt.dat" と "ttt.txt" が同一であることを確認するために、あなたが作成した T.T.T. の詩のバージョンから余分な空白をすべて削除してください。

**Exercise 173.** テキストファイルからすべての冠詞 (articles) を削除するプログラムを設計しなさい。このプログラムはファイル名 `n` を受け取り、ファイルを読み込み、冠詞を削除して、その結果を "no-articles-" に `n` を連結した名前のファイルに出力します。この演習では、冠詞とは "a"、"an"、"the" の 3 つの単語のいずれかを指します。

元のテキストの行と単語への構造が変換後も維持されるように `read-words/line` を使用しなさい。プログラムが設計できたら、Piet Hein の詩に対して実行しなさい。

**Exercise 174.** テキストファイルを数値的に符号化 (encode) するプログラムを設計しなさい。単語内の各文字は、0 から 256 の間の値を持つ 3 文字の数値文字列として符号化されるべきです。Figure 69 は 1 文字に対する私たちの符号化関数を示しています。始める前に、これらの関数について説明しなさい。

ヒント：(1) ファイルの行と単語への構造を保持するために `read-words/line` を使用すること。(2) `explode` についてもう一度よく読んでおくこと。

**Exercise 175.** Unix コマンドの `wc` をシミュレートする BSL プログラムを設計しなさい。このコマンドの目的は、与えられたファイル内の 1String、単語、および行の数を数えることです。すなわち、このコマンドはファイル名を受け取り、3 つの数値からなる値を生成します。(なぜあなたのコンピュータの `wc` は、あなたが作成したコードとわずかに異なる結果を返す可能性があるのでしょうか?)

**Note:** Unix は古いオペレーティングシステムであり、Linux、macOS、そしてある程度現代の Windows によってエミュレートされています。

```
+-----+
| Figure 70: Transpose a matrix |
| |
| ; Matrix -> Matrix |
| ; transposes the given matrix along the diagonal |
| |
| (define wor1 (cons 11 (cons 21 '()))) |
| (define wor2 (cons 12 (cons 22 '()))) |
| (define tam1 (cons wor1 (cons wor2 '()))) |
| |
+-----+
```



```
| (check-expect (transpose mat1) tam1) |
|
| (define (transpose lln) |
| (cond |
| [(empty? (first lln)) '()] |
| [else (cons (first* lln) (transpose (rest* lln)))])) |
+-----+

```

**Exercise 176.** 数学の先生がこれまでに行列 (matrix) の計算を教えてくれたかもしれませんが。原則として、行列とは単に数値の長方形を意味します。以下は行列に対する可能なデータ表現の1つです：

```
; A Matrix is one of:
; - (cons Row '())
; - (cons Row Matrix)
; constraint all rows in matrix are of the same length

; A Row is one of:
; - '()
; - (cons Number Row)

```

行列に関する制約に注意してください。データ定義を熟読し、数 11, 12, 21, 22 からなる  $2 \times 2$  の行列をこのデータ表現に変換しなさい。立ち止まって、このデータの例を自力で考え出すまでは読み進めないでください。

5 秒で解くパズルの解答は以下のとおりです：

```
(define row1 (cons 11 (cons 12 '())))
(define row2 (cons 21 (cons 22 '())))
(define mat1 (cons row1 (cons row2 '())))

```

もし自力で作成できなかった場合は、今すぐじっくりと確認してください。

Figure 70 の関数は、行列の要素を転置 (transpose) するという重要な数学的操作を実装しています。転置するとは、対角線（すなわち左上から右下への直線）に沿って要素を反転させることを意味します。

立ち止まって！手作業で mat1 を転置してから、Figure 70 を読みなさい。なぜ transpose は (empty? (first lln)) を尋ねるのでしょうか？

この定義は2つの補助関数を前提としています：

- first\*: 行列を受け取り、最初の列を数値のリストとして生成します。
- rest\*: 行列を受け取り、最初の列を取り除きます。結果は行列になります。

これらの関数の定義がまだ手元になくても、transpose がどのように動作するのは理解できるはずです。また、これまでに学んだ設計レシピではこの関数を設計できないことも理解できるはずです。その理由を説明しなさい。

ウィッシュリストにあるこれら 2 つの関数を設計しなさい。その後、いくつかのテストケースを追加して transpose の設計を完成させなさい。

#### 10.4 グラフィカルエディタ再訪 (A Graphical Editor, Revisited)

「グラフィカルエディタ (A Graphical Editor)」では、対話的なグラフィカル 1 行エディタの設計を扱いました。そこではエディタの状態を表現する 2 通りの異なる方法が提案され、その両方を探索することが促されました。すなわち、ペアの文字列を含む構造体、または文字列と現在の位置を指すインデックスを組み合わせた構造体です (演習問題 87 を参照)。

3 つ目の代替案は、2 つの 1String のリストを組み合わせた構造体を使用することです。

```
(define-struct editor [pre post])
; An Editor is a structure:
; (make-editor Lo1S Lo1S)
; An Lo1S is one of:
; - '()
; - (cons 1String Lo1S)
```

なぜそうするのか疑問に思う前に、2 つのデータ例を作成してみましょう。

```
(define good
 (cons "g" (cons "o" (cons "o" (cons "d" '())))))
(define all
 (cons "a" (cons "l" (cons "l" '()))))
(define lla
 (cons "l" (cons "l" (cons "a" '()))))
```

```
; data example 1:
(make-editor all good)
```

```
; data example 2:
(make-editor lla good)
```

これら 2 つの例は、解釈を書き下すことがどれほど重要であることを示しています。editor の 2 つのフィールドがカーソルの左側と右側の文字を表していることは明確ですが、これら 2 つの例は構造体型の解釈方法に少なくとも 2 通りのやり方があることを示しています。

1. (make-editor pre post) は、pre の文字がカーソルの前にあり、post の文字がカーソルの後にある、結合されたテキストが (string-append (implode pre) (implode post)) であることを意味する可能性があります。implode は 1String のリストを String に変換することを思い出してください。
2. (make-editor pre post) は、pre の文字が逆順でカーソルの前にある、という意味である可能性も同様にあります。その場合、表示されるエディタ内のテキストは次のように得られます: (string-append (implode (rev pre)) (implode post))。関数 rev は 1String のリストを受け取り、それを逆順にしなければなりません。

rev の完全な定義がなくても、それがどのように機能するかは想像できるでしょう。この理解を利用して、最初のデータ例を最初の解釈に従って情報に変換した場合と、2 番目のデータ例を2番目の解釈に従って処理した場合に、同じエディタ表示が得られることを確認してください。

[image: pict\_101.png]

どちらの解釈も優れた選択肢ですが、2 番目の解釈を使用するとプログラムの設計が大幅に単純化されることが分かります。本節の残りの部分ではこの点を示し、同時に構造体内部でのリストの使用方法を説明します。このレッスンを適切に理解するには、「グラフィカルエディタ (A Graphical Editor)」の演習問題を解いておく必要があります。

データ定義の意味を理解するには明らかにこの関数が必要ですので、まずは rev から始めましょう。そのヘッダー情報はわかりやすいものです。

```
; lols -> lols
; produces a reverse version of the given list
```

```
(check-expect
 (rev (cons "a" (cons "b" (cons "c" '()))))
 (cons "c" (cons "b" (cons "a" '()))))
```

```
(define (rev l) l)
```

念のため、テストケースとして「明白な」例を1つ追加しました。何が必要かを確実に理解するために、追加の例をいくつか足してみてもよいでしょう。

rev のテンプレートは、通常のリスト用テンプレートです。

```
(define (rev l)
 (cond
 [(empty? l)...]
 [else (... (first l)...
 ... (rev (rest l))...)]))
```

ケースは2つあり、2 番目のケースにはいくつかのセクタ式と1つの自己参照式が含まれています。

テンプレートの最初の節を埋めるのは簡単です。空リストの逆順バージョンは空リストです。2 番目の節については、再びコーディングの質問を使用します。

- (first l) は lString のリストの最初の要素です。
- (rest l) はリストの残りの部分です。そして
- (rev (rest l)) はリストの残りの部分の逆順です。

ここで手を止めましょう！これらのヒントをもとに rev の設計を完成させてみてください。

Figure 71: Tabulating for rev									
l	(first ↵ l)	(rest ↵ l)	(rev ↵ (rest l))	(rev l)					
(first l)									
(rest l)									
(rev (rest l))									
(cons "a" ↵ '())									
(cons "a" ↵ (cons "b" ↵ (cons "c" ↵ ...									
'()))									
(cons "a" ↵ (cons "b" ↵ (cons "c" ↵ (cons "b" ↵ (cons "c" ↵ (cons "a" ↵ '()))									
'()))									
(cons "a" ↵ (cons "b" ↵ (cons "c" ↵ (cons "b" ↵ (cons "c" ↵ (cons "a" ↵ (cons "b" ↵ (cons "c" ↵ (cons "b" ↵ (cons "c" ↵ (cons "a" ↵ '()))									
'()))									
(cons "a" ↵ (cons "b" ↵ (cons "c" ↵ (cons "b" ↵ (cons "c" ↵ (cons "a" ↵ (cons "b" ↵ (cons "c" ↵ (cons "b" ↵ (cons "c" ↵ (cons "a" ↵ (cons "b" ↵ (cons "c" ↵ (cons "b" ↵ (cons "c" ↵ (cons "a" ↵ '()))									
'()))									
(cons "a" ↵ (cons "b" ↵ (cons "c" ↵ (cons "b" ↵ (cons "c" ↵ (cons "a" ↵ (cons "b" ↵ (cons "c" ↵ (cons "b" ↵ (cons "c" ↵ (cons "a" ↵ (cons "b" ↵ (cons "c" ↵ (cons "b" ↵ (cons "c" ↵ (cons "a" ↵ (cons "b" ↵ (cons "c" ↵ (cons "b" ↵ (cons "c" ↵ (cons "a" ↵ '()))									
'()))									

これらのヒントで行き詰まった場合は、例から表を作成することを思い出してください。Figure 71 は、2つの例 `(cons "a" '())` および `(cons "a" (cons "b" (cons "c" '())))` に対する表を示しています。2番目の例は特にわかりやすいものです。後ろから2番目の列を見ると、`(rev (rest l))` が `(cons "c" (cons "b" '()))` を生成することで、仕事の大部分を果たしていることがわかります。希望する結果は `(cons "c" (cons "b" (cons "a" '())))` なので、`rev` は再帰の結果の末尾に何らかの方法で "a" を追加しなければなりません。実際、`(rev (rest l))` は常にリストの残りの部分の逆順であるため、その末尾に `(first l)` を追加するだけで十分であることは明らかです。リストの末尾に要素を追加する関数はまだありませんが、それを「おねだり (wish)」して、関数定義を完成させるために使用することができます。

```
(define (rev l)
 (cond
 [(empty? l) '()]
 [else (add-at-end (rev (rest l)) (first l))]))
```

以下は、add-at-end に対する拡張されたウィッシュリスト・エントリです。

```
; lols 1String -> lols
; creates a new list by adding s to the end of l
```

```
(check-expect
 (add-at-end (cons "c" (cons "b" '())) "a")
 (cons "c" (cons "b" (cons "a" '()))))
```

```
(define (add-at-end l s)
 l)
```

これが「拡張」されているのは、テストケースとして定式化された例が付属しているためです。この例は rev の例から導き出されたものであり、まさにこのウィッシュリスト・エントリの動機となった例そのものです。読み進める前に、add-at-end が空リストを受け取る例をご自身で作成してみてください。

add-at-end もリスト処理関数であるため、テンプレートは皆様が熟知しているものを名前変更しただけです。

```
(define (add-at-end l s)
 (cond
 [(empty? l)...]
 [else (... (first l)...
 ... (add-at-end (rest l) s)...)]))
```

これを関数定義として完成させるために、ステップ5のレシピの質問に従って進めます。最初の質問は、「基本 (basic)」ケース、すなわちここでの最初のケースに対する回答を定式化することです。提案された演習に取り組んだのであれば、

```
(add-at-end '() s)
```

の結果が常に (cons s '()) になることがわかるでしょう。結局のところ、結果はリストでなければならず、そのリストには与えられた 1String が含まれていなければならないからです。

次の2つの質問は、「複雑な」ケースまたは「自己参照」ケースに関するものです。2番目の cond 行の式が何を計算するかはわかっています。最初の式は与えられたリストから最初の 1String を取り出し、2番目の式は「(rest l) の末尾に s を追加して新しいリストを作成」します。つまり、目的記述 (purpose statement) が、関数がここで何を生成すべきかを指示しています。ここから、関数は再帰の結果に (first l) を再び追加しなければならないことが明確になります。

```
(define (add-at-end l s)
 (cond
```

```

[(empty? l) (cons s '())]
[else
 (cons (first l) (add-at-end (rest l) s))])

```

例としてのテストを実行して、この関数が動作すること、したがって rev も動作することを確認してください。もちろん、BSL が 1String のリストを含む任意の与えられたリストを反転する関数をすでに提供していることを知っても驚くべきではありません。そして当然ながら、それは reverse と呼ばれています。

**Exercise 177.** 関数 create-editor を設計しなさい。この関数は 2 つの文字列を受け取り、Editor を生成します。最初の文字列はカーソルの左側のテキストであり、2 番目の文字列はカーソルの右側のテキストです。本節の残りの部分は、この関数に依存しています。

この時点で、グラフィカルな 1 行エディタに対するデータの表現方法を完全に理解しているはずですが、「世界プログラムの設計 (Designing World Programs)」での対話型プログラムの設計戦略に従って、物理定数（例えばエディタの幅や高さなど）およびグラフィック定数（例えばカーソルなど）を定義する必要があります。以下は私たちの定義例です。

```

(define HEIGHT 20); the height of the editor
(define WIDTH 200); its width
(define FONT-SIZE 16); the font size
(define FONT-COLOR "black"); the font color

(define MT (empty-scene WIDTH HEIGHT))
(define CURSOR (rectangle 1 HEIGHT "solid" "red"))

```

しかし重要な点は、イベントハンドラおよびエディタの状態を描画する関数に対するウィッシュリストを書き出すことです。2htdp/universe ティーチパックがこれらの関数のヘッダー情報を定めていることを思い出してください。

```

; Editor -> Image
; renders an editor as an image of the two texts
; separated by the cursor
(define (editor-render e) MT)

``racket
; Editor -> Image
(define (editor-render e)
 (place-image/align
 (beside (editor-text (editor-pre e))
 CURSOR
 (editor-text (editor-post e)))
 1 1
 "left" "top"
 MT))

; Editor -> Image
; renders an editor as an image of the two texts
; separated by the cursor
(define (editor-render e) MT)

```

```

; Editor KeyEvent -> Editor
; deals with a key event, given some editor
(define (editor-kh ed ke) ed)

; Editor -> Image
(define (editor-render e)
 (place-image/align
 (beside (editor-text (editor-pre e))
 CURSOR
 (editor-text (editor-post e)))
 1 1
 "left" "top"
 MT))

```

; Editor KeyEvent -> Editor ; deals with a key event, given some editor (define (editor-kh ed ke) ed)

さらに、「世界プログラムの設計 (Designing World Programs)」では、プログラムの `main` 関数を書き出すことが求められます。

```

````racket
; main: String -> Editor
; launches the editor given some initial string
(define (main s)
  (big-bang (create-editor s "")
    [on-key editor-kh]
    [to-draw editor-render]))

; main: String -> Editor
; launches the editor given some initial string
(define (main s)
  (big-bang (create-editor s "")
    [on-key editor-kh]
    [to-draw editor-render]))

```

このプログラムの初期エディタを決定するために、演習問題 177 (exercise 177) を読み返してください。

次にどのウィッシュに取り組むかはどちらでも構いませんが、ここでは最初に editor-kh を設計し、次に editor-render を設計することにします。ヘッダー情報はすでにあるので、キーイベントハンドラの機能を 2 つの例で説明しましょう。

```

(check-expect (editor-kh (create-editor "" "") "e")
  (create-editor "e" ""))

(check-expect
  (editor-kh (create-editor "cd" "fgh") "e")
  (create-editor "cde" "fgh"))

```

これら両方の例は、キーボードで文字「e」を押したときに何が起こるかを示しています。コンピュータは、エディタの現在の状態と "e" に対して関数 `editor-kh` を実行します。最初の例ではエディタは空であり、これは結果が文字 "e" とそれに続くカーソルだけを持つエディタになることを意味します。2 番目の例では、カーソルは文字列 "cd" と "fgh" の間にあり、したがって結果は "cde" と "fgh" の間にカーソルがあるエディタになります。要するに、この関数は常にカーソル位置に通常の文字を挿入します。

読み進める前に、文字を削除するためにバックスペース ("`\b`") キーを押したとき、カーソルを移動するために "`left`" や "`right`" の矢印キーを押したとき、あるいはその他の矢印キーを押したときに `editor-kh` がどのように機能するかを示す例を作成してみてください。どのような場合でも、エディタが空のとき、カーソルがエディタ内の空でない文字列の左端または右端にあるとき、そして中央にあるときに何が起こるべきかを考慮してください。ここで区間 (interval) を扱っているわけではありませんが、「極端な」ケースに対する例を作成しておくことは、依然として良い考えです。

テストケースを用意したら、テンプレートを開発する段階です。`editor-kh` の場合、2 つの複雑な形態のデータを消費する関数を扱うことになります。1 つはリストを含む構造体であり、もう 1 つは文字列の大きな列挙体 (enumeration) です。一般的に言って、このような設計ケースでは改善された設計レシピが要求されます。しかしこのようなケースでは、入力の 1 つ、すなわちキーストローク (キー入力) を最初に処理すべきであることも明白です。

そうは言っても、テンプレートは関数がどの `KeyEvent` を受け取ったかをチェックするための大きな `cond` 式にすぎません。

```
(define (editor-kh ed k)
  (cond
    [(key=? k "left")...]
    [(key=? k "right")...]
    [(key=? k "\b")...]
    [(key=? k "\t")...]
    [(key=? k "\r")...]
    [(= (string-length k) 1)...]
    [else...]))
```

`cond` 式は `KeyEvent` `t` データ定義に完完全には一致していません。なぜなら、一部の `KeyEvent` は特別な注意を必要とし ("`left`" や "`など`")、一部は特殊であるため無視される必要があり ("`”`" や "```")、また一部は一つの大きなグループ (通常のキー) に分類されるべきだからです。

Exercise 178. `editor-kh` のテンプレートが、長さ 1 の文字列であるかをチェックする前に "`\t`" と "`\r`" を処理している理由を説明せよ。

第 5 ステップ——関数の定義——では、条件式の各節に個別に立ち向かいます。最初の節は、カーソルを移動させ、エディタの文字列コンテンツには手を加えない結果を要求します。2 番目の節も同様です。しかし 3 番目の節は、エディタのコンテンツから文字を (もし文字が存在すれば) 削除することを要求します。最後に、6 番目の `cond` 節はカーソル位置

への文字の追加に関係しています。最初の基本的なガイドラインに従い、私たちはウィッシュュリストを広範に活用し、タスクごとに1つの関数を想像します。

```
(define (editor-kh ed k)
  (cond
    [(key=? k "left") (editor-lft ed)]
    [(key=? k "right") (editor-rgt ed)]
    [(key=? k "\b") (editor-del ed)]
    [(key=? k "\t") ed]
    [(key=? k "\r") ed]
    [(= (string-length k) 1) (editor-ins ed k)]
    [else ed]))
```

editor-kh の定義から分かるように、ウィッシュュリストの4つの関数のうち3つは同じシグネチャを持っています。

```
; Editor -> Editor
```

最後の関数は1つではなく2つの引数を取ります。

```
; Editor 1String -> Editor
```

最初の3つの関数に対する正しい願いの定式化はあなたに委ね、ここでは4番目の関数に焦点を当てます。

目的文と関数ヘッダーから始めましょう。

```
; insert the 1String k between pre and post
(define (editor-ins ed k)
  ed)
```

目的は問題文からそのまま取ってきたものです。関数ヘッダーの構築には、Editor のインスタンスが必要です。pre と post は現在のインスタンスの構成要素であるため、単にそれらを再び組み立てるだけです。

次に、editor-kh の例から editor-ins の例を導き出します。

```
(check-expect
  (editor-ins (make-editor '() '()) "e")
  (make-editor (cons "e" '()) '()))

(check-expect
  (editor-ins
    (make-editor (cons "d" '())
      (cons "f" (cons "g" '()))))
    "e")
  (make-editor (cons "e" (cons "d" '()))
    (cons "f" (cons "g" '()))))
```

Editor の解釈を用いて、これらの例を検討してください。すなわち、与えられたエディタが情報の観点から何を意味しているのか、そして関数呼び出しがその観点から何を達成す

ることになっているのかを理解しているか確認してください。このケースにおいては、エディタの視覚表現を描くことが情報を適切に表現するためベストです。

第4ステップでは、テンプレートの開発が求められます。第1引数は構造体であることが保証されており、第2引数はアトミックなデータである文字列です。言い換えれば、テンプレートは与えられたエディタの表現からパーツを取り出しているだけです。

```
(define (editor-ins ed k)
  (... ed... k...
  ... (editor-pre ed)...
  ... (editor-post ed)...))
```

テンプレートにはパラメータも利用可能であるため列挙されていることを思い出してください。

テンプレートと例から、editor-ins は与えられたエディタの pre フィールドと post フィールドからエディタを作成し、前者の先頭に k を追加すればよいと結論付けるのは比較的容易です。

```
(define (editor-ins ed k)
  (make-editor (cons k (editor-pre ed))
               (editor-post ed)))
```

(editor-pre ed) と (editor-post ed) の両方が lString のリストであっても、補助関数を設計する必要はありません。望む結果を得るには、リストを作成する cons を使えば十分です。

この時点で、あなたは2つのことをすべきです。第一に、この関数のテストを実行すること。第二に、Editor の解釈を用いて、なぜこの関数が挿入を行えるのかを抽象的に説明することです。そして、これだけでは物足りないなら、このシンプルな定義を exercise 84 の定義と比較し、あちらの定義では補助関数が必要であったのに対し、ここでの私たちの定義ではなぜ必要ないのかを考えてみるとよいでしょう。

Exercise 179. 次の関数を設計せよ。

```
; Editor -> Editor
; moves the cursor position one lString left,
; if possible
(define (editor-lft ed) ed)

; Editor -> Editor
; moves the cursor position one lString right,
; if possible
(define (editor-rgt ed) ed)

; Editor -> Editor
; deletes a lString to the left of the cursor,
; if possible
(define (editor-del ed) ed)
```

繰り返しになりますが、十分な数の例を使って詳しく確認することが非常に重要です。

Editor の描画 (rendering) 関数を設計することには、いくつか新しいものの小さな課題があります。第 1 の課題は、十分な数のテストケースを開発することです。一方では、考えられる組み合わせのカバレッジが求められます。カーソルの左側の空文字列、右側の空文字列、そして両方の文字列が空である場合です。他方では、2http/image ティーチパックが提供する関数をいくつか実験してみることも必要です。具体的には、テキスト画像として描画された 2 つの文字列のピースを合成する方法が必要であり、テキスト画像を空の画像フレーム (MT) に配置する方法が必要です。以下は、(create-editor "pre" "post") の結果に対する画像を作成するために行うことです。

```
(place-image/align
  (beside (text "pre" FONT-SIZE FONT-COLOR)
    CURSOR
    (text "post" FONT-SIZE FONT-COLOR))
  1 1
  "left" "top"
  MT)
```

これを上のエディタの画像と比較すると、いくつかの違いに気づくでしょう。しかし正確なレイアウトはこの演習の目的において本質的ではなく、また改訂されたレイアウトによって問題が極端に簡単になるわけでもないため、問題ありません。いずれにせよ、DrRacket の対話領域で実験して、お気に入りのエディタ表示を見つけてください。

これでテンプレートを開発する準備が整いました。少なくともここまでは作成できるはずです。

```
(define (editor-render e)
  (... (editor-pre e)... (editor-post e)))
```

与えられた引数は 2 つのフィールドを持つ構造体型にすぎません。しかしそれらの値は 1String のリストであり、テンプレートをさらに洗練させたくなくなるかもしれません。そうしてはいけません！ その代わりに、あるデータ定義が別の複雑なデータ定義を参照している場合、ウィッシュリストを使用する方がよいということを心に留めておいてください。

十分な数の例を消化していれば、ウィッシュリストに何を載せたいかも分かっているはずです。それは、文字列を適切なサイズと色のテキスト画像に変換する 1 つの関数です。この関数を editor-text と呼びましょう。そうすると、editor-render の定義は単に editor-text を 2 回使用し、その結果を beside と place-image で合成するだけになります。

```
; Editor -> Image
(define (editor-render e)
  (place-image/align
    (beside (editor-text (editor-pre e))
      CURSOR
      (editor-text (editor-post e)))
    1 1
    "left" "top"
    MT))
```

この定義は式を3段階の深さにネストしていますが、仮想の editor-text 関数を使用することで、非常に読みやすくなっています。

残るは editor-text の設計です。editor-render の設計から、editor-text は 1String のリストを受け取り、テキスト画像を生成することが分かっています。

```
; lols -> Image
; renders a list of 1Strings as a text image
(define (editor-text s)
  (text "" FONT-SIZE FONT-COLOR))
```

このダミ一定義は、空のテキスト画像を生成します。

editor-text が何を計算することになっているかを実証するために、例を詳しく見ていきます。例となる入力

```
(create-editor "pre" "post")
```

であり、これは editor-render の説明にも使用されたもので、以下と等価です。

```
(make-editor
  (cons "e" (cons "r" (cons "p" '()))))
  (cons "p" (cons "o" (cons "s" (cons "t" '())))))
```

私たちは2番目のリストを editor-text のサンプル入力として選びます。そして期待される結果は editor-render の例から分かっています。

```
(check-expect
  (editor-text
    (cons "p" (cons "o" (cons "s" (cons "t" '())))))
  (text "post" FONT-SIZE FONT-COLOR))
```

読み進める前に、2つ目の例を作成してみるとよいでしょう。

editor-text が 1String のリストを受け取ることを踏まえれば、手際よくテンプレートを書き出すことができます。

```
(define (editor-text s)
  (cond
    [(empty? s)...]
    [else (... (first s)
      ... (editor-text (rest s))...))]))
```

結局のところ、テンプレートは関数の入力を記述するデータ定義によって規定されます。しかし、Editor の解釈を理解し心に留めていれば、テンプレートは必要ありません。そこでは文字列を 1String のリストに変換するために explode が使用されています。当然ながら、その逆の計算を行う implode という関数が存在します。すなわち、

```
> (implode (cons "p" (cons "o" (cons "s" (cons "t" '())))))
"post"
```

この関数を使用すると、editor-text の定義は例から関数本体へとほんのわずかなステップで到達できます。

```
(define (editor-text s)
  (text (implode s) FONT-SIZE FONT-COLOR))
```

Exercise 180. implode を使わずに editor-text を設計せよ。

本当の驚きは、これら 2 つの関数をテストするときに訪れます。editor-text のテストは成功するものの、editor-render のテストは失敗します。失敗箇所を調べると、カーソルの左側の文字列—— "pre" —— が逆順に組版されていることが分かります。エディタのこの部分の状態が逆順で表現されていることを忘れていたのです。幸いなことに、2 つの関数の単体テストはどちらの関数が誤っているかを正確に突き止め、その関数のどこが間違っているのかを伝え、問題を解決する方法すら提案してくれます。

```
(define (editor-render ed)
  (place-image/align
    (beside (editor-text (reverse (editor-pre ed)))
            CURSOR
            (editor-text (editor-post ed)))
    1 1
    "left" "top"
    MT))
```

この定義は、ed の pre フィールドに対して reverse 関数を使用しています。

Note 現代のアプリケーションでは、ユーザーがマウス（またはその他のジェスチャーベースのデバイス）を使ってカーソルを配置できるようになっています。原理的にはこの機能をエディタに追加することは可能ですが、A Graphical Editor, with Mouse のセクションまでその実施を保留します。

```
+-----+
| Figure 68: Counting the words on a line |
|                                         |
| ; An LN is one of:                     |
| ; - '()                               |
| ; - (cons Los LN)                     |
| ; interpretation a list of lines, each is a list of Strings |
|                                         |
| (define line0 (cons "hello" (cons "world" '()))) |
| (define line1 '())                     |
|                                         |
| (define ln0 '())                       |
| (define ln1 (cons line0 (cons line1 '()))) |
|                                         |
| ; LN -> List-of-numbers                 |
| ; determines the number of words on each line |
|                                         |
| (check-expect (words-on-line ln0) '()) |
+-----+
```

```
| (check-expect (words-on-line ln1) (cons 2 (cons 0 '()))) |
| (define (words-on-line ln) |
|   (cond |
|     [(empty? ln) '()] |
|     [else (cons (length (first ln)) |
|                 (words-on-line (rest ln)))])) |
+-----+

```

11 構成による設計 (Design by Composition)

これまでに学んできたように、プログラムとは複雑な成果物であり、その作成には相互に協力し合う数多くの関数を設計することが求められます。設計者が「いつ複数の関数を設計すべきか」、そして「それらの関数をどのようにして1つのプログラムへと合成するか」を理解していれば、この協力関係はうまく機能します。

相互に関連する関数を設計する必要性には、これまでに何度も遭遇してきました。問題文の中にいくつかの異なるタスクが示唆されており、それぞれのタスクを個別の関数として実現するのが最適である場合もあれば、あるデータ定義が別のデータ定義を参照しており、前者のデータを処理する関数が後者のデータを処理する関数に依存している場合もあります。

本章では、数多くの関数を合成するプログラムの設計が求められるさまざまなシナリオを紹介します。こうした設計をサポートするために、本章では関数を分割し合成するためのいくつかの非公式なガイドラインを提示します。ただし、これらの例では複雑な形式のリストを扱う必要があるため、本章はまず簡潔なリスト表記に関するセクションから始まります。

11.1 list 関数 (The list Function)

現時点で、単に1つのリストを作成するためだけに、とりわけ大量の値を含むリストを作成するために、非常に多くの `cons` を書き続けることにうんざりしているかもしれません。幸いなことに、プログラマの生活のこの部分を簡素化する仕組みを提供する、追加の教育用言語が用意されています。BSL+ も同様です。

注記: あなたは BSL を卒業しました。学習のために「Language (言語)」メニューを使用して「Beginning Student with List Abbreviations (リスト略記付き初級学生)」を選択する時が来ました。

主要な革新は `list` であり、これは任意の数の値を受け取ってリストを作成します。`list` を理解する最も簡単な方法は、それを略記法として考えることです。具体的には、以下の形式のすべての式：

```
(list exp-1... exp-n)
```

は、 n 個の一連の `cons` 式を表します：

```
(cons exp-1 (cons... (cons exp-n '())))
```

ここで `'()` はリストの要素ではなく、実際にはリストの残りの部分であることに注意してください。以下は3つの例を示した表です：

short-hand	long-hand	<code>(list "ABC")</code>	<code>(cons "ABC" '())</code>	<code>(list #false #true)</code>
		<code>(cons #false (cons #true '()))</code>	<code>(list 123)</code>	<code>(cons 1 (cons 2 (cons 3 '())))</code>

これらはそれぞれ、1 個、2 個、および 3 個の要素を持つリストを導入しています。

もちろん、list は値だけでなく式にも適用できます：

```
> (list (+ 0 1) (+ 1 1))
(list 1 2)
> (list (/ 1 0) (+ 1 1))
/ :division by zero
```

リストが構築される前に、式が評価されなければなりません。式の評価中にエラーが発生した場合、リストが形成されることはありません。要するに、list は任意個の引数を取る他のあらゆる基本操作と同様に振る舞います。その結果がたまたま cons で構築されたリストであるというだけです。

list を使用すると、多数の要素を持つリストや、リストや構造体を含むリストの表記が大幅に簡素化されます。以下はその例です：

```
(list 0 1 2 3 4 5 6 7 8 9)
```

このリストには 10 個の要素が含まれており、cons で作成すると 10 回の cons の使用と 1 回の '() のインスタンスが必要になります。同様に、以下のリスト：

```
(list (list "bob" 0 "a")
      (list "carl" 1 "a")
      (list "dana" 2 "b")
      (list "erik" 3 "c")
      (list "frank" 4 "a")
      (list "grant" 5 "b")
      (list "hank" 6 "c")
      (list "ian" 7 "a")
      (list "john" 8 "d")
      (list "karel" 9 "e"))
```

は 11 回の list の使用を必要としますが、これは 40 回の cons と 11 回の追加の '() の使用と鮮やかな対比をなしています。

Exercise 181. list を使って、以下のリストと同等なものを構築してください。

1. (cons "a" (cons "b" (cons "c" (cons "d" '()))))
2. (cons (cons1 (cons2 '())) '())
3. (cons "a" (cons (cons1 '()) (cons #false '())))
4. (cons (cons "a" (cons2 '())) (cons "hello" '()))

次の例にも挑戦してみてください。

```
(cons (cons 1 (cons 2 '()))
      (cons (cons 2 '())
            '()))
```


まず、各リストおよびネストされた各リストに何個の要素が含まれているかを特定することから始めてください。回答の表現には `check-expect` を使用してください。これにより、略記したものが正式な表記と本当に同一であるか確認できます。

Exercise 182. `cons` と `'()` を使って、以下のリストと同等なものを構成してください。

1. `(list012345)`
2. `(list(list"he"0)(list"it"1)(list"lui"14))`
3. `(list1(list12)(list123))`

回答の表現には `check-expect` を使用してください。

Exercise 183. 場合によっては、リストが `cons` と `list` の両方を用いて形成されることがあります。

1. `(cons"a"(list0#false))`
2. `(list(cons1(cons13'())))`
3. `(cons(list1(list13'()))'())`
4. `(list'()'')(cons1'())`
5. `(cons"a"(cons(list1)(list#false'())))`

以下の各式を、`cons` のみ、または `list` のみを使用して書き直してください。回答の確認には `check-expect` を使用してください。

Exercise 184. 以下の式の値を求めてください。

1. `(list(string=?"a""b")#false)`
2. `(list(+1020)(*1020)/(1020))`
3. `(list"dana""jane""mary""laura")`

回答の表現には `check-expect` を使用してください。

Exercise 185. あなたは BSL から `first` と `rest` について知っていますが、BSL+ にはそれらよりもさらに多くのセレクタが用意されています。以下の式の値を求めてください。

1. `(first(list123))`
2. `(rest(list123))`
3. `(second(list123))`

`third` や `fourth` が存在するかどうかをドキュメントで調べてみてください。

11.2 関数の合成 (Composing Functions)

『How to Design Programs』では、プログラムとは定義の集まり（構造型定義、データ定義、定数定義、関数定義）であると説明されています。関数間の役割分担を指導するために、同節では大まかなガイドラインも提案されています。

注記: テストも忘れないでください。

1つのタスクにつき1つの関数を設計すること。問題における数量間の依存関係ごとに補助関数の定義を定式化すること。

本書のこの部分では、補助関数に関するもう1つのガイドラインを導入します。

1つのデータ定義につき1つのテンプレートを設計すること。1つのデータ定義が第2のデータ定義を指し示している場合、補助関数の定義を定式化すること。

本節では、設計プロセスにおいて追加の補助関数が求められる可能性がある特定の箇所、すなわちテンプレートから本格的な定義を作成する「定義ステップ」に着目します。テンプレートを完全な関数定義に変えるということは、テンプレートのサブ式の値を組み合わせて最終的な解を作成することを意味します。その際、補助関数の必要性を示唆するいくつかの状況に遭遇するかもしれません：

1. データの値の合成に特定の適用領域の知識（たとえば、2つの（コンピュータ）画像の合成、会計、音楽、あるいは科学など）が必要な場合は、補助関数を設計します。
2. データの値の合成に利用可能な値のケース分けが必要な場合（たとえば、数値が正、ゼロ、負のいずれであるかに依存する場合）は、cond 式を使用します。cond が複雑に見える場合は、テンプレートの式を引数とし、その本体を cond 式とする補助関数を設計します。
3. データの値の合成において、自己参照的なデータ定義（リスト、自然数、またはそれに類するもの）の要素を処理しなければならない場合は、補助関数を設計します。
4. すべてが失敗した場合は、より一般的な関数を設計し、メイン関数をその一般化された関数の特定の用途として定義する必要があるかもしれません。この提案は直感に反するように思えるかもしれませんが、驚くほど多くのケースで求められます。

最後の2つの基準は、これまでに例は登場したものの、詳細には議論していない状況です。続く2つの節では、さらなる例を用いてこれらの原則を説明します。

ただし、先へ進む前に、プログラム設計の管理における鍵は、よく言及される以下を維持することであることを思い出してください：

ウィッシュリスト (Wish List)

プログラムを完成させるために設計しなければならない関数ヘッダーのリストを維持すること。完全な関数ヘッダーを書き留めておくことで、完成したプログラムの部分をテストできるようになります。これは、多くのテストが失敗したとしても有効です。当然ながら、ウィッシュリストが空になった時には、すべてのテストが合格し、すべての関数がテストでカバーされているはずです。

ウィッシュリストに関数を追加する前に、その関数のようなものが使用中の言語のティーチパックに既に存在しないか、あるいは似たようなものが既にウィッシュリスト

に載っていないかを確認する必要があります。BSL、BSL+、そして実際あらゆるプログラミング言語は、多くの組み込み操作と多くのティーチパック関数を提供しています。時間がある時や必要が生じた時には、選択した言語を調べて、何が提供されているかを把握しておくべきです。

11.3 再帰する補助関数 (Auxiliary Functions that Recur)

人は常にものをソート（並べ替え）する必要があり、プログラムも同様です。投資アドバイザーは各保有資産が生み出す利益によってポートフォリオをソートします。ゲームプログラムはスコアに応じてプレイヤーのリストをソートします。そしてメールプログラムは日付、送信者、あるいはその他の基準に従ってメッセージをソートします。

一般に、データの各ペアを比較して順序付けることができれば、一連のアイテムをソートできます。あらゆる種類のデータに比較プリミティブが存在するわけではありませんが、比較プリミティブを持つデータを私たちは皆 1 つ知っています。それは数値です。したがって、本節では単純化されつつも非常に代表的な例題を扱います。

例題 実数のリストをソートする関数を設計せよ。

以下の演習問題では、この関数を他のデータに適応させる方法を明らかにします。

問題文には他のタスクについての言及がなく、またソートが他のタスクを示唆しているようにも見えないため、単に設計レシピに従います。ソートとは、一連の数値を並べ替えることを意味します。この言い換えは、関数の入力と出力に対する自然なデータ定義、ひいてはそのシグネチャを意味します。List-of-numbers の定義があることを前提とすると、最初のステップは簡単です。

```
; List-of-numbers -> List-of-numbers
; produces a sorted version of alon
(define (sort> alon)
  alon)
```

alon を返すことで、関数のシグネチャに関する限り結果が適切であることが保証されますが、一般に、与えられたリストはソートされておらず、この結果は誤りです。

例を作成する段になると、問題文が非常にあいまいであることがすぐに明らかになります。以前と同様に、例の作成を整理するために List-of-numbers のデータ定義を使用します。データ定義は 2 つの節で構成されているため、2 つの例が必要です。明らかに、sort> が '() に適用された場合、結果は '() でなければなりません。問題は、

```
(cons 12 (cons 20 (cons -5 '())))
```

に対する結果がどうあるべきかです。このリストはソートされていませんが、ソートする方法は 2 つあります。

- (cons20(cons12(cons-5'())))、つまり数値が降順に並べられたリスト。
- (cons-5(cons12(cons20'())))、つまり数値が昇順に並べられたリスト。

現実世界の状況であれば、ここで問題を提示した人に明確化を求める必要があります。ここでは降順の選択肢を選びます。昇順の選択肢を設計しても、異なる障害が生じるわけではありません。

この決定により、ヘッダー素材の改訂が必要になります。

```
; List-of-numbers -> List-of-numbers
; rearranges alon in descending order

(check-expect (sort> '()) '())
(check-expect (sort> (list 3 2 1)) (list 3 2 1))
(check-expect (sort> (list 1 2 3)) (list 3 2 1))
(check-expect (sort> (list 12 20 -5))
              (list 20 12 -5))

(define (sort> alon)
  alon)
```

ヘッダー素材には、単体テストとして再定式化され、list を使用した例が含まれるようになりました。後者に違和感がある場合は、双方向の変換を練習するために cons を使ってテストを再定式化してください。追加の2つの例に関しては、それらは sort> が昇順および降順で既にソートを再定式化してください。追加された2つの例については、sort> が昇順および降順で既にソートされているリストに対して機能することを要求しています。

次に、データ定義を関数テンプレートに変換する必要があります。数値のリストは以前にも扱ったことがあるため、このステップは簡単です。

```
(define (sort> alon)
  (cond
    [(empty? alon)...]
    [else (... (first alon)...
               ... (sort> (rest alon))...)]))
```

このテンプレートを使用することで、ようやくプログラム開発の興味深い部分に取りかかることができます。シンプルなケースから始めて、cond 式の各ケースを個別に考慮します。sort> の入力がない場合、例で指定されているように答えは '() です。sort> の入力が cons されたリストである場合、テンプレートは役立つ可能性のある2つの式を示唆しています。

- (first alon) は入力から最初の数を抽出します。
- (sort>(rest alon)) は、関数の目的記述に従って (rest alon) を降順に並べ替えます。

これらの抽象的な答えを明確にするために、2番目の例を使用してこれらの構成要素を詳細に説明しましょう。sort> が (list 12 20 -5) を処理するとき、

1. (first alon) は 12 であり、
2. (rest alon) は (list 20 -5) であり、
3. (sort>(rest alon)) は、このリストが既にソートされているため (list 20 -5) を生成します。

意図した答えを生成するために、`sort>` は最後のリストの 2 つの数の間に 12 を挿入しなければなりません。より一般的には、(`sort>(restalon)`) の結果の中の適切な位置に (`firstalon`) を挿入する式を見つけなければなりません。それができれば、ソートは簡単に解決できる問題となります。

ソートされたリストに数を挿入することは、明らかに単純なタスクではありません。項目の適切な位置を見つけるために、ソートされたリストを検索することが求められます。任意のリストを検索するには補助関数が必要です。なぜならリストのサイズは任意であり、前のセクションの項目 3 により、任意のサイズの値を処理するには補助関数のデザインが必要となるからです。

そこで、新しいウィッシュリスト (`wish-list`) のエントリがこちらです。

```
; Number List-of-numbers -> List-of-numbers
; inserts n into the sorted list of numbers alon
(define (insert n alon) alon)
```

つまり、`insert` は数と降順にソートされたリストを受け取り、前者を後者に挿入することによってソートされたリストを生成します。

`insert` があれば、`sort>` の定義を完成させるのは簡単です。

```
(define (sort> alon)
  (cond
    [(empty? alon) '()]
    [else
     (insert (first alon) (sort> (rest alon)))]))
```

最終的な結果を生成するために、`sort>` は空でないリストの最初の項目を取り出し、残りの部分のソート済みバージョンを計算し、`insert` を使用してこれら 2 つのピースから完全にソートされたリストを生成します。

手を止めてください！ プログラムをこのままでテストしてみましょう。一部のテストケースは成功し、一部は失敗します。それは進展です。デザインの次のステップは、機能的な例の作成です。`insert` の最初の入力は何の数であるため、5 を使用し、2 番目の入力の例を作成するために `List-of-numbers` のデータ定義を使用します。

まず、数と `()` が与えられたときに `insert` が何を生成すべきかを考慮します。`insert` の目的記述によると、出力はリストでなければならず、2 番目の入力のすべての数を含んでいなければならない、最初の引数を含んでいなければなりません。これは以下を示唆しています。

```
(check-expect (insert 5 '()) (list 5))
```

次に、1 つの項目だけを持つ空でないリストを使用します。

```
(check-expect (insert 5 (list 6)) (list 6 5))
(check-expect (insert 5 (list 4)) (list 5 4))
```

これらが期待される結果となる理由は以前と同様です。第一に、結果には2番目のリストのすべての数と追加の数が含まれていなければなりません。第二に、結果はソートされていなければなりません。

最後に、複数の項目を含むリストの例を作成しましょう。実際、そのような例は `sort>` の例、特に2番目の `cond` 節の分析から導き出すことができます。そこから、12が `(list20-5)` の適切な位置に挿入される場合にのみ `sort>` が機能することがわかります。

```
(check-expect (insert 12 (list 20 -5))
              (list 20 12 -5))
```

つまり、`insert` には2番目のリストが与えられ、それは降順にソートされています。

例の開発が教えてくれることに注意してください。`insert` 関数は、与えられた `n` よりも小さい最初の数を見つけなければなりません。そのような数が存在しない場合、関数は最終的にリストの末尾に到達し、`n` を末尾に追加しなければなりません。さて、テンプレートに進む前に、追加の例を作成してみてください。そのためには、`sort>` の補足的な例を使用するとよいでしょう。

`sort>` とは対照的に、`insert` 関数は2つの入力を受け取ります。最初の入力が数でありアトムミックであることがわかっているため、テンプレート開発では2番目の引数である数値のリストに集中できます。

```
(define (insert n alon)
  (cond
    [(empty? alon)...]
    [else (... (first alon)...
               ... (insert n (rest alon))...)]))
```

このテンプレートと `sort>` のテンプレートとの唯一の違いは、こちらでは追加の引数 `n` を考慮する必要があることです。

`insert` のテンプレートの空欄を埋めるために、再びケースバイケースで進めます。最初のケースは空のリストに関するものです。最初の例によれば、`(listn)` は `n` と `alon` からソートされたリストを構成するため、最初の `cond` 節で必要な式となります。

2番目のケースは最初のケースよりも複雑であるため、figure 53からの質問に従います。

1. `(firstalon)` は `alon` の最初の数です。
2. `(restalon)` は `alon` の残りの部分であり、`alon` と同様に降順でソートされています。
3. `(insertn(restalon))` は、`n` と `(restalon)` 上の数からソートされたリストを生成します。

問題は、最終的な答えを得るためにこれらのデータをどのように組み合わせるかです。

これらすべてを具体的にするために、いくつかの例を検討してみましょう。

```
(insert 7 (list 6 5 4))
```

ここでは `n` は7であり、2番目の入力のどの数よりも大きいです。リストの最初の項目を見るだけでそのことがわかります。それは6ですが、リストはソートされているため、リ

スト上の他のすべての数は6よりもさらに小さくなります。したがって、(list654)に7をconsするだけで十分です。

対照的に、適用が次のような場合：

```
(insert 0 (list 6 2 1 -1))
```

nは実際にリストの残りの部分に挿入されなければなりません。より具体的には、(firstalon)は6、(restalon)は(list21-1)であり、(insertn(restalon))は目的記述に従って(list210-1)を生成します。その最後のリストに6を追加し直すことで、(insert0(list621-1))に対する望ましい答えが得られます。

完全な関数定義を得るには、これらの例を一般化する必要があります。ケース分析は、nが(firstalon)以上であるかどうかを判定するネストされた条件式を示唆しています。

- もしそうであれば、alonはすでにソートされているため、alon内のすべての項目はnよりも小さくなります。その場合の答えは(consnalon)です。
- しかし、もしnが(firstalon)よりも小さい場合、関数はalon内にnを挿入するための適切な位置をまだ見つけていません。結果の最初の項目は(firstalon)でなければならず、nは(restalon)に挿入されなければなりません。この場合における最終結果は(cons (firstalon) (insert n (restalon)))となります。なぜなら、このリストにはnとalonのすべての項目がソートされた順序で含まれているからです。それこそが私たちの必要としているものです。

この議論をBSL+に翻訳するには、そのようなケースに対してif式が必要になります。条件は($\geq$ n(firstalon))であり、2つの分岐に対する式はすでに定式化されています。

Figure 72には完全なソートプログラムが含まれています。これをDrRacketの定義領域にコピーし、テストケースを元に戻して、プログラムをテストしてください。これで基盤となるすべてのテストが成功し、すべての式をカバーするはずです。

用語 ソートのためのこの特定のプログラムは、プログラミングの文献では挿入ソート(insertion sort)として知られています。後ほど、まったく異なるデザイン戦略を使用して、リストをソートする代替方法を学びます。

```
+-----+
| Figure 72: Sorting Lists of numbers |
|                                     |
| ; List-of-numbers -> List-of-numbers |
| ; produces a sorted version of l |
| (define (sort> l) |
|   (cond |
|     [(empty? l) '()] |
|     [(cons? l) (insert (first l) (sort> (rest l)))] |
|   ) |
|                                     |
| ; Number List-of-numbers -> List-of-numbers |
| ; inserts n into the sorted list of numbers l |
| (define (insert n l) |
```



```
| (cond                                     |
|   [(empty? l) (cons n '())]             |
|   [else (if (>= n (first l))             |
|             (cons n l)                  |
|             (cons (first l) (insert n (rest l))))])) |
+-----+

```

Exercise 186. 『合間 1 : 初級学生言語 (Intermezzo 1: Beginning Student Language)』をもう一度見直してください。この合間では BSL とテストの定式化方法が説明されています。後者の方法の 1 つに、ある式が特定のプロパティを満たすかどうかを判定する `check-satisfied` があります。演習問題 145 の `sorted?` を使用して、`check-satisfied` による `sort>` のテストを再定式化してください。

次に、この関数定義を考えてみましょう。

```
; List-of-numbers -> List-of-numbers
; produces a sorted version of l
(define (sort>/bad l)
  (list 9 8 7 6 5 4 3 2 1 0))

```

`sort>/bad` がソート関数ではないことを示すテストケースを作成できますか？ `check-satisfied` を使用してこのテストケースを定式化できますか？

注釈 (1) ここで驚くかもしれないのは、テストを作成するために関数を定義することです。現実の世界では、このステップは一般的であり、時にはテストのための関数（その関数自体のテストなどを含め）を実際にデザインする必要があります。(2) `check-satisfied` を使用したテストの定式化は、`check-expect`（あるいは他の形式）を使用するよりも簡単な場合があります、また少し一般的でもあります。述語が関数のすべての可能な入力と出力の関係を完全に記述している場合、コンピューター科学者はそれを仕様 (specification) と呼びます。『lambda による仕様記述 (Specifying with lambda)』では、`sort>` を完全に仕様化する方法を説明しています。

Exercise 187. スコアによってゲームプレイヤーのリストをソートするプログラムをデザインせよ。

```
(define-struct gp [name score])
; A GamePlayer is a structure:
;   (make-gp String Number)
; interpretation (make-gp p s) represents player p who
; scored a maximum of s points

```

ヒント `GamePlayer` の 2 つの要素を比較する関数を定式化せよ。

Exercise 188. 日付によって電子メールのリストをソートするプログラムをデザインせよ。

```
(define-struct email [from date message])
; An Email Message is a structure:
;   (make-email String Number String)

```



```
; interpretation (make-email f d m) represents text m
; sent by f, d seconds after the beginning of time
```

また、送信者名によって電子メールメッセージのリストをソートするプログラムも作成せよ。2つの文字列をアルファベット順に比較するには、string<? プリミティブを使用すること。

Exercise 189. 以下は関数 search です。

```
; Number List-of-numbers -> Boolean
(define (search n alon)
  (cond
    [(empty? alon) #false]
    [else (or (= (first alon) n)
               (search n (rest alon)))])])
```

この関数はある数が数値のリストに含まれているかどうかを判定します。目的の数がリストに含まれていないことを突き止めるために、関数はリスト全体を走査しなければならない場合があります。

数値のソート済みリストの中に数があるかどうかを判定する関数 search-sorted を作成せよ。この関数は、リストがソートされているという事実を活用しなければならない。

Exercise 190. lString のリストを受け取り、すべての接頭辞 (prefix) のリストを生成する関数 prefixes をデザインせよ。リスト p のすべての項目にわたって p と l が同一である場合、リスト p は l の接頭辞です。例えば、(list "a" "b" "c") はそれ自体の接頭辞であり、(list "a" "b" "c" "d") の接頭辞でもあります。

lString のリストを受け取り、すべての接尾辞 (suffix) を生成する関数 suffixes をデザインせよ。リスト s のすべての項目にわたって s と l が末尾から同一である場合、リスト s は l の接尾辞です。例えば、(list "b" "c" "d") はそれ自体の接尾辞であり、(list "a" "b" "c" "d") の接尾辞でもあります。

11.4 一般化する補助関数 (Auxiliary Functions that Generalize)

時として、補助関数は単なる小さなヘルパー関数にとどまらず、より一般的な問題に対する解法となることがあります。このような補助関数が必要になるのは、問題の記述が狭すぎる場合です。プログラマがデザインレシピの手順を進める中で、「自然な」解決策が間違っていることに気づくことがあります。この壊れた解決策を分析することで、わずかに異なるがより一般的な問題の記述と、その一般的問題の解決策を元の問題に利用する簡単な方法が示唆されることがあります。

次の問題への解法を通して、この考え方を説明します。

ノート: この問題は Paul C. Fisher によって提案されました。

サンプル問題 与えられたシーンに多角形を追加する関数をデザインしなさい。

基礎的な幾何学（ドメイン）知識を覚えていない場合に備えて、多角形の（単純化した）定義を添えておきます。

多角形とは、少なくとも3つの点（同一直線上にない）が3つの直線でつながれた平面図形です。

したがって、多角形の自然なデータ表現の1つは Posn のリストとなります。例えば、以下の2つの定義

```
(define triangle-p (list (make-posn 20 10) (make-posn 20 20) (make-posn 30 20)))  
(define square-p (list (make-posn 10 10) (make-posn 20 10) (make-posn 20 20)  
  (make-posn 10 20)))
```

は、その名前が示す通り、三角形と正方形を導入しています。ここで、'() や (list(make-posn3040)) を多角形としてどう解釈すべきか疑問に思うかもしれませんが、答えは「これらは多角形を表していない」です。多角形は少なくとも3つの点で構成されるため、多角形の優れたデータ表現とは、少なくとも3つの Posn を持つリストの集合です。

空でない温度のリスト（「空でないリスト」の節の NEList-of-temperatures）のデータ定義の構築に従えば、多角形のデータ表現を定式化するのは簡単です。

```
; A Polygon is one of:  
; - (list Posn Posn Posn)  
; - (cons Posn Polygon)
```

第1条項は3つの Posn のリストが Polygon であることを意味し、第2条項は既存の Polygon に Posn を cons することで別の多角形が作られることを意味します。このデータ定義は、その条項の1つで list を使用するまさに最初の例であるため、この文脈での略記から正式な表記への変換を確実に確認できるように、cons を使って詳細に記述しておきます。

```
; a Polygon is one of:  
; - (cons Posn (cons Posn (cons Posn '())))  
; - (cons Posn Polygon)
```

重要なのは、単純に選んだデータ表現（単なる Posn のリスト）では、意図した情報を正しく表現できない場合があるということです。初期の探索段階でデータ定義を修正することは普通であり、実際、デザインプロセスの残りの段階でそのような修正が必きない可能性があるということです。初期の探求段階でデータ定義を改定することは正常なことです。実際、設計プロセスの残りの部分でそのような改定が必要になることも時々あります。しかし、体系的なアプローチを守っている限り、データ定義への変更は設計の残りの部分に自然に伝播させることができます。

2 番目のステップでは、関数のシグネチャ、目的の記述 (purpose statement)、ヘッダーが必要です。問題文では 1 つのタスクのみが言及されており、その他のタスクは暗示されていないため、1 つの関数から始めます：

```
; a plain background image
(define MT (empty-scene 50 50))

; Image Polygon -> Image
; renders the given polygon p into img
(define (render-poly img p)
  img)
```

MT の追加定義が必要とされるのは、例の作成を簡素化するためです。

最初の例には、上述の三角形を使用します。2htdp/image ティーチパックを軽く見ると、三角形の 3 本の線を描画するために必要な関数は scene+line であることが分かります：

```
(check-expect
  (render-poly MT triangle-p)
  (scene+line
    (scene+line
      (scene+line MT 20 10 20 20 "red")
      20 20 30 20 "red")
    30 20 20 10 "red"))
```

最も内側の scene+line は、最初の Posn から 2 番目の Posn への線を描画し、中央のものは 2 番目と 3 番目の Posn を使用し、最も外側の scene+line は 3 番目と最初の Posn を接続します。

注: 当然ながら、この式を正しく取得するために DrRacket の対話領域で実験を行いました。

最初かつ最小の多角形が三角形であることを踏まえると、2 番目の例としては長方形や正方形が自然に候補に挙がります。ここでは square-p を使用します：

```
(check-expect
  (render-poly MT square-p)
  (scene+line
    (scene+line
      (scene+line
        (scene+line MT 10 10 20 10 "red")
        20 10 20 20 "red")
        20 20 10 20 "red")
    10 20 10 10 "red"))
```

正方形は三角形より 1 点多いだけであり、描画も容易です。方眼紙にこれらの図形を描いてみてもよいでしょう。

テンプレートの構築は挑戦を投げかけます。具体的には、figure 52 の 1 番目と 2 番目の質問は、データ定義が明確なサブセットを区別しているか、そしてそれらをどのように区別するかを問うています。最初の節においてデータ定義が三角形を他のすべての多角形から明確に区別している一方で、その 2 つをどのように識別するかはすぐには明らかではありません。両方の節が Posn のリストを記述しているからです。最初の節は 3 つの Posn のリストを記述し、2 番目の節は少なくとも 4 つの要素を持つ Posn のリストを記述しています。したがって、1 つの選択肢は、与えられた多角形が 3 要素の長さであるかどうかを尋ねることです：

```
(= (length p) 3)
```

最初の節の省略なしのバージョン、すなわち

```
(cons Posn (cons Posn (cons Posn '()))))
```

を使用することは、最初の条件を定式化する 2 番目の方法を示唆します。すなわち、rest 関数を 3 回適用した後に、与えられた Polygon が空であるかをチェックすることです：

```
(empty? (rest (rest (rest p))))
```

すべての Polygon は少なくとも 3 つの Posn で構成されているため、rest を 3 回使用することは正当です。length とは異なり、rest は明確な操作的意味を持つプリミティブで理解しやすい操作です。これは cons 構造の 2 番目のフィールドを選択するだけであり、それ以外のことは何もしません。

注: 自身で作成した（再帰的な）関数よりも、組み込みの述語やセレクターを使って条件を定式化の方が真に優れています。説明については *Intermezzo 5: The Cost of Computation* を参照してください。

figure 52 の残りの質問には直接的な回答が存在するため、次のようなテンプレートが得られます：

```
(define (render-poly img p)
  (cond
    [(empty? (rest (rest (rest p))))
     (... (first p)... img...
      ... (second p)...
      ... (third p)...)]
    [else (... (first p)...
      ... (render-poly img (rest p))...)]))
```

最初の節において p は三角形を表しているため、それは正確に 3 つの Posn から構成されている必要があり、これらは first、second、および third を介して抽出されます。2 番目の節では、p は 1 つの Posn と 1 つの Polygon で構成されており、(firstp) と (restp) を正当化します。前者は p から Posn を抽出し、後者は Polygon を抽出します。したがって、その周囲に自己参照的な関数呼び出しを追加します。また、この節における (firstp) の処

理や、最初の節における 3 つの Posn の処理には、補助関数の設計が必要になるかもしれないということを頭に入れておく必要があります。

これで、1 度に 1 つの節を処理しながら関数定義に集中する準備が整いました。最初の節は三角形に関するものであり、直感的な答えを示唆しています。具体的には、3 つの Posn が存在し、render-poly は 50×50 ピクセルの空のシーンの中でこれら 3 つを接続する必要があります。Posn が別のデータ定義であることを考慮すると、次のような明白なウィッシュリスト（欲しい関数リスト）項目が得られます：

```
; Image Posn Posn -> Image
; draws a red line from Posn p to Posn q into im
(define (render-line im p q)
  im)
```

この関数を使用すると、render-poly における最初の cond 節は次のようになります：

```
(render-line
  (render-line
    (render-line MT (first p) (second p))
    (second p) (third p))
  (third p) (first p))
```

この式は、最初から 2 番目、2 番目から 3 番目、そして 3 番目から最初の Posn へと線を引くことで、与えられた Polygon p を明らかに三角形として描画します。

2 番目の cond 節は、1 つの Posn が拡張された Polygon に関するものです。テンプレートの中には 2 つの式が見られ、figure 53 に従って、これらの式が何を計算するかを再確認します：

1. (firstp) は最初の Posn を抽出します。
2. (restp) は p から Polygon を抽出します。
3. (render-polyimg(restp)) は (restp) を描画します。これは関数の目的記述（purpose statement）が述べている通りです。

問題は、与えられた Polygon p を描画するためにこれらのパーツをどのように使用するかです。

思いつくかもしれないアイディアの 1 つは、(restp) が少なくとも 3 つの Posn で構成されているということです。したがって、この埋め込まれた Polygon から少なくとも 1 つの Posn を抽出し、(firstp) をこの追加の点と接続することが可能です。BSL+ コードでこのアイディアを表すと、次のようになります：

```
(render-line (render-poly MT (rest p)) (first p)
  (second p))
```

上述のように、ハイライトされたサブ式は、埋め込まれた Polygon を 50×50 の空のシーンに描画します。render-line の使用により、p の最初から 2 番目の Posn までの 1 本の線がこのシーンに追加されます。

私たちの分析は、むしろ自然で完全な関数定義を示唆しています：

```
(define (render-poly img p)
  (cond
    [(empty? (rest (rest (rest p))))]
      (render-line
        (render-line
          (render-line MT (first p) (second p))
            (second p) (third p))
          (third p) (first p))]
    [else
      (render-line (render-poly img (rest p))
        (first p)
        (second p))]))
```

render-line の設計は、本書の第 1 部で解いたような問題です。したがって、上記の関数をテストできるように、最終的な定義のみを提供します：

```
; Image Posn Posn -> Image
; renders a line from p to q into img
(define (render-line img p q)
  (scene+line
    img
    (posn-x p) (posn-y p) (posn-x q) (posn-y q)
    "red"))
```

手を止めよう！ render-line のテストを作成してください。

最後に、関数をテストしなければなりません。render-poly のテストは失敗します。一方で、テストが失敗することは、普通の消費者に影響を与える前に問題を発見することがテストの目的であるため幸運です。他方で、デザインレシピに従い、かなり自然な選択をしたにもかかわらず関数が動作しないため、この欠陥は不幸なことです。

手を止めよう！ なぜテストが失敗すると思いますか？ render-poly のテンプレート内のパーツのイメージを描いてみてください。次に、それらを結合する線を描きます。あるいは、DrRacket の対話領域で実験してみてください：

```
> (render-poly MT square-p)
[image:pict_102.png]
```

画像は、render-poly が (restp) の 3 つの点を接続し、次に (firstp) を (restp) の最初の点、すなわち (secondp) に接続していることを示しています。(restsquare-p) を直接 render-poly の入力として使用する対話によって、この主張を簡単に検証できます：

```
> (render-poly MT (rest square-p))
[image:pict_103.png]
```

さらに、元の正方形に別の点、たとえば (make-posn4030) を追加した場合に render-poly が何を描画するのか疑問に思うかもしれません：


```
> (render-poly MT (cons (make-posn 40 30) square-p))  
[image:pict_104.png]
```

望ましい五角形の代わりに、render-poly は与えられた Polygon の末尾にある三角形を常に描画し、それ以外は三角形に先行する Posn 同士を接続するだけです。

実験により私たちの設計の問題点が確認される一方で、この関数が「ほぼ正しい」ことも示唆されます。Posn のリストによって指定された連続する点を接続し、末尾の三角形の最初から最後の Posn までの線を引きます。もしこの最後のステップをスキップすれば、関数は単に「点を接続する」だけになり、「開いた」多角形を描くことになります。そして最初と最後の点を接続することで、タスクを完了できるでしょう。

言い換えれば、私たちの失敗の分析は、2 ステップの解決策を提案しています：

1. より一般的な問題を解決する。
2. この一般的な問題の解決策を使用して、元の問題を解決する。

まず、一般的な問題の記述から始めます：

サンプル問題 与えられた点群の間の接続を、与えられたシーンに描画する関数を設計しなさい。

render-poly の設計はこの問題をほぼ解決していますが、この関数をほぼゼロから設計します。まず、データ定義が必要です。少なくともいくつかの点がない限り、点を接続することは意味をなしません。単純にするため、少なくとも 1 つの点から進めます：

```
; An NELoP is one of:  
; - (cons Posn '())  
; - (cons Posn NELoP)
```

次に、「点を接続する」関数のシグネチャ、目的の記述、およびヘッダーを定式化します：

```
; Image NELoP -> Image  
; connects the dots in p by rendering lines in img  
(define (connect-dots img p)  
  MT)
```

第3に、render-poly のための例をこの新しい関数に合わせて適用します。私たちの失敗分析が示しているように、この関数は p 内の最初の Posn を 2 番目の Posn に、2 番目を 3 番目に、3 番目を 4 番目に、というように、何にも接続されない最後の Posn に至るまで順番に接続します。以下は、3 つの Posn のリストである最初の et (check-expect (connect-dots MT triangle-p) (scene+line (scene+line MT 20 20 30 20 “red”) 20 10 20 20 “red”))

期待される値は2本の線を持つ画像である。1本目は最初の `Posn` から2番目の `Posn` への線、もう1本は2番目から3番目の `Posn` への線である番目の `Posn` への線です。

****Exercise 191.**** `render-poly` 関数の2番目の例を `connect-dots` に合わせて適用しなさい。

第4に、空でないリストを処理する関数のためのテンプレートを使用します。

```
```racket
(define (connect-dots img p)
 (cond
 [(empty? (rest p)) (... (first p)...)]
 [else (... (first p)...
 ... (connect-dots img (rest p))...)])))
```

このテンプレートには2つの節があります。1つは1つの Posn からなるリストのためのもので、2つ目は複数の Posn を持つリストのためのものです。どちらの場合も少なくとも1つの Posn が存在するため、テンプレートの両方の節に (first p) が含まれています。2つ目の節には、データ定義の2番目の節における自己参照を思い出させるために (connect-dots img (rest p)) も含まれています。

第5の、そして中心となるステップは、テンプレートを関数定義へと変換することです。最初の節が最も単純なので、そこから始めます。すでに述べたように、与えられたリストに Posn が1つしか含まれていない場合、何かを接続することはできません。したがって、関数は最初の cond 節から単に MT を返します。2番目の cond 節については、テンプレートの式が何を計算するかを思い出してみましょう。

1. (first p) は最初の Posn を抽出します。
2. (rest p) は p から NELoP を抽出します。
3. (connect-dots img (rest p)) は、img 内に線を描画することによって (rest p) 内の点を接続します。

render-poly を設計する最初の試みから、connect-dots は (connect-dots img (rest p)) の結果に1本の線、すなわち (first p) から (second p) への線を追加する必要があることが分かっています。p が2番目の Posn を含んでいることは分かっています。そうでなければ cond の評価が最初の節を選択していたはずだからです。

すべてをまとめると、次のような定義が得られます。

```
(define (connect-dots img p)
 (cond
 [(empty? (rest p)) img]
 [else
 (render-line
 (connect-dots img (rest p))
 (first p)
 (second p))]))
```

この定義は、render-poly より2つ多くの Posn のリストに対応しているにもかかわらず、欠陥のあったバージョンの render-poly よりもシンプルに見えます。

**注:** この議論は非形式的なものです。集合や関数の関係に関するこのような主張について形式的な議論が必要になった場合は、論理学を学ぶ必要があるでしょう。実際、



本書の設計プロセスは論理学から深い影響を受けており、計算機科学における論理学の講座は自然な補完となります。一般的に、工学にとっての解析学 (analysis) と同じ役割を、計算にとっての論理学 (logic) が果たしています。

逆に、connect-dots は render-poly を一般化 (generalize) していると言えます。後者へのすべての入力、前者への入力でもあります。データ定義の観点から言えば、すべての Polygon は NELoP でもあります。しかし、Polygon ではない NELoP も数多く存在します。正確に言えば、2つの要素または1つの要素を含むすべての Posn のリストは NELoP に属しますが、Polygon には属しません。しかし、ここでの重要な洞察は、ある関数が別の関数よりも多くの入力を扱わなければならないからといって、前者が後者より複雑になるとは限らないということです。一般化によって、関数定義が単純化されることがしばしばあります。

上で説明したように、render-poly は connect-dots を使用して、与えられた Polygon のすべての連続する Posn を接続できます。そしてタスクを完了するには、与えられた Polygon の最初の Posn から最後の Posn への線を追加する必要があります。コードの観点からは、これは単に2つの関数、すなわち connect-dots と render-line を合成することを意味します。ただし、Polygon から最後の Posn を抽出する関数も必要です。この願いが叶えば、render-poly の定義は1行で済みます。

```
; Image Polygon -> Image
; adds an image of p to img
(define (render-poly img p)
 (render-line (connect-dots img p)
 (first p)
 (last p)))
```

last に対するウィッシュリストの項目を作成するのは簡単です。

```
; Polygon -> Posn
; extracts the last item from p
```

また、last は一般的に有用な関数になり得るため、NELoP からの入力に対して設計する方がよいかもしれないことも明らかです。

```
; NELoP -> Posn
; extracts the last item from p
(define (last p)
 (first p))
```

ちょっと待ってください！(Stop!) なぜ last のスタブ定義において first を使用することが許容されるのでしょうか？

**Exercise 192.** Polygon に対して last を使用することがなぜ許容されるのか議論しなさい。また、connect-dots のテンプレートを last に適用できる理由についても議論しなさい。

```
(define (last p)
 (cond
```

```

[(empty? (rest p)) (... (first p)...)]
[else (... (first p)... (last (rest p))...))]

```

最後に、last の例を作成し、それらをテストに変換して、図 73 の last の定義が作成した例で正しく動作することを確認しなさい。

```

+-----+
| Figure 73: Drawing a polygon |
| |
| ; Image Polygon -> Image |
| ; adds a corner of p to img |
| (define (render-poly img p) |
| (render-line (connect-dots img p) (first p) (last p))) |
| |
| ; Image NELoP -> Image |
| ; connects the Posns in p in an image |
| (define (connect-dots img p) |
| (cond |
| [(empty? (rest p)) img] |
| [else (render-line (connect-dots img (rest p)) |
| (first p) |
| (second p))])) |
| |
| ; Image Posn Posn -> Image |
| ; draws a red line from Posn p to Posn q into im |
| (define (render-line im p q) |
| (scene+line |
| im (posn-x p) (posn-y p) (posn-x q) (posn-y q) "red")) |
| |
| ; Polygon -> Posn |
| ; extracts the last item from p |
| (define (last p) |
| (cond |
| [(empty? (rest (rest (rest p)))) (third p)] |
| [else (last (rest p))])) |
| |
+-----+

```

要約すると、render-poly の開発によって、連続する点のリストを接続するという一般化された問題を検討することに自然と導かれました。そして、一般化された関数と他の補助関数を合成する関数を定義することで、元の問題を解決できます。したがって、プログラムは、比較的シンプルなメイン関数である render-poly と、作業の大部分を行う複雑な補助関数で構成されます。このような設計アプローチが一般的であり、プログラムを設計し整理するための優れた方法であることを、今後何度も目の当たりにすることでしょう。

**Exercise 193.** 以下は render-poly を定義するためのさらに 2 つのアイデアです。

- render-poly は p の最後の要素を p の先頭に cons し、その後に connect-dots を呼び出すことができる。

- `render-poly` は Polygon 上で動作する `add-at-end` のバージョンを介して、`p` の最初の要素を `p` の末尾に追加することができる。

両方のアイデアを使って `render-poly` を定義しなさい。両方の定義がテストケースに合格することを確認しなさい。

**Exercise 194.** 最後の `Posn` が接続される追加の `Posn` を受け取るように `connect-dots` を変更しなさい。次に、この新しいバージョンの `connect-dots` を使用するように `render-poly` を変更しなさい。

```
(check-expect (connect-dots MT triangle-p)
 (scene+line
 (scene+line MT 20 20 30 20 "red")
 20 10 20 20 "red"))

(define (connect-dots img p)
 (cond
 [(empty? (rest p)) (... (first p)...)]
 [else (... (first p)...
 ... (connect-dots img (rest p))...))]))
```

当然ながら、`last` のような関数は本格的なプログラミング言語で提供されており、`render-poly` のような機能は `2htdp/image` ティーチパックで提供されています。なぜ私たちがわざわざこれらの関数を設計したのか不思議に思うかもしれませんが、本書および本節のタイトルを考えてみてください。目標は、役に立つ関数を設計すること（だけ）ではなく、コードをいかに体系的に設計するかを学ぶことです。具体的には、本節は設計プロセスにおける「一般化 (generalization)」という概念についてのものです。この概念の詳細については、Abstraction (抽象化) および Accumulators (アキュムレータ) を参照してください。

## 12 プロジェクト：リスト (Projects: Lists)

本章ではいくつかの発展的な演習問題を取り上げます。これらはいずれも、設計の構成要素についての理解を確実なものにすることを目的としています。すなわち、バッチプログラムおよび対話型プログラムの設計、構成による設計 (design by composition)、設計ウィッシュリスト (design wish lists)、そして関数のための設計レシピ (design recipe) です。最初の節では、英語の辞書や iTunes ライブラリといった現実世界のデータを扱う問題を扱います。単語パズルの問題には 2 つの節を費やします。1 つは構成による設計を示すため、もう 1 つは問題の中核に取り組むためです。残りの節では、ゲームや有限状態機械 (finite-state machines) を扱います。

**注:** 本章は `2http/batch-io` ティーチパックに依存しています。

### 12.1 現実世界のデータ：辞書 (Real-World Data: Dictionaries)

現実世界の情報は大量に存在する傾向があり、だからこそプログラムを使ってそれを処理することには大きな意味があります。例えば、辞書にはわずかに十数個の単語だけでなく、数十万個もの単語が含まれています。このような大量の情報を処理したい場合は、小さな例を用いてプログラムを慎重に設計しなければなりません。プログラムが正しく動作することを十分に確認できたら、現実世界のデータで実行して実際の結果を得ます。もしプログラムが遅すぎてこの大量のデータを処理できない場合は、各関数とその仕組みについて振り返ってみてください。冗長な計算を排除できないか検討しましょう。

**注:** 性能に関する問題については、「生成基底の再帰 (Generative Recursion)」を参照してください。そこに至るまでは、性能上の問題を適切に探求できるように、プログラムを体系的に設計することに焦点を当てます。

```
+-----+
| Figure 74: Reading a dictionary |
| |
| ; On OS X: |
| (define LOCATION "/usr/share/dict/words") |
| ; On LINUrds |
| ; On WINDOWS: borrow the word file from your Linux friend |
| |
| ; A Dictionary is a List-of-strings. |
| (define AS-LIST (read-lines LOCATION)) |
+-----+
```

図 74 (Figure 74) は、英語の辞書全体を読み込むために必要な 1 行のコードを示しています。このような辞書がどれほど大きいかを実感するために、図のコードをお使いのコンピュータに合わせて調整し、`length` を使用してご自身の辞書に何単語あるかを測定してみてください。私たちが測定した本日 (2017 年 7 月 25 日) の辞書には 235,886 語が含まれていました。

```

+-----+
| Figure 74: Reading a dictionary |
| |
| ; On OS X: |
| (define LOCATION "/usr/share/dict/words") |
| ; On LINUX: /usr/share/dict/words or /var/lib/dict/words |
| ; On WINDOWS: borrow the word file from your Linux friend |
| |
| ; A Dictionary is a List-of-strings. |
| (define AS-LIST (read-lines LOCATION)) |
+-----+

```

以下の演習問題では、アルファベットの文字 (letter) が重要な役割を果たします。図 74 のコードを調整したものに加えて、プログラムの先頭に以下を追加するとよいでしょう。

```

; A Letter is one of the following 1Strings:
; - "a"
; - ...
; - "z"
; or, equivalently, a member? of this list:
(define LETTERS
 (explode "abcdefghijklmnopqrstuvwxyz"))

```

ヒント: 演習問題の例やテストを作成するには list を使用してください。

**Exercise 195.** Letter と Dictionary を受け取り、与えられた Dictionary 内の何個の単語が与えられた Letter で始まるかを数える関数 `starts-with#` をデザインしなさい。関数が正しく動作することを確認したら、お使いのコンピュータの辞書で "e" で始まる単語が何個あるか、また "z" で始まる単語が何個あるかを求めなさい。

**Exercise 196.** `count-by-letter` をデザインしなさい。この関数は Dictionary を受け取り、与えられた辞書の中で各文字が単語の先頭文字として使われている回数を数えます。その結果は、文字とカウントを組み合わせたデータである Letter-Count のリスト (list of Letter-Counts) になります。

関数のデザインが完了したら、お使いのコンピュータの辞書内のすべての文字に対して何個の単語が存在するかを求めなさい。

**Exercise 197.** `most-frequent` をデザインしなさい。この関数は Dictionary を受け取ります。与えられた Dictionary 内で単語の先頭文字として最も頻繁に現れる文字に対する Letter-Count を生成します。

お使いのコンピュータの辞書で最も頻繁に使用される先頭文字は何ですか？ また、それは何回使用されていますか？

デザイン選択に関する注記 この演習問題では、前問の解法と、Letter-Count のリストから正しいペアを選ぶ関数との合成が求められます。後者の関数をデザインするには、次の 2 つの方法があります。

- カウントが最大のペアを選ぶ関数をデザインする。
- ソートされたペアのリストから最初の要素を選択する関数をデザインする。

両方のデザインを検討しなさい。あなたはどちらを好みますか？その理由はなぜですか？

**Exercise 198.** words-by-first-letter をデザインしなさい。この関数は Dictionary を受け取り、Letter ごとに 1 つの Dictionary を含む Dictionary のリスト (list of Dictionarys) を生成します。

この新しい関数を使用して、演習 197 の most-frequent を再デザインしなさい。新しい関数を most-frequent.v2 と命名しなさい。デザインが完了したら、お使いのコンピュータの辞書に対して、これら 2 つの関数が同じ結果を計算することを確認しなさい。

```
(check-expect
 (most-frequent AS-LIST)
 (most-frequent.v2 AS-LIST))
```

デザイン選択に関する注記 words-by-first-letter においては、与えられた辞書に特定の文字で始まる単語が含まれていない場合の状況に対処するための選択肢があります。

- 1 つの選択肢は、結果として生じる空の辞書を全体 Result から除外することです。これを行うことで、関数のテストと most-frequent.v2 のデザインの両方がシンプルになりますが、補助関数のデザインが必要になります。
- もう 1 つの選択肢は、ある文字の単語を探した結果として、単語が存在しない場合であっても '()' を結果に含めることです。この選択肢は、最初の選択肢で必要となる補助関数を回避できますが、most-frequent.v2 のデザインに複雑さを加えます。

中間データと脱森林化に関する注記 単語をカウントする関数のこの第 2 バージョンは、その構成部分がカウントされること以外には実質的な目的を持たない巨大な中間データ構造を作成することによって、目的の結果を計算します。場合によっては、プログラミング言語が 2 つの関数を 1 つに融合させることで、これらを自動的に排除します。これはプログラムの変換であり、「脱森林化 (deforestation)」とも呼ばれます。プログラミング言語がプログラムを脱森林化しないとわかっており、プログラムが十分な速度でデータを処理できない場合は、このようなデータ構造を排除することを検討してください。

## 12.2 実世界データ: iTunes (Real-World Data: iTunes)

Apple の iTunes ソフトウェアは、音楽、ビデオ、TV 番組などを収集するために広く利用されています。自身の iTunes アプリケーションが収集した情報を分析したいと思うかもしれません。そのデータベースを抽出することは、実のところ非常に簡単です。アプリケーションの File メニューを選択し、Library を選んでから Export を選択します。すると、iTunes 情報のいわゆる XML 表現をエクスポートできます。XML の処理は「Project: The Commerce of XML」で詳しく扱われています。ここでは、情報を取得するために [2htdp/itunes](http://2htdp.org/) ティーチパックに依存します。具体的には、このティーチパックを使用することで、iTunes ライブラリに含まれる音楽トラックを取得できます。

**Note:** このような独自の形式は定期的に変更されるため、本書には2つのサンプルファイル `itunes.xml` と `itunes2.xml` が付属しています。ご自身のファイルの代わりに、これらを使用しても構いません。

詳細には違いがありますが、iTunes ライブラリは各音楽トラックに対して以下のような種類の情報を保持しています（場合によってはこれより少し少ないこともあります）。

- Track ID: ライブラリ内でのトラックの一意の識別子。例: 442
- Name: トラックのタイトル。Wild Child
- Artist: 制作アーティスト。Enya
- Album: トラックが所属するアルバムのタイトル。A Day Without
- Genre: トラックに割り当てられた音楽ジャンル。New Age
- Kind: 音楽のエンコーディング。MPEG audio file
- Size: ファイルのサイズ。4562044
- Total Time: トラックの長さ（ミリ秒単位）。227996
- Track Number: アルバム内でのトラックの順序。2
- Track Count: アルバムに含まれるトラックの数。11
- Year: リリース年。2000
- Date Added: トラックが追加された日時。2002-7-17 3:55:14
- Play Count: 再生回数。20
- Play Date: 最後に再生された日時。3388484113 Unix 秒
- Play Date UTC: 最後に再生された UTC 日時。2011-5-17 17:35:13

いつものように、最初のタスクはこの情報に対する BSL データ表現を選択することです。本節では、音楽トラックに対して2つの表現を使用します。1つは構造体に基づく表現、もう1つはリストに基づく表現です。前者は、すべての情報が利用可能な場合にのみトラックごとに固定数の属性を記録するのに対し、後者は利用可能なあらゆる情報をデータとして表現します。それぞれが特定の用途に適しており、用途によっては両方の表現が役立ちます。

**Note:** 2htdp/batch-io ティーチパックに加えて、本節は 2htdp/itunes ティーチパックに依存します。

```
+-----+
| Figure 75: Representing iTunes tracks as structures (the structures) |
| |
| ; the 2htdp/itunes teachpack documentation, part 1: |
| |
| ; An LTracks is one of: |
| ; - '() |
| ; - (cons Track LTracks) |
| |
| (define-struct track
```

```

| (define-struct track
| [name artist album time track# added play# played])
| ; A Track is a structure:
| ; (make-track String String String N N Date N Date)
| ; interpretation An instance records in order: the track's
| ; title, its producing artist, to which album it belongs,
| ; its playing time in milliseconds, its position within the
| ; album, the date it was added, how often it has been
| ; played, and the date when it was last played
|
| (define-struct date [year month day hour minute second])
| ; A Date is a structure:
| ; (make-date N N N N N N)
| ; interpretation An instance records six pieces of information:
| ; the date's year, month (between 1 and 12 inclusive),
| ; day (between 1 and 31), hour (between 0
| ; and 23), minute (between 0 and 59), and
| ; second (also between 0 and 59).
+-----+

+-----+
| Figure 76: Representing iTunes tracks as structures (the functions) |
|
| ; Any Any Any Any Any Any Any Any -> Track or #false
| ; creates an instance of Track for legitimate inputs
| ; otherwise it produces #false
| (define (create-track name artist album time
| track# added play# played)
| ...)
|
| ; Any Any Any Any Any Any -> Date or #false
| ; creates an instance of Date for legitimate inputs
| ; otherwise it produces #false
| (define (create-date y mo day h m s)
| ...)
|
| ; String -> LTracks
| ; creates a list-of-tracks representation from the
| ; text in file-name (an XML export from iTunes)
| (define (read-itunes-as-tracks file-name)
| ...)
+-----+

```

図75と図76は、2htdp/itunes ティーチパックによって実装されているトラックの構造体に基づく表現を導入しています。track 構造体には8つのフィールドがあり、それぞれがトラックの特定のプロパティを表しています。ほとんどのフィールドにはStringやNなどのアトミックな種類のデータが含まれ、他のフィールドには6つのフィールドを持つ構造体であるDateが含まれています。2htdp/itunes ティーチパックは、track と date の構造体



型のすべての述語とセレクタをエクスポートしますが、コンストラクタの代わりにチェック機能付きコンストラクタを提供します。

```
+-----+
| Figure 75: Representing iTunes tracks as structures (the structures) |
| |
| ; the 2http/itunes teachpack documentation, part 1: |
| |
| ; An LTracks is one of: |
| ; - '()' |
| ; - (cons Track LTracks) |
| |
| (define-struct track |
| [name artist album time track# added play# played]) |
| ; A Track is a structure: |
| ; (make-track String String String N N Date N Date) |
| ; interpretation An instance records in order: the track's |
| ; title, its producing artist, to which album it belongs, |
| ; its playing time in milliseconds, its position within the |
| ; album, the date it was added, how often it has been |
| ; played, and the date when it was last played |
| |
| (define-struct date [year month day hour minute second]) |
| ; A Date is a structure: |
| ; (make-date N N N N N N) |
| ; interpretation An instance records six pieces of information: |
| ; the date's year, month (between 1 and 12 inclusive), |
| ; day (between 1 and 31), hour (between 0 |
| ; and 23), minute (between 0 and 59), and |
| ; second (also between 0 and 59). |
| |
+-----+
```

2http/itunes ティーチパックの説明の最後の要素は、iTunes の XML ライブラリ記述を読み込んでトラックのリストである LTracks を返す関数です。iTunes アプリから XML ライブラリをエクスポートしたら、以下のコードスニペットを実行してすべてのレコードを取得できます。

```
; modify the following to use your chosen name
(define ITUNES-LOCATION "itunes.xml")

; LTracks
(define itunes-tracks
 (read-itunes-as-tracks ITUNES-LOCATION))
```

このスニペットを、iTunes の XML エクスポートと同じフォルダに保存してください。例として itunes-tracks を使用しないように注意してください。例として使うには大きすぎます。実際、非常に大きいため、DrRacket で BSL プログラムを実行するたびにファイルを読み込むと多大な時間がかかる可能性があります。そのため、関数を設計している間は、

この2行目をコメントアウトしておくといよいでしょう。iTunes コレクションに関する情報を計算したいときだけ、コメントアウトを解除してください。

**Exercise 199.** 重要なデータ定義はすでに提供されていますが、設計レシピの最初のステップはまだ不完全です。Date、Track、および LTracks の例を作成してください。これらの例は、以降の演習問題で入力として重宝します。

**Exercise 200.** LTracks の要素を受け取り、総再生時間を返す関数 `total-time` を設計してください。プログラムが完成したら、お使いの iTunes コレクションの総再生時間を計算してください。

**Exercise 201.** `select-all-album-titles` を設計してください。この関数は LTracks を受け取り、アルバムタイトルのリストを `List-of-strings` として返します。

また、関数 `create-set` を設計してください。この関数は `List-of-strings` を受け取り、与えられたリスト内の各 String を正確に1度だけ含むリストを構築します。ヒント: String である `s` が与えられたリストの先頭にあり、かつリストの残りの部分にも存在する場合、`create-set` は `s` を残しません。

最後に、LTracks を受け取り、重複のないアルバムタイトルのリストを返す `select-album-titles/unique` を設計してください。この関数を使用して、お使いの iTunes コレクション内のすべてのアルバムタイトルを特定し、そこに含まれる異なるアルバムの数も求めてください。

**Exercise 202.** `select-album` を設計してください。この関数はアルバムのタイトルと LTracks を受け取ります。後者から、指定されたアルバムに属するトラックのリストを抽出します。

**Exercise 203.** `select-album-date` を設計してください。この関数はアルバムのタイトル、日付 (Date)、および LTracks を受け取ります。後者から、指定されたアルバムに属し、かつ指定された日付より後に再生されたトラックのリストを抽出します。ヒント: 2 つの Date を受け取り、1 つ目が2 つ目よりも前であるかどうかを判定する関数を設計する必要があります。

**Exercise 204.** `select-albums` を設計してください。この関数は LTracks の要素を受け取ります。アルバムごとに1つの LTracks を含む、LTracks のリストを返します。各アルバムはそのタイトルによって一意に識別され、結果には1回だけ登場します。ヒント: (1) 前の演習問題の解法のいくつかを利用するとよいでしょう。(2) グループ化を行う関数は2つのリスト (アルバムタイトルのリストとトラックのリスト) を受け取り、後者が補助関数に渡されるまではアトムックとして扱います。演習問題 196 を参照してください。

用語: `select-` で始まる名前の関数は、いわゆるデータベースクエリ (database queries) です。詳細は「Project: Database」を参照してください。

```
+-----+
| Figure 77: Representing iTunes tracks as lists |
|
```

```

; the 2http/itunes teachpack documentation, part 2:
|
| ; An LLists is one of:
| ; - '()
| ; - (cons LAssoc LLists)
|
| ; An LAssoc is one of:
| ; - '()
| ; - (cons Association LAssoc)
| ;
| ; An Association is a list of two items:
| ; (cons String (cons BSDN '()))
|
| ; A BSDN is one of:
| ; - Boolean
| ; - Number
| ; - String
| ; - Date
|
| ; String -> LLists
| ; creates a list of lists representation for all tracks in
| ; file-name, which must be an XML export from iTunes
| (define (read-itunes-as-lists file-name)
| ...)
+-----+

```

図 77 は、2http/itunes ティーチパックがリストを使ってどのようにトラックを表現しているかを示しています。LLists はトラック表現のリストであり、そのそれぞれが String と 4 種類の値のいずれかをペアにしたリストのリストです。read-itunes-as-lists 関数は iTunes の XML ライブラリを読み込み、LLists の要素を返します。したがって、以下の定義をプログラムに追加すれば、すべてのトラック情報にアクセスできるようになります。

```

; modify the following to use your chosen name
(define ITUNES-LOCATION "itunes.xml")

; LLists
(define list-tracks
 (read-itunes-as-lists ITUNES-LOCATION))

```

そして、iTunes ライブラリが保存されているのと同じフォルダに保存します。

**Exercise 205.** LAssoc と LLists の例、つまりトラックのリスト表現およびそのようなトラックのリストの例を作成してください。

**Exercise 206.** 関数 find-association を設計してください。この関数は、key という名前の String、LAssoc、および default という名前の Any の要素という 3 つの引数を受け取ります。最初の要素が key と等しい最初の Association を返すか、そのような Association が存在しない場合は default を返します。

注: この関数を設計した後に、assoc について調べてみてください。

**Exercise 207.** LLists を受け取り、総再生時間を返す total-time/list を設計してください。ヒント: まず演習問題 206 を解いてください。

設計が完了したら、お使いの iTunes コレクションの総再生時間を計算してください。この結果を演習問題 200 の total-time 関数が計算した時間と比較してください。なぜ違いがあるのでしょうか？

**Exercise 208.** boolean-attributes を設計してください。この関数は LLists を受け取り、Boolean 属性に関連付けられている String のリストを返します。ヒント: 演習問題 201 の create-set を使用してください。

完了したら、お使いの iTunes ライブラリがトラックに採用しているブール値属性の数を特定してください。それらは意味をなしていますか？

注: リストに基づく表現は、構造体に基づく表現ほど整理されていません。この文脈では「半構造化 (semi-structured)」という言葉が使われることがあります。このようなリスト表現は、めったに現れず構造体に適合しないプロパティにも対応できます。人々は未知の情報を探索するためにこのような表現をよく使用し、フォーマットが周知のものとなった後に構造体を導入します。可能である場合に LAssoc を Track に変換する関数 track-as-struct を設計してください。

### 12.3 単語ゲーム、合成の例証 (Word Games, Composition Illustrated)

新聞や雑誌の単語パズルを解いたことがある人もいるでしょう。次の問題を考えてみてください：

サンプル問題: 与えられた単語と同じ文字から構成されるすべての単語を見つけよ。  
例えば “cat” は “act” と綴ることもできる。

例を通して考えてみましょう。あなたに “dear” という単語が与えられたとします。この 4 つの文字の並べ替え（配置）は、全部で 24 通りあります：

ader aedr aerd adre arde areddaer eadr eard dare rade raeddear edar erad drae  
rdae readdera edra erda drea rdea reda

このリストの中には、正しい単語が 3 つあります：“read”、“dear”、“dare” です。

**注:** 単語に同じ文字が 2 回含まれている場合、すべての並べ替えのコレクションには同じ文字列の複製が複数含まれることがあります。私たちの目的においては、これで問題ありません。現実的なプログラムでは、リストの代わりに集合 (set) を使用して重複するエントリを回避したくなるかもしれません。「リストと集合に関する注意」を参照してください。**注終わり**

あらゆる可能な並べ替えを系統的に列挙することは、英語辞書における検索と同様に、明らかにプログラムの役目です。本節では検索関数の設計を取り扱い、もう一方の課題の解決は次節に委ねます。この2つを分離することで、最初の節では系統的なプログラム設計のハイレベルな考え方に集中することができます。

**注:** 現実世界の辞書を扱う方法については、「現実世界のデータ: 辞書」を参照してください。

手作業でこの問題をどのように解決するか、少し想像してみましょう。もし十分な時間があれば、与えられた単語に含まれるすべての文字の可能な並べ替えをすべて列挙し、その中から辞書にも存在する異形（バリエーション）だけを選び出すことでしょう。明らかにプログラムもこのように進めることができ、これは合成 (composition) による自然な設計を示唆しています。しかし、いつものように私たちは系統的に進め、まずは入力と出力のデータ表現を選択することから始めます。

少なくとも一見したところでは、単語を String として表現し、結果を単語のリスト、すなわち List-of-strings として表現するのが自然です。この選択に基づいて、シグネチャと目的の記述を策定できます：

```
; String -> List-of-strings
; finds all words that use the same letters as s
(define (alternative-words s)
 ...)
```

次に、いくつか例が必要です。与えられた単語が “cat” の場合、c, a, t という3つの文字を扱います。いろいろ試してみると、これらの文字の並べ替えは cat, cta, tca, tac, act, atc の6通りあることがわかります。このうち2つが実際の単語です：“cat” と “act” です。alternative-words は String のリストを生成するため、結果を表現する方法は (list "act" "cat") と (list "cat" "act") の2通りあります。幸いにも、BSL には関数が2つの可能な結果のうちいずれか1つを返すことを表現する手段が備わっています：

```
(check-member-of (alternative-words "cat")
 (list "act" "cat")
 (list "cat" "act"))
```

手を止めましょう (Stop!)！ドキュメントで check-member-of について調べて読んでください。

この例を詳しく調べることで、2つの問題が浮き彫りになります：

- 1つ目はテストに関する問題です。例えば「rat」という単語を使ったとします。これには「rat」、「tar」、「art」の3つの選択肢があります。この場合、関数の結果となり得る6つのリストを策定しなければなりません。4つの可能な選択肢がある「dear」のような単語では、テストの策定はさらに困難になります。

- 2つ目の問題は単語の表現の選択に関するものです。最初はStringが自然に思えますが、例から明らかなように、私たちの関数の一部は単語を文字の列として捉え、自由自在に並べ替えることができる必要があります。String内の文字を並べ替えることも可能ですが、文字のリスト (lists of letters) の方が明らかにこの目的に適しています。

まずはテストの問題から、順を追って対処していきましょう。

alternative-words と "rat" に対するテストを策定したいとします。上記より、結果には "rat"、"tar"、"art" が含まれていなければならないことはわかっていますが、これらの単語がどのような順序で結果に現れるかはわかりません。

このような状況では、check-satisfied が役に立ちます。これを用いて、String のリストに私たちの3つのStringが含まれているかどうかをチェックする関数と一緒に使用できます：

**注：**「合間 1: 初級学生言語」を参照してください。

```
; List-of-strings -> Boolean
(define (all-words-from-rat? w)
 (and (member? "rat" w)
 (member? "art" w)
 (member? "tar" w)))
```

この関数を使えば、alternative-words のテストを策定するのは簡単です：

```
(check-satisfied (alternative-words "rat")
 all-words-from-rat?)
```

**データと設計に関する注意：**この議論が示唆しているのは、alternative-words 関数が構築するのはリストではなく集合 (set) であるということです。差異の詳細な説明については、「リストと集合に関する注意」を参照してください。ここでは、集合とは値の順序やそれらの値が何回出現するかを考慮しない値のコレクションを表すということを知っておけば十分です。プログラミング言語が集合のデータ表現をサポートしていない場合、プログラマーはここでの List-of-strings 表現のように、近い代替手段に頼る傾向があります。プログラムが大きくなるにつれて、この選択はプログラマーを悩ませるかもしれませんが、この種の問題への対処は2冊目の本のテーマです。 **注終わり**

```
+-----+
| Figure 78: Finding alternative words |
| |
| ; List-of-strings -> Boolean |
| (define (all-words-from-rat? w) |
| (and |
| (member? "rat" w) (member? "art" w) (member? "tar" w))) |
| |
| ; String -> List-of-strings |
| ; finds all words that the letters of some given word spell |
| |
| (check-member-of (alternative-words "cat")
```

```

| (list "act" "cat")
| (list "cat" "act"))
|
| (check-satisfied (alternative-words "rat")
| all-words-from-rat?)
|
| (define (alternative-words s)
| (in-dictionary
| (words->strings (arrangements (string->word s)))))
|
| ; List-of-words -> List-of-strings
| ; turns all Words in low into Strings
| (define (words->strings low) '())
|
| ; List-of-strings -> List-of-strings
| ; picks out all those Strings that occur in the dictionary
| (define (in-dictionary los) '())
+-----+

```

単語の表現に関する問題については、次の節に先送りします。具体的には、次の節で (1) 文字の並べ替えに適した Word のデータ表現、(2) List-of-words のデータ定義、(3) Word を List-of-words (すなわちすべての可能な並べ替えのリスト) にマッピングする関数を導入することにします：

```
; A Word is...
```

```
; A List-of-words is...
```

```
; Word -> List-of-words
; finds all rearrangements of word
(define (arrangements word)
 (list word))
```

**Exercise 209.** 上記により、2つの追加の願いが残されます：Stringを受け取り、対応するWordを生成する関数、および逆方向の関数です。ウィッシュリストのエントリは以下のとおりです：

```
; String -> Word
; converts s to the chosen word representation
(define (string->word s)...)

; Word -> String
; converts w to a string
(define (word->string w)...)

```

次節で Word のデータ定義を参照し、string->word と word->string の定義を完成させなさい。ヒント: BSL が提供する関数の一覧を参照するとよいでしょう。

これら2つの小さな問題が解決したところで、alternative-words の設計に戻しましょう。これで手元には、(1) シグネチャ、(2) 目的の記述、(3) 例とテスト、(4) データ表現の選択に関する洞察、(5) 問題を2つの大きなステップに分解する方法の考えが揃いました。

したがって、テンプレートを作成する代わりに、私たちが考えている合成を書き出します：

```
(in-dictionary (arrangements s))
```

この式は、与えられた単語 *s* に対して、arrangements を使用して文字のすべての可能な並べ替えのリストを作成し、in-dictionary を使用して辞書にも存在する並べ替えを選択することを意味しています。

手を止めましょう (Stop!) ! 合成がうまくいくことを確認するために、2つの関数のシグネチャを調べてみてください。正確には何をチェックする必要がありますか？

この式が捉えきれていないのは4番目の点、すなわち文字を並べ替えるためにプレーンな文字列を使わないという決定です。*s* を arrangements に渡す前に、それを単語に変換する必要があります。幸いなことに、演習問題 209 はまさにそのような関数を求めています：

```
(in-dictionary
 (... (arrangements (string->word s))))
```

同様に、得られた単語のリストを文字列のリストに変換する必要があります。演習問題 209 は単一の単語を変換する関数を求めていましたが、ここではそれらのリストを扱う関数が必要です。別の願いをする時間です：

```
(in-dictionary
 (words->strings
 (arrangements (string->word s))))
```

手を止めましょう (Stop!) ! words->strings のシグネチャは何であり、その目的は何ですか？

図 78 にすべてのピースがまとめられています。以下の演習問題では、残りの関数を設計することが求められます。

**Exercise 210.** 図 78 で指定されている words->strings 関数の設計を完成させなさい。ヒント: 演習問題 209 の解法を使用しなさい。

**Exercise 211.** 図 78 で指定されている in-dictionary の設計を完成させなさい。ヒント: 辞書の読み込み方については「現実世界のデータ: 辞書」を参照しなさい。実世界のデータ: 辞書 (Real-World Data: Dictionaries)」を参照しなさい。

## 12.4 単語ゲーム、問題の中核 (Word Games, the Heart of the Problem)

目標は、Word を受け取り、その単語の文字ごとの並べ替えのリストを生成する関数 arrangements を設計することです。この発展的な演習は、深いウィッシュリスト (wish list)、すなわち1つの関数を完成させるたびに増えていくように思える希望する関数のリストの必要性を強化します。



**注記:** 数学的な用語では順列 (permutations) です。

前述のように、String (文字列) は単語の表現として機能させることもできますが、String はアトム (不可分) であり、arrangements がその文字を並べ替える必要があるという事実そのものが、異なる表現を求めています。したがって、私たちが選択した単語のデータ表現は、入力の各要素が文字を表す 1String のリストです。

```
; A Word is one of:
; - '() or
; - (cons 1String Word)
; interpretation a Word is a List of 1Strings (letters)
```

**Exercise 212.** List-of-words のデータ定義を書き下してください。Word と List-of-words の例を作成してください。最後に、上記の機能例を check-expect で定式化してください。完全な例の代わりに、たとえば "d" と "e" のようなわずか 2 文字の単語で作業することを検討してください。

arrangements のテンプレートは、リスト処理関数のテンプレートです。

```
; Word -> List-of-words
; creates all rearrangements of the letters in w
(define (arrangements w)
 (cond
 [(empty? w)...]
 [else (... (first w)...
 ... (arrangements (rest w))...)]))
```

第 5 ステップの準備として、テンプレートの cond 行を見てみましょう。

1. 入力がある場合、入力の可能な並べ替えは 1 つしかありません。それは '()' という単語です。したがって結果は (list '()) となります。これは空リストを唯一の要素として含むリストです。
2. それ以外の場合、単語には最初の文字が存在し、(first w) がその文字となります。また、再帰呼び出しによって、単語の残りの部分に対するすべての可能な並べ替えのリストが生成されます。たとえば、リストが (list "d" "e" "r") の場合、再帰呼び出しは (arrangements (list "e" "r")) です。これは (cons (list "e" "r") (cons (list "r" "e") '())) という結果を生成します。リスト全体に対して可能なすべての並べ替えを得るには、これらのすべての単語の先頭、末尾、および可能なすべての文字の間に、最初の要素 (この例では "d") を挿入しなければなりません。

私たちの分析によると、多くの異なる単語のすべての位置に 1 つの文字をなんとか挿入できれば、arrangements を完成させることができると示唆されます。このタスク記述の最後の側面は、暗黙のうちにリストについて言及しており、本章のアドバイスに従って補助関数を求めています。この関数を insert-everywhere/in-all-words と呼び、それを使って arrangements の定義を完成させましょう。

```
(define (arrangements w)
 (cond
 [(empty? w) (list '())]
 [else (insert-everywhere/in-all-words (first w)
 (arrangements (rest w)))])
```

**Exercise 213.** `insert-everywhere/in-all-words` を設計してください。この関数は `1String` と単語のリストを消費します。結果は第2引数のような単語のリストになりますが、与えられたリストのすべての単語の先頭、すべての文字の間、および末尾に第1引数が挿入されます。

完全なウィッシュリストのエントリから始めてください。空リストのテスト、1文字の単語を持つリストのテスト、2文字の単語を持つリストのテストなどを補足してください。次に進む前に、以下の3つのヒントを注意深く学習してください。

ヒント (1) 上記の例を再考してください。それは、`"d"` を単語 `(list "e" "r")` および `(list "r" "e")` に挿入する必要があることを示しています。したがって、次の適用例は自然な例の候補の1つです。

```
(insert-everywhere/in-all-words "d"
 (cons (list "e" "r")
 (cons (list "r" "e")
 '())))
```

(2) 2つのリストを消費してその2つのリストの連結を生成する BSL+ の操作 `append` を使用したくなるでしょう。

```
> (append (list "a" "b" "c") (list "d" "e"))
(list "a" "b" "c" "d" "e")
```

`append` のような関数の開発は「Simultaneous Processing」(同時処理) のテーマです。

(3) この演習問題の解法は一連の関数です。忍耐強くデザインレシピに従い、体系的にウィッシュリストに取り組んでください。

**Exercise 214.** `arrangements` を「Word Games, Composition Illustrated」(単語ゲーム、プログラム合成の例示) の不完全なプログラムと統合してください。全テストスイートがパスすることを確認した後、お気に入りのいくつかの例で実行してください。

## 12.5 ワームに餌を与える (Feeding Worms)

Worm (Snake としても知られています) は、最も古いコンピューターゲームの1つです。ゲームが始まると、1匹のワームと1つの餌が現れます。ワームは壁に向かって移動しています。壁に到達させてはいけません。さもないとゲームオーバーになります。代わりに、矢印キーを使ってワームの動きをコントロールします。

ゲームの目標は、ワームにできるだけ多くの餌を食べさせることです。ワームが餌を食べるにつれて、体長が長くなり、より多くのセグメントが現れます。1つの餌が消化されると、別の餌が現れます。しかし、ワームの成長はワーム自身を危険にさらします。十分に

長く成長すると、自分自身に衝突する可能性があり、もし衝突すれば、その場合もゲームオーバーとなります。

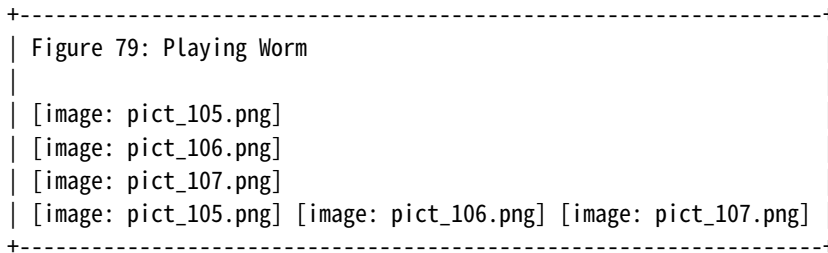


Figure 79 は、ゲームが実際にどのように動作するかを示す一連のスクリーンショットを表示しています。左側には初期設定が表示されています。ワームは1つの赤いセグメント、つまりその頭部のみで構成されています。ワームは緑色のディスクとして表示されている餌に向かって移動しています。中央のスクリーンショットは、ワームがまさに餌を食べようとしている状況を示しています。一番右のスクリーンショットでは、ワームが右の壁に衝突しています。ゲームは終了し、プレイヤーは 11 点を獲得しました。

以下の演習問題では、Worm ゲームの設計と実装をガイドします。Structures in Lists と同様に、これらの演習問題は反復的洗練 (iterative refinement) を通じて難度の高い問題に取り組む方法を示しています。つまり、対話型プログラム全体を一度に設計するのではなく、イテレーション (iteration) と呼ばれるいくつかの段階に分けて設計します。各イテレーションでは、詳細を追加してプログラムを洗練させ、あなたや顧客が満足するまで繰り返します。演習問題の結果に満足がいけない場合は、自由にバリエーションを作成してください。

**Exercise 215.** 1 セグメントのワームを継続的に移動させ、プレイヤーが 4 方向の矢印キーでワームの移動をコントロールできるようにするワールドプログラムを設計しなさい。プログラムは、ワームの唯一のセグメントを描画するために赤いディスクを使用する必要があります。クロックのティックごとに、ワームはその直径分だけ移動する必要があります。

ヒント (1) ワールドプログラムの設計方法を思い出すために Designing World Programs を再読してください。worm-main 関数を定義するときは、クロックが刻むレートを引数として使用します。レートの記述方法については on-tick のドキュメントを参照してください。(2) ワームのデータ表現を開発するときは、物理的表現と論理的表現という 2 種類の異なる表現の使用を検討してください。物理的表現はキャンバス上のワームの実際の物理的位置を追跡し、論理的表現はワームが左および上から何セグメント (の幅) 離れているかをカウントします。「ゲーム」の物理的外観 (ワームのセグメントのサイズ、ゲームボックスのサイズ) を変更しやすいのは、この 2 つのうちどちらでしょうか？

**Exercise 216.** ワームがワールドの壁に到達した場合に停止するように、exercise 215 のプログラムを修正しなさい。この条件によってプログラムが停止したとき、ワールドシーンの左下に "worm hit border" というテキストを含む最終シーンを描画する必要があります。

す。ヒント 最後のワールドを特別な方法で描画するために、big-bang の stop-when 句を使用できます。

**Exercise 217.** 尻尾を持つワームのデータ表現を開発しなさい。ワームの尻尾は、「接続された」セグメントの（空である可能性もある）シーケンスです。ここでの「接続された」とは、あるセグメントの座標がその前のセグメントの座標と高々 1 つの方向にしか変わらないことを意味します。話を単純に保つため、すべてのセグメント（頭部セグメントと尻尾セグメント）を同一に扱ってください。

ここで、複数セグメントのワームに対応するように exercise 215 のプログラムを修正しなさい。シンプルに保ちましょう：(1) プログラムはすべてのワームセグメントを赤いディスクとして描画してよく、(2) ワームが壁や自分自身に衝突する可能性は無視してください。ヒント ワームの移動を実現する 1 つの方法は、移動している方向にセグメントを 1 つ追加し、最後のセグメントを削除することです。

```
+-----+
| Figure 80: Random placement of food |
| |
| ; Posn -> Posn |
| ;??? |
| (check-satisfied (food-create (make-posn 1 1)) not=-1-1?) |
| (define (food-create p) |
| (food-check-create |
| p (make-posn (random MAX) (random MAX)))) |
| |
| ; Posn Posn -> Posn |
| ; generative recursion |
| ;??? |
| (define (food-check-create p candidate) |
| (if (equal? p candidate) (food-create p) candidate)) |
| |
| ; Posn -> Boolean |
| ; use for testing only |
| (define (not=-1-1? p) |
| (not (and (= (posn-x p) 1) (= (posn-y p) 1)))) |
| |
+-----+
```

**Exercise 218.** ワームがワールドの壁に衝突した場合、または自分自身に衝突した場合に停止するように、exercise 217 のプログラムを再設計しなさい。ワームが壁に衝突したためか、それとも自分自身に衝突したためかにプログラムが停止した理由を説明するために、exercise 216 と同様のメッセージを表示してください。

ヒント (1) ワームが自分自身に衝突するかどうかを判定するには、移動した場合に頭部の位置が古い尻尾のセグメントのいずれかと一致するかどうかを確認します。(2) member? 関数について調べてみてください。

**Exercise 219.** exercise 218 のプログラムに餌を装備させなさい。いかなる時点でも、ボックスには 1 つの餌が含まれている必要があります。シンプルにするため、1 つの餌はワー

ムのセグメントと同じサイズとします。ワームの頭部が餌と同じ位置にあるとき、ワームは餌を食べます。これは、ワームの尻尾が1セグメント拡張されることを意味します。餌が食べられると、別の餌が異なる場所に現れます。

ゲームに餌を追加するには、ワールド状態のデータ表現を変更する必要があります。状態にはワームに加えて、餌の表現、特にその現在の位置が含まれるようになります。ゲーム表現の変更は、イベントを処理するための新しい関数を示唆しますが、これらの関数は (exercise 218 からの) ワーム用の関数とそのテストケースを再利用できます。また、ティックハンドラはワームを移動させるだけでなく、食べるプロセスと新しい餌の作成を管理しなければならないことも意味します。

プログラムはボックス内にランダムに餌を配置する必要があります。これを適切に行うには、まだ見たことのない設計技法である、いわゆる生成再帰 (generative recursion。Generative Recursion で紹介されています) が必要となるため、figure 80 にこれらの関数を提供しています。ただし、これらを使用する前に、MAX が 1 より大きいと仮定してこれらの関数がどのように動作するかを説明し、その後目的文 (purpose statement) を公式化しなさい。

**Note:** random の動作については、マニュアルまたは exercise 99 をお読みください。

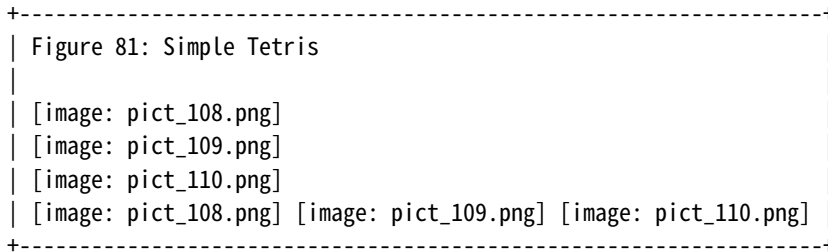
ヒント (1) 「食べる」ことの1つの解釈方法は、頭部が以前餌があった場所に移動し、頭部が以前あった場所に挿入された1つのセグメントによって尻尾が伸びる、と言うことです。なぜこの解釈は関数として設計しやすいのでしょうか？ (2) この最後のステップにおいて、worm-main 関数に2番目のパラメータとして、big-bang がワールドの現在の状態を別ウィンドウで表示するかどうかを決定する Boolean 値を追加することが役立つとわかりました。この情報を要求する方法については、state のドキュメントを参照してください。

この最後の演習問題を終わると、完全なワームゲームが完成します。ここで worm-main 関数を修正して、最終的なワームの長さを返すようにしてください。次に、DrRacket の (Racket メニューにある) Create Executable を使用して、プログラムを BSL+ について知っている人だけでなく、誰でも起動できるものに変換してください。

ゲームを真に自分自身のゲームにするために、特別な工夫を追加したくなるかもしれません。私たちは、面白いゲーム終了メッセージ、周りに複数の異なる餌を用意すること、部屋に追加の障害物を配置すること、その他いくつかのアイデアを試してみました。あなたならどのようなアイデアを思いつきますか？ の他いくつかのアイデアを試してみました。あなたはどんなことを思いつきますか？

## 12.6 簡易テトリス (Simple Tetris)

テトリス (Tetris) は、ソフトウェア黎明期のもう1つのゲームです。本格的なテトリスゲームの設計はわずかな見返りに対して膨大な労力を要するため、本節では簡略化されたバージョンに焦点を当てます。意欲的であれば、実際のテトリスの仕組みを調べて本格的なバージョンを設計してみてください。



私たちの簡略化されたバージョンでは、ゲームはシーンの上部から個々のブロックが落下することから始まります。ブロックの1つが地面に着地すると静止し、ランダムな場所から別のブロックが落下し始めます。プレイヤーは「左」および「右」の矢印キーで落下中のブロックを操作できます。ブロックがキャンバスの床、あるいはすでに静止しているブロックの上に到達すると、静止して動かさなくなります。短時間でブロックは積み上がっていきます。もしブロックの積み上がりがキャンバスの天井に達すると、ゲームオーバーです。当然ながら、このゲームの目的はできるだけ多くのブロックを着地させることです。アイデアのイラストについては figure 81 を参照してください。

この説明を踏まえて、「Designing World Programs (ワールドプログラムの設計)」にある対話型プログラムの設計ガイドラインに取り組むことができます。ガイドラインでは、定数プロパティと変数プロパティを分離することが求められます。前とができ、後者は簡易テトリスゲームの考え得るすべての状態を構成するデータを表します。以下にいくつかの例を挙げます：

- ゲームの幅と高さは、ブロックと同様に固定されています。BSL+ の観点からは、次のような定義が望ましいでしょう： (define WIDTH 10);# of blocks, horizontally(define SIZE 10);blocks are squares(define SCENE-SIZE (\* WIDTH SIZE)) (define BLOCK;red squares with black rims (overlay (square (- SIZE 1) "solid" "red") (square SIZE "outline" "black"))))読み進める前に、これらの定義を説明してください。
- ブロックの「景観 (landscapes)」は、ゲームごと、またクロックティックごとに異なります。これをもっと正確にしましょう。ブロックの見た目は同じままであり、その位置が異なります。

これで、落下中のブロックと地面のブロックの景観のためのデータ表現を設計するという中心的な問題が残ります。落下中のブロックに関しては、ここでも2つの可能性が考えられます。1つは「物理的」表現を選択することであり、もう1つは「論理的」表現です。物理的表現はキャンバス上のブロックの実際の物理的位置を追跡し、論理的表現はブロックが左および上からブロック幅何個分離れているかを数えます。静止しているブロックに関しては、個々のブロックよりもさらに多くの選択肢があります。物理的位置のリスト、論理的位置のリスト、積み上がった高さのリストなどです。

**Note:** 関連する設計上の決定については exercise 215 を参照してください。

この節では、データ表現をあらかじめ選択しておきます：

```

(define-struct tetris [block landscape])
(define-struct block [x y])

; A Tetris is a structure:
; (make-tetris Block Landscape)
; A Landscape is one of:
; - '()
; - (cons Block Landscape)
; A Block is a structure:
; (make-block N N)

; interpretations
; (make-block x y) depicts a block whose left
; corner is (* x SIZE) pixels from the left and
; (* y SIZE) pixels from the top;
; (make-tetris b0 (list b1 b2...)) means b0 is the
; dropping block, while b1, b2, and... are resting

```

これが論理的表現と呼ばれるものです。なぜなら、座標はブロックの物理的な位置を反映しておらず、原点からのブロックサイズの個数を表しているだけだからです。この選択は、 $x$  が常に 0 と WIDTH の間 (WIDTH は含まない) であり、 $y$  が 0 と HEIGHT の間 (HEIGHT は含まない) であることを意味しますが、私たちはこの知識を無視します。

**Exercise 220.** テトリスゲームの状態に対する定義のように、複雑なデータ定義が提示された場合は、まず多様なデータコレクションのインスタンスを作成することから始めます。以下は、後で関数の例として使用できる、示唆に富んだ例の名称です：

```

(define landscape0...)
(define block-dropping...)
(define tetris0...)
(define tetris0-drop...)
...
(define block-landed (make-block 0 (- HEIGHT 1)))
...
(define block-on-block (make-block 0 (- HEIGHT 2)))

```

与えられた Tetris のインスタンスを Image に変換するプログラム `tetris-render` を設計しなさい。DrRacket の対話領域 (interactions area) を使用して、(きわめて) シンプルなデータ例のいくつかを描画する式を作成しなさい。次に、それらの関数例をユニットテストおよび関数自体として定式化しなさい。

**Exercise 221.** キャンバスの上部から直線状に落下し、床または既に静止しているブロックの上に着地するブロックを表示する対話型プログラム `tetris-main` を設計しなさい。`tetris-main` への入力により、クロックが刻むレートを決定するようにしなさい。レートの指定方法については `on-tick` のドキュメントを参照しなさい。

ブロックが着地したかどうかを発見するために、ブロックを落下させて床の上にあるか、または静止しているブロックのリストにあるいずれかのブロックと重なっているかを

チェックすることを推奨します。ヒント：組込み関数 `member?` についておさらいしてください。

ブロックが着地したとき、プログラムは直ちに現在の列の右隣の列から降下する別のブロックを生成する必要があります。現在のブロックが既に最も右の列にある場合、次のブロックは最も左の列を使用する必要があります。あるいは、現在の列とは異なる列をランダムに選択する関数 `block-generate` を定義しなさい。インスピレーションを得るには `exercise 219` を参照してください。

**Exercise 222.** プレイヤーが落下中のブロックの水平方向の動きを操作できるように、`exercise 221` のプログラムを修正しなさい。プレイヤーが「left」矢印キーを押すたびに、列 0 にあるか、またはその左側に既に静止ブロックの山がある場合を除き、落下中のブロックは左に 1 列シフトする必要があります。同様に、プレイヤーが「right」を押すたびに、可能であれば落下中のブロックは右に 1 列移動する必要があります。

**Exercise 223.** `exercise 222` のプログラムに `stop-when` 句を備え付けなさい。いずれかの列にキャンバスの天井に「触れる」のに十分なブロックが含まれると、ゲームは終了します。

`exercise 223` を解くと、最小限のテトリスゲームが完成します。友達に見せる前に、少し磨きをかけたくなるかもしれません。例えば、最終キャンバスにプレイヤーがいくつブロックを積み上げることができたかを示すテキストを表示することができます。あるいは、すべてのキャンバスにそのようなテキストを含めることもできます。選択はあなた次第です。

## 12.7 本格的なスペースウォーズ (Full Space War)

「要素の列挙と構造体 (Itemizations and Structures)」では、アクションがほとんどないスペースインベーダーゲームをほのめかしていました。プレイヤーは地上部隊を前後に動かすことしかできませんでした。「リストとワールド (Lists and World)」では、プレイヤーが希望するだけ多くの弾を発射できるようになります。この節では、このゲームを完成させるのに役立つ練習問題を出題します。

いつものように、UFO が地球に着陸しようとしています。プレイヤーの任務は UFO の着陸を防ぐことです。この目的のために、ゲームには任意の数の弾を発射できる戦車が用意されています。これらの弾の 1 つが UFO の重心に十分に近づくと、ゲームは終了し、プレイヤーの勝ちとなります。UFO が地面に十分に近づくと、プレイヤーの負けとなります。

**Exercise 224.** 前の 2 つの節から学んだ教訓を活用し、ゲームの機能を 1 つずつ追加しながら、ゲームの拡張をゆっくりと設計しなさい。常に設計レシピを使用し、補助関数のガイドラインを頼りにしなさい。このゲームが気に入ったら、他の機能を追加してみてください。ランニングテキストを表示する、戦車を破壊できる電荷を UFO に装備する、攻撃する UFO の艦隊全体を作成する、そして何よりも想像力を働かせてください。

UFO と戦車が撃ち合うのが好きではない場合は、同じアイディアを使って、似たような文明的なゲームを作成してください。



### Exercise 225. 消火ゲームを設計しなさい。

このゲームは、広大な森林で火災が猛威を振るう西部諸州を舞台にしています。これは空中消火活動をシミュレートしたものです。具体的には、プレイヤーは地面の火災に水を投下する飛行機のパイロットとして行動します。プレイヤーは飛行機の水平方向の移動と水の投下を制御します。

作成するゲームソフトウェアは、地面のランダムな場所に火災を発生させます。火災の数を制限し、現在燃えている火災の数やその他の要因の関数にしてもよいでしょう。ゲームの目的は、制限時間内にすべての火災を消火することです。ヒント：このゲームを作成するには、この章で説明されている反復的な設計アプローチを使用してください。

### 12.8 有限状態機械 (Finite State Machines)

有限状態機械 (FSM: Finite state machine) と正規表現は、プログラミングにおいてどこにでも存在する要素です。「有限状態の世界 (Finite State Worlds)」で説明したように、状態機械はワールドプログラムについて考える 1 つの方法です。逆に、演習問題 109 は、FSM を実装し、プレイヤーが特定の一連のキープレスをを行ったかどうかをチェックするワールドプログラムの設計方法を示しています。

思い出されるかもしれませんが、有限状態機械は正規表現と等価です。したがって、コンピュータ科学者は、演習問題 109 の

$a(b|c)^*d$

のような、特定の正規表現に一致するキー入力の列を FSM が受容する (accepts) と言いがちです。もし別のパターン、例えば

$a(b|c)^*a$

を認識するプログラムが欲しい場合、既存のプログラムを適切に修正するだけで済みます。これら 2 つのプログラムは互いに似たものになり、いくつかの異なる正規表現に対してこの演習を繰り返すと、最終的には似たような見た目のプログラムが大量に出来上がることになります。

自然な考え方としては、一般解、つまり FSM のデータ表現を受け取り、プレイヤーがマッチするキーのシーケンスを押したかどうかを認識するワールドプログラムを探すことです。本節では、まさにそのようなワールドプログラムの設計を提示しますが、これは大幅に単純化されたものです。具体的には、この FSM には初期状態や終了状態がなく、マッチングでは実際のキー入力を無視します。代わりに、いずれかのキーが押されるたびに、ある状態から別の状態への遷移が起こります。さらに、状態を色の文字列とすることを要求します。そうすれば、FSM を解釈するプログラムは、単に現在の状態を色として表示できます。

設計上の選択に関する注記 ここに一般化のもう 1 つの試みがあります。

サンプル問題 与えられた FSM を特定の KeyEvents のリスト上で解釈するプログラムを設計しなさい。つまり、このプログラムは FSM のデータ表現と文字列を受け取ります。文字列が FSM に対応する正規表現に一致する場合、その結果は #true になり、それ以外の場合は #false になります。

しかしながら、判明することですが、最初の 2 つのパートの原則だけでこのプログラムを設計することはできません。実際、この問題を解決するには「バックトラックするアルゴリズム (Algorithms that Backtrack)」まで待つ必要があります。演習問題 476 を参照してください。+-----

```
+ | Figure 82: Representing and interpreting finite state machines in general | |
| |; An FSM is one of: | |; -
'|() | |; - (cons Transition FSM)
| | | |
(define-struct transition [current next]) | |; A Transition is a
structure: | |; (make-transition FSM-State FSM-State)
| | | |; FSM-State is a Color.
| | | |; interpretation An FSM
represents the transitions that a | |; finite state machine can take from one state
to another | |; in reaction to keystrokes |
+-----+
```

単純化された問題の記述は、FSM の表現のためのデータ定義の必要性、その状態の性質、および画像としての見た目を含み、いくつかの点を要求しています。図 82 (Figure 82) はこの情報をまとめたものです。まず FSM のデータ定義から始まります。お分かりのように、FSM は単なる Transition のリストです。ワールドプログラムが任意の FSM に対して動作するようにしたいので、リストを使用する必要があります。それは、有限であるものの、任意に大きい数の状態を意味します。各 Transition は、構造体の中で 2 つの状態を組み合わせます。現在の状態 (current state) と次の状態 (next state)、つまりプレイヤーがキーを押したときにマシンが遷移する状態です。データ定義の最後の部分では、状態は単なる色の名前であると述べています。

```
+-----+
| Figure 82: Representing and interpreting finite state machines in general |
| |; An FSM is one of: | |
| |; - '|'() |
| |; - (cons Transition FSM) |
| |
| (define-struct transition [current next]) |
| |; A Transition is a structure: |
| |; (make-transition FSM-State FSM-State) |
| |
| |; FSM-State is a Color. |
| |
| |; interpretation An FSM represents the transitions that a |
+-----+
```

```
|; finite state machine can take from one state to another
|; in reaction to keystrokes
+-----+
```

**Exercise 226.** 状態のための同等性述語 `state=?` を設計しなさい。

この定義は複雑であるため、設計レシピに従って例を作成します。

```
(define fsm-traffic
 (list (make-transition "red" "green")
 (make-transition "green" "yellow")
 (make-transition "yellow" "red")))

(define fsm-traffic
 (list (make-transition "red" "green")
 (make-transition "green" "yellow")
 (make-transition "yellow" "red")))
```

この遷移表が信号機を記述していることに、おそらくお気づきでしょう。その最初の遷移は、信号機が“red”から“green”へ跳ぶことを示し、2番目の遷移は“green”から“yellow”への遷移を表し、最後の遷移は“yellow”から“red”への遷移です。

**Exercise 227.** BW Machine は、すべてのキーイベントに対して黒から白へ、そして白から黒へと反転する FSM です。BW Machine のデータ表現を定式化しなさい。

明らかに、私たちの問題に対する解決策はワールドプログラムです。

```
; FSM ->???
; match the keys pressed with the given FSM
(define (simulate an-fsm)
 (big-bang...
 [to-draw...]
 [on-key...]))
```

このプログラムは FSM を受け取ることになっていますが、プログラムが何を生成すべきかについては手がかりがありません。プレイヤーのキープレスに応じて与えられた FSM のように振る舞うため、このプログラムを `simulate` と呼びます。

とにかくワールドプログラムの設計レシピに従って、どこまで進められるかを見てみましょう。設計レシピは、「現実世界」において変化するものと変化しないものを区別するよう教えてくれます。`simulate` 関数は FSM のインスタンスを受け取りますが、この FSM が変化しないこともわかっています。変化するのはマシンの現在の状態です。

この分析は以下のデータ定義を示唆しています。

```
; A SimulationState.v1 is an FSM-State.
```

ワールドプログラムの設計レシピによれば、このデータ定義によりメイン関数が完成します。

**注記:** empty-image 定数は「不可視の」画像を表します。これは、描画関数のヘッダーを書き出すための優れたデフォルト値です。

```
(define (simulate.v1 fsm0)
 (big-bang initial-state
 [to-draw render-state.v1]
 [on-key find-next-state.v1]))
```

そして、2つのエントリを持つウィッシュュリストを意味します。

```
; SimulationState.v1 -> Image
; renders a world state as an image
(define (render-state.v1 s)
 empty-image)

; SimulationState.v1 KeyEvent -> SimulationState.v1
; finds the next state from ke and cs
(define (find-next-state.v1 cs ke)
 cs)
```

このスケッチは2つの疑問を提起します。第一に、まさに最初の SimulationState.v1 がどのように決定されるかという問題です。現在、選択されている状態 initial-state は、この問題について警告するために灰色でマーキングされています。第二に、ウィッシュュリストの2番目のエントリは、いくらかの狼狽を引き起こすに違いありません。

与えられているのが現在の状態とキー入力だけである場合、find-next-state は一体どのようにして次の状態を見つけることができるのでしょうか？

単純化された問題の記述によれば、キー入力の正確な性質が無関係であるため、この疑問は特に当てはまります。どのキーが押されたかに関係なく、FSM は次の状態へと遷移するからです。

この2番目の問題が露呈しているのは、BSL+ の根本的な限界です。この限界を理解するために、まずは回避策 (work-around) から始めます。基本的に、この分析は find-next-state 関数が現在の状態だけでなく FSM も受け取り、遷移のリストを検索して次の状態を選択できるようにすることを要求します。言い換えると、ワールドの状態には FSM の現在の状態と FSM 自体の両方が含まれていなければなりません。

**注記:** 最初の2人のコンピュータ科学者であるアロンゾ・チャーチとアラン・チューリングは、1930年代にすべてのプログラミング言語が数値上の特定関数を計算できることを証明しました。したがって、彼らはすべてのプログラミング言語が同等であると主張しました。本書の第一著者はこれに同意しません。彼は、プログラマーが解法を表現することを言語がどのように許可しているかに基づいて、言語を区別します。

```
(define-struct fs [fsm current])
; A SimulationState.v2 is a structure:
; (make-fs FSM FSM-State)
```

ワールド設計レシピによれば、この変更はキーイベントハンドラもこの組み合わせを返さなければならないことを意味します。

```
; SimulationState.v2 -> Image
; renders a world state as an image
(define (render-state.v2 s)
 empty-image)

; SimulationState.v2 KeyEvent -> SimulationState.v2
; finds the next state from ke and cs
(define (find-next-state.v2 cs ke)
 cs)
```

最後に、メイン関数は2つの引数、すなわち FSM とその最初の状態を受け取らなければならなくなりました。結局のところ、simulate が受け取るさまざまな FSM にはあらゆる種類の状態が備わっており、それらすべてが同じ初期状態を持っていると仮定することはできません。改訂された関数ヘッダーは以下の通りです。

```
; FSM FSM-State -> SimulationState.v2
; match the keys pressed with the given FSM
(define (simulate.v2 an-fsm s0)
 (big-bang (make-fs an-fsm s0)
 [to-draw state-as-colored-square]
 [on-key find-next-state]))
```

信号機の FSM の例に戻りましょう。このマシンについては、マシンと “red” に simulate を適用するのが最善でしょう。

```
(simulate.v2 fsm-traffic "red")
```

止まって考えよう！なぜ信号機にとって “red” が良いのだと思いますか？

**注記:** エンジニアは “red” を安全な状態 (safe state) と呼びます。

表現力に関する注記 回避策を踏まえると、これで BSL の限界を説明できます。与えられた FSM はシミュレーションの過程で変化しないにもかかわらず、その記述はワールドの状態の一部にならなければなりません。理想的には、プログラムは FSM の記述が一定のままであることを表現すべきですが、代わりにプログラムは FSM を常に変化する状態の一部として扱わなければなりません。プログラムの読者は、big-bang の最初の部分だけからこの事実を推測することはできません。

本書の次のパートでは、新しいプログラミング言語と特定の言語構造である ISL とローカル定義 (local definitions) の導入によって、この難題を解決します。詳細については、「ローカル定義は表現力を追加する (Local Definitions Add Expressive Power)」を参照し

てください。現時点で、ウィッシュリストに戻り、そのエントリを1つずつ処理していくことができます。最初のもの、state-as-colored-square の設計は非常に単純であるため、完全な定義をそのまま提示します。

```
; SimulationState.v2 -> Image
; renders current world state as a colored square
```

```
(check-expect (state-as-colored-square
 (make-fs fsm-traffic "red"))
 (square 100 "solid" "red"))
```

```
(define (state-as-colored-square an-fsm)
 (square 100 "solid" (fs-current an-fsm)))
```

対照的に、キーイベントハンドラの設計はいくらかの議論に値します。ヘッダーの内容を思い出してください。

```
; SimulationState.v2 KeyEvent -> SimulationState.v2
; finds the next state from an-fsm and ke
(define (find-next-state an-fsm ke)
 an-fsm)
```

設計レシピによれば、ハンドラはワールドの状態と KeyEvent を受け取り、ワールドの次の状態を生成しなければなりません。シグネチャを平易な言葉で明確に表現することは、例の設計のガイドにもなります。最初の2つの例は以下の通りです。

```
(check-expect
 (find-next-state (make-fs fsm-traffic "red") "n")
 (make-fs fsm-traffic "green"))
(check-expect
 (find-next-state (make-fs fsm-traffic "red") "a")
 (make-fs fsm-traffic "green"))
```

これらの例が言っているのは、現在の状態が fsm-traffic マシンとその “red” 状態を組み合わせている場合、プレイヤーがキーボードで “n” または “a” のどちらを叩いたかに関係なく、結果は同じ FSM と “green” を組み合わせたものになるということです。もう1つの例を以下に示します。

```
(check-expect
 (find-next-state (make-fs fsm-traffic "green") "q")
 (make-fs fsm-traffic "yellow"))
```

先を読む前に、この例を解釈してください。他の例を思い出すことができますか？

この関数は構造体を受け取るため、構造体処理のためのテンプレートを書き出します。

```
(define (find-next-state an-fsm ke)
 (... (fs-fsm an-fsm).. (fs-current an-fsm)...))
```

さらに、望ましい結果が SimulationState.v2 であるため、適切なコンストラクタを追加してテンプレートを洗練させることができます。

```
(define (find-next-state an-fsm ke)
 (make-fs
 ... (fs-fsm an-fsm)... (fs-current an-fsm)...))
```

例が示唆しているのは、抽出された FSM が新しい SimulationState.v2 の最初のコンポーネントになり、この関数は実際には現在の状態と与えられた FSM を構成する Transitions のリストから次の状態を計算するだけでよいということです。後者は任意に長いので、私たちは願い (wish) を作成します。それは、リストをトラバースして current 状態が (fs-current an-fsm) である Transition を探す find 関数です。そして、定義を完成させます。

```
(define (find-next-state an-fsm ke)
 (make-fs
 (fs-fsm an-fsm)
 (find (fs-fsm an-fsm) (fs-current an-fsm))))
```

新しい願いの定式化は以下の通りです。

```
; FSM FSM-State -> FSM-State
; finds the state representing current in transitions
; and retrieves the next field
(check-expect (find fsm-traffic "red") "green")
(check-expect (find fsm-traffic "green") "yellow")
(check-error (find fsm-traffic "black")
 "not found: black")
(define (find transitions current)
 current)
```

これらの例は find-next-state の例から導き出されています。

止まって考えよう！ いくつか追加の例を作成し、それから演習問題に取り組みなさい。

**Exercise 228.** find の設計を完成させなさい。

補助関数がテストされたら、simulate を使用して fsm-traffic や演習問題 227 の BW Machine を動かして遊びなさい。

私たちのシミュレーションプログラムは意図的にかなり制限されています。特に、プレイヤーがどのキーを押すかに応じてある状態から別の状態に遷移する FSM を表現するために使用することはできません。しかし、体系的な設計を踏まえれば、プログラムを拡張してそのような機能を持たせることができます。

**Exercise 229.** 以下は Transition の改訂されたデータ定義です。

```
(define-struct ktransition [current key next])
; A Transition.v2 is a structure:
; (make-ktransition FSM-State KeyEvent FSM-State)
```

Transition.v2 のリストを使用して演習問題 109 の FSM を表現しなさい。エラーと最終状態は無視してかまいません。

キー入力を適切な方法で処理するように simulate の設計を修正しなさい。データ例の適応から始めて、設計レシピに従いなさい。

改訂されたプログラムを使用して、以下のキー入力シーケンスにおける演習問題 109 の FSM の実行をシミュレートしなさい：“a”、“b”、“b”、“c”、“d”。

有限状態機械には通常、初期状態と最終状態が伴います。FSM を「実行する」プログラムが最終状態に達したとき、プログラムは停止すべきです。最後の演習問題では、これらの概念を導入するためにもう一度 FSM のデータ表現を改訂します。

**Exercise 230.** FSMs に関する以下のデータ表現を考えなさい。

```
(define-struct fsm [initial transitions final])
(define-struct transition [current key next])
; An FSM.v2 is a structure:
; (make-fsm FSM-State LOT FSM-State)
; A LOT is one of:
; - '()
; - (cons Transition.v3 LOT)
; A Transition.v3 is a structure:
; (make-transition FSM-State KeyEvent FSM-State)
```

このコンテキストにおいて演習問題 109 の FSM を表現しなさい。

FSM.v2 を受け取り、プレイヤーのキー入力上でそれを実行する関数 fsm-simulate を設計しなさい。キー入力のシーケンスによって FSM.v2 が最終状態に達した場合、fsm-simulate は停止します。ヒント: この関数は、与えられた fsm 構造体の initial フィールドを使用して現在の状態を追跡します。

反復的洗練に関する注記 (Note on Iterative Refinement) これら最後の 2 つのプロジェクトは、「反復的洗練による設計 (design by iterative refinement)」の概念を導入しています。基本的な考え方は、最初のプログラムは望ましい動作のほんの一部しか実装せず、次のプログラムはもう少し実装し、これを繰り返していくというものです。最終的に、望ましい動作のすべて、あるいは少なくとも顧客を満足させるのに十分な動作を示すプログラムが得られます。詳細については、「反復的洗練 (Iterative Refinement)」を参照してください。

```
+-----+
| Figure 82: Representing and interpreting finite state machines in general |
| |
| ; An FSM is one of: |
| ; - '() |
| ; - (cons Transition FSM) |
| |
| (define-struct transition [current next]) |
| ; A Transition is a structure: |
| ; (make-transition FSM-State FSM-State) |
| |
| ; FSM-State is a Color. |
+-----+
```



  ; interpretation An FSM represents the transitions that a  ; finite state machine can take from one state to another  ; in reaction to keystrokes +-----+	     
-------------------------------------------------------------------------------------------------------------------------------------------------------------------------	----------------

## 13 まとめ (Summary)

本書のこの第2部は、任意に大きなデータ (arbitrarily large data) を扱うプログラムの設計について扱っています。簡単に想像できるように、ソフトウェアはあらかじめサイズ制限が指定されていない情報を扱うときに特に有用であり、「任意に大きなデータ」を扱えるようになることは、本物のプログラマーになるための道のりにおいて極めて重要なステップです。このような観点から、以下の3つのレッスンを持ち帰ることをお勧めします。

1. このパートでは、データ定義における自己参照 (self-references) および相互参照 (cross-references) を扱うために設計レシピを洗練させます。前者の出現は再帰関数の設計を要求し、後者の出現は補助関数を要求します。
2. 複雑な問題は、個別の問題への分解 (decomposition) を必要とします。問題を分解する際、2つの要素が必要になります。それは、個別の問題を解決する関数と、それらの個別の解決策を単一の解決策へと合成 (composition) するデータ定義です。個別のプログラムに時間を費やした後、合成が確実に機能するように、必要なデータ定義とともに自らの「願い (wishes)」を定式化する必要があります。分解と合成の設計 (decomposition-composition design) は、問題記述が暗黙的または明示的に補助タスクに言及している場合、関数のコーディング手順において (別の) 任意に大きなデータの一部分のトラバース (全件処理) が求められる場合、そして——驚くべきことかもしれませんが——一般的な問題のほうが、問題記述で説明されている特定の問題よりもいくぶん解決しやすい場合に特に有用です。
3. 具象的・実用的な知識 (プラグマティクス) が重要です。big-bang プログラムを設計したい場合は、そのさまざまな句 (clause) とそれらが何を達成するかを理解する必要があります。あるいは、数学的な問題を解くプログラムを設計することがタスクであるなら、選択した言語とそのライブラリがどのような数学的演算を提供しているかをあらかじめ確認しておくべきです。

このパートでは、任意に大きなデータの優れた例として主にリストに焦点を当てました。なぜなら、リストは Haskell、Lisp、ML、Racket、Scheme などの言語で実用的に便利だからです。しかし、ここでの考え方は、ファイル、ファイルフォルダ、データベースなど、あらゆる種類の同様のデータに適用できます。

『相互に絡み合うデータ』(Intertwined Data) では、「大きな」構造化データの探求を継続し、最も複雑な種類のデータに対して設計レシピがどのようにスケール (適用) するかを示します。それまでの間、次のパートでは、皆さんが現時点で抱えているかもしれない重要な懸念、すなわち「プログラマーの仕事とは、同じ種類のプログラムを何度も何度も何度も繰り返し作成することばかりなのだろうか」という疑問に対処します。

## **Intermezzo 2: Quote, Unquote**

(今後の翻訳作業のためのプレースホルダー)

### III 抽象化 (Abstraction)

## 導入：抽象化の必要性

多くのデータ定義と関数定義は似ています。例えば、String のリストの定義は Number のリストの定義と、データクラスの名前と「String」「Number」という単語の 2 箇所しか異なりませ。同様に、String のリストから特定の文字列を探す関数は、Number of lists から特定の数を探す関数とほとんど区別がつきません。

経験から、このような類似性は問題を引き起こすことがわかっています。類似性は、プログラマがコードを（物理的または精神的に）コピーすることから生じます。プログラマが別の問題に直面したとき、既存の解決策をコピーして新しい問題に合わせて修正します。この行動は「本物の」プログラミングだけでなく、スプレッドシートや数学的モデリングの世界でも見られます。

しかし、コードのコピーは以下のような問題を引き起こします： - ミスのコピー - 同じ修正を多くのコピーに適用する必要性 - 基礎データ定義が変更されたときにすべてのコピーを探して修正するコスト

このプロセスは高価でエラーが発生しやすく、プログラミングチームに不必要なコストを課します。

良いプログラマは、プログラミング言語が許す限り類似性を排除しようとしています。

「プログラムはエッセイのようなものです。最初のバージョンは下書きであり、下書きには編集が必要です。」

「排除する」とは、プログラマがプログラムの最初のドラフトを書き留め、類似性（および他の問題）を見つけ、それらを取り除くことを意味します。最後のステップでは、**抽象化**を行うか、既存の（抽象化された）関数を使用します。このプロセスを何度か繰り返すことで、プログラムを満足のいく形にすることがよくあります。

このパートの前半では、関数とデータ定義の類似性を抽象化する方法を示します。プログラマはこのプロセスの結果も「抽象化」と呼びます。後半は、既存の抽象化の使用と、このプロセスを容易にする新しい言語要素についてです。このパートの例はリストの領域から取られていますが、考え方は普遍的に適用可能です。

---

## 14 類似性はどこにでもある (Similarities Everywhere)

Arbitrarily Large Data の演習（の一部）を解いた人は、多くの解決策が似ていることを知っているでしょう。実際、類似性は、ある問題の解決策をコピーして次の問題の解決策を作成する誘惑に駆られるかもしれません。

しかし、**thou shall not steal code**（コードを盗んてはならない）、たとえ自分のコードであってもです。代わりに、類似したコード片を**抽象化**しなければなりません。この章では、その抽象化の方法を教えます。

私たちの手段は「Intermediate Student Language」（ISL）に特有のものです。

DrRacket で、「Language」メニューの「How to Design Programs」サブメニューから「Intermediate Student」を選択してください。

ほとんどすべての他のプログラミング言語も同様の手段を提供しています。オブジェクト指向言語では、追加の抽象化メカニズムが見つかるかもしれません。いずれにせよ、これらのメカニズムは本章で述べる基本的な特性を共有しており、ここで説明する設計の考え方は他の文脈でも適用されます。

### 14.1 関数における類似性 (Similarities in Functions)

設計レシピは、関数の基本的な組織を決定します。なぜなら、テンプレートは関数の目的に関係なくデータ定義から作成されるからです。したがって、同じ種類のデータを消費する関数は似ているのは当然です。

以下は 2 つの非常に似た関数です（図 86）：

```
; Los -> Boolean
; does l contain "dog"
(define (contains-dog? l)
 (cond
 [(empty? l) #false]
 [else
 (or
 (string=? (first l) "dog")
 (contains-dog? (rest l)))]))

; Los -> Boolean
; does l contain "cat"
(define (contains-cat? l)
 (cond
 [(empty? l) #false]
 [else
 (or
 (string=? (first l) "cat")
 (contains-cat? (rest l)))]))
```

図 86: 2 つの類似した関数

これらの関数はほぼ区別が付きません。各々は文字列のリストを消費し、本体は2つの節を持つ cond 式で構成されます。入力が空リストなら #false を返し、or を使って最初の要素が目的の文字列かどうかを調べ、そうでなければ再帰的に残りを調べます。唯一の違いは比較する文字列です（図 86 の “dog” と “cat”）。

良いプログラマは、このような密接に関連する関数を複数定義するのを避けます。代わりに、探す文字列を追加の引数として受け取る 1 つの汎用関数を定義します：

```
; String Los -> Boolean
; determines whether l contains the string s
(define (contains? s l)
 (cond
 [(empty? l) #false]
 [else
 (or (string=? (first l) s)
 (contains? s (rest l)))])))
```

これで contains-dog? と contains-cat? は 1 行で定義できます：

```
; Los -> Boolean
; does l contain "dog"
(define (contains-dog? l)
 (contains? "dog" l))

; Los -> Boolean
; does l contain "cat"
(define (contains-cat? l)
 (contains? "cat" l))
```

図 87: 2 つの類似した関数、再考

これが**関数的抽象化 (Functional Abstraction)**です。異なるバージョンの関数を抽象化することは、プログラムから類似性を排除する一つの方法であり、類似性を排除することで、長期にわたってプログラムを健全に維持することが容易になります。

**Exercise 235.** contains? を使って “atom”, “basic”, “zoo” を探す関数を定義せよ。

**Exercise 236.** 次の 2 つの関数のテストスイートを作成せよ：

```
; Lon -> Lon
; adds 1 to each item on l
(define (add1* l)
 (cond
 [(empty? l) '()]
 [else
 (cons (add1 (first l)) (add1* (rest l)))])))

; Lon -> Lon
; adds 5 to each item on l
(define (plus5 l)
 (cond
```

```

[(empty? l) '()]
[else
 (cons (+ (first l) 5) (plus5 (rest l)))))]

```

これらを抽象化せよ。抽象化した関数を使って上記2つを1行で再定義し、テストで確認せよ。最後に、各数から2を引く関数を設計せよ。

## 14.2 異なる類似性 (Different Similarities)

contains-dog? と contains-cat? のケースでは抽象化は簡単に見えました。2つの関数定義を比較し、リテラル文字列を関数パラメータに置き換え、元の関数が抽象関数を使って簡単に定義できることを確認するだけでした。この種の抽象化は非常に自然なため、本書の前半2つのパートでも特に断ることなく登場していました。

本節では、これと同じ原理が、より強力な抽象化の形態をどのように生み出すかを示します。図 88 を見てください。どちらの関数も数値のリストと閾値を消費します。左側は閾値未満のすべての数値のリストを生成し、右側は閾値を超えるすべての数値のリストを生成します。

```

; Lon Number -> Lon
; select those numbers on l
; that are below t
(define (small l t)
 (cond
 [(empty? l) '()]
 [else
 (cond
 [(< (first l) t) (cons (first l) (small (rest l) t))]
 [else (small (rest l) t)]])))

; Lon Number -> Lon
; select those numbers on l
; that are above t
(define (large l t)
 (cond
 [(empty? l) '()]
 [else
 (cond
 [(> (first l) t) (cons (first l) (large (rest l) t))]
 [else (large (rest l) t)]])))

```

図 88: さらに2つの類似した関数

この2つの関数は、与えられたリストの数値が結果の一部になるべきかどうかを決定する「比較演算子」の1箇所だけが異なります。左側の関数は<を使用し、右側は>を使用しています。それ以外は、関数名を除いて2つの関数は同一です。

最初の例に従って、追加のパラメータを使ってこれら2つの関数を抽象化しましょう。今回は、追加のパラメータは文字列ではなく、比較演算子（関数）を表します：



```
(define (extract R l t)
 (cond
 [(empty? l) '()]
 [else
 (cond
 [(R (first l) t)
 (cons (first l) (extract R (rest l) t))]
 [else
 (extract R (rest l) t))])]))
```

この新しい関数を適用するには、3つの引数を指定する必要があります：2つの数値を比較する関数R、数値のリストl、そして閾値tです。この関数は、(R i t) が #true に評価されるl内のすべての要素iを抽出します。

[!NOTE] ここで、この定義が意味をなすのか疑問に思うかもしれません。私たちは何の苦労もなく、「関数を消費する関数」を作成しました。微分演算や不定積分を学んだことがあるなら、これらが関数を消費して関数を生成する関数であることを知っているでしょう。しかし、本コースではその知識を仮定しません。私たちの単純な学習言語ISLはこのような関数をサポートしており、このような関数を定義することは優れたプログラマの最も強力なツールの1つです。

テストを実行すると、(extract < l t) が (small l t) と同じ結果を計算することがわかります：

```
(check-expect (extract < '() 5) (small '() 5))
(check-expect (extract < '(3) 5) (small '(3) 5))
(check-expect (extract < '(1 6 4) 5) (small '(1 6 4) 5))
```

同様に、(extract > l t) は (large l t) と同じ結果を生成します。つまり、元の2つの関数を次のように再定義できます：

```
; Lon Number -> Lon
(define (small-1 l t) (extract < l t))
```

```
; Lon Number -> Lon
(define (large-1 l t) (extract > l t))
```

重要な洞察は、small-1 と large-1 が1行で定義できることだけではありません。extract のような抽象関数があれば、他の場所でも有効に活用できます：  
 \* (extract = l t): l 内の t と等しい数値をすべて抽出します。  
 \* (extract <= l t): l 内の t 以下の数値をすべて抽出します。  
 \* (extract >= l t): l 内の t 以上の数値をすべて抽出します。

実際、extract の第1引数はISLの定義済み演算である必要はありません。2つの引数を消費して真偽値を生成する任意の関数を使用できます。次の例を考えてみましょう：

```
; Number Number -> Boolean
; is the area of a square with side x larger than c
```

```
(define (squared>? x c)
 (> (* x x) c))
```

すなわち、squared>? は平方  $x^2 > c$  が成り立つかどうかを検査し、extract で使用できます：

```
(extract squared>? (list 3 4 5) 10)
```

この適用により、(list 3 4 5) の中で平方した値が 10 より大きい数値（この場合は 4 と 5）が抽出されます。

**Exercise 237.** DrRacket で (squared>? 3 10) と (squared>? 4 10) を評価せよ。(squared>? 5 10) はどうなるか。

抽象化された関数定義は、元の関数よりも便利になり得ることがわかりました。たとえば、contains? は contains-dog? や contains-cat? よりも便利であり、extract は small や large よりも便利です。抽象化のこれらの利点は、文書作成、スプレッドシート、小さなアプリ、大規模な産業プロジェクトなど、あらゆるレベルのプログラミングで得られます。

また、抽象化のもう 1 つの重要な側面は、これらの関数に対して**単一の制御点 (Single Point of Control)** を持てることです。抽象関数に間違いがあることが判明した場合、その定義を修正するだけで、他のすべての定義も修正されます。同様に、抽象関数の計算を高速化する方法を見つければ、この関数を使用して定義されたすべての関数が追加の労力なしに改善されます。

```
; Nelon -> Number
; determines the smallest number on l
(define (inf l)
 (cond
 [(empty? (rest l)) (first l)]
 [else
 (if (< (first l) (inf (rest l)))
 (first l)
 (inf (rest l)))])])

; Nelon -> Number
; determines the largest number on l
(define (sup l)
 (cond
 [(empty? (rest l)) (first l)]
 [else
 (if (> (first l) (sup (rest l)))
 (first l)
 (sup (rest l)))])])
```

図 89: リスト内の最小値 (inf) と最大値 (sup) の探索

**Exercise 238.** 図 89 の 2 つの関数を 1 つの関数に抽象化せよ。どちらも空でない数値のリスト (Nelon) を消費し、1 つの数値を生成します。左側はリスト内の最小値を生成し、右側は最大値を生成します。

抽象関数を用いて inf-1 と sup-1 を定義せよ。以下の 2 つのリストでテストせよ：

```
(list 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1)
(list 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25)
```

なぜこれらの関数は長いリストの一部で遅くなるのか。元の関数を min (2 つの数値から小さい方を選択) および max (大きい方を選択) を使って修正せよ。その後、再度抽象化し、inf-2 と sup-2 を定義して、同じ入力でテストせよ。なぜこれらのバージョンははるかに高速になるのか。

### 14.3 データ定義における類似性 (Similarities in Data Definitions)

ここで、次の 2 つのデータ定義を詳しく見てみましょう：

```
; An Lon (List-of-numbers) is one of:
; - '()
; - (cons Number Lon)

; An Los (List-of-strings) is one of:
; - '()
; - (cons String Los)
```

左側は数値のリストを導入し、右側は文字列のリストを記述しています。そして、これら 2 つのデータ定義は非常に似ています。類似した関数と同様に、2 つのデータ定義は異なる名前を使用していますが、名前は任意であるためこれは重要ではありません。唯一の本当の違いは、2 番目の節の cons の最初の位置にあり、リストがどのような種類の要素を含むかを指定している点です。

この 1 つの違いを抽象化するために、データ定義を関数であるかのように扱います。パラメータを導入し、異なる型 (要素) を参照していた場所にこのパラメータを使用します：

```
; A [List-of ITEM] is one of:
; - '()
; - (cons ITEM [List-of ITEM])
```

このような抽象的なデータ定義を **パラメータ化されたデータ定義 (Parametric Data Definitions)** と呼びます。大雑把に言えば、パラメータ化されたデータ定義は、関数が特定の値から抽象化するのと同様に、特定のデータコレクションへの参照から抽象化します。

問題は、もちろん、これらのパラメータがどのような範囲をとるかです。関数のパラメータが「未知の値」を表すのに対し、データ定義のパラメータは「データのクラス全体 (型)」を表します。パラメータ化されたデータ定義に具体的なデータコレクションの名前を割り当てるプロセスを **インスタンス化 (Instantiation)** と呼びます。以下は List-of 抽象化のサンプルインスタンス化です： \* [List-of Number] と書くとき、ITEM が Number を表すため、これは List-of-numbers の別名になります。\* 同様に、[List-of String] は List-of-strings と同じデータクラスを定義します。\* 以下のような在庫レコード IR のクラスを定義した場合： racket (define-struct IR [name price]) ; An IR is a structure: (make-IR String Number) [List-of IR] は在庫レコードのリストの名前になります。

慣例として、データ定義のパラメータには大文字の名前（例：ITEM, X, Y）を使用します。

この抽象データ定義が正当であることを確認するには、パラメータ ITEM に実際のデータ定義名（たとえば Number）を代入し、自己参照（再帰）を展開してみます。得られる定義は従来の数値リストの定義と完全に一致します。

2 番目の例として、次の構造体定義から始めましょう：

```
(define-struct point [hori veri])
```

この構造体タイプを使用する 2 つの異なるデータ定義を示します：

```
; A Pair-boolean-string is a structure:
; (make-point Boolean String)
```

```
; A Pair-number-image is a structure:
; (make-point Number Image)
```

この場合、データ定義は 2 箇所異なります。hori フィールドに対応する違いと、veri フィールドに対応する違いです。したがって、抽象データ定義を作成するには 2 つのパラメータを導入する必要があります：

```
; A [CP H V] is a structure:
; (make-point H V)
```

ここで、H は hori フィールドのデータコレクション用のパラメータであり、V は veri フィールドに表示されるデータコレクション用のパラメータです。

2 つのパラメータを持つデータ定義をインスタンス化するには、データコレクションの 2 つの名前が必要です。パラメータに Number と Image を使用すると、数値と画像を組み合わせたポイント構造体のコレクションを記述する [CP Number Image] が得られます。対照的に、[CP Boolean String] は真偽値と文字列をポイント構造体で組み合わせます。

**Exercise 239.** 2 つの要素のリストは、ISL プログラミングで頻繁に使用されるもう 1 つのデータ形式です。以下は、2 つのパラメータを持つデータ定義です：

```
; A [List X Y] is a List:
; (cons X (cons Y '()))
```

この定義をインスタンス化して、以下のデータクラスを記述せよ：1. 数値のペア（Number と Number） 2. 数値と 1String のペア 3. 文字列と真偽値のペア

また、これら 3 つのデータ定義のそれぞれについて、具体的な例を 1 つずつ作成せよ。

パラメータ化されたデータ定義が得られたら、それらを組み合わせて大きな効果を得ることができます。次の定義を考えてみましょう：

```
; [List-of [CP Boolean Image]]
```

最も外側の表記は [List-of ...] であり、これはリストを扱っていることを意味します。このリストがどのようなデータを含むかという問いに答えるには、内部の式を調べる必要

があります：[CP Boolean Image]。これは真偽値と画像をポイント内で組み合わせたものです。含意として、上記は真偽値と画像を組み合わせたポイントのリストです。同様に、

```
; [CP Number [List-of Image]]
```

は、1つの数値と画像のリストを組み合わせたCPのインスタンス化です。

**Exercise 240.** 奇妙ですが類似した次の2つのデータ定義があります：

```
; An LStr is one of:
; - String
; - (make-layer LStr)

; An LNum is one of:
; - Number
; - (make-layer LNum)
```

どちらのデータ定義も、次の構造体定義に依存しています：

```
(define-struct layer [stuff])
```

両者について具体例を作成せよ。2つを抽象化したデータ定義を作成せよ。そして、抽象定義をインスタンス化して元の定義を復元せよ。

**Exercise 241.** NEList-of-temperatures と NEList-of-Booleans の定義を比較せよ。そして、空でないリストを表す抽象データ定義 NEList-of を策定せよ。

**Exercise 242.** 以下は、もう1つのパラメータ化されたデータ定義です：

```
; A [Maybe X] is one of:
; - #false
; - X
```

これらのデータ定義を解釈せよ：[Maybe String], [Maybe [List-of String]], [List-of [Maybe String]]。

次の関数署名は何を意味するか：

```
; String [List-of String] -> [Maybe [List-of String]]
; returns the remainder of los starting with s
; #false otherwise
(check-expect (occurs "a" (list "b" "a" "d" "e")) (list "d" "e"))
(check-expect (occurs "a" (list "b" "c" "d")) #false)
(define (occurs s los) los)
```

設計レシピの残りのステップを実行せよ。

## 14.4 関数は値である (Functions Are Values)

本パートの関数は、プログラムの評価に関する私たちの理解を拡張します。関数が数値だけでなく、文字列や画像を消費することは理解しやすいでしょう。構造体やリストは少し拡張が必要ですが、結局のところ有限の「もの」です。しかし、関数を消費する関数は奇

妙に思えます。実際、このアイデア自体が最初のインターメッツォ（BSL）の規則に2つの方法で違反しています：

1. プリミティブや関数の名前が、適用式の引数として使用されている。2.

パラメータが、適用式の「関数位置」で使用されている。

問題を詳しく説明すると、ISLの文法がBSLとどのように異なるかがわかります。第1に、式言語は定義内の関数名とプリミティブ演算の名前を含めるべきです。第2に、適用式の最初の位置は、関数名やプリミティブ演算以外のもの（変数や関数のパラメータなど）を許容しなければなりません。

文法への変更は評価規則への変更を要求するようには見えますが、変更されるのは「値の集合」だけです。具体的には、関数を関数の引数として受け入れるために、最も単純な変更は「関数やプリミティブ演算もまた値である」とすることです。

**Exercise 243.** DrRacket の定義領域に以下が含まれていると仮定します：

```
(define (f x) x)
```

以下の式のうち、値（値の定義を満たすもの）を特定せよ：1. (cons f '())

2. (f f) 3. (cons f (cons 10 (cons (f 10) '()))))

なぜそれらが値である（または値でない）のかを説明せよ。

**Exercise 244.** 次の記述が ISL において合法（文法的に正しい）である理由を論じよ：1.

(define (f x) (x 10)) 2. (define (f x) (x f)) 3. (define (f x y) (x 'a y 'b))

あなたの推論を説明せよ。

**Exercise 245.** function=at-1.2-3-and-5.775? 関数を設計せよ。数値から数値への2つの関数が与えられたとき、この関数は、1.2、3、および -5.775 に対して2つの関数が同じ結果を生成するかどうかを判定します。

数学者は、同じ入力を与えられたときに同じ結果を計算する場合、2つの関数は等しいと言います。数値から数値への2つの関数が等しいかどうかを判定する一般関数 function=? を定義することは可能でしょうか。もし可能なら定義せよ。不可能なら、その理由を説明し、これが「簡単に定義できるアイデアであるにもかかわらず、その関数を定義できない最初の例」に遭遇したことを意味するインプリケーションを考慮せよ。

## 14.5 関数を使った計算 (Computing with Functions)

BSL+ から ISL への移行により、関数を引数として使用し、適用式の最初の位置にパラメータ名を使用できるようになりました。DrRacket はこれらの位置にある名前を他の場所と同様に処理しますが、当然ながら結果として関数を期待します。驚くべきことに、代数の法則を単純に適用するだけで、ISL プログラムを評価することができます。

extract の評価がどのように機能するかを見てみましょう。明らかに、

```
(extract < '() 5) == '()
```

が成り立ちます。「Intermezzo 1」の代入モデルを使用して、関数の本体で計算を続けることができます。パラメータ  $R, l, t$  が、引数  $<, '(), 5$  にそれぞれ置き換えられます。ここからは、条件式の評価から始まるプレーンな計算です：

```
==
(cond
 [(empty? '()) '()]
 [else
 (cond
 [(< (first '()) 5)
 (cons (first '()) (extract < (rest '()) 5))]
 [else (extract < (rest '()) 5)]]])
==
(cond
 [#true '()]
 [else
 (cond
 [(< (first '()) 5)
 (cons (first '()) (extract < (rest '()) 5))]
 [else (extract < (rest '()) 5)]]])
== '()
```

次に、要素が1つのリストを考えます：

```
(extract < (cons 4 '()) 5)
```

リストの唯一の要素は4であり、 $(< 4 5)$  は真であるため、結果は  $(cons 4 '())$  になるはずです。評価の最初のステップは次のようになります：

```
(extract < (cons 4 '()) 5)
==
(cond
 [(empty? (cons 4 '())) '()]
 [else
 (cond
 [(< (first (cons 4 '())) 5)
 (cons (first (cons 4 '()))
 (extract < (rest (cons 4 '())) 5))]
 [else (extract < (rest (cons 4 '())) 5)]]])
```

やはり、 $R$  は  $<$ ,  $l$  は  $(cons 4 '())$ ,  $t$  は  $5$  に置き換えられます。残りは簡単です：

```
(cond
 [(empty? (cons 4 '())) '()]
 [else
 (cond
 [(< (first (cons 4 '())) 5)
 (cons (first (cons 4 '()))
 (extract < (rest (cons 4 '())) 5))]
 [else (extract < (rest (cons 4 '())) 5)]]])
==
```



```

(cond
 [#false '()]
 [else
 (cond
 [(< (first (cons 4 '())) 5)
 (cons (first (cons 4 '()))
 (extract < (rest (cons 4 '())) 5))]]
 [else (extract < (rest (cons 4 '())) 5)]]])
=
(cond
 [(< (first (cons 4 '())) 5)
 (cons (first (cons 4 '()))
 (extract < (rest (cons 4 '())) 5))]]
 [else (extract < (rest (cons 4 '())) 5)]]
=
(cond
 [(< 4 5)
 (cons (first (cons 4 '()))
 (extract < (rest (cons 4 '())) 5))]]
 [else (extract < (rest (cons 4 '())) 5)]]

```

ここが重要なステップであり、代入された位置で < が実際に使用されています。そして、計算が続きます：

```

=
(cond
 [#true
 (cons (first (cons 4 '()))
 (extract < (rest (cons 4 '())) 5))]]
 [else (extract < (rest (cons 4 '())) 5)]]
=
(cons 4 (extract < (rest (cons 4 '())) 5))
=
(cons 4 (extract < '() 5))
=
(cons 4 '())

```

最後の例は、2つの要素のリストへの適用です：

```

(extract < (cons 6 (cons 4 '())) 5)
= (extract < (cons 4 '()) 5)
= (cons 4 (extract < '() 5))
= (cons 4 '())

```

ステップ1は、閾値未満でない場合に extract がリストの最初の要素を除外するケースを扱っています。

**Exercise 246.** DrRacket の Stepper (ステップ実行) を使用して、最後の計算のステップ1を確認せよ。



**Exercise 247.** DrRacket の Stepper を使用して `(extract < (cons 8 (cons 4 '())) 5)` を評価せよ。

**Exercise 248.** DrRacket の Stepper で `(squared>? 3 10)` と `(squared>? 4 10)` を評価せよ。

次の対話を考えてみます：

```
> (extract squared>? (list 3 4 5) 10)
(list 4 5)
```

Stepper が示すステップの一部を以下に示します：

```
(extract squared>? (list 3 4 5) 10)
; (1) ==
(cond
 [(empty? (list 3 4 5)) '()]
 [else
 (cond
 [(squared>? (first (list 3 4 5)) 10)
 (cons (first (list 3 4 5)) (extract squared>? (rest (list 3 4 5)) 10))]
 [else (extract squared>? (rest (list 3 4 5)) 10)]]])

; (2) == ... ==
(cond
 [(squared>? 3 10)
 (cons (first (list 3 4 5)) (extract squared>? (rest (list 3 4 5)) 10))]
 [else (extract squared>? (rest (list 3 4 5)) 10)])
```

Stepper を使用して、行 (1) から (2) へのステップを確認せよ。さらにステップを進めて、ステップ (2) から最終結果までの隙間を埋めよ。各ステップを代入モデルや関数の評価ルールを用いて説明せよ。

**Exercise 249.** 関数は「値」であり、引数、結果、リストの要素になり得ます。以下の定義と式を DrRacket の定義ウィンドウに入力し、Stepper を使ってこのプログラムがどのように動作するかを調べよ：

```
(define (f x) x)
(cons f '())
(f f)
(cons f (cons 10 (cons (f 10) '()))))
```

---

## 15 抽象化の設計 (Designing Abstractions)

本質的に、抽象化とは「具体的なものをパラメータに変換すること」です。前章でもこれを行いました。類似した関数定義を抽象化するには、定義内の具体的な値を置き換えるパラメータを追加します。類似したデータ定義を抽象化するには、パラメータ化されたデータ定義を作成します。他のプログラミング言語に触れると、その抽象化メカニズムもパラメータの導入を必要とすることを目にするでしょう（それが関数の引数であるとは限りませんが）。

### 15.1 例からの抽象化 (Abstractions from Examples)

あなたが足し算を初めて学んだとき、具体的な例を使って作業しました。親はおそらく指を使って小さな2つの数を足すことを教えたでしょう。後になって、任意の2つの数を足す方法を学びました。そこで初めてある種の抽象化に触れました。さらに後で、摂氏から華氏への温度変換や、与えられた速度と時間で車が移動する距離を計算する式を定式化することを学びました。要するに、非常に具体的な例から抽象的な関係へと移行したのです。

```
; List-of-numbers -> List-of-numbers
; converts a list of Celsius temperatures to Fahrenheit
(define (cf* l)
 (cond
 [(empty? l) '()]
 [else
 (cons (C2F (first l)) (cf* (rest l))))])

; Inventory -> List-of-strings
; extracts the names of toys from an inventory
(define (names i)
 (cond
 [(empty? i) '()]
 [else
 (cons (IR-name (first i)) (names (rest i))))])

; Number -> Number
; converts one Celsius temperature to Fahrenheit
(define (C2F c)
 (+ (* 9/5 c) 32))

(define-struct IR [name price])
; An IR is a structure: (make-IR String Number)
; An Inventory is one of:
; - '()
; - (cons IR Inventory)
```

図 90: 似た 2 つの関数

本節では、例から抽象化を作成するための設計レシピを紹介します。抽象化を作成すること自体は簡単ですが、既存の抽象化を見つけて利用するという「難しい部分」は次の節に譲ります。

「類似性はどこにでもある」のエッセンスを思い出してください。2つの具体的な定義から始め、それらを比較し、違いをマークし、そして抽象化します。抽象化を作成する手順は、ほとんどこれだけです：

### 1. 類似点と相違点の比較 (Compare)

ほぼ同じで、名前と特定の位置にある具体的な値だけが異なる2つの関数定義を見つけたら、それらを比較して相違点をマークします。もし違いが複数箇所にある場合は、対応する違い同士を結びつけます。図 90 は、類似した関数定義のペアを示しています。2つの関数はリストを処理し、各要素に関数を適用しています。違いは「どの関数を要素に適用するか」という点だけです。

### 2. パラメータ化 (Parameterize)

相違点としてマークした部分を新しいパラメータ名 (引数) に置き換え、パラメータリストに追加します。図 90 の例では、相違点を `g` という名前に置き換えます。これにより本質的な違いが排除されます。次に、非本質的な違い (関数名やリスト引数の名前など) を統一します。関数名を `map1`、リストパラメータを `k` と呼ぶことにすると、図 91 のように、完全に同一の関数定義が得られます。racket (define (map1 k g) (cond [(empty? k) '()] [else (cons (g (first k)) (map1 (rest k) g))])) 図 91: 抽象化された同一の関数定義

違いが複数ある場合は、対応する違いのラインごとに1つの追加パラメータを導入します。そして、すべての再帰呼び出し部分にも追加パラメータを忘れずに引き渡すように変更します。

3. 検証と再定義 (Validate) 新しい抽象関数が、元の関数の正しい抽象化であることを検証します。これは、元の関数を抽象関数を使って定義し直し、既存のテストスイートが正常に通るか確認することを意味します。元の関数が `f-original` で引数を 1 つ消費し、抽象関数が `abstract` であるとし、`f-original` が他方と `val` という値の使用において異なる場合、次の定義を行います：

```
(define (f-from-abstract x)
 (abstract x val))
```

この `f-from-abstract` が `f-original` と同等であることをテストします。先ほどの例に戻ると：

```
(define (cf*-from-map1 l) (map1 l C2F))
(define (names-from-map1 i) (map1 i IR-name))
```

元のテストケースが以下のように定義されていたとします：

```
(check-expect (cf* (list 100 0 -40)) (list 212 32 -40))
(check-expect (names (list (make-IR "doll" 21.0) (make-IR "bear" 13.0)))
 (list "doll" "bear"))
```

これらを次のように書き換えて実行し、成功を確認します：

```
(check-expect (cf*-from-map1 (list 100 0 -40)) (list 212 32 -40))
(check-expect (names-from-map1 (list (make-IR "doll" 21.0) (make-IR "bear" 13.0)))
 (list "doll" "bear"))
```

4. **署名の定式化 (Signature)** 新しい抽象関数に署名 (Signature) を与えます。これについては 15.2 で詳しく扱います。

新しい抽象化を行ったら、他に使える用途がないか確認します。用途が多ければ、その抽象化は本当に有用です。たとえば、map1 を使って数値リストの各要素に 1 を加える関数は以下のように定義できます：

```
; List-of-numbers -> List-of-numbers
(define (add1-to-each l)
 (map1 l add1))
```

このような汎用的な関数がすでに言語の一部として提供されていることもよくあります (Chapter 16 を参照)。

```
; Number -> [List-of Number]
; tabulates sin between n and 0 (incl.) in a list
(define (tab-sin n)
 (cond
 [(= n 0) (list (sin 0))]
 [else (cons (sin n) (tab-sin (sub1 n)))]))

; Number -> [List-of Number]
; tabulates sqrt between n and 0 (incl.) in a list
(define (tab-sqrt n)
 (cond
 [(= n 0) (list (sqrt 0))]
 [else (cons (sqrt n) (tab-sqrt (sub1 n)))]))
```

図 92: 演習 250 のための類似した関数

```
; [List-of Number] -> Number
; computes the sum of the numbers on l
(define (sum l)
 (cond
 [(empty? l) 0]
 [else (+ (first l) (sum (rest l)))]))

; [List-of Number] -> Number
; computes the product of the numbers on l
(define (product l)
 (cond
 [(empty? l) 1]
 [else (* (first l) (product (rest l)))]))
```

図 93: 演習 251 のための類似した関数

**Exercise 250.** 図 92 の 2 つの関数の抽象化である `tabulate` を設計せよ。設計が完了したら、それを使って `sqr` (平方) と `tan` の表を作成する関数を定義せよ。

**Exercise 251.** 図 93 の 2 つの関数の抽象化である `fold1` を設計せよ。

**Exercise 252.** 図 94 の 2 つの関数の抽象化である `fold2` を設計せよ。この演習と演習 251 を比較せよ。どちらも `product` 関数を含んでいますが、この演習は 2 番目の関数 `image*` が `Posn` のリストを消費して `Image` を生成するため、難易度が高くなっています。得られた設計と前演習の設計を比較し、署名の抽象化についての洞察を得よ。

```
; [List-of Number] -> Number
(define (product l)
 (cond
 [(empty? l) 1]
 [else (* (first l) (product (rest l)))]))

; [List-of Posn] -> Image
(define (image* l)
 (cond
 [(empty? l) emt]
 [else (place-dot (first l) (image* (rest l)))]))

; Posn Image -> Image
(define (place-dot p img)
 (place-image dot (posn-x p) (posn-y p) img))

; 描画用定数
(define emt (empty-scene 100 100))
(define dot (circle 3 "solid" "red"))
```

図 94: 演習 252 のための類似した関数

## 15.2 署名における類似性 (Similarities in Signatures)

関数の署名 (Signature) は、その再利用の鍵となります。したがって、抽象化された関数を表現するのに十分な「最も一般的な署名」を記述する方法を学ぶ必要があります。

基本的に、署名はデータ定義と同じです。どちらもデータのクラスを指定しますが、データ定義はクラスに名前を付けるのに対し、署名はそうしません。それでも、以下のコード：

```
; Number Boolean -> String
(define (f n b) "hello world")
```

の最初の行は、`Number` と `Boolean` を消費して `String` を生成するすべての関数のクラスを表しています。

したがって、抽象化の設計レシピは署名にも適用できます。類似した署名を比較し、相違点を強調し、それらをパラメータで置き換えます。

`cf*` と `names` の元の署名を比較してみましょう： `* cf* : [List-of Number] -> [List-of Number]` \*  
`names : [List-of IR] -> [List-of String]`

比較すると、矢印の左側で `Number` と `IR` が異なり、右側で `Number` と `String` が異なります。これら 2 つの相違点をパラメータ `X` と `Y` に置き換えると、次のようになります：

```
; [List-of X] -> [List-of Y]
```

このシグネチャに現れる変数  $x$  と  $y$  は、データ定義のパラメータ（Chapter 14.3）と同じようにデータのクラス全体を表します。これらをインスタンス化（代入）すると、元の署名が得られます： $*x$  と  $y$  に `Number` を代入すると、`cf*` の署名になります。 $*x$  に `IR`、 $y$  に `String` を代入すると、`names` の署名になります。

次に、追加の関数引数を受け取る `map1` の署名を考えます。`map1` の署名は次のようになります：

```
; [X Y] [List-of X] [X -> Y] -> [List-of Y]
```

具体的には、`map1` は  $x$  という型に属する要素のリストを消費します。また、 $x$  の要素を消費して  $y$  という（また別の）型の要素を生成する関数を消費します。そして、結果として  $y$  のリストを返します。

もう 1 つの例として、演習 252 の `product` と `image*` の署名を抽象化します： $*product$  の署名：`[List-of Number] Number [Number Number -> Number] -> Number`  $*image*$  の署名：`[List-of Posn] Image [Posn Image -> Image] -> Image`

これらを比較すると、すべての `Number` が一様に対応しているわけではなく：

1. ある `Number` は `Posn` に対応している（リストの要素）。2. 別の

`Number` は `Image` に対応している（ベースケースと結果の型）。

したがって、2 つのパラメータ  $x$  と  $y$  を導入し、`fold2` の署名は次のようになります：

```
; [X Y] [List-of X] Y [X Y -> Y] -> Y
```

$x$  と  $y$  の両方に `Number` を代入すると `product` の署名が得られ、 $x$  に `Posn`、 $y$  に `Image` を代入すると `image*` の署名が得られることを確認してください。

署名を抽象化するためのステップを以下にまとめます：1. **シグネチャの比較**: 2 つの類似した関数の具体的な署名を比較し、相違点をマークします。2. **パラメータの抽出**: 抽象化によって追加されたパラメータが何を表しているかを特定し、対応するプレースホルダーを割り当てます。

3. **変数の置き換え**: 相違点を表す型パラメータ ( $x$ ,

$y$  など) を導入し、統一された汎用的な署名を作成します。4. **検証**: 型パラメータに具体的な型を代入し、元の個々の署名が正しく復元されることを確認します。

**Exercise 253.** 以下のそれぞれの署名は関数のクラスを表しています：1. `[Number -> Boolean]` 2. `[Boolean String -> Boolean]` 3. `[Number Number Number -> Number]` 4. `[Number -> [List-of Number]]` 5. `[[List-of Number] -> Boolean]`

それぞれのクラスに属する具体的な関数の例を ISL で記述せよ。

**Exercise 254.** 以下の関数の署名を策定せよ： $*sort-n$ : 数値のリストと、2 つの数値を比較して真偽値を返す関数を消費し、ソートされた数値のリストを生成する。 $*sort-s$ : 文字列のリストと、2 つの文字列を比較して真偽値を返す関数を消費し、ソートされた文字列のリストを生成する。

その後、2つの署名を抽象化せよ。さらに、一般化された署名が、在庫レコード (IR) のリストをソートする関数の署名としてインスタンス化できることを示せ。

**Exercise 255.** 以下の関数の署名を策定せよ： \* map-n: 数値のリストと、数値から数値への関数を消費し、数値のリストを生成する。 \* map-s: 文字列のリストと、文字列から文字列への関数を消費し、文字列のリストを生成する。

その後、2つの署名を抽象化せよ。また、一般化された署名が上記 map1 の署名としてインスタンス化できることを示せ。

### 15.3 単一の制御点 (Single Point of Control)

プログラムは文章の下書き (ドラフト) のようなものです。誤字の修正、文法エラーの修正、文章の一貫性の確保、そして重複の排除などの編集作業は、文章を良くするために不可欠です。誰も同じことを何度も繰り返す文章を読みたくはありませんし、そのようなプログラムも読みたくはありません。

重複を排除して抽象化を作成することには、多くの利点があります。定義がシンプルになり、既存の関数の潜在的な不具合 (境界の不一致など) が明らかになります。しかし、最も重要な利点は、共通の機能に対して**単一の制御点 (Single Point of Control)** を作ることです。

機能の定義を1箇所にまとめることで、プログラムのメンテナンスが非常に容易になります： \* バグ (間違い) を発見したとき、1箇所だけを修正すればよくなります。 \* 新しいデータ形式に対応する必要があるとき、1箇所にコードを追加するだけで済みます。 \* アルゴリズムを改善したり高速化したりしたとき、1箇所の変更ですべての利用箇所が恩恵を受けます。

もしコードをコピー & ペーストしていた場合、すべてのコピー箇所を探し出して修正しなければならず、ミスの修正漏れが発生する危険性が非常に高くなります。

したがって、次のガイドラインを強く推奨します：

[!IMPORTANT] コードをコピーして修正する代わりに、抽象化を作成せよ。

抽象化の構築はプログラミングスキルを高めるための最も強力な練習の1つです。優れたプログラマは、重複を見つけると積極的にプログラムをリファクタリング (編集) し、新しい抽象化を構築して制御点を1箇所に集中させます。

### 15.4 テンプレートからの抽象化 (Abstractions from Templates)

リストを処理する多くの関数は、データ定義から自動的に導かれる共通のテンプレートに従っています。したがって、関数が互いに似通った形になるのは必然です。

実は、私たちは個々の関数からだけでなく、**テンプレートそのものから直接抽象化を作成することができます**。リストを走査する関数の基本テンプレートを思い出してください：

```
(define (fun-for-l l)
 (cond
 [(empty? l) ...]
 [else (... (first l) ... (fun-for-l (rest l)) ...)]))
```

このテンプレートには、各節に1つずつ、合計2つの「空欄 (gap)」があります。リスト処理関数を定義するとき、最初の節の空欄をベース値で埋め、2番目の節の空欄を「組み合わせる（結合する）関数」で埋めます。この結合関数は、リストの最初の要素と、残りのリストに対する再帰呼び出しの結果を組み合わせます。

この仕組みをパラメータ化して抽象関数を定義すると、次のようになります：

```
; [X Y] [List-of X] Y [X Y -> Y] -> Y
(define (reduce l base combine)
 (cond
 [(empty? l) base]
 [else
 (combine (first l)
 (reduce (rest l) base combine))]))
```

この reduce を使うと、リスト処理関数の多くを1行で定義できるようになります。たとえば、数値のリストの総和 sum と総乗 product は次のように再定義できます：

```
; [List-of Number] -> Number
(define (sum lon) (reduce lon 0 +))

; [List-of Number] -> Number
(define (product lon) (reduce lon 1 *))
```

sum では、空リストの場合のベース値は0であり、組み合わせる関数は+です。product では、ベース値は1であり、組み合わせる関数は\*です。このように、多くのリスト処理関数が reduce を使って簡潔に再定義できます。

---



## 17 無名関数 (Nameless Functions)

### 17.1 lambda から生まれる関数 (Functions from lambda)

lambda の構文は次のとおりです：

```
(lambda (variable-1 ... variable-N) expression)
```

特徴はキーワード `lambda` です。続いて括弧で囲まれた変数列、そして結果を計算する式があります。

例（1 引数）：

```
(lambda (x) (expt 10 x))
(lambda (x) (if (> x 0) x (- 0 x)))
(lambda (x) (string-append "hello " x))
```

これらは無名関数です。インタラクショナルエリアで直接使用できます。

### 17.2 lambda を使った計算 (Computing with lambda)

lambda 式は関数値として評価されます。

### 17.3 lambda を使った抽象化 (Abstracting with lambda)

lambda を使ってその場で関数を作成し、`map` や `filter` と組み合わせます。

```
(map (lambda (x) (* x x)) (list 1 2 3))
```

### 17.4 lambda を使った仕様 (Specifying with lambda)

テストケースや目的文で `lambda` を使って動作を指定できます。

### 17.5 lambda を使った表現 (Representing with lambda)

lambda は関数を値として表現するための強力な手段です。

## **Intermezzo 3: Scope and Abstraction**

(今後の翻訳作業のためのプレースホルダー)

## IV 絡み合ったデータ (Intertwined Data)

(今後の翻訳作業のためのプレースホルダー)

## **Intermezzo 4: The Nature of Numbers**

(今後の翻訳作業のためのプレースホルダー)

## V 生成的再帰 (Generative Recursion)

(今後の翻訳作業のためのプレースホルダー)

## **Intermezzo 5: The Cost of Computation**

(今後の翻訳作業のためのプレースホルダー)

## VI 蓄積子 (Accumulators)

(今後の翻訳作業のためのプレースホルダー)

## **エピローグ: 先へ進む (Epilogue: Moving On)**

(今後の翻訳作業のためのプレースホルダー)



## 付録 A: Quick — 絵で学ぶ Racket 入門

原題: Quick: An Introduction to Racket with Pictures

著者: Matthew Flatt

原本: extracted/appendix/quick/original\_markdown\_00\_index.md

このチュートリアルは、絵を描くライブラリのひとつを使って、Racket プログラミング言語への短い入門を提供します。たとえ芸術的な目的で Racket を使うつもりがなくても、絵のライブラリは興味深く、理解を深める例を支えてくれます。結局のところ、一枚の絵は五百個の「hello world」に匹敵するのです。

同様に、例は DrRacket で実行することを想定しています。DrRacket は、たとえ最終的に Emacs や vi、その他のエディタで Racket を使うとしても、言語とシステムの感触をつかむいちばん速い方法です。

### 1 用意……

Racket をダウンロードしてインストールし、DrRacket を起動してください。

### 2 セット……

**注:** DrRacket のドキュメントを見ると、DrRacket IDE の概要がわかります。

絵を描くには、まず絵用の関数を読み込む必要があります。それらはスライド発表を作るためのライブラリの一部です。次を **定義エリア** (DrRacket で見える上側のテキスト領域) にコピーしてください。

```
#lang slideshow
```

それから **Run** ボタンをクリックします。テキストキャレットが下側のテキスト領域——**対話エリア (interactions area)** ——へ移るのが見えるはずです。

以前に DrRacket を使ったことがある場合は、Run をクリックする前に、メニュー **Language | Choose Language…** から「ソースに宣言された言語を使う」ように DrRacket をリセットする必要があるかもしれません。

### 3 スタート！

対話ウィンドウの `>` のあとに式を打ち、Enter を押すと、DrRacket はその式を評価して結果を表示します。式は単なる値でもよく、たとえば数値 5 や文字列 "art gallery" です。

```
> 5
5
> "art gallery"
"art gallery"
```

式は関数呼び出しでもかまいません。関数を呼ぶには、関数名の前に開き括弧を置き、続けて引数となる式を書き、閉じ括弧を置きます。次のようにします。

```
> (circle 10)
[image:pict.png]
```

circle 関数の結果は **絵 (picture) の値** で、数値や文字列と同様に、式の結果として表示されます。circle への引数は、円の大きさ (ピクセル単位) を決めます。予想どおり、rectangle 関数もあり、こちらは引数を 1 つではなく 2 つ取ります。

```
> (rectangle 10 20)
[image:pict_2.png]
```

わざと circle に誤った個数の引数を与えて、何が起きるか試してみてください。

```
> (circle 10 20)
circle: arity mismatch;
 the expected number of arguments does not match the given
number
 expected: 1 plus optional arguments with keywords
#:border-color and #:border-width
 given: 2
arguments...:
 10
 20
```

DrRacket は、エラーを引き起こした式をピンクで強調表示します (ただし、このドキュメントではピンクの強調は示されていません)。

circle や rectangle のような基本的な絵の構築子に加え、絵を組み合わせる hc-append 関数があります。Racket で関数呼び出しを入れ子にし始めると、次のようになります。

```
> (hc-append (circle 10) (rectangle 10 20))
[image:pict_3.png]
```

名前 hc-append のハイフンは識別子の一部です。hc マイナス append ではありません。関数名が h で始まるのは絵を **水平 (horizontal)** に組み合わせるからで、次の文字が c なのは絵を **垂直方向の中央 (center)** に揃えるからです。

他にどんな関数があるか——たとえば絵を垂直に積み、左揃えにする方法は？——と気になったら、DrRacket でテキストキャレットを hc-append という名前の上に置き、**F1** キーを押してください。ブラウザが開き、hc-append のドキュメントへのリンクが表示されます。リンクをクリックすると、他にもたくさんの関数が見つかります。

この文書を HTML で読んでいる場合は、hc-append や、このチュートリアルで使われている他のインポートされた識別子をクリックしてもかまいません。

## 4 定義

特定の円や長方形の絵を何度も使うなら、名前を付けるほうが簡単です。定義エリア (上側) に戻り、次の 2 つの定義を追加して、定義エリア全体が次のようになるようにしてください。

```
#lang slideshow
(define c (circle 10))
(define r (rectangle 10 20))
```

もう一度 **Run** をクリックします。これで、単に `c` や `r` と打てます。

```
> r
[image:pict_4.png]
> (hc-append c r)
[image:pict_5.png]
> (hc-append 20 c r c)
[image:pict_6.png]
```

見てのとおりに、`hc-append` は絵の引数の前に **省略可能な数値引数** を受け取り、また任意個数の絵の引数を受け取ります。数値が与えられたとき、それは絵と絵の間に入れる間隔を指定します。

`c` と `r` の `define` 形式は、定義エリアではなく対話エリアで評価することもできました。ただし実際には、**定義エリアがプログラム本体**——保存するファイル——であり、対話エリアは一時的な探索やデバッグ用です。

プログラムに **関数定義** を追加しましょう。関数定義も図形の定義と同じく `define` を使いますが、関数名の前に開き括弧を置き、対応する閉じ括弧の前に引数名を書きます。

```
(define (square n)
; A semi-colon starts a line comment.
; The expression below is the function body.
 (filled-rectangle n n))
```

(コメント: セミコロン ; は行コメントの開始です。その下の式が関数本体です。)

定義の構文は、関数呼び出しの構文と対応しています。

```
> (square 10)
[image:pict_7.png]
```

定義を対話エリアで評価できるのと同様に、式を定義エリアに書くこともできます。プログラムを実行すると、定義エリアにある式の結果が対話エリアに表示されます。これ以降、例の定義と式はまとめて書きますので、好きなエリアに置いてください。ただし例は互いに積み重なるので、少なくとも定義は定義エリアに置くのがよいです。

## 5 局所束縛

`define` 形式は、局所的な束縛を作るために、いくつかの場所で使えます。たとえば関数本体の内側でも使えます。

```
(define (four p)
 (define two-p (hc-append p p))
 (vc-append two-p two-p))
```

Racketeer (Racket 使い) は、局所束縛に `let` や `let*` 形式を使うこともあります。 `let` の利点は、任意の式の位置で使えることです。また、識別子ごとに別々の `define` を書く必要がなく、一度に多くの識別子を束縛できます。

```
(define (checker p1 p2)
 (let ([p12 (hc-append p1 p2)]
 [p21 (hc-append p2 p1)])
 (vc-append p12 p21)))
```

`let` 形式は複数の識別子を **同時に** 束縛するので、束縛どうしが互いに参照し合うことはできません。対照的に `let*` 形式は、後の束縛が前の束縛を使うことを許します。

```
(define (checkerboard p)
 (let* ([rp (colorize p "red")]
 [bp (colorize p "black")]
 [c (checker rp bp)]
 [c4 (four c)])
 (four c4)))
```

## 6 関数は値である

`circle` を関数として呼び出す代わりに、式として単に `circle` を評価してみてください。

```
> circle
#<procedure:circle>
```

つまり識別子 `circle` は関数 (別名「手続き」) に束縛されており、ちょうど `c` が円の絵に束縛されているのと同じです。円の絵と違って関数を完全に表示する簡単な方法はないので、DrRacket は単に `#<procedure:circle>` と表示します。

この例は、関数も数値や絵と同じく **値** であることを示します (表示がきれいではないとしても)。関数は値なので、他の関数を引数として受け取る関数を定義できます。

```
(define (series mk)
 (hc-append 4 (mk 5) (mk 10) (mk 20)))
```

関数を引数として受け取る関数を呼ぶとき、引数となる関数が他のどこでも必要ないことがよくあります。そのたびに `define` で関数を書くのは面倒です。名前を考え、定義を置く場所を見つけなければならないからです。代わりに `lambda` を使うと、**無名関数** を作れます。

```
> (series (lambda (size) (checkerboard (square size))))
[image:pict_13.png]
```

`lambda` の直後の括弧付きの名前が関数の引数で、そのあとの式が関数本体です。「function」や「procedure」ではなく「lambda」という言葉を使うのは、Racket の歴史と文化の一部です。

関数用の `define` 形式は、実際には、値として `lambda` を使う単純な `define` の省略形です。たとえば `series` の定義は次のようにも書けます。

```
(define series
 (lambda (mk)
 (hc-append 4 (mk 5) (mk 10) (mk 20))))
```

多くの Racketeer は、lambda に展開するより、define の省略形の関数形式を好みます。

## 7 レキシカルスコープ

Racket は **レキシカルスコープ** の言語です。つまり、識別子が式として使われるとき、その式の **テキスト上の環境** の何かが、識別子の束縛を決めます。この規則は lambda 本体の中の識別子にも、他のどこでも同様に当てはまります。

次の rgb-series 関数では、各 lambda 形式の中の mk の使用は、rgb-series の引数を指します。それがテキスト上スコープにある束縛だからです。

```
(define (rgb-series mk)
 (vc-append
 (series (lambda (sz) (colorize (mk sz) "red"))))
 (series (lambda (sz) (colorize (mk sz) "green"))))
 (series (lambda (sz) (colorize (mk sz) "blue")))))
```

別の例です。rgb-maker は関数を受け取り、元の関数を覚えて使う新しい関数を返します。

```
(define (rgb-maker mk)
 (lambda (sz)
 (vc-append (colorize (mk sz) "red")
 (colorize (mk sz) "green")
 (colorize (mk sz) "blue")))))
```

rgb-maker で関数を合成すると、rgb-series を使う場合と比べて、絵の中のオブジェクトの並び方が違うことに注意してください。

## 8 リスト

Racket は、もともと「LISt Processor」を意味した言語 Lisp から多くのスタイルを受け継いでおり、リストはいまも Racket の重要な一部です。

list 関数は任意個数の引数を取り、与えられた値を含むリストを返します。

```
> (list "red" "green" "blue")
'("red" "green" "blue")
> (list (circle 10) (square 10))
'([image:pict_18.png] [image:pict_19.png])
```

見てのとおり、リストは **単一引用符** のあとに、リスト要素の表示を囲む一対の括弧として表示されます。ここには混乱の余地があります。括弧は (circle 10) のような **式** にも、'("red" "green" "blue") のような **表示結果** にも使われるからです。違いの鍵は引用符で、別のところで議論されています。違いを強調するため、ドキュメントと DrRacket では、結果の括弧は式の括弧と異なり青で表示されます。

リストがあれば、いつか各要素に何かしたくなるでしょう。map 関数はリストと、リストの各要素に適用する関数を取り、その関数の結果を組み合わせた新しいリストを返します。

```
(define (rainbow p)
 (map (lambda (color)
 (colorize p color))
 (list "red" "orange" "yellow" "green" "blue" "purple")))
```

リストと一緒に使える別の関数が apply です。map と同様、関数とリストを取りますが、apply に与える関数は、要素を一つずつではなく、**すべての引数を一度に** 受け取るべきです。apply は、任意個数の引数を取る関数——たとえば vc-append——と特に相性がよいです。

```
> (apply vc-append (rainbow (square 5)))
[image:pict_26.png]
```

(vc-append (rainbow (square 5))) ではうまくいかないことに注意してください。vc-append は引数としてリストを欲しがらず、引数として **絵** を欲しがり、しかも任意個数受け取れるからです。apply は、「多くの引数を欲しがる関数」と「それらの引数のリストという単一の値」のあいだの橋渡しをします。

## 9 モジュール

定義ウィンドウのプログラムが次で始まるので、

```
#lang slideshow
```

定義ウィンドウに書くコードはすべて **モジュール** の内側にあります。さらに、そのモジュールは最初に slideshow が指すモジュールからすべてをインポートします。slideshow は絵を作る関数に加え、list や map のようなよく使う関数もエクスポートします。

追加のライブラリをインポートするには require 形式を使います。たとえばライブラリ pict/flash は filled-flash 関数を提供します。

```
> (filled-flash 40 30)
[image:pict_27.png]
```

モジュールの名前付けと配布にはさまざまな方法があります。

- 一部のモジュールは Racket 配布に同梱されているか、そうでなければ **コレクション** の階層にインストールされます。たとえばモジュール名 pict/flash は、「"pict" コレクション内のファイル "flash.rkt" で実装されたモジュール」を意味します。モジュール名にスラッシュが含まれない場合は、"main.rkt" ファイルを指します。
- 一部のモジュールの集まりは **パッケージ** として配布されます。パッケージは DrRacket の **File** メニューの **Install Package...** からインストールできるほか、コマンドラインツール raco pkg でもインストールできます。たとえば "avl" パッケージを入れると avl モジュールが使えます。パッケージは <https://pkgs.racket-lang.org/> に登録することも、Git リポジトリ・Web サイト・ファイル・ディレクトリから直接入れることもできます。詳細は Package Management in Racket を参照してください。

- 一部のモジュールは、特定のコレクションやパッケージに属さず、他のモジュールに対する **相対位置** に置かれます。たとえば DrRacket で、ここまでの定義をファイル "quick.rkt" に保存し、次の行を追加したとします。

```
(provide rainbow square)
```

すると、同じディレクトリに新しいタブまたはウィンドウを開き、次の新しいプログラム "use.rkt" を書けます。

```
#lang racket
(require "quick.rkt")
(rainbow (square 5))
```

"use.rkt" を実行すると、正方形の虹色リストが出力になります。"use.rkt" は初期インポートとして racket を使っており、それ自体は絵を作る関数を供給しませんが、require と関数呼び出し構文は提供します。

**注:** 定義を保存するには、DrRacket の **Save Definitions** メニュー項目を使います。

Racketeer は通常、新しいプログラムやライブラリを、相対パスとコレクションベースのパスで互いにインポートし合うモジュールとして書きます。こうして開発したプログラムやライブラリが他の人にも有用そうなら、特に実装が Git リポジトリでホストされている場合、パッケージとして登録できます。

## 10 マクロ

試してみる別のライブラリです。

```
> (code (circle 10))
[image:img0.png]
```

結果は円そのものではなく、式として使えば円を生成する **コードの絵** です。言い換えると、code は関数ではなく、絵を作るための **新しい構文形式** です。code 付きの開き括弧のあいだにある部分は式ではなく、code 構文形式によって操作されます。

これは前節で「racket は require と関数呼び出し構文を提供する」と言った意味を説明する助けになります。ライブラリは関数のような **値** のエクスポートに限られず、新しい構文形式も定義できます。この意味で、Racket は厳密には「ひとつの言語」ではなく、言語を拡張したり全く新しい言語を作ったりできるように言語を構造化する **アイデア** に近いものです。

新しい構文形式を導入する一つの方法が、define-syntax と syntax-rules です。

```
(define-syntax pict+code
 (syntax-rules ()
 [(pict+code expr)
 (hc-append 10
 expr
 (code expr))]))
```



この種の定義は **マクロ** です。(pict+code expr) の部分はマクロの使用パターンで、プログラム中のそのパターンの出現は、対応するテンプレート (hc-append 10 expr (code expr)) の出現に置き換えられます。特に (pict+code (circle 10)) は expr が (circle 10) としてパターンに一致するので、(hc-append 10 (circle 10) (code (circle 10))) に置き換えられます。

もちろん、この種の構文拡張は両刃の剣です。新しい言語を発明すると、言いたいことを言いやすくなる一方、他の人が理解しにくくなることもあります。実際、Racket の開発者は Racket コードを含む講演や論文を絶えず行っており、そうした成果物に関わる全員が code を知っている価値があります。

実際、このドキュメントのソースを見てみるとよいでしょう。#lang で始まりますが、それ以外はあまり Racket らしく見えません。それでも、私たちはソースを Racket プログラムとして実行することでこのドキュメントを組み立てています。文書を書くのに十分なほど Racket の構文を拡張するには syntax-rules 以上のものが必要ですが、Racket の構文拡張はかなり遠くまで連れて行ってくれます。

## 11 オブジェクト

オブジェクトシステムは、Racket ユーザーが学んで使う価値のある、洗練された言語拡張の別の例です。lambda があっても、ときにオブジェクトは関数より適しており、オブジェクトはグラフィカルユーザインタフェースで特にうまく働きます。Racket の GUI およびグラフィックスシステムの API は、オブジェクトとクラスの言葉で表現されています。

クラスシステム自体は racket/class ライブラリで実装され、racket/gui/base ライブラリが GUI と描画のクラスを提供します。慣例として、クラス名は % で終わります。

```
(require racket/class
 racket/gui/base)
(define f (new frame% [label "My Art"]
 [width 300]
 [height 300]
 [alignment '(center center)]))
```

new 形式はクラスのインスタンスを作り、label や width のような初期化引数は名前付きで与えます。send 形式はオブジェクトのメソッド（たとえば show）を、メソッド名のあとの引数とともに呼び出します。この場合の引数 #t は真偽値定数の「真」です。

slideshow で生成された絵は、フレーム内のキャンバスのような描画コンテキストへ絵を描くために、グラフィックスツールボックスの描画コマンドを使う関数をカプセル化しています。slideshow の make-pict-drawer 関数は、絵の描画関数を公開します。キャンバスの描画コールバックで make-pict-drawer を使い、キャンバスへ絵を描けます。

```
(define (add-drawing p)
 (let ([drawer (make-pict-drawer p)])
 (new canvas% [parent f]
 [style '(border)]
 [paint-callback (lambda (self dc)
 (drawer dc 0 0))])))
```



[image:img2.png]

各キャンバスはフレームの等しい部分を埋めるように伸びます。フレームがデフォルトで子をそのように管理するからです。

## 12 ここからどこへ行くか

この Racket 入門は、Lisp や Scheme を紹介・区別する伝統的な話題の多くを意図的に避けています。前置の算術記法、シンボル、リストの `quote/quasiquote`、`eval`、第一級継続、そしてすべての構文が実は変装した `lambda` にすぎない、という考え方です。それらはすべて Racket の一部ですが、Racket の日常的なプログラミングの主な材料ではありません。

代わりに、Racket プログラマは通常、関数、レコード、オブジェクト、例外、正規表現、モジュール、スレッドでプログラムします。つまり、Scheme がしばしば語られるような「ミニマリスト」な言語ではなく、Racket は豊富なライブラリとツールを持つ豊かな言語を提供します。

プログラミングが初めての方、あるいは教科書をじっくり進める忍耐がある方には、**How to Design Programs (プログラムの設計方法)** を読むことをおすすめします。すでに読んだ方、あるいはその本がどこへ連れて行くかを見たい方は、*Continue: Web Applications in Racket* を見てください。

経験のあるプログラマが、絵ではなくシステム寄りの視点から Racket の旅を続けるなら、次の目的地は *More: Systems Programming with Racket* です。

代わりに、完全な Racket 言語とツールを深く学び始めるなら、*The Racket Guide* へ進んでください。

## 付録 B: How to Design Programs 言語リファレンス

原本: [extracted/appendix/htdp-langs/original\\_markdown\\_00\\_index.md](https://github.com/htdp-langs/original_markdown_00_index.md)

初学者向けに、説明文を日本語へ翻訳しています。コード・シグネチャ・実行例は原文のまま保持します。

### How to Design Programs 言語

このマニュアルで説明されている言語は、How to Design Programs の本と一緒に使うために DrRacket が提供するものです。

これらの言語のプログラムを DrRacket で実行すると、実行されなかった部分がオレンジと黒で強調表示されます。この色は、まだテストされていない部分についてプログラマにフィードバックするためのものです。この色を見たくなければ、`check-expect` でプログラムをテストしてください。もちろん、色が見えないからといって十分にテスト済みとは限りません。各部分が（少なくとも 1 回）実行された、という意味にすぎません。

通常これらの言語は DrRacket の Choose Language ダイアログで選びますが、DrRacket や他のエディタでは、コードの先頭行に `#lang` 言語指定を書いてもアクセスできます。

- 初級学生言語 `#lang htdp/bsl`
- リスト略記つき初級学生言語 `#lang htdp/bsl+`
- 中級学生言語 `#lang htdp/isl`
- `lambda` つき中級学生言語 `#lang htdp/isl+`
- 上級学生言語 `#lang htdp/asl`

### 目次

- 1.1 あらかじめ定義された変数
- 1.2 テンプレート変数
- 1.3 構文
- 1.4 シグネチャ
  - 1.4.1 シグネチャ形式
  - 1.4.2 構造体シグネチャ
- 1.5 あらかじめ定義された関数
- 1.6 数: 整数・有理数・実数・複素数・正確数・非正確数
- 1.7 真偽値
- 1.8 シンボル
- 1.9 リスト
- 1.10 Posn
- 1.11 文字
- 1.12 文字列
- 1.13 画像
- 1.14 その他
- 1.15 シグネチャ

- 2.1 あらかじめ定義された変数
- 2.2 テンプレート変数
- 2.3 リスト略記つき初級の構文
- 2.4 共通の構文
- 2.5 シグネチャ
- 2.5.1 シグネチャ形式
- 2.5.2 構造体シグネチャ
- 2.6 あらかじめ定義された関数
- 2.7 数: 整数・有理数・実数・複素数・正確数・非正確数
- 2.8 真偽値
- 2.9 シンボル
- 2.10 リスト
- 2.11 Posn
- 2.12 文字
- 2.13 文字列
- 2.14 画像
- 2.15 その他
- 2.16 シグネチャ
- 3.1 あらかじめ定義された変数
- 3.2 テンプレート変数
- 3.3 中級の構文
- 3.4 共通の構文
- 3.5 シグネチャ
- 3.5.1 シグネチャ形式
- 3.5.2 構造体シグネチャ
- 3.6 あらかじめ定義された関数
- 3.7 数: 整数・有理数・実数・複素数・正確数・非正確数
- 3.8 真偽値
- 3.9 シンボル
- 3.10 リスト
- 3.11 Posn
- 3.12 文字
- 3.13 文字列
- 3.14 画像
- 3.15 その他
- 3.16 シグネチャ
- 3.17 数 (緩和条件)
- 3.18 文字列 (緩和条件)
- 3.19 Posn
- 3.20 高階関数

- 4.1 あらかじめ定義された変数
- 4.2 テンプレート変数
- 4.3 lambda つき中級の構文
- 4.4 共通の構文
- 4.5 あらかじめ定義された関数
- 4.6 シグネチャ
- 4.6.1 シグネチャ形式
- 4.6.2 構造体シグネチャ
- 4.7 数: 整数・有理数・実数・複素数・正確数・非正確数
- 4.8 真偽値
- 4.9 シンボル
- 4.10 リスト
- 4.11 Posn
- 4.12 文字
- 4.13 文字列
- 4.14 画像
- 4.15 その他
- 4.16 シグネチャ
- 4.17 数 (緩和条件)
- 4.18 文字列 (緩和条件)
- 4.19 Posn
- 4.20 高階関数
- 4.21 数 (緩和条件・拡張)
- 4.22 高階関数 (lambda つき)
- 5.1 あらかじめ定義された変数
- 5.2 テンプレート変数
- 5.3 上級の構文
- 5.4 共通の構文
- 5.5 あらかじめ定義された関数
- 5.6 シグネチャ
- 5.6.1 シグネチャ形式
- 5.6.2 構造体シグネチャ
- 5.7 数: 整数・有理数・実数・複素数・正確数・非正確数
- 5.8 真偽値
- 5.9 シンボル
- 5.10 リスト
- 5.11 Posn
- 5.12 文字
- 5.13 文字列
- 5.14 画像

- 5.15 その他
- 5.16 シグネチャ
- 5.17 数（緩和条件）
- 5.18 文字列（緩和条件）
- 5.19 Posn
- 5.20 高階関数
- 5.21 数（緩和条件・拡張）
- 5.22 高階関数（lambda つき）
- 5.23 読み取りと表示
- 5.24 ベクタ
- 5.25 ボックス
- 5.26 ハッシュ表

## 付録 B-1: Beginning Student Language (#lang htdp/bsl)

原本: extracted/appendix/htdp-langs/original\_markdown\_01\_beginner.md

初学者向けに、説明文を日本語へ翻訳しています。コード・シグネチャ・実行例は原文のまま保持します。

### 1 Beginning Student (初級学生言語)

文法の記法では、 $X\cdots$  (太字の点) という書き方で、 $X$  が任意の回数 (0 回、1 回、またはそれ以上) 現れてよいことを示します。別に、文法はテンプレートで使う識別子として  $\cdots$  も定義します。

Beginning Student Language の説明は、How to Design Programs/2e の Intermezzo 1 を参照してください。

program	=	def-or-expr...
def-or-expr	=	definition expr test-case library-require
definition	=	(define (name variable variable..... (define name expr) (define name (lambda (variable vari... (define-struct name (name...))
expr	=	(name expr expr...) (cond [expr expr]... [expr expr]) (cond [expr expr]... [else expr]) (if expr expr expr) (and expr expr expr...) (or expr expr expr...) name 'name '() number boolean string character (signature signature-form)
signature-declaration	=	(: name signature-form)
signature-form	=	(enum expr...) (mixed signature-form...) (signature-form... -> signature-fo... (ListOf signature-form) signature-variable expr
signature-variable	=	%name
test-case	=	(check-expect expr expr)

			(check-random expr expr)
			(check-within expr expr expr)
			(check-member-of expr expr...)
			(check-range expr expr expr)
			(check-satisfied expr name)
			(check-error expr expr)
			(check-error expr)
library-require	=		(require string)
			(require (lib string string...))
			(require (planet string package))
package	=		(string string number number)

名前 (name) または変数 (variable) は、空白や次の文字を含まない文字の並びです。

“, ‘ ` () [] {}”;#

数 (number) とは、123、3/2、5.5 のような数です。

真偽値 (boolean) は #true または #false のいずれかです。

#true 定数の別表記は #t、true、#T です。同様に #f、false、#F も #false として認識されます。

シンボル (symbol) は、クォート文字に続く名前です。シンボルは 42、‘()、#false などと同じく値です。

文字列 (string) は、一對の ” で囲まれた文字の並びです。シンボルと違い、文字列は文字に分割したり、さまざまな関数で操作したりできます。たとえば “abcdef”、“This is a string”、および “This is a string with ” inside” はすべて文字列です。

文字 (character) は # で始まり、その文字の名前を持ちます。たとえば #、## は文字です。

関数呼び出しでは、開き括弧の直後に現れる関数は、define や define-struct で定義した関数、またはあらかじめ定義された関数のいずれかです。

## 1.1 あらかじめ定義された変数

[値]	
empty: empty?	

空リスト。

[値]	
true: boolean?	

#true の値。

```
+-----+
| [值] |
| |
| false: boolean? |
+-----+
```

#false の値。

## 1.2 テンプレート変数

```

+-----+
| [構文] |
| |
| .. |
+-----+

```

関数定義がテンプレートであることを示すためのプレースホルダ。

```

+-----+
| [構文] |
| |
| ... |
+-----+

```

関数定義がテンプレートであることを示すためのプレースホルダ。

[構文]	
...	

関数定義がテンプレートであることを示すためのプレースホルダ。

[構文]	
.....	

関数定義がテンプレートであることを示すためのプレースホルダ。

+	-----	+
	[構文]	
	.....	
+	-----	+

関数定義がテンプレートであることを示すためのプレースホルダ。

### 1.3 構文

-----

[構文]



```
|
| (define (name variable variable...) expression) |
+-----+
```

name という名前の関数を定義します。expression は関数本体です。関数が呼ばれると、引数の値が変数の代わりに本体へ挿入されます。関数は、その新しい式の値を返します。

関数名は、他の関数や変数と同じであってはなりません。

```
+-----+
| [構文] |
| |
| (define name expression) |
+-----+
```

expression の値で name という変数を定義します。変数名は他の関数や変数と同じであってはならず、name 自身が expression に現れてはなりません。

```
+-----+
| [構文] |
| |
| (define name (lambda (variable variable...) expression)) |
+-----+
```

An alternate way へ defining 関数 s. name は name の 関数, であり できない be same と して という の another 関数 または 変数.

A lambda できない be used outside の この alternate syntax.

```
+-----+
| [構文] |
| |
| 'name |
+-----+
```

クォートされた名前はシンボルです。シンボルは 0 や '() と同じく値です。

```
+-----+
| [構文] |
| |
| (define-struct structure-name (field-name...)) |
+-----+
```

structure-name という新しい構造体を定義します。構造体のフィールドは field-names によって名付けられます。define-struct のあと、次の新しい関数が使えます。

- make-structure-name: 構造体のフィールド数と同じ個数の引数を取り、その構造体の新しいインスタンスを作ります。
- structure-name-field-name: 構造体のインスタンスを取り、field-name という名前のフィールドの値を返します。

- `structure-name?`: 任意の値を取り、その値が構造体のインスタンスなら `#true` を返します。

`define-struct` が導入する新しい関数の名前は、他の関数や変数と同じであってはいけません。同じだと `define-struct` はエラーを報告します。

```
+-----+
| [構文] |
| |
| (name expression expression...) |
+-----+
```

`Calls` 関数 `named name`. 値の `call` は値の `name's body` のとき every one の関数's 変数 `s` は `replaced` によって 値 `s` の `corresponding` 式 `s`.

関数 `named name` must defined before it できる be called. 数の引数 式 `s` でなければならない same として 数の引数 `s expected` によって 関数.

```
+-----+
| [構文] |
| |
| (cond [question-expression answer-expression]...) |
| (cond [question-expression answer-expression]... [else answer-expression]) |
| (cond [question-expression answer-expression] |
| ... |
| [else answer-expression]) |
| |
| ``racket |
| (cond [question-expression answer-expression] |
| ... |
| [else answer-expression]) |
+-----+
```

Chooses 節 based 上の ある 条件. `cond` finds first question-式 という 評価結果は `#true`, then 評価する corresponding answer-式.

どの `question-expression` も `#true` に評価されない場合、`cond` の値は `else` 節の `answer-expression` です。 `else` がなければ、`cond` はエラーを報告します。 `question-expression` の結果が `#true` でも `#false` でもない場合も、`cond` はエラーを報告します。

`else` は `cond` の外では使えません。

```
+-----+
| [構文] |
| |
| (if question-expression |
| then-answer-expression |
| else-answer-expression) |
+-----+
```

```
| ``racket
| (if question-expression
| then-answer-expression
| else-answer-expression)
| ...
+-----+
```

とき: 値の question-式は #true, もし評価する then-answer-式. とき: test は #false, もし評価する else-answer-式.

もし question-式は neither #true nor #false, もし reports エラー.

```
+-----+
| [構文]
|
| (and expression expression expression...)
+-----+
```

すべての expression が #true なら #true に評価されます。いずれかの expression が #false なら、and 式は #false に評価されます（その式より右の式は評価されません）。

もし任意の の式 s evaluate へ 値 other than #true または #false, と reports エラー.

```
+-----+
| [構文]
|
| (or expression expression expression...)
+-----+
```

いずれかの expression が #true になった時点で #true に評価されます（その式より右の式は評価されません）。すべての expression が #false なら、or 式は #false に評価されます。

もし任意の の式 s evaluate へ 値 other than #true または #false, または reports エラー.

```
+-----+
| [構文]
|
| (check-expect expression expected-expression)
+-----+
```

最初の expression が expected-expression と同じ値に評価されることを検査します。

```
(check-expect (fahrenheit->celsius 212) 100)
(check-expect (fahrenheit->celsius -40) -40)
```

```
(define (fahrenheit->celsius f)
 (* 5/9 (- f 32)))
```

check-expect 式は、学生プログラムのトップレベルに置く必要があります。また、検査対象の関数定義より前を含め、プログラムのどこにでも置けます。そこに check-expect を置くことで、プログラマは動く例とともにプログラムの意図を将来の読者に伝えられ、関数

本体そのものを読む必要がしばしばなくなります。check-expect (およびすべての check 形式) の構文エラーは意図的に実行時まで遅延されるので、学生は完全な関数ヘッダを必ずしも書かなくてもテストを書けます。

expr または expected-expr が不正確数や関数値を生成するのはエラーです。不正確数については、単純な等価で比べるのは原則として誤りです。代わりに両者が小さな区間内にあるかをチェックします。check-within を参照してください。関数については (Intermediate 以上を参照)、関数どうしを比較するのは証明上不可能です。

```
+-----+
| [構文] |
| |
| (check-random expression expected-expression) |
+-----+
```

最初の expression が expected-expression と同じ値に評価されることをチェックします。

form supplies same 乱数-数 generator へ both parts. もし both parts request 乱数 数 s から same interval の中の same order, they receive same 乱数 数 s.

ここでは simple example の where check-乱数 は useful:

```
(define WIDTH 100)
(define HEIGHT (* 2 WIDTH))

(define-struct player (name x y))
; A Player is (make-player String Nat Nat)

; String -> Player

(check-random (create-randomly-placed-player "David Van Horn")
 (make-player "David Van Horn" (random WIDTH) (random HEIGHT)))

(define (create-randomly-placed-player name)
 (make-player name (random WIDTH) (random HEIGHT)))
```

Note how 乱数 は called 上の same 数 s の中の same order の中の both parts の check-乱数. もし two parts call 乱数 のための different intervals, they は likely へ fail:

```
; String -> Player

(check-random (create-randomly-placed-player "David Van Horn")
 (make-player "David Van Horn" (random WIDTH) (random HEIGHT)))

(define (create-randomly-placed-player name)
 (a-helper-function name (random HEIGHT)))

; String Number -> Player
(define (a-helper-function name height)
 (make-player name (random WIDTH) height))
```

Because 引数へ a-helper-関数 は evaluated first, 乱数 は first called のための interval [0,HEIGHT) と then のための [0,WIDTH), という is, の中の different order than の中の preceding check-乱数.

それはエラーのための expr または expected-expr へ produce 関数値 または in 正確数; see note 上の check-expect のための details.

```
+-----+
| [構文] |
| |
| (check-satisfied expression predicate) |
+-----+
```

最初の expression が、指定された述語（1 引数の関数）を満たすことを検査します。「満たす」とは「その関数が与えられた値に対して #true を返す」という意味です。

ここでは simple examples のための check-satisfied:

```
> (check-satisfied 1 odd?)
The test passed!

> (check-satisfied 1 even?)
Ran 1 test. 0 tests passed.
Check failures:
Actual value | 1 | does not satisfy even?.
```

In general check-satisfied empowers プログラム designers へ use defined 関数 s へ formulate test suites:

```
; [cons Number [List-of Number]] -> Boolean
; a function for testing htdp-sort

(check-expect (sorted? (list 1 2 3)) #true)
(check-expect (sorted? (list 2 1 3)) #false)

(define (sorted? l)
 (cond
 [(empty? (rest l)) #true]
 [else (and (<= (first l) (second l)) (sorted? (rest l)))]))

; [List-of Number] -> [List-of Number]
; create a sorted version of the given list of numbers

(check-satisfied (htdp-sort (list 1 2 0 3)) sorted?)

(define (htdp-sort l)
 (cond
 [(empty? l) l]
 [else (insert (first l) (htdp-sort (rest l)))]))
```

```

; Number [List-of Number] -> [List-of Number]
; insert x into l at proper place
; assume l is arranged in ascending order
; the result is sorted in the same way
(define (insert x l)
 (cond
 [(empty? l) (list x)]
 [else (if (<= x (first l)) (cons x l) (cons (first l) (insert x (rest l))))]))

```

And yes, results of htdp-sort satisfy sorted? 述語:

```

> (check-satisfied (htdp-sort (list 1 2 0 3)) sorted?)
The test passed!

```

```

+-----+
| [構文] |
| |
| (check-within expression expected-expression delta) |
+-----+

```

expression という式の値が、expected-expression という式が生成する値と構造的に等しいかを検査します。最初の式の中の各数は、2 番目の式の対応する数から delta 以内でなければなりません。

```

(define-struct roots (x sqrt))
; RT is [List-of (make-roots Number Number)]

(define (root-of a)
 (make-roots a (sqrt a)))

(define (roots-table xs)
 (cond
 [(empty? xs) '()]
 [else (cons (root-of (first xs)) (roots-table (rest xs)))]))

```

Due へ presence の in 正確数 s の中の nested data, check-within は correct choice のための testing, と test succeeds もし delta は reasonably large:

例:

```

> (check-within (roots-table (list 1.0 2.0 3.0))
 roots 1.0 1.0) (make-roots 2 1.414) (list (make-roots 3 1.713))
0.1)
The test passed!

```

対照的に、のとき delta は small, test fails:

例:

```

> (check-within (roots-table (list 2.0))
 1.414) (list (make-roots 2
#i1e-5)
Ran 1 test.

```

0 tests passed.

Check failures:

```
Actual value | '((make-roots 2.0
1.4142135623730951)) | is not within 1e-5 of expected value | '((make-roots 2 1.414)) | .
at line 5, column 0
```

それはエラーのための式sまたはexpected-式へproduce関数値; see note 上の check-expect のための details.

もし delta は ない 数, check-within reports エラー.

```
+-----+
| [構文] |
| (check-error expression expected-error-message) |
| (check-error expression) |
+-----+
```

expression がエラーを報告することを検査します。expected-error-message がある場合は、エラーメッセージがその値と一致することも検査します。

ここでは typical beginner example という calls のための use の check-エラー:

```
(define sample-table
 '(("matthias" 10)
 ("matthew" 20)
 ("robby" -1)
 ("shriram" 18)))

; [List-of [list String Number]] String -> Number
; determine the number associated with s in table

(define (lookup table s)
 (cond
 [(empty? table) (error (string-append s " not found"))]
 [else (if (string=? (first (first table)) s)
 (second (first table))
 (lookup (rest table))))])
```

Consider following two examples の中の この context:

例:

```
> (check-expect (lookup sample-table "matthew") 20)
The test passed!
```

例:

```
> (check-error (lookup sample-table "kathi") "kathi not found")
The test passed!
```

```
+-----+
| [構文] |
| |
| (check-member-of expression expression expression...) |
+-----+
```

that the value of the first expression is that of one of the following expressions を検査します。

```
; [List-of X] -> X
; pick a random element from the given list l
(define (pick-one l)
 (list-ref l (random (length l))))
```

例:

```
> (check-member-of (pick-one '("a" "b" "c")) "a" "b" "c")
The test passed!
```

それはエラーのための任意のの式 s へ produce 関数 値; see note 上の check-expect のための details.

```
+-----+
| [構文] |
| |
| (check-range expression low-expression high-expression) |
+-----+
```

that the value of the first expression is a number in between the value of the low-expression and the high-expression, inclusive を検査します。

A check-range form は best used へ delimit possible results の 関数 s という compute in 正確 数 s:

```
(define EPSILON 0.001)

; [Real -> Real] Real -> Real
; what is the slope of f at x?
(define (differentiate f x)
 (slope f (- x EPSILON) (+ x EPSILON)))

; [Real -> Real] Real Real -> Real
(define (slope f left right)
 (/ (- (f right) (f left))
 2 EPSILON))
```

```
(check-range (differentiate sin 0) 0.99 1.0)
```

それはエラーのための式, low-式, または high-式 へ produce 関数 値; see note 上の check-expect のための details.



```
+-----+
| [構文] |
| |
| (require string) |
+-----+
```

Makes 定義 *s* の モジュール specified によって 文字列 available の中の current モジュール (i.e., current file), where 文字列 refers へ file relative へ current file.

The string is constrained in several ways to avoid problems with different path conventions on different platforms: a / is a directory separator, . always means the current directory, .. always means the parent directory, path elements can use only a through z (uppercase or lowercase), 0 through 9, -, \_, and ., and the string cannot be empty or contain a leading or trailing /.

```
+-----+
| [構文] |
| |
| (require module-name) |
+-----+
```

Accesses file の中の installed ライブラリ. ライブラリ name は 識別子 とともに same constraints として のための relative-path 文字列 (though without quotes), とともに additional constraint という it ～してはならない contain a..

```
+-----+
| [構文] |
| |
| (require (lib string string...)) |
+-----+
```

Accesses a file in an installed library, making its definitions available in the current module (i.e., the current file). The first string names the library file, and the remaining strings name the collection (and sub-collection, and so on) where the file is installed. Each string is constrained in the same way as for the (requirestring) form.

```
+-----+
| [構文] |
| |
| (require (planet string (string string number number))) |
+-----+
```

Accesses ライブラリ という は distributed 上の internet via PPlaneT server, making it 定義 *s* available の中の current モジュール (i.e., current file).

full grammar のための planet requires は given の中の Importing と Exporting: require と provide, but best place へ find examples の syntax は 上の the PPlaneT server, 中の description の specific package.

## 1.4 シグネチャ

Signatures do not have to be comment: They can also be part of the > code. When a signature is attached to a function, DrRacket will check > that program uses the function in accordance with the signature and > display signature violations along with the test results. A signature is a regular value, and is specified as a > signature form, a > special syntax that only works with: signature declarations > and inside signature expressions.

```
+-----+
| [構文] |
| |
| (: name signature-form) |
+-----+
```

これ attaches シグネチャ specified によって シグネチャ-form へ 定義 の name. There でなければならぬ 定義 の name somewhere の中の プログラム.

```
(: age Integer)
(define age 42)

(: area-of-square (Number -> Number))
(define (area-of-square len)
 (sqr len))
```

On running プログラム, Racket checks whether シグネチャ s attached with: actually match 値 の 変数. もし they don't, Racket reports シグネチャ violation along とともに test failures.

たとえば、この piece の code:

```
(: age Integer)
(define age "fortytwo")
```

Yields この output:

```
+-----+
| `1 signature violation.` |
+-----+
| `Signature violations:` |
| `got "fortytwo" at line 2, column 12, signature... |
+-----+
```

Note という シグネチャ violation does ない stop running プログラム.

```
+-----+
| [構文] |
| |
| |
+-----+
```

```
| (signature signature-form) |
+-----+
```

これ返すシグネチャ described によってシグネチャ-form として 値.

### 1.4.1 シグネチャ形式

Any expression can be a signature form, in which case the signature is > the value returned by that expression. There are a few special > signature forms, however: In a signature form, any name that starts with a % is a > signature variable that stands for any signature depending on how > the signature is used. Example:

```
(: same (%a -> %a))
```

```
(define (same x) x)
```

```
+-----+
| [構文] |
| |
| (input-signature-form... -> output-signature-form) |
+-----+
```

これシグネチャ form describes 関数 とともに inputs described によって input-シグネチャ-forms と output described によって output-シグネチャ-form.

```
+-----+
| [構文] |
| |
| (enum expr...) |
+-----+
```

これシグネチャ describes enumeration の 値 s returned によって exprs.

例:

```
(: cute? ((enum "cat" "snake") -> Boolean))
```

```
(define (cute? pet)
 (cond
 [(string=? pet "cat") #t]
 [(string=? pet "snake") #f]))
```

```
+-----+
| [構文] |
| |
| (mixed signature-form...) |
+-----+
```

これシグネチャ describes mixed data, i.e. itemization where 各 の cases has シグネチャ described によってシグネチャ-form.

例:

```
(define SIGS (signature (mixed Aim Fired)))
```

```
+-----+
| [構文] |
| |
| (ListOf signature-form) |
+-----+
```

これ シグネチャ describes リスト where elements は described によって シグネチャ-form.

```
+-----+
| [構文] |
| |
| (predicate expression) |
+-----+
```

これ シグネチャ describes 値 s through 述語: 式 must evaluate へ 関数 の one 引数 という 返す 真偽値. シグネチャ matches すべての 値 s のための であり 述語 返す #true.

### 1.4.2 構造体シグネチャ

A define-struct form defines two additional names that can be > used in signatures. For a struct called struct, these > are Struct and StructOf. Note that > these names are capitalized. In particular, a struct called > Struct, will also define Struct and > StructOf. Moreover, when forming the additional > names, hyphens are removed, and each letter following a hyphen is > capitalized - so a struct called foo-bar will define > FooBar and FooBarOf. Struct is a signature that describes struct values > from this structure type. StructOf is a function > that takes as input a signature for each field. It returns a > signature describing values of this structure type, additionally > describing the values of the fields of the value.

```
(define-struct pair [fst snd])
```

```
(: add-pair ((PairOf Number Number) -> Number))
```

```
(define (add-pair p)
 (+ (pair-fst p) (pair-snd p)))
```

### 1.5 あらかじめ定義された関数

remaining subsections リスト それらの 関数 s という は built へ プログラム ming language. All other 関数 s は imported から teachpack または でなければならない defined の中の プログラム.

## 1.6 数: 整数・有理数・実数・複素数・正確数・非正確数

```
+-----+
| [手続き] |
| |
| (* x y z...) → number |
| x: number |
| y: number |
| z: number |
+-----+
```

すべての数を掛け合わせます。

```
> (* 5 3)
15
> (* 5 3 2)
30
```

```
+-----+
| [手続き] |
| |
| (+ x y z...) → number |
| x: number |
| y: number |
| z: number |
+-----+
```

すべての数を足し合わせます。

```
> (+ 2/3 1/16)
35/48
> (+ 3 2 5 8)
18
```

```
+-----+
| [手続き] |
| |
| (- x y...) → number |
| x: number |
| y: number |
+-----+
```

Subtracts second (and following) 数(s) から first; negates 数 もし there は only one 引数.

```
> (- 5)
-5
> (- 5 3)
2
> (- 5 3 1)
1
```

```
+-----+
| [手続き] |
| |
+-----+
```

```
| (/ x y z...) → number |
| x: number |
| y: number |
| z: number |
+-----+
```

第 1 引数を第 2（およびそれ以降）の数で割ります。

```
> (/ 12 2)
6
> (/ 12 2 3)
2
```

```
+-----+
| [手続き] |
| |
| (< x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

two or more (real) numbers for less-than を比較します。

```
> (< 42 2/5)
#false
```

```
+-----+
| [手続き] |
| |
| (<= x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

two or more (real) numbers for less-than or equality を比較します。

```
> (<= 42 2/5)
#false
```

```
+-----+
| [手続き] |
| |
| (= x y z...) → boolean? |
| x: number |
| y: number |
| z: number |
+-----+
```

two or more numbers for equality を比較します。

```
> (= 42 2/5)
#false
```

```

+-----+
| [手続き] |
| |
| (> x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+

```

two or more (real) numbers for greater-than を比較します。

```

> (> 42 2/5)
#true

```

```

+-----+
| [手続き] |
| |
| (>= x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+

```

two or more (real) numbers for greater-than or equality を比較します。

```

> (>= 42 42)
#true

```

```

+-----+
| [手続き] |
| |
| (abs x) → real |
| x: real |
+-----+

```

判定する: absolute 値 の 実数 数.

```

> (abs -12)
12

```

```

+-----+
| [手続き] |
| |
| (acos x) → number |
| x: number |
+-----+

```

the arccosine (inverse of cos) of a number を計算します。

```

> (acos 0)
#i1.5707963267948966

```

```

+-----+
| [手続き] |
+-----+

```

```
|
| (add1 x) → number |
| x: number |
+-----+
```

Increments given 数.

```
> (add1 2)
3
```

```
+-----+
| [手続き] |
| |
| (angle x) → real |
| x: number |
+-----+
```

the angle from a complex number を取り出します。

```
> (angle (make-polar 3 4))
#i-2.2831853071795867
```

```
+-----+
| [手続き] |
| |
| (asin x) → number |
| x: number |
+-----+
```

the arcsine (inverse of sin) of a number を計算します。

```
> (asin 0)
0
```

```
+-----+
| [手続き] |
| |
| (atan x) → number |
| x: number |
+-----+
```

計算する: arctangent の given 数:

```
> (atan 0)
0
> (atan 0.5)
#i0.4636476090008061
```

Also comes in a two-argument version where (atany x) computes (atan(/yx)) but the signs of y and x determine the quadrant of the result and the result tends to be more accurate than that of the 1-argument version in borderline cases:

```
> (atan 3 4)
#i0.6435011087932844
```



```
> (atan -2 -1)
#i-2.0344439357957027
```

```
+-----+
| [手続き] |
| |
| (ceiling x) → integer |
| x: real |
+-----+
```

判定する: closest 整数 (正確 または in 正確) above 実数 数. 参照: round.

```
> (ceiling 12.3)
#i13.0
```

```
+-----+
| [手続き] |
| |
| (complex? x) → boolean? |
| x: any/c |
+-----+
```

some value is complex かどうかを判定します。

```
> (complex? 1-2i)
#true
```

```
+-----+
| [手続き] |
| |
| (conjugate x) → number |
| x: number |
+-----+
```

Flips sign の imaginary part の 複素数 数.

```
> (conjugate 3+4i)
3-4i
> (conjugate -2-5i)
-2+5i
> (conjugate (make-polar 3 4))
#i-1.960930862590836+2.2704074859237844i
```

```
+-----+
| [手続き] |
| |
| (cos x) → number |
| x: number |
+-----+
```

the cosine of a number (radians) を計算します。

```
> (cos pi)
#i-1.0
```

```

+-----+
| [手続き] |
| |
| (cosh x) → number |
| x: number |
+-----+

```

the hyperbolic cosine of a number を計算します。

```

> (cosh 10)
#i11013.232920103324

```

```

+-----+
| [手続き] |
| |
| (current-seconds) → integer |
+-----+

```

判定する: current time の中の seconds elapsed (since platform-specific starting date).

```

> (current-seconds)
1779843321

```

```

+-----+
| [手続き] |
| |
| (denominator x) → integer |
| x: rational? |
+-----+

```

the denominator of a rational を計算します。

```

> (denominator 2/3)
3

```

```

+-----+
| [値] |
| |
| e: real |
+-----+

```

Euler's 数.

```

> e
#i2.718281828459045

```

```

+-----+
| [手続き] |
| |
| (even? x) → boolean? |
| x: integer |
+-----+

```

some integer (exact or inexact) is even or not かどうかを判定します。

```
> (even? 2)
```

```
#true
```

```
+-----+
| [手続き] |
| |
| (exact->inexact x) → number |
| x: number |
+-----+
```

an exact number to an inexact one を変換します。

```
> (exact->inexact 12)
```

```
#i12.0
```

```
+-----+
| [手続き] |
| |
| (exact? x) → boolean? |
| x: number |
+-----+
```

some number is exactかどうかを判定します。

```
> (exact? (sqrt 2))
```

```
#false
```

```
+-----+
| [手続き] |
| |
| (exp x) → number |
| x: number |
+-----+
```

判定する: e raised へ 数.

```
> (exp -2)
```

```
#i0.1353352832366127
```

```
+-----+
| [手続き] |
| |
| (expt x y) → number |
| x: number |
| y: number |
+-----+
```

the power of the first to the second number, which is to say, exponentiation を計算します。

```
> (expt 16 1/2)
```

```
4
```

```
> (expt 3 -4)
```

```
1/81
```

```

+-----+
| [手続き] |
| |
| (floor x) → integer |
| x: real |
+-----+

```

判定する: closest 整数 (正確 または in 正確) below 実数 数. 参照: round.

```

> (floor 12.3)
#i12.0

```

```

+-----+
| [手続き] |
| |
| (gcd x y...) → integer |
| x: integer |
| y: integer |
+-----+

```

判定する: greatest common divisor の two 整数 s (正確 または in 正確).

```

> (gcd 6 12 8)
2

```

```

+-----+
| [手続き] |
| |
| (imag-part x) → real |
| x: number |
+-----+

```

the imaginary part from a complex number を取り出します。

```

> (imag-part 3+4i)
4

```

```

+-----+
| [手続き] |
| |
| (inexact->exact x) → number |
| x: number |
+-----+

```

Approximates in 正確 数 によって 正確 one.

```

> (inexact->exact 12.0)
12

```

```

+-----+
| [手続き] |
| |
| (inexact? x) → boolean? |
+-----+

```

```
| x: number |
+-----+
```

some number is inexactかどうかを判定します。

```
> (inexact? 1-2i)
#false
```

```
+-----+
| [手続き] |
| |
| (integer->char x) → char |
| x: exact-integer? |
+-----+
```

Looks up 文字 という corresponds へ given 正確 整数 の中の ASCII table (if any).

```
> (integer->char 42)
#*
```

```
+-----+
| [手続き] |
| |
| (integer-sqrt x) → complex |
| x: integer |
+-----+
```

the integer or imaginary-integer square root of an integer を計算します。

```
> (integer-sqrt 11)
3
> (integer-sqrt -11)
0+3i
```

```
+-----+
| [手続き] |
| |
| (integer? x) → boolean? |
| x: any/c |
+-----+
```

some value is an integer (exact or inexact)かどうかを判定します。

```
> (integer? (sqrt 2))
#false
```

```
+-----+
| [手続き] |
| |
| (lcm x y...) → integer |
| x: integer |
| y: integer |
+-----+
```

判定する: least common multiple の two 整数 s (正確 または in 正確).

```
> (lcm 6 12 8)
```

```
24
```

```
+-----+
| [手続き] |
| |
| (log x) → number |
| x: number |
+-----+
```

判定する: base-e logarithm の数.

```
> (log 12)
```

```
#i2.4849066497880004
```

```
+-----+
| [手続き] |
| |
| (magnitude x) → real |
| x: number |
+-----+
```

判定する: magnitude の複素数 数.

```
> (magnitude (make-polar 3 4))
```

```
#i2.9999999999999996
```

```
+-----+
| [手続き] |
| |
| (make-polar x y) → number |
| x: real |
| y: real |
+-----+
```

a complex from a magnitude and angle を作成します。

```
> (make-polar 3 4)
```

```
#i-1.960930862590836-2.2704074859237844i
```

```
+-----+
| [手続き] |
| |
| (make-rectangular x y) → number |
| x: real |
| y: real |
+-----+
```

a complex from a real and an imaginary part を作成します。

```
> (make-rectangular 3 4)
```

```
3+4i
```

```
+-----+
| [手続き] |
| |
```

```
|
| (max x y...) → real |
| x: real |
| y: real |
+-----+
```

判定する: largest 数—aka, maximum.

```
> (max 3 2 8 7 2 9 0)
9
```

```
+-----+
| [手続き] |
| (min x y...) → real |
| x: real |
| y: real |
+-----+
```

判定する: smallest 数—aka, minimum.

```
> (min 3 2 8 7 2 9 0)
0
```

```
+-----+
| [手続き] |
| (modulo x y) → integer |
| x: integer |
| y: integer |
+-----+
```

Finds remainder の division の first 数 によって second:

```
> (modulo 9 2)
1
> (modulo 3 -4)
-1
```

```
+-----+
| [手続き] |
| (negative? x) → boolean? |
| x: real |
+-----+
```

some real number is strictly smaller than zero かどうかを判定します。

```
> (negative? -2)
#true
```

```
+-----+
| [手続き] |
| |
+-----+
```

```
| (number->string x) → string |
| x: number |
+-----+
```

a number to a string を変換します。

```
> (number->string 42)
"42"
```

```
+-----+
| [手続き] |
| |
| (number->string-digits x p) → string |
| x: number |
| p: posint |
+-----+
```

数 x を、指定した桁数の文字列に変換します。

```
> (number->string-digits 0.9 2)
"0.9"
> (number->string-digits pi 4)
"3.1416"
```

```
+-----+
| [手続き] |
| |
| (number? n) → boolean? |
| n: any/c |
+-----+
```

～かどうかを判定する: ある 値 は 数:

```
> (number? "hello world")
#false
> (number? 42)
#true
```

```
+-----+
| [手続き] |
| |
| (numerator x) → integer |
| x: rational? |
+-----+
```

有理数の分子を計算します。

```
> (numerator 2/3)
2
```

```
+-----+
| [手続き] |
| |
| (odd? x) → boolean? |
+-----+
```



```
| x: integer |
+-----+
```

some integer (exact or inexact) is odd or not かどうかを判定します。

```
> (odd? 2)
#false
```

```
+-----+
| [値] |
| pi: real |
+-----+
```

ratio の circle's circumference へ その diameter.

```
> pi
#i3.141592653589793
```

```
+-----+
| [手続き] |
| (positive? x) → boolean? |
| x: real |
+-----+
```

some real number is strictly larger than zero かどうかを判定します。

```
> (positive? -2)
#false
```

```
+-----+
| [手続き] |
| (quotient x y) → integer |
| x: integer |
| y: integer |
+-----+
```

最初の整数（被除数）を 2 番目の整数（除数）で割り、商を求めます。

```
> (quotient 9 2)
4
> (quotient 3 4)
0
```

```
+-----+
| [手続き] |
| (random x) → natural? |
| x: (and/c natural? positive?) |
+-----+
```

与えられた正確な自然数より小さい、乱数の自然数を生成します。

```
> (random 42)
```

```
26
```

```
+-----+
| [手続き] |
| |
| (rational? x) → boolean? |
| x: any/c |
+-----+
```

ある値が有理数かどうかを判定します。

```
> (rational? 1)
```

```
#true
```

```
> (rational? -2.349)
```

```
#true
```

```
> (rational? #i1.23456789)
```

```
#true
```

```
> (rational? (sqrt -1))
```

```
#false
```

```
> (rational? pi)
```

```
#true
```

```
> (rational? e)
```

```
#true
```

```
> (rational? 1-2i)
```

```
#false
```

対話が示すように、教育用言語は予想より多くの数を有理数とみなします。特に pi は、数学的な  $\pi$  の有限近似にすぎないため有理数です。rational? は、これらの数を分数として考えてよいという示唆だと捉えてください。

```
+-----+
| [手続き] |
| |
| (real-part x) → real |
| x: number |
+-----+
```

複素数から実部を取り出します。

```
> (real-part 3+4i)
```

```
3
```

```
+-----+
| [手続き] |
| |
| (real? x) → boolean? |
| x: any/c |
+-----+
```

ある値が実数かどうかを判定します。

```
> (real? 1-2i)
#false
```

```
+-----+
| [手続き] |
| |
| (remainder x y) → integer |
| x: integer |
| y: integer |
+-----+
```

判定する: remainder の dividing first によって second 整数 (正確 または in 正確).

```
> (remainder 9 2)
1
> (remainder 3 4)
3
```

```
+-----+
| [手続き] |
| |
| (round x) → integer |
| x: real |
+-----+
```

実数を整数に丸めます (同点のときは偶数へ丸めます)。floor と ceiling も参照してください。

```
> (round 12.3)
#i12.0
```

```
+-----+
| [手続き] |
| |
| (sgn x) → (union 1 #i1.0 0 #i0.0 -1 #i-1.0) |
| x: real |
+-----+
```

判定する: sign の 実数 数.

```
> (sgn -12)
-1
```

```
+-----+
| [手続き] |
| |
| (sin x) → number |
| x: number |
+-----+
```

数 (ラジアン) の正弦を計算します。

```
> (sin pi)
#i1.2246467991473532e-16
```

```

+-----+
| [手続き] |
| |
| (sinh x) → number |
| x: number |
+-----+

```

数の双曲線正弦を計算します。

```

> (sinh 10)
#i11013.232874703393

```

```

+-----+
| [手続き] |
| |
| (sqr x) → number |
| x: number |
+-----+

```

数の 2 乗を計算します。

```

> (sqr 8)
64

```

```

+-----+
| [手続き] |
| |
| (sqrt x) → number |
| x: number |
+-----+

```

数の平方根を計算します。

```

> (sqrt 9)
3
> (sqrt 2)
#i1.4142135623730951

```

```

+-----+
| [手続き] |
| |
| (sub1 x) → number |
| x: number |
+-----+

```

与えられた数を 1 減らします。

```

> (sub1 2)
1

```

```

+-----+
| [手続き] |
| |
| (tan x) → number |
+-----+

```

```
| x: number |
+-----+
```

数（ラジアン）の正接を計算します。

```
> (tan pi)
#i-1.2246467991473532e-16
```

```
+-----+
| [手続き] |
| |
| (zero? x) → boolean? |
| x: number |
+-----+
```

some number is zero or not かどうかを判定します。

```
> (zero? 2)
#false
```

## 1.7 真偽値

```
+-----+
| [手続き] |
| |
| (boolean->string x) → string |
| x: boolean? |
+-----+
```

与えられた真偽値に対する文字列を生成します。

```
> (boolean->string #false)
"#false"
> (boolean->string #true)
"#true"
```

```
+-----+
| [手続き] |
| |
| (boolean=? x y) → boolean? |
| x: boolean? |
| y: boolean? |
+-----+
```

2つの真偽値が等しいかを判定します。

```
> (boolean=? #true #false)
#false
```

```
+-----+
| [手続き] |
| |
| (boolean? x) → boolean? |
| x: any/c |
+-----+
```

値が真偽値かどうかを判定します。

```
> (boolean? 42)
#false
> (boolean? #false)
#true
```

```
+-----+
| [手続き] |
| |
| (false? x) → boolean? |
| x: any/c |
+-----+
```

a value is false かどうかを判定します。

```
> (false? #false)
#true
```

```
+-----+
| [手続き] |
| |
| (not x) → boolean? |
| x: boolean? |
+-----+
```

真偽値を否定します。

```
> (not #false)
#true
```

## 1.8 シンボル

```
+-----+
| [手続き] |
| |
| (symbol->string x) → string |
| x: symbol |
+-----+
```

a symbol to a string を変換します。

```
> (symbol->string 'c)
"c"
```

```
+-----+
| [手続き] |
| |
| (symbol=? x y) → boolean? |
| x: symbol |
| y: symbol |
+-----+
```

2つのシンボルが等しいかを判定します。

```
> (symbol=? 'a 'b)
#false
```

```
+-----+
| [手続き] |
| |
| (symbol? x) → boolean? |
| x: any/c |
+-----+
```

値がシンボルかどうかを判定します。

```
> (symbol? 'a)
#true
```

## 1.9 リスト

```
+-----+
| [手続き] |
| |
| (append x y z...) → list? |
| x: list? |
| y: list? |
| z: list? |
+-----+
```

複数のリストの項目を連結して、1つのリストを作ります。

```
> (append (cons 1 (cons 2 '())) (cons "a" (cons "b" empty)))
(list 1 2 "a" "b")
```

```
+-----+
| [手続き] |
| |
| (assoc x l) → (union (listof any) #false) |
| x: any/c |
| l: (listof any) |
+-----+
```

l 上で first が x と equal? である最初の対を返します。なければ #false を返します。

```
> (assoc "hello" '(("world" 2) ("hello" 3) ("good" 0)))
(list "hello" 3)
```

```
+-----+
| [手続き] |
| |
| (assq x l) → (union #false cons?) |
| x: any/c |
| l: list? |
+-----+
```

～かどうかを判定する: ある item は first item の pair の中の リスト の pairs. (それ compares items とともに eq?.)

```
> a
(list (list 'a 22) (list 'b 8) (list 'c 70))
> (assq 'b a)
(list 'b 8)
```

```
+-----+
| [手続き] |
| |
| (caaar x) → any/c |
| x: list? |
+-----+
```

LISP 風セレクタ: (car (car (car x)))。

```
> w
(list (list (list (list "bye") 3) #true) 42)
> (caaar w)
(list "bye")
```

```
+-----+
| [手続き] |
| |
| (caadr x) → any/c |
| x: list? |
+-----+
```

LISP 風セレクタ: (car (car (cdr x)))。

```
> (caadr (cons 1 (cons (cons 'a '()) (cons (cons 'd '()) '()))))
'a
```

```
+-----+
| [手続き] |
| |
| (caar x) → any/c |
| x: list? |
+-----+
```

LISP 風セレクタ: (car (car x))。

```
> y
(list (list (list 1 2 3) #false "world"))
> (caar y)
(list 1 2 3)
```

```
+-----+
| [手続き] |
| |
| (cadar x) → any/c |
| x: list? |
+-----+
```

LISP 風セレクタ: (car (cdr (car x)))。



```
> w
(list (list (list (list "bye") 3) #true) 42)
> (cadar w)
#true
```

```
+-----+
| [手続き] |
| |
| (caddr x) → any/c |
| x: list? |
+-----+
```

LISP 風セレクタ: (car (cdr (cdr (cdr x)))).

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (caddr v)
4
```

```
+-----+
| [手続き] |
| |
| (caddr x) → any/c |
| x: list? |
+-----+
```

LISP 風セレクタ: (car (cdr (cdr x))).

```
> x
(list 2 "hello" #true)
> (caddr x)
#true
```

```
+-----+
| [手続き] |
| |
| (cadr x) → any/c |
| x: list? |
+-----+
```

LISP 風セレクタ: (car (cdr x)).

```
> x
(list 2 "hello" #true)
> (cadr x)
"hello"
```

```
+-----+
| [手続き] |
| |
| (car x) → any/c |
| x: cons? |
+-----+
```

空でないリストの先頭要素を取り出します。

```
> x
(list 2 "hello" #true)
> (car x)
2
```

```
+-----+
| [手続き] |
| |
| (cdaar x) → any/c |
| x: list? |
+-----+
```

LISP 風セクタ: (cdr (car (car x)))。

```
> w
(list (list (list (list "bye") 3) #true) 42)
> (cdaar w)
(list 3)
```

```
+-----+
| [手続き] |
| |
| (cdadr x) → any/c |
| x: list? |
+-----+
```

LISP 風セクタ: (cdr (car (cdr x)))。

```
> (cdadr (list 1 (list 2 "a") 3))
(list "a")
```

```
+-----+
| [手続き] |
| |
| (cdar x) → list? |
| x: list? |
+-----+
```

LISP 風セクタ: (cdr (car x))。

```
> y
(list (list (list 1 2 3) #false "world"))
> (cdar y)
(list #false "world")
```

```
+-----+
| [手続き] |
| |
| (cddar x) → any/c |
| x: list? |
+-----+
```

LISP 風セクタ: (cdr (cdr (car x)))

```
> w
(list (list (list (list "bye") 3) #true) 42)
> (cddar w)
'()
```

```
+-----+
| [手続き] |
| |
| (cdddr x) → any/c |
| x: list? |
+-----+
```

LISP 風セクタ: (cdr (cdr (cdr x)))。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (cdddr v)
(list 4 5 6 7 8 9 'A)
```

```
+-----+
| [手続き] |
| |
| (cddr x) → list? |
| x: list? |
+-----+
```

LISP 風セクタ: (cdr (cdr x))。

```
> x
(list 2 "hello" #true)
> (cddr x)
(list #true)
```

```
+-----+
| [手続き] |
| |
| (cdr x) → any/c |
| x: cons? |
+-----+
```

空でないリストの残り (rest) を取り出します。

```
> x
(list 2 "hello" #true)
> (cdr x)
(list "hello" #true)
```

```
+-----+
| [手続き] |
| |
| (cons x y) → list? |
| x: any/c |
+-----+
```

```
| y: list? |
+-----+
```

a list を構築します。

```
> (cons 1 '())
(cons 1 '())
```

```
+-----+
| [手続き] |
| |
| (cons? x) → boolean? |
| x: any/c |
+-----+
```

ある値が構築されたリストかどうかを判定します。

```
> (cons? (cons 1 '()))
#true
> (cons? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (eighth x) → any/c |
| x: list? |
+-----+
```

空でないリストの 8 番目の項目を選びます。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (eighth v)
8
```

```
+-----+
| [手続き] |
| |
| (empty? x) → boolean? |
| x: any/c |
+-----+
```

値が空リストかどうかを判定します。

```
> (empty? '())
#true
> (empty? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (fifth x) → any/c |
+-----+
```

```
| x: list? |
+-----+
```

空でないリストの 5 番目の項目を選びます。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (fifth v)
5
```

```
+-----+
| [手続き] |
| |
| (first x) → any/c |
| x: cons? |
+-----+
```

空でないリストの先頭要素を取り出します。

```
> x
(list 2 "hello" #true)
> (first x)
2
```

```
+-----+
| [手続き] |
| |
| (fourth x) → any/c |
| x: list? |
+-----+
```

空でないリストの 4 番目の項目を選びます。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (fourth v)
4
```

```
+-----+
| [手続き] |
| |
| (length l) → natural? |
| l: list? |
+-----+
```

リスト上の項目数を評価します。

```
> x
(list 2 "hello" #true)
> (length x)
3
```

```
+-----+
| [手続き] |
+-----+
```

```

| (list x...) → list? |
| x: any/c |
+-----+

```

a list of its arguments を構築します。

```

> (list 1 2 3 4 5 6 7 8 9 0)
(cons 1 (cons 2 (cons 3 (cons 4 (cons 5 (cons 6 (cons 7 (cons 8 (cons 9 (cons 0 '()))))))))))

```

```

+-----+
| [手続き] |
| (list* x... l) → list? |
| x: any/c |
| l: list? |
+-----+

```

a list by adding multiple items to a list を構築します。

```

> x
(list 2 "hello" #true)
> (list* 4 3 x)
(list 4 3 2 "hello" #true)

```

```

+-----+
| [手続き] |
| (list-ref x i) → any/c |
| x: list? |
| i: natural? |
+-----+

```

リストからインデックスで指定した項目を取り出します。

```

> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (list-ref v 9)
'A

```

```

+-----+
| [手続き] |
| (list? x) → boolean? |
| x: any/c |
+-----+

```

与えられた値がリストかどうかを調べます。

```

> (list? 42)
#false
> (list? '())
#true

```

```
> (list? (cons 1 (cons 2 '())))
#true
```

```
+-----+
| [手続き] |
| |
| (make-list i x) → list? |
| i: natural? |
| x: any/c |
+-----+
```

a list of i copies of x を構築します。

```
> (make-list 3 "hello")
(cons "hello" (cons "hello" (cons "hello" '())))
```

```
+-----+
| [手続き] |
| |
| (member x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

値がリスト上にあるかどうかを判定します (equal? で比較)。

```
> x
(list 2 "hello" #true)
> (member "hello" x)
#true
```

```
+-----+
| [手続き] |
| |
| (member? x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

値がリスト上にあるかどうかを判定します (equal? で比較)。

```
> x
(list 2 "hello" #true)
> (member? "hello" x)
#true
```

```
+-----+
| [手続き] |
| |
| (memq x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

値  $x$  がリスト  $l$  上にあるかを判定します。  $x$  と  $l$  上の項目の比較には `eq?` を使います。

```
> x
(list 2 "hello" #true)
> (memq (list (list 1 2 3)) x)
#false
```

```
+-----+
| [手続き] |
| |
| (memq? x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

値  $x$  がリスト  $l$  上にあるかを判定します。  $x$  と  $l$  上の項目の比較には `eq?` を使います。

```
> x
(list 2 "hello" #true)
> (memq? (list (list 1 2 3)) x)
#false
```

```
+-----+
| [手続き] |
| |
| (memv x l) → (or/c #false list) |
| x: any/c |
| l: list? |
+-----+
```

～かどうかを判定する: ある 値 は 上の リスト もし so, it produces suffix の リスト という starts とともに  $x$  もし not, it produces false. (それ compares 値  $s$  とともに `eqv?` 述語.)

```
> x
(list 2 "hello" #true)
> (memv (list (list 1 2 3)) x)
#false
```

```
+-----+
| [値] |
| |
| null: list |
+-----+
```

空リストの別名

```
> null
'()
```

```
+-----+
| [手続き] |
| |
| (null? x) → boolean? |
+-----+
```



```
| x: any/c |
+-----+
```

値が空リストかどうかを判定します。

```
> (null? '())
#true
> (null? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (range start end step) → list? |
| start: number |
| end: number |
| step: number |
+-----+
```

start から end までステップで進み、数のリストを構築します。

```
> (range 0 10 2)
(cons 0 (cons 2 (cons 4 (cons 6 (cons 8 '())))))
```

```
+-----+
| [手続き] |
| |
| (remove x l) → list? |
| x: any/c |
| l: list? |
+-----+
```

与えられたリストと同様のリストを、与えられた項目の最初の出現を除いて構築します (値は equal? で比較します)。

```
> x
(list 2 "hello" #true)
> (remove "hello" x)
(list 2 #true)
> hello-2
(list 2 "hello" #true "hello")
> (remove "hello" hello-2)
(list 2 #true "hello")
```

```
+-----+
| [手続き] |
| |
| (remove-all x l) → list? |
| x: any/c |
| l: list? |
+-----+
```

与えられたリストと同様のリストを、与えられた項目のすべての出現を除いて構築します（値は `equal?` で比較します）。

```
> x
(list 2 "hello" #true)
> (remove-all "hello" x)
(list 2 #true)
> hello-2
(list 2 "hello" #true "hello")
> (remove-all "hello" hello-2)
(list 2 #true)
```

```
+-----+
| [手続き] |
| |
| (rest x) → any/c |
| x: cons? |
+-----+
```

空でないリストの残り（`rest`）を取り出します。

```
> x
(list 2 "hello" #true)
> (rest x)
(list "hello" #true)
```

```
+-----+
| [手続き] |
| |
| (reverse l) → list |
| l: list? |
+-----+
```

a reversed version of a list を作成します。

```
> x
(list 2 "hello" #true)
> (reverse x)
(list #true "hello" 2)
```

```
+-----+
| [手続き] |
| |
| (second x) → any/c |
| x: list? |
+-----+
```

空でないリストの 2 番目の項目を選びます。

```
> x
(list 2 "hello" #true)
> (second x)
"hello"
```

```
+-----+
| [手続き] |
| |
| (seventh x) → any/c |
| x: list? |
+-----+
```

空でないリストの 7 番目の項目を選びます。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (seventh v)
7
```

```
+-----+
| [手続き] |
| |
| (sixth x) → any/c |
| x: list? |
+-----+
```

空でないリストの 6 番目の項目を選びます。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (sixth v)
6
```

```
+-----+
| [手続き] |
| |
| (third x) → any/c |
| x: list? |
+-----+
```

空でないリストの 3 番目の項目を選びます。

```
> x
(list 2 "hello" #true)
> (third x)
#true
```

## 1.10 Posn

```
+-----+
| [手続き] |
| |
| (make-posn x y) → posn |
| x: any/c |
| y: any/c |
+-----+
```

2 つの任意の値から posn を構築します。

```
> (make-posn 3 3)
(make-posn 3 3)
> (make-posn "hello" #true)
(make-posn "hello" #true)
```

```
+-----+
| [手続き] |
| |
| (posn-x p) → any/c |
| p: posn |
+-----+
```

posn の x 成分を取り出します。

```
> p
(make-posn 2 -3)
> (posn-x p)
2
```

```
+-----+
| [手続き] |
| |
| (posn-y p) → any/c |
| p: posn |
+-----+
```

posn の y 成分を取り出します。

```
> p
(make-posn 2 -3)
> (posn-y p)
-3
```

```
+-----+
| [手続き] |
| |
| (posn? x) → boolean? |
| x: any/c |
+-----+
```

its input is a posn かどうかを判定します。

```
> q
(make-posn "bye" 2)
> (posn? q)
#true
> (posn? 42)
#false
```

## 1.11 文字

```
+-----+
| [手続き] |
| |
| |
+-----+
```

```
| (char->integer c) → integer |
| c: char |
+-----+
```

与えられた文字に対応する数を ASCII 表（あれば）から調べます。

```
> (char->integer #\a)
97
> (char->integer #\z)
122
```

```
+-----+
| [手続き] |
| |
| (char-alphabetic? c) → boolean? |
| c: char |
+-----+
```

a character represents an alphabetic characterかどうかを判定します。

```
> (char-alphabetic? #\Q)
#true
```

```
+-----+
| [手続き] |
| |
| (char-ci<=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

文字が、大文字小文字を区別せず昇順に並んでいるかを判定します。

```
> (char-ci<=? #\b #\B)
#true
> (char<=? #\b #\B)
#false
```

```
+-----+
| [手続き] |
| |
| (char-ci<? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

文字が、大文字小文字を区別せず厳密な昇順に並んでいるかを判定します。

```
> (char-ci<? #\B #\C)
#true
> (char<? #\b #\B)
#false
```

```
+-----+
| [手続き] |
| (char-ci=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

two characters are equal in a case-insensitive mannerかどうかを判定します。

```
> (char-ci=? #\b #\B)
#true
```

```
+-----+
| [手続き] |
| (char-ci>=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

文字が、大文字小文字を区別せず降順に並んでいるかを判定します。

```
> (char-ci>=? #\b #\C)
#false
> (char>=? #\b #\C)
#true
```

```
+-----+
| [手続き] |
| (char-ci>? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

文字が、大文字小文字を区別せず厳密な降順に並んでいるかを判定します。

```
> (char-ci>? #\b #\B)
#false
> (char>? #\b #\B)
#true
```

```
+-----+
| [手続き] |
| (char-downcase c) → char |
| c: char |
+-----+
```

対応する小文字を生成します。

```
> (char-downcase #\T)
#\t
```

```
+-----+
| [手続き] |
| |
| (char-lower-case? c) → boolean? |
| c: char |
+-----+
```

a character is a lower-case characterかどうかを判定します。

```
> (char-lower-case? #\T)
#false
```

```
+-----+
| [手続き] |
| |
| (char-numeric? c) → boolean? |
| c: char |
+-----+
```

a character represents a digitかどうかを判定します。

```
> (char-numeric? #\9)
#true
```

```
+-----+
| [手続き] |
| |
| (char-upcase c) → char |
| c: char |
+-----+
```

対応する大文字を生成します。

```
> (char-upcase #\t)
#\T
```

```
+-----+
| [手続き] |
| |
| (char-upper-case? c) → boolean? |
| c: char |
+-----+
```

a character is an upper-case characterかどうかを判定します。

```
> (char-upper-case? #\T)
#true
```

```
+-----+
| [手続き] |
| |
```

```

|
| (char-whitespace? c) → boolean? |
| c: char |
+-----+

```

a character represents spaceかどうかを判定します。

```

> (char-whitespace? #\tab)
#true

```

```

+-----+
| [手続き] |
|
| (char<=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+

```

文字が昇順に並んでいるかを判定します。

```

> (char<=? #\a #\a #\b)
#true

```

```

+-----+
| [手続き] |
|
| (char<? x d e...) → boolean? |
| x: char |
| d: char |
| e: char |
+-----+

```

文字が厳密な昇順に並んでいるかを判定します。

```

> (char<? #\a #\b #\c)
#true

```

```

+-----+
| [手続き] |
|
| (char=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+

```

文字が等しいかを判定します。

```

> (char=? #\b #\a)
#false

```

```

+-----+
| [手続き] |
|

```



```
|
| (char>=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

文字が降順に並んでいるかを判定します。

```
> (char>=? #\b #\b #\a)
#true
```

```
+-----+
| [手続き] |
|
| (char>? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

文字が厳密な降順に並んでいるかを判定します。

```
> (char>? #\A #\z #\a)
#false
```

```
+-----+
| [手続き] |
|
| (char? x) → boolean? |
| x: any/c |
+-----+
```

a value is a characterかどうかを判定します。

```
> (char? "a")
#false
> (char? #\a)
#true
```

## 1.12 文字列

```
+-----+
| [手続き] |
|
| (explode s) → (listof string) |
| s: string |
+-----+
```

文字列を 1 文字の文字列のリストに変換します。

```
> (explode "cat")
(list "c" "a" "t")
```

```
+-----+
| [手続き] |
| |
| (format f x...) → string |
| f: string |
| x: any/c |
+-----+
```

文字列を整形し、値を埋め込むこともできます。

```
> (format "Dear Dr. ~a:" "Flatt")
"Dear Dr. Flatt:"
> (format "Dear Dr. ~s:" "Flatt")
"Dear Dr. \"Flatt\":"
```

```
+-----+
| [手続き] |
| |
| (implode l) → string |
| l: list? |
+-----+
```

1 文字の文字列のリストを 1 つの文字列に連結します。

```
> (implode (cons "c" (cons "a" (cons "t" '()))))
"cat"
```

```
+-----+
| [手続き] |
| |
| (int->string i) → string |
| i: integer |
+-----+
```

an integer in [0,55295] or [57344 1114111] to a 1-letter string を変換します。

```
> (int->string 65)
"A"
```

```
+-----+
| [手続き] |
| |
| (list->string l) → string |
| l: list? |
+-----+
```

a s list of characters into a string を変換します。

```
> (list->string (cons #\c (cons #\a (cons #\t '()))))
"cat"
```

```
+-----+
| [手続き] |
| |
+-----+
```

```
| (make-string i c) → string |
| i: natural? |
| c: char |
+-----+
```

c から長さ i の文字列を生成します。

```
> (make-string 3 #\d)
"ddd"
```

```
+-----+
| [手続き] |
| |
| (replicate i s) → string |
| i: natural? |
| s: string |
+-----+
```

s を i 回複製します。

```
> (replicate 3 "h")
"hhh"
```

```
+-----+
| [手続き] |
| |
| (string c...) → string? |
| c: char |
+-----+
```

与えられた文字から文字列を作ります。

```
> (string #\d #\o #\g)
"dog"
```

```
+-----+
| [手続き] |
| |
| (string->int s) → integer |
| s: string |
+-----+
```

a 1-letter string to an integer in [0,55295] or [57344, 1114111] を変換します。

```
> (string->int "a")
97
```

```
+-----+
| [手続き] |
| |
| (string->list s) → (listof char) |
| s: string |
+-----+
```

a string into a list of characters を変換します。

```
> (string->list "hello")
(list #\h #\e #\l #\l #\o)
```

```
+-----+
| [手続き] |
| |
| (string->number s) → (union number #false) |
| s: string |
+-----+
```

a string into a number, produce false if impossible を変換します。

```
> (string->number "-2.03")
-2.03
> (string->number "1-2i")
1-2i
```

```
+-----+
| [手続き] |
| |
| (string->symbol s) → symbol |
| s: string |
+-----+
```

a string into a symbol を変換します。

```
> (string->symbol "hello")
'hello
```

```
+-----+
| [手続き] |
| |
| (string-alphabetic? s) → boolean? |
| s: string |
+-----+
```

文字列中のすべての「文字」がアルファベットかどうかを判定します。

```
> (string-alphabetic? "123")
#false
> (string-alphabetic? "cat")
#true
```

```
+-----+
| [手続き] |
| |
| (string-append s t z...) → string |
| s: string |
| t: string |
| z: string |
+-----+
```

複数の文字列の文字を連結します。

```
> (string-append "hello" " " "world" " " "good bye")
"hello world good bye"
```

```
+-----+
| [手続き] |
| |
| (string-ci<=? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

文字列が、辞書順で昇順かつ大文字小文字を区別しない順序かを判定します。

```
> (string-ci<=? "hello" "WORLD")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci<? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

文字列が、辞書順で厳密な昇順かつ大文字小文字を区別しない順序かを判定します。

```
> (string-ci<? "hello" "WORLD")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci=? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

すべての文字列が、大文字小文字を無視して文字ごとに等しいかを判定します。

```
> (string-ci=? "hello" "Hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci>=? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

文字列が、辞書順で降順かつ大文字小文字を区別しない順序かを判定します。

```
> (string-ci>? "WORLD" "hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci>? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

文字列が、辞書順で厳密な降順かつ大文字小文字を区別しない順序かを判定します。

```
> (string-ci>? "WORLD" "hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string-contains-ci? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

最初の文字列が 2 番目の文字列に、大文字小文字を無視して現れるかを判定します。

```
> (string-contains-ci? "At" "caT")
#true
```

```
+-----+
| [手続き] |
| |
| (string-contains? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

最初の文字列が 2 番目の文字列に文字どおり現れるかを判定します。

```
> (string-contains? "at" "cat")
#true
```

```
+-----+
| [手続き] |
| |
| (string-copy s) → string |
| s: string |
+-----+
```

文字列をコピーします。

```
> (string-copy "hello")
"hello"
```

```
+-----+
| [手続き] |
| |
+-----+
```

```
| (string-downcase s) → string |
| s: string |
+-----+
```

a string like the given one with all ‘letters’ as lower case を生成します。

```
> (string-downcase "CAT")
"cat"
> (string-downcase "cAt")
"cat"
```

```
+-----+
| [手続き] |
| |
| (string-ith s i) → 1string? |
| s: string |
| i: natural? |
+-----+
```

the ith 1-letter substring from s を取り出します。

```
> (string-ith "hello world" 1)
"e"
```

```
+-----+
| [手続き] |
| |
| (string-length s) → nat |
| s: string |
+-----+
```

判定する: length の 文字列.

```
> (string-length "hello world")
11
```

```
+-----+
| [手続き] |
| |
| (string-lower-case? s) → boolean? |
| s: string |
+-----+
```

all ‘letters’ in the string are lower caseかどうかを判定します。

```
> (string-lower-case? "CAT")
#false
```

```
+-----+
| [手続き] |
| |
| (string-numeric? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are numericかどうかを判定します。

```
> (string-numeric? "123")
#true
> (string-numeric? "1-2i")
#false
```

```
+-----+
| [手続き] |
| |
| (string-ref s i) → char |
| s: string |
| i: natural? |
+-----+
```

the ith character from s を取り出します。

```
> (string-ref "cat" 2)
#\t
```

```
+-----+
| [手続き] |
| |
| (string-upcase s) → string |
| s: string |
+-----+
```

a string like the given one with all 'letters' as upper case を生成します。

```
> (string-upcase "cat")
"CAT"
> (string-upcase "cAt")
"CAT"
```

```
+-----+
| [手続き] |
| |
| (string-upper-case? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are upper caseかどうかを判定します。

```
> (string-upper-case? "CAT")
#true
```

```
+-----+
| [手続き] |
| |
| (string-whitespace? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are white spaceかどうかを判定します。



```
> (string-whitespace? (string-append " " (string #\tab #\newline #\return)))
#true
```

```
+-----+
| [手続き] |
| |
| (string<=? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

the strings are ordered in a lexicographically increasing mannerかどうかを判定します。

```
> (string<=? "hello" "hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string<? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

the strings are ordered in a lexicographically strictly increasing mannerかどうかを判定します。

```
> (string<? "hello" "world")
#true
```

```
+-----+
| [手続き] |
| |
| (string=? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

all strings are equal, character for characterかどうかを判定します。

```
> (string=? "hello" "world")
```

```
#false
```

```
> (string=? "bye" "bye")
```

```
#true
```

```
+-----+
| [手続き] |
| |
| (string>=? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

the strings are ordered in a lexicographically decreasing mannerかどうかを判定します。

```
> (string>=? "world" "hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string>? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

the strings are ordered in a lexicographically strictly decreasing mannerかどうかを判定します。

```
> (string>? "world" "hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string? x) → boolean? |
| x: any/c |
+-----+
```

a value is a stringかどうかを判定します。

```
> (string? "hello world")
#true
> (string? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (substring s i j) → string |
| s: string |
| i: natural? |
| j: natural? |
+-----+
```

the substring starting at i up to j (or the end if j is not provided) を取り出します。

```
> (substring "hello world" 1 5)
"ello"
> (substring "hello world" 1 8)
"ello wo"
> (substring "hello world" 4)
"o world"
```

### 1.13 画像

```
+-----+
| [手続き] |
| |
| (image=? i j) → boolean? |
| i: image |
| j: image |
+-----+
```

two images are equalかどうかを判定します。

```
> c1
[image:pict.png]
> (image=? (circle 5 "solid" "green") c1)
#false
> (image=? (circle 10 "solid" "green") c1)
#true
```

```
+-----+
| [手続き] |
| |
| (image? x) → boolean? |
| x: any/c |
+-----+
```

a value is an imageかどうかを判定します。

```
> c1
[image:pict_2.png]
> (image? c1)
#true
```

### 1.14 その他

```
+-----+
| [手続き] |
| |
| (≈ x y eps) → boolean? |
| x: number |
| y: number |
| eps: non-negative-real |
+-----+
```

whether x and y are within eps of either other を検査します。

```
> (≈ 1.01 1.0 0.1)
#true
> (≈ 1.01 1.5 0.1)
#false
```

```
+-----+
| [値] |
| |
+-----+
```

```
| eof: eof-object? |
+-----+
```

A 値 という represents end の file:

```
> eof
#<eof>
```

```
+-----+
| [手続き] |
| |
| (eof-object? x) → boolean? |
| x: any/c |
+-----+
```

some value is the end-of-file value かどうかを判定します。

```
> (eof-object? eof)
#true
> (eof-object? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (eq? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+
```

two values are equivalent from the computer's perspective (intensional) かどうかを判定します。

```
> (eq? (cons 1 '()) (cons 1 '()))
#false
> one
(list 1)
> (eq? one one)
#true
```

```
+-----+
| [手続き] |
| |
| (equal? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+
```

two values are structurally equal where basic values are compared with the eqv? predicate かどうかを判定します。

```
> (equal? (make-posn 1 2) (make-posn (- 2 1) (+ 1 1)))
#true
```

```
+-----+
| [手続き] |
| |
| (equal~? x y z) → boolean? |
| x: any/c |
| y: any/c |
| z: non-negative-real |
+-----+
```

x and y like equal? but uses =~ in the case of numbers を比較します。

```
> (equal~? (make-posn 1.01 1.0) (make-posn 1.01 0.99) 0.2)
#true
```

```
+-----+
| [手続き] |
| |
| (eqv? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+
```

two values are equivalent from the perspective of all functions that can be applied to it (extensional)かどうかを判定します。

```
> (eqv? (cons 1 '()) (cons 1 '()))
#false
> one
(list 1)
> (eqv? one one)
#true
```

```
+-----+
| [手続き] |
| |
| (error x...) → void? |
| x: any/c |
+-----+
```

Signals an error, combining the given values into an error message. If any of the values' printed representations is too long, it is truncated and “...” is put into the string. If the first value is a symbol, it is suffixed with a colon and the result pre-pended on to the error message.

```
> zero
0
> (if (= zero 0) (error "can't divide by 0") (/ 1 zero))
can't divide by 0
```

```
+-----+
| [手続き] |
| |
+-----+
```

```
| (exit) → void |
+-----+
```

Evaluating (exit) terminates running プログラム.

```
+-----+
| [手続き] |
| |
| (identity x) → any/c |
| x: any/c |
+-----+
```

x を返します。

```
> (identity 42)
42
> (identity c1)
[image:pict_3.png]
> (identity "hello")
"hello"
```

```
+-----+
| [手続き] |
| |
| (struct? x) → boolean? |
| x: any/c |
+-----+
```

some value is a structure かどうかを判定します。

```
> (struct? (make-posn 1 2))
#true
> (struct? 43)
#false
```

### 1.15 シグネチャ

```
+-----+
| [値] |
| |
| Any: signature? |
+-----+
```

any value のシグネチャ。

```
+-----+
| [値] |
| |
| Boolean: signature? |
+-----+
```

booleans のシグネチャ。

```
+-----+
| [値] |
| Char: signature? |
+-----+
```

characters のシグネチャ。

```
+-----+
| [手続き] |
| (ConsOf first-sig rest-sig) → signature? |
| first-sig: signature? |
| rest-sig: signature? |
+-----+
```

a cons pair のシグネチャ。

```
+-----+
| [値] |
| EmptyList: signature? |
+-----+
```

the empty list のシグネチャ。

```
+-----+
| [値] |
| False: signature? |
+-----+
```

just false のシグネチャ。

```
+-----+
| [値] |
| Integer: signature? |
+-----+
```

integers のシグネチャ。

```
+-----+
| [値] |
| Natural: signature? |
+-----+
```

natural numbers のシグネチャ。

```
+-----+
| [値] |
| |
+-----+
```

```
| Number: signature? |
+-----+
```

arbitrary numbers のシグネチャ。

```
+-----+
| [値] |
| |
| Rational: signature? |
+-----+
```

rational numbers のシグネチャ。

```
+-----+
| [値] |
| |
| Real: signature? |
+-----+
```

real numbers のシグネチャ。

```
+-----+
| [値] |
| |
| String: signature? |
+-----+
```

strings のシグネチャ。

```
+-----+
| [値] |
| |
| Symbol: signature? |
+-----+
```

symbols のシグネチャ。

```
+-----+
| [値] |
| |
| True: signature? |
+-----+
```

just true のシグネチャ。



## 付録 B-2: Beginning Student with List Abbreviations (#lang htdp/bsl+)

原本: `extracted/appendix/htdp-langs/original_markdown_02_beginner-abbr.md`

文法の記法では、太字のドット **X**… を用いて、**X** が任意の回数 (0 回、1 回、またはそれ以上) 現れうることを示します。また別途、文法はテンプレートで使う識別子として … も定義しています。

Beginning Student Language の説明は、『How to Design Programs/2e』の Intermezzo 1 を参照してください。

クオートしたリストの説明は、『How to Design Programs/2e』の Intermezzo 2 を参照してください。

program	=	def-or-expr...
def-or-expr	=	definition expr test-case library-require
definition	=	(define (name variable variable..... (define name expr) (define name (lambda (variable vari... (define-struct name (name...))
expr	=	(name expr expr...) (prim-op expr...) (cond [expr expr]... [expr expr]) (cond [expr expr]... [else expr]) (if expr expr expr) (and expr expr expr...) (or expr expr expr...) name 'quoted 'quasiquoted '() number boolean string character
signature-declaration	=	(: name signature-form)
signature-form	=	(enum expr...) (mixed signature-form...) (signature-form... -> signature-fo... (ListOf signature-form) signature-variable expr
signature-variable	=	%name

quoted	=	name number string character (quoted...) 'quoted 'quoted ,`quoted ,@`quoted
quasiquoted	=	name number string character (quasiquoted...) 'quasiquoted 'quasiquoted ,`expr ,@`expr
test-case	=	(check-expect expr expr) (check-random expr expr) (check-within expr expr expr) (check-member-of expr expr...) (check-range expr expr expr) (check-satisfied expr name) (check-error expr expr) (check-error expr)
library-require	=	(require string) (require (lib string string...)) (require (planet string package))
package	=	(string string number number)

名前 (name) または変数 (variable) は、空白や次の文字を含まない文字の並びです。

“ , ` ( ) [ ] { } ; #

数 (number) とは、123、3/2、5.5 のような数です。

真偽値 (boolean) は #true または #false のいずれかです。

#true 定数の別表記は #t、true、#T です。同様に #f、false、#F も #false として認識されます。

シンボル (symbol) は、クオート文字に続く名前です。シンボルは 42、`()、#false などと同じく値です。

文字列 (string) は、一対の ” で囲まれた文字の並びです。シンボルと違い、文字列は文字に分割したり、さまざまな関数で操作したりできます。たとえば “abcdef”、“This is a string”、および “This is a string with " inside” はすべて文字列です。

文字 (character) は # で始まり、その文字の名前を持ちます。たとえば #、## は文字です。

関数呼び出しでは、開き括弧の直後に現れる関数は、define や define-struct で定義した関数、またはあらかじめ定義された関数のいずれかです。

## 2.1 あらかじめ定義された変数

```
+-----+
| [値] |
| |
| empty: empty? |
+-----+
```

空リスト。

```
+-----+
| [値] |
| |
| true: boolean? |
+-----+
```

#true の値。

```
+-----+
| [値] |
| |
| false: boolean? |
+-----+
```

#false の値。

## 2.2 テンプレート変数

```
+-----+
| [構文] |
| |
| .. |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
| ... |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
| |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

## 2.3 リスト略記つき初級の構文

```
+-----+
| [構文] |
| 'name |
+-----+
```

クォートされた name はシンボルです。クォートされた部分は、入れ子のリストの略記です。

通常このクォートは ' で書き、'(applebanana) のようにしますが、quote を使って (quote(applebanana)) と書くこともできます。

```
+-----+
| [構文] |
| 'name |
+-----+
```

quote と似ていますが、式への「アンクォート」による脱出も許します。

通常、準クォートはバッククォートで書き、(apple,(+12)) のようにしますが、quasiquote を使って (quasiquote(apple,(+12))) と書くこともできます。

```
+-----+
| [構文] |
| ,expression |
+-----+
```

準クォートが1つのとき、,expression はクォートから脱出し、評価した式の結果を略記リストに挿入します。

準クォートが複数あるとき、,expression は文字どおりの ,expression であり、expression に対する準クォートの段数を1つ減らします。

通常、アンクォートは, で書きますが、unquote と書くこともできます。

```
+-----+
| [構文] |
| |
| ,@expression |
+-----+
```

準クォートが1つのとき、,@expression はクォートから脱出し、評価結果であるリストを略記リストに継ぎ足し（スプライス）します。

準クォートが複数あるとき、スプライシング・アンクォートはアンクォートと同様で、準クォートの段数を1つ減らします。

通常、スプライシング・アンクォートは,@ で書きますが、unquote-splicing と書くこともできます。

## 2.4 共通の構文

次の構文は、Beginner with List Abbreviations レベルでも Beginning Student レベルと同じ振る舞いをします。

```
+-----+
| [構文] |
| |
| (define (name variable variable...) expression) |
+-----+
```

name という名前の関数を定義します。expression は関数本体です。関数が呼ばれると、引数の値が変数の代わりに本体へ挿入されます。関数は、その新しい式の値を返します。

関数名は、他の関数や変数と同じであってはなりません。

```
+-----+
| [構文] |
| |
| (define name expression) |
+-----+
```

expression の値で name という変数を定義します。変数名は他の関数や変数と同じであってはならず、name 自身が expression に現れてはなりません。

```
+-----+
| [構文] |
| |
| (define name (lambda (variable variable...) expression)) |
+-----+
```

関数を定義する別の書き方です。name は関数の名前で、他の関数や変数と同じであってはいけません。

lambda は、この別構文の外では使えません。

```
+-----+
| [構文] |
| |
| (define-struct structure-name (field-name...)) |
+-----+
```

structure-name という新しい構造体を定義します。構造体のフィールドは field-names で名付けられます。define-struct のあと、次の新しい関数が使えます：

- make-structure-name: 構造体のフィールド数と同じ個数の引数を取り、その構造体の新しいインスタンスを作ります。
- structure-name-field-name: 構造体のインスタンスを取り、field-name で名付けられたフィールドの値を返します。
- structure-name?: 任意の値を取り、その値が構造体のインスタンスなら #true を返します。

define-struct が導入する新しい関数の名前は、他の関数や変数と同じであってはいけません。同じだと define-struct はエラーを報告します。

```
+-----+
| [構文] |
| |
| (name expression expression...) |
+-----+
```

Calls 関数 named name. 値の call は 値の name's body のとき every one の関数's 変数 s は replaced によって 値 s の corresponding 式 s.

関数 named name must defined before it できる be called. 数の引数 式 s でなければならない same として 数の引数 s expected によって 関数.

```
+-----+
| [構文] |
| |
| (cond [question-expression answer-expression]...) |
| (cond [question-expression answer-expression]... [else answer-expression]) |
| (cond [question-expression answer-expression] |
| ... |
| [else answer-expression]) |
| |
| ``racket |
| (cond [question-expression answer-expression] |
| ... |
| [else answer-expression]) |
+-----+
```

Chooses 節 based 上の ある 条件. cond finds first question-式 という 評価結果は #true, then 評価する corresponding answer-式.

If none of the question-expressions evaluates to #true, cond's value is the answer-expression of the else clause. If there is no else, cond reports an error. If the result of a question-expression is neither #true nor #false, cond also reports an error.

else は cond の外では使えません。

```
+-----+
| [構文] |
| |
| (if question-expression
| then-answer-expression
| else-answer-expression)
| |
| ``racket
| (if question-expression
| then-answer-expression
| else-answer-expression)
| ...
+-----+
```

とき: 値の question-式は #true, もし評価する then-answer-式. とき: test は #false, もし評価する else-answer-式.

もし question-式は neither #true nor #false, もし reports エラー.

```
+-----+
| [構文] |
| |
| (and expression expression expression...) |
+-----+
```

Evaluates へ #true もしすべての式 s は #true. もし任意の式は #false, と式評価結果は #false (and 式 s へ right のという式はない evaluated.)

もし任意の の式 s evaluate へ 値 other than #true または #false, と reports エラー.

```
+-----+
| [構文] |
| |
| (or expression expression expression...) |
+-----+
```

expressions の 1 つが #true になった時点で #true に評価されます (その式より右の式は評価されません)。すべてが #false なら、or 式は #false に評価されます。

expressions のいずれかが #true でも #false でもない値に評価されると、or はエラーを報告します。

```
+-----+
| [構文] |
| |
| |
+-----+
```

```
| (check-expect expression expected-expression) |
+-----+
```

最初の expression が expected-expression と同じ値に評価されることを検査します。

```
(check-expect (fahrenheit->celsius 212) 100)
(check-expect (fahrenheit->celsius -40) -40)
```

```
(define (fahrenheit->celsius f)
 (* 5/9 (- f 32)))
```

check-expect 式は、学習者プログラムのトップレベルに置く必要があります。また、検査対象の関数定義の前を含め、プログラムのどこに現れてもかまいません。そこに check-expect を置くことで、プログラムは動作する例によってプログラムの意図を将来の読者に伝え、本体の関数定義を読まなくても済むことがしばしばあります。check-expect (およびすべての check 形式) の構文エラーは、意図的に実行時まで遅らせます。学生が完全な関数ヘッダを必ずしも書かなくてもテストを書けるようにするためです。

expr または expected-expr が不正確数や関数値を生むのはエラーです。不正確数については、単純な等価比較をするのは道義的に正しくありません。代わりに両方が小さな区間内にあるかを調べます。check-within を参照してください。関数については (Intermediate 以上)、比較は証明可能なほど不可能です。

```
+-----+
| [構文] |
| |
| (check-random expression expected-expression) |
+-----+
```

最初の expression が expected-expression と同じ値に評価されることを検査します。

この形式は、両方の部分に同じ乱数生成器を与えます。両方の部分が同じ区間から同じ順序で乱数を要求すれば、同じ乱数を受け取ります。

check-random が役立つ簡単な例です：

```
(define WIDTH 100)
(define HEIGHT (* 2 WIDTH))

(define-struct player (name x y))
; A Player is (make-player String Nat Nat)

; String -> Player

(check-random (create-randomly-placed-player "David Van Horn")
 (make-player "David Van Horn" (random WIDTH) (random HEIGHT)))

(define (create-randomly-placed-player name)
 (make-player name (random WIDTH) (random HEIGHT)))
```



check-random の両方の部分で、random が同じ数に対して同じ順序で呼ばれていることに注意してください。2つの部分が異なる区間で random を呼ぶと、失敗しやすくなります：

```
; String -> Player

(check-random (create-randomly-placed-player "David Van Horn")
 (make-player "David Van Horn" (random WIDTH) (random HEIGHT)))

(define (create-randomly-placed-player name)
 (a-helper-function name (random HEIGHT)))

; String Number -> Player
(define (a-helper-function name height)
 (make-player name (random WIDTH) height))
```

a-helper-function への引数が先に評価されるため、random はまず区間 [0,HEIGHT) で呼ばれ、次に [0,WIDTH) で呼ばれます。つまり、直前の check-random とは異なる順序です。

expr または expected-expr が関数値や不正確数を生むのはエラーです。詳細は check-expect の注意を参照してください。

```
+-----+
| [構文] |
| |
| (check-satisfied expression predicate) |
+-----+
```

最初の expression が、名前付きの predicate (1 引数の関数) を満たすことを検査します。「満たす」とは「その関数が与えられた値に対して #true を返す」という意味です。

check-satisfied の簡単な例です：

```
> (check-satisfied 1 odd?)
The test passed!

> (check-satisfied 1 even?)
Ran 1 test. 0 tests passed.
Check failures:
Actual value | 1 | does not satisfy even?.
```

一般に check-satisfied により、プログラム設計者は定義した関数を使ってテスト一式を組み立てられます：

```
; [cons Number [List-of Number]] -> Boolean
; a function for testing htdp-sort

(check-expect (sorted? (list 1 2 3)) #true)
(check-expect (sorted? (list 2 1 3)) #false)
```

```
(define (sorted? l)
 (cond
 [(empty? (rest l)) #true]
 [else (and (<= (first l) (second l)) (sorted? (rest l)))]))
```

```
; [List-of Number] -> [List-of Number]
; create a sorted version of the given list of numbers
```

```
(check-satisfied (htdp-sort (list 1 2 0 3)) sorted?)
```

```
(define (htdp-sort l)
 (cond
 [(empty? l) l]
 [else (insert (first l) (htdp-sort (rest l)))]))
```

```
; Number [List-of Number] -> [List-of Number]
; insert x into l at proper place
; assume l is arranged in ascending order
; the result is sorted in the same way
```

```
(define (insert x l)
 (cond
 [(empty? l) (list x)]
 [else (if (<= x (first l)) (cons x l) (cons (first l) (insert x (rest l))))]))
```

そして実際、htdp-sortの結果はsorted? 述語を満たします：

```
> (check-satisfied (htdp-sort (list 1 2 0 3)) sorted?)
```

```
+-----+
| [構文] |
| |
| (check-within expression expected-expression delta) |
+-----+
```

expression という式の値が、expected-expression という式が生む値と構造的に等しいかを検査します。最初の式の中のすべての数は、2 つ目の式の対応する数の delta 以内でなければなりません。

```
(define-struct roots (x sqrt))
; RT is [List-of (make-roots Number Number)]
```

```
(define (root-of a)
 (make-roots a (sqrt a)))
```

```
(define (roots-table xs)
 (cond
 [(empty? xs) '()]
 [else (cons (root-of (first xs)) (roots-table (rest xs)))]))
```

入れ子のデータに不正確数が含まれるため、検査には check-within が適切で、delta が十分大きければテストは成功します：

```
> (check-within (roots-table (list 1.0 2.0 3.0)) (list (make-roots 1.0 1.0) (make-roots 2 1.414) (make-roots 3 1.713)) 0.1)
The test passed!
```

例：

expressions または expected-expression が関数値を生むのはエラーです。詳細は check-expect の注意を参照してください。

```

+-----+
| [構文] |
+-----+
| (check-error expression expected-error-message) |
| (check-error expression) |
+-----+

```

check-error の使用が求められる、初学者によくある例です：

```
(define sample-table
 (('("matthias" 10)
 ("matthew" 20)
 ("robby" -1)
 ("shriram" 18)))

; [List-of [List String Number]] String -> Number
; determine the number associated with s in table

(define (lookup table s)
 (cond
 [(empty? table) (error (string-append s " not found"))]
 [else (if (string=? (first (first table)) s)
```

```
(second (first table))
(lookup (rest table))))))
```

この文脈での次の2つの例を考えてください：

例:

```
> (check-expect (lookup sample-table "matthew") 20)
The test passed!
```

例:

```
> (check-error (lookup sample-table "kathi") "kathi not found")
The test passed!
```

```
+-----+
| [構文] |
| |
| (check-member-of expression expression expression...) |
+-----+
```

最初の expression の値が、続く expressions のいずれかと同じであることを検査します。

```
; [List-of X] -> X
; pick a random element from the given list l
(define (pick-one l)
 (list-ref l (random (length l))))
```

例:

```
> (check-member-of (pick-one '("a" "b" "c")) "a" "b" "c")
The test passed!
```

expressions のいずれかが関数値を生むのはエラーです。詳細は check-expect の注意を参照してください。

```
+-----+
| [構文] |
| |
| (check-range expression low-expression high-expression) |
+-----+
```

最初の expression の値が、low-expression と high-expression の値の間（両端を含む）にある数であることを検査します。

check-range 形式は、不正確数を計算する関数のありうる結果の範囲を区切るのに最適です：

```
(define EPSILON 0.001)

; [Real -> Real] Real -> Real
; what is the slope of f at x?
(define (differentiate f x)
 (slope f (- x EPSILON) (+ x EPSILON)))
```

```
; [Real -> Real] Real Real -> Real
(define (slope f left right)
 (/ (- (f right) (f left))
 2 EPSILON))
```

```
(check-range (differentiate sin 0) 0.99 1.0)
```

expression、low-expression、または high-expression が関数値を生むのはエラーです。詳細は check-expect の注意を参照してください。

```
+-----+
| [構文] |
| |
| (require string) |
+-----+
```

string で指定したモジュールの定義を、現在のモジュール（つまり現在のファイル）で使えるようにします。string は現在のファイルからの相対パスのファイルを指します。

string には、プラットフォームごとのパス規約の違いを避けるため、いくつかの制約があります：/ はディレクトリ区切り、. は常にカレントディレクトリ、.. は常に親ディレクトリ、パス要素に使えるのは a から z（大文字小文字）、0 から 9、-、\_、. のみで、string は空であってはならず、先頭や末尾の / も含んではいけません。

```
+-----+
| [構文] |
| |
| (require module-name) |
+-----+
```

インストール済みライブラリ内のファイルにアクセスします。ライブラリ名は、相対パスの string と同じ制約を持つ識別子です（ただし引用符は不要）が、さらに .. を含んではいけないという制約があります。

```
+-----+
| [構文] |
| |
| (require (lib string string...)) |
+-----+
```

インストール済みライブラリ内のファイルにアクセスし、その定義を現在のモジュール（つまり現在のファイル）で使えるようにします。最初の string はライブラリファイル名で、残りの strings はファイルがインストールされているコレクション（およびサブコレクションなど）の名前です。各 string は (require string) 形式と同じ制約を受けます。

```
+-----+
| [構文] |
| |
| |
+-----+
```

```
| (require (planet string (string string number number))) |
+-----+
```

インターネット上のPLaneTサーバ経由で配布されるライブラリにアクセスし、その定義を現在のモジュール（つまり現在のファイル）で使えるようにします。

planet の require の完全な文法は「Importing and Exporting: require and provide」にあります。構文の例を探すいちばん良い場所はPLaneTサーバ上の、特定パッケージの説明です。

## 2.5 シグネチャ

Signatures do not have to be comment: They can also be part of the > code. When a signature is attached to a function, DrRacket will check > that program uses the function in accordance with the signature and > display signature violations along with the test results. A signature is a regular value, and is specified as a > signature form, a > special syntax that only works with: signature declarations > and inside signature expressions.

```
+-----+
| [構文] |
| |
| (: name signature-form) |
+-----+
```

これ attaches シグネチャ specified によってシグネチャ-form へ 定義 の name. There でなければならない 定義 の name somewhere の中の プログラム.

```
(: age Integer)
(define age 42)

(: area-of-square (Number -> Number))
(define (area-of-square len)
 (sqr len))
```

On running プログラム, Racket checks whether シグネチャ s attached with: actually match 値 の 変数. もし they don't, Racket reports シグネチャ violation along とともに test failures.

たとえば、この piece の code:

```
(: age Integer)
(define age "fortytwo")
```

Yields この output:

```
+-----+
| `1 signature violation.` |
+-----+
```

```
| `Signature violations:`
| `got "fortytwo" at line 2, column 12, signature...`
+-----+
```

Note という シグネチャ violation does ない stop running プログラム.

```
+-----+
| [構文] |
| (signature signature-form) |
+-----+
```

これ 返す シグネチャ described によって シグネチャ-form として 値.

### 2.5.1 シグネチャ形式

Any expression can be a signature form, in which case the signature is > the value returned by that expression. There are a few special > signature forms, however: In a signature form, any name that starts with a % is a > signature variable that stands for any signature depending on how > the signature is used. Example:

```
(: same (%a -> %a))
```

```
(define (same x) x)
```

```
+-----+
| [構文] |
| (input-signature-form... -> output-signature-form) |
+-----+
```

これ シグネチャ form describes 関数 とともに inputs described によって input-シグネチャ-forms と output described によって output-シグネチャ-form.

```
+-----+
| [構文] |
| (enum expr...) |
+-----+
```

これ シグネチャ describes enumeration の 値 s returned によって exprs.

例:

```
(: cute? ((enum "cat" "snake") -> Boolean))
```

```
(define (cute? pet)
 (cond
 [(string=? pet "cat") #t]
 [(string=? pet "snake") #f]))
```

```
+-----+
| [構文] |
| |
| (mixed signature-form...) |
+-----+
```

これシグネチャ describes mixed data, i.e. itemization where 各の cases has シグネチャ described によって シグネチャ-form.

例:

```
(define SIGS (signature (mixed Aim Fired)))
```

```
+-----+
| [構文] |
| |
| (ListOf signature-form) |
+-----+
```

これ シグネチャ describes リスト where elements は described によって シグネチャ-form.

```
+-----+
| [構文] |
| |
| (predicate expression) |
+-----+
```

これ シグネチャ describes 値 s through 述語: 式 must evaluate へ 関数の one 引数 という 返す 真偽値. シグネチャ matches すべての 値 s のための であり 述語 返す #true.

## 2.5.2 構造体シグネチャ

define-struct 形式は、シグネチャで使える2つの追加名を定義します。struct という構造体に対しては、Struct と StructOf です。これらの名前は大文字で始まることに注意してください。とくに Struct という構造体も Struct と StructOf を定義します。さらに追加名を作るとき、ハイフンは取り除かれ、ハイフンの直後の文字は大文字になります。したがって foo-bar という構造体は FooBar と FooBarOf を定義します。Struct は、この構造体型の構造体値を表すシグネチャです。StructOf は各フィールドのシグネチャを入力として取る関数で、この構造体型の値を表すシグネチャを返し、あわせて値の各フィールドの値も記述します。

```
(define-struct pair [fst snd])
```

```
(: add-pair ((PairOf Number Number) -> Number))
```

```
(define (add-pair p)
 (+ (pair-fst p) (pair-snd p)))
```



## 2.6 あらかじめ定義された関数

remaining subsections リスト それらの 関数 `s` という は built-in プログラム `ming` language. All other 関数 `s` は imported から `teachpack` または でなければならぬ `defined` の中の プログラム.

## 2.7 数: 整数・有理数・実数・複素数・正確数・非正確数

```
+-----+
| [手続き] |
| |
| (* x y z...) → number |
| x: number |
| y: number |
| z: number |
+-----+
```

すべての数を掛け合わせます。

```
> (* 5 3)
15
> (* 5 3 2)
30
```

```
+-----+
| [手続き] |
| |
| (+ x y z...) → number |
| x: number |
| y: number |
| z: number |
+-----+
```

すべての数を足し合わせます。

```
> (+ 2/3 1/16)
35/48
> (+ 3 2 5 8)
18
```

```
+-----+
| [手続き] |
| |
| (- x y...) → number |
| x: number |
| y: number |
+-----+
```

Subtracts second (and following) 数(s) から first; negates 数 もし there は only one 引数.

```
> (- 5)
-5
> (- 5 3)
```

2

> (- 5 3 1)

1

```
+-----+
| [手続き] |
| |
| (/ x y z...) → number |
| x: number |
| y: number |
| z: number |
+-----+
```

第1引数を第2（およびそれ以降）の数で割ります。

> (/ 12 2)

6

> (/ 12 2 3)

2

```
+-----+
| [手続き] |
| |
| (< x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

2つ以上の（実）数が小さい順かを比較します。

> (< 42 2/5)

#false

```
+-----+
| [手続き] |
| |
| (<= x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

2つ以上の（実）数が小さいか等しい順かを比較します。

> (<= 42 2/5)

#false

```
+-----+
| [手続き] |
| |
| (= x y z...) → boolean? |
| x: number |
| y: number |
+-----+
```

```
| z: number |
+-----+
```

2 つ以上の数が等しいかを比較します。

```
> (= 42 2/5)
#false
```

```
+-----+
| [手続き] |
| |
| (> x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

2 つ以上の（実）数が大きい順かを比較します。

```
> (> 42 2/5)
#true
```

```
+-----+
| [手続き] |
| |
| (>= x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

2 つ以上の（実）数が大きいかな等しい順かを比較します。

```
> (>= 42 42)
#true
```

```
+-----+
| [手続き] |
| |
| (abs x) → real |
| x: real |
+-----+
```

判定する: absolute 値 の 実数 数.

```
> (abs -12)
12
```

```
+-----+
| [手続き] |
| |
| (acos x) → number |
| x: number |
+-----+
```

the arccosine (inverse of cos) of a number を計算します。

```
> (acos 0)
#i1.5707963267948966
```

```
+-----+
| [手続き] |
| |
| (add1 x) → number |
| x: number |
+-----+
```

Increments given 数.

```
> (add1 2)
3
```

```
+-----+
| [手続き] |
| |
| (angle x) → real |
| x: number |
+-----+
```

the angle from a complex number を取り出します。

```
> (angle (make-polar 3 4))
#i-2.2831853071795867
```

```
+-----+
| [手続き] |
| |
| (asin x) → number |
| x: number |
+-----+
```

the arcsine (inverse of sin) of a number を計算します。

```
> (asin 0)
0
```

```
+-----+
| [手続き] |
| |
| (atan x) → number |
| x: number |
+-----+
```

計算する: arctangent の given 数:

```
> (atan 0)
0
> (atan 0.5)
#i0.4636476090008061
```

Also comes in a two-argument version where (atanx) computes (atan(/yx)) but the signs of y and x determine the quadrant of the result and the result tends to be more accurate than that of the 1-argument version in borderline cases:

```
> (atan 3 4)
#i0.6435011087932844
> (atan -2 -1)
#i-2.0344439357957027
```

```
+-----+
| [手続き] |
| |
| (ceiling x) → integer |
| x: real |
+-----+
```

判定する: closest 整数 (正確 または in 正確) above 実数 数. 参照: round.

```
> (ceiling 12.3)
#i13.0
```

```
+-----+
| [手続き] |
| |
| (complex? x) → boolean? |
| x: any/c |
+-----+
```

some value is complexかどうかを判定します。

```
> (complex? 1-2i)
#true
```

```
+-----+
| [手続き] |
| |
| (conjugate x) → number |
| x: number |
+-----+
```

Flips sign of the imaginary part of the complex number.

```
> (conjugate 3+4i)
3-4i
> (conjugate -2-5i)
-2+5i
> (conjugate (make-polar 3 4))
#i-1.960930862590836+2.2704074859237844i
```

```
+-----+
| [手続き] |
| |
| (cos x) → number |
+-----+
```

```
| x: number |
+-----+
```

the cosine of a number (radians) を計算します。

```
> (cos pi)
#i-1.0
```

```
+-----+
| [手続き] |
| |
| (cosh x) → number |
| x: number |
+-----+
```

the hyperbolic cosine of a number を計算します。

```
> (cosh 10)
#i11013.232920103324
```

```
+-----+
| [手続き] |
| |
| (current-seconds) → integer |
+-----+
```

判定する: current time の中の seconds elapsed (since platform-specific starting date).

```
> (current-seconds)
1779843327
```

```
+-----+
| [手続き] |
| |
| (denominator x) → integer |
| x: rational? |
+-----+
```

the denominator of a rational を計算します。

```
> (denominator 2/3)
3
```

```
+-----+
| [値] |
| |
| e: real |
+-----+
```

ネイピア数（オイラー数）です。

```
> e
#i2.718281828459045
```

```
+-----+
| [手続き] |
+-----+
```

```

|
| (even? x) → boolean? |
| x: integer |
+-----+

```

some integer (exact or inexact) is even or not かどうかを判定します。

```

> (even? 2)
#true

```

```

+-----+
| [手続き] |
| |
| (exact->inexact x) → number |
| x: number |
+-----+

```

an exact number to an inexact one を変換します。

```

> (exact->inexact 12)
#i12.0

```

```

+-----+
| [手続き] |
| |
| (exact? x) → boolean? |
| x: number |
+-----+

```

some number is exact かどうかを判定します。

```

> (exact? (sqrt 2))
#false

```

```

+-----+
| [手続き] |
| |
| (exp x) → number |
| x: number |
+-----+

```

判定する: e raised へ 数.

```

> (exp -2)
#i0.1353352832366127

```

```

+-----+
| [手続き] |
| |
| (expt x y) → number |
| x: number |
| y: number |
+-----+

```

the power of the first to the second number, which is to say, exponentiation を計算します。

```
> (expt 16 1/2)
```

```
4
```

```
> (expt 3 -4)
```

```
1/81
```

```
+-----+
| [手続き] |
| |
| (floor x) → integer |
| x: real |
+-----+
```

判定する: closest 整数 (正確 または in 正確) below 実数 数. 参照: round.

```
> (floor 12.3)
```

```
#i12.0
```

```
+-----+
| [手続き] |
| |
| (gcd x y...) → integer |
| x: integer |
| y: integer |
+-----+
```

判定する: greatest common divisor の two 整数 s (正確 または in 正確).

```
> (gcd 6 12 8)
```

```
2
```

```
+-----+
| [手続き] |
| |
| (imag-part x) → real |
| x: number |
+-----+
```

the imaginary part from a complex number を取り出します。

```
> (imag-part 3+4i)
```

```
4
```

```
+-----+
| [手続き] |
| |
| (inexact->exact x) → number |
| x: number |
+-----+
```

Approximates in 正確 数 によって 正確 one.



```
> (inexact->exact 12.0)
```

```
12
```

```
+-----+
| [手続き] |
| |
| (inexact? x) → boolean? |
| x: number |
+-----+
```

some number is inexactかどうかを判定します。

```
> (inexact? 1-2i)
```

```
#false
```

```
+-----+
| [手続き] |
| |
| (integer->char x) → char |
| x: exact-integer? |
+-----+
```

Looks up 文字 という corresponds へ given 正確 整数 の中の ASCII table (if any).

```
> (integer->char 42)
```

```
#\"
```

```
+-----+
| [手続き] |
| |
| (integer-sqrt x) → complex |
| x: integer |
+-----+
```

the integer or imaginary-integer square root of an integer を計算します。

```
> (integer-sqrt 11)
```

```
3
```

```
> (integer-sqrt -11)
```

```
0+3i
```

```
+-----+
| [手続き] |
| |
| (integer? x) → boolean? |
| x: any/c |
+-----+
```

some value is an integer (exact or inexact)かどうかを判定します。

```
> (integer? (sqrt 2))
```

```
#false
```

```
+-----+
| [手続き] |
| |
+-----+
```

```

|
| (lcm x y...) → integer |
| x: integer |
| y: integer |
+-----+

```

判定する: least common multiple の two 整数 s (正確 または in 正確).

```

> (lcm 6 12 8)
24

```

```

+-----+
| [手続き] |
|
| (log x) → number |
| x: number |
+-----+

```

判定する: base-e logarithm の 数.

```

> (log 12)
#i2.4849066497880004

```

```

+-----+
| [手続き] |
|
| (magnitude x) → real |
| x: number |
+-----+

```

判定する: magnitude の 複素数 数.

```

> (magnitude (make-polar 3 4))
#i2.9999999999999996

```

```

+-----+
| [手続き] |
|
| (make-polar x y) → number |
| x: real |
| y: real |
+-----+

```

a complex from a magnitude and angle を作成します。

```

> (make-polar 3 4)
#i-1.960930862590836-2.2704074859237844i

```

```

+-----+
| [手続き] |
|
| (make-rectangular x y) → number |
| x: real |
+-----+

```

```
| y: real |
+-----+
```

a complex from a real and an imaginary part を作成します。

```
> (make-rectangular 3 4)
3+4i
```

```
+-----+
| [手続き] |
| (max x y...) → real |
| x: real |
| y: real |
+-----+
```

判定する: largest 数—aka, maximum.

```
> (max 3 2 8 7 2 9 0)
9
```

```
+-----+
| [手続き] |
| (min x y...) → real |
| x: real |
| y: real |
+-----+
```

判定する: smallest 数—aka, minimum.

```
> (min 3 2 8 7 2 9 0)
0
```

```
+-----+
| [手続き] |
| (modulo x y) → integer |
| x: integer |
| y: integer |
+-----+
```

Finds remainder の division の first 数 によって second:

```
> (modulo 9 2)
```

```
1
```

```
> (modulo 3 -4)
```

```
-1
```

```
+-----+
| [手続き] |
| (negative? x) → boolean? |
+-----+
```

```
| x: real |
+-----+
```

some real number is strictly smaller than zeroかどうかを判定します。

```
> (negative? -2)
#true
```

```
+-----+
| [手続き] |
| |
| (number->string x) → string |
| x: number |
+-----+
```

a number to a string を変換します。

```
> (number->string 42)
"42"
```

```
+-----+
| [手続き] |
| |
| (number->string-digits x p) → string |
| x: number |
| p: posint |
+-----+
```

a number x to a string with the specified number of digits を変換します。

```
> (number->string-digits 0.9 2)
"0.9"
> (number->string-digits pi 4)
"3.1416"
```

```
+-----+
| [手続き] |
| |
| (number? n) → boolean? |
| n: any/c |
+-----+
```

～かどうかを判定する: ある 値 は 数:

```
> (number? "hello world")
#false
> (number? 42)
#true
```

```
+-----+
| [手続き] |
| |
| (numerator x) → integer |
+-----+
```

```
| x: rational? |
+-----+
```

the numerator of a rational を計算します。

```
> (numerator 2/3)
2
```

```
+-----+
| [手続き] |
| |
| (odd? x) → boolean? |
| x: integer |
+-----+
```

some integer (exact or inexact) is odd or not かどうかを判定します。

```
> (odd? 2)
#false
```

```
+-----+
| [値] |
| |
| pi: real |
+-----+
```

ratio of the circle's circumference to its diameter.

```
> pi
#i3.141592653589793
```

```
+-----+
| [手続き] |
| |
| (positive? x) → boolean? |
| x: real |
+-----+
```

some real number is strictly larger than zero かどうかを判定します。

```
> (positive? -2)
#false
```

```
+-----+
| [手続き] |
| |
| (quotient x y) → integer |
| x: integer |
| y: integer |
+-----+
```

Divides first 整数—also called dividend—by second—known as divisor—to obtain quotient.

```
> (quotient 9 2)
```

```
4
```

```
> (quotient 3 4)
```

```
0
```

```
+-----+
| [手続き] |
| |
| (random x) → natural? |
| x: (and/c natural? positive?) |
+-----+
```

Generates 乱数 natural 数 less than ある given 正確 natural.

```
> (random 42)
```

```
41
```

```
+-----+
| [手続き] |
| |
| (rational? x) → boolean? |
| x: any/c |
+-----+
```

some value is a rational number かどうかを判定します。

```
> (rational? 1)
```

```
#true
```

```
> (rational? -2.349)
```

```
#true
```

```
> (rational? #i1.23456789)
```

```
#true
```

```
> (rational? (sqrt -1))
```

```
#false
```

```
> (rational? pi)
```

```
#true
```

```
> (rational? e)
```

```
#true
```

```
> (rational? 1-2i)
```

```
#false
```

As the interactions show, the teaching languages considers many more numbers as rationals than expected. In particular, pi is a rational number because it is only a finite approximation to the mathematical  $\pi$ . Think of rational? as a suggestion to think of these numbers as fractions.

```
+-----+
| [手続き] |
| |
| (real-part x) → real |
| x: number |
+-----+
```

the real part from a complex number を取り出します。

```
> (real-part 3+4i)
3
```

```
+-----+
| [手続き] |
| |
| (real? x) → boolean? |
| x: any/c |
+-----+
```

some value is a real number かどうかを判定します。

```
> (real? 1-2i)
#false
```

```
+-----+
| [手続き] |
| |
| (remainder x y) → integer |
| x: integer |
| y: integer |
+-----+
```

判定する: remainder の dividing first によって second 整数 (正確 または in 正確).

```
> (remainder 9 2)
1
> (remainder 3 4)
3
```

```
+-----+
| [手続き] |
| |
| (round x) → integer |
| x: real |
+-----+
```

Rounds 実数 数へ 整数 (rounds へ even へ break ties). 参照: floor と ceiling.

```
> (round 12.3)
#i12.0
```

```
+-----+
| [手続き] |
| |
| (sgn x) → (union 1 #i1.0 0 #i0.0 -1 #i-1.0) |
| x: real |
+-----+
```

判定する: sign の 実数 数.

```
> (sgn -12)
-1
```

```

+-----+
| [手続き] |
| |
| (sin x) → number |
| x: number |
+-----+

```

the sine of a number (radians) を計算します。

```

> (sin pi)
#i1.2246467991473532e-16

```

```

+-----+
| [手続き] |
| |
| (sinh x) → number |
| x: number |
+-----+

```

the hyperbolic sine of a number を計算します。

```

> (sinh 10)
#i11013.232874703393

```

```

+-----+
| [手続き] |
| |
| (sqr x) → number |
| x: number |
+-----+

```

the square of a number を計算します。

```

> (sqr 8)
64

```

```

+-----+
| [手続き] |
| |
| (sqrt x) → number |
| x: number |
+-----+

```

the square root of a number を計算します。

```

> (sqrt 9)
3
> (sqrt 2)
#i1.4142135623730951

```

```

+-----+
| [手続き] |
| |
| (sub1 x) → number |
+-----+

```



```
| x: number |
+-----+
```

Decrements given 数.

```
> (sub1 2)
```

```
1
```

```
+-----+
| [手続き] |
| |
| (tan x) → number |
| x: number |
+-----+
```

the tangent of a number (radians) を計算します。

```
> (tan pi)
```

```
#i-1.2246467991473532e-16
```

```
+-----+
| [手続き] |
| |
| (zero? x) → boolean? |
| x: number |
+-----+
```

some number is zero or not かどうかを判定します。

```
> (zero? 2)
```

```
#false
```

## 2.8 真偽値

```
+-----+
| [手続き] |
| |
| (boolean->string x) → string |
| x: boolean? |
+-----+
```

Produces 文字列 のための given 真偽値

```
> (boolean->string #false)
```

```
"#false"
```

```
> (boolean->string #true)
```

```
"#true"
```

```
+-----+
| [手続き] |
| |
| (boolean=? x y) → boolean? |
| x: boolean? |
| y: boolean? |
+-----+
```

two booleans are equalかどうかを判定します。

```
> (boolean=? #true #false)
#false
```

```
+-----+
| [手続き] |
| |
| (boolean? x) → boolean? |
| x: any/c |
+-----+
```

値が真偽値かどうかを判定します。

```
> (boolean? 42)
#false
> (boolean? #false)
#true
```

```
+-----+
| [手続き] |
| |
| (false? x) → boolean? |
| x: any/c |
+-----+
```

a value is falseかどうかを判定します。

```
> (false? #false)
#true
```

```
+-----+
| [手続き] |
| |
| (not x) → boolean? |
| x: boolean? |
+-----+
```

Negates 真偽値 値.

```
> (not #false)
#true
```

## 2.9 シンボル

```
+-----+
| [手続き] |
| |
| (symbol->string x) → string |
| x: symbol |
+-----+
```

a symbol to a string を変換します。

```
> (symbol->string 'c)
"c"
```

```
+-----+
| [手続き] |
| |
| (symbol=? x y) → boolean? |
| x: symbol |
| y: symbol |
+-----+
```

two symbols are equalかどうかを判定します。

```
> (symbol=? 'a 'b)
#false
```

```
+-----+
| [手続き] |
| |
| (symbol? x) → boolean? |
| x: any/c |
+-----+
```

値がシンボルかどうかを判定します。

```
> (symbol? 'a)
#true
```

## 2.10 リスト

```
+-----+
| [手続き] |
| |
| (append x y z...) → list? |
| x: list? |
| y: list? |
| z: list? |
+-----+
```

a single list from several, by concatenation of the items を作成します。

```
> (append (cons 1 (cons 2 '())) (cons "a" (cons "b" empty)))
(list 1 2 "a" "b")
```

```
+-----+
| [手続き] |
| |
| (assoc x l) → (union (listof any) #false) |
| x: any/c |
| l: (listof any) |
+-----+
```

l 上で first が x と equal? な最初の対を返します。なければ #false を返します。

```
> (assoc "hello" '(("world" 2) ("hello" 3) ("good" 0)))
(list "hello" 3)
```

```
+-----+
| [手続き] |
| |
| (assq x l) → (union #false cons?) |
| x: any/c |
| l: list? |
+-----+
```

～かどうかを判定する: ある item は first item の pair の中の リスト の pairs. (それ compares items とともに eq?.)

```
> a
(list (list 'a 22) (list 'b 8) (list 'c 70))
> (assq 'b a)
(list 'b 8)
```

```
+-----+
| [手続き] |
| |
| (caaar x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (car(car(carx))).

```
> w
(list (list (list (list "bye") 3) #true) 42)
> (caaar w)
(list "bye")
```

```
+-----+
| [手続き] |
| |
| (caadr x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (car(car(cdrx))).

```
> (caadr (cons 1 (cons (cons 'a '()) (cons (cons 'd '()) '()))))
'a
```

```
+-----+
| [手続き] |
| |
| (caar x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (car(carx)).

```
> y
(list (list (list 1 2 3) #false "world"))
> (caar y)
(list 1 2 3)
```

```
+-----+
| [手続き] |
| |
| (cadar x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (car(cdr(carx)))。

```
> w
(list (list (list (list "bye") 3) #true) 42)
> (cadar w)
#true
```

```
+-----+
| [手続き] |
| |
| (caddr x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (car(cdr(cdr(cdrx))))。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (caddr v)
4
```

```
+-----+
| [手続き] |
| |
| (caddr x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (car(cdr(cdrx)))。

```
> x
(list 2 "hello" #true)
> (caddr x)
#true
```

```
+-----+
| [手続き] |
| |
| (cadr x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (car(cdrx))。

```
> x
(list 2 "hello" #true)
> (cadr x)
"hello"
```

```
+-----+
| [手続き] |
| |
| (car x) → any/c |
| x: cons? |
+-----+
```

空でないリストの先頭要素を取り出します。

```
> x
(list 2 "hello" #true)
> (car x)
2
```

```
+-----+
| [手続き] |
| |
| (cdaar x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (cdr(car(carx)))。

```
> w
(list (list (list (list "bye") 3) #true) 42)
> (cdaar w)
(list 3)
```

```
+-----+
| [手続き] |
| |
| (cdadr x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (cdr(car(cdrx)))。

```
> (cdadr (list 1 (list 2 "a") 3))
(list "a")
```

```
+-----+
| [手続き] |
| |
| (cdar x) → list? |
| x: list? |
+-----+
```

LISP 風の selector: (cdr(carx))。

```
> y
(list (list (list 1 2 3) #false "world"))
> (cdar y)
(list #false "world")
```

```
+-----+
| [手続き] |
| |
| (cddar x) → any/c |
| x: list? |
+-----+
```

LISP 風の選択子: (cdr (cdr (car x)))

```
> w
(list (list (list (list "bye") 3) #true) 42)
> (cddar w)
'()
```

```
+-----+
| [手続き] |
| |
| (cdddr x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (cdr(cdr(cdrx)))。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (cdddr v)
(list 4 5 6 7 8 9 'A)
```

```
+-----+
| [手続き] |
| |
| (cddr x) → list? |
| x: list? |
+-----+
```

LISP 風の selector: (cdr(cdrx))。

```
> x
(list 2 "hello" #true)
> (cddr x)
(list #true)
```

```
+-----+
| [手続き] |
| |
| (cdr x) → any/c |
+-----+
```

```
| x: cons? |
+-----+
```

空でないリストの残り (rest) を取り出します。

```
> x
(list 2 "hello" #true)
> (cdr x)
(list "hello" #true)
```

```
+-----+
| [手続き] |
| |
| (cons x y) → list? |
| x: any/c |
| y: list? |
+-----+
```

a list を構築します。

```
> (cons 1 '())
(cons 1 '())
```

```
+-----+
| [手続き] |
| |
| (cons? x) → boolean? |
| x: any/c |
+-----+
```

some value is a constructed list かどうかを判定します。

```
> (cons? (cons 1 '()))
#true
> (cons? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (eighth x) → any/c |
| x: list? |
+-----+
```

the eighth item of a non-empty list を選択します。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (eighth v)
8
```

```
+-----+
| [手続き] |
| |
+-----+
```



```
| (empty? x) → boolean? |
| x: any/c |
+-----+
```

値が空リストかどうかを判定します。

```
> (empty? '())
#true
> (empty? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (fifth x) → any/c |
| x: list? |
+-----+
```

the fifth item of a non-empty list を選択します。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (fifth v)
5
```

```
+-----+
| [手続き] |
| |
| (first x) → any/c |
| x: cons? |
+-----+
```

空でないリストの先頭要素を取り出します。

```
> x
(list 2 "hello" #true)
> (first x)
2
```

```
+-----+
| [手続き] |
| |
| (fourth x) → any/c |
| x: list? |
+-----+
```

the fourth item of a non-empty list を選択します。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (fourth v)
4
```

```

+-----+
| [手続き] |
| |
| (length l) → natural? |
| l: list? |
+-----+

```

リスト上の要素の個数を求めます。

```

> x
(list 2 "hello" #true)
> (length x)
3

```

```

+-----+
| [手続き] |
| |
| (list x...) → list? |
| x: any/c |
+-----+

```

引数からなるリストを組み立てます。

```

> (list 1 2 3 4 5 6 7 8 9 0)
(cons 1 (cons 2 (cons 3 (cons 4 (cons 5 (cons 6 (cons 7 (cons 8 (cons 9 (cons 0 '()))))))))))

```

```

+-----+
| [手続き] |
| |
| (list* x... l) → list? |
| x: any/c |
| l: list? |
+-----+

```

複数の要素をリストに加えてリストを組み立てます。

```

> x
(list 2 "hello" #true)
> (list* 4 3 x)
(list 4 3 2 "hello" #true)

```

```

+-----+
| [手続き] |
| |
| (list-ref x i) → any/c |
| x: list? |
| i: natural? |
+-----+

```

the indexed item from the list を取り出します。

```

> v
(list 1 2 3 4 5 6 7 8 9 'A)

```

```
> (list-ref v 9)
```

```
'A
```

```
+-----+
| [手続き] |
| |
| (list? x) → boolean? |
| x: any/c |
+-----+
```

whether the given value is a list を検査します。

```
> (list? 42)
```

```
#false
```

```
> (list? '())
```

```
#true
```

```
> (list? (cons 1 (cons 2 '())))
```

```
#true
```

```
+-----+
| [手続き] |
| |
| (make-list i x) → list? |
| i: natural? |
| x: any/c |
+-----+
```

a list of i copies of x を構築します。

```
> (make-list 3 "hello")
```

```
(cons "hello" (cons "hello" (cons "hello" '())))
```

```
+-----+
| [手続き] |
| |
| (member x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

値がリスト上にあるかどうかを判定します (equal? で比較)。

```
> x
```

```
(list 2 "hello" #true)
```

```
> (member "hello" x)
```

```
#true
```

```
+-----+
| [手続き] |
| |
| (member? x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

値がリスト上にあるかどうかを判定します (equal? で比較)。

```
> x
(list 2 "hello" #true)
> (member? "hello" x)
#true
```

[手続き]
(memq x l) → boolean?
x: any/c
l: list?

some value x is on some list l, using eq? to compare x with items on l かどうかを判定します。

```
> x
(list 2 "hello" #true)
> (memq (list (list 1 2 3)) x)
#false
```

[手続き]
(memq? x l) → boolean?
x: any/c
l: list?

some value x is on some list l, using eq? to compare x with items on l かどうかを判定します。

```
> x
(list 2 "hello" #true)
> (memq? (list (list 1 2 3)) x)
#false
```

[手続き]
(memv x l) → (or/c #false list)
x: any/c
l: list?

〜かどうかを判定する: ある 値 は 上の リスト もし so, it produces suffix の リスト という starts とともに x もし not, it produces false. (それ compares 値 s とともに eqv? 述語.)

```
> x
(list 2 "hello" #true)
```

```
> (memv (list (list 1 2 3)) x)
#false
```

```
+-----+
| [値] |
| |
| null: list |
+-----+
```

空リストの別名です。

```
> null
'()
```

```
+-----+
| [手続き] |
| |
| (null? x) → boolean? |
| x: any/c |
+-----+
```

値が空リストかどうかを判定します。

```
> (null? '())
#true
> (null? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (range start end step) → list? |
| start: number |
| end: number |
| step: number |
+-----+
```

start から end まで step で進む数のリストを組み立てます。

```
> (range 0 10 2)
(cons 0 (cons 2 (cons 4 (cons 6 (cons 8 '())))))
```

```
+-----+
| [手続き] |
| |
| (remove x l) → list? |
| x: any/c |
| l: list? |
+-----+
```

与えられたリストと同様のリストを、与えられた項目の最初の出現を除いて組み立てます (値は equal? で比較)。

```

> x
(list 2 "hello" #true)
> (remove "hello" x)
(list 2 #true)
> hello-2
(list 2 "hello" #true "hello")
> (remove "hello" hello-2)
(list 2 #true "hello")

```

```

+-----+
| [手続き] |
| |
| (remove-all x l) → list? |
| x: any/c |
| l: list? |
+-----+

```

与えられたリストと同様のリストを、与えられた項目のすべての出現を除いて組み立てます（値はequal? で比較）。

```

> x
(list 2 "hello" #true)
> (remove-all "hello" x)
(list 2 #true)
> hello-2
(list 2 "hello" #true "hello")
> (remove-all "hello" hello-2)
(list 2 #true)

```

```

+-----+
| [手続き] |
| |
| (rest x) → any/c |
| x: cons? |
+-----+

```

空でないリストの残り（rest）を取り出します。

```

> x
(list 2 "hello" #true)
> (rest x)
(list "hello" #true)

```

```

+-----+
| [手続き] |
| |
| (reverse l) → list |
| l: list? |
+-----+

```

a reversed version of a list を作成します。

```
> x
(list 2 "hello" #true)
> (reverse x)
(list #true "hello" 2)

+-----+
| [手続き] |
| |
| (second x) → any/c |
| x: list? |
+-----+
```

the second item of a non-empty list を選択します。

```
> x
(list 2 "hello" #true)
> (second x)
"hello"

+-----+
| [手続き] |
| |
| (seventh x) → any/c |
| x: list? |
+-----+
```

the seventh item of a non-empty list を選択します。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (seventh v)
7

+-----+
| [手続き] |
| |
| (sixth x) → any/c |
| x: list? |
+-----+
```

the sixth item of a non-empty list を選択します。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (sixth v)
6

+-----+
| [手続き] |
| |
| (third x) → any/c |
| x: list? |
+-----+
```

the third item of a non-empty list を選択します。

```
> x
(list 2 "hello" #true)
> (third x)
#true
```

## 2.11 Posn

```
+-----+
| [手続き] |
| |
| (make-posn x y) → posn |
| x: any/c |
| y: any/c |
+-----+
```

2つの任意の値から posn を組み立てます。

```
> (make-posn 3 3)
(make-posn 3 3)
> (make-posn "hello" #true)
(make-posn "hello" #true)
```

```
+-----+
| [手続き] |
| |
| (posn-x p) → any/c |
| p: posn |
+-----+
```

the x component of a posn を取り出します。

```
> p
(make-posn 2 -3)
> (posn-x p)
2
```

```
+-----+
| [手続き] |
| |
| (posn-y p) → any/c |
| p: posn |
+-----+
```

the y component of a posn を取り出します。

```
> p
(make-posn 2 -3)
> (posn-y p)
-3
```

```
+-----+
| [手続き] |
| |
```



```

| |
| (posn? x) → boolean? |
| x: any/c |
+-----+

```

入力が posn かどうかを判定します。

```

> q
(make-posn "bye" 2)
> (posn? q)
#true
> (posn? 42)
#false

```

## 2.12 文字

```

+-----+
| [手続き] |
| |
| (char->integer c) → integer |
| c: char |
+-----+

```

Looks up 数 という corresponds へ given 文字 の中の ASCII table (if any).

```

> (char->integer #\a)
97
> (char->integer #\z)
122

```

```

+-----+
| [手続き] |
| |
| (char-alphabetic? c) → boolean? |
| c: char |
+-----+

```

文字がアルファベット文字を表すかを判定します。

```

> (char-alphabetic? #\Q)
#true

```

```

+-----+
| [手続き] |
| |
| (char-ci<=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+

```

文字が、大文字小文字を区別せず増加順かを判定します。

```
> (char-ci<=? #\b #\B)
#true
> (char<=? #\b #\B)
#false
```

```
+-----+
| [手続き] |
| |
| (char-ci<=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

文字が、大文字小文字を区別せず厳密に増加順かを判定します。

```
> (char-ci<=? #\B #\C)
#true
> (char<=? #\b #\B)
#false
```

```
+-----+
| [手続き] |
| |
| (char-ci<=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

two characters are equal in a case-insensitive mannerかどうかを判定します。

```
> (char-ci=? #\b #\B)
#true
```

```
+-----+
| [手続き] |
| |
| (char-ci>=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

文字が、大文字小文字を区別せず減少順に並んでいるかを判定します。

```
> (char-ci>=? #\b #\C)
#false
> (char>=? #\b #\C)
#true
```

```
+-----+
| [手続き] |
| |
| |
+-----+
```

```
| (char-ci>? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

文字が、大文字小文字を区別せず厳密に減少順に並んでいるかを判定します。

```
> (char-ci>? #\b #\B)
#false
> (char>? #\b #\B)
#true
```

```
+-----+
| [手続き] |
| |
| (char-downcase c) → char |
| c: char |
+-----+
```

対応する小文字を返します。

```
> (char-downcase #\T)
#\t
```

```
+-----+
| [手続き] |
| |
| (char-lower-case? c) → boolean? |
| c: char |
+-----+
```

文字が小文字かどうかを判定します。

```
> (char-lower-case? #\T)
#false
```

```
+-----+
| [手続き] |
| |
| (char-numeric? c) → boolean? |
| c: char |
+-----+
```

文字が数字を表すかを判定します。

```
> (char-numeric? #\9)
#true
```

```
+-----+
| [手続き] |
| |
| (char-upcase c) → char |
```

```
| c: char |
+-----+
```

対応する大文字を返します。

```
> (char-upcase #\t)
#\T
```

```
+-----+
| [手続き] |
| |
| (char-upper-case? c) → boolean? |
| c: char |
+-----+
```

文字が大文字かどうかを判定します。

```
> (char-upper-case? #\T)
#true
```

```
+-----+
| [手続き] |
| |
| (char-whitespace? c) → boolean? |
| c: char |
+-----+
```

文字が空白を表すかを判定します。

```
> (char-whitespace? #\tab)
#true
```

```
+-----+
| [手続き] |
| |
| (char<=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

文字が増加順かを判定します。

```
> (char<=? #\a #\a #\b)
#true
```

```
+-----+
| [手続き] |
| |
| (char<? x d e...) → boolean? |
| x: char |
| d: char |
| e: char |
+-----+
```

文字が厳密に増加順かを判定します。

```
> (char<? #\a #\b #\c)
#true
```

```
+-----+
| [手続き] |
| |
| (char=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

文字が等しいかを判定します。

```
> (char=? #\b #\a)
#false
```

```
+-----+
| [手続き] |
| |
| (char>=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

文字が減少順に並んでいるかを判定します。

```
> (char>=? #\b #\b #\a)
#true
```

```
+-----+
| [手続き] |
| |
| (char>? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

文字が厳密に減少順に並んでいるかを判定します。

```
> (char>? #\A #\z #\a)
#false
```

```
+-----+
| [手続き] |
| |
| (char? x) → boolean? |
| x: any/c |
+-----+
```

値が文字かどうかを判定します。

```
> (char? "a")
#false
> (char? #\a)
#true
```

## 2.13 文字列

```
+-----+
| [手続き] |
| |
| (explode s) → (listof string) |
| s: string |
+-----+
```

文字列を 1 文字ずつの文字列のリストに変換します。

```
> (explode "cat")
(list "c" "a" "t")
```

```
+-----+
| [手続き] |
| |
| (format f x...) → string |
| f: string |
| x: any/c |
+-----+
```

Formats 文字列, possibly embedding 値 s.

```
> (format "Dear Dr. ~a:" "Flatt")
"Dear Dr. Flatt:"
> (format "Dear Dr. ~s:" "Flatt")
"Dear Dr. \"Flatt\":"
```

```
+-----+
| [手続き] |
| |
| (implode l) → string |
| l: list? |
+-----+
```

1 文字ずつの文字列のリストを 1 つの文字列に連結します。

```
> (implode (cons "c" (cons "a" (cons "t" '()))))
"cat"
```

```
+-----+
| [手続き] |
| |
| (int->string i) → string |
| i: integer |
+-----+
```

an integer in [0,55295] or [57344 1114111] to a 1-letter string を変換します。

```
> (int->string 65)
"A"
```

```
+-----+
| [手続き] |
| |
| (list->string l) → string |
| l: list? |
+-----+
```

a s list of characters into a string を変換します。

```
> (list->string (cons #\c (cons #\a (cons #\t '()))))
"cat"
```

```
+-----+
| [手続き] |
| |
| (make-string i c) → string |
| i: natural? |
| c: char |
+-----+
```

a string of length i from c を生成します。

```
> (make-string 3 #\d)
"ddd"
```

```
+-----+
| [手続き] |
| |
| (replicate i s) → string |
| i: natural? |
| s: string |
+-----+
```

s を i 回繰り返します。

```
> (replicate 3 "h")
"hhh"
```

```
+-----+
| [手続き] |
| |
| (string c...) → string? |
| c: char |
+-----+
```

Builds 文字列 of the given 文字 s.

```
> (string #\d #\o #\g)
"dog"
```

```

+-----+
| [手続き] |
| |
| (string->int s) → integer |
| s: string |
+-----+

```

a 1-letter string to an integer in [0,55295] or [57344, 1114111] を変換します。

```

> (string->int "a")
97

```

```

+-----+
| [手続き] |
| |
| (string->list s) → (listof char) |
| s: string |
+-----+

```

a string into a list of characters を変換します。

```

> (string->list "hello")
(list #\h #\e #\l #\l #\o)

```

```

+-----+
| [手続き] |
| |
| (string->number s) → (union number #false) |
| s: string |
+-----+

```

a string into a number, produce false if impossible を変換します。

```

> (string->number "-2.03")
-2.03
> (string->number "1-2i")
1-2i

```

```

+-----+
| [手続き] |
| |
| (string->symbol s) → symbol |
| s: string |
+-----+

```

a string into a symbol を変換します。

```

> (string->symbol "hello")
'hello

```

```

+-----+
| [手続き] |
| |
| (string-alphabetic? s) → boolean? |
+-----+

```



```
| s: string |
+-----+
```

文字列中のすべての「文字」がアルファベットかを判定します。

```
> (string-alphabetic? "123")
#false
> (string-alphabetic? "cat")
#true
```

```
+-----+
| [手続き] |
| |
| (string-append s t z...) → string |
| s: string |
| t: string |
| z: string |
+-----+
```

複数の文字列の文字を連結します。

```
> (string-append "hello" " " "world" " " "good bye")
"hello world good bye"
```

```
+-----+
| [手続き] |
| |
| (string-ci<=? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

文字列が、大文字小文字を区別せず辞書式に増加順かを判定します。

```
> (string-ci<=? "hello" "WORLD")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci<? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

文字列が、大文字小文字を区別せず辞書式に厳密増加順かを判定します。

```
> (string-ci<? "hello" "WORLD")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci=? s t) → boolean? |
+-----+
```

```
| s: string |
| t: string |
+-----+
```

大文字小文字を問わず、すべての文字列が文字ごとに等しいかを判定します。

```
> (string-ci=? "hello" "Hello")
#true
```

```
+-----+
| [手続き] |
| (string-ci=? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

文字列が、大文字小文字を区別せず辞書式に減少順かを判定します。

```
> (string-ci>? "WORLD" "hello")
#true
```

```
+-----+
| [手続き] |
| (string-ci>? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

文字列が、大文字小文字を区別せず辞書式に厳密減少順かを判定します。

```
> (string-ci>? "WORLD" "hello")
#true
```

```
+-----+
| [手続き] |
| (string-contains-ci? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

大文字小文字を無視して、最初の文字列が2番目の文字列に現れるかを判定します。

```
> (string-contains-ci? "At" "caT")
#true
```

```
+-----+
| [手続き] |
| (string-contains? s t) → boolean? |
| s: string |
+-----+
```

```
| t: string |
+-----+
```

最初の文字列が 2 番目の文字列に文字どおり現れるかを判定します。

```
> (string-contains? "at" "cat")
#true
```

```
+-----+
| [手続き] |
| |
| (string-copy s) → string |
| s: string |
+-----+
```

Copies 文字列.

```
> (string-copy "hello")
"hello"
```

```
+-----+
| [手続き] |
| |
| (string-downcase s) → string |
| s: string |
+-----+
```

与えられた文字列と同様で、すべての「文字」を小文字にした文字列を返します。

```
> (string-downcase "CAT")
"cat"
> (string-downcase "cAt")
"cat"
```

```
+-----+
| [手続き] |
| |
| (string-ith s i) → 1string? |
| s: string |
| i: natural? |
+-----+
```

the ith 1-letter substring from s を取り出します。

```
> (string-ith "hello world" 1)
"e"
```

```
+-----+
| [手続き] |
| |
| (string-length s) → nat |
| s: string |
+-----+
```

判定する: length の 文字列.

```
> (string-length "hello world")
```

```
11
```

```
+-----+
| [手続き] |
| |
| (string-lower-case? s) → boolean? |
| s: string |
+-----+
```

文字列中のすべての「文字」が小文字かを判定します。

```
> (string-lower-case? "CAT")
```

```
#false
```

```
+-----+
| [手続き] |
| |
| (string-numeric? s) → boolean? |
| s: string |
+-----+
```

文字列中のすべての「文字」が数字かを判定します。

```
> (string-numeric? "123")
```

```
#true
```

```
> (string-numeric? "1-2i")
```

```
#false
```

```
+-----+
| [手続き] |
| |
| (string-ref s i) → char |
| s: string |
| i: natural? |
+-----+
```

the ith character from s を取り出します。

```
> (string-ref "cat" 2)
```

```
#\t
```

```
+-----+
| [手続き] |
| |
| (string-upcase s) → string |
| s: string |
+-----+
```

与えられた文字列と同様で、すべての「文字」を大文字にした文字列を返します。

```
> (string-upcase "cat")
```

```
"CAT"
```

```
> (string-upcase "cAt")
"CAT"
```

```
+-----+
| [手続き] |
| |
| (string-upper-case? s) → boolean? |
| s: string |
+-----+
```

文字列中のすべての「文字」が大文字かを判定します。

```
> (string-upper-case? "CAT")
#true
```

```
+-----+
| [手続き] |
| |
| (string-whitespace? s) → boolean? |
| s: string |
+-----+
```

文字列中のすべての「文字」が空白かを判定します。

```
> (string-whitespace? (string-append " " (string #\tab #\newline #\return)))
#true
```

```
+-----+
| [手続き] |
| |
| (string<=? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

文字列が辞書式に増加順かを判定します。

```
> (string<=? "hello" "hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string<? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

文字列が辞書式に厳密増加順かを判定します。

```
> (string<? "hello" "world")
#true
```

```
+-----+
| [手続き] |
| |
```

```

| |
| (string=? s t) → boolean? |
| s: string |
| t: string |
+-----+

```

すべての文字列が文字ごとに等しいかを判定します。

```

> (string=? "hello" "world")
#false
> (string=? "bye" "bye")
#true

```

```

+-----+
| [手続き] |
| |
| (string>=? s t) → boolean? |
| s: string |
| t: string |
+-----+

```

文字列が辞書式に減少順かを判定します。

```

> (string>=? "world" "hello")
#true

```

```

+-----+
| [手続き] |
| |
| (string>? s t) → boolean? |
| s: string |
| t: string |
+-----+

```

文字列が辞書式に厳密減少順かを判定します。

```

> (string>? "world" "hello")
#true

```

```

+-----+
| [手続き] |
| |
| (string? x) → boolean? |
| x: any/c |
+-----+

```

a value is a string かどうかを判定します。

```

> (string? "hello world")
#true
> (string? 42)
#false

```

```

+-----+
| [手続き] |
| |
| (substring s i j) → string |
| s: string |
| i: natural? |
| j: natural? |
+-----+

```

the substring starting at *i* up to *j* (or the end if *j* is not provided) を取り出します。

```

> (substring "hello world" 1 5)
"ello"
> (substring "hello world" 1 8)
"ello wo"
> (substring "hello world" 4)
"o world"

```

## 2.14 画像

```

+-----+
| [手続き] |
| |
| (image=? i j) → boolean? |
| i: image |
| j: image |
+-----+

```

2つの画像が等しいかを判定します。

```

> c1
[image:pict_4.png]
> (image=? (circle 5 "solid" "green") c1)
#false
> (image=? (circle 10 "solid" "green") c1)
#true

```

```

+-----+
| [手続き] |
| |
| (image? x) → boolean? |
| x: any/c |
+-----+

```

値が画像かどうかを判定します。

```

> c1
[image:pict_5.png]
> (image? c1)
#true

```

## 2.15 その他

```
+-----+
| [手続き] |
| |
| (≈ x y eps) → boolean? |
| x: number |
| y: number |
| eps: non-negative-real |
+-----+
```

whether x and y are within eps of either other を検査します。

```
> (≈ 1.01 1.0 0.1)
#true
> (≈ 1.01 1.5 0.1)
#false
```

```
+-----+
| [値] |
| |
| eof: eof-object? |
+-----+
```

ファイルの終わりを表す値です：

```
> eof
#<eof>
```

```
+-----+
| [手続き] |
| |
| (eof-object? x) → boolean? |
| x: any/c |
+-----+
```

some value is the end-of-file value かどうかを判定します。

```
> (eof-object? eof)
#true
> (eof-object? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (eq? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+
```

two values are equivalent from the computer's perspective (intensional) かどうかを判定します。



```
> (eq? (cons 1 '()) (cons 1 '()))
#false
> one
(list 1)
> (eq? one one)
#true
```

```
+-----+
| [手続き] |
| |
| (equal? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+
```

two values are structurally equal where basic values are compared with the eqv? predicateかどうかを判定します。

```
> (equal? (make-posn 1 2) (make-posn (- 2 1) (+ 1 1)))
#true
```

```
+-----+
| [手続き] |
| |
| (equal~? x y z) → boolean? |
| x: any/c |
| y: any/c |
| z: non-negative-real |
+-----+
```

x and y like equal? but uses =~ in the case of numbers を比較します。

```
> (equal~? (make-posn 1.01 1.0) (make-posn 1.01 0.99) 0.2)
#true
```

```
+-----+
| [手続き] |
| |
| (eqv? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+
```

two values are equivalent from the perspective of all functions that can be applied to it (extensional)かどうかを判定します。

```
> (eqv? (cons 1 '()) (cons 1 '()))
#false
> one
(list 1)
> (eqv? one one)
#true
```

```

+-----+
| [手続き] |
| |
| (error x...) → void? |
| x: any/c |
+-----+

```

Signals an error, combining the given values into an error message. If any of the values' printed representations is too long, it is truncated and “...” is put into the string. If the first value is a symbol, it is suffixed with a colon and the result pre-pended on to the error message.

```

> zero
0
> (if (= zero 0) (error "can't divide by 0") (/ 1 zero))
can't divide by 0

```

```

+-----+
| [手続き] |
| |
| (exit) → void |
+-----+

```

Evaluating (exit) terminates running プログラム.

```

+-----+
| [手続き] |
| |
| (identity x) → any/c |
| x: any/c |
+-----+

```

x を返します。

```

> (identity 42)
42
> (identity c1)
[image:pict_6.png]
> (identity "hello")
"hello"

```

```

+-----+
| [手続き] |
| |
| (struct? x) → boolean? |
| x: any/c |
+-----+

```

some value is a structure かどうかを判定します。

```

> (struct? (make-posn 1 2))
#true

```

```
> (struct? 43)
#false
```

## 2.16 シグネチャ

```
+-----+
| [値] |
| |
| Any: signature? |
+-----+
```

any value のシグネチャ。

```
+-----+
| [値] |
| |
| Boolean: signature? |
+-----+
```

booleans のシグネチャ。

```
+-----+
| [値] |
| |
| Char: signature? |
+-----+
```

characters のシグネチャ。

```
+-----+
| [手続き] |
| |
| (ConsOf first-sig rest-sig) → signature? |
| first-sig: signature? |
| rest-sig: signature? |
+-----+
```

a cons pair のシグネチャ。

```
+-----+
| [値] |
| |
| EmptyList: signature? |
+-----+
```

the empty list のシグネチャ。

```
+-----+
| [値] |
| |
| False: signature? |
+-----+
```

just false のシグネチャ。

```

+-----+
| [値] |
| Integer: signature? |
+-----+

```

integers のシグネチャ。

```

+-----+
| [値] |
| Natural: signature? |
+-----+

```

natural numbers のシグネチャ。

```

+-----+
| [値] |
| Number: signature? |
+-----+

```

arbitrary numbers のシグネチャ。

```

+-----+
| [値] |
| Rational: signature? |
+-----+

```

rational numbers のシグネチャ。

```

+-----+
| [値] |
| Real: signature? |
+-----+

```

real numbers のシグネチャ。

```

+-----+
| [値] |
| String: signature? |
+-----+

```

strings のシグネチャ。

```

+-----+
| [値] |
| Symbol: signature? |
+-----+

```

symbols のシグネチャ。

```
+-----+
| [値] |
| |
| True: signature? |
+-----+
```

just true のシグネチャ。

## 付録 B-3: Intermediate Student (#lang htdp/isl)

原本: extracted/appendix/htdp-langs/original\_markdown\_03\_intermediate.md

初学者向けに、説明文を日本語へ翻訳しています。コード・シグネチャ・実行例は原文のまま保持します。

### 3 Intermediate Student (中級学生言語)

文法の記法では、 $X\cdots$  (太字の点) という書き方で、 $X$  が任意の回数 (0 回、1 回、またはそれ以上) 現れてよいことを示します。別に、文法はテンプレートで使う識別子として  $\cdots$  も定義します。

program	=	def-or-expr...
def-or-expr	=	definition expr test-case library-require
definition	=	(define (name variable variable..... (define name expr) (define name (lambda (variable vari... (define-struct name (name...))
expr	=	(local [definition...] expr) (letrec ([name expr-for-let]...) e... (let ([name expr-for-let]...) expr) (let* ([name expr-for-let]...) exp... (name expr expr...) (cond [expr expr]... [expr expr]) (cond [expr expr]... [else expr]) (if expr expr expr) (and expr expr expr...) (or expr expr expr...) (time expr) name 'quoted 'quasiquoted '() number boolean string character (signature signature-form)
expr-for-let	=	(lambda (variable variable...) exp... expr
signature-declaration	=	(: name signature-form)
signature-form	=	(enum expr...) (mixed signature-form...) (signature-form... -> signature-fo...

			(ListOf signature-form)
			signature-variable
			expr
signature-variable	=		%name
quoted	=		name
			number
			string
			character
			(quoted...)
			'quoted
			'quoted
			`,`quoted
			`,`@`quoted
quasiquoted	=		name
			number
			string
			character
			(quasiquoted...)
			'quasiquoted
			'quasiquoted
			`,`expr
			`,`@`expr
test-case	=		(check-expect expr expr)
			(check-random expr expr)
			(check-within expr expr expr)
			(check-member-of expr expr...)
			(check-range expr expr expr)
			(check-satisfied expr expr)
			(check-error expr expr)
			(check-error expr)
library-require	=		(require string)
			(require (lib string string...))
			(require (planet string package))
package	=		(string string number number)

名前 (name) または変数 (variable) は、空白や次の文字を含まない文字の並びです。

“; ‘ ` () [] {}”; #

数 (number) とは、123、3/2、5.5 のような数です。

真偽値 (boolean) は #true または #false のいずれかです。

#true 定数の別表記は #t、true、#T です。同様に #f、false、#F も #false として認識されます。

シンボル (symbol) は、クオート文字に続く名前です。シンボルは 42、`()、#false などと同じく値です。

文字列 (string) は、一對の ” で囲まれた文字の並びです。シンボルと違い、文字列は文字に分割したり、さまざまな関数で操作したりできます。たとえば “abcdef”、“This is a string”、および “This is a string with " inside” はすべて文字列です。

文字 (character) は # で始まり、その文字の名前を持ちます。たとえば #、## は文字です。

関数呼び出しでは、開き括弧の直後に現れる関数は、define や define-struct で定義した関数、またはあらかじめ定義された関数のいずれかです。

### 3.1 あらかじめ定義された変数

```
+-----+
| [値] |
| |
| empty: empty? |
+-----+
```

空リスト。

```
+-----+
| [値] |
| |
| true: boolean? |
+-----+
```

#true の値。

```
+-----+
| [値] |
| |
| false: boolean? |
+-----+
```

#false の値。

### 3.2 テンプレート変数

```
+-----+
| [構文] |
| |
| .. |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
| ... |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。



```
+-----+
| [構文] |
| |
| |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
| |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
| |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

### 3.3 中級の構文

```
+-----+
| [構文] |
| |
| (local [definition...] expression) |
+-----+
```

Groups related 定義 s のための use の中の 式. Each 定義 できる be either define または define-struct.

When evaluating local, each definition is evaluated in order, and finally the body expression is evaluated. Only the expressions within the local (including the right-hand-sides of the definitions and the expression) may refer to the names defined by the definitions. If a name defined in the local is the same as a top-level binding, the inner one “shadows” the outer one. That is, inside the local, any references to that name refer to the inner one.

```
+-----+
| [構文] |
| |
| (letrec ([name expr-for-let]...) expression) |
+-----+
```

local と同様ですが、with a simpler syntax. Each name defines a variable (or a function) with the value of the corresponding expr-for-let. If expr-for-let is a lambda, letrec defines a function, otherwise it defines a variable.

```
+-----+
| [構文] |
| |
| (let* ([name expr-for-let]...) expression) |
+-----+
```

letrec と同様ですが、each name can only be used in expression, and in expr-for-lets occurring after that name。

```
+-----+
| [構文] |
| |
| (let ([name expr-for-let]...) expression) |
+-----+
```

letrec と同様ですが、the defined names can be used only in the last expression, not the expr-for-lets next to the names。

```
+-----+
| [構文] |
| |
| (time expression) |
+-----+
```

Measures the time taken to evaluate expression. After evaluating expression, time prints out the time taken by the evaluation (including real time, time taken by the CPU, and the time spent collecting free memory). The value of time is the same as that of expression.

### 3.4 共通の構文

following syntaxes behave same の中の Intermediate level として they did の中の Beginning Student とともに List Abbreviations level.

```
+-----+
| [構文] |
| |
| 'name |
+-----+
```

A quoted name は シンボル. A quoted part は abbreviation のための nested リスト s.

通常、この quotation は written とともに ‘, like’(applebanana), but it できる also be written とともに quote, like (quote(applebanana)).

```
+-----+
| [構文] |
| |
| 'name |
+-----+
```

同様: quote, but also allows escaping へ 式 “unquotes.”

通常、quasi-quotations は written とともに backquote, `,` like `(apple,(+12))`, but they できる also be written とともに quasiquote, like `(quasiquote(apple,(+12)))`.

```
+-----+
| [構文] |
| |
| ,expression |
+-----+
```

のもとで single quasiquote, `,`式 escapes から quote へ include evaluated 式 whose result は inserted へ abbreviated リスト.

のもとで multiple quasiquotes, `,`式 は 実数 ly literal `,`式, decrementing quasiquote count によって one のための 式.

通常、unquote は written with `,,` but it できる also be written とともに unquote.

```
+-----+
| [構文] |
| |
| ,@expression |
+-----+
```

のもとで single quasiquote, `,@`式 escapes から quote へ include evaluated 式 whose result は リスト へ splice へ abbreviated リスト.

のもとで multiple quasiquotes, splicing unquote は like unquote; という is, it decrements quasiquote count によって one.

通常、splicing unquote は written with `,,` but it できる also be written とともに unquote-splicing.

```
+-----+
| [構文] |
| |
| (define (name variable variable...) expression) |
+-----+
```

name という名前の関数を定義します。expression は関数本体です。関数が呼ばれると、引数の値が変数の代わりに本体へ挿入されます。関数は、その新しい式の値を返します。

関数名は、他の関数や変数と同じであってはなりません。

```
+-----+
| [構文] |
| |
| (define name expression) |
+-----+
```

expression の値で name という変数を定義します。変数名は他の関数や変数と同じであってはならず、name 自身が expression に現れてはなりません。

```
+-----+
| [構文] |
| |
| (define name (lambda (variable variable...) expression)) |
+-----+
```

An alternate way へ defining 関数 s. name は name の 関数, であり できない be same と  
して という の another 関数 または 変数.

A lambda できない be used outside の この alternate syntax.

```
+-----+
| [構文] |
| |
| (define-struct structure-name (field-name...)) |
+-----+
```

Defines new 構造体 called 構造体-name. 構造体's fields は named によって field-  
names. のあと define-struct, following new 関数 s は available:

- make-structure-name: takes a number of arguments equal to the number of fields  
in the structure, and creates a new instance of that structure.
- structure-name-field-name: takes an instance of the structure and returns the value  
in the field named by field-name.
- structure-name?: takes any value, and returns #true if the value is an instance of the  
structure.

name の new 関数 s introduced によって define-struct ~してはならない be same とし  
て という の other 関数 s または 変数 s, そうでなければ define-struct reports エラー.

```
+-----+
| [構文] |
| |
| (name expression expression...) |
+-----+
```

Calls 関数 named name. 値 の call は 値 の name's body のとき every one の 関数's 変  
数 s は replaced によって 値 s の corresponding 式 s.

関数 named name must defined before it できる be called. 数 の 引数 式 s でなければな  
らない same として 数 の 引数 s expected によって 関数.

```
+-----+
| [構文] |
| |
| (cond [question-expression answer-expression]...) |
| (cond [question-expression answer-expression]... [else answer-expression]) |
| (cond [question-expression answer-expression] |
| ... |
| [else answer-expression]) |
+-----+
```

```

| ``racket
| (cond [question-expression answer-expression]
| ...
| [else answer-expression])
| ...
+-----+

```

Chooses 節 based 上の ある 条件. cond finds first question-式 という 評価結果は #true, then 評価する corresponding answer-式.

If none of the question-expressions evaluates to #true, cond's value is the answer-expression of the else clause. If there is no else, cond reports an error. If the result of a question-expression is neither #true nor #false, cond also reports an error.

else は cond の外では使えません。

```

+-----+
| [構文] |
| (if question-expression
| then-answer-expression
| else-answer-expression)
|
| ``racket
| (if question-expression
| then-answer-expression
| else-answer-expression)
| ...
+-----+

```

とき: 値 の question-式 は #true, もし 評価する then-answer-式. とき: test は #false, もし 評価する else-answer-式.

もし question-式 は neither #true nor #false, もし reports エラー.

```

+-----+
| [構文] |
| (and expression expression expression...) |
+-----+

```

Evaluates へ #true もし すべての 式 s は #true. もし 任意の 式 は #false, と 式 評価結果は #false (and 式 s へ right の という 式 は ない evaluated.)

もし 任意の の 式 s evaluate へ 値 other than #true または #false, と reports エラー.

```

+-----+
| [構文] |
|
+-----+

```

```
| (or expression expression expression...) |
+-----+
```

to #true as soon as one of the expressions is #true (and the expressions to the right of that expression are not evaluated.) If all of the expressions are #false, the or expression evaluates to #false を評価します。

もし任意の の式 s evaluate へ 値 other than #true または #false, または reports エラー.

```
+-----+
| [構文] |
| |
| (check-expect expression expected-expression) |
+-----+
```

that the first expression evaluates to the same value as the expected-expression を検査します。

```
(check-expect (fahrenheit->celsius 212) 100)
(check-expect (fahrenheit->celsius -40) -40)
```

```
(define (fahrenheit->celsius f)
 (* 5/9 (- f 32)))
```

A check-expect expression must be placed at the top-level of a student program. Also it may show up anywhere in the program, including ahead of the tested function definition. By placing check-expects there, a programmer conveys to a future reader the intention behind the program with working examples, thus making it often superfluous to read the function definition proper. Syntax errors in check-expect (and all check forms) are intentionally delayed to run time so that students can write tests without necessarily writing complete function headers.

It is an error for expr or expected-expr to produce an inexact number or a function value. As for inexact numbers, it is morally wrong to compare them for plain equality. Instead one tests whether they are both within a small interval; see check-within. As for functions (see Intermediate and up), it is provably impossible to compare functions.

```
+-----+
| [構文] |
| |
| (check-random expression expected-expression) |
+-----+
```

that the first expression evaluates to the same value as the expected-expression を検査します。

form supplies same 乱数-数 generator へ both parts. もし both parts request 乱数 数 s から same interval の中の same order, they receive same 乱数 数 s.

ここでは simple example の where check-乱数 は useful:

```

(define WIDTH 100)
(define HEIGHT (* 2 WIDTH))

(define-struct player (name x y))
; A Player is (make-player String Nat Nat)

; String -> Player

(check-random (create-randomly-placed-player "David Van Horn")
 (make-player "David Van Horn" (random WIDTH) (random HEIGHT)))

(define (create-randomly-placed-player name)
 (make-player name (random WIDTH) (random HEIGHT)))

```

Note how 乱数 is called 上の same 数 s の中の same order の中の both parts の check-乱数. もし two parts call 乱数 のための different intervals, they は likely へ fail:

```

; String -> Player

(check-random (create-randomly-placed-player "David Van Horn")
 (make-player "David Van Horn" (random WIDTH) (random HEIGHT)))

(define (create-randomly-placed-player name)
 (a-helper-function name (random HEIGHT)))

; String Number -> Player
(define (a-helper-function name height)
 (make-player name (random WIDTH) height))

```

Because 引数 へ a-helper-関数 は evaluated first, 乱数 は first called のための interval [0,HEIGHT) と then のための [0,WIDTH), という is, の中の different order than の中の preceding check-乱数.

それはエラー のための expr または expected-expr へ produce 関数 値 または in 正確 数; see note 上の check-expect のための details.

```

+-----+
| [構文] |
| |
| (check-satisfied expression predicate) |
+-----+

```

Checks という first 式 satisfies named 述語 (関数 の one 引数). Recall という “satisfies” means “the 関数 produces #true のための given 値.”

ここでは simple examples のための check-satisfied:

```

> (check-satisfied 1 odd?)
The test passed!

```

```
> (check-satisfied 1 even?)
Ran 1 test. 0 tests passed.
Check failures:
Actual value | 1 | does not satisfy even?.
at line 3, column 0
```

In general check-satisfied empowers プログラム designers へ use defined 関数 s へ formulate test suites:

```
; [cons Number [List-of Number]] -> Boolean
; a function for testing htdp-sort

(check-expect (sorted? (list 1 2 3)) #true)
(check-expect (sorted? (list 2 1 3)) #false)

(define (sorted? l)
 (cond
 [(empty? (rest l)) #true]
 [else (and (<= (first l) (second l)) (sorted? (rest l)))]))

; [List-of Number] -> [List-of Number]
; create a sorted version of the given list of numbers

(check-satisfied (htdp-sort (list 1 2 0 3)) sorted?)

(define (htdp-sort l)
 (cond
 [(empty? l) l]
 [else (insert (first l) (htdp-sort (rest l)))]))

; Number [List-of Number] -> [List-of Number]
; insert x into l at proper place
; assume l is arranged in ascending order
; the result is sorted in the same way
(define (insert x l)
 (cond
 [(empty? l) (list x)]
 [else (if (<= x (first l)) (cons x l) (cons (first l) (insert x (rest l))))])
```

And yes, results の htdp-sort satisfy sorted? 述語:

```
> (check-satisfied (htdp-sort (list 1 2 0 3)) sorted?)
+-----+
| [構文] |
| |
| (check-within expression expected-expression delta) |
+-----+
```

whether the value of the expression expression is structurally equal to the value produced by the expected-expression expression; every number in the first expression



must be within delta of the corresponding number in the second expression を検査します。

```
(define-struct roots (x sqrt))
; RT is [List-of (make-roots Number Number)]

(define (root-of a)
 (make-roots a (sqrt a)))

(define (roots-table xs)
 (cond
 [(empty? xs) '()]
 [else (cons (root-of (first xs)) (roots-table (rest xs))))])
```

Due への presence の in 正確 数 s の中の nested data, check-within は correct choice のための testing, と test succeeds もし delta は reasonably large:

例:

```
> (check-within (roots-table (list 1.0 2.0 3.0))
 roots 1.0 1.0) (list (make-roots 2 1.414) (make-roots 3 1.713))
0.1
The test passed!
```

対照的に、のとき delta は small, test fails:

例:

```
> (check-within (roots-table (list 2.0))
 1.414) (list (make-roots 2
#1e-5)
Ran 1 test.
0 tests passed.
Check failures:

Actual value | '((make-roots 2.0
1.4142135623730951)) | is not within 1e-5 of expected value | '((make-roots 2 1.414)) | .

at line 5, column 0
```

それはエラーのための式 s または expected-式 へ produce 関数 値; see note 上の check-expect のための details.

もし delta は ない 数, check-within reports エラー.

```
+-----+
| [構文] |
| (check-error expression expected-error-message) |
| (check-error expression) |
+-----+
```

that the expression reports an error, where the error messages matches the value of expected-error-message, if it is present を検査します。

ここでは typical beginner example という calls のための use の check-エラー:

```
(define sample-table
 '(("matthias" 10)
 ("matthew" 20)
 ("robby" -1)
 ("shriram" 18)))

; [List-of [list String Number]] String -> Number
; determine the number associated with s in table

(define (lookup table s)
 (cond
 [(empty? table) (error (string-append s " not found"))]
 [else (if (string=? (first (first table)) s)
 (second (first table))
 (lookup (rest table))))])
```

Consider following two examples の中の この context:

例:

```
> (check-expect (lookup sample-table "matthew") 20)
The test passed!
```

例:

```
> (check-error (lookup sample-table "kathi") "kathi not found")
The test passed!
```

```
+-----+
| [構文] |
| |
| (check-member-of expression expression expression...) |
+-----+
```

that the value of the first expression is that of one of the following expressions を検査します。

```
; [List-of X] -> X
; pick a random element from the given list l
(define (pick-one l)
 (list-ref l (random (length l))))
```

例:

```
> (check-member-of (pick-one '("a" "b" "c")) "a" "b" "c")
The test passed!
```

それはエラーのための任意の式  $s$  へ produce 関数 値; see note 上の check-expect のための details.

```
+-----+
| [構文] |
| |
| (check-range expression low-expression high-expression) |
+-----+
```

that the value of the first expression is a number in between the value of the low-expression and the high-expression, inclusive を検査します。

A check-range form は best used へ delimit possible results の 関数  $s$  という compute in 正確 数  $s$ :

```
(define EPSILON 0.001)

; [Real -> Real] Real -> Real
; what is the slope of f at x?
(define (differentiate f x)
 (slope f (- x EPSILON) (+ x EPSILON)))

; [Real -> Real] Real Real -> Real
(define (slope f left right)
 (/ (- (f right) (f left))
 2 EPSILON))
```

```
(check-range (differentiate sin 0) 0.99 1.0)
```

それはエラーのための式, low-式, または high-式 へ produce 関数 値; see note 上の check-expect のための details.

```
+-----+
| [構文] |
| |
| (require string) |
+-----+
```

Makes 定義  $s$  の モジュール specified によって 文字列 available の中の current モジュール (i.e., current file), where 文字列 refers へ file relative へ current file.

The string is constrained in several ways to avoid problems with different path conventions on different platforms: a / is a directory separator,. always means the current directory,.. always means the parent directory, path elements can use only a through z (uppercase or lowercase), 0 through 9, -, \_, and., and the string cannot be empty or contain a leading or trailing /.

```
+-----+
| [構文] |
| |
```

```
| (require module-name) |
+-----+
```

Accesses file の中の installed ライブラリ. ライブラリ name は 識別子 とともに same constraints として のための relative-path 文字列 (though without quotes), とともに additional constraint という it ～してはならない contain a..

```
+-----+
| [構文] |
| |
| (require (lib string string...)) |
+-----+
```

Accesses a file in an installed library, making its definitions available in the current module (i.e., the current file). The first string names the library file, and the remaining strings name the collection (and sub-collection, and so on) where the file is installed. Each string is constrained in the same way as for the (requirestring) form.

```
+-----+
| [構文] |
| |
| (require (planet string (string string number number))) |
+-----+
```

Accesses ライブラリ という は distributed 上の internet via PPlaneT server, making it 定義 s available の中の current モジュール (i.e., current file).

full grammar のための planet requires は given の中の Importing と Exporting: require と provide, but best place へ find examples の syntax は 上の the PPlaneT server, 中の description の specific package.

### 3.5 シグネチャ

Signatures do not have to be comment: They can also be part of the > code. When a signature is attached to a function, DrRacket will check > that program uses the function in accordance with the signature and > display signature violations along with the test results. A signature is a regular value, and is specified as a > signature form, a > special syntax that only works with: signature declarations > and inside signature expressions.

```
+-----+
| [構文] |
| |
| (: name signature-form) |
+-----+
```

これ attaches シグネチャ specified によって シグネチャ-form へ 定義 の name. There でなければならぬ 定義 の name somewhere の中の プログラム.

```
(: age Integer)
(define age 42)

(: area-of-square (Number -> Number))
(define (area-of-square len)
 (sqr len))
```

On running プログラム, Racket checks whether シグネチャ s attached with: actually match 値 の 変数. もし they don't, Racket reports シグネチャ violation along とともに test failures.

たとえば、この piece の code:

```
(: age Integer)
(define age "fortytwo")
```

Yields この output:

```
+-----+
| `1 signature violation.` |
+-----+
| `Signature violations:` |
| `got "fortytwo" at line 2, column 12, signature... |
+-----+
```

Note という シグネチャ violation does ない stop running プログラム.

```
+-----+
| [構文] |
| |
| (signature signature-form) |
+-----+
```

これ 返す シグネチャ described によって シグネチャ-form として 値.

### 3.5.1 シグネチャ形式

Any expression can be a signature form, in which case the signature is > the value returned by that expression. There are a few special > signature forms, however: In a signature form, any name that starts with a % is a > signature variable that stands for any signature depending on how > the signature is used. Example:

```
(: same (%a -> %a))

(define (same x) x)

+-----+
| [構文] |
| |
```

```
|
| (input-signature-form... -> output-signature-form) |
+-----+
```

これ シグネチャ form describes 関数 とともに inputs described によって input-シグネチャ-forms と output described によって output-シグネチャ-form.

```
+-----+
| [構文] |
| |
| (enum expr...) |
+-----+
```

これ シグネチャ describes enumeration の 値 s returned によって exprs.

例:

```
(: cute? ((enum "cat" "snake") -> Boolean))
```

```
(define (cute? pet)
 (cond
 [(string=? pet "cat") #t]
 [(string=? pet "snake") #f]))
```

```
+-----+
| [構文] |
| |
| (mixed signature-form...) |
+-----+
```

これ シグネチャ describes mixed data, i.e. itemization where 各 の cases has シグネチャ described によって シグネチャ-form.

例:

```
(define SIGS (signature (mixed Aim Fired)))
```

```
+-----+
| [構文] |
| |
| (ListOf signature-form) |
+-----+
```

これ シグネチャ describes リスト where elements は described によって シグネチャ-form.

```
+-----+
| [構文] |
| |
| (predicate expression) |
+-----+
```

これシグネチャ describes 値 s through 述語: 式 must evaluate へ 関数 の one 引数 とい  
う 返す 真偽値. シグネチャ matches すべての 値 s のための であり 述語 返す #true.

### 3.5.2 構造体シグネチャ

A define-struct form defines two additional names that can be > used in signatures. For a struct called struct, these > are Struct and StructOf. Note that > these names are capitalized. In particular, a struct called > Struct, will also define Struct and > StructOf. Moreover, when forming the additional > names, hyphens are removed, and each letter following a hyphen is > capitalized - so a struct called foo-bar will define > FooBar and FooBarOf. Struct is a signature that describes struct values > from this structure type. StructOf is a function > that takes as input a signature for each field. It returns a > signature describing values of this structure type, additionally > describing the values of the fields of the value.

```
(define-struct pair [fst snd])

(: add-pair ((PairOf Number Number) -> Number))
(define (add-pair p)
 (+ (pair-fst p) (pair-snd p)))
```

### 3.6 あらかじめ定義された関数

remaining subsections リスト それらの 関数 s という は built へ プログラム ming language. All other 関数 s は imported から teachpack または でなければならない defined の中の プログラム.

### 3.7 数: 整数・有理数・実数・複素数・正確数・非正確数

```
+-----+
| [手続き] |
| |
| (- x y...) -> number |
| x: number |
| y: number |
+-----+
```

Subtracts second (and following) 数(s) から first; negates 数 もし there は only one 引数.

```
> (- 5)
-5
> (- 5 3)
2
> (- 5 3 1)
1
```

```
+-----+
| [手続き] |
| |
| |
+-----+
```

```
| (< x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

two or more (real) numbers for less-than を比較します。

```
> (< 42 2/5)
#false
```

```
+-----+
| [手続き] |
| (<= x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

two or more (real) numbers for less-than or equality を比較します。

```
> (<= 42 2/5)
#false
```

```
+-----+
| [手続き] |
| (> x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

two or more (real) numbers for greater-than を比較します。

```
> (> 42 2/5)
#true
```

```
+-----+
| [手続き] |
| (>= x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

two or more (real) numbers for greater-than or equality を比較します。

```
> (>= 42 42)
#true
```



```

+-----+
| [手続き] |
| |
| (abs x) → real |
| x: real |
+-----+

```

判定する: absolute 値 の 実数 数.

```

> (abs -12)
12

```

```

+-----+
| [手続き] |
| |
| (acos x) → number |
| x: number |
+-----+

```

the arccosine (inverse of cos) of a number を計算します。

```

> (acos 0)
#i1.5707963267948966

```

```

+-----+
| [手続き] |
| |
| (add1 x) → number |
| x: number |
+-----+

```

Increments given 数.

```

> (add1 2)
3

```

```

+-----+
| [手続き] |
| |
| (angle x) → real |
| x: number |
+-----+

```

the angle from a complex number を取り出します。

```

> (angle (make-polar 3 4))
#i-2.2831853071795867

```

```

+-----+
| [手続き] |
| |
| (asin x) → number |
| x: number |
+-----+

```

the arcsine (inverse of sin) of a number を計算します。

```
> (asin 0)
0
```

```
+-----+
| [手続き] |
| |
| (atan x) → number |
| x: number |
+-----+
```

計算する: arctangent の given 数:

```
> (atan 0)
0
> (atan 0.5)
#i0.4636476090008061
```

Also comes in a two-argument version where (atany x) computes (atan(/yx)) but the signs of y and x determine the quadrant of the result and the result tends to be more accurate than that of the 1-argument version in borderline cases:

```
> (atan 3 4)
#i0.6435011087932844
> (atan -2 -1)
#i-2.0344439357957027
```

```
+-----+
| [手続き] |
| |
| (ceiling x) → integer |
| x: real |
+-----+
```

判定する: closest 整数 (正確 または in 正確) above 実数 数. 参照: round.

```
> (ceiling 12.3)
#i13.0
```

```
+-----+
| [手続き] |
| |
| (complex? x) → boolean? |
| x: any/c |
+-----+
```

some value is complex かどうかを判定します。

```
> (complex? 1-2i)
#true
```

```
+-----+
| [手続き] |
| |
```

```
|
| (conjugate x) → number |
| x: number |
+-----+
```

Flips sign of the imaginary part of a complex number.

```
> (conjugate 3+4i)
3-4i
> (conjugate -2-5i)
-2+5i
> (conjugate (make-polar 3 4))
#i-1.960930862590836+2.2704074859237844i
```

```
+-----+
| [手続き] |
| |
| (cos x) → number |
| x: number |
+-----+
```

the cosine of a number (radians) を計算します。

```
> (cos pi)
#i-1.0
```

```
+-----+
| [手続き] |
| |
| (cosh x) → number |
| x: number |
+-----+
```

the hyperbolic cosine of a number を計算します。

```
> (cosh 10)
#i11013.232920103324
```

```
+-----+
| [手続き] |
| |
| (current-seconds) → integer |
+-----+
```

判定する: current time の中の seconds elapsed (since platform-specific starting date).

```
> (current-seconds)
1779843334
```

```
+-----+
| [手続き] |
| |
| (denominator x) → integer |
+-----+
```

```
| x: rational? |
+-----+
```

the denominator of a rational を計算します。

```
> (denominator 2/3)
3
```

```
+-----+
| [値] |
| e: real |
+-----+
```

Euler's 数.

```
> e
#i2.718281828459045
```

```
+-----+
| [手続き] |
| (even? x) → boolean? |
| x: integer |
+-----+
```

some integer (exact or inexact) is even or not かどうかを判定します。

```
> (even? 2)
#true
```

```
+-----+
| [手続き] |
| (exact->inexact x) → number |
| x: number |
+-----+
```

an exact number to an inexact one を変換します。

```
> (exact->inexact 12)
#i12.0
```

```
+-----+
| [手続き] |
| (exact? x) → boolean? |
| x: number |
+-----+
```

some number is exact かどうかを判定します。

```
> (exact? (sqrt 2))
#false
```

```

+-----+
| [手続き] |
| |
| (exp x) → number |
| x: number |
+-----+

```

判定する:  $e$  raised  $\wedge$  数.

```

> (exp -2)
#i0.1353352832366127

```

```

+-----+
| [手続き] |
| |
| (expt x y) → number |
| x: number |
| y: number |
+-----+

```

the power of the first to the second number, which is to say, exponentiation を計算します。

```

> (expt 16 1/2)
4
> (expt 3 -4)
1/81

```

```

+-----+
| [手続き] |
| |
| (floor x) → integer |
| x: real |
+-----+

```

判定する: closest 整数 (正確 または in 正確) below 実数 数. 参照: round.

```

> (floor 12.3)
#i12.0

```

```

+-----+
| [手続き] |
| |
| (gcd x y...) → integer |
| x: integer |
| y: integer |
+-----+

```

判定する: greatest common divisor の two 整数 s (正確 または in 正確).

```

> (gcd 6 12 8)
2

```

```

+-----+
| [手続き] |
| |
| (imag-part x) → real |
| x: number |
+-----+

```

the imaginary part from a complex number を取り出します。

```

> (imag-part 3+4i)
4

```

```

+-----+
| [手続き] |
| |
| (inexact->exact x) → number |
| x: number |
+-----+

```

Approximates in 正確 数 によって 正確 one.

```

> (inexact->exact 12.0)
12

```

```

+-----+
| [手続き] |
| |
| (inexact? x) → boolean? |
| x: number |
+-----+

```

some number is inexact かどうかを判定します。

```

> (inexact? 1-2i)
#false

```

```

+-----+
| [手続き] |
| |
| (integer->char x) → char |
| x: exact-integer? |
+-----+

```

Looks up 文字 という corresponds へ given 正確 整数 の中の ASCII table (if any).

```

> (integer->char 42)
#*

```

```

+-----+
| [手続き] |
| |
| (integer-sqrt x) → complex |
| x: integer |
+-----+

```

the integer or imaginary-integer square root of an integer を計算します。

```
> (integer-sqrt 11)
```

```
3
```

```
> (integer-sqrt -11)
```

```
0+3i
```

```
+-----+
| [手続き] |
| |
| (integer? x) → boolean? |
| x: any/c |
+-----+
```

some value is an integer (exact or inexact) かどうかを判定します。

```
> (integer? (sqrt 2))
```

```
#false
```

```
+-----+
| [手続き] |
| |
| (lcm x y...) → integer |
| x: integer |
| y: integer |
+-----+
```

判定する: least common multiple の two 整数 s (正確 または in 正確).

```
> (lcm 6 12 8)
```

```
24
```

```
+-----+
| [手続き] |
| |
| (log x) → number |
| x: number |
+-----+
```

判定する: base-e logarithm の 数.

```
> (log 12)
```

```
#i2.4849066497880004
```

```
+-----+
| [手続き] |
| |
| (magnitude x) → real |
| x: number |
+-----+
```

判定する: magnitude の 複素数 数.

```
> (magnitude (make-polar 3 4))
```

```
#i2.9999999999999996
```

```

+-----+
| [手続き] |
| |
| (make-polar x y) → number |
| x: real |
| y: real |
+-----+

```

a complex from a magnitude and angle を作成します。

```

> (make-polar 3 4)
#i-1.960930862590836-2.2704074859237844i

```

```

+-----+
| [手続き] |
| |
| (make-rectangular x y) → number |
| x: real |
| y: real |
+-----+

```

a complex from a real and an imaginary part を作成します。

```

> (make-rectangular 3 4)
3+4i

```

```

+-----+
| [手続き] |
| |
| (max x y...) → real |
| x: real |
| y: real |
+-----+

```

判定する: largest 数—aka, maximum.

```

> (max 3 2 8 7 2 9 0)
9

```

```

+-----+
| [手続き] |
| |
| (min x y...) → real |
| x: real |
| y: real |
+-----+

```

判定する: smallest 数—aka, minimum.

```

> (min 3 2 8 7 2 9 0)
0

```

```

+-----+
| [手続き] |
| |

```



```

|
| (modulo x y) → integer |
| x: integer |
| y: integer |
+-----+

```

Finds remainder of the division of the first number by the second:

```
> (modulo 9 2)
```

```
1
```

```
> (modulo 3 -4)
```

```
-1
```

```

+-----+
| [手続き] |
| |
| (negative? x) → boolean? |
| x: real |
+-----+

```

some real number is strictly smaller than zeroかどうかを判定します。

```
> (negative? -2)
```

```
#true
```

```

+-----+
| [手続き] |
| |
| (number->string x) → string |
| x: number |
+-----+

```

a number to a string を変換します。

```
> (number->string 42)
```

```
"42"
```

```

+-----+
| [手続き] |
| |
| (number->string-digits x p) → string |
| x: number |
| p: posint |
+-----+

```

a number x to a string with the specified number of digits を変換します。

```
> (number->string-digits 0.9 2)
```

```
"0.9"
```

```
> (number->string-digits pi 4)
```

```
"3.1416"
```

```

+-----+
| [手続き] |

```

```

|
| (number? n) → boolean? |
| n: any/c |
+-----+

```

～かどうかを判定する: ある 値 は 数:

```

> (number? "hello world")
#false
> (number? 42)
#true

```

```

+-----+
| [手続き] |
| |
| (numerator x) → integer |
| x: rational? |
+-----+

```

the numerator of a rational を計算します。

```

> (numerator 2/3)
2

```

```

+-----+
| [手続き] |
| |
| (odd? x) → boolean? |
| x: integer |
+-----+

```

some integer (exact or inexact) is odd or not かどうかを判定します。

```

> (odd? 2)
#false

```

```

+-----+
| [値] |
| |
| pi: real |
+-----+

```

ratio の circle's circumference へ その diameter.

```

> pi
#i3.141592653589793

```

```

+-----+
| [手続き] |
| |
| (positive? x) → boolean? |
| x: real |
+-----+

```

some real number is strictly larger than zero かどうかを判定します。

```
> (positive? -2)
#false
```

```
+-----+
| [手続き] |
| |
| (quotient x y) → integer |
| x: integer |
| y: integer |
+-----+
```

Divides first 整数—also called dividend—by second—known として divisor—to obtain quotient.

```
> (quotient 9 2)
4
> (quotient 3 4)
0
```

```
+-----+
| [手続き] |
| |
| (random x) → natural?
| x: (and/c natural? positive?) |
+-----+
```

Generates 乱数 natural 数 less than ある given 正確 natural.

```
> (random 42)
30
```

```
+-----+
| [手続き] |
| |
| (rational? x) → boolean? |
| x: any/c |
+-----+
```

some value is a rational number かどうかを判定します。

```
> (rational? 1)
#true
> (rational? -2.349)
#true
> (rational? #i1.23456789)
#true
> (rational? (sqrt -1))
#false
> (rational? pi)
#true
> (rational? e)
#true
```

```
> (rational? 1-2i)
#false
```

As the interactions show, the teaching languages considers many more numbers as rationals than expected. In particular, pi is a rational number because it is only a finite approximation to the mathematical  $\pi$ . Think of rational? as a suggestion to think of these numbers as fractions.

```
+-----+
| [手続き] |
| |
| (real-part x) → real |
| x: number |
+-----+
```

the real part from a complex number を取り出します。

```
> (real-part 3+4i)
3
```

```
+-----+
| [手続き] |
| |
| (real? x) → boolean? |
| x: any/c |
+-----+
```

some value is a real numberかどうかを判定します。

```
> (real? 1-2i)
#false
```

```
+-----+
| [手続き] |
| |
| (remainder x y) → integer |
| x: integer |
| y: integer |
+-----+
```

判定する: remainder の dividing first によって second 整数 (正確 または in 正確).

```
> (remainder 9 2)
```

```
1
```

```
> (remainder 3 4)
```

```
3
```

```
+-----+
| [手続き] |
| |
| (round x) → integer |
| x: real |
+-----+
```

Rounds 実数 数へ整数 (roundsへevenへbreak ties). 参照: floor と ceiling.

```
> (round 12.3)
#i12.0
```

```
+-----+
| [手続き] |
| |
| (sgn x) → (union 1 #i1.0 0 #i0.0 -1 #i-1.0) |
| x: real |
+-----+
```

判定する: sign の 実数 数.

```
> (sgn -12)
-1
```

```
+-----+
| [手続き] |
| |
| (sin x) → number |
| x: number |
+-----+
```

the sine of a number (radians) を計算します。

```
> (sin pi)
#i1.2246467991473532e-16
```

```
+-----+
| [手続き] |
| |
| (sinh x) → number |
| x: number |
+-----+
```

the hyperbolic sine of a number を計算します。

```
> (sinh 10)
#i11013.232874703393
```

```
+-----+
| [手続き] |
| |
| (sqr x) → number |
| x: number |
+-----+
```

the square of a number を計算します。

```
> (sqr 8)
64
```

```
+-----+
| [手続き] |
+-----+
```

```

| |
| (sqrt x) → number |
| x: number |
+-----+

```

the square root of a number を計算します。

```

> (sqrt 9)
3
> (sqrt 2)
#i1.4142135623730951

```

```

+-----+
| [手続き] |
| |
| (sub1 x) → number |
| x: number |
+-----+

```

Decrements given 数.

```

> (sub1 2)
1

```

```

+-----+
| [手続き] |
| |
| (tan x) → number |
| x: number |
+-----+

```

the tangent of a number (radians) を計算します。

```

> (tan pi)
#i-1.2246467991473532e-16

```

```

+-----+
| [手続き] |
| |
| (zero? x) → boolean? |
| x: number |
+-----+

```

some number is zero or not かどうかを判定します。

```

> (zero? 2)
#false

```

### 3.8 真偽値

```

+-----+
| [手続き] |
| |
| (boolean->string x) → string |

```

```
| x: boolean? |
+-----+
```

Produces 文字列 のための given 真偽値

```
> (boolean->string #false)
"#false"
> (boolean->string #true)
"#true"
```

```
+-----+
| [手続き] |
| |
| (boolean=? x y) → boolean? |
| x: boolean? |
| y: boolean? |
+-----+
```

two booleans are equalかどうかを判定します。

```
> (boolean=? #true #false)
#false
```

```
+-----+
| [手続き] |
| |
| (boolean? x) → boolean? |
| x: any/c |
+-----+
```

値が真偽値かどうかを判定します。

```
> (boolean? 42)
#false
> (boolean? #false)
#true
```

```
+-----+
| [手続き] |
| |
| (false? x) → boolean? |
| x: any/c |
+-----+
```

a value is falseかどうかを判定します。

```
> (false? #false)
#true
```

```
+-----+
| [手続き] |
| |
| (not x) → boolean? |
+-----+
```

```
| x: boolean? |
+-----+
```

Negates 真偽値 値.

```
> (not #false)
#true
```

### 3.9 シンボル

```
+-----+
| [手続き] |
| |
| (symbol->string x) → string |
| x: symbol |
+-----+
```

a symbol to a string を変換します。

```
> (symbol->string 'c)
"c"
```

```
+-----+
| [手続き] |
| |
| (symbol=? x y) → boolean? |
| x: symbol |
| y: symbol |
+-----+
```

two symbols are equalかどうかを判定します。

```
> (symbol=? 'a 'b)
#false
```

```
+-----+
| [手続き] |
| |
| (symbol? x) → boolean? |
| x: any/c |
+-----+
```

値がシンボルかどうかを判定します。

```
> (symbol? 'a)
#true
```

### 3.10 リスト

```
+-----+
| [手続き] |
| |
| (append l...) → (listof any) |
| l: (listof any) |
+-----+
```



a single list from several, by concatenation of the items. In ISL and up: append also works when applied to one list or none を作成します。

```
> (append (cons 1 (cons 2 '())) (cons "a" (cons "b" '())))
(list 1 2 "a" "b")
> (append)
'()
```

```
+-----+
| [手続き] |
| |
| (assoc x l) → (union (listof any) #false) |
| x: any/c |
| l: (listof any) |
+-----+
```

the first pair on l whose first is equal? to x; otherwise it produces #false を生成します。

```
> (assoc "hello" '(("world" 2) ("hello" 3) ("good" 0)))
(list "hello" 3)
```

```
+-----+
| [手続き] |
| |
| (assq x l) → (union #false cons?) |
| x: any/c |
| l: list? |
+-----+
```

～かどうかを判定する: ある item は first item の pair の中の リスト の pairs. (それ compares items とともに eq?.)

```
> a
(list (list 'a 22) (list 'b 8) (list 'c 70))
> (assq 'b a)
(list 'b 8)
```

```
+-----+
| [手続き] |
| |
| (caaar x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (car(car(carx)))。

```
> w
(list (list (list (list "bye" 3) #true) 42)
> (caaar w)
(list "bye")
```

```
+-----+
| [手続き] |
```

```

| |
| (caadr x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(car(cdrx)))。

```

> (caadr (cons 1 (cons (cons 'a '()) (cons (cons 'd '()) '()))))
'a

```

```

+-----+
| [手続き] |
| |
| (caar x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(carx))。

```

> y
(list (list (list 1 2 3) #false "world"))
> (caar y)
(list 1 2 3)

```

```

+-----+
| [手続き] |
| |
| (cadar x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(cdr(carx)))。

```

> w
(list (list (list (list "bye") 3) #true) 42)
> (cadar w)
#true

```

```

+-----+
| [手続き] |
| |
| (caddr x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(cdr(cdr(cdrx))))。

```

> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (caddr v)
4

```

```

+-----+
| [手続き] |
| |

```

(caddr x) → any/c
x: list?

LISP 風の selector: (car(cdr(cdrx)))。

```
> x
(list 2 "hello" #true)
> (caddr x)
#true
```

[手続き]
(cadr x) → any/c
x: list?

LISP 風の selector: (car(cdrx))。

```
> x
(list 2 "hello" #true)
> (cadr x)
"hello"
```

[手続き]
(car x) → any/c
x: cons?

空でないリストの先頭要素を取り出します。

```
> x
(list 2 "hello" #true)
> (car x)
2
```

[手続き]
(cdaar x) → any/c
x: list?

LISP 風の selector: (cdr(car(carx)))。

```
> w
(list (list (list (list "bye") 3) #true) 42)
> (cdaar w)
(list 3)
```

```

+-----+
| [手続き] |
| |
| (cdadr x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (cdr(car(cdrx)))。

```

> (cdadr (list 1 (list 2 "a") 3))
(list "a")

```

```

+-----+
| [手続き] |
| |
| (cdar x) → list? |
| x: list? |
+-----+

```

LISP 風の selector: (cdr(carx))。

```

> y
(list (list (list 1 2 3) #false "world"))
> (cdar y)
(list #false "world")

```

```

+-----+
| [手続き] |
| |
| (cddar x) → any/c |
| x: list? |
+-----+

```

LISP-style selector: (cdr(cdr(carx)))

```

> w
(list (list (list (list "bye") 3) #true) 42)
> (cddar w)
'()

```

```

+-----+
| [手続き] |
| |
| (cdddr x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (cdr(cdr(cdrx)))。

```

> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (cdddr v)
(list 4 5 6 7 8 9 'A)

```

```

+-----+
| [手続き] |
| |
| (cddr x) → list? |
| x: list? |
+-----+

```

LISP 風の selector: (cdr(cdrx))。

```

> x
(list 2 "hello" #true)
> (cddr x)
(list #true)

```

```

+-----+
| [手続き] |
| |
| (cdr x) → any/c |
| x: cons? |
+-----+

```

空でないリストの残り (rest) を取り出します。

```

> x
(list 2 "hello" #true)
> (cdr x)
(list "hello" #true)

```

```

+-----+
| [手続き] |
| |
| (cons x y) → list? |
| x: any/c |
| y: list? |
+-----+

```

a list を構築します。

```

> (cons 1 '())
(cons 1 '())

```

```

+-----+
| [手続き] |
| |
| (cons? x) → boolean? |
| x: any/c |
+-----+

```

some value is a constructed list かどうかを判定します。

```

> (cons? (cons 1 '()))
#true
> (cons? 42)
#false

```

```

+-----+
| [手続き] |
| |
| (eighth x) → any/c |
| x: list? |
+-----+

```

the eighth item of a non-empty list を選択します。

```

> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (eighth v)
8

```

```

+-----+
| [手続き] |
| |
| (empty? x) → boolean? |
| x: any/c |
+-----+

```

値が空リストかどうかを判定します。

```

> (empty? '())
#true
> (empty? 42)
#false

```

```

+-----+
| [手続き] |
| |
| (fifth x) → any/c |
| x: list? |
+-----+

```

the fifth item of a non-empty list を選択します。

```

> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (fifth v)
5

```

```

+-----+
| [手続き] |
| |
| (first x) → any/c |
| x: cons? |
+-----+

```

空でないリストの先頭要素を取り出します。

```

> x
(list 2 "hello" #true)

```

```
> (first x)
```

```
2
```

```
+-----+
| [手続き] |
| |
| (fourth x) → any/c |
| x: list? |
+-----+
```

the fourth item of a non-empty list を選択します。

```
> v
```

```
(list 1 2 3 4 5 6 7 8 9 'A)
```

```
> (fourth v)
```

```
4
```

```
+-----+
| [手続き] |
| |
| (length l) → natural? |
| l: list? |
+-----+
```

the number of items on a list を評価します。

```
> x
```

```
(list 2 "hello" #true)
```

```
> (length x)
```

```
3
```

```
+-----+
| [手続き] |
| |
| (list x...) → list? |
| x: any/c |
+-----+
```

a list of its arguments を構築します。

```
> (list 1 2 3 4 5 6 7 8 9 0)
```

```
(cons 1 (cons 2 (cons 3 (cons 4 (cons 5 (cons 6 (cons 7 (cons 8 (cons 9 (cons 0 '()))))))))))
```

```
+-----+
| [手続き] |
| |
| (list* x... l) → list? |
| x: any/c |
| l: list? |
+-----+
```

a list by adding multiple items to a list を構築します。

```

> x
(list 2 "hello" #true)
> (list* 4 3 x)
(list 4 3 2 "hello" #true)

+-----+
| [手続き] |
| |
| (list-ref x i) → any/c |
| x: list? |
| i: natural? |
+-----+

```

the indexed item from the list を取り出します。

```

> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (list-ref v 9)
'A

+-----+
| [手続き] |
| |
| (list? x) → boolean? |
| x: any/c |
+-----+

```

whether the given value is a list を検査します。

```

> (list? 42)
#false
> (list? '())
#true
> (list? (cons 1 (cons 2 '())))
#true

+-----+
| [手続き] |
| |
| (make-list i x) → list? |
| i: natural? |
| x: any/c |
+-----+

```

a list of i copies of x を構築します。

```

> (make-list 3 "hello")
(cons "hello" (cons "hello" (cons "hello" '())))

+-----+
| [手続き] |
| |
| (member x l) → boolean? |
| x: any/c |
+-----+

```



```
| l: list? |
+-----+
```

値がリスト上にあるかどうかを判定します (equal? で比較)。

```
> x
(list 2 "hello" #true)
> (member "hello" x)
#true

+-----+
| [手続き] |
| |
| (member? x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

値がリスト上にあるかどうかを判定します (equal? で比較)。

```
> x
(list 2 "hello" #true)
> (member? "hello" x)
#true

+-----+
| [手続き] |
| |
| (memq x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

some value x is on some list l, using eq? to compare x with items on l かどうかを判定します。

```
> x
(list 2 "hello" #true)
> (memq (list (list 1 2 3)) x)
#false

+-----+
| [手続き] |
| |
| (memq? x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

some value x is on some list l, using eq? to compare x with items on l かどうかを判定します。

```
> x
(list 2 "hello" #true)
> (memq? (list (list 1 2 3)) x)
#false
```

```
+-----+
| [手続き] |
| |
| (memv x l) → (or/c #false list) |
| x: any/c |
| l: list? |
+-----+
```

～かどうかを判定する: ある 値 は 上の リスト もし so, it produces suffix の リスト という starts とともに x もし not, it produces false. (それ compares 値 s とともに eqv? 述語.)

```
> x
(list 2 "hello" #true)
> (memv (list (list 1 2 3)) x)
#false
```

```
+-----+
| [値] |
| |
| null: list |
+-----+
```

Another name のための 空リスト

```
> null
'()
```

```
+-----+
| [手続き] |
| |
| (null? x) → boolean? |
| x: any/c |
+-----+
```

値が空リストかどうかを判定します。

```
> (null? '())
#true
> (null? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (range start end step) → list? |
| start: number |
| end: number |
+-----+
```

```
| step: number |
+-----+
```

a list of numbers by stepping from start to end を構築します。

```
> (range 0 10 2)
(cons 0 (cons 2 (cons 4 (cons 6 (cons 8 '())))))
```

```
+-----+
| [手続き] |
| |
| (remove x l) → list? |
| x: any/c |
| l: list? |
+-----+
```

a list like the given one, with the first occurrence of the given item removed (comparing values with equal?) を構築します。

```
> x
(list 2 "hello" #true)
> (remove "hello" x)
(list 2 #true)
> hello-2
(list 2 "hello" #true "hello")
> (remove "hello" hello-2)
(list 2 #true "hello")
```

```
+-----+
| [手続き] |
| |
| (remove-all x l) → list? |
| x: any/c |
| l: list? |
+-----+
```

a list like the given one, with all occurrences of the given item removed (comparing values with equal?) を構築します。

```
> x
(list 2 "hello" #true)
> (remove-all "hello" x)
(list 2 #true)
> hello-2
(list 2 "hello" #true "hello")
> (remove-all "hello" hello-2)
(list 2 #true)
```

```
+-----+
| [手続き] |
| |
| (rest x) → any/c |
+-----+
```

```
| x: cons? |
+-----+
```

空でないリストの残り (rest) を取り出します。

```
> x
(list 2 "hello" #true)
> (rest x)
(list "hello" #true)
```

```
+-----+
| [手続き] |
| |
| (reverse l) → list |
| l: list? |
+-----+
```

a reversed version of a list を作成します。

```
> x
(list 2 "hello" #true)
> (reverse x)
(list #true "hello" 2)
```

```
+-----+
| [手続き] |
| |
| (second x) → any/c |
| x: list? |
+-----+
```

the second item of a non-empty list を選択します。

```
> x
(list 2 "hello" #true)
> (second x)
"hello"
```

```
+-----+
| [手続き] |
| |
| (seventh x) → any/c |
| x: list? |
+-----+
```

the seventh item of a non-empty list を選択します。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (seventh v)
7
```

```
+-----+
| [手続き] |
| |
```

```

| |
| (sixth x) → any/c |
| x: list? |
+-----+

```

the sixth item of a non-empty list を選択します。

```

> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (sixth v)
6

```

```

+-----+
| [手続き] |
| |
| (third x) → any/c |
| x: list? |
+-----+

```

the third item of a non-empty list を選択します。

```

> x
(list 2 "hello" #true)
> (third x)
#true

```

### 3.11 Posn

```

+-----+
| [手続き] |
| |
| (make-posn x y) → posn |
| x: any/c |
| y: any/c |
+-----+

```

a posn from two arbitrary values を構築します。

```

> (make-posn 3 3)
(make-posn 3 3)
> (make-posn "hello" #true)
(make-posn "hello" #true)

```

```

+-----+
| [手続き] |
| |
| (posn-x p) → any/c |
| p: posn |
+-----+

```

the x component of a posn を取り出します。

```

> p
(make-posn 2 -3)

```

```
> (posn-x p)
```

```
2
```

```
+-----+
| [手続き] |
| |
| (posn-y p) → any/c |
| p: posn |
+-----+
```

the y component of a posn を取り出します。

```
> p
```

```
(make-posn 2 -3)
```

```
> (posn-y p)
```

```
-3
```

```
+-----+
| [手続き] |
| |
| (posn? x) → boolean? |
| x: any/c |
+-----+
```

its input is a posn かどうかを判定します。

```
> q
```

```
(make-posn "bye" 2)
```

```
> (posn? q)
```

```
#true
```

```
> (posn? 42)
```

```
#false
```

### 3.12 文字

```
+-----+
| [手続き] |
| |
| (char->integer c) → integer |
| c: char |
+-----+
```

Looks up 数 という corresponds へ given 文字 の中の ASCII table (if any).

```
> (char->integer #\a)
```

```
97
```

```
> (char->integer #\z)
```

```
122
```

```
+-----+
| [手続き] |
| |
| (char-alphabetic? c) → boolean? |
+-----+
```

```
| c: char |
+-----+
```

a character represents an alphabetic characterかどうかを判定します。

```
> (char-alphabetic? #\Q)
#true
```

```
+-----+
| [手続き] |
| (char-ci<=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are ordered in an increasing and case-insensitive mannerかどうかを判定します。

```
> (char-ci<=? #\b #\B)
#true
> (char<=? #\b #\B)
#false
```

```
+-----+
| [手続き] |
| (char-ci<? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are ordered in a strictly increasing and case-insensitive mannerかどうかを判定します。

```
> (char-ci<? #\B #\c)
#true
> (char<? #\b #\B)
#false
```

```
+-----+
| [手続き] |
| (char-ci=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

two characters are equal in a case-insensitive mannerかどうかを判定します。

```
> (char-ci=? #\b #\B)
#true
```

```
+-----+
| [手続き] |
| |
| (char-ci>=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are sorted in a decreasing and case-insensitive mannerかどうかを判定します。

```
> (char-ci>=? #\b #\C)
#false
> (char>=? #\b #\C)
#true
```

```
+-----+
| [手続き] |
| |
| (char-ci>? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are sorted in a strictly decreasing and case-insensitive mannerかどうかを判定します。

```
> (char-ci>? #\b #\B)
#false
> (char>? #\b #\B)
#true
```

```
+-----+
| [手続き] |
| |
| (char-downcase c) → char |
| c: char |
+-----+
```

the equivalent lower-case character を生成します。

```
> (char-downcase #\T)
#\t
```

```
+-----+
| [手続き] |
| |
| (char-lower-case? c) → boolean? |
+-----+
```



```
| c: char |
+-----+
```

a character is a lower-case characterかどうかを判定します。

```
> (char-lower-case? #\T)
#false
```

```
+-----+
| [手続き] |
| |
| (char-numeric? c) → boolean? |
| c: char |
+-----+
```

a character represents a digitかどうかを判定します。

```
> (char-numeric? #\9)
#true
```

```
+-----+
| [手続き] |
| |
| (char-upcase c) → char |
| c: char |
+-----+
```

the equivalent upper-case characterを生成します。

```
> (char-upcase #\t)
#\T
```

```
+-----+
| [手続き] |
| |
| (char-upper-case? c) → boolean? |
| c: char |
+-----+
```

a character is an upper-case characterかどうかを判定します。

```
> (char-upper-case? #\T)
#true
```

```
+-----+
| [手続き] |
| |
| (char-whitespace? c) → boolean? |
| c: char |
+-----+
```

a character represents spaceかどうかを判定します。

```
> (char-whitespace? #\tab)
#true
```

```
+-----+
| [手続き] |
| (char<=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are ordered in an increasing mannerかどうかを判定します。

```
> (char<=? #\a #\a #\b)
#true
```

```
+-----+
| [手続き] |
| (char<? x d e...) → boolean? |
| x: char |
| d: char |
| e: char |
+-----+
```

the characters are ordered in a strictly increasing mannerかどうかを判定します。

```
> (char<? #\a #\b #\c)
#true
```

```
+-----+
| [手続き] |
| (char=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are equalかどうかを判定します。

```
> (char=? #\b #\a)
#false
```

```
+-----+
| [手続き] |
| (char>=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are sorted in a decreasing mannerかどうかを判定します。

```
> (char>=? #\b #\b #\a)
#true
```

```
+-----+
| [手続き] |
| |
| (char>? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are sorted in a strictly decreasing mannerかどうかを判定します。

```
> (char>? #\A #\z #\a)
#false
```

```
+-----+
| [手続き] |
| |
| (char? x) → boolean? |
| x: any/c |
+-----+
```

a value is a characterかどうかを判定します。

```
> (char? "a")
#false
> (char? #\a)
#true
```

### 3.13 文字列

```
+-----+
| [手続き] |
| |
| (explode s) → (listof string) |
| s: string |
+-----+
```

Translates 文字列へリストの1-letter 文字列 s.

```
> (explode "cat")
(list "c" "a" "t")
```

```
+-----+
| [手続き] |
| |
| (format f x...) → string |
| f: string |
| x: any/c |
+-----+
```

Formats 文字列, possibly embedding 値 s.

```
> (format "Dear Dr. ~a:" "Flatt")
"Dear Dr. Flatt:"
> (format "Dear Dr. ~s:" "Flatt")
"Dear Dr. \"Flatt\":"
```

```
+-----+
| [手続き] |
| |
| (implode l) → string |
| l: list? |
+-----+
```

Concatenates リスト の 1-letter 文字列 s へ one 文字列.

```
> (implode (cons "c" (cons "a" (cons "t" '()))))
"cat"
```

```
+-----+
| [手続き] |
| |
| (int->string i) → string |
| i: integer |
+-----+
```

an integer in [0,55295] or [57344 111411] to a 1-letter string を変換します。

```
> (int->string 65)
"A"
```

```
+-----+
| [手続き] |
| |
| (list->string l) → string |
| l: list? |
+-----+
```

a s list of characters into a string を変換します。

```
> (list->string (cons #\c (cons #\a (cons #\t '()))))
"cat"
```

```
+-----+
| [手続き] |
| |
| (make-string i c) → string |
| i: natural? |
| c: char |
+-----+
```

a string of length i from c を生成します。

```
> (make-string 3 #\d)
"ddd"
```

```

+-----+
| [手続き] |
| |
| (replicate i s) → string |
| i: natural? |
| s: string |
+-----+

```

Replicates *s* *i* times.

```

> (replicate 3 "h")
"hhh"

```

```

+-----+
| [手続き] |
| |
| (string c...) → string? |
| c: char |
+-----+

```

Builds 文字列 of the given 文字 *s*.

```

> (string #\d #\o #\g)
"dog"

```

```

+-----+
| [手続き] |
| |
| (string->int s) → integer |
| s: string |
+-----+

```

a 1-letter string to an integer in [0,55295] or [57344, 1114111] を変換します。

```

> (string->int "a")
97

```

```

+-----+
| [手続き] |
| |
| (string->list s) → (listof char) |
| s: string |
+-----+

```

a string into a list of characters を変換します。

```

> (string->list "hello")
(list #\h #\e #\l #\l #\o)

```

```

+-----+
| [手続き] |
| |
| (string->number s) → (union number #false) |
+-----+

```

```
| s: string |
+-----+
```

a string into a number, produce false if impossible を変換します。

```
> (string->number "-2.03")
-2.03
> (string->number "1-2i")
1-2i
```

```
+-----+
| [手続き] |
| |
| (string->symbol s) → symbol |
| s: string |
+-----+
```

a string into a symbol を変換します。

```
> (string->symbol "hello")
'hello
```

```
+-----+
| [手続き] |
| |
| (string-alphabetic? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are alphabeticかどうかを判定します。

```
> (string-alphabetic? "123")
#false
> (string-alphabetic? "cat")
#true
```

```
+-----+
| [手続き] |
| |
| (string-contains-ci? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

the first string appears in the second one without regard to the case of the lettersかどうかを判定します。

```
> (string-contains-ci? "At" "caT")
#true
```

```
+-----+
| [手続き] |
| |
| (string-contains? s t) → boolean? |
+-----+
```

```
| s: string |
| t: string |
+-----+
```

the first string appears literally in the second oneかどうかを判定します。

```
> (string-contains? "at" "cat")
#true
```

```
+-----+
| [手続き] |
| |
| (string-copy s) → string |
| s: string |
+-----+
```

Copies 文字列.

```
> (string-copy "hello")
"hello"
```

```
+-----+
| [手続き] |
| |
| (string-downcase s) → string |
| s: string |
+-----+
```

a string like the given one with all ‘letters’ as lower case を生成します。

```
> (string-downcase "CAT")
"cat"
> (string-downcase "cAt")
"cat"
```

```
+-----+
| [手続き] |
| |
| (string-ith s i) → 1string? |
| s: string |
| i: natural? |
+-----+
```

the ith 1-letter substring from s を取り出します。

```
> (string-ith "hello world" 1)
"e"
```

```
+-----+
| [手続き] |
| |
| (string-length s) → nat |
| s: string |
+-----+
```

判定する: length の 文字列.

```
> (string-length "hello world")
11
```

```
+-----+
| [手続き] |
| (string-lower-case? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are lower case かどうかを判定します。

```
> (string-lower-case? "CAT")
#false
```

```
+-----+
| [手続き] |
| (string-numeric? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are numeric かどうかを判定します。

```
> (string-numeric? "123")
#true
> (string-numeric? "1-2i")
#false
```

```
+-----+
| [手続き] |
| (string-ref s i) → char |
| s: string |
| i: natural? |
+-----+
```

the ith character from s を取り出します。

```
> (string-ref "cat" 2)
#\t
```

```
+-----+
| [手続き] |
| (string-upcase s) → string |
| s: string |
+-----+
```

a string like the given one with all 'letters' as upper case を生成します。

```
> (string-upcase "cat")
"CAT"
```



```
> (string-upcase "cAt")
"CAT"
```

```
+-----+
| [手続き] |
| |
| (string-upper-case? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are upper caseかどうかを判定します。

```
> (string-upper-case? "CAT")
#true
```

```
+-----+
| [手続き] |
| |
| (string-whitespace? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are white spaceかどうかを判定します。

```
> (string-whitespace? (string-append " " (string #\tab #\newline #\return)))
#true
```

```
+-----+
| [手続き] |
| |
| (string? x) → boolean? |
| x: any/c |
+-----+
```

a value is a stringかどうかを判定します。

```
> (string? "hello world")
#true
> (string? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (substring s i j) → string |
| s: string |
| i: natural? |
| j: natural? |
+-----+
```

the substring starting at i up to j (or the end if j is not provided) を取り出します。

```
> (substring "hello world" 1 5)
"ello"
```

```
> (substring "hello world" 1 8)
"ello wo"
> (substring "hello world" 4)
"o world"
```

### 3.14 画像

```
+-----+
| [手続き] |
| |
| (image=? i j) → boolean? |
| i: image |
| j: image |
+-----+
```

two images are equalかどうかを判定します。

```
> c1
[image:pict_7.png]
> (image=? (circle 5 "solid" "green") c1)
#false
> (image=? (circle 10 "solid" "green") c1)
#true
```

```
+-----+
| [手続き] |
| |
| (image? x) → boolean? |
| x: any/c |
+-----+
```

a value is an imageかどうかを判定します。

```
> c1
[image:pict_8.png]
> (image? c1)
#true
```

### 3.15 その他

```
+-----+
| [手続き] |
| |
| (≈ x y eps) → boolean? |
| x: number |
| y: number |
| eps: non-negative-real |
+-----+
```

whether x and y are within eps of either other を検査します。

```
> (≈ 1.01 1.0 0.1)
#true
```

```
> (= 1.01 1.5 0.1)
#false
```

```
+-----+
| [値] |
| |
| eof: eof-object? |
+-----+
```

A 値 という represents end の file:

```
> eof
#<eof>
```

```
+-----+
| [手続き] |
| |
| (eof-object? x) → boolean? |
| x: any/c |
+-----+
```

some value is the end-of-file value かどうかを判定します。

```
> (eof-object? eof)
#true
> (eof-object? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (eq? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+
```

two values are equivalent from the computer's perspective (intensional) かどうかを判定します。

```
> (eq? (cons 1 '()) (cons 1 '()))
#false
> one
(list 1)
> (eq? one one)
#true
```

```
+-----+
| [手続き] |
| |
| (equal? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+
```

two values are structurally equal where basic values are compared with the `eqv?` predicate かどうかを判定します。

```
> (equal? (make-posn 1 2) (make-posn (- 2 1) (+ 1 1)))
#true
```

```
+-----+
| [手続き] |
| |
| (equal~? x y z) → boolean? |
| x: any/c |
| y: any/c |
| z: non-negative-real |
+-----+
```

x and y like `equal?` but uses `=~` in the case of numbers を比較します。

```
> (equal~? (make-posn 1.01 1.0) (make-posn 1.01 0.99) 0.2)
#true
```

```
+-----+
| [手続き] |
| |
| (eqv? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+
```

two values are equivalent from the perspective of all functions that can be applied to it (extensional) かどうかを判定します。

```
> (eqv? (cons 1 '()) (cons 1 '()))
#false
> one
(list 1)
> (eqv? one one)
#true
```

```
+-----+
| [手続き] |
| |
| (error x...) → void? |
| x: any/c |
+-----+
```

Signals an error, combining the given values into an error message. If any of the values' printed representations is too long, it is truncated and “...” is put into the string. If the first value is a symbol, it is suffixed with a colon and the result pre-pended on to the error message.

```
> zero
0
```

```
> (if (= zero 0) (error "can't divide by 0") (/ 1 zero))
can't divide by 0
```

```
+-----+
| [手続き] |
| |
| (exit) → void |
+-----+
```

Evaluating (exit) terminates running プログラム.

```
+-----+
| [手続き] |
| |
| (identity x) → any/c |
| x: any/c |
+-----+
```

x を返します。

```
> (identity 42)
42
> (identity c1)
[image:pict_9.png]
> (identity "hello")
"hello"
```

```
+-----+
| [手続き] |
| |
| (struct? x) → boolean? |
| x: any/c |
+-----+
```

some value is a structureかどうかを判定します。

```
> (struct? (make-posn 1 2))
#true
> (struct? 43)
#false
```

### 3.16 シグネチャ

```
+-----+
| [値] |
| |
| Any: signature? |
+-----+
```

any value のシグネチャ。

```
+-----+
| [値] |
| |
```

```
| Boolean: signature? |
+-----+
```

booleans のシグネチャ。

```
+-----+
| [値] |
| |
| Char: signature? |
+-----+
```

characters のシグネチャ。

```
+-----+
| [手続き] |
| |
| (ConsOf first-sig rest-sig) → signature? |
| first-sig: signature? |
| rest-sig: signature? |
+-----+
```

a cons pair のシグネチャ。

```
+-----+
| [値] |
| |
| EmptyList: signature? |
+-----+
```

the empty list のシグネチャ。

```
+-----+
| [値] |
| |
| False: signature? |
+-----+
```

just false のシグネチャ。

```
+-----+
| [値] |
| |
| Integer: signature? |
+-----+
```

integers のシグネチャ。

```
+-----+
| [値] |
| |
| Natural: signature? |
+-----+
```

natural numbers のシグネチャ。

```

+-----+
| [値] |
| |
| Number: signature? |
+-----+

```

arbitrary numbers のシグネチャ。

```

+-----+
| [値] |
| |
| Rational: signature? |
+-----+

```

rational numbers のシグネチャ。

```

+-----+
| [値] |
| |
| Real: signature? |
+-----+

```

real numbers のシグネチャ。

```

+-----+
| [値] |
| |
| String: signature? |
+-----+

```

strings のシグネチャ。

```

+-----+
| [値] |
| |
| Symbol: signature? |
+-----+

```

symbols のシグネチャ。

```

+-----+
| [値] |
| |
| True: signature? |
+-----+

```

just true のシグネチャ。

### 3.17 数（緩和条件）

```

+-----+
| [手続き] |
| |
| (* x...) → number |
+-----+

```

```
| x: number |
+-----+
```

Multiplies すべての given 数 s. In ISL と up: \* works のとき applied へ only one 数 または none.

```
> (* 5 3)
15
> (* 5 3 2)
30
> (* 2)
2
> (*)
1
```

```
+-----+
| [手続き] |
| |
| (+ x...) → number |
| x: number |
+-----+
```

Adds すべての given 数 s. In ISL と up: + works のとき applied へ only one 数 または none.

```
> (+ 2/3 1/16)
35/48
> (+ 3 2 5 8)
18
> (+ 1)
1
> (+)
0
```

```
+-----+
| [手続き] |
| |
| (/ x y...) → number |
| x: number |
| y: number |
+-----+
```

Divides first によって すべての remaining 数 s. In ISL と up: / computes inverse のとき applied へ one 数.

```
> (/ 12 2)
6
> (/ 12 2 3)
2
> (/ 3)
1/3
```



```
+-----+
| [手続き] |
| |
| (= x...) → number |
| x: number |
+-----+
```

numbers for equality. In ISL and up: = works when applied to only one number を比較します。

```
> (= 10 10)
#true
> (= 11)
#true
> (= 0)
#true
```

### 3.18 文字列（緩和条件）

```
+-----+
| [手続き] |
| |
| (string-append s...) → string |
| s: string |
+-----+
```

Concatenates 文字 s の several 文字列 s.

```
> (string-append "hello" " " "world" " " "good bye")
"hello world good bye"
```

```
+-----+
| [手続き] |
| |
| (string-ci<=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically increasing and case-insensitive manner かどうかを判定します。

```
> (string-ci<=? "hello" "WORLD" "zoo")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci<? s t x...) → boolean? |
| s: string |
| t: string |
+-----+
```

```
| x: string |
+-----+
```

the strings are ordered in a lexicographically strictly increasing and case-insensitive mannerかどうかを判定します。

```
> (string-ci<? "hello" "WORLD" "zoo")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

all strings are equal, character for character, regardless of caseかどうかを判定します。

```
> (string-ci=? "hello" "Hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci>=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically decreasing and case-insensitive mannerかどうかを判定します。

```
> (string-ci>? "zoo" "WORLD" "hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci>? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically strictly decreasing and case-insensitive mannerかどうかを判定します。

```
> (string-ci>? "zoo" "WORLD" "hello")
#true
```

```

+-----+
| [手続き] |
| (string<=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+

```

the strings are ordered in a lexicographically increasing mannerかどうかを判定します。

```

> (string<=? "hello" "hello" "world" "zoo")
#true

```

```

+-----+
| [手続き] |
| (string<? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+

```

the strings are ordered in a lexicographically strictly increasing mannerかどうかを判定します。

```

> (string<? "hello" "world" "zoo")
#true

```

```

+-----+
| [手続き] |
| (string=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+

```

all strings are equal, character for characterかどうかを判定します。

```

> (string=? "hello" "world")
#false
> (string=? "bye" "bye")
#true

```

```

+-----+
| [手続き] |
| (string>=? s t x...) → boolean? |
| s: string |
| t: string |
+-----+

```

```
| x: string |
+-----+
```

the strings are ordered in a lexicographically decreasing mannerかどうかを判定します。

```
> (string>=? "zoo" "zoo" "world" "hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string>? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically strictly decreasing mannerかどうかを判定します。

```
> (string>? "zoo" "world" "hello")
#true
```

### 3.19 Posn

```
+-----+
| [手続き] |
| |
| (posn) → signature |
+-----+
```

posns のシグネチャ。

### 3.20 高階関数

```
+-----+
| [手続き] |
| |
| (andmap p? l...) → boolean |
| p?: (X... -> boolean) |
| l: (listof X) |
+-----+
```

～かどうかを判定する: p? holds のためのすべての items の l…:

```
(andmap p (list x-1... x-n)) = (and (p x-1)... (p x-n))
```

```
(andmap p (list x-1... x-n) (list y-1... y-n)) = (and (p x-1 y-1)... (p x-n y-n))
```

```
> (andmap odd? '(1 3 5 7 9))
#true
```

```
> (andmap even? '())
#true
```

Making sure すべての数  $s$  は below ある threshold:

```
> (define (small-enough? x) (< x 3))
> (andmap small-enough? '(0 1 2))
#true
```

Checking という すべての items の中の first リスト satisfy corresponding predicate の中の 2nd:

```
> (define (and-satisfies? x f) (f x))
> (andmap and-satisfies? (list 0 1 2) (list odd? even? positive?))
#false
```

```
+-----+
| [手続き] |
| |
| (apply f x-1... l) → Y |
| f: (X-1... X-N → Y) |
| x-1: X-1 |
| l: (list X-i+1... X-N) |
+-----+
```

Applies 関数 using items から リスト として 引数 s:

```
(apply f (list x-1... x-n)) = (f x-1... x-n)
```

```
> a-list
(list 0 1 2 3 4 5 6 7 8 9)
> (apply max a-list)
9
```

```
+-----+
| [手続き] |
| |
| (argmax f l) → X |
| f: (X → real) |
| l: (listof X) |
+-----+
```

the (first) element of the list that maximizes the output of the function を探します。

```
> (argmax second '((sam 98) (carl 78) (vincent 93) (asumu 99)))
(list 'asumu 99)
```

```
+-----+
| [手続き] |
| |
| (argmin f l) → X |
| f: (X → real) |
| l: (listof X) |
+-----+
```

the (first) element of the list that minimizes the output of the function を探します。

```
> (argmin second '((sam 98) (carl 78) (vincent 93) (asumu 99)))
(list 'carl 78)
```

```
+-----+
| [手続き] |
| |
| (build-list n f) → (listof X) |
| n: nat |
| f: (nat -> X) |
+-----+
```

構築する: リスト によって applying f へ 数 s between 0 と (-n1):

```
(build-list n f) = (list (f 0)... (f (- n 1)))
```

```
> (build-list 22 add1)
(list 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22)
```

Creating diagonal matrix:

```
> (define (diagonalize i) (local ((define (off j) (if (= i j) 1 0)))
 (build-list 3 off)))
> (build-list 3 diagonalize)
(list (list 1 0 0) (list 0 1 0) (list 0 0 1))
```

```
+-----+
| [手続き] |
| |
| (build-string n f) → string |
| n: nat |
| f: (nat -> char) |
+-----+
```

構築する: 文字列 によって applying f へ 数 s between 0 と (-n1):

```
(build-string n f) = (string (f 0)... (f (- n 1)))
```

```
> (build-string 10 integer->char)
"\u0000\u0001\u0002\u0003\u0004\u0005\u0006\a\b\t"
```

Making alphabet:

```
> (define (starting-at-a x) (integer->char (+ 65 x)))
> (build-string 26 starting-at-a)
"ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

```
+-----+
| [手続き] |
| |
| (compose f g) → (X -> Z) |
| f: (Y -> Z) |
| g: (X -> Y) |
+-----+
```

(説明) Composes sequence の procedures へ single procedure:

```
(compose f g)
```

is equivalent to

```
(define (f-after-g x)
 (f (g x)))
```

```
> (map (compose add1 second) '((add 3) (sub 2) (mul 4)))
(list 4 3 5)
```

```
+-----+
| [手続き] |
| |
| (filter p? l) → (listof X) |
| p?: (X → boolean) |
| l: (listof X) |
| |
+-----+
```

a list from all those items on a list for which the predicate holds を構築します。

```
> (filter odd? '(0 1 2 3 4 5 6 7 8 9))
(list 1 3 5 7 9)
```

Keep only 数  $s$  という は large enough:

```
> (define (large-enough? x) (>= x 3))
> (filter large-enough? '(0 1 2 3 4 5 6 7 8 9))
(list 3 4 5 6 7 8 9)
```

```
+-----+
| [手続き] |
| |
| (foldl f base l...) → Y |
| f: (X... Y → Y) |
| base: Y |
| l: (listof X) |
| |
+-----+
```

```
(foldl f base (list x-1... x-n)) = (f x-n... (f x-1 base))
```

```
(foldl f base (list x-1... x-n) (list y-1... y-n))
= (f x-n y-n... (f x-1 y-1 base))
```

```
> (foldl + 0 '(0 1 2 3 4 5 6 7 8 9))
45
```

```
> (foldl cons '() '(a b c))
(list 'c 'b 'a)
```

```
+-----+
| [手続き] |
| |
| (foldr f base l...) → Y |
| f: (X... Y → Y) |
| |
+-----+
```

```

| base: Y |
| l: (listof X) |
+-----+

```

```

(foldr f base (list x-1... x-n)) = (f x-1... (f x-n base))

```

```

(foldr f base (list x-1... x-n) (list y-1... y-n))
= (f x-1 y-1... (f x-n y-n base))

```

```

> (foldr + 0 '(0 1 2 3 4 5 6 7 8 9))
45

```

Append すべての rests のすべての リスト s:

```

> (define (append-rests f r) (append (rest f) r))
> (foldr append-rests '() '((1 a) (2 b c) (3 d e f)))
(list 'a 'b 'c 'd 'e 'f)

```

Add two リスト s の数 s:

```

> (define (add-two-lists x y r) (+ x y r))
> (foldr add-two-lists 0 '(1 2 3) '(10 11 12))
39

```

```

+-----+
| [手続き] |
| |
| (map f l...) → (listof Z) |
| f: (X... → Z) |
| l: (listof X) |
+-----+

```

構築する: new リスト によって applying 関数 へ 各 item 上の one または more existing リスト s:

```

(map f (list x-1... x-n)) = (list (f x-1)... (f x-n))
(map f (list x-1... x-n) (list y-1... y-n)) = (list (f x-1 y-1)... (f x-n y-n))
> (map add1 (list 3 -4.01 2/5))
(list 4 #i-3.01 1.4)

```

Mapping user-defined 関数:

```

> (define (tag-with-a x) (list "a" (+ x 1)))
> (map tag-with-a (list 3 -4.01 2/5))
(list (list "a" 4) (list "a" #i-3.01) (list "a" 1.4))

```

Mapping over two リスト s:

```

> (define (add-and-multiply x y) (+ x (* x y)))
> (map add-and-multiply (list 3 -4 2/5) '(1 2 3))
(list 6 -12 1.6)

```



```
+-----+
| [手続き] |
| |
| (memf p? l) → (union #false (listof X)) |
| p?: (X → any) |
| l: (listof X) |
+-----+
```

#false if p? produces false for all items on l. If p? produces #true for any of the items on l, memf returns the sub-list starting from that item を生成します。

```
> (memf odd? '(2 4 6 3 8 0))
(list 3 8 0)
```

```
+-----+
| [手続き] |
| |
| (ormap p? l...) → boolean |
| p?: (X... → boolean) |
| l: (listof X) |
+-----+
```

～かどうかを判定する: p? holds のための at least one items の l:

```
(ormap p (list x-1... x-n)) = (or (p x-1)... (p x-n))
```

```
(ormap p (list x-1... x-n) (list y-1... y-n)) = (or (p x-1 y-1)... (p x-n y-n))
```

```
> (ormap odd? '(1 3 5 7 9))
#true
```

```
> (ormap even? '())
#false
```

Making sure at least one 数 は below ある threshold:

```
> (define (a-small-one? x) (< x 3))
```

```
> (ormap a-small-one? '(6 7 8 1 5))
#true
```

Checking という one item の中の first リスト satisfy corresponding predicate の中の 2nd:

```
> (define (or-satisfies? x f) (f x))
```

```
> (ormap or-satisfies? (list 0 1 2) (list odd? even? positive?))
#true
```

```
+-----+
| [手続き] |
| |
| (procedure? x) → boolean? |
| x: any |
+-----+
```

true if the value is a procedure を生成します。

```
> (procedure? cons)
#true
```

```
> (procedure? add1)
#true
```

Checking プログラム mer-defined 関数:

```
> (define (my-function x) x)
> (procedure? my-function)
#true
```

```
+-----+
| [手続き] |
| |
| (quicksort l comp) → (listof X) |
| l: (listof X) |
| comp: (X X → boolean) |
+-----+
```

Sorts items 上の l, の中の order according へ comp (using quicksort algorithm).

```
> (quicksort '(6 7 2 1 3 4 0 5 9 8) <)
(list 0 1 2 3 4 5 6 7 8 9)
```

```
+-----+
| [手続き] |
| |
| (sort l comp) → (listof X) |
| l: (listof X) |
| comp: (X X → boolean) |
+-----+
```

Sorts items 上の l, の中の order according へ comp.

```
> (sort '(6 7 2 1 3 4 0 5 9 8) <)
(list 0 1 2 3 4 5 6 7 8 9)
```

## 付録 B-4: Intermediate Student with lambda (#lang htdp/isl+)

原本: extracted/appendix/htdp-langs/original\_markdown\_04\_intermediate-lam.md

初学者向けに、説明文を日本語へ翻訳しています。コード・シグネチャ・実行例は原文のまま保持します。

### 4 Intermediate Student with Lambda (lambda つき中級)

文法の記法では、 $X\cdots$  (太字の点) という書き方で、 $X$  が任意の回数 (0 回、1 回、またはそれ以上) 現れてよいことを示します。別に、文法はテンプレートで使う識別子として  $\cdots$  も定義します。

program	=	def-or-expr...
def-or-expr	=	definition expr test-case library-require
definition	=	(define (name variable variable..... (define name expr) (define-struct name (name...))
expr	=	(lambda (variable variable...) exp... ( $\lambda$ (variable variable...) expr) (local [definition...] expr) (letrec ([name expr]...) expr) (let ([name expr]...) expr) (let* ([name expr]...) expr) (expr expr expr...) (cond [expr expr]... [expr expr]) (cond [expr expr]... [else expr]) (if expr expr expr) (and expr expr expr...) (or expr expr expr...) (time expr) name prim-op 'quoted 'quasiquoted '() number boolean string character (signature signature-form)
signature-declaration	=	(: name signature-form)
signature-form	=	(enum expr...)

			(mixed signature-form...)
			(signature-form... -> signature-fo...
			(ListOf signature-form)
			signature-variable
			expr
signature-variable	=		%name
quoted	=		name
			number
			string
			character
			(quoted...)
			'quoted
			'quoted
			`,`quoted
			`,`@`quoted
quasiquoted	=		name
			number
			string
			character
			(quasiquoted...)
			'quasiquoted
			'quasiquoted
			`,`expr
			`,`@`expr
test-case	=		(check-expect expr expr)
			(check-random expr expr)
			(check-within expr expr expr)
			(check-member-of expr expr...)
			(check-range expr expr expr)
			(check-satisfied expr expr)
			(check-error expr expr)
			(check-error expr)
library-require	=		(require string)
			(require (lib string string...))
			(require (planet string package))
package	=		(string string number number)

名前 (name) または変数 (variable) は、空白や次の文字を含まない文字の並びです。

“`,`()[]{}|;#

数 (number) とは、123、3/2、5.5 のような数です。

真偽値 (boolean) は #true または #false のいずれかです。

#true 定数の別表記は #t、true、#T です。同様に #f、false、#F も #false として認識されます。

シンボル (symbol) は、クォート文字に続く名前です。シンボルは 42、'()、#false などと同じく値です。

文字列 (string) は、一對の " で囲まれた文字の並びです。シンボルと違い、文字列は文字に分割したり、さまざまな関数で操作したりできます。たとえば "abcdef"、"This is a string"、および "This is a string with " inside" はすべて文字列です。

文字 (character) は # で始まり、その文字の名前を持ちます。たとえば #、## は文字です。

関数呼び出しでは、開き括弧の直後に現れる関数は、define や define-struct で定義した関数、またはあらかじめ定義された関数のいずれかです。

#### 4.1 あらかじめ定義された変数

```
+-----+
| [値] |
| |
| empty: empty? |
+-----+
```

空リスト。

```
+-----+
| [値] |
| |
| true: boolean? |
+-----+
```

#true の値。

```
+-----+
| [値] |
| |
| false: boolean? |
+-----+
```

#false の値。

#### 4.2 テンプレート変数

```
+-----+
| [構文] |
| |
| .. |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
| ... |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
| |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
| |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
| |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

### 4.3 lambda つき中級の構文

```
+-----+
| [構文] |
| |
| (lambda (variable variable...) expression) |
+-----+
```

与えられた変数と同じ個数の引数を取り、本体が expression である関数を作ります。

```
+-----+
| [構文] |
| |
| (λ (variable variable...) expression) |
+-----+
```

ギリシャ文字λは lambda の同義語です。

```
+-----+
| [構文] |
| |
| (expression expression expression...) |
+-----+
```

最初の expression を評価して得られる関数を呼び出します。呼び出しの値は、その関数本体の中の名前付き変数が、対応する式の値で置き換えられたときの本体の値です。

呼び出される関数は、その関数呼び出しより前に現れる定義から来るか、lambda 式から来なければなりません。引数の式の個数は、関数が期待する引数の個数と同じでなければなりません。

```
+-----+
| [構文] |
| |
| (local [definition...] expression) |
+-----+
```

expression の中で使う関連する定義をまとめます。各 definition は define または define-struct のいずれかです。

local を評価するとき、各定義が順番に評価され、最後に本体の expression が評価されます。定義によって導入された名前を参照できるのは、local の内側の式だけです（定義の右辺と本体の expression を含みます）。local で定義した名前がトップレベルの束縛と同じ場合、内側のものが外側のものを「影で覆い（シャドウ）」ます。つまり、local の内側では、その名前への参照は内側の定義を指します。

```
+-----+
| [構文] |
| |
| (letrec ([name expr-for-let]...) expression) |
+-----+
```

local に似ていますが、構文がより簡単です。各 name は、対応する expr-for-let の値を持つ変数（または関数）を定義します。expr-for-let が lambda なら letrec は関数を定義し、そうでなければ変数を定義します。

```
+-----+
| [構文] |
| |
| (let* ([name expr-for-let]...) expression) |
+-----+
```

letrec に似ていますが、各 name は本体の expression と、その名前より後に現れる expr-for-let の中だけで使えます。

```
+-----+
| [構文] |
| |
| (let ([name expr-for-let]...) expression) |
+-----+
```

letrec に似ていますが、定義された名前は最後の expression の中だけで使え、名前の隣の expr-for-let の中では使えません。

```
+-----+
| [構文] |
| |
| |
+-----+
```

```
| (time expression) |
+-----+
```

expression の評価にかかった時間を測ります。expression を評価したあと、time はその評価にかかった時間（実時間、CPU 時間、空きメモリの回収に使った時間を含む）を表示します。time の値は expression の値と同じです。

#### 4.4 共通の構文（Common Syntaxes）

次の構文は、Intermediate with Lambda レベルでも Intermediate Student レベルと同じ動きをします。

```
+-----+
| [構文] |
| |
| (define (name variable variable...) expression) |
+-----+
```

name という名前の関数を定義します。expression は関数本体です。関数が呼ばれると、引数の値が変数の代わりに本体へ挿入されます。関数は、その新しい式の値を返します。関数名は、他の関数や変数と同じであってはいけません。

```
+-----+
| [構文] |
| |
| (define name expression) |
+-----+
```

expression の値で name という変数を定義します。変数名は他の関数や変数と同じであってはいならず、name 自身が expression に現れてはいけません。

```
+-----+
| [構文] |
| |
| 'name |
+-----+
```

クオートされた name はシンボルです。クオートされた部分は、入れ子のリストの略記です。

通常、このクオートは ' で書き、例えば '(apple banana) のようにしますが、quote を使って (quote (apple banana)) のように書くこともできます。

```
+-----+
| [構文] |
| |
| 'name |
+-----+
```

quote に似ていますが、式への「アンクオート」による脱出も許します。



通常、準クオートはバッククオート で書き、例えば(`apple ,(+ 1 2)`) のようにしますが、`quasiquote` を使って (`quasiquote (apple ,(+ 1 2))`) のように書くこともできます。

```
+-----+
| [構文] |
| |
| ,expression |
+-----+
```

単一の準クオートの下では、`,expression` はクオートから脱出して、評価された式の結果を略記リストへ挿入します。

複数の準クオートの下では、`,expression` は文字どおりの `,expression` であり、`expression` に対する準クオートの段数を 1 つ減らします。

通常、アンクオートは `,` で書きますが、`unquote` で書くこともできます。

```
+-----+
| [構文] |
| |
| ,@expression |
+-----+
```

単一の準クオートの下では、`,@expression` はクオートから脱出して、評価結果がリストである式を、略記リストへ継ぎ足します (スプライスします)。

複数の準クオートの下では、スプライシング・アンクオートはアンクオートと同様で、準クオートの段数を 1 つ減らします。

通常、スプライシング・アンクオートは `,@` で書きますが、`unquote-splicing` で書くこともできます。

```
+-----+
| [構文] |
| |
| (define-struct structure-name (field-name...)) |
+-----+
```

`structure-name` という新しい構造体を定義します。構造体のフィールドは `field-name` たちで名付けられます。`define-struct` のあと、次の新しい関数が使えます。

- `make-structure-name`: 構造体のフィールド数と同じ個数の引数を取り、その構造体の新しいインスタンスを作ります。
- `structure-name-field-name`: 構造体のインスタンスを取り、`field-name` というフィールドの値を返します。
- `structure-name?`: 任意の値を取り、その値がその構造体のインスタンスなら `#true` を返します。

`define-struct` が導入する新しい関数の名前は、他の関数や変数と同じであってはなりません。同じだと `define-struct` はエラーを報告します。

```

+-----+
| [構文] |
|
| (cond [question-expression answer-expression]...)
| (cond [question-expression answer-expression]... [else answer-expression])
| (cond [question-expression answer-expression]
| ...
| [else answer-expression])
|
| ``racket
| (cond [question-expression answer-expression]
| ...
| [else answer-expression])
|
+-----+

```

ある条件に基づいて節を選びます。cond は #true に評価される最初の question-expression を見つけ、対応する answer-expression を評価します。

どの question-expression も #true に評価されない場合、cond の値は else 節の answer-expression です。else がなければ、cond はエラーを報告します。question-expression の結果が #true でも #false でもない場合も、cond はエラーを報告します。

else は cond の外では使えません。

```

+-----+
| [構文] |
|
| (if question-expression
| then-answer-expression
| else-answer-expression)
|
| ``racket
| (if question-expression
| then-answer-expression
| else-answer-expression)
| ...
|
+-----+

```

question-expression の値が #true のとき、if は then-answer-expression を評価します。テストが #false のとき、if は else-answer-expression を評価します。

question-expression が #true でも #false でもない場合、if はエラーを報告します。

```

+-----+
| [構文] |
|
| (and expression expression expression...)
|
+-----+

```

すべての expression が #true なら #true に評価されます。いずれかの expression が #false なら、and 式は #false に評価されます（その式より右の式は評価されません）。

いずれかの expression が #true でも #false でもない値に評価された場合、and はエラーを報告します。

```
+-----+
| [構文] |
| |
| (or expression expression expression...) |
+-----+
```

いずれかの expression が #true になった時点で #true に評価されます（その式より右の式は評価されません）。すべての expression が #false なら、or 式は #false に評価されます。

いずれかの expression が #true でも #false でもない値に評価された場合、or はエラーを報告します。

```
+-----+
| [構文] |
| |
| (check-expect expression expected-expression) |
+-----+
```

最初の expression が expected-expression と同じ値に評価されることをチェックします。

```
(check-expect (fahrenheit->celsius 212) 100)
(check-expect (fahrenheit->celsius -40) -40)
```

```
(define (fahrenheit->celsius f)
 (* 5/9 (- f 32)))
```

check-expect 式は学生プログラムのトップレベルに置かなければなりません。また、プログラムのどこに置いてもよく、テスト対象の関数定義より前でも構いません。そこに check-expect を置くことで、プログラマは動く例を通じて将来の読者にプログラムの意図を伝え、しばしば関数定義本体そのものを読む必要をなくします。check-expect（およびすべての check 形式）の構文エラーは、意図的に実行時まで遅延されます。学生が完全な関数ヘッダを必ずしも書かずにテストを書けるようにするためです。

expr または expected-expr が不正確数や関数値を生成するのはエラーです。不正確数については、単純な等価性で比較するのは道徳的に正しくありません。代わりに両者が小さな区間内にあるかを調べます。check-within を参照してください。関数については（Intermediate 以上）、関数を比較することは証明可能なほど不可能です。

```
+-----+
| [構文] |
| |
| (check-random expression expected-expression) |
+-----+
```

最初の expression が expected-expression と同じ値に評価されることをチェックします。

この形式は、両側に同じ乱数生成器を供給します。両側が同じ区間の乱数を同じ順番で要求すれば、同じ乱数を受け取ります。

ここでは simple example の where check-乱数 は useful:

```
(define WIDTH 100)
(define HEIGHT (* 2 WIDTH))

(define-struct player (name x y))
; A Player is (make-player String Nat Nat)

; String -> Player

(check-random (create-randomly-placed-player "David Van Horn")
 (make-player "David Van Horn" (random WIDTH) (random HEIGHT)))
```

```
(define (create-randomly-placed-player name)
 (make-player name (random WIDTH) (random HEIGHT)))
```

check-random の両側で、random が同じ数に対して同じ順番で呼ばれていることに注意してください。両側が異なる区間で random を呼ぶと、失敗しやすくなります。

```
; String -> Player

(check-random (create-randomly-placed-player "David Van Horn")
 (make-player "David Van Horn" (random WIDTH) (random HEIGHT)))
```

```
(define (create-randomly-placed-player name)
 (a-helper-function name (random HEIGHT)))
```

```
; String Number -> Player
(define (a-helper-function name height)
 (make-player name (random WIDTH) height))
```

a-helper-function への引数が先に評価されるため、random は最初に区間 [0,HEIGHT) で、次に [0,WIDTH) で呼ばれます。つまり、前の check-random とは異なる順番です。

expr または expected-expr が関数値や不正確数を生成するのはエラーです。詳細は check-expect の注を参照してください。

```
+-----+
| [構文] |
| |
| (check-satisfied expression predicate) |
+-----+
```

最初の expression が、名前付きの predicate (1 引数の関数) を満たすことをチェックします。「満たす」とは「その関数が与えられた値に対して #true を生成する」という意味です。

check-satisfied の簡単な例を次に示します。

```
> (check-satisfied 1 odd?)
The test passed!

> (check-satisfied 1 even?)
Ran 1 test. 0 tests passed.
Check failures:
Actual value | 1 | does not satisfy even?.
```



```
at line 3, column 0
```

一般に check-satisfied は、プログラム設計者が定義した関数を使ってテスト式を書き表す力を与えます。

```
; [cons Number [List-of Number]] -> Boolean
; a function for testing htdp-sort
```

```
(check-expect (sorted? (list 1 2 3)) #true)
(check-expect (sorted? (list 2 1 3)) #false)
```

```
(define (sorted? l)
 (cond
 [(empty? (rest l)) #true]
 [else (and (<= (first l) (second l)) (sorted? (rest l)))]))
```

```
; [List-of Number] -> [List-of Number]
; create a sorted version of the given list of numbers
```

```
(check-satisfied (htdp-sort (list 1 2 0 3)) sorted?)
```

```
(define (htdp-sort l)
 (cond
 [(empty? l) l]
 [else (insert (first l) (htdp-sort (rest l)))]))
```

```
; Number [List-of Number] -> [List-of Number]
; insert x into l at proper place
; assume l is arranged in ascending order
; the result is sorted in the same way
```

```
(define (insert x l)
 (cond
 [(empty? l) (list x)]
 [else (if (<= x (first l)) (cons x l) (cons (first l) (insert x (rest l))))]))
```

そして実際、htdp-sort の結果は sorted? 述語を満たします。

```
> (check-satisfied (htdp-sort (list 1 2 0 3)) sorted?)
```

```
+-----+
| [構文] |
| |
```

expression 式の値が expected-expression 式の生成する値と構造的に等しいかを検査します。最初の式の中のすべての数は、2 番目の式の対応する数から delta 以内でなければなりません。

入れ子のデータに不正確数が含まれるため、テストには check-within が正しい選択であり、delta が十分大きければテストは成功します。

```
> (check-within (roots-table (list 1.0 2.0 3.0)) (list (make-roots 1.0 1.0) (make-roots 2 1.414) (make-roots 3 1.713)) 0.1)
The test passed!
```

```
> (check-within (roots-table (list 2.0)) (list (make-roots 2
1.414))) #ile-5)
Ran 1 test.
0 tests passed.
Check failures:
```

Actual value | '((make-roots 2.0  
1.4142135623730951)) | is not within 1e-5 of expected value | '((make-roots 2 1.414)) | .

at line 5, column 0

delta が数でない場合、check-within はエラーを報告します。

534

```
| (check-error expression) |
+-----+
```

expression がエラーを報告することを確認します。expected-error-message がある場合は、エラーメッセージがその値と一致することも確認します。

check-error の使用が求められる典型的な初学者の例を次に示します。

```
(define sample-table
 '(("matthias" 10)
 ("matthew" 20)
 ("robby" -1)
 ("shriram" 18)))

; [List-of [List String Number]] String -> Number
; determine the number associated with s in table

(define (lookup table s)
 (cond
 [(empty? table) (error (string-append s " not found"))]
 [else (if (string=? (first (first table)) s)
 (second (first table))
 (lookup (rest table))))])
```

この文脈での次の 2 つの例を考えてください。

例:

```
> (check-expect (lookup sample-table "matthew") 20)
The test passed!
```

例:

```
> (check-error (lookup sample-table "kathi") "kathi not found")
The test passed!
```

```
+-----+
| [構文] |
| |
| (check-member-of expression expression expression...) |
+-----+
```

最初の expression の値が、続く expression のいずれかと同じであることを確認します。

```
; [List-of X] -> X
; pick a random element from the given list l
(define (pick-one l)
 (list-ref l (random (length l))))
```

例:

```
> (check-member-of (pick-one '("a" "b" "c")) "a" "b" "c")
The test passed!
```

いずれかの expression が関数値を生成するのはエラーです。詳細は check-expect の注を参照してください。

```
+-----+
| [構文] |
| |
| (check-range expression low-expression high-expression) |
+-----+
```

最初の expression の値が、low-expression の値と high-expression の値の間（両端を含む）の数であることを検査します。

check-range 形式は、不正確数を計算する関数の取り得る結果の範囲を区切るのに最も適しています。

```
(define EPSILON 0.001)

; [Real -> Real] Real -> Real
; what is the slope of f at x?
(define (differentiate f x)
 (slope f (- x EPSILON) (+ x EPSILON)))

; [Real -> Real] Real Real -> Real
(define (slope f left right)
 (/ (- (f right) (f left))
 2 EPSILON))
```

```
(check-range (differentiate sin 0) 0.99 1.0)
```

expression、low-expression、high-expression のいずれかが関数値を生成するのはエラーです。詳細は check-expect の注を参照してください。

```
+-----+
| [構文] |
| |
| (require string) |
+-----+
```

string で指定されたモジュールの定義を、現在のモジュール（つまり現在のファイル）で使えるようにします。string は現在のファイルからの相対パスのファイルを指します。

string には、プラットフォームごとのパス規約の問題を避けるための制約がいくつかあります。/ はディレクトリ区切り、. は常に現在のディレクトリ、.. は常に親ディレクトリを意味し、パス要素に使えるのは a から z（大文字小文字どちらも）、0 から 9、-、\_、. だけで、文字列は空であってはならず、先頭や末尾の / を含んではなりません。

```
+-----+
| [構文] |
| |
+-----+
```



```
| (require module-name) |
+-----+
```

インストール済みライブラリ内のファイルにアクセスします。ライブラリ名は、相対パス文字列と同じ制約を持つ識別子です（ただし引用符はありません）。さらに .. を含んではならないという制約があります。

```
+-----+
| [構文] |
| |
| (require (lib string string...)) |
+-----+
```

インストール済みライブラリ内のファイルにアクセスし、その定義を現在のモジュール（つまり現在のファイル）で使えるようにします。最初の string がライブラリファイル名で、残りの string がファイルがインストールされているコレクション（およびサブコレクションなど）の名前です。各 string は (require string) 形式と同じ制約を受けます。

```
+-----+
| [構文] |
| |
| (require (planet string (string string number number))) |
+-----+
```

インターネット上の PPlaneT サーバ経由で配布されているライブラリにアクセスし、その定義を現在のモジュール（つまり現在のファイル）で使えるようにします。

planet の require の完全な文法は Importing and Exporting: require and provide にありますが、構文の例を探すのにいちばんよい場所は、PPlaneT サーバ上の特定のパッケージの説明です。

## 4.5 あらかじめ定義された関数

## 4.6 シグネチャ

Signatures do not have to be comment: They can also be part of the > code. When a signature is attached to a function, DrRacket will check > that program uses the function in accordance with the signature and > display signature violations along with the test results. A signature is a regular value, and is specified as a > signature form, a > special syntax that only works with: signature declarations > and inside signature expressions.

```
+-----+
| [構文] |
| |
| (: name signature-form) |
+-----+
```

これ attaches シグネチャ specified によって シグネチャ-form へ 定義 の name. There でなければならぬ 定義 の name somewhere の中の プログラム.

```
(: age Integer)
(define age 42)

(: area-of-square (Number -> Number))
(define (area-of-square len)
 (sqr len))
```

On running プログラム, Racket checks whether シグネチャ s attached with: actually match 値 の 変数. もし they don't, Racket reports シグネチャ violation along とともに test failures.

たとえば、この piece の code:

```
(: age Integer)
(define age "fortytwo")
```

Yields この output:

```
+-----+
| `1 signature violation.` |
+-----+
| `Signature violations:` |
| `got "fortytwo" at line 2, column 12, signature... |
+-----+
```

Note という シグネチャ violation does ない stop running プログラム.

```
+-----+
| [構文] |
| (signature signature-form) |
+-----+
```

これ 返す シグネチャ described によって シグネチャ-form として 値.

#### 4.6.1 シグネチャ形式

Any expression can be a signature form, in which case the signature is > the value returned by that expression. There are a few special > signature forms, however: In a signature form, any name that starts with a % is a > signature variable that stands for any signature depending on how > the signature is used. Example:

```
(: same (%a -> %a))

(define (same x) x)

+-----+
| [構文] |
```

```
|
| (input-signature-form... -> output-signature-form) |
+-----+
```

これ シグネチャ form describes 関数 とともに inputs described によって input-シグネチャ-forms と output described によって output-シグネチャ-form.

```
+-----+
| [構文] |
| |
| (enum expr...) |
+-----+
```

これ シグネチャ describes enumeration の 値 s returned によって exprs.

例:

```
(: cute? ((enum "cat" "snake") -> Boolean))
```

```
(define (cute? pet)
 (cond
 [(string=? pet "cat") #t]
 [(string=? pet "snake") #f]))
```

```
+-----+
| [構文] |
| |
| (mixed signature-form...) |
+-----+
```

これ シグネチャ describes mixed data, i.e. itemization where 各 の cases has シグネチャ described によって シグネチャ-form.

例:

```
(define SIGS (signature (mixed Aim Fired)))
```

```
+-----+
| [構文] |
| |
| (ListOf signature-form) |
+-----+
```

これ シグネチャ describes リスト where elements は described によって シグネチャ-form.

```
+-----+
| [構文] |
| |
| (predicate expression) |
+-----+
```

このシグネチャは述語を通じて値を記述します。expression は 1 引数で真偽値を返す関数に評価されなければなりません。シグネチャは、その述語が #true を返すすべての値と一致します。

#### 4.6.2 構造体シグネチャ

A define-struct form defines two additional names that can be used in signatures. For a struct called struct, these are Struct and StructOf. Note that these names are capitalized. In particular, a struct called Struct, will also define Struct and StructOf. Moreover, when forming the additional names, hyphens are removed, and each letter following a hyphen is capitalized - so a struct called foo-bar will define FooBar and FooBarOf. Struct is a signature that describes struct values from this structure type. StructOf is a function that takes as input a signature for each field. It returns a signature describing values of this structure type, additionally describing the values of the fields of the value.

```
(define-struct pair [fst snd])
```

```
(: add-pair ((PairOf Number Number) -> Number))
```

```
(define (add-pair p)
 (+ (pair-fst p) (pair-snd p)))
```

remaining subsections リスト それらの 関数 s という は built-in プログラム ming language. All other 関数 s は imported から teachpack または でなければならぬ defined の中の プログラム.

#### 4.7 数: 整数・有理数・実数・複素数・正確数・非正確数

```
+-----+
| [手続き] |
| |
| (- x y...) → number |
| x: number |
| y: number |
+-----+
```

第 1 の数から第 2 (およびそれ以降) の数を引きます。引数が 1 つだけのときは、その数の符号を反転します。

```
> (- 5)
-5
> (- 5 3)
2
> (- 5 3 1)
1
```

```
+-----+
| [手続き] |
```

```
|
| (< x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

two or more (real) numbers for less-than を比較します。

```
> (< 42 2/5)
#false
```

```
+-----+
| [手続き] |
| (<= x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

two or more (real) numbers for less-than or equality を比較します。

```
> (<= 42 2/5)
#false
```

```
+-----+
| [手続き] |
| (> x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

two or more (real) numbers for greater-than を比較します。

```
> (> 42 2/5)
#true
```

```
+-----+
| [手続き] |
| (>= x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

two or more (real) numbers for greater-than or equality を比較します。

```
> (>= 42 42)
#true
```

```

+-----+
| [手続き] |
| |
| (abs x) → real |
| x: real |
+-----+

```

判定する: absolute 値 の 実数 数.

```

> (abs -12)
12

```

```

+-----+
| [手続き] |
| |
| (acos x) → number |
| x: number |
+-----+

```

the arccosine (inverse of cos) of a number を計算します。

```

> (acos 0)
#i1.5707963267948966

```

```

+-----+
| [手続き] |
| |
| (add1 x) → number |
| x: number |
+-----+

```

与えられた数を 1 増やします。

```

> (add1 2)
3

```

```

+-----+
| [手続き] |
| |
| (angle x) → real |
| x: number |
+-----+

```

the angle from a complex number を取り出します。

```

> (angle (make-polar 3 4))
#i-2.2831853071795867

```

```

+-----+
| [手続き] |
| |
| (asin x) → number |
| x: number |
+-----+

```

the arcsine (inverse of sin) of a number を計算します。

```
> (asin 0)
0
```

```
+-----+
| [手続き] |
| |
| (atan x) → number |
| x: number |
+-----+
```

計算する: arctangent の given 数:

```
> (atan 0)
0
> (atan 0.5)
#i0.4636476090008061
```

Also comes in a two-argument version where (atany x) computes (atan(/yx)) but the signs of y and x determine the quadrant of the result and the result tends to be more accurate than that of the 1-argument version in borderline cases:

```
> (atan 3 4)
#i0.6435011087932844
> (atan -2 -1)
#i-2.0344439357957027
```

```
+-----+
| [手続き] |
| |
| (ceiling x) → integer |
| x: real |
+-----+
```

判定する: closest 整数 (正確 または in 正確) above 実数 数. 参照: round.

```
> (ceiling 12.3)
#i13.0
```

```
+-----+
| [手続き] |
| |
| (complex? x) → boolean? |
| x: any/c |
+-----+
```

some value is complex かどうかを判定します。

```
> (complex? 1-2i)
#true
```

```
+-----+
| [手続き] |
| |
```

```

|
| (conjugate x) → number |
| x: number |
+-----+

```

複素数の虚部の符号を反転します。

```

> (conjugate 3+4i)
3-4i
> (conjugate -2-5i)
-2+5i
> (conjugate (make-polar 3 4))
#i-1.960930862590836+2.2704074859237844i

```

```

+-----+
| [手続き] |
| |
| (cos x) → number |
| x: number |
+-----+

```

the cosine of a number (radians) を計算します。

```

> (cos pi)
#i-1.0

```

```

+-----+
| [手続き] |
| |
| (cosh x) → number |
| x: number |
+-----+

```

the hyperbolic cosine of a number を計算します。

```

> (cosh 10)
#i11013.232920103324

```

```

+-----+
| [手続き] |
| |
| (current-seconds) → integer |
+-----+

```

判定する: current time の中の seconds elapsed (since platform-specific starting date).

```

> (current-seconds)
1779843339

```

```

+-----+
| [手続き] |
| |
| (denominator x) → integer |
+-----+

```



```
| x: rational? |
+-----+
```

the denominator of a rational を計算します。

```
> (denominator 2/3)
3
```

```
+-----+
| [値] |
| e: real |
+-----+
```

Euler's 数.

```
> e
#i2.718281828459045
```

```
+-----+
| [手続き] |
| (even? x) → boolean? |
| x: integer |
+-----+
```

some integer (exact or inexact) is even or not かどうかを判定します。

```
> (even? 2)
#true
```

```
+-----+
| [手続き] |
| (exact->inexact x) → number |
| x: number |
+-----+
```

an exact number to an inexact one を変換します。

```
> (exact->inexact 12)
#i12.0
```

```
+-----+
| [手続き] |
| (exact? x) → boolean? |
| x: number |
+-----+
```

some number is exact かどうかを判定します。

```
> (exact? (sqrt 2))
#false
```

```

+-----+
| [手続き] |
| |
| (exp x) → number |
| x: number |
+-----+

```

判定する:  $e$  raised  $\wedge$  数.

```

> (exp -2)
#i0.1353352832366127

```

```

+-----+
| [手続き] |
| |
| (expt x y) → number |
| x: number |
| y: number |
+-----+

```

the power of the first to the second number, which is to say, exponentiation を計算します。

```

> (expt 16 1/2)
4
> (expt 3 -4)
1/81

```

```

+-----+
| [手続き] |
| |
| (floor x) → integer |
| x: real |
+-----+

```

判定する: closest 整数 (正確 または in 正確) below 実数 数. 参照: round.

```

> (floor 12.3)
#i12.0

```

```

+-----+
| [手続き] |
| |
| (gcd x y...) → integer |
| x: integer |
| y: integer |
+-----+

```

判定する: greatest common divisor の two 整数 s (正確 または in 正確).

```

> (gcd 6 12 8)
2

```

```

+-----+
| [手続き] |
| |
| (imag-part x) → real |
| x: number |
+-----+

```

the imaginary part from a complex number を取り出します。

```

> (imag-part 3+4i)
4

```

```

+-----+
| [手続き] |
| |
| (inexact->exact x) → number |
| x: number |
+-----+

```

不正確数を正確数で近似します。

```

> (inexact->exact 12.0)
12

```

```

+-----+
| [手続き] |
| |
| (inexact? x) → boolean? |
| x: number |
+-----+

```

some number is inexact かどうかを判定します。

```

> (inexact? 1-2i)
#false

```

```

+-----+
| [手続き] |
| |
| (integer->char x) → char |
| x: exact-integer? |
+-----+

```

与えられた正確整数に対応する文字を ASCII 表から調べます（あれば）。

```

> (integer->char 42)
#*

```

```

+-----+
| [手続き] |
| |
| (integer-sqrt x) → complex |
| x: integer |
+-----+

```

the integer or imaginary-integer square root of an integer を計算します。

```
> (integer-sqrt 11)
```

```
3
```

```
> (integer-sqrt -11)
```

```
0+3i
```

```
+-----+
| [手続き] |
| |
| (integer? x) → boolean? |
| x: any/c |
+-----+
```

some value is an integer (exact or inexact) かどうかを判定します。

```
> (integer? (sqrt 2))
```

```
#false
```

```
+-----+
| [手続き] |
| |
| (lcm x y...) → integer |
| x: integer |
| y: integer |
+-----+
```

判定する: least common multiple の two 整数 s (正確 または in 正確).

```
> (lcm 6 12 8)
```

```
24
```

```
+-----+
| [手続き] |
| |
| (log x) → number |
| x: number |
+-----+
```

判定する: base-e logarithm の 数.

```
> (log 12)
```

```
#i2.4849066497880004
```

```
+-----+
| [手続き] |
| |
| (magnitude x) → real |
| x: number |
+-----+
```

判定する: magnitude の 複素数 数.

```
> (magnitude (make-polar 3 4))
```

```
#i2.9999999999999996
```

```

+-----+
| [手続き] |
| |
| (make-polar x y) → number |
| x: real |
| y: real |
+-----+

```

a complex from a magnitude and angle を作成します。

```

> (make-polar 3 4)
#i-1.960930862590836-2.2704074859237844i

```

```

+-----+
| [手続き] |
| |
| (make-rectangular x y) → number |
| x: real |
| y: real |
+-----+

```

a complex from a real and an imaginary part を作成します。

```

> (make-rectangular 3 4)
3+4i

```

```

+-----+
| [手続き] |
| |
| (max x y...) → real |
| x: real |
| y: real |
+-----+

```

判定する: largest 数—aka, maximum.

```

> (max 3 2 8 7 2 9 0)
9

```

```

+-----+
| [手続き] |
| |
| (min x y...) → real |
| x: real |
| y: real |
+-----+

```

判定する: smallest 数—aka, minimum.

```

> (min 3 2 8 7 2 9 0)
0

```

```

+-----+
| [手続き] |
| |

```

```

|
| (modulo x y) → integer |
| x: integer |
| y: integer |
+-----+

```

Finds remainder of the division of the first number by the second:

```
> (modulo 9 2)
```

```
1
```

```
> (modulo 3 -4)
```

```
-1
```

```

+-----+
| [手続き] |
|
| (negative? x) → boolean? |
| x: real |
+-----+

```

some real number is strictly smaller than zeroかどうかを判定します。

```
> (negative? -2)
```

```
#true
```

```

+-----+
| [手続き] |
|
| (number->string x) → string |
| x: number |
+-----+

```

a number to a string を変換します。

```
> (number->string 42)
```

```
"42"
```

```

+-----+
| [手続き] |
|
| (number->string-digits x p) → string |
| x: number |
| p: posint |
+-----+

```

a number x to a string with the specified number of digits を変換します。

```
> (number->string-digits 0.9 2)
```

```
"0.9"
```

```
> (number->string-digits pi 4)
```

```
"3.1416"
```

```

+-----+
| [手続き] |

```

```

|
| (number? n) → boolean? |
| n: any/c |
+-----+

```

ある値が数かどうかを判定します。

```

> (number? "hello world")
#false
> (number? 42)
#true

```

```

+-----+
| [手続き] |
| |
| (numerator x) → integer |
| x: rational? |
+-----+

```

the numerator of a rational を計算します。

```

> (numerator 2/3)
2

```

```

+-----+
| [手続き] |
| |
| (odd? x) → boolean? |
| x: integer |
+-----+

```

some integer (exact or inexact) is odd or not かどうかを判定します。

```

> (odd? 2)
#false

```

```

+-----+
| [値] |
| |
| pi: real |
+-----+

```

ratio の circle's circumference へ その diameter.

```

> pi
#i3.141592653589793

```

```

+-----+
| [手続き] |
| |
| (positive? x) → boolean? |
| x: real |
+-----+

```

some real number is strictly larger than zero かどうかを判定します。

```
> (positive? -2)
#false
```

```
+-----+
| [手続き] |
| |
| (quotient x y) → integer |
| x: integer |
| y: integer |
+-----+
```

Divides first 整数—also called dividend—by second—known として divisor—to obtain quotient.

```
> (quotient 9 2)
4
> (quotient 3 4)
0
```

```
+-----+
| [手続き] |
| |
| (random x) → natural?
| x: (and/c natural? positive?) |
+-----+
```

Generates 乱数 natural 数 less than ある given 正確 natural.

```
> (random 42)
27
```

```
+-----+
| [手続き] |
| |
| (rational? x) → boolean? |
| x: any/c |
+-----+
```

some value is a rational number かどうかを判定します。

```
> (rational? 1)
#true
> (rational? -2.349)
#true
> (rational? #i1.23456789)
#true
> (rational? (sqrt -1))
#false
> (rational? pi)
#true
> (rational? e)
#true
```



```
> (rational? 1-2i)
#false
```

As the interactions show, the teaching languages considers many more numbers as rationals than expected. In particular, pi is a rational number because it is only a finite approximation to the mathematical  $\pi$ . Think of rational? as a suggestion to think of these numbers as fractions.

```
+-----+
| [手続き] |
| |
| (real-part x) → real |
| x: number |
+-----+
```

the real part from a complex number を取り出します。

```
> (real-part 3+4i)
3
```

```
+-----+
| [手続き] |
| |
| (real? x) → boolean? |
| x: any/c |
+-----+
```

some value is a real numberかどうかを判定します。

```
> (real? 1-2i)
#false
```

```
+-----+
| [手続き] |
| |
| (remainder x y) → integer |
| x: integer |
| y: integer |
+-----+
```

判定する: remainder の dividing first によって second 整数 (正確 または in 正確).

```
> (remainder 9 2)
```

```
1
```

```
> (remainder 3 4)
```

```
3
```

```
+-----+
| [手続き] |
| |
| (round x) → integer |
| x: real |
+-----+
```

実数を整数に丸めます（同点のときは偶数へ丸めます）。floor と ceiling も参照してください。

```
> (round 12.3)
#i12.0
```

```
+-----+
| [手続き] |
| |
| (sgn x) → (union 1 #i1.0 0 #i0.0 -1 #i-1.0) |
| x: real |
+-----+
```

判定する: sign の実数数.

```
> (sgn -12)
-1
```

```
+-----+
| [手続き] |
| |
| (sin x) → number |
| x: number |
+-----+
```

the sine of a number (radians) を計算します。

```
> (sin pi)
#i1.2246467991473532e-16
```

```
+-----+
| [手続き] |
| |
| (sinh x) → number |
| x: number |
+-----+
```

the hyperbolic sine of a number を計算します。

```
> (sinh 10)
#i11013.232874703393
```

```
+-----+
| [手続き] |
| |
| (sqr x) → number |
| x: number |
+-----+
```

the square of a number を計算します。

```
> (sqr 8)
64
```

```

+-----+
| [手続き] |
| |
| (sqrt x) → number |
| x: number |
+-----+

```

the square root of a number を計算します。

```

> (sqrt 9)
3
> (sqrt 2)
#i1.4142135623730951

```

```

+-----+
| [手続き] |
| |
| (sub1 x) → number |
| x: number |
+-----+

```

与えられた数を 1 減らします。

```

> (sub1 2)
1

```

```

+-----+
| [手続き] |
| |
| (tan x) → number |
| x: number |
+-----+

```

the tangent of a number (radians) を計算します。

```

> (tan pi)
#i-1.2246467991473532e-16

```

```

+-----+
| [手続き] |
| |
| (zero? x) → boolean? |
| x: number |
+-----+

```

some number is zero or not かどうかを判定します。

```

> (zero? 2)
#false

```

## 4.8 真偽値

```

+-----+
| [手続き] |
| |
| |
+-----+

```

```
| (boolean->string x) → string |
| x: boolean? |
+-----+
```

与えられた真偽値の文字列を生成します。

```
> (boolean->string #false)
"#false"
> (boolean->string #true)
"#true"
```

```
+-----+
| [手続き] |
| |
| (boolean=? x y) → boolean? |
| x: boolean? |
| y: boolean? |
+-----+
```

two booleans are equalかどうかを判定します。

```
> (boolean=? #true #false)
#false
```

```
+-----+
| [手続き] |
| |
| (boolean? x) → boolean? |
| x: any/c |
+-----+
```

値が真偽値かどうかを判定します。

```
> (boolean? 42)
#false
> (boolean? #false)
#true
```

```
+-----+
| [手続き] |
| |
| (false? x) → boolean? |
| x: any/c |
+-----+
```

a value is falseかどうかを判定します。

```
> (false? #false)
#true
```

```
+-----+
| [手続き] |
| |
| (not x) → boolean? |
+-----+
```

```
| x: boolean? |
+-----+
```

真偽値を否定します。

```
> (not #false)
#true
```

## 4.9 シンボル

```
+-----+
| [手続き] |
| |
| (symbol->string x) → string |
| x: symbol |
+-----+
```

a symbol to a string を変換します。

```
> (symbol->string 'c)
"c"
```

```
+-----+
| [手続き] |
| |
| (symbol=? x y) → boolean? |
| x: symbol |
| y: symbol |
+-----+
```

two symbols are equalかどうかを判定します。

```
> (symbol=? 'a 'b)
#false
```

```
+-----+
| [手続き] |
| |
| (symbol? x) → boolean? |
| x: any/c |
+-----+
```

値がシンボルかどうかを判定します。

```
> (symbol? 'a)
#true
```

## 4.10 リスト

```
+-----+
| [手続き] |
| |
| (append l...) → (listof any) |
| l: (listof any) |
+-----+
```

複数のリストから、要素を連結して1つのリストを作ります。ISL 以上では、append は1つのリストや0個のリストにも適用できます。

```
> (append (cons 1 (cons 2 '())) (cons "a" (cons "b" '())))
(list 1 2 "a" "b")
> (append)
'()
```

```
+-----+
| [手続き] |
| |
| (assoc x l) → (union (listof any) #false) |
| x: any/c |
| l: (listof any) |
+-----+
```

the first pair on l whose first is equal? to x; otherwise it produces #false を生成します。

```
> (assoc "hello" '(("world" 2) ("hello" 3) ("good" 0)))
(list "hello" 3)
```

```
+-----+
| [手続き] |
| |
| (assq x l) → (union #false cons?) |
| x: any/c |
| l: list? |
+-----+
```

ある要素が、対のリストの中のある対の先頭要素かどうかを判定します (要素は eq? で比較します)。

```
> a
(list (list 'a 22) (list 'b 8) (list 'c 70))
> (assq 'b a)
(list 'b 8)
```

```
+-----+
| [手続き] |
| |
| (caaar x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (car(car(carx)))。

```
> w
(list (list (list (list "bye") 3) #true) 42)
> (caaar w)
(list "bye")
```

```
+-----+
| [手続き] |
| |
```

```

| |
| (caadr x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(car(cdrx)))。

```

> (caadr (cons 1 (cons (cons 'a '()) (cons (cons 'd '()) '()))))
'a

```

```

+-----+
| [手続き] |
| |
| (caar x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(carx))。

```

> y
(list (list (list 1 2 3) #false "world"))
> (caar y)
(list 1 2 3)

```

```

+-----+
| [手続き] |
| |
| (cadar x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(cdr(carx)))。

```

> w
(list (list (list (list "bye") 3) #true) 42)
> (cadar w)
#true

```

```

+-----+
| [手続き] |
| |
| (caddr x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(cdr(cdr(cdrx))))。

```

> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (caddr v)
4

```

```

+-----+
| [手続き] |
| |

```

```

| |
| (caddr x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(cdr(cdrx)))。

```

> x
(list 2 "hello" #true)
> (caddr x)
#true

```

```

+-----+
| [手続き] |
| |
| (cadr x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(cdrx))。

```

> x
(list 2 "hello" #true)
> (cadr x)
"hello"

```

```

+-----+
| [手続き] |
| |
| (car x) → any/c |
| x: cons? |
+-----+

```

空でないリストの先頭要素を取り出します。

```

> x
(list 2 "hello" #true)
> (car x)
2

```

```

+-----+
| [手続き] |
| |
| (cdaar x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (cdr(car(carx)))。

```

> w
(list (list (list (list "bye") 3) #true) 42)
> (cdaar w)
(list 3)

```



```
+-----+
| [手続き] |
| |
| (cdadr x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (cdr(car(cdrx)))。

```
> (cdadr (list 1 (list 2 "a") 3))
(list "a")
```

```
+-----+
| [手続き] |
| |
| (cdar x) → list? |
| x: list? |
+-----+
```

LISP 風の selector: (cdr(carx))。

```
> y
(list (list (list 1 2 3) #false "world"))
> (cdar y)
(list #false "world")
```

```
+-----+
| [手続き] |
| |
| (cddar x) → any/c |
| x: list? |
+-----+
```

LISP 風の選択子: (cdr (cdr (car x)))

```
> w
(list (list (list (list "bye") 3) #true) 42)
> (cddar w)
'()
```

```
+-----+
| [手続き] |
| |
| (cdddr x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (cdr(cdr(cdrx)))。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (cdddr v)
(list 4 5 6 7 8 9 'A)
```

```

+-----+
| [手続き] |
| |
| (cddr x) → list? |
| x: list? |
+-----+

```

LISP 風の selector: (cdr(cdrx))。

```

> x
(list 2 "hello" #true)
> (cddr x)
(list #true)

```

```

+-----+
| [手続き] |
| |
| (cdr x) → any/c |
| x: cons? |
+-----+

```

空でないリストの残り (rest) を取り出します。

```

> x
(list 2 "hello" #true)
> (cdr x)
(list "hello" #true)

```

```

+-----+
| [手続き] |
| |
| (cons x y) → list? |
| x: any/c |
| y: list? |
+-----+

```

a list を構築します。

```

> (cons 1 '())
(cons 1 '())

```

```

+-----+
| [手続き] |
| |
| (cons? x) → boolean? |
| x: any/c |
+-----+

```

some value is a constructed list かどうかを判定します。

```

> (cons? (cons 1 '()))
#true
> (cons? 42)
#false

```

```
+-----+
| [手続き] |
| |
| (eighth x) → any/c |
| x: list? |
+-----+
```

the eighth item of a non-empty list を選択します。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (eighth v)
8
```

```
+-----+
| [手続き] |
| |
| (empty? x) → boolean? |
| x: any/c |
+-----+
```

値が空リストかどうかを判定します。

```
> (empty? '())
#true
> (empty? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (fifth x) → any/c |
| x: list? |
+-----+
```

the fifth item of a non-empty list を選択します。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (fifth v)
5
```

```
+-----+
| [手続き] |
| |
| (first x) → any/c |
| x: cons? |
+-----+
```

空でないリストの先頭要素を取り出します。

```
> x
(list 2 "hello" #true)
```

```
> (first x)
```

```
2
```

```
+-----+
| [手続き] |
| |
| (fourth x) → any/c |
| x: list? |
+-----+
```

the fourth item of a non-empty list を選択します。

```
> v
```

```
(list 1 2 3 4 5 6 7 8 9 'A)
```

```
> (fourth v)
```

```
4
```

```
+-----+
| [手続き] |
| |
| (length l) → natural? |
| l: list? |
+-----+
```

the number of items on a list を評価します。

```
> x
```

```
(list 2 "hello" #true)
```

```
> (length x)
```

```
3
```

```
+-----+
| [手続き] |
| |
| (list x...) → list? |
| x: any/c |
+-----+
```

a list of its arguments を構築します。

```
> (list 1 2 3 4 5 6 7 8 9 0)
```

```
(cons 1 (cons 2 (cons 3 (cons 4 (cons 5 (cons 6 (cons 7 (cons 8 (cons 9 (cons 0 '()))))))))))))
```

```
+-----+
| [手続き] |
| |
| (list* x... l) → list? |
| x: any/c |
| l: list? |
+-----+
```

a list by adding multiple items to a list を構築します。

```
> x
(list 2 "hello" #true)
> (list* 4 3 x)
(list 4 3 2 "hello" #true)
```

```
+-----+
| [手続き] |
| |
| (list-ref x i) → any/c |
| x: list? |
| i: natural? |
+-----+
```

the indexed item from the list を取り出します。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (list-ref v 9)
'A
```

```
+-----+
| [手続き] |
| |
| (list? x) → boolean? |
| x: any/c |
+-----+
```

whether the given value is a list を検査します。

```
> (list? 42)
#false
> (list? '())
#true
> (list? (cons 1 (cons 2 '())))
#true
```

```
+-----+
| [手続き] |
| |
| (make-list i x) → list? |
| i: natural? |
| x: any/c |
+-----+
```

a list of i copies of x を構築します。

```
> (make-list 3 "hello")
(cons "hello" (cons "hello" (cons "hello" '())))
```

```
+-----+
| [手続き] |
| |
| (member x l) → boolean? |
| x: any/c |
+-----+
```

```
| l: list? |
+-----+
```

値がリスト上にあるかどうかを判定します (equal? で比較)。

```
> x
(list 2 "hello" #true)
> (member "hello" x)
#true

+-----+
| [手続き] |
| |
| (member? x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

値がリスト上にあるかどうかを判定します (equal? で比較)。

```
> x
(list 2 "hello" #true)
> (member? "hello" x)
#true

+-----+
| [手続き] |
| |
| (memq x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

some value x is on some list l, using eq? to compare x with items on l かどうかを判定します。

```
> x
(list 2 "hello" #true)
> (memq (list (list 1 2 3)) x)
#false

+-----+
| [手続き] |
| |
| (memq? x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

some value x is on some list l, using eq? to compare x with items on l かどうかを判定します。

```
> x
(list 2 "hello" #true)
> (memq? (list (list 1 2 3)) x)
#false
```

```
+-----+
| [手続き] |
| |
| (memv x l) → (or/c #false list) |
| x: any/c |
| l: list? |
+-----+
```

ある値がリスト上にあるかどうかを判定し、ある場合は x から始まるリストの接尾辞を生成し、ない場合は false を生成します（値は eqv? 述語で比較します）。

```
> x
(list 2 "hello" #true)
> (memv (list (list 1 2 3)) x)
#false
```

```
+-----+
| [値] |
| |
| null: list |
+-----+
```

空リストの別名です。

```
> null
'()
```

```
+-----+
| [手続き] |
| |
| (null? x) → boolean? |
| x: any/c |
+-----+
```

値が空リストかどうかを判定します。

```
> (null? '())
#true
> (null? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (range start end step) → list? |
| start: number |
| end: number |
+-----+
```

```
| step: number |
+-----+
```

a list of numbers by stepping from start to end を構築します。

```
> (range 0 10 2)
(cons 0 (cons 2 (cons 4 (cons 6 (cons 8 '())))))
```

```
+-----+
| [手続き] |
| |
| (remove x l) → list? |
| x: any/c |
| l: list? |
+-----+
```

a list like the given one, with the first occurrence of the given item removed (comparing values with equal?) を構築します。

```
> x
(list 2 "hello" #true)
> (remove "hello" x)
(list 2 #true)
> hello-2
(list 2 "hello" #true "hello")
> (remove "hello" hello-2)
(list 2 #true "hello")
```

```
+-----+
| [手続き] |
| |
| (remove-all x l) → list? |
| x: any/c |
| l: list? |
+-----+
```

a list like the given one, with all occurrences of the given item removed (comparing values with equal?) を構築します。

```
> x
(list 2 "hello" #true)
> (remove-all "hello" x)
(list 2 #true)
> hello-2
(list 2 "hello" #true "hello")
> (remove-all "hello" hello-2)
(list 2 #true)
```

```
+-----+
| [手続き] |
| |
| (rest x) → any/c |
+-----+
```



```
| x: cons? |
+-----+
```

空でないリストの残り (rest) を取り出します。

```
> x
(list 2 "hello" #true)
> (rest x)
(list "hello" #true)
+-----+
| [手続き] |
| |
| (reverse l) → list |
| l: list? |
+-----+
```

a reversed version of a list を作成します。

```
> x
(list 2 "hello" #true)
> (reverse x)
(list #true "hello" 2)
+-----+
| [手続き] |
| |
| (second x) → any/c |
| x: list? |
+-----+
```

the second item of a non-empty list を選択します。

```
> x
(list 2 "hello" #true)
> (second x)
"hello"
+-----+
| [手続き] |
| |
| (seventh x) → any/c |
| x: list? |
+-----+
```

the seventh item of a non-empty list を選択します。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (seventh v)
7
+-----+
| [手続き] |
```

```

|
| (sixth x) → any/c |
| x: list? |
+-----+

```

the sixth item of a non-empty list を選択します。

```

> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (sixth v)
6

```

```

+-----+
| [手続き] |
|
| (third x) → any/c |
| x: list? |
+-----+

```

the third item of a non-empty list を選択します。

```

> x
(list 2 "hello" #true)
> (third x)
#true

```

#### 4.11 Posn

```

+-----+
| [手続き] |
|
| (make-posn x y) → posn |
| x: any/c |
| y: any/c |
+-----+

```

a posn from two arbitrary values を構築します。

```

> (make-posn 3 3)
(make-posn 3 3)
> (make-posn "hello" #true)
(make-posn "hello" #true)

```

```

+-----+
| [手続き] |
|
| (posn-x p) → any/c |
| p: posn |
+-----+

```

the x component of a posn を取り出します。

```

> p
(make-posn 2 -3)

```

```
> (posn-x p)
```

```
2
```

```
+-----+
| [手続き] |
| |
| (posn-y p) → any/c |
| p: posn |
+-----+
```

the y component of a posn を取り出します。

```
> p
```

```
(make-posn 2 -3)
```

```
> (posn-y p)
```

```
-3
```

```
+-----+
| [手続き] |
| |
| (posn? x) → boolean? |
| x: any/c |
+-----+
```

its input is a posn かどうかを判定します。

```
> q
```

```
(make-posn "bye" 2)
```

```
> (posn? q)
```

```
#true
```

```
> (posn? 42)
```

```
#false
```

## 4.12 文字

```
+-----+
| [手続き] |
| |
| (char->integer c) → integer |
| c: char |
+-----+
```

与えられた文字に対応する数を ASCII 表から調べます (あれば)。

```
> (char->integer #\a)
```

```
97
```

```
> (char->integer #\z)
```

```
122
```

```
+-----+
| [手続き] |
| |
| (char-alphabetic? c) → boolean? |
+-----+
```

```
| c: char |
+-----+
```

a character represents an alphabetic characterかどうかを判定します。

```
> (char-alphabetic? #\Q)
#true
```

```
+-----+
| [手続き] |
| |
| (char-ci<=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are ordered in an increasing and case-insensitive mannerかどうかを判定します。

```
> (char-ci<=? #\b #\B)
#true
> (char<=? #\b #\B)
#false
```

```
+-----+
| [手続き] |
| |
| (char-ci<? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are ordered in a strictly increasing and case-insensitive mannerかどうかを判定します。

```
> (char-ci<? #\B #\c)
#true
> (char<? #\b #\B)
#false
```

```
+-----+
| [手続き] |
| |
| (char-ci=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

two characters are equal in a case-insensitive mannerかどうかを判定します。

```
> (char-ci=? #\b #\B)
#true
```

```
+-----+
| [手続き] |
| (char-ci>=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are sorted in a decreasing and case-insensitive mannerかどうかを判定します。

```
> (char-ci>=? #\b #\C)
#false
> (char>=? #\b #\C)
#true
```

```
+-----+
| [手続き] |
| (char-ci>? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are sorted in a strictly decreasing and case-insensitive mannerかどうかを判定します。

```
> (char-ci>? #\b #\B)
#false
> (char>? #\b #\B)
#true
```

```
+-----+
| [手続き] |
| (char-downcase c) → char |
| c: char |
+-----+
```

the equivalent lower-case character を生成します。

```
> (char-downcase #\T)
#\t
```

```
+-----+
| [手続き] |
| (char-lower-case? c) → boolean? |
+-----+
```

```
| c: char |
+-----+
```

a character is a lower-case characterかどうかを判定します。

```
> (char-lower-case? #\T)
#false
```

```
+-----+
| [手続き] |
| |
| (char-numeric? c) → boolean? |
| c: char |
+-----+
```

a character represents a digitかどうかを判定します。

```
> (char-numeric? #\9)
#true
```

```
+-----+
| [手続き] |
| |
| (char-upcase c) → char |
| c: char |
+-----+
```

the equivalent upper-case characterを生成します。

```
> (char-upcase #\t)
#\T
```

```
+-----+
| [手続き] |
| |
| (char-upper-case? c) → boolean? |
| c: char |
+-----+
```

a character is an upper-case characterかどうかを判定します。

```
> (char-upper-case? #\T)
#true
```

```
+-----+
| [手続き] |
| |
| (char-whitespace? c) → boolean? |
| c: char |
+-----+
```

a character represents spaceかどうかを判定します。

```
> (char-whitespace? #\tab)
#true
```

```
+-----+
| [手続き] |
| (char<=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are ordered in an increasing mannerかどうかを判定します。

```
> (char<=? #\a #\a #\b)
#true
```

```
+-----+
| [手続き] |
| (char<? x d e...) → boolean? |
| x: char |
| d: char |
| e: char |
+-----+
```

the characters are ordered in a strictly increasing mannerかどうかを判定します。

```
> (char<? #\a #\b #\c)
#true
```

```
+-----+
| [手続き] |
| (char=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are equalかどうかを判定します。

```
> (char=? #\b #\a)
#false
```

```
+-----+
| [手続き] |
| (char>=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are sorted in a decreasing mannerかどうかを判定します。

```
> (char>=? #\b #\b #\a)
#true
```

```
+-----+
| [手続き] |
| |
| (char>? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are sorted in a strictly decreasing mannerかどうかを判定します。

```
> (char>? #\A #\z #\a)
#false
```

```
+-----+
| [手続き] |
| |
| (char? x) → boolean? |
| x: any/c |
+-----+
```

a value is a characterかどうかを判定します。

```
> (char? "a")
#false
> (char? #\a)
#true
```

#### 4.13 文字列

```
+-----+
| [手続き] |
| |
| (explode s) → (listof string) |
| s: string |
+-----+
```

Translates 文字列へ リストの 1-letter 文字列 s.

```
> (explode "cat")
(list "c" "a" "t")
```

```
+-----+
| [手続き] |
| |
| (format f x...) → string |
| f: string |
| x: any/c |
+-----+
```

Formats 文字列, possibly embedding 値 s.



```
> (format "Dear Dr. ~a:" "Flatt")
"Dear Dr. Flatt:"
> (format "Dear Dr. ~s:" "Flatt")
"Dear Dr. \"Flatt\":"
```

```
+-----+
| [手続き] |
| |
| (implode l) → string |
| l: list? |
+-----+
```

Concatenates リスト の 1-letter 文字列 s へ one 文字列.

```
> (implode (cons "c" (cons "a" (cons "t" '()))))
"cat"
```

```
+-----+
| [手続き] |
| |
| (int->string i) → string |
| i: integer |
+-----+
```

an integer in [0,55295] or [57344 111411] to a 1-letter string を変換します。

```
> (int->string 65)
"A"
```

```
+-----+
| [手続き] |
| |
| (list->string l) → string |
| l: list? |
+-----+
```

a s list of characters into a string を変換します。

```
> (list->string (cons #\c (cons #\a (cons #\t '()))))
"cat"
```

```
+-----+
| [手続き] |
| |
| (make-string i c) → string |
| i: natural? |
| c: char |
+-----+
```

a string of length i from c を生成します。

```
> (make-string 3 #\d)
"ddd"
```

```

+-----+
| [手続き] |
| |
| (replicate i s) → string |
| i: natural? |
| s: string |
+-----+

```

Replicates s i times.

```

> (replicate 3 "h")
"hhh"

```

```

+-----+
| [手続き] |
| |
| (string c...) → string? |
| c: char |
+-----+

```

Builds 文字列 of the given 文字 s.

```

> (string #\d #\o #\g)
"dog"

```

```

+-----+
| [手続き] |
| |
| (string->int s) → integer |
| s: string |
+-----+

```

a 1-letter string to an integer in [0,55295] or [57344, 1114111] を変換します。

```

> (string->int "a")
97

```

```

+-----+
| [手続き] |
| |
| (string->list s) → (listof char) |
| s: string |
+-----+

```

a string into a list of characters を変換します。

```

> (string->list "hello")
(list #\h #\e #\l #\l #\o)

```

```

+-----+
| [手続き] |
| |
| (string->number s) → (union number #false) |
+-----+

```

```
| s: string |
+-----+
```

a string into a number, produce false if impossible を変換します。

```
> (string->number "-2.03")
-2.03
> (string->number "1-2i")
1-2i
```

```
+-----+
| [手続き] |
| |
| (string->symbol s) → symbol |
| s: string |
+-----+
```

a string into a symbol を変換します。

```
> (string->symbol "hello")
'hello
```

```
+-----+
| [手続き] |
| |
| (string-alphabetic? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are alphabeticかどうかを判定します。

```
> (string-alphabetic? "123")
#false
> (string-alphabetic? "cat")
#true
```

```
+-----+
| [手続き] |
| |
| (string-contains-ci? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

the first string appears in the second one without regard to the case of the lettersかどうかを判定します。

```
> (string-contains-ci? "At" "caT")
#true
```

```
+-----+
| [手続き] |
| |
| (string-contains? s t) → boolean? |
+-----+
```

```

| s: string |
| t: string |
+-----+

```

the first string appears literally in the second oneかどうかを判定します。

```

> (string-contains? "at" "cat")
#true

```

```

+-----+
| [手続き] |
| |
| (string-copy s) → string |
| s: string |
+-----+

```

Copies 文字列.

```

> (string-copy "hello")
"hello"

```

```

+-----+
| [手続き] |
| |
| (string-downcase s) → string |
| s: string |
+-----+

```

a string like the given one with all ‘letters’ as lower case を生成します。

```

> (string-downcase "CAT")
"cat"
> (string-downcase "cAt")
"cat"

```

```

+-----+
| [手続き] |
| |
| (string-ith s i) → 1string? |
| s: string |
| i: natural? |
+-----+

```

the ith 1-letter substring from s を取り出します。

```

> (string-ith "hello world" 1)
"e"

```

```

+-----+
| [手続き] |
| |
| (string-length s) → nat |
| s: string |
+-----+

```

判定する: length の 文字列.

```
> (string-length "hello world")
11
```

```
+-----+
| [手続き] |
| |
| (string-lower-case? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are lower case かどうかを判定します。

```
> (string-lower-case? "CAT")
#false
```

```
+-----+
| [手続き] |
| |
| (string-numeric? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are numeric かどうかを判定します。

```
> (string-numeric? "123")
#true
> (string-numeric? "1-2i")
#false
```

```
+-----+
| [手続き] |
| |
| (string-ref s i) → char |
| s: string |
| i: natural? |
+-----+
```

the ith character from s を取り出します。

```
> (string-ref "cat" 2)
#\t
```

```
+-----+
| [手続き] |
| |
| (string-upcase s) → string |
| s: string |
+-----+
```

a string like the given one with all 'letters' as upper case を生成します。

```
> (string-upcase "cat")
"CAT"
```

```
> (string-upcase "cAt")
"CAT"
```

```
+-----+
| [手続き] |
| |
| (string-upper-case? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are upper caseかどうかを判定します。

```
> (string-upper-case? "CAT")
#true
```

```
+-----+
| [手続き] |
| |
| (string-whitespace? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are white spaceかどうかを判定します。

```
> (string-whitespace? (string-append " " (string #\tab #\newline #\return)))
#true
```

```
+-----+
| [手続き] |
| |
| (string? x) → boolean? |
| x: any/c |
+-----+
```

a value is a stringかどうかを判定します。

```
> (string? "hello world")
#true
> (string? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (substring s i j) → string |
| s: string |
| i: natural? |
| j: natural? |
+-----+
```

the substring starting at i up to j (or the end if j is not provided) を取り出します。

```
> (substring "hello world" 1 5)
"ello"
```

```
> (substring "hello world" 1 8)
"ello wo"
> (substring "hello world" 4)
"o world"
```

#### 4.14 画像

```
+-----+
| [手続き] |
| |
| (image=? i j) → boolean? |
| i: image |
| j: image |
+-----+
```

two images are equalかどうかを判定します。

```
> c1
[image:pict_10.png]
> (image=? (circle 5 "solid" "green") c1)
#false
> (image=? (circle 10 "solid" "green") c1)
#true
```

```
+-----+
| [手続き] |
| |
| (image? x) → boolean? |
| x: any/c |
+-----+
```

a value is an imageかどうかを判定します。

```
> c1
[image:pict_11.png]
> (image? c1)
#true
```

#### 4.15 その他

```
+-----+
| [手続き] |
| |
| (≈ x y eps) → boolean? |
| x: number |
| y: number |
| eps: non-negative-real |
+-----+
```

whether x and y are within eps of either other を検査します。

```
> (≈ 1.01 1.0 0.1)
#true
```

```
> (= 1.01 1.5 0.1)
#false
```

```
+-----+
| [値] |
| |
| eof: eof-object? |
+-----+
```

A 値 という represents end の file:

```
> eof
#<eof>
```

```
+-----+
| [手続き] |
| |
| (eof-object? x) → boolean? |
| x: any/c |
+-----+
```

some value is the end-of-file value かどうかを判定します。

```
> (eof-object? eof)
#true
> (eof-object? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (eq? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+
```

two values are equivalent from the computer's perspective (intensional) かどうかを判定します。

```
> (eq? (cons 1 '()) (cons 1 '()))
#false
> one
(list 1)
> (eq? one one)
#true
```

```
+-----+
| [手続き] |
| |
| (equal? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+
```



two values are structurally equal where basic values are compared with the `eqv?` predicateかどうかを判定します。

```
> (equal? (make-posn 1 2) (make-posn (- 2 1) (+ 1 1)))
#true
```

```
+-----+
| [手続き] |
| |
| (equal~? x y z) → boolean? |
| x: any/c |
| y: any/c |
| z: non-negative-real |
+-----+
```

`x` and `y` like `equal?` but uses `=~` in the case of numbers を比較します。

```
> (equal~? (make-posn 1.01 1.0) (make-posn 1.01 0.99) 0.2)
#true
```

```
+-----+
| [手続き] |
| |
| (eqv? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+
```

two values are equivalent from the perspective of all functions that can be applied to it (extensional)かどうかを判定します。

```
> (eqv? (cons 1 '()) (cons 1 '()))
#false
> one
(list 1)
> (eqv? one one)
#true
```

```
+-----+
| [手続き] |
| |
| (error x...) → void? |
| x: any/c |
+-----+
```

Signals an error, combining the given values into an error message. If any of the values' printed representations is too long, it is truncated and “...” is put into the string. If the first value is a symbol, it is suffixed with a colon and the result pre-pended on to the error message.

```
> zero
0
```

```
> (if (= zero 0) (error "can't divide by 0") (/ 1 zero))
can't divide by 0
```

```
+-----+
| [手続き] |
| |
| (exit) → void |
+-----+
```

Evaluating (exit) terminates running プログラム.

```
+-----+
| [手続き] |
| |
| (identity x) → any/c |
| x: any/c |
+-----+
```

x を返します。

```
> (identity 42)
42
> (identity c1)
[image:pict_12.png]
> (identity "hello")
"hello"
```

```
+-----+
| [手続き] |
| |
| (struct? x) → boolean? |
| x: any/c |
+-----+
```

some value is a structureかどうかを判定します。

```
> (struct? (make-posn 1 2))
#true
> (struct? 43)
#false
```

#### 4.16 シグネチャ

```
+-----+
| [値] |
| |
| Any: signature? |
+-----+
```

any value のシグネチャ。

```
+-----+
| [値] |
| |
```

```
| Boolean: signature? |
+-----+
```

booleans のシグネチャ。

```
+-----+
| [値] |
| |
| Char: signature? |
+-----+
```

characters のシグネチャ。

```
+-----+
| [手続き] |
| |
| (ConsOf first-sig rest-sig) → signature? |
| first-sig: signature? |
| rest-sig: signature? |
+-----+
```

a cons pair のシグネチャ。

```
+-----+
| [値] |
| |
| EmptyList: signature? |
+-----+
```

the empty list のシグネチャ。

```
+-----+
| [値] |
| |
| False: signature? |
+-----+
```

just false のシグネチャ。

```
+-----+
| [値] |
| |
| Integer: signature? |
+-----+
```

integers のシグネチャ。

```
+-----+
| [値] |
| |
| Natural: signature? |
+-----+
```

natural numbers のシグネチャ。

```
+-----+
| [値] |
| |
| Number: signature? |
+-----+
```

arbitrary numbers のシグネチャ。

```
+-----+
| [値] |
| |
| Rational: signature? |
+-----+
```

rational numbers のシグネチャ。

```
+-----+
| [値] |
| |
| Real: signature? |
+-----+
```

real numbers のシグネチャ。

```
+-----+
| [値] |
| |
| String: signature? |
+-----+
```

strings のシグネチャ。

```
+-----+
| [値] |
| |
| Symbol: signature? |
+-----+
```

symbols のシグネチャ。

```
+-----+
| [値] |
| |
| True: signature? |
+-----+
```

just true のシグネチャ。

#### 4.17 数（緩和条件）

#### 4.18 文字列（緩和条件）

```
+-----+
| [手続き] |
| |
```

```
|
| (string-append s...) → string |
| s: string |
+-----+
```

Concatenates 文字 s の several 文字列 s.

```
> (string-append "hello" " " "world" " " "good bye")
"hello world good bye"
```

```
+-----+
| [手続き] |
| |
| (string-ci<=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically increasing and case-insensitive manner  
かどうかを判定します。

```
> (string-ci<=? "hello" "WORLD" "zoo")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci<? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically strictly increasing and case-insensitive manner  
かどうかを判定します。

```
> (string-ci<? "hello" "WORLD" "zoo")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

all strings are equal, character for character, regardless of case かどうかを判定します。

```
> (string-ci=? "hello" "Hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci>=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically decreasing and case-insensitive manner  
かどうかを判定します。

```
> (string-ci>? "zoo" "WORLD" "hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci>? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically strictly decreasing and case-insensitive manner  
かどうかを判定します。

```
> (string-ci>? "zoo" "WORLD" "hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string<=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically increasing manner  
かどうかを判定します。

```
> (string<=? "hello" "hello" "world" "zoo")
#true
```

```
+-----+
| [手続き] |
| |
| (string<? s t x...) → boolean? |
| s: string |
| t: string |
+-----+
```

```
| x: string |
+-----+
```

the strings are ordered in a lexicographically strictly increasing mannerかどうかを判定します。

```
> (string<? "hello" "world" "zoo")
#true
```

```
+-----+
| [手続き] |
| |
| (string=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

all strings are equal, character for characterかどうかを判定します。

```
> (string=? "hello" "world")
#false
> (string=? "bye" "bye")
#true
```

```
+-----+
| [手続き] |
| |
| (string>=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically decreasing mannerかどうかを判定します。

```
> (string>=? "zoo" "zoo" "world" "hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string>? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically strictly decreasing mannerかどうかを判定します。

```
> (string=? "zoo" "world" "hello")
#true
```

#### 4.19 Posn

```
+-----+
| [手続き] |
| |
| (posn) → signature |
+-----+
```

posns のシグネチャ。

#### 4.20 高階関数

##### 4.21 数（緩和条件・拡張）

```
+-----+
| [手続き] |
| |
| (* x...) → number |
| x: number |
+-----+
```

Multiplies すべての given 数 s. In ISL と up: \* works のとき applied へ only one 数 または none.

```
> (* 5 3)
15
> (* 5 3 2)
30
> (* 2)
2
> (*)
1
```

```
+-----+
| [手続き] |
| |
| (+ x...) → number |
| x: number |
+-----+
```

Adds すべての given 数 s. In ISL と up: + works のとき applied へ only one 数 または none.

```
> (+ 2/3 1/16)
35/48
> (+ 3 2 5 8)
18
> (+ 1)
1
```



```
> (+)
```

```
0
```

```
+-----+
| [手続き] |
| |
| (/ x y...) → number |
| x: number |
| y: number |
| |
+-----+
```

Divides first によって すべての remaining 数 s. In ISL と up: / computes inverse のとき applied へ one 数.

```
> (/ 12 2)
```

```
6
```

```
> (/ 12 2 3)
```

```
2
```

```
> (/ 3)
```

```
1/3
```

```
+-----+
| [手続き] |
| |
| (= x...) → number |
| x: number |
| |
+-----+
```

numbers for equality. In ISL and up: = works when applied to only one number を比較します。

```
> (= 10 10)
```

```
#true
```

```
> (= 11)
```

```
#true
```

```
> (= 0)
```

```
#true
```

## 4.22 高階関数 (lambda つき)

```
+-----+
| [手続き] |
| |
| (andmap p? [l]) → boolean |
| p?: (X... -> boolean) |
| l: (listof X) =... |
| |
+-----+
```

～かどうかを判定する: p? holds のための すべての items の l…:

```
(andmap p (list x-1... x-n)) = (and (p x-1)... (p x-n))
```

```
(andmap p (list x-1... x-n) (list y-1... y-n)) = (and (p x-1 y-1)... (p x-n y-n))
```

```

> (andmap odd? '(1 3 5 7 9))
#true
> threshold
3
> (andmap (lambda (x) (< x threshold)) '(0 1 2))
#true
> (andmap even? '())
#true
> (andmap (lambda (x f) (f x)) (list 0 1 2) (list odd? even? positive?))
#false

```

```

+-----+
| [手続き] |
| |
| (apply f x-1... l) → Y |
| f: (X-1... X-N → Y) |
| x-1: X-1 |
| l: (list X-i+1... X-N) |
+-----+

```

Applies 関数 using items から リスト として 引数 s:

```
(apply f (list x-1... x-n)) = (f x-1... x-n)
```

```

> a-list
(list 0 1 2 3 4 5 6 7 8 9)
> (apply max a-list)
9

```

```

+-----+
| [手続き] |
| |
| (argmax f l) → X |
| f: (X → real) |
| l: (listof X) |
+-----+

```

the (first) element of the list that maximizes the output of the function を探します。

```

> (argmax second '((sam 98) (carl 78) (vincent 93) (asumu 99)))
(list 'asumu 99)

```

```

+-----+
| [手続き] |
| |
| (argmin f l) → X |
| f: (X → real) |
| l: (listof X) |
+-----+

```

the (first) element of the list that minimizes the output of the function を探します。

```

> (argmin second '((sam 98) (carl 78) (vincent 93) (asumu 99)))
(list 'carl 78)

```

```

+-----+
| [手続き] |
| |
| (build-list n f) → (listof X) |
| n: nat |
| f: (nat -> X) |
+-----+

```

構築する: リスト によって applying f へ 数 s between 0 と (-n1):

```

(build-list n f) = (list (f 0)... (f (- n 1)))

> (build-list 22 add1)
(list 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22)
> i
3
> (build-list 3 (lambda (j) (+ j i)))
(list 3 4 5)
> (build-list 5 (lambda (i) (build-list 5
(lambda (j) (if (= i j) 1 0))))))
(list (list 1 0 0 0 0) (list 0 1 0 0 0) (list 0 0 1 0 0) (list 0 0 0 1 0) (list 0 0 0 0 1))

```

```

+-----+
| [手続き] |
| |
| (build-string n f) → string |
| n: nat |
| f: (nat -> char) |
+-----+

```

構築する: 文字列 によって applying f へ 数 s between 0 と (-n1):

```

(build-string n f) = (string (f 0)... (f (- n 1)))

> (build-string 10 integer->char)
"\u0000\u0001\u0002\u0003\u0004\u0005\u0006\a\b\t"
> (build-string 26 (lambda (x) (integer->char (+ 65 x))))
"ABCDEFGHIJKLMNOPQRSTUVWXYZ"

```

```

+-----+
| [手続き] |
| |
| (compose f g) → (X -> Z) |
| f: (Y -> Z) |
| g: (X -> Y) |
+-----+

```

(説明) Composes sequence の procedures へ single procedure:

```

(compose f g) = (lambda (x) (f (g x)))

> ((compose add1 second) '(add 3))
4

```

```
> (map (compose add1 second) '((add 3) (sub 2) (mul 4)))
(list 4 3 5)
```

```
+-----+
| [手続き] |
| |
| (filter p? l) → (listof X) |
| p?: (X → boolean) |
| l: (listof X) |
+-----+
```

a list from all those items on a list for which the predicate holds を構築します。

```
> (filter odd? '(0 1 2 3 4 5 6 7 8 9))
(list 1 3 5 7 9)
> threshold
3
> (filter (lambda (x) (>= x threshold)) '(0 1 2 3 4 5 6 7 8 9))
(list 3 4 5 6 7 8 9)
```

```
+-----+
| [手続き] |
| |
| (foldl f base l...) → Y |
| f: (X... Y → Y) |
| base: Y |
| l: (listof X) |
+-----+
```

```
(foldl f base (list x-1... x-n)) = (f x-n... (f x-1 base))
```

```
(foldl f base (list x-1... x-n) (list x-1... x-n))
```

```
= (f x-n y-n... (f x-1 y-1 base))
```

```
> (foldl + 0 '(0 1 2 3 4 5 6 7 8 9))
```

```
45
```

```
> a-list
```

```
(list 0 1 2 3 4 5 6 7 8 9)
```

```
> (foldl (lambda (x r) (if (> x threshold) (cons (* 2 x) r) r)) '() a-list)
```

```
(list 18 16 14 12 10 8)
```

```
> (foldl (lambda (x y r) (+ x y r)) 0 '(1 2 3) '(10 11 12))
```

```
39
```

```
+-----+
| [手続き] |
| |
| (foldr f base l...) → Y |
| f: (X... Y → Y) |
| base: Y |
| l: (listof X) |
+-----+
```

```
(foldr f base (list x-1... x-n)) = (f x-1... (f x-n base))
```

```

(foldr f base (list x-1... x-n) (list y-1... y-n))
= (f x-1 y-1... (f x-n y-n base))

> (foldr + 0 '(0 1 2 3 4 5 6 7 8 9))
45

> a-list
(list 0 1 2 3 4 5 6 7 8 9)
> (foldr (lambda (x r) (if (> x threshold) (cons (* 2 x) r) r)) '() a-list)
(list 8 10 12 14 16 18)
> (foldr (lambda (x y r) (+ x y r)) 0 '(1 2 3) '(10 11 12))
39

```

```

+-----+
| [手続き] |
| (for-each f l...) → void? |
| f: (any... → any) |
| l: (listof any) |
+-----+

```

a function to each item on one or more lists for effect only を適用します。

Although Intermediate Student とともに Lambda provides for-each 関数, it は intended へ be used の中の Advanced Student level.

```

+-----+
| [手続き] |
| (map f l...) → (listof Z) |
| f: (X... → Z) |
| l: (listof X) |
+-----+

```

構築する: new リスト によって applying 関数 へ 各 item 上の one または more existing リスト s:

```

(map f (list x-1... x-n)) = (list (f x-1)... (f x-n))
(map f (list x-1... x-n) (list y-1... y-n)) = (list (f x-1 y-1)... (f x-n y-n))

> (map add1 (list 3 -4.01 2/5))
(list 4 #i-3.01 1.4)

> (define (tag-with-a x) (list "a" (+ x 1)))
> (map tag-with-a (list 3 -4.01 2/5))
(list (list "a" 4) (list "a" #i-3.01) (list "a" 1.4))

> (define (add-and-multiply x y) (+ x (* x y)))
> (map add-and-multiply (list 3 -4 2/5) '(1 2 3))
(list 6 -12 1.6)

```

```

+-----+
| [手続き] |
| |
| |
+-----+

```

```
| (memf p? l) → (union #false (listof X)) |
| p?: (X → any) |
| l: (listof X) |
+-----+
```

#false if p? produces false for all items on l. If p? produces #true for any of the items on l, memf returns the sub-list starting from that item を生成します。

```
> (memf odd? '(2 4 6 3 8 0))
(list 3 8 0)
```

```
+-----+
| [手続き] |
| |
| (ormap p? l) → boolean |
| p?: (X → boolean) |
| l: (listof X) |
+-----+
```

～かどうかを判定する: p? holds のための at least one items の l:

```
(ormap p (list x-1... x-n)) = (or (p x-1)... (p x-n))
```

```
(ormap p (list x-1... x-n) (list y-1... y-n)) = (or (p x-1 y-1)... (p x-n y-n))
```

```
> (ormap odd? '(1 3 5 7 9))
```

```
#true
```

```
> threshold
```

```
3
```

```
> (ormap (lambda (x) (< x threshold)) '(6 7 8 1 5))
```

```
#true
```

```
> (ormap even? '())
```

```
#false
```

```
> (ormap (lambda (x f) (f x)) (list 0 1 2) (list odd? even? positive?))
```

```
#true
```

```
+-----+
| [手続き] |
| |
| (procedure? x) → boolean? |
| x: any |
+-----+
```

true if the value is a procedure を生成します。

```
> (procedure? cons)
```

```
#true
```

```
> (procedure? add1)
```

```
#true
```

```
> (procedure? (lambda (x) (> x 22)))
```

```
#true
```

```
+-----+
| [手続き] |
| |
```

```

| (quicksort l comp) → (listof X) |
| l: (listof X) |
| comp: (X X → boolean) |
+-----+

```

Sorts items 上の l, の中の order according へ comp (using quicksort algorithm).

```

> (quicksort '(6 7 2 1 3 4 0 5 9 8) <)
(list 0 1 2 3 4 5 6 7 8 9)

```

```

+-----+
| [手続き] |
| (sort l comp) → (listof X) |
| l: (listof X) |
| comp: (X X → boolean) |
+-----+

```

Sorts items 上の l, の中の order according へ comp.

```

> (sort '(6 7 2 1 3 4 0 5 9 8) <)
(list 0 1 2 3 4 5 6 7 8 9)

```

## 付録 B-5: Advanced Student (#lang htdp/asl)

原本: extracted/appendix/htdp-langs/original\_markdown\_05\_advanced.md

初学者向けに、説明文を日本語へ翻訳しています。コード・シグネチャ・実行例は原文のまま保持します。

### 5 Advanced Student (上級学生言語)

文法の記法では、 $X \cdots$  (太字の点) という書き方で、 $X$  が任意の回数 (0 回、1 回、またはそれ以上) 現れてよいことを示します。別に、文法はテンプレートで使う識別子として  $\cdots$  も定義します。

program	=	def-or-expr...
def-or-expr	=	definition expr test-case library-require signature-declaration
definition	=	(define (name variable...) expr) (define name expr) (define-struct name (name...)) (define-datatype name (name name....
expr	=	(begin expr expr...) (begin0 expr expr...) (set! variable expr) (delay expr) (lambda (variable...) expr) (λ (variable...) expr) (local [definition...] expr) (letrec ([name expr]...) expr) (shared ([name expr]...) expr) (let ([name expr]...) expr) (let name ([name expr]...) expr) (let* ([name expr]...) expr) (recur name ([name expr]...) expr) (expr expr...) (cond [expr expr]... [expr expr]) (cond [expr expr]... [else expr]) (case expr [(choice choice...) exp... [(choice choice...) exp... (case expr [(choice choice...) exp... (case expr [(choice choice...) exp... [else expr]) (match expr [pattern expr]...) (if expr expr expr) (when expr expr)



		(unless expr expr)
		(and expr expr expr...)
		(or expr expr expr...)
		(time expr)
		name
		'quoted
		'quasiquoted
		'()
		number
		boolean
		string
		character
		(signature signature-form)
choice	=	name
pattern	=	number
		-
		name
		number
		true
		false
		string
		character
		'quoted
		'quasiquoted-pattern
		(cons pattern pattern)
		(list pattern...)
		(list* pattern...)
		(struct id (pattern...))
		(vector pattern...)
		(box pattern)
quasiquoted-pattern	=	name
		number
		string
		character
		(quasiquoted-pattern...)
		'quasiquoted-pattern
		'quasiquoted-pattern
		`,`pattern
		`,`@`pattern
signature-declaration	=	(: name signature-form)
signature-form	=	(enum expr...)
		(mixed signature-form...)
		(signature-form... -> signature-fo...)
		(ListOf signature-form)
		signature-variable
signature-variable	=	expr
quoted	=	%name
	=	name
		number

					string	
					character	
					(quoted...)	
					'quoted	
					'quoted	
					`,'quoted	
					`,@'quoted	
quasiquoted		=			name	
					number	
					string	
					character	
					(quasiquoted...)	
					'quasiquoted	
					'quasiquoted	
					`,'expr	
					`,@'expr	
test-case		=			(check-expect expr expr)	
					(check-random expr expr)	
					(check-within expr expr expr)	
					(check-error expr expr...)	
					(check-member-of expr expr expr)	
					(check-satisfied expr expr)	
					(check-range expr expr)	
					(check-range expr)	
library-require		=			(require string)	
					(require (lib string string...))	
					(require (planet string package))	
package		=			(string string number number)	

名前 (name) または変数 (variable) は、空白や次の文字を含まない文字の並びです。

“; ‘ ` () [] {} |; #

数 (number) とは、123、3/2、5.5 のような数です。

真偽値 (boolean) は #true または #false のいずれかです。

#true 定数の別表記は #t、true、#T です。同様に #f、false、#F も #false として認識されます。

シンボル (symbol) は、クオート文字に続く名前です。シンボルは 42、'()、#false などと同じく値です。

文字列 (string) は、一対の ” で囲まれた文字の並びです。シンボルと違い、文字列は文字に分割したり、さまざまな関数で操作したりできます。たとえば “abcdef”、“This is a string”、および “This is a string with " inside” はすべて文字列です。

文字 (character) は # で始まり、その文字の名前を持ちます。たとえば #、## は文字です。

関数呼び出しでは、開き括弧の直後に現れる関数は、define や define-struct で定義した関数、またはあらかじめ定義された関数のいずれかです。

### 5.1 あらかじめ定義された変数

```
+-----+
| [値] |
| |
| empty: empty? |
+-----+
```

空リスト。

```
+-----+
| [値] |
| |
| true: boolean? |
+-----+
```

#true の値。

```
+-----+
| [値] |
| |
| false: boolean? |
+-----+
```

#false の値。

### 5.2 テンプレート変数

```
+-----+
| [構文] |
| |
| .. |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
| ... |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
| |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

```
+-----+
| [構文] |
| |
+-----+
```

関数定義がテンプレートであることを示すためのプレースホルダ。

### 5.3 上級の構文

In Advanced, set! できる be used へ mutate 変数 s, と define-struct's 構造体 s は mutable. define と lambda できる define 関数 s の zero 引数 s, と 関数呼び出し s できる invoke 関数 s の zero 引数 s.

```
+-----+
| [構文] |
| (lambda (variable...) expression) |
+-----+
```

a function that takes as many arguments as given variables, and whose body is expression を作成します。

```
+-----+
| [構文] |
| (λ (variable...) expression) |
+-----+
```

Greek letter λ は synonym のための lambda.

```
+-----+
| [構文] |
| (expression expression...) |
+-----+
```

Calls the function that results from evaluating the first expression. The value of the call is the value of function's body when every instance of name's variables are replaced by the values of the corresponding expressions.

The function being called must come from either a definition appearing before the function call, or from a lambda expression. The number of argument expressions must be the same as the number of arguments expected by the function.

```
+-----+
| [構文] |
| |
| (define-datatype datatype-name [variant-name field-name...]) |
+-----+
```

A short-hand のための defining group の related 構造体 s. following define-datatype:

```
(define-datatype datatype-name
 [variant-name field-name #,...]
...)
```

is equivalent to:

```
(define (datatype-name? x)
 (or (variant-name? x)...))
(define-struct variant-name (field-name...))
...
```

```
+-----+
| [構文] |
| |
| (begin expression expression...) |
+-----+
```

the expressions in order from left to right. The value of the begin expression is the value of the last expression を評価します。

```
+-----+
| [構文] |
| |
| (begin0 expression expression...) |
+-----+
```

the expressions in order from left to right. The value of the begin expression is the value of the first expression を評価します。

```
+-----+
| [構文] |
| |
| (set! variable expression) |
+-----+
```

expression, and then changes the value of the variable to have expression's value. The variable must be defined by define, letrec, let\*, let, or local を評価します。

```
+-----+
| [構文] |
| |
| (delay expression) |
+-----+
```

a “promise” to evaluate expression. The expression is not evaluated until the promise is forced with force; when the promise is forced, the result is recorded, so that any further force of the promise immediately produces the remembered value を生成します。

```
+-----+
| [構文] |
| |
| (shared ([name expression]...) expression) |
+-----+
```

letrec と同様ですが、when an expression next to an id is a cons, list, vector, quasiquoted expression, or make-struct-name from a define-struct, the expression can refer directly to any name, not just names defined earlier. Thus, shared can be used to create cyclic data structures。

```
+-----+
| [構文] |
| |
| (recur name ([name expression]...) expression) |
+-----+
```

A short-hand syntax for recursive loops. The first name corresponds to the name of the recursive function. The names in the parenthesis are the function’s arguments, and each corresponding expression is a value supplied for that argument in an initial starting call of the function. The last expression is the body of the function.

More precisely, following recur:

```
(recur func-name ([arg-name arg-expression]...)
 body-expression)
```

is equivalent to:

```
(local [(define (func-name arg-name...) body-expression)]
 (func-name arg-expression...))
```

```
+-----+
| [構文] |
| |
| (let name ([name expression]...) expression) |
+-----+
```

An alternate syntax のための recur.

```
+-----+
| [構文] |
| |
| (case expression [(choice...) expression]... [(choice...) expression]) |
+-----+
```

A case form contains one or more clauses. Each clause contains a choices (in parentheses)—either numbers or names—and an answer expression. The initial expression is evaluated, and its value is compared to the choices in each clause, where the lines are considered in order. The first line that contains a matching choice provides an answer expression whose value is the result of the whole case expression. Numbers match with the numbers in the choices, and symbols match with the names. If none of the lines contains a matching choice, it is an error.

```
+-----+
| [構文] |
| |
| (case expression [(choice...) expression]... [else expression]) |
+-----+
```

これ form の case は similar へ prior one, except という final else 節は taken もし no 節 contains choice matching 値の initial 式.

```
+-----+
| [構文] |
| |
| (match expression [pattern expression]...) |
+-----+
```

A match form contains one or more clauses that are surrounded by square brackets. Each clause contains a pattern—a description of a value—and an answer expression. The initial expression is evaluated, and its value is matched against the pattern in each clause, where the clauses are considered in order. The first clause that contains a matching pattern provides an answer expression whose value is the result of the whole match expression. This expression may reference identifiers defined in the matching pattern. If none of the clauses contains a matching pattern, it is an error.

```
+-----+
| [構文] |
| |
| (when question-expression body-expression) |
+-----+
```

If question-expression evaluates to true, the result of the when expression is the result of evaluating the body-expression, otherwise the result is (void) and the body-expression is not evaluated. If the result of evaluating the question-expression is neither true nor false, it is an error.

```
+-----+
| [構文] |
| |
| (unless question-expression body-expression) |
+-----+
```

when と同様ですが、the body-expression is evaluated when the question-expression produces false instead of true。

#### 5.4 共通の構文

following syntaxes behave same の中の Advanced level として they did の中の Intermediate Student とともに Lambda level.

```
+-----+
| [構文] |
| |
| (local [definition...] expression) |
+-----+
```

Groups related 定義 s のための use の中の 式. Each 定義 できる be either define または define-struct.

When evaluating local, each definition is evaluated in order, and finally the body expression is evaluated. Only the expressions within the local (including the right-hand-sides of the definitions and the expression) may refer to the names defined by the definitions. If a name defined in the local is the same as a top-level binding, the inner one “shadows” the outer one. That is, inside the local, any references to that name refer to the inner one.

```
+-----+
| [構文] |
| |
| (letrec ([name expr-for-let]...) expression) |
+-----+
```

local と同様ですが、with a simpler syntax. Each name defines a variable (or a function) with the value of the corresponding expr-for-let. If expr-for-let is a lambda, letrec defines a function, otherwise it defines a variable。

```
+-----+
| [構文] |
| |
| (let* ([name expr-for-let]...) expression) |
+-----+
```

letrec と同様ですが、each name can only be used in expression, and in expr-for-lets occuring after that name。

```
+-----+
| [構文] |
| |
| (let ([name expr-for-let]...) expression) |
+-----+
```



letrec と同様ですが、the defined names can be used only in the last expression, not the expr-for-lets next to the names。

```
+-----+
| [構文] |
| |
| (time expression) |
+-----+
```

Measures the time taken to evaluate expression. After evaluating expression, time prints out the time taken by the evaluation (including real time, time taken by the CPU, and the time spent collecting free memory). The value of time is the same as that of expression.

```
+-----+
| [構文] |
| |
| (define (name variable variable...) expression) |
+-----+
```

name という名前の関数を定義します。expression は関数本体です。関数が呼ばれると、引数の値が変数の代わりに本体へ挿入されます。関数は、その新しい式の値を返します。関数名は、他の関数や変数と同じであってはいけません。

```
+-----+
| [構文] |
| |
| (define name expression) |
+-----+
```

expression の値で name という変数を定義します。変数名は他の関数や変数と同じであってはいならず、name 自身が expression に現れてはいけません。

```
+-----+
| [構文] |
| |
| (define-struct structure-name (field-name...)) |
+-----+
```

Defines new 構造体 called 構造体-name. 構造体's fields は named によって field-names. のあと define-struct, following new 関数 s は available:

- make-structure-name: takes a number of arguments equal to the number of fields in the structure, and creates a new instance of that structure.
- structure-name-field-name: takes an instance of the structure and returns the value in the field named by field-name.
- structure-name?: takes any value, and returns #true if the value is an instance of the structure.

name の new 関数 s introduced によって define-struct ~してはならない be same として という の other 関数 s または 変数 s, そうでなければ define-struct reports エラー.

(説明) In Advanced, define-struct introduces one additional 関数:

- **set-structure-name-field-name!**

takes an instance of the structure and a value, and mutates the instance's field to the given value.

```
+-----+
| [構文] |
| (cond [question-expression answer-expression]...) |
| (cond [question-expression answer-expression]... [else answer-expression]) |
| (cond [question-expression answer-expression] |
| ... |
| [else answer-expression]) |
| ``racket |
| (cond [question-expression |
| ... |
| [else answer-expression]) |
| ... |
+-----+
```

Chooses 節 based 上の ある 条件. cond finds first question-式 という 評価結果は #true, then 評価する corresponding answer-式.

If none of the question-expressions evaluates to #true, cond's value is the answer-expression of the else clause. If there is no else, cond reports an error. If the result of a question-expression is neither #true nor #false, cond also reports an error.

else は cond の外では使えません。

```
+-----+
| [構文] |
| (if question-expression |
| then-answer-expression |
| else-answer-expression) |
| ``racket |
| (if question-expression |
| then-answer-expression |
| else-answer-expression) |
| ... |
+-----+
```

とき: 値 の question-式 は #true, もし 評価する then-answer-式. とき: test は #false, もし 評価する else-answer-式.

もし question-式 は neither #true nor #false, もし reports エラー.

```
+-----+
| [構文] |
| |
| (and expression expression expression...) |
+-----+
```

Evaluates へ #true もし すべての式 s は #true. もし 任意の式 は #false, と 式 評価結果は #false (and 式 s へ right の という 式 は ない evaluated.)

もし 任意の の 式 s evaluate へ 値 other than #true または #false, と reports エラー.

```
+-----+
| [構文] |
| |
| (or expression expression expression...) |
+-----+
```

to #true as soon as one of the expressions is #true (and the expressions to the right of that expression are not evaluated.) If all of the expressions are #false, the or expression evaluates to #false を評価します。

もし 任意の の 式 s evaluate へ 値 other than #true または #false, または reports エラー.

```
+-----+
| [構文] |
| |
| (check-expect expression expected-expression) |
+-----+
```

that the first expression evaluates to the same value as the expected-expression を検査します。

```
(check-expect (fahrenheit->celsius 212) 100)
(check-expect (fahrenheit->celsius -40) -40)
```

```
(define (fahrenheit->celsius f)
 (* 5/9 (- f 32)))
```

A check-expect expression must be placed at the top-level of a student program. Also it may show up anywhere in the program, including ahead of the tested function definition. By placing check-expects there, a programmer conveys to a future reader the intention behind the program with working examples, thus making it often superfluous to read the function definition proper. Syntax errors in check-expect (and all check forms) are intentionally delayed to run time so that students can write tests without necessarily writing complete function headers.

It is an error for expr or expected-expr to produce an inexact number or a function value. As for inexact numbers, it is morally wrong to compare them for plain equality.

Instead one tests whether they are both within a small interval; see `check-within`. As for functions (see Intermediate and up), it is provably impossible to compare functions.

```
+-----+
| [構文] |
| |
| (check-random expression expected-expression) |
+-----+
```

that the first expression evaluates to the same value as the expected-expression を検査します。

form supplies same 乱数-数 generator へ both parts. もし both parts request 乱数 数  $s$  から same interval の中の same order, they receive same 乱数 数  $s$ .

ここでは simple example の where `check-乱数` は useful:

```
(define WIDTH 100)
(define HEIGHT (* 2 WIDTH))

(define-struct player (name x y))
; A Player is (make-player String Nat Nat)

; String -> Player

(check-random (create-randomly-placed-player "David Van Horn")
 (make-player "David Van Horn" (random WIDTH) (random HEIGHT)))

(define (create-randomly-placed-player name)
 (make-player name (random WIDTH) (random HEIGHT)))
```

Note how 乱数 は called 上の same 数  $s$  の中の same order の中の both parts の `check-乱数`. もし two parts call 乱数 のための different intervals, they は likely へ fail:

```
; String -> Player

(check-random (create-randomly-placed-player "David Van Horn")
 (make-player "David Van Horn" (random WIDTH) (random HEIGHT)))

(define (create-randomly-placed-player name)
 (a-helper-function name (random HEIGHT)))

; String Number -> Player
(define (a-helper-function name height)
 (make-player name (random WIDTH) height))
```

Because 引数 へ a-helper-関数 は evaluated first, 乱数 は first called のための interval  $[0, HEIGHT)$  と then のための  $[0, WIDTH)$ , という is, の中の different order than の中の preceding `check-乱数`.

それはエラーのための `expr` または `expected-expr` へ produce 関数 値 または in 正確 数;  
see note 上の `check-expect` のための details.

```
+-----+
| [構文] |
| |
| (check-satisfied expression predicate) |
+-----+
```

Checks という first 式 satisfies named 述語 (関数の one 引数). Recall という “satisfies” means “the 関数 produces #true のための given 値.”

ここでは simple examples のための `check-satisfied`:

```
> (check-satisfied 1 odd?)
The test passed!

> (check-satisfied 1 even?)
Ran 1 test. 0 tests passed.
Check failures:
Actual value | 1 | does not satisfy even?.
```



```
at line 3, column 0
```

In general `check-satisfied` empowers プログラム designers へ use defined 関数 `s` へ formulate test suites:

```
; [cons Number [List-of Number]] -> Boolean
; a function for testing htdp-sort
```

```
(check-expect (sorted? (list 1 2 3)) #true)
(check-expect (sorted? (list 2 1 3)) #false)
```

```
(define (sorted? l)
 (cond
 [(empty? (rest l)) #true]
 [else (and (<= (first l) (second l)) (sorted? (rest l)))]))
```

```
; [List-of Number] -> [List-of Number]
; create a sorted version of the given list of numbers
```

```
(check-satisfied (htdp-sort (list 1 2 0 3)) sorted?)
```

```
(define (htdp-sort l)
 (cond
 [(empty? l) l]
 [else (insert (first l) (htdp-sort (rest l)))]))
```

```
; Number [List-of Number] -> [List-of Number]
; insert x into l at proper place
; assume l is arranged in ascending order
; the result is sorted in the same way
```

And yes, results of htdp-sort satisfy sorted? 述語:

[構文]
(check-within expression expected-expression delta)

```
(define (roots-table xs)
 (cond
 [(empty? xs) '()]
 [else (cons (root-of (first xs)) (roots-table (rest xs)))])
```

例:

対照的に、のとき  $\delta$  は small, test fails:

例:

Actual value	'((make-roots 2.0

```
1.4142135623730951)) | is not within 1e-5 of expected value | '((make-roots 2 1.414)) | .
|_____| at line 5, column 0
```

それはエラーのための式sまたはexpected-式へproduce関数値; see note 上の check-expect のための details.

もし delta は ない数, check-within reports エラー.

```
+-----+
| [構文] |
| (check-error expression expected-error-message) |
| (check-error expression) |
+-----+
```

that the expression reports an error, where the error messages matches the value of expected-error-message, if it is present を 検査します。

ここでは typical beginner example という calls のための use の check-エラー:

```
(define sample-table
 '(("matthias" 10)
 ("matthew" 20)
 ("robby" -1)
 ("shriram" 18)))

; [List-of [List String Number]] String -> Number
; determine the number associated with s in table

(define (lookup table s)
 (cond
 [(empty? table) (error (string-append s " not found"))]
 [else (if (string=? (first (first table)) s)
 (second (first table))
 (lookup (rest table)))]))
```

Consider following two examples の中の この context:

例:

```
> (check-expect (lookup sample-table "matthew") 20)
The test passed!
```

例:

```
> (check-error (lookup sample-table "kathi") "kathi not found")
The test passed!
```

```
+-----+
| [構文] |
| |
| |
```

```
| (check-member-of expression expression expression...) |
+-----+
```

that the value of the first expression is that of one of the following expressions を検査します。

```
; [List-of X] -> X
; pick a random element from the given list l
(define (pick-one l)
 (list-ref l (random (length l))))
```

例:

```
> (check-member-of (pick-one '("a" "b" "c")) "a" "b" "c")
The test passed!
```

それはエラーのための任意の の式 s へ produce 関数 値; see note 上の check-expect のための details.

```
+-----+
| [構文] |
| |
| (check-range expression low-expression high-expression) |
+-----+
```

that the value of the first expression is a number in between the value of the low-expression and the high-expression, inclusive を検査します。

A check-range form は best used へ delimit possible results の 関数 s という compute in 正確 数 s:

```
(define EPSILON 0.001)

; [Real -> Real] Real -> Real
; what is the slope of f at x?
(define (differentiate f x)
 (slope f (- x EPSILON) (+ x EPSILON)))

; [Real -> Real] Real Real -> Real
(define (slope f left right)
 (/ (- (f right) (f left))
 2 EPSILON))
```

```
(check-range (differentiate sin 0) 0.99 1.0)
```

それはエラーのための 式, low-式, または high-式 へ produce 関数 値; see note 上の check-expect のための details.

```
+-----+
| [構文] |
| |
| |
+-----+
```



```
| (require string) |
+-----+
```

Makes 定義 *s* の モジュール specified によって 文字列 available の中の current モジュール (i.e., current file), where 文字列 refers へ file relative へ current file.

The string is constrained in several ways to avoid problems with different path conventions on different platforms: a / is a directory separator, . always means the current directory, .. always means the parent directory, path elements can use only a through z (uppercase or lowercase), 0 through 9, -, \_, and ., and the string cannot be empty or contain a leading or trailing /.

```
+-----+
| [構文] |
| |
| (require module-name) |
+-----+
```

Accesses file の中の installed ライブラリ. ライブラリ name は 識別子 とともに same constraints として のための relative-path 文字列 (though without quotes), とともに additional constraint という it ～してはならない contain a..

```
+-----+
| [構文] |
| |
| (require (lib string string...)) |
+-----+
```

Accesses a file in an installed library, making its definitions available in the current module (i.e., the current file). The first string names the library file, and the remaining strings name the collection (and sub-collection, and so on) where the file is installed. Each string is constrained in the same way as for the (requirestring) form.

```
+-----+
| [構文] |
| |
| (require (planet string (string string number number))) |
+-----+
```

Accesses ライブラリ という は distributed 上の internet via PPlaneT server, making it 定義 *s* available の中の current モジュール (i.e., current file).

full grammar のための planet requires は given の中の Importing と Exporting: require と provide, but best place へ find examples の syntax は 上の the PPlaneT server, の中の description の specific package.

## 5.5 あらかじめ定義された関数

## 5.6 シグネチャ

Signatures do not have to be comment: They can also be part of the > code. When a signature is attached to a function, DrRacket will check > that program uses the function in accordance with the signature and > display signature violations along with the test results. A signature is a regular value, and is specified as a > signature form, a > special syntax that only works with: signature declarations > and inside signature expressions.

```
+-----+
| [構文] |
| |
| (: name signature-form) |
+-----+
```

これ attaches シグネチャ specified によって シグネチャ-form へ 定義 の name. There でなければならぬ 定義 の name somewhere の中の プログラム.

```
(: age Integer)
(define age 42)

(: area-of-square (Number -> Number))
(define (area-of-square len)
 (sqr len))
```

On running プログラム, Racket checks whether シグネチャ s attached with: actually match 値 の 変数. もし they don't, Racket reports シグネチャ violation along とともに test failures.

たとえば、この piece の code:

```
(: age Integer)
(define age "fortytwo")
```

Yields この output:

```
+-----+
| `1 signature violation.` |
+-----+
| `Signature violations:` |
| `got "fortytwo" at line 2, column 12, signature... |
+-----+
```

Note という シグネチャ violation does ない stop running プログラム.

```
+-----+
| [構文] |
| |
| |
+-----+
```

```
| (signature signature-form) |
+-----+
```

これ返すシグネチャ described によってシグネチャ-form として 値.

### 5.6.1 シグネチャ形式

Any expression can be a signature form, in which case the signature is > the value returned by that expression. There are a few special > signature forms, however: In a signature form, any name that starts with a % is a > signature variable that stands for any signature depending on how > the signature is used. Example:

```
(: same (%a -> %a))
```

```
(define (same x) x)
```

```
+-----+
| [構文] |
| |
| (input-signature-form... -> output-signature-form) |
+-----+
```

これシグネチャ form describes 関数 とともに inputs described によって input-シグネチャ-forms と output described によって output-シグネチャ-form.

```
+-----+
| [構文] |
| |
| (enum expr...) |
+-----+
```

これシグネチャ describes enumeration の 値 s returned によって exprs.

例:

```
(: cute? ((enum "cat" "snake") -> Boolean))
```

```
(define (cute? pet)
 (cond
 [(string=? pet "cat") #t]
 [(string=? pet "snake") #f]))
```

```
+-----+
| [構文] |
| |
| (mixed signature-form...) |
+-----+
```

これシグネチャ describes mixed data, i.e. itemization where 各 の cases has シグネチャ described によってシグネチャ-form.

例:

```
(define SIGS (signature (mixed Aim Fired)))
```

```
+-----+
| [構文] |
| |
| (ListOf signature-form) |
+-----+
```

これ シグネチャ describes リスト where elements は described によって シグネチャ-form.

```
+-----+
| [構文] |
| |
| (predicate expression) |
+-----+
```

これ シグネチャ describes 値 s through 述語: 式 must evaluate へ 関数 の one 引数 という 返す 真偽値. シグネチャ matches すべての 値 s のための であり 述語 返す #true.

### 5.6.2 構造体シグネチャ

A advanced form defines two additional names that can be > used in signatures. For a struct called struct, these > are Struct and StructOf. Note that > these names are capitalized. In particular, a struct called > Struct, will also define Struct and > StructOf. Moreover, when forming the additional > names, hyphens are removed, and each letter following a hyphen is > capitalized - so a struct called foo-bar will define > FooBar and FooBarOf. Struct is a signature that describes struct values > from this structure type. StructOf is a function > that takes as input a signature for each field. It returns a > signature describing values of this structure type, additionally > describing the values of the fields of the value.

```
(define-struct pair [fst snd])
```

```
(: add-pair ((PairOf Number Number) -> Number))
```

```
(define (add-pair p)
 (+ (pair-fst p) (pair-snd p)))
```

remaining subsections リスト それらの 関数 s という は built へ プログラム ming language. All other 関数 s は imported から teachpack または でなければならない defined の中の プログラム.

### 5.7 数: 整数・有理数・実数・複素数・正確数・非正確数

```
+-----+
| [手続き] |
| |
| |
+-----+
```

```
| (- x y...) → number |
| x: number |
| y: number |
+-----+
```

Subtracts second (and following) 数(s) から first; negates 数 もし there は only one 引数.

```
> (- 5)
-5
> (- 5 3)
2
> (- 5 3 1)
1
```

```
+-----+
| [手続き] |
| |
| (< x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

two or more (real) numbers for less-than を比較します。

```
> (< 42 2/5)
#false
```

```
+-----+
| [手続き] |
| |
| (<= x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

two or more (real) numbers for less-than or equality を比較します。

```
> (<= 42 2/5)
#false
```

```
+-----+
| [手続き] |
| |
| (> x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

two or more (real) numbers for greater-than を比較します。

```
> (> 42 2/5)
```

```
#true
```

```
+-----+
| [手続き] |
| |
| (>= x y z...) → boolean? |
| x: real |
| y: real |
| z: real |
+-----+
```

two or more (real) numbers for greater-than or equality を比較します。

```
> (>= 42 42)
```

```
#true
```

```
+-----+
| [手続き] |
| |
| (abs x) → real |
| x: real |
+-----+
```

判定する: absolute 値 の 実数 数.

```
> (abs -12)
```

```
12
```

```
+-----+
| [手続き] |
| |
| (acos x) → number |
| x: number |
+-----+
```

the arccosine (inverse of cos) of a number を計算します。

```
> (acos 0)
```

```
#i1.5707963267948966
```

```
+-----+
| [手続き] |
| |
| (add1 x) → number |
| x: number |
+-----+
```

Increments given 数.

```
> (add1 2)
```

```
3
```

```
+-----+
| [手続き] |
| |
+-----+
```

```

|
| (angle x) → real
| x: number
+-----+

```

the angle from a complex number を取り出します。

```

> (angle (make-polar 3 4))
#i-2.2831853071795867

```

```

+-----+
| [手続き] |
|
| (asin x) → number
| x: number
+-----+

```

the arcsine (inverse of sin) of a number を計算します。

```

> (asin 0)
0

```

```

+-----+
| [手続き] |
|
| (atan x) → number
| x: number
+-----+

```

計算する: arctangent の given 数:

```

> (atan 0)
0
> (atan 0.5)
#i0.4636476090008061

```

Also comes in a two-argument version where (atany x) computes (atan(/yx)) but the signs of y and x determine the quadrant of the result and the result tends to be more accurate than that of the 1-argument version in borderline cases:

```

> (atan 3 4)
#i0.6435011087932844
> (atan -2 -1)
#i-2.0344439357957027

```

```

+-----+
| [手続き] |
|
| (ceiling x) → integer
| x: real
+-----+

```

判定する: closest 整数 (正確 または in 正確) above 実数 数. 参照: round.

```
> (ceiling 12.3)
#i13.0
```

```
+-----+
| [手続き] |
| |
| (complex? x) → boolean? |
| x: any/c |
+-----+
```

some value is complexかどうかを判定します。

```
> (complex? 1-2i)
#true
```

```
+-----+
| [手続き] |
| |
| (conjugate x) → number |
| x: number |
+-----+
```

Flips sign of the imaginary part of the complex number.

```
> (conjugate 3+4i)
3-4i
> (conjugate -2-5i)
-2+5i
> (conjugate (make-polar 3 4))
#i-1.960930862590836+2.2704074859237844i
```

```
+-----+
| [手続き] |
| |
| (cos x) → number |
| x: number |
+-----+
```

the cosine of a number (radians) を計算します。

```
> (cos pi)
#i-1.0
```

```
+-----+
| [手続き] |
| |
| (cosh x) → number |
| x: number |
+-----+
```

the hyperbolic cosine of a number を計算します。

```
> (cosh 10)
#i11013.232920103324
```



```
+-----+
| [手続き] |
| |
| (current-seconds) → integer |
+-----+
```

判定する: current time の中の seconds elapsed (since platform-specific starting date).

```
> (current-seconds)
1779843347
```

```
+-----+
| [手続き] |
| |
| (denominator x) → integer |
| x: rational? |
+-----+
```

the denominator of a rational を計算します。

```
> (denominator 2/3)
3
```

```
+-----+
| [値] |
| |
| e: real |
+-----+
```

Euler's 数.

```
> e
#i2.718281828459045
```

```
+-----+
| [手続き] |
| |
| (even? x) → boolean? |
| x: integer |
+-----+
```

some integer (exact or inexact) is even or not かどうかを判定します。

```
> (even? 2)
#true
```

```
+-----+
| [手続き] |
| |
| (exact->inexact x) → number |
| x: number |
+-----+
```

an exact number to an inexact one を変換します。

```
> (exact->inexact 12)
```

```
#i12.0
```

```
+-----+
| [手続き] |
| |
| (exact? x) → boolean? |
| x: number |
+-----+
```

some number is exact かどうかを判定します。

```
> (exact? (sqrt 2))
```

```
#false
```

```
+-----+
| [手続き] |
| |
| (exp x) → number |
| x: number |
+-----+
```

判定する: e raised へ 数.

```
> (exp -2)
```

```
#i0.1353352832366127
```

```
+-----+
| [手続き] |
| |
| (expt x y) → number |
| x: number |
| y: number |
+-----+
```

the power of the first to the second number, which is to say, exponentiation を計算します。

```
> (expt 16 1/2)
```

```
4
```

```
> (expt 3 -4)
```

```
1/81
```

```
+-----+
| [手続き] |
| |
| (floor x) → integer |
| x: real |
+-----+
```

判定する: closest 整数 (正確 または in 正確) below 実数 数. 参照: round.

```
> (floor 12.3)
```

```
#i12.0
```

```

+-----+
| [手続き] |
| |
| (gcd x y...) → integer |
| x: integer |
| y: integer |
+-----+

```

判定する: greatest common divisor の two 整数 s (正確 または in 正確).

```

> (gcd 6 12 8)
2

```

```

+-----+
| [手続き] |
| |
| (imag-part x) → real |
| x: number |
+-----+

```

the imaginary part from a complex number を取り出します。

```

> (imag-part 3+4i)
4

```

```

+-----+
| [手続き] |
| |
| (inexact->exact x) → number |
| x: number |
+-----+

```

Approximates in 正確 数 によって 正確 one.

```

> (inexact->exact 12.0)
12

```

```

+-----+
| [手続き] |
| |
| (inexact? x) → boolean? |
| x: number |
+-----+

```

some number is inexact かどうかを判定します。

```

> (inexact? 1-2i)
#false

```

```

+-----+
| [手続き] |
| |
| (integer->char x) → char |
+-----+

```

```
| x: exact-integer? |
+-----+
```

Looks up 文字 という corresponds へ given 正確 整数 の中の ASCII table (if any).

```
> (integer->char 42)
#*
```

```
+-----+
| [手続き] |
| |
| (integer-sqrt x) → complex |
| x: integer |
+-----+
```

the integer or imaginary-integer square root of an integer を計算します。

```
> (integer-sqrt 11)
3
> (integer-sqrt -11)
0+3i
```

```
+-----+
| [手続き] |
| |
| (integer? x) → boolean? |
| x: any/c |
+-----+
```

some value is an integer (exact or inexact) かどうかを判定します。

```
> (integer? (sqrt 2))
#false
```

```
+-----+
| [手続き] |
| |
| (lcm x y...) → integer |
| x: integer |
| y: integer |
+-----+
```

判定する: least common multiple の two 整数 s (正確 または in 正確).

```
> (lcm 6 12 8)
24
```

```
+-----+
| [手続き] |
| |
| (log x) → number |
| x: number |
+-----+
```

判定する: base-e logarithm の 数.

```
> (log 12)
#i2.4849066497880004
```

```
+-----+
| [手続き] |
| |
| (magnitude x) → real |
| x: number |
+-----+
```

判定する: magnitude の複素数 数.

```
> (magnitude (make-polar 3 4))
#i2.9999999999999996
```

```
+-----+
| [手続き] |
| |
| (make-polar x y) → number |
| x: real |
| y: real |
+-----+
```

a complex from a magnitude and angle を作成します。

```
> (make-polar 3 4)
#i-1.960930862590836-2.2704074859237844i
```

```
+-----+
| [手続き] |
| |
| (make-rectangular x y) → number |
| x: real |
| y: real |
+-----+
```

a complex from a real and an imaginary part を作成します。

```
> (make-rectangular 3 4)
3+4i
```

```
+-----+
| [手続き] |
| |
| (max x y...) → real |
| x: real |
| y: real |
+-----+
```

判定する: largest 数—aka, maximum.

```
> (max 3 2 8 7 2 9 0)
9
```

```

+-----+
| [手続き] |
| |
| (min x y...) → real |
| x: real |
| y: real |
| |
+-----+

```

判定する: smallest 数—aka, minimum.

```

> (min 3 2 8 7 2 9 0)
0

```

```

+-----+
| [手続き] |
| |
| (modulo x y) → integer |
| x: integer |
| y: integer |
| |
+-----+

```

Finds remainder の division の first 数 によって second:

```

> (modulo 9 2)
1
> (modulo 3 -4)
-1

```

```

+-----+
| [手続き] |
| |
| (negative? x) → boolean? |
| x: real |
| |
+-----+

```

some real number is strictly smaller than zero かどうかを判定します。

```

> (negative? -2)
#true

```

```

+-----+
| [手続き] |
| |
| (number->string x) → string |
| x: number |
| |
+-----+

```

a number to a string を変換します。

```

> (number->string 42)
"42"

```

```

+-----+
| [手続き] |
| |
+-----+

```

```

|
| (number->string-digits x p) → string |
| x: number |
| p: posint |
+-----+

```

a number  $x$  to a string with the specified number of digits を変換します。

```

> (number->string-digits 0.9 2)
"0.9"
> (number->string-digits pi 4)
"3.1416"

```

```

+-----+
| [手続き] |
| |
| (number? n) → boolean? |
| n: any/c |
+-----+

```

～かどうかを判定する: ある 値 は 数:

```

> (number? "hello world")
#false
> (number? 42)
#true

```

```

+-----+
| [手続き] |
| |
| (numerator x) → integer |
| x: rational? |
+-----+

```

the numerator of a rational を計算します。

```

> (numerator 2/3)
2

```

```

+-----+
| [手続き] |
| |
| (odd? x) → boolean? |
| x: integer |
+-----+

```

some integer (exact or inexact) is odd or not かどうかを判定します。

```

> (odd? 2)
#false

```

```

+-----+
| [値] |
| |
+-----+

```

```
| pi: real |
+-----+
```

ratio of the circle's circumference to its diameter.

```
> pi
#i3.141592653589793
```

```
+-----+
| [手続き] |
| |
| (positive? x) → boolean? |
| x: real |
+-----+
```

some real number is strictly larger than zeroかどうかを判定します。

```
> (positive? -2)
#false
```

```
+-----+
| [手続き] |
| |
| (quotient x y) → integer |
| x: integer |
| y: integer |
+-----+
```

Divides first 整数—also called dividend—by second—known として divisor—to obtain quotient.

```
> (quotient 9 2)
4
> (quotient 3 4)
0
```

```
+-----+
| [手続き] |
| |
| (random x) → natural |
| x: natural |
+-----+
```

Generates 乱数 数. もし given one 引数 乱数 返す natural 数 less than given natural. In ASL, もし given no 引数 s, 乱数 generates 乱数 in 正確 数 between 0.0 と 1.0 exclusive.

```
> (random)
#i0.3731303691424236
> (random)
#i0.7347732274403869
> (random 42)
6
```



```
> (random 42)
```

```
25
```

```
+-----+
| [手続き] |
| |
| (rational? x) → boolean? |
| x: any/c |
+-----+
```

some value is a rational numberかどうかを判定します。

```
> (rational? 1)
```

```
#true
```

```
> (rational? -2.349)
```

```
#true
```

```
> (rational? #i1.23456789)
```

```
#true
```

```
> (rational? (sqrt -1))
```

```
#false
```

```
> (rational? pi)
```

```
#true
```

```
> (rational? e)
```

```
#true
```

```
> (rational? 1-2i)
```

```
#false
```

As the interactions show, the teaching languages considers many more numbers as rationals than expected. In particular, pi is a rational number because it is only a finite approximation to the mathematical  $\pi$ . Think of rational? as a suggestion to think of these numbers as fractions.

```
+-----+
| [手続き] |
| |
| (real-part x) → real |
| x: number |
+-----+
```

the real part from a complex numberを取り出します。

```
> (real-part 3+4i)
```

```
3
```

```
+-----+
| [手続き] |
| |
| (real? x) → boolean? |
| x: any/c |
+-----+
```

some value is a real numberかどうかを判定します。

```
> (real? 1-2i)
#false
```

```
+-----+
| [手続き] |
| |
| (remainder x y) → integer |
| x: integer |
| y: integer |
+-----+
```

判定する: remainder の dividing first によって second 整数 (正確 または in 正確).

```
> (remainder 9 2)
1
> (remainder 3 4)
3
```

```
+-----+
| [手続き] |
| |
| (round x) → integer |
| x: real |
+-----+
```

Rounds 実数 数へ 整数 (rounds へ even へ break ties). 参照: floor と ceiling.

```
> (round 12.3)
#i12.0
```

```
+-----+
| [手続き] |
| |
| (sgn x) → (union 1 #i1.0 0 #i0.0 -1 #i-1.0) |
| x: real |
+-----+
```

判定する: sign の 実数 数.

```
> (sgn -12)
-1
```

```
+-----+
| [手続き] |
| |
| (sin x) → number |
| x: number |
+-----+
```

the sine of a number (radians) を計算します。

```
> (sin pi)
#i1.2246467991473532e-16
```

```

+-----+
| [手続き] |
| |
| (sinh x) → number |
| x: number |
+-----+

```

the hyperbolic sine of a number を計算します。

```

> (sinh 10)
#i11013.232874703393

```

```

+-----+
| [手続き] |
| |
| (sqr x) → number |
| x: number |
+-----+

```

the square of a number を計算します。

```

> (sqr 8)
64

```

```

+-----+
| [手続き] |
| |
| (sqrt x) → number |
| x: number |
+-----+

```

the square root of a number を計算します。

```

> (sqrt 9)
3
> (sqrt 2)
#i1.4142135623730951

```

```

+-----+
| [手続き] |
| |
| (sub1 x) → number |
| x: number |
+-----+

```

Decrements given 数.

```

> (sub1 2)
1

```

```

+-----+
| [手続き] |
| |
| (tan x) → number |
+-----+

```

```
| x: number |
+-----+
```

the tangent of a number (radians) を計算します。

```
> (tan pi)
#i-1.2246467991473532e-16
```

```
+-----+
| [手続き] |
| |
| (zero? x) → boolean? |
| x: number |
+-----+
```

some number is zero or not かどうかを判定します。

```
> (zero? 2)
#false
```

## 5.8 真偽値

```
+-----+
| [手続き] |
| |
| (boolean->string x) → string |
| x: boolean? |
+-----+
```

Produces 文字列 のための given 真偽値

```
> (boolean->string #false)
"#false"
> (boolean->string #true)
"#true"
```

```
+-----+
| [手続き] |
| |
| (boolean=? x y) → boolean? |
| x: boolean? |
| y: boolean? |
+-----+
```

two booleans are equal かどうかを判定します。

```
> (boolean=? #true #false)
#false
```

```
+-----+
| [手続き] |
| |
| (boolean? x) → boolean? |
| x: any/c |
+-----+
```

値が真偽値かどうかを判定します。

```
> (boolean? 42)
#false
> (boolean? #false)
#true
```

```
+-----+
| [手続き] |
| |
| (false? x) → boolean? |
| x: any/c |
+-----+
```

a value is false かどうかを判定します。

```
> (false? #false)
#true
```

```
+-----+
| [手続き] |
| |
| (not x) → boolean? |
| x: boolean? |
+-----+
```

Negates 真偽値 値.

```
> (not #false)
#true
```

## 5.9 シンボル

```
+-----+
| [手続き] |
| |
| (symbol->string x) → string |
| x: symbol |
+-----+
```

a symbol to a string を変換します。

```
> (symbol->string 'c)
"c"
```

```
+-----+
| [手続き] |
| |
| (symbol=? x y) → boolean? |
| x: symbol |
| y: symbol |
+-----+
```

two symbols are equal かどうかを判定します。

```
> (symbol=? 'a 'b)
#false
```

```
+-----+
| [手続き] |
| |
| (symbol? x) → boolean? |
| x: any/c |
+-----+
```

値がシンボルかどうかを判定します。

```
> (symbol? 'a)
#true
```

## 5.10 リスト

```
+-----+
| [手続き] |
| |
| (append l...) → (listof any) |
| l: (listof any) |
+-----+
```

a single list from several. In ASL, list\* also deals with cyclic lists を作成します。

```
+-----+
| [手続き] |
| |
| (assoc x l) → (union (listof any) #false) |
| x: any/c |
| l: (listof any) |
+-----+
```

the first pair on l whose first is equal? to x; otherwise it produces #false を生成します。

```
> (assoc "hello" '(("world" 2) ("hello" 3) ("good" 0)))
(list "hello" 3)
```

```
+-----+
| [手続き] |
| |
| (assq x l) → (union #false cons?) |
| x: any/c |
| l: list? |
+-----+
```

～かどうかを判定する: ある item は first item の pair の中の リスト の pairs. (それ compares items とともに eq?.)

```
> a
(list (list 'a 22) (list 'b 8) (list 'c 70))
> (assq 'b a)
(list 'b 8)
```

```

+-----+
| [手続き] |
| |
| (caaar x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(car(carx)))。

```

> w
(list (list (list (list "bye") 3) #true) 42)
> (caaar w)
(list "bye")

```

```

+-----+
| [手続き] |
| |
| (caadr x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(car(cdrx)))。

```

> (caadr (cons 1 (cons (cons 'a '()) (cons (cons 'd '()) '()))))
'a

```

```

+-----+
| [手続き] |
| |
| (caar x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(carx))。

```

> y
(list (list (list 1 2 3) #false "world"))
> (caar y)
(list 1 2 3)

```

```

+-----+
| [手続き] |
| |
| (cadar x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(cdr(carx)))。

```

> w
(list (list (list (list "bye") 3) #true) 42)
> (cadar w)
#true

```

```

+-----+
| [手続き] |
| |
| (caddr x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(cdr(cdr(cdrx)))).

```

> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (caddr v)
4

```

```

+-----+
| [手続き] |
| |
| (caddr x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(cdr(cdrx))).

```

> x
(list 2 "hello" #true)
> (caddr x)
#true

```

```

+-----+
| [手続き] |
| |
| (cadr x) → any/c |
| x: list? |
+-----+

```

LISP 風の selector: (car(cdrx)).

```

> x
(list 2 "hello" #true)
> (cadr x)
"hello"

```

```

+-----+
| [手続き] |
| |
| (car x) → any/c |
| x: cons? |
+-----+

```

空でないリストの先頭要素を取り出します。

```

> x
(list 2 "hello" #true)

```



```
> (car x)
```

```
2
```

```
+-----+
| [手続き] |
| |
| (cdaar x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (cdr(car(carx)))。

```
> w
```

```
(list (list (list (list "bye") 3) #true) 42)
```

```
> (cdaar w)
```

```
(list 3)
```

```
+-----+
| [手続き] |
| |
| (cdadr x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (cdr(car(cdrx)))。

```
> (cdadr (list 1 (list 2 "a") 3))
```

```
(list "a")
```

```
+-----+
| [手続き] |
| |
| (cdar x) → list? |
| x: list? |
+-----+
```

LISP 風の selector: (cdr(carx))。

```
> y
```

```
(list (list (list 1 2 3) #false "world"))
```

```
> (cdar y)
```

```
(list #false "world")
```

```
+-----+
| [手続き] |
| |
| (cddar x) → any/c |
| x: list? |
+-----+
```

LISP-style selector: (cdr(cdr(carx)))

```
> w
```

```
(list (list (list (list "bye") 3) #true) 42)
```

```
> (cddar w)
'()
```

```
+-----+
| [手続き] |
| |
| (cdddr x) → any/c |
| x: list? |
+-----+
```

LISP 風の selector: (cdr(cdr(cdrx)))。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (cdddr v)
(list 4 5 6 7 8 9 'A)
```

```
+-----+
| [手続き] |
| |
| (cddr x) → list? |
| x: list? |
+-----+
```

LISP 風の selector: (cdr(cdrx))。

```
> x
(list 2 "hello" #true)
> (cddr x)
(list #true)
```

```
+-----+
| [手続き] |
| |
| (cdr x) → any/c |
| x: cons? |
+-----+
```

空でないリストの残り (rest) を取り出します。

```
> x
(list 2 "hello" #true)
> (cdr x)
(list "hello" #true)
```

```
+-----+
| [手続き] |
| |
| (cons x l) → (listof X) |
| x: X |
| l: (listof X) |
+-----+
```

a list. In ASL, cons creates a mutable list を構築します。

```

+-----+
| [手続き] |
| |
| (cons? x) → boolean? |
| x: any/c |
+-----+

```

some value is a constructed listかどうかを判定します。

```

> (cons? (cons 1 '()))
#true
> (cons? 42)
#false

```

```

+-----+
| [手続き] |
| |
| (eighth x) → any/c |
| x: list? |
+-----+

```

the eighth item of a non-empty list を選択します。

```

> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (eighth v)
8

```

```

+-----+
| [手続き] |
| |
| (empty? x) → boolean? |
| x: any/c |
+-----+

```

値が空リストかどうかを判定します。

```

> (empty? '())
#true
> (empty? 42)
#false

```

```

+-----+
| [手続き] |
| |
| (fifth x) → any/c |
| x: list? |
+-----+

```

the fifth item of a non-empty list を選択します。

```

> v
(list 1 2 3 4 5 6 7 8 9 'A)

```

```
> (fifth v)
```

```
5
```

```
+-----+
| [手続き] |
| |
| (first x) → any/c |
| x: cons? |
+-----+
```

空でないリストの先頭要素を取り出します。

```
> x
```

```
(list 2 "hello" #true)
```

```
> (first x)
```

```
2
```

```
+-----+
| [手続き] |
| |
| (for-each f l...) → void? |
| f: (any... → any) |
| l: (listof any) |
+-----+
```

Applies 関数へ 各 item 上の one または more リスト s のための effect only:

```
(for-each f (list x-1... x-n)) = (begin (f x-1)... (f x-n))
```

```
> (for-each (lambda (x) (begin (display x) (newline))) '(1 2 3))
```

```
123
```

```
+-----+
| [手続き] |
| |
| (fourth x) → any/c |
| x: list? |
+-----+
```

the fourth item of a non-empty list を選択します。

```
> v
```

```
(list 1 2 3 4 5 6 7 8 9 'A)
```

```
> (fourth v)
```

```
4
```

```
+-----+
| [手続き] |
| |
| (length l) → natural? |
| l: list? |
+-----+
```

the number of items on a list を評価します。

```
> x
(list 2 "hello" #true)
> (length x)
3
```

```
+-----+
| [手続き] |
| |
| (list x...) → list? |
| x: any/c |
+-----+
```

a list of its arguments を構築します。

```
> (list 1 2 3 4 5 6 7 8 9 0)
(cons 1 (cons 2 (cons 3 (cons 4 (cons 5 (cons 6 (cons 7 (cons 8 (cons 9 (cons 0 '()))))))))))
```

```
+-----+
| [手続き] |
| |
| (list* x... l) → (listof any) |
| x: any |
| l: (listof any) |
+-----+
```

a list by adding multiple items to a list. In ASL, list\* also deals with cyclic lists を構築します。

```
+-----+
| [手続き] |
| |
| (list-ref x i) → any/c |
| x: list? |
| i: natural? |
+-----+
```

the indexed item from the list を取り出します。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (list-ref v 9)
'A
```

```
+-----+
| [手続き] |
| |
| (list? x) → boolean? |
| x: any/c |
+-----+
```

whether the given value is a list を検査します。

```
> (list? 42)
#false
```

```
> (list? '())
#true
> (list? (cons 1 (cons 2 '())))
#true
```

```
+-----+
| [手続き] |
| |
| (make-list i x) → list? |
| i: natural? |
| x: any/c |
+-----+
```

a list of i copies of x を構築します。

```
> (make-list 3 "hello")
(cons "hello" (cons "hello" (cons "hello" '())))
```

```
+-----+
| [手続き] |
| |
| (member x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

値がリスト上にあるかどうかを判定します (equal? で比較)。

```
> x
(list 2 "hello" #true)
> (member "hello" x)
#true
```

```
+-----+
| [手続き] |
| |
| (member? x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

値がリスト上にあるかどうかを判定します (equal? で比較)。

```
> x
(list 2 "hello" #true)
> (member? "hello" x)
#true
```

```
+-----+
| [手続き] |
| |
| (memq x l) → boolean? |
| x: any/c |
+-----+
```

```
| l: list? |
+-----+
```

some value x is on some list l, using eq? to compare x with items on lかどうかを判定します。

```
> x
(list 2 "hello" #true)
> (memq (list (list 1 2 3)) x)
#false
```

```
+-----+
| [手続き] |
| |
| (memq? x l) → boolean? |
| x: any/c |
| l: list? |
+-----+
```

some value x is on some list l, using eq? to compare x with items on lかどうかを判定します。

```
> x
(list 2 "hello" #true)
> (memq? (list (list 1 2 3)) x)
#false
```

```
+-----+
| [手続き] |
| |
| (memv x l) → (or/c #false list) |
| x: any/c |
| l: list? |
+-----+
```

〜かどうかを判定する: ある 値 は 上の リスト もし so, it produces suffix の リスト という startsとともに x もし not, it produces false. (それ compares 値 sとともに eqv? 述語.)

```
> x
(list 2 "hello" #true)
> (memv (list (list 1 2 3)) x)
#false
```

```
+-----+
| [値] |
| |
| null: list |
+-----+
```

Another name のための 空リスト

```
> null
'()
```

```

+-----+
| [手続き] |
| |
| (null? x) → boolean? |
| x: any/c |
+-----+

```

値が空リストかどうかを判定します。

```

> (null? '())
#true
> (null? 42)
#false

```

```

+-----+
| [手続き] |
| |
| (range start end step) → list? |
| start: number |
| end: number |
| step: number |
+-----+

```

a list of numbers by stepping from start to end を構築します。

```

> (range 0 10 2)
(cons 0 (cons 2 (cons 4 (cons 6 (cons 8 '())))))

```

```

+-----+
| [手続き] |
| |
| (remove x l) → list? |
| x: any/c |
| l: list? |
+-----+

```

a list like the given one, with the first occurrence of the given item removed (comparing values with equal?) を構築します。

```

> x
(list 2 "hello" #true)
> (remove "hello" x)
(list 2 #true)
> hello-2
(list 2 "hello" #true "hello")
> (remove "hello" hello-2)
(list 2 #true "hello")

```

```

+-----+
| [手続き] |
| |
| (remove-all x l) → list? |
| x: any/c |
+-----+

```



```
| l: list? |
+-----+
```

a list like the given one, with all occurrences of the given item removed (comparing values with equal?) を構築します。

```
> x
(list 2 "hello" #true)
> (remove-all "hello" x)
(list 2 #true)
> hello-2
(list 2 "hello" #true "hello")
> (remove-all "hello" hello-2)
(list 2 #true)
```

```
+-----+
| [手続き] |
| |
| (rest x) → any/c |
| x: cons? |
+-----+
```

空でないリストの残り (rest) を取り出します。

```
> x
(list 2 "hello" #true)
> (rest x)
(list "hello" #true)
```

```
+-----+
| [手続き] |
| |
| (reverse l) → list |
| l: list? |
+-----+
```

a reversed version of a list を作成します。

```
> x
(list 2 "hello" #true)
> (reverse x)
(list #true "hello" 2)
```

```
+-----+
| [手続き] |
| |
| (second x) → any/c |
| x: list? |
+-----+
```

the second item of a non-empty list を選択します。

```
> x
(list 2 "hello" #true)
```

```
> (second x)
"hello"
```

```
+-----+
| [手続き] |
| |
| (seventh x) → any/c |
| x: list? |
+-----+
```

the seventh item of a non-empty list を選択します。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (seventh v)
7
```

```
+-----+
| [手続き] |
| |
| (sixth x) → any/c |
| x: list? |
+-----+
```

the sixth item of a non-empty list を選択します。

```
> v
(list 1 2 3 4 5 6 7 8 9 'A)
> (sixth v)
6
```

```
+-----+
| [手続き] |
| |
| (third x) → any/c |
| x: list? |
+-----+
```

the third item of a non-empty list を選択します。

```
> x
(list 2 "hello" #true)
> (third x)
#true
```

## 5.11 Posn

```
+-----+
| [手続き] |
| |
| (make-posn x y) → posn |
| x: any/c |
| y: any/c |
+-----+
```

a posn from two arbitrary values を構築します。

```
> (make-posn 3 3)
(make-posn 3 3)
> (make-posn "hello" #true)
(make-posn "hello" #true)
```

```
+-----+
| [手続き] |
| |
| (posn-x p) → any/c |
| p: posn |
+-----+
```

the x component of a posn を取り出します。

```
> p
(make-posn 2 -3)
> (posn-x p)
2
```

```
+-----+
| [手続き] |
| |
| (posn-y p) → any/c |
| p: posn |
+-----+
```

the y component of a posn を取り出します。

```
> p
(make-posn 2 -3)
> (posn-y p)
-3
```

```
+-----+
| [手続き] |
| |
| (posn? x) → boolean? |
| x: any/c |
+-----+
```

its input is a posn かどうかを判定します。

```
> q
(make-posn "bye" 2)
> (posn? q)
#true
> (posn? 42)
#false
```

```
+-----+
| [手続き] |
| |
| |
+-----+
```

```
| (set-posn-x! p x) → void? |
| p: posn |
| x: any |
+-----+
```

the x component of a posn を更新します。

```
> p
(make-posn 2 -3)
> (set-posn-x! p 678)
> p
(make-posn 678 -3)
```

```
+-----+
| [手続き] |
| |
| (set-posn-y! p x) → void |
| p: posn |
| x: any |
+-----+
```

the y component of a posn を更新します。

```
> q
(make-posn "bye" 2)
> (set-posn-y! q 678)
> q
(make-posn "bye" 678)
```

## 5.12 文字

```
+-----+
| [手続き] |
| |
| (char->integer c) → integer |
| c: char |
+-----+
```

Looks up 数 という corresponds へ given 文字 の中の ASCII table (if any).

```
> (char->integer #\a)
97
> (char->integer #\z)
122
```

```
+-----+
| [手続き] |
| |
| (char-alphabetic? c) → boolean? |
| c: char |
+-----+
```

a character represents an alphabetic characterかどうかを判定します。

```
> (char-alphabetic? #\Q)
#true
```

```
+-----+
| [手続き] |
| |
| (char-ci=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are ordered in an increasing and case-insensitive mannerかどうかを判定します。

```
> (char-ci=? #\b #\B)
#true
> (char=? #\b #\B)
#false
```

```
+-----+
| [手続き] |
| |
| (char-ci<? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are ordered in a strictly increasing and case-insensitive mannerかどうかを判定します。

```
> (char-ci<? #\B #\C)
#true
> (char<? #\b #\B)
#false
```

```
+-----+
| [手続き] |
| |
| (char-ci=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

two characters are equal in a case-insensitive mannerかどうかを判定します。

```
> (char-ci=? #\b #\B)
#true
```

```
+-----+
| [手続き] |
| |
```

```
|
| (char-ci>=? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are sorted in a decreasing and case-insensitive mannerかどうかを判定します。

```
> (char-ci>=? #\b #\C)
#false
> (char>=? #\b #\C)
#true
```

```
+-----+
| [手続き] |
|
| (char-ci>? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+
```

the characters are sorted in a strictly decreasing and case-insensitive mannerかどうかを判定します。

```
> (char-ci>? #\b #\B)
#false
> (char>? #\b #\B)
#true
```

```
+-----+
| [手続き] |
|
| (char-downcase c) → char |
| c: char |
+-----+
```

the equivalent lower-case character を生成します。

```
> (char-downcase #\T)
#\t
```

```
+-----+
| [手続き] |
|
| (char-lower-case? c) → boolean? |
| c: char |
+-----+
```

a character is a lower-case characterかどうかを判定します。

```
> (char-lower-case? #\T)
#false
```

```
+-----+
| [手続き] |
| |
| (char-numeric? c) → boolean? |
| c: char |
+-----+
```

a character represents a digit かどうかを判定します。

```
> (char-numeric? #\9)
#true
```

```
+-----+
| [手続き] |
| |
| (char-upcase c) → char |
| c: char |
+-----+
```

the equivalent upper-case character を生成します。

```
> (char-upcase #\t)
#\T
```

```
+-----+
| [手続き] |
| |
| (char-upper-case? c) → boolean? |
| c: char |
+-----+
```

a character is an upper-case character かどうかを判定します。

```
> (char-upper-case? #\T)
#true
```

```
+-----+
| [手続き] |
| |
| (char-whitespace? c) → boolean? |
| c: char |
+-----+
```

a character represents space かどうかを判定します。

```
> (char-whitespace? #\tab)
#true
```

```
+-----+
| [手続き] |
| |
| (char<=? c d e...) → boolean? |
+-----+
```

```

| c: char
| d: char
| e: char
+-----+

```

the characters are ordered in an increasing mannerかどうかを判定します。

```

> (char<=? #\a #\a #\b)
#true

```

```

+-----+
| [手続き]
|
| (char<? x d e...) → boolean?
| x: char
| d: char
| e: char
+-----+

```

the characters are ordered in a strictly increasing mannerかどうかを判定します。

```

> (char<? #\a #\b #\c)
#true

```

```

+-----+
| [手続き]
|
| (char=? c d e...) → boolean?
| c: char
| d: char
| e: char
+-----+

```

the characters are equalかどうかを判定します。

```

> (char=? #\b #\a)
#false

```

```

+-----+
| [手続き]
|
| (char>=? c d e...) → boolean?
| c: char
| d: char
| e: char
+-----+

```

the characters are sorted in a decreasing mannerかどうかを判定します。

```

> (char>=? #\b #\b #\a)
#true

```

```

+-----+
| [手続き]
|

```



```

| (char? c d e...) → boolean? |
| c: char |
| d: char |
| e: char |
+-----+

```

the characters are sorted in a strictly decreasing mannerかどうかを判定します。

```

> (char? #\A #\z #\a)
#false

```

```

+-----+
| [手続き] |
| (char? x) → boolean? |
| x: any/c |
+-----+

```

a value is a characterかどうかを判定します。

```

> (char? "a")
#false
> (char? #\a)
#true

```

### 5.13 文字列

```

+-----+
| [手続き] |
| (explode s) → (listof string) |
| s: string |
+-----+

```

Translates 文字列へリストの1-letter 文字列 s.

```

> (explode "cat")
(list "c" "a" "t")

```

```

+-----+
| [手続き] |
| (format f x...) → string |
| f: string |
| x: any/c |
+-----+

```

Formats 文字列, possibly embedding 値 s.

```

> (format "Dear Dr. ~a:" "Flatt")
"Dear Dr. Flatt:"
> (format "Dear Dr. ~s:" "Flatt")
"Dear Dr. \"Flatt\":"

```

```

+-----+
| [手続き] |
| |
| (implode l) → string |
| l: list? |
+-----+

```

Concatenates リスト の 1-letter 文字列 s へ one 文字列.

```

> (implode (cons "c" (cons "a" (cons "t" '()))))
"cat"

```

```

+-----+
| [手続き] |
| |
| (int->string i) → string |
| i: integer |
+-----+

```

an integer in [0,55295] or [57344 1114111] to a 1-letter string を変換します。

```

> (int->string 65)
"A"

```

```

+-----+
| [手続き] |
| |
| (list->string l) → string |
| l: list? |
+-----+

```

a s list of characters into a string を変換します。

```

> (list->string (cons #\c (cons #\a (cons #\t '()))))
"cat"

```

```

+-----+
| [手続き] |
| |
| (make-string i c) → string |
| i: natural? |
| c: char |
+-----+

```

a string of length i from c を生成します。

```

> (make-string 3 #\d)
"ddd"

```

```

+-----+
| [手続き] |
| |
| (replicate i s) → string |
| i: natural? |
+-----+

```

```
| s: string |
+-----+
```

Replicates s i times.

```
> (replicate 3 "h")
"hhh"
```

```
+-----+
| [手続き] |
| |
| (string c...) → string? |
| c: char |
+-----+
```

Builds 文字列 of the given 文字 s.

```
> (string #\d #\o #\g)
"dog"
```

```
+-----+
| [手続き] |
| |
| (string->int s) → integer |
| s: string |
+-----+
```

a 1-letter string to an integer in [0,55295] or [57344, 1114111] を変換します。

```
> (string->int "a")
97
```

```
+-----+
| [手続き] |
| |
| (string->list s) → (listof char) |
| s: string |
+-----+
```

a string into a list of characters を変換します。

```
> (string->list "hello")
(list #\h #\e #\l #\l #\o)
```

```
+-----+
| [手続き] |
| |
| (string->number s) → (union number #false) |
| s: string |
+-----+
```

a string into a number, produce false if impossible を変換します。

```
> (string->number "-2.03")
-2.03
```

```
> (string->number "1-2i")
```

```
1-2i
```

```
+-----+
| [手続き] |
| |
| (string->symbol s) → symbol |
| s: string |
+-----+
```

a string into a symbol を変換します。

```
> (string->symbol "hello")
```

```
'hello
```

```
+-----+
| [手続き] |
| |
| (string-alphabetic? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are alphabeticかどうかを判定します。

```
> (string-alphabetic? "123")
```

```
#false
```

```
> (string-alphabetic? "cat")
```

```
#true
```

```
+-----+
| [手続き] |
| |
| (string-contains-ci? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

the first string appears in the second one without regard to the case of the lettersかどうかを判定します。

```
> (string-contains-ci? "At" "caT")
```

```
#true
```

```
+-----+
| [手続き] |
| |
| (string-contains? s t) → boolean? |
| s: string |
| t: string |
+-----+
```

the first string appears literally in the second oneかどうかを判定します。

```
> (string-contains? "at" "cat")
#true
```

```
+-----+
| [手続き] |
| |
| (string-copy s) → string |
| s: string |
+-----+
```

Copies 文字列.

```
> (string-copy "hello")
"hello"
```

```
+-----+
| [手続き] |
| |
| (string-downcase s) → string |
| s: string |
+-----+
```

a string like the given one with all ‘letters’ as lower case を生成します。

```
> (string-downcase "CAT")
"cat"
> (string-downcase "cAt")
"cat"
```

```
+-----+
| [手続き] |
| |
| (string-ith s i) → 1string? |
| s: string |
| i: natural? |
+-----+
```

the ith 1-letter substring from s を取り出します。

```
> (string-ith "hello world" 1)
"e"
```

```
+-----+
| [手続き] |
| |
| (string-length s) → nat |
| s: string |
+-----+
```

判定する: length の 文字列.

```
> (string-length "hello world")
11
```

```
+-----+
| [手続き] |
| |
| (string-lower-case? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are lower case かどうかを判定します。

```
> (string-lower-case? "CAT")
#false
```

```
+-----+
| [手続き] |
| |
| (string-numeric? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are numeric かどうかを判定します。

```
> (string-numeric? "123")
#true
> (string-numeric? "1-2i")
#false
```

```
+-----+
| [手続き] |
| |
| (string-ref s i) → char |
| s: string |
| i: natural? |
+-----+
```

the ith character from s を取り出します。

```
> (string-ref "cat" 2)
#\t
```

```
+-----+
| [手続き] |
| |
| (string-upcase s) → string |
| s: string |
+-----+
```

a string like the given one with all 'letters' as upper case を生成します。

```
> (string-upcase "cat")
"CAT"
> (string-upcase "cAt")
"CAT"
```

```
+-----+
| [手続き] |
| |
| (string-upper-case? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are upper caseかどうかを判定します。

```
> (string-upper-case? "CAT")
#true
```

```
+-----+
| [手続き] |
| |
| (string-whitespace? s) → boolean? |
| s: string |
+-----+
```

all 'letters' in the string are white spaceかどうかを判定します。

```
> (string-whitespace? (string-append " " (string #\tab #\newline #\return)))
#true
```

```
+-----+
| [手続き] |
| |
| (string? x) → boolean? |
| x: any/c |
+-----+
```

a value is a stringかどうかを判定します。

```
> (string? "hello world")
#true
> (string? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (substring s i j) → string |
| s: string |
| i: natural? |
| j: natural? |
+-----+
```

the substring starting at i up to j (or the end if j is not provided) を取り出します。

```
> (substring "hello world" 1 5)
"ello"
> (substring "hello world" 1 8)
"ello wo"
```

```
> (substring "hello world" 4)
"o world"
```

## 5.14 画像

```
+-----+
| [手続き] |
| |
| (image=? i j) → boolean? |
| i: image |
| j: image |
+-----+
```

two images are equal かどうかを判定します。

```
> c1
[image:pict_13.png]
> (image=? (circle 5 "solid" "green") c1)
#false
> (image=? (circle 10 "solid" "green") c1)
#true
```

```
+-----+
| [手続き] |
| |
| (image? x) → boolean? |
| x: any/c |
+-----+
```

a value is an image かどうかを判定します。

```
> c1
[image:pict_14.png]
> (image? c1)
#true
```

## 5.15 その他

```
+-----+
| [手続き] |
| |
| (≈ x y eps) → boolean? |
| x: number |
| y: number |
| eps: non-negative-real |
+-----+
```

whether x and y are within eps of either other を検査します。

```
> (≈ 1.01 1.0 0.1)
#true
> (≈ 1.01 1.5 0.1)
#false
```



```

+-----+
| [手続き] |
| |
| (current-milliseconds) → exact-integer |
+-----+

```

the current “time” in fixnum milliseconds (possibly negative) を返します。

```

> (current-milliseconds)
1779843342996

```

```

+-----+
| [値] |
| |
| eof: eof-object? |
+-----+

```

A 値 という represents end の file:

```

> eof
#<eof>

```

```

+-----+
| [手続き] |
| |
| (eof-object? x) → boolean? |
| x: any/c |
+-----+

```

some value is the end-of-file value かどうかを判定します。

```

> (eof-object? eof)
#true
> (eof-object? 42)
#false

```

```

+-----+
| [手続き] |
| |
| (eq? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+

```

two values are equivalent from the computer’s perspective (intensional) かどうかを判定します。

```

> (eq? (cons 1 '()) (cons 1 '()))
#false
> one
(list 1)
> (eq? one one)
#true

```

```

+-----+
| [手続き] |
| |
| (equal? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+

```

two values are structurally equal where basic values are compared with the eqv? predicate かどうかを判定します。

```

> (equal? (make-posn 1 2) (make-posn (- 2 1) (+ 1 1)))
#true

```

```

+-----+
| [手続き] |
| |
| (equal~? x y z) → boolean? |
| x: any/c |
| y: any/c |
| z: non-negative-real |
+-----+

```

x and y like equal? but uses =~ in the case of numbers を比較します。

```

> (equal~? (make-posn 1.01 1.0) (make-posn 1.01 0.99) 0.2)
#true

```

```

+-----+
| [手続き] |
| |
| (eqv? x y) → boolean? |
| x: any/c |
| y: any/c |
+-----+

```

two values are equivalent from the perspective of all functions that can be applied to it (extensional) かどうかを判定します。

```

> (eqv? (cons 1 '()) (cons 1 '()))
#false
> one
(list 1)
> (eqv? one one)
#true

```

```

+-----+
| [手続き] |
| |
| (error x...) → void? |
| x: any/c |
+-----+

```

Signals an error, combining the given values into an error message. If any of the values' printed representations is too long, it is truncated and "...” is put into the string. If the first value is a symbol, it is suffixed with a colon and the result pre-pended on to the error message.

```
> zero
0
> (if (= zero 0) (error "can't divide by 0") (/ 1 zero))
can't divide by 0
```

```
+-----+
| [手続き] |
| |
| (exit) → void |
+-----+
```

Evaluating (exit) terminates running プログラム.

```
+-----+
| [手続き] |
| |
| (force v) → any |
| v: any |
+-----+
```

the delayed value; see also delay を探します。

```
+-----+
| [手続き] |
| |
| (gensym) → symbol? |
+-----+
```

Generates new シンボル, different から すべての シンボル s の中の プログラム.

```
> (gensym)
'g8579052
```

```
+-----+
| [手続き] |
| |
| (identity x) → any/c |
| x: any/c |
+-----+
```

x を返します。

```
> (identity 42)
42
> (identity c1)
[image:pict_15.png]
> (identity "hello")
"hello"
```

```

+-----+
| [手続き] |
| |
| (promise? x) → boolean? |
| x: any |
+-----+

```

a value is delayedかどうかを判定します。

```

+-----+
| [手続き] |
| |
| (sleep sec) → void |
| sec: positive-num |
+-----+

```

Causes プログラムへ sleep のための given 数の seconds.

```

+-----+
| [手続き] |
| |
| (struct? x) → boolean? |
| x: any/c |
+-----+

```

some value is a structureかどうかを判定します。

```

> (struct? (make-posn 1 2))
#true
> (struct? 43)
#false

```

```

+-----+
| [手続き] |
| |
| (void) → void? |
+-----+

```

a void value を生成します。

```

> (void)

```

```

+-----+
| [手続き] |
| |
| (void? x) → boolean? |
| x: any |
+-----+

```

a value is voidかどうかを判定します。

```

> (void? (void))
#true

```

```
> (void? 42)
#false
```

## 5.16 シグネチャ

```
+-----+
| [値] |
| |
| Any: signature? |
+-----+
```

any value のシグネチャ。

```
+-----+
| [値] |
| |
| Boolean: signature? |
+-----+
```

booleans のシグネチャ。

```
+-----+
| [値] |
| |
| Char: signature? |
+-----+
```

characters のシグネチャ。

```
+-----+
| [手続き] |
| |
| (ConsOf first-sig rest-sig) → signature? |
| first-sig: signature? |
| rest-sig: signature? |
+-----+
```

a cons pair のシグネチャ。

```
+-----+
| [値] |
| |
| EmptyList: signature? |
+-----+
```

the empty list のシグネチャ。

```
+-----+
| [値] |
| |
| False: signature? |
+-----+
```

just false のシグネチャ。

```

+-----+
| [値] |
| Integer: signature? |
+-----+

```

integers のシグネチャ。

```

+-----+
| [値] |
| Natural: signature? |
+-----+

```

natural numbers のシグネチャ。

```

+-----+
| [値] |
| Number: signature? |
+-----+

```

arbitrary numbers のシグネチャ。

```

+-----+
| [値] |
| Rational: signature? |
+-----+

```

rational numbers のシグネチャ。

```

+-----+
| [値] |
| Real: signature? |
+-----+

```

real numbers のシグネチャ。

```

+-----+
| [値] |
| String: signature? |
+-----+

```

strings のシグネチャ。

```

+-----+
| [値] |
| Symbol: signature? |
+-----+

```

symbols のシグネチャ。

```
+-----+
| [値] |
| |
| True: signature? |
+-----+
```

just true のシグネチャ。

## 5.17 数（緩和条件）

## 5.18 文字列（緩和条件）

```
+-----+
| [手続き] |
| |
| (string-append s...) → string |
| s: string |
+-----+
```

Concatenates 文字 s の several 文字列 s.

```
> (string-append "hello" " " "world" " " "good bye")
"hello world good bye"
```

```
+-----+
| [手続き] |
| |
| (string-ci<=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically increasing and case-insensitive manner  
かどうかを判定します。

```
> (string-ci<=? "hello" "WORLD" "zoo")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci<? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically strictly increasing and case-insensitive  
manner かどうかを判定します。

```
> (string-ci=? "hello" "WORLD" "zoo")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

all strings are equal, character for character, regardless of caseかどうかを判定します。

```
> (string-ci=? "hello" "Hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci>=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically decreasing and case-insensitive mannerかどうかを判定します。

```
> (string-ci>? "zoo" "WORLD" "hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string-ci>? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically strictly decreasing and case-insensitive mannerかどうかを判定します。

```
> (string-ci>? "zoo" "WORLD" "hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string<=? s t x...) → boolean? |
| s: string |
| t: string |
+-----+
```



```
| x: string |
+-----+
```

the strings are ordered in a lexicographically increasing mannerかどうかを判定します。

```
> (string<=? "hello" "hello" "world" "zoo")
#true
```

```
+-----+
| [手続き] |
| |
| (string<? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically strictly increasing mannerかどうかを判定します。

```
> (string<? "hello" "world" "zoo")
#true
```

```
+-----+
| [手続き] |
| |
| (string=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

all strings are equal, character for characterかどうかを判定します。

```
> (string=? "hello" "world")
#false
> (string=? "bye" "bye")
#true
```

```
+-----+
| [手続き] |
| |
| (string>=? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically decreasing mannerかどうかを判定します。

```
> (string>=? "zoo" "zoo" "world" "hello")
#true
```

```
+-----+
| [手続き] |
| |
| (string>? s t x...) → boolean? |
| s: string |
| t: string |
| x: string |
+-----+
```

the strings are ordered in a lexicographically strictly decreasing mannerかどうかを判定します。

```
> (string>? "zoo" "world" "hello")
#true
```

## 5.19 Posn

```
+-----+
| [手続き] |
| |
| (posn) → signature |
+-----+
```

posns のシグネチャ。

## 5.20 高階関数

### 5.21 数（緩和条件・拡張）

```
+-----+
| [手続き] |
| |
| (* x...) → number |
| x: number |
+-----+
```

Multiplies すべての given 数 s. In ISL と up: \* works のとき applied へ only one 数 または none.

```
> (* 5 3)
15
> (* 5 3 2)
30
> (* 2)
2
> (*)
1
```

```
+-----+
| [手続き] |
| |
+-----+
```

```

|
| (+ x...) → number
| x: number
+-----+

```

Adds すべての given 数 s. In ISL と up: + works のとき applied へ only one 数 または none.

```

> (+ 2/3 1/16)
35/48
> (+ 3 2 5 8)
18
> (+ 1)
1
> (+)
0

```

```

+-----+
| [手続き] |
|
| (/ x y...) → number
| x: number
| y: number
+-----+

```

Divides first によって すべての remaining 数 s. In ISL と up: / computes inverse のとき applied へ one 数.

```

> (/ 12 2)
6
> (/ 12 2 3)
2
> (/ 3)
1/3

```

```

+-----+
| [手続き] |
|
| (= x...) → number
| x: number
+-----+

```

numbers for equality. In ISL and up: = works when applied to only one number を比較します。

```

> (= 10 10)
#true
> (= 11)
#true
> (= 0)
#true

```

## 5.22 高階関数 (lambda つき)

```
+-----+
| [手続き] |
| |
| (andmap p? [l]) → boolean |
| p?: (X... -> boolean) |
| l: (listof X) =... |
+-----+
```

～かどうかを判定する: p? holds のための すべての items の l…:

```
(andmap p (list x-1... x-n)) = (and (p x-1)... (p x-n))
(andmap p (list x-1... x-n) (list y-1... y-n)) = (and (p x-1 y-1)... (p x-n y-n))
> (andmap odd? '(1 3 5 7 9))
#true
> threshold
3
> (andmap (lambda (x) (< x threshold)) '(0 1 2))
#true
> (andmap even? '())
#true
> (andmap (lambda (x f) (f x)) (list 0 1 2) (list odd? even? positive?))
#false
```

```
+-----+
| [手続き] |
| |
| (apply f x-1... l) → Y |
| f: (X-1... X-N -> Y) |
| x-1: X-1 |
| l: (list X-i+1... X-N) |
+-----+
```

Applies 関数 using items から リスト として 引数 s:

```
(apply f (list x-1... x-n)) = (f x-1... x-n)
> a-list
(list 0 1 2 3 4 5 6 7 8 9)
> (apply max a-list)
9
```

```
+-----+
| [手続き] |
| |
| (argmax f l) → X |
| f: (X -> real) |
| l: (listof X) |
+-----+
```

the (first) element of the list that maximizes the output of the function を探します。

```
> (argmax second '((sam 98) (carl 78) (vincent 93) (asumu 99)))
(list 'asumu 99)
```

```
+-----+
| [手続き] |
| |
| (argmin f l) → X |
| f: (X → real) |
| l: (listof X) |
+-----+
```

the (first) element of the list that minimizes the output of the function を探します。

```
> (argmin second '((sam 98) (carl 78) (vincent 93) (asumu 99)))
(list 'carl 78)
```

```
+-----+
| [手続き] |
| |
| (build-list n f) → (listof X) |
| n: nat |
| f: (nat → X) |
+-----+
```

構築する: リスト によって applying f へ 数 s between 0 と (-n1):

```
(build-list n f) = (list (f 0)... (f (- n 1)))
```

```
> (build-list 22 add1)
(list 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22)
> i
3
> (build-list 3 (lambda (j) (+ j i)))
(list 3 4 5)
> (build-list 5 (lambda (i) (build-list 5
(lambda (j) (if (= i j) 1 0)))))
(list (list 1 0 0 0 0) (list 0 1 0 0 0) (list 0 0 1 0 0) (list 0 0 0 1 0) (list 0 0 0 0 1))
```

```
+-----+
| [手続き] |
| |
| (build-string n f) → string |
| n: nat |
| f: (nat → char) |
+-----+
```

構築する: 文字列 によって applying f へ 数 s between 0 と (-n1):

```
(build-string n f) = (string (f 0)... (f (- n 1)))
```

```
> (build-string 10 integer->char)
"\u0000\u0001\u0002\u0003\u0004\u0005\u0006\a\b\t"
> (build-string 26 (lambda (x) (integer->char (+ 65 x))))
"ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

```

+-----+
| [手続き] |
| |
| (compose f g) → (X → Z) |
| f: (Y → Z) |
| g: (X → Y) |
| |
+-----+

```

(説明) Composes sequence の procedures へ single procedure:

```

(compose f g) = (lambda (x) (f (g x)))
> ((compose add1 second) '(add 3))
4
> (map (compose add1 second) '((add 3) (sub 2) (mul 4)))
(list 4 3 5)

```

```

+-----+
| [手続き] |
| |
| (filter p? l) → (listof X) |
| p?: (X → boolean) |
| l: (listof X) |
| |
+-----+

```

a list from all those items on a list for which the predicate holds を構築します。

```

> (filter odd? '(0 1 2 3 4 5 6 7 8 9))
(list 1 3 5 7 9)
> threshold
3
> (filter (lambda (x) (>= x threshold)) '(0 1 2 3 4 5 6 7 8 9))
(list 3 4 5 6 7 8 9)

```

```

+-----+
| [手続き] |
| |
| (foldl f base l...) → Y |
| f: (X... Y → Y) |
| base: Y |
| l: (listof X) |
| |
+-----+

```

```

(foldl f base (list x-1... x-n)) = (f x-n... (f x-1 base))
(foldl f base (list x-1... x-n) (list x-1... x-n))
= (f x-n y-n... (f x-1 y-1 base))
> (foldl + 0 '(0 1 2 3 4 5 6 7 8 9))
45
> a-list
(list 0 1 2 3 4 5 6 7 8 9)
> (foldl (lambda (x r) (if (> x threshold) (cons (* 2 x) r) r)) '() a-list)
(list 18 16 14 12 10 8)

```

```
> (foldl (lambda (x y r) (+ x y r)) 0 '(1 2 3) '(10 11 12))
39
```

```
+-----+
| [手続き] |
| |
| (foldr f base l...) → Y |
| f: (X... Y → Y) |
| base: Y |
| l: (listof X) |
+-----+
```

```
(foldr f base (list x-1... x-n)) = (f x-1... (f x-n base))
```

```
(foldr f base (list x-1... x-n) (list y-1... y-n))
= (f x-1 y-1... (f x-n y-n base))
```

```
> (foldr + 0 '(0 1 2 3 4 5 6 7 8 9))
45
```

```
> a-list
```

```
(list 0 1 2 3 4 5 6 7 8 9)
```

```
> (foldr (lambda (x r) (if (> x threshold) (cons (* 2 x) r) r)) '() a-list)
(list 8 10 12 14 16 18)
```

```
> (foldr (lambda (x y r) (+ x y r)) 0 '(1 2 3) '(10 11 12))
39
```

```
+-----+
| [手続き] |
| |
| (map f l...) → (listof Z) |
| f: (X... → Z) |
| l: (listof X) |
+-----+
```

構築する: new リスト によって applying 関数 へ 各 item 上の one または more existing リスト s:

```
(map f (list x-1... x-n)) = (list (f x-1)... (f x-n))
```

```
(map f (list x-1... x-n) (list y-1... y-n)) = (list (f x-1 y-1)... (f x-n y-n))
```

```
> (map add1 (list 3 -4.01 2/5))
(list 4 #i-3.01 1.4)
```

```
> (define (tag-with-a x) (list "a" (+ x 1)))
tag-with-a:this name was defined previously and cannot be re-defined
```

```
> (map tag-with-a (list 3 -4.01 2/5))
(list (list "a" 4) (list "a" #i-3.01) (list "a" 1.4))
```

```
> (define (add-and-multiply x y) (+ x (* x y)))
add-and-multiply:this name was defined previously and cannot be re-defined
```

```
> (map add-and-multiply (list 3 -4 2/5) '(1 2 3))
(list 6 -12 1.6)
```

```

+-----+
| [手続き] |
| |
| (memf p? l) → (union #false (listof X)) |
| p?: (X → any) |
| l: (listof X) |
+-----+

```

#false if p? produces false for all items on l. If p? produces #true for any of the items on l, memf returns the sub-list starting from that item を生成します。

```

> (memf odd? '(2 4 6 3 8 0))
(list 3 8 0)

```

```

+-----+
| [手続き] |
| |
| (ormap p? l) → boolean |
| p?: (X → boolean) |
| l: (listof X) |
+-----+

```

～かどうかを判定する: p? holds のための at least one items の l:

```
(ormap p (list x-1... x-n)) = (or (p x-1)... (p x-n))
```

```
(ormap p (list x-1... x-n) (list y-1... y-n)) = (or (p x-1 y-1)... (p x-n y-n))
```

```
> (ormap odd? '(1 3 5 7 9))
```

```
#true
```

```
> threshold
```

```
3
```

```
> (ormap (lambda (x) (< x threshold)) '(6 7 8 1 5))
```

```
#true
```

```
> (ormap even? '())
```

```
#false
```

```
> (ormap (lambda (x f) (f x)) (list 0 1 2) (list odd? even? positive?))
```

```
#true
```

```

+-----+
| [手続き] |
| |
| (procedure? x) → boolean? |
| x: any |
+-----+

```

true if the value is a procedure を生成します。

```
> (procedure? cons)
```

```
#true
```

```
> (procedure? add1)
```

```
#true
```



```
> (procedure? (lambda (x) (> x 22)))
#true
```

```
+-----+
| [手続き] |
| |
| (quicksort l comp) → (listof X) |
| l: (listof X) |
| comp: (X X → boolean) |
+-----+
```

Sorts items 上の l, の中の order according へ comp (using quicksort algorithm).

```
> (quicksort '(6 7 2 1 3 4 0 5 9 8) <)
(list 0 1 2 3 4 5 6 7 8 9)
```

```
+-----+
| [手続き] |
| |
| (sort l comp) → (listof X) |
| l: (listof X) |
| comp: (X X → boolean) |
+-----+
```

Sorts items 上の l, の中の order according へ comp.

```
> (sort '(6 7 2 1 3 4 0 5 9 8) <)
(list 0 1 2 3 4 5 6 7 8 9)
```

## 5.23 読み取りと表示

```
+-----+
| [手続き] |
| |
| (display x) → void |
| x: any |
+-----+
```

Prints 引数 へ stdout (without quotes 上の シンボル s と 文字列 s, etc.).

```
> (display 10)
10
> (display "hello")
hello
> (display 'hello)
hello
```

```
+-----+
| [手続き] |
| |
| (newline) → void |
+-----+
```

Prints newline.

```

+-----+
| [手続き] |
| |
| (pretty-print x) → void |
| x: any |
+-----+

```

Pretty prints S-式 s (like write).

```

> (pretty-print '((1 2 3) ((a) ("hello world" #true) (((false "good bye"))))))
((1 2 3) ((a) ("hello world" #true) (((false "good bye")))))
> (pretty-print (build-list 10 (lambda (i) (build-list 10 (lambda (j) (= i j))))))
(#true #false #false #false #false #false #false #false #false #false) (#false #true #false
#false #false #false #false #false #false #false) (#false #false #true #false #false #false
#false #false #false #false) (#false #false #false #true #false #false #false #false #false
#false) (#false #false #false #false #true #false #false #false #false #false) (#false #false
#false #false #true #false #false #false #false #false) (#false #false #false #false #false
#false #true #false #false #false) (#false #false #false #false #false #false #false #false #true
#false #false) (#false #false #false #false #false #false #false #false #false #true) (#false
#false #false #false #false #false #false #false #false #true))

```

```

+-----+
| [手続き] |
| |
| (print x) → void |
| x: any |
+-----+

```

Prints 引数として 値.

```

> (print 10)
10
> (print "hello")
"hello"
> (print 'hello)
'hello

```

```

+-----+
| [手続き] |
| |
| (printf f x...) → void |
| f: string |
| x: any |
+-----+

```

Formats rest の 引数 s according へ first 引数 と print it.

```

+-----+
| [手続き] |
| |
| (read) → sexp |
+-----+

```

Reads input から user.

```
+-----+
| [手続き] |
| |
| (with-input-from-file f p) → any |
| f: string |
| p: (-> any) |
+-----+
```

Opens named input file f と allows p へ read から it.

```
+-----+
| [手続き] |
| |
| (with-input-from-string s p) → any |
| s: string |
| p: (-> any) |
+-----+
```

Turns s へ input のための read operations の中の p.

```
> (with-input-from-string "hello" read)
'hello
> (string-length (symbol->string (with-input-from-string "hello" read)))
5
```

```
+-----+
| [手続き] |
| |
| (with-output-to-file f p) → any |
| f: string |
| p: (-> any) |
+-----+
```

Opens named output file f と allows p へ write へ it.

```
+-----+
| [手続き] |
| |
| (with-output-to-string p) → any |
| p: (-> any) |
+-----+
```

a string from all write/display/print operations in p を生成します。

```
> (with-output-to-string (lambda () (display 10)))
"10"
```

```
+-----+
| [手続き] |
| |
| (write x) → void |
+-----+
```

```
| x: any |
+-----+
```

Prints 引数  $\rightarrow$  stdout (in traditional style という は somewhere between print と display).

```
> (write 10)
10
> (write "hello")
"hello"
> (write 'hello)
hello
```

## 5.24 ベクタ

```
+-----+
| [手続き] |
| |
| (build-vector n f) $\rightarrow$ (vectorof X) |
| n: nat |
| f: (nat $\rightarrow$ X) |
+-----+
```

a vector by applying f to the numbers 0 through (-n1) を構築します。

```
> (build-vector 5 add1)
(vector 1 2 3 4 5)
```

```
+-----+
| [手続き] |
| |
| (list->vector l) $\rightarrow$ (vectorof X) |
| l: (listof X) |
+-----+
```

Transforms l  $\rightarrow$  vector.

```
> (list->vector (list "hello" "world" "good" "bye"))
(vector "hello" "world" "good" "bye")
```

```
+-----+
| [手続き] |
| |
| (make-vector n x) $\rightarrow$ (vectorof X) |
| n: number |
| x: X |
+-----+
```

a vector of n copies of x を構築します。

```
> (make-vector 5 0)
(vector 0 0 0 0 0)
```

```

+-----+
| [手続き] |
| |
| (vector x...) → (vector X...) |
| x: X |
+-----+

```

a vector from the given values を構築します。

```

> (vector 1 2 3 -1 -2 -3)
(vector 1 2 3 -1 -2 -3)

```

```

+-----+
| [手続き] |
| |
| (vector->list v) → (listof X) |
| v: (vectorof X) |
+-----+

```

Transforms v へ リスト.

```

> (vector->list (vector 'a 'b 'c))
(list 'a 'b 'c)

```

```

+-----+
| [手続き] |
| |
| (vector-length v) → nat |
| v: (vector X) |
+-----+

```

判定する: length の v.

```

> v
(vector "a" "b" "c" "d" "e")
> (vector-length v)
5

```

```

+-----+
| [手続き] |
| |
| (vector-ref v n) → X |
| v: (vector X) |
| n: nat |
+-----+

```

the nth element from v を取り出します。

```

> v
(vector "a" "b" "c" "d" "e")
> (vector-ref v 3)
"d"

```

```

+-----+
| [手続き] |
| |
| (vector-set! v n x) → void |
| v: (vectorof X) |
| n: nat |
| x: X |
+-----+

```

v at position n to be x を更新します。

```

> v
(vector "a" "b" "c" "d" "e")
> (vector-set! v 3 77)
> v
(vector "a" "b" "c" 77 "e")

```

```

+-----+
| [手続き] |
| |
| (vector? x) → boolean |
| x: any |
+-----+

```

a value is a vector かどうかを判定します。

```

> v
(vector "a" "b" "c" 77 "e")
> (vector? v)
#true
> (vector? 42)
#false

```

## 5.25 ボックス

```

+-----+
| [手続き] |
| |
| (box x) → box? |
| x: any/c |
+-----+

```

a box を構築します。

```

> (box 42)
(box 42)

```

```

+-----+
| [手続き] |
| |
| (box? x) → boolean? |
| x: any/c |
+-----+

```

a value is a boxかどうかを判定します。

```
> b
(box 33)
> (box? b)
#true
> (box? 42)
#false
```

```
+-----+
| [手続き] |
| |
| (set-box! b x) → void |
| b: box? |
| x: any/c |
+-----+
```

a box を更新します。

```
> b
(box 33)
> (set-box! b 31)
> b
(box 31)
```

```
+-----+
| [手続き] |
| |
| (unbox b) → any |
| b: box? |
+-----+
```

the boxed value を取り出します。

```
> b
(box 31)
> (unbox b)
31
```

## 5.26 ハッシュ表

```
+-----+
| [手続き] |
| |
| (hash-copy h) → hash |
| h: hash |
+-----+
```

Copies hash table.

```
+-----+
| [手続き] |
| |
| (hash-count h) → integer |
+-----+
```

```
| h: hash |
+-----+
```

判定する: 数の keys mapped によって hash table.

```
> ish
(make-immutable-hash (list (list 'b 69) (list 'r 999) (list 'c 42) (list 'e 61)))
> (hash-count ish)
4
```

```
+-----+
| [手続き] |
| |
| (hash-eq? h) → boolean |
| h: hash |
+-----+
```

a hash table uses eq? for comparisons かどうかを判定します。

```
> hsh
(make-hash (list (list 'c 42) (list 'r 999) (list 'b 69) (list 'e 61)))
> (hash-eq? hsh)
#false
> heq
(make-hasheq (list (list 'b 69) (list 'r 999) (list 'c 42) (list 'e 61)))
> (hash-eq? heq)
#true
```

```
+-----+
| [手続き] |
| |
| (hash-equal? h) → boolean |
| h: hash? |
+-----+
```

a hash table uses equal? for comparisons かどうかを判定します。

```
> ish
(make-immutable-hash (list (list 'b 69) (list 'r 999) (list 'c 42) (list 'e 61)))
> (hash-equal? ish)
#true
> ieq
(make-immutable-hasheq (list (list 'b 69) (list 'r 999) (list 'c 42) (list 'e 61)))
> (hash-equal? ieq)
#false
```

```
+-----+
| [手続き] |
| |
| (hash-eqv? h) → boolean |
| h: hash |
+-----+
```



a hash table uses eqv? for comparisonsかどうかを判定します。

```
> heq
(make-hasheq (list (list 'b 69) (list 'r 999) (list 'c 42) (list 'e 61)))
> (hash-eqv? heq)
#false
> heqv
(make-hasheqv (list (list 'b 69) (list 'r 999) (list 'c 42) (list 'e 61)))
> (hash-eqv? heqv)
#true
```

```
+-----+
| [手続き] |
| |
| (hash-for-each h f) → void? |
| h: (hash X Y) |
| f: (X Y → any) |
+-----+
```

a function to each mapping of a hash table for effect onlyを適用します。

```
> hsh
(make-hash (list (list 'c 42) (list 'r 999) (list 'b 69) (list 'e 61)))
> (hash-for-each hsh (lambda (ky vl) (hash-set! hsh ky (+ vl 1))))
> hsh
(make-hash (list (list 'c 43) (list 'r 1000) (list 'b 70) (list 'e 62)))
```

```
+-----+
| [手続き] |
| |
| (hash-has-key? h x) → boolean |
| h: (hash X Y) |
| x: X |
+-----+
```

a key is associated with a value in a hash tableかどうかを判定します。

```
> ish
(make-immutable-hash (list (list 'b 69) (list 'r 999) (list 'c 42) (list 'e 61)))
> (hash-has-key? ish 'b)
#true
> hsh
(make-hash (list (list 'c 43) (list 'r 1000) (list 'b 70) (list 'e 62)))
> (hash-has-key? hsh 'd)
#false
```

```
+-----+
| [手続き] |
| |
| (hash-map h f) → (listof Z) |
| h: (hash X Y) |
| f: (X Y → Z) |
+-----+
```

a new list by applying a function to each mapping of a hash table を構築します。

```
> ish
(make-immutable-hash (list (list 'b 69) (list 'r 999) (list 'c 42) (list 'e 61)))
> (hash-map ish list)
(list (list 'b 69) (list 'r 999) (list 'c 42) (list 'e 61))
```

```
+-----+
| [手続き] |
| |
| (hash-ref h k) → Y |
| h: (hash X Y) |
| k: X |
+-----+
```

the value associated with a key from a hash table; the three argument case allows a default value or default value computation を取り出します。

```
> hsh
(make-hash (list (list 'c 43) (list 'r 1000) (list 'b 70) (list 'e 62)))
> (hash-ref hsh 'b)
```

70

```
+-----+
| [手続き] |
| |
| (hash-ref! h k v) → Y |
| h: (hash X Y) |
| k: X |
| v: Y |
+-----+
```

the value associated with a key from a mutable hash table; if the key does not have an mapping, the third argument is used as the value (or used to compute the value) and is added to the hash table associated with the key を取り出します。

```
> hsh
(make-hash (list (list 'c 43) (list 'r 1000) (list 'b 70) (list 'e 62)))
> (hash-ref! hsh 'd 99)
99
> hsh
(make-hash (list (list 'c 43) (list 'd 99) (list 'r 1000) (list 'b 70) (list 'e 62)))
```

```
+-----+
| [手続き] |
| |
| (hash-remove h k) → (hash X Y) |
| h: (hash X Y) |
| k: X |
+-----+
```

an immutable hash table with one less mapping than an existing immutable hash table  
を構築します。

```
> ish
(make-immutable-hash (list (list 'b 69) (list 'r 999) (list 'c 42) (list 'e 61)))
> (hash-remove ish 'b)
(make-immutable-hash (list (list 'c 42) (list 'r 999) (list 'e 61)))
```

```
+-----+
| [手続き] |
| |
| (hash-remove! h x) → void |
| h: (hash X Y) |
| x: X |
+-----+
```

Removes mapping から mutable hash table.

```
> hsh
(make-hash (list (list 'c 43) (list 'd 99) (list 'r 1000) (list 'b 70) (list 'e 62)))
> (hash-remove! hsh 'r)
> hsh
(make-hash (list (list 'c 43) (list 'd 99) (list 'b 70) (list 'e 62)))
```

```
+-----+
| [手続き] |
| |
| (hash-set h k v) → (hash X Y) |
| h: (hash X Y) |
| k: X |
| v: Y |
+-----+
```

an immutable hash table with one new mapping from an existing immutable hash  
table を構築します。

```
> (hash-set ish 'a 23)
(make-immutable-hash (list (list 'b 69) (list 'r 999) (list 'c 42) (list 'a 23) (list 'e 61)))
```

```
+-----+
| [手続き] |
| |
| (hash-set! h k v) → void? |
| h: (hash X Y) |
| k: X |
| v: Y |
+-----+
```

a mutable hash table with a new mapping を更新します。

```
> hsh
(make-hash (list (list 'c 43) (list 'd 99) (list 'b 70) (list 'e 62)))
> (hash-set! hsh 'a 23)
```

```
> hsh
(make-hash (list (list 'c 43) (list 'a 23) (list 'd 99) (list 'b 70) (list 'e 62)))
```

```
+-----+
| [手続き] |
| |
| (hash-update h k f) → (hash X Y) |
| h: (hash X Y) |
| k: X |
| f: (Y -> Y) |
+-----+
```

Composes hash-ref と hash-set へ update existing mapping; third 引数 は used へ compute new mapping 値; fourth 引数 は used として third 引数 へ hash-ref.

```
> (hash-update ish 'b (lambda (old-b) (+ old-b 1)))
(make-immutable-hash (list (list 'b 70) (list 'r 999) (list 'c 42) (list 'e 61)))
```

```
+-----+
| [手続き] |
| |
| (hash-update! h k f) → void? |
| h: (hash X Y) |
| k: X |
| f: (Y -> Y) |
+-----+
```

Composes hash-ref と hash-set! へ update existing mapping; third 引数 は used へ compute new mapping 値; fourth 引数 は used として third 引数 へ hash-ref.

```
> hsh
(make-hash (list (list 'c 43) (list 'a 23) (list 'd 99) (list 'b 70) (list 'e 62)))
> (hash-update! hsh 'b (lambda (old-b) (+ old-b 1)))
> hsh
(make-hash (list (list 'c 43) (list 'a 23) (list 'd 99) (list 'b 71) (list 'e 62)))
```

```
+-----+
| [手続き] |
| |
| (hash? x) → boolean |
| x: any |
+-----+
```

a value is a hash table かどうかを判定します。

```
> ish
(make-immutable-hash (list (list 'b 69) (list 'r 999) (list 'c 42) (list 'e 61)))
> (hash? ish)
#true
> (hash? 42)
#false
```

```

+-----+
| [手続き] |
| |
| (make-hash) → (hash X Y) |
+-----+

```

a mutable hash table from an optional list of mappings that uses equal? for comparisons を構築します。

```

> (make-hash)
(make-hash)
> (make-hash '((b 69) (e 61) (i 999)))
(make-hash (list (list 'i 999) (list 'b 69) (list 'e 61)))

```

```

+-----+
| [手続き] |
| |
| (make-hasheq) → (hash X Y) |
+-----+

```

a mutable hash table from an optional list of mappings that uses eq? for comparisons を構築します。

```

> (make-hasheq)
(make-hasheq)
> (make-hasheq '((b 69) (e 61) (i 999)))
(make-hasheq (list (list 'i 999) (list 'b 69) (list 'e 61)))

```

```

+-----+
| [手続き] |
| |
| (make-hasheqv) → (hash X Y) |
+-----+

```

a mutable hash table from an optional list of mappings that uses eqv? for comparisons を構築します。

```

> (make-hasheqv)
(make-hasheqv)
> (make-hasheqv '((b 69) (e 61) (i 999)))
(make-hasheqv (list (list 'i 999) (list 'b 69) (list 'e 61)))

```

```

+-----+
| [手続き] |
| |
| (make-immutable-hash) → (hash X Y) |
+-----+

```

an immutable hash table from an optional list of mappings that uses equal? for comparisons を構築します。

```

> (make-immutable-hash)
(make-immutable-hash)

```

```
> (make-immutable-hash '((b 69) (e 61) (i 999)))
(make-immutable-hash (list (list 'b 69) (list 'e 61) (list 'i 999)))
```

```
+-----+
| [手続き] |
| |
| (make-immutable-hasheq) → (hash X Y) |
+-----+
```

an immutable hash table from an optional list of mappings that uses eq? for comparisons を構築します。

```
> (make-immutable-hasheq)
(make-immutable-hasheq)
> (make-immutable-hasheq '((b 69) (e 61) (i 999)))
(make-immutable-hasheq (list (list 'b 69) (list 'e 61) (list 'i 999)))
```

```
+-----+
| [手続き] |
| |
| (make-immutable-hasheqv) → (hash X Y) |
+-----+
```

an immutable hash table from an optional list of mappings that uses eqv? for comparisons を構築します。

```
> (make-immutable-hasheqv)
(make-immutable-hasheqv)
> (make-immutable-hasheqv '((b 69) (e 61) (i 999)))
(make-immutable-hasheqv (list (list 'b 69) (list 'e 61) (list 'i 999)))
```